Universidad Peruana de Ciencias Aplicadas

Carrera de Ingeniería de Software

1ACC0238

Aplicaciones para Dispositivos Móviles

NRC

13975

Informe del Trabajo Final

Docente

Quevedo Velasco, David Gerardo

Equipo

ADM

Proyecto

AniTec

Integrantes:

Código	Apellidos y Nombres
U202217853	Beingolea Montalvo, Sebastian Martin
U202315165	Melgarejo Quiroz, Josep Eliu
U20231C019	Ortega Muñoz, Saul
U202215979	Sanchez Silva, Luciana Celeste
U20221C554	Villanueva Rodriguez, Giuseppe Adrian

Período 202620

Octubre 2026

Registro de versiones del informe

Las entradas de la versión 1 corresponden al avance AV1 del curso de Aplicaciones para Dispositivos Móviles, las de la versión 2, al TB1 (Sprint 1) y las de la versión 3, al AV2 (Sprint 2). La fecha indica la actualización de este registro.

Versión	Fecha	Autor	Descripción de modificación
1.0	15/09/2026	Josep Eliu Melgarejo Quiroz	Creación del documento de colaboración en GitHub
1.1	16/09/2026	Sebastian Martin Beingolea Montalvo	Redacción del capítulo I: Introducción.
1.2	16/09/2026	Saul Ortega Muñoz	Redacción de las secciones 2.1 a 2.4 del capítulo II.
1.3	16/09/2026	Josep Eliu Melgarejo Quiroz	Redacción de las secciones 2.6.1 a 2.6.9 del capítulo II.
1.4	16/09/2026	Giuseppe Villanueva Rodriguez	Realizó una entrevista.
1.5	16/09/2026	Luciana Celeste Sanchez Silva	Redacción de las secciones 2.5 a 2.6 del capítulo II.
2.0	25/09/2026	Josep Eliu Melgarejo Quiroz	Sprint Planning 1: definición del Sprint Goal, de la capacidad del equipo y de las historias comprometidas del Sprint 1.
2.1	01/10/2026	Sebastian Martin Beingolea Montalvo	Evidencias del backend en el capítulo IV: despliegue en Render y documentación con Swagger.
2.2	02/10/2026	Saul Ortega Muñoz	Wireframe y mock-up móvil de la Landing Page, y evidencias de desarrollo, ejecución, pruebas y despliegue de la Landing Page.
2.3	02/10/2026	Giuseppe Villanueva Rodriguez	Configuración y despliegue de las aplicaciones móviles (sección 4.1), registro en Firebase y evidencias de las pruebas de Android.
2.4	03/10/2026	Luciana Celeste Sanchez Silva	Wireframes, wireflows, user flow diagrams y evidencias del prototipo de las aplicaciones móviles en el capítulo III.
2.5	04/10/2026	Josep Eliu Melgarejo Quiroz	Documentación completa del Sprint 1 en el capítulo IV, actualización del Product Backlog, student outcomes e integración de los cambios del equipo.
3.0	25/10/2026	Josep Eliu Melgarejo Quiroz	Sprint Planning 2: Sprint Goal, historias comprometidas y ajuste del Product Backlog (secciones 2.4.3 y 4.2.2.1).

Versión	Fecha	Autor	Descripción de modificación
3.1	25/10/2026	Josep Eliu Melgarejo Quiroz	Documentación del Sprint 2 en el capítulo IV (4.2.2): matriz LACX, Sprint Backlog, evidencias de desarrollo, pruebas, ejecución, servicios y despliegue, y colaboración del equipo.
3.2	25/10/2026	Josep Eliu Melgarejo Quiroz	Actualización de la configuración de despliegue (4.1): aplicación Flutter, firma de las compilaciones, distribución en Firebase App Distribution, datos de demostración y límite de conexiones del backend.
3.3	25/10/2026	Josep Eliu Melgarejo Quiroz	Actualización del registro de versiones, del Project Report Collaboration Insights, del Student Outcome y del anexo F para el AV2.

Project Report Collaboration Insights

Repositorios del proyecto

- Informe: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/Informe
- Landing page: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-landing-page
- Frontend web: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-frontend
- Backend: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend
- Android: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android
- Flutter: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-flutter

AV1. Distribución del trabajo

Durante AV1, el equipo adaptó el informe y el diseño de AniTec al alcance de una aplicación móvil. Las contribuciones documentadas para los cinco integrantes actuales son las siguientes:

Integrante	Contribución en AV1
Sebastian Martin Beingolea Montalvo	Redactó el capítulo I: Introducción.
Saul Ortega Muñoz	Redactó las secciones 2.1 a 2.4 del capítulo II.
Josep Eliu Melgarejo Quiroz	Redactó las secciones 2.6.1 a 2.6.9 del capítulo II.
Giuseppe Villanueva Rodriguez	Realizó una entrevista.
Luciana Celeste Sanchez Silva	Redactó las secciones 2.5 a 2.6 del capítulo II.

TB1. Distribución del trabajo

Durante el TB1 (Sprint 1), el equipo implementó la Landing Page, el backend y la aplicación Android, y documentó el sprint en los capítulos III y IV del informe. Las contribuciones de los cinco integrantes, según las tareas del Sprint Backlog, son las siguientes:

Integrante	Contribución en TB1
Sebastian Martin Beingolea Montalvo	Conectó la aplicación con la API, agregó los corrales y las operaciones masivas al backend, documentó los endpoints con OpenAPI, protegió la sesión del usuario y creó las pruebas automatizadas del backend. Documentó las evidencias del backend en el informe.
Saul Ortega Muñoz	Diseñó el wireframe y el mock-up móvil de la Landing Page y construyó sus secciones, su diseño adaptable y el cambio de idioma. Desarrolló las pantallas de registro, inicio de sesión, mantenimiento de la sesión y ficha técnica del animal.
Josep Eliu Melgarejo Quiroz	Desarrolló el almacenamiento local de la aplicación Android, la navegación por rol, la lista de animales con búsqueda, el registro individual y masivo de animales y la fotografía del animal. Dirigió la planificación del Sprint 1 y mantiene la estructura del informe.

Integrante	Contribución en TB1
Giuseppe Villanueva Rodriguez	Creó el proyecto Android y organizó su código por capas, y desarrolló la edición de animales y la pantalla de inicio del ganadero. Escribió las pruebas de la aplicación y registró las aplicaciones en Firebase.
Luciana Celeste Sanchez Silva	Elaboró los mock-ups, wireframes, wireflows, user flow diagrams y el prototipo de las aplicaciones móviles. Desarrolló las pantallas de fincas, corrales y registros sanitarios.

AV2. Distribución del trabajo

Durante el AV2 (Sprint 2), el equipo completó el backend y la aplicación Android, construyó la primera versión de la aplicación Flutter, distribuyó las dos aplicaciones en Firebase App Distribution y documentó el sprint en el capítulo IV del informe. Las tareas se repartieron en partes iguales, con 40 horas estimadas por integrante. Las contribuciones, según las tareas del Sprint Backlog 2, son las siguientes:

Integrante	Contribución en AV2
Sebastian Martin Beingolea Montalvo	Filtró los datos por usuario en el servidor, integró el retorno móvil de Stripe y la confirmación del pago, cargó los planes de suscripción, agregó el archivado de animales y el nombre base del alta masiva con los datos de demostración, configuró el servicio publicado y escribió las pruebas de la aplicación Flutter.
Saul Ortega Muñoz	Implementó el escáner de códigos QR con ML Kit y la búsqueda manual, los dispositivos IoT, la arquitectura y el diseño del proyecto Flutter, las pruebas de integración y BDD del backend y los mensajes de error y el idioma. Actualizó la Landing Page con el vínculo de descarga.
Josep Eliu Melgarejo Quiroz	Desarrolló la vista del veterinario, el trabajo sin conexión, las suscripciones con Stripe en Android, la corrección de la navegación inferior y el permiso de red local para depurar. Dirigió la planificación del Sprint 2, mantiene la estructura del informe y documentó el sprint.
Giuseppe Villanueva Rodriguez	Implementó la analítica y el archivado de animales en Android, la compilación firmada con R8, la distribución de las dos aplicaciones en Firebase y las pruebas de Android. Verificó el escáner QR y el pago de prueba en un dispositivo físico.
Luciana Celeste Sanchez Silva	Construyó las pantallas de actividades y de finanzas, el nombre base del formulario de lotes y, en Flutter, el inicio de sesión y el inicio del ganadero. Elaboró los videos del AV2.

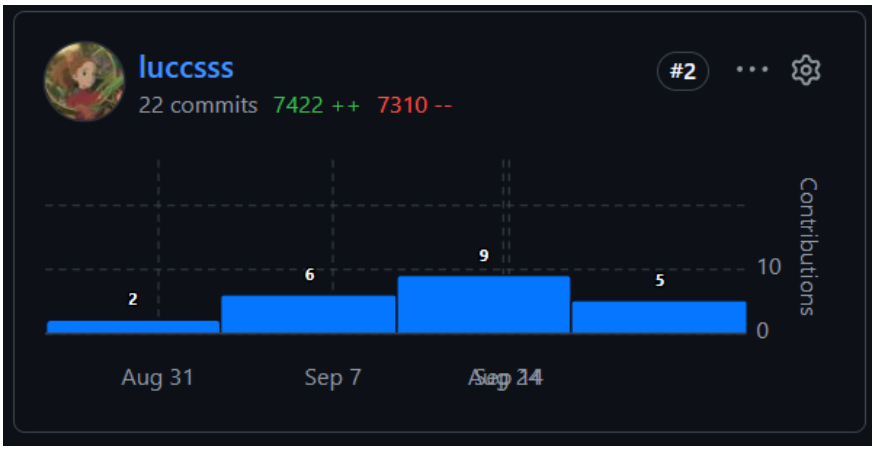
GitHub Collaboration Insights

El repositorio del informe conserva el historial de los cambios documentales del equipo. Las siguientes capturas muestran el grafo general y los commits asociados a cada integrante durante el avance.

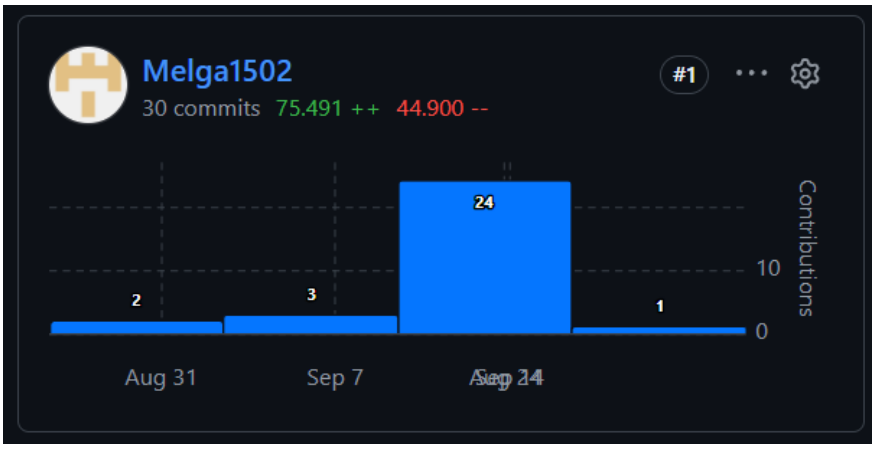
Grafo general de commits AV1

Commits por integrante AV1

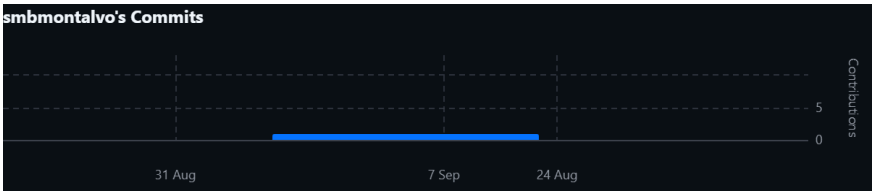
Luciana Celeste Sanchez Silva



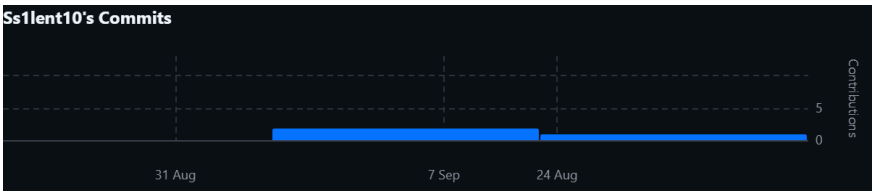
Josep Eliu Melgarejo Quiroz



Sebastian Martin Beingolea Montalvo

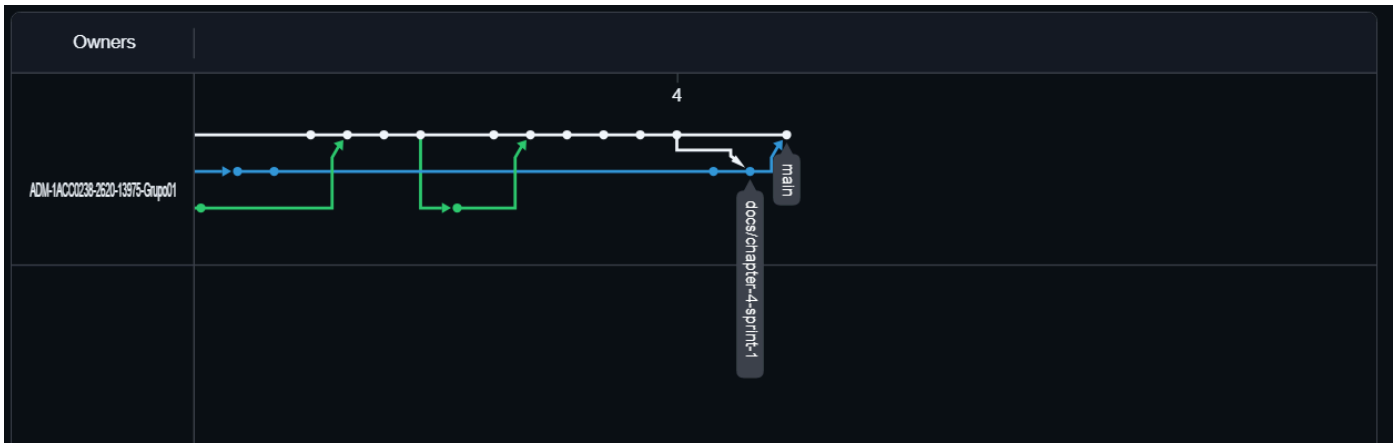


Saul Ortega Muñoz



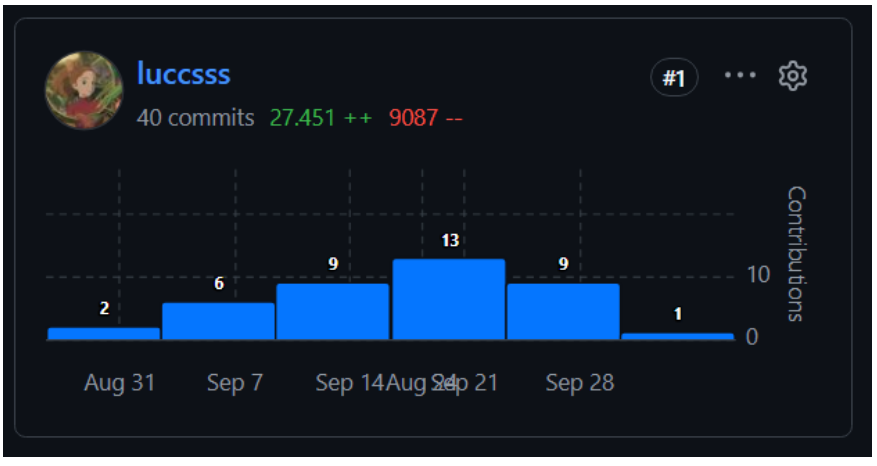
Giuseppe Villanueva Rodriguez

Grafo general de commits TB1

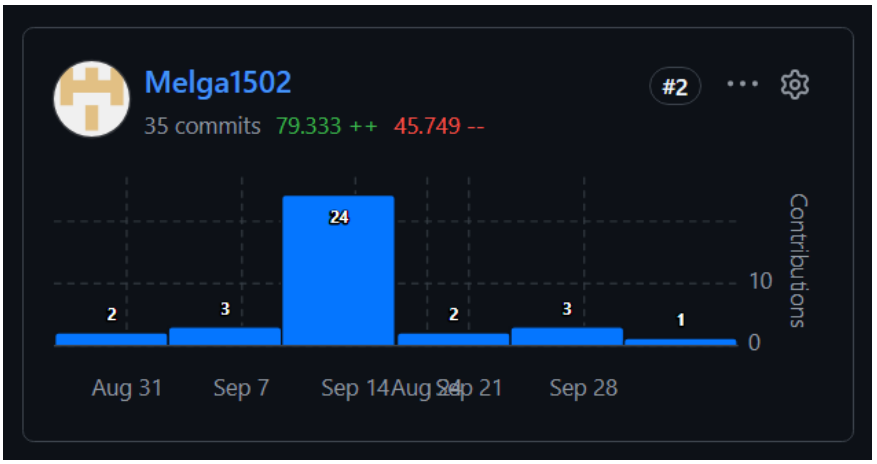


Commits por integrante TB1

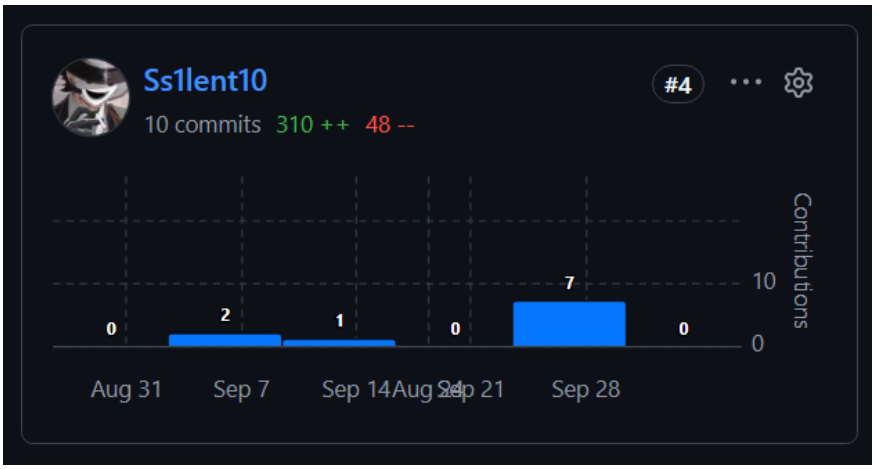
Luciana Celeste Sanchez Silva



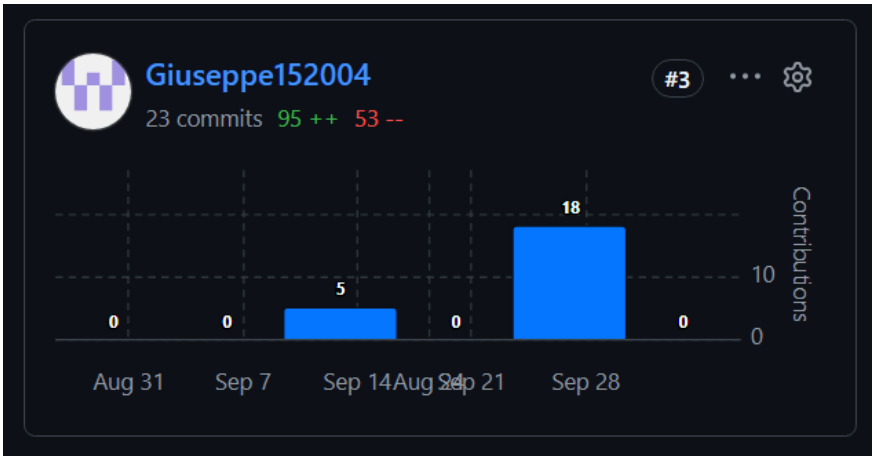
Josep Eliu Melgarejo Quiroz



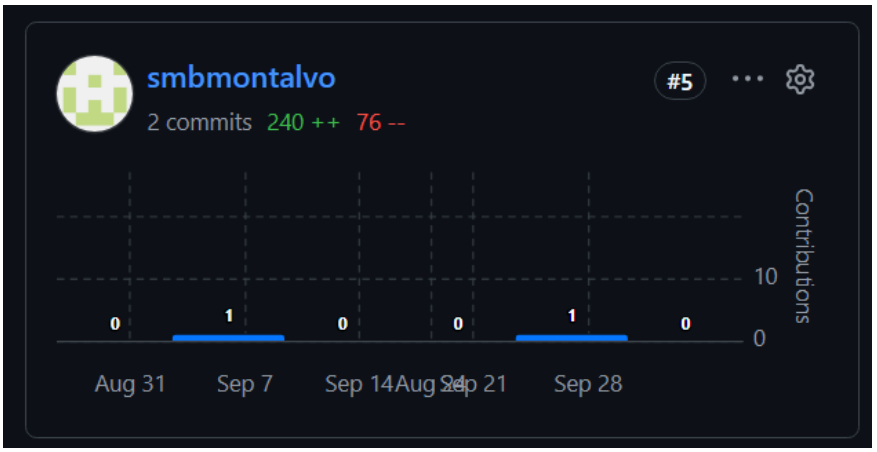
Sebastian Martin Beingolea Montalvo



Saul Ortega Muñoz



Giuseppe Villanueva Rodriguez



Grafo general de commits AV2

Captura pendiente

Reemplazar por la captura real de la evidencia

Commits por integrante AV2

Luciana Celeste Sanchez Silva

Captura pendiente

Reemplazar por la captura real de la evidencia

Josep Eliu Melgarejo Quiroz

Sebastian Martin Beingolea Montalvo

Saul Ortega Muñoz

Giuseppe Villanueva Rodriguez

Captura pendiente
Reemplazar por la captura real de la evidencia

Contenido

- Student Outcome
- Objetivos SMART

Capítulo I: Presentación

- 1.1. Startup Profile
  - 1.1.1. Descripción de la Startup
  - 1.1.2. Perfiles de integrantes del equipo
- 1.2. Solution Profile
  - 1.2.1. Antecedentes y problemática
  - 1.2.2. Lean UX Process
    - 1.2.2.1. Lean UX Problem Statements
    - 1.2.2.2. Lean UX Assumptions
    - 1.2.2.3. Lean UX Hypothesis Statements
    - 1.2.2.4. Lean UX Canvas
- 1.3. Segmentos objetivo

Capítulo II: Requirements Development and Software Solution Design

- 2.1. Competidores
  - 2.1.1. Análisis competitivo
  - 2.1.2. Estrategias y tácticas frente a competidores
- 2.2. Entrevistas
  - 2.2.1. Diseño de entrevistas
  - 2.2.2. Registro de entrevistas
  - 2.2.3. Análisis de entrevistas
- 2.3. Needfinding
  - 2.3.1. User Personas
  - 2.3.2. User Task Matrix
  - 2.3.3. User Journey Mapping
  - 2.3.4. Empathy Mapping
  - 2.3.5. Big Picture EventStorming
  - 2.3.6. Ubiquitous Language
- 2.4. Requirements specification
  - 2.4.1. User Stories
  - 2.4.2. Impact Mapping
  - 2.4.3. Product Backlog
- 2.5. Strategic-Level Domain-Driven Design
  - 2.5.1. EventStorming

        - 2.5.1.1. Candidate Context Discovery
        - 2.5.1.2. Domain Message Flows Modeling
        - 2.5.1.3. Bounded Context Canvases
    - 2.5.2. Context Mapping
    - 2.5.3. Software Architecture
        - 2.5.3.1. Software Architecture Context Level Diagrams
        - 2.5.3.2. Software Architecture Container Level Diagrams
        - 2.5.3.3. Software Architecture Deployment Diagrams
- 2.6. Tactical-Level Domain-Driven Design
    - 2.6.1. Bounded Context: Identity and Access Management
        - 2.6.1.1. Domain Layer
        - 2.6.1.2. Interface Layer
        - 2.6.1.3. Application Layer
        - 2.6.1.4. Infrastructure Layer
        - 2.6.1.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.1.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.1.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.1.6.2. Bounded Context Database Design Diagram
    - 2.6.2. Bounded Context: Profile Management
        - 2.6.2.1. Domain Layer
        - 2.6.2.2. Interface Layer
        - 2.6.2.3. Application Layer
        - 2.6.2.4. Infrastructure Layer
        - 2.6.2.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.2.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.2.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.2.6.2. Bounded Context Database Design Diagram
    - 2.6.3. Bounded Context: Livestock Management
        - 2.6.3.1. Domain Layer
        - 2.6.3.2. Interface Layer
        - 2.6.3.3. Application Layer
        - 2.6.3.4. Infrastructure Layer
        - 2.6.3.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.3.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.3.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.3.6.2. Bounded Context Database Design Diagram
    - 2.6.4. Bounded Context: Sanitary Management
        - 2.6.4.1. Domain Layer
        - 2.6.4.2. Interface Layer
        - 2.6.4.3. Application Layer
        - 2.6.4.4. Infrastructure Layer
        - 2.6.4.5. Bounded Context Software Architecture Component Level Diagrams

        - 2.6.4.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.4.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.4.6.2. Bounded Context Database Design Diagram
    - 2.6.5. Bounded Context: Veterinary Collaboration
        - 2.6.5.1. Domain Layer
        - 2.6.5.2. Interface Layer
        - 2.6.5.3. Application Layer
        - 2.6.5.4. Infrastructure Layer
        - 2.6.5.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.5.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.5.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.5.6.2. Bounded Context Database Design Diagram
    - 2.6.6. Bounded Context: Activity Management
        - 2.6.6.1. Domain Layer
        - 2.6.6.2. Interface Layer
        - 2.6.6.3. Application Layer
        - 2.6.6.4. Infrastructure Layer
        - 2.6.6.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.6.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.6.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.6.6.2. Bounded Context Database Design Diagram
    - 2.6.7. Bounded Context: Financial Management
        - 2.6.7.1. Domain Layer
        - 2.6.7.2. Interface Layer
        - 2.6.7.3. Application Layer
        - 2.6.7.4. Infrastructure Layer
        - 2.6.7.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.7.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.7.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.7.6.2. Bounded Context Database Design Diagram
    - 2.6.8. Bounded Context: Subscription Management
        - 2.6.8.1. Domain Layer
        - 2.6.8.2. Interface Layer
        - 2.6.8.3. Application Layer
        - 2.6.8.4. Infrastructure Layer
        - 2.6.8.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.8.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.8.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.8.6.2. Bounded Context Database Design Diagram
    - 2.6.9. Bounded Context: Analytics and Reporting
        - 2.6.9.1. Domain Layer
        - 2.6.9.2. Interface Layer

        - 2.6.9.3. Application Layer
        - 2.6.9.4. Infrastructure Layer
        - 2.6.9.5. Bounded Context Software Architecture Component Level Diagrams
        - 2.6.9.6. Bounded Context Software Architecture Code Level Diagrams
            - 2.6.9.6.1. Bounded Context Domain Layer Class Diagrams
            - 2.6.9.6.2. Bounded Context Database Design Diagram

Capítulo III: Solution UI/UX Design

- 3.1. Product design
    - 3.1.1. Style Guidelines
        - 3.1.1.1. General Style Guidelines
        - 3.1.1.2. Web Style Guidelines
        - 3.1.1.3. Mobile Style Guidelines
    - 3.1.2. Information Architecture
        - 3.1.2.1. Organization Systems
        - 3.1.2.2. Labelling Systems
        - 3.1.2.3. SEO Tags and Meta Tags
        - 3.1.2.4. Searching Systems
        - 3.1.2.5. Navigation Systems
    - 3.1.3. Landing Page UI Design
        - 3.1.3.1. Landing Page Wireframe
        - 3.1.3.2. Landing Page Mock-up
    - 3.1.4. Mobile Applications UX/UI Design
        - 3.1.4.1. Mobile Applications Wireframes
        - 3.1.4.2. Mobile Applications Wireflow Diagrams
        - 3.1.4.3. Mobile Applications Mock-ups
        - 3.1.4.4. Mobile Applications User Flow Diagrams
        - 3.1.4.5. Mobile Applications Prototyping

Capítulo IV: Product Implementation & Validation

- 4. Product Implementation & Validation

    - 4.1. Software Configuration Management
        - 4.1.1. Software Development Environment Configuration
        - 4.1.2. Source Code Management
        - 4.1.3. Source Code Style Guide & Conventions
        - 4.1.4. Software Deployment Configuration
    - 4.2. Landing Page & Mobile Application Implementation
        - 4.2.1. Sprint 1
            - 4.2.1.1. Sprint Planning 1
            - 4.2.1.2. Aspect Leaders and Collaborators

        - 4.2.1.3. Sprint Backlog 1
        - 4.2.1.4. Development Evidence for Sprint Review
        - 4.2.1.5. Testing Suite Evidence for Sprint Review
        - 4.2.1.6. Execution Evidence for Sprint Review
        - 4.2.1.7. Services Documentation Evidence for Sprint Review
        - 4.2.1.8. Software Deployment Evidence for Sprint Review
        - 4.2.1.9. Team Collaboration Insights during Sprint
    - 4.2.2. Sprint 2
        - 4.2.2.1. Sprint Planning 2
        - 4.2.2.2. Aspect Leaders and Collaborators
        - 4.2.2.3. Sprint Backlog 2
        - 4.2.2.4. Development Evidence for Sprint Review
        - 4.2.2.5. Testing Suite Evidence for Sprint Review
        - 4.2.2.6. Execution Evidence for Sprint Review
        - 4.2.2.7. Services Documentation Evidence for Sprint Review
        - 4.2.2.8. Software Deployment Evidence for Sprint Review
        - 4.2.2.9. Team Collaboration Insights during Sprint
  - 4.3. Validation Interviews
    - 4.3.1. Diseño de Entrevistas
    - 4.3.2. Registro de Entrevistas
    - 4.3.3. Evaluaciones según heurísticas
- Conclusiones
- Bibliografía
- Anexos

Student Outcome

ABET - EAC - Student Outcome 7

Criterio: La capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas.

En el siguiente cuadro se describen las acciones realizadas y los enunciados de conclusiones por parte de cada integrante y del grupo durante el AV1, el TB1 y el AV2, que permiten sustentar el logro del ABET - EAC - Student Outcome 7.

Criterio específico	Acciones realizadas	Conclusiones
Actualiza conceptos y conocimientos necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de software.	**Luciana Celeste Sanchez Silva:** **AV1:** Revisó los conceptos de Lean UX, segmentación de usuarios y diseño centrado en las necesidades del ganadero y del veterinario. Aplicó estos conocimientos al actualizar el Startup Profile, el Solution Profile, los Problem Statements, Assumptions e Hypothesis Statements para orientar AniTec hacia una aplicación móvil. **TB1:** Aprendió a convertir los diseños de Figma en pantallas reales de la aplicación, pensando en lo que necesita el ganadero al usarla en el campo. Aplicó este conocimiento al elaborar los mock-ups móviles y al construir las pantallas de fincas, corrales y registros sanitarios con formularios claros y mensajes fáciles de entender. **AV2:** Aprendió a llevar un diseño a una segunda tecnología, Flutter, conservando los colores, los textos y los estados de carga, vacío y error, a cuidar los montos y los formularios de las finanzas y a guardar la sesión del usuario de forma cifrada. Aplicó este conocimiento al construir las pantallas de actividades y de finanzas de Android, el nombre base opcional del formulario de lotes con su vista previa en inglés y español, y el inicio de sesión con	**AV1:** El equipo actualizó conocimientos de Lean UX, Requirements Engineering, Domain-Driven Design, arquitectura C4 y desarrollo para dispositivos móviles. Estos conocimientos se aplicaron directamente en la corrección de los capítulos I y II, la definición del alcance de Android y Flutter, la elaboración de requisitos trazables y el diseño estratégico y táctico de los nueve bounded contexts. Como resultado, AniTec cuenta con una base documental coherente con el alcance del proyecto y preparada para orientar su posterior implementación móvil. **TB1:** El equipo pasó del diseño a la implementación y tuvo que aprender nuevas herramientas para construir la Landing Page, el servidor y la aplicación Android. Cada integrante aplicó lo aprendido en las tareas del Sprint 1, lo que permitió entregar una Landing Page publicada, un servidor desplegado y probado, y una aplicación que permite registrarse, iniciar sesión y gestionar fincas, corrales, animales y registros sanitarios.

Criterio específico	Acciones realizadas	Conclusiones
	sesión segura y el inicio del ganadero con cambio de idioma en Flutter. **Josep Eliu Melgarejo Quiroz:** **AV1:** Analizó los requisitos del proyecto y estudió la estructura requerida para la especificación de requisitos móviles. Aplicó lo aprendido al reorganizar el índice del informe, adaptar las User Stories y Technical Stories, y distribuir el Product Backlog en tres sprints alineados con los hitos del curso. **TB1:** Aprendió a construir una aplicación móvil que guarda la información en el teléfono y muestra a cada usuario solo lo que le corresponde. Aplicó este conocimiento al desarrollar la lista de animales con búsqueda, el registro de animales uno por uno o varios a la vez, la fotografía del animal y la organización de la información de cada rol. **AV2:** Aprendió a reproducir un error con una prueba antes de corregirlo, a guardar el trabajo sin conexión para enviarlo después, a integrar un pago externo en una aplicación móvil y a atender el permiso de red local que Android 17 exige para llegar al servidor del equipo. Aplicó este conocimiento al desarrollar la vista del veterinario, la cola de operaciones sin conexión, las suscripciones con Stripe, la corrección de la navegación inferior y el permiso de red local en depuración, y al documentar el Sprint 2 en el informe. **Sebastian Martin Beingolea Montalvo:** **AV1:** Profundizó en los fundamentos de Domain-Driven Design, EventStorming y lenguaje ubicuo. Aplicó estos conceptos en la identificación y delimitación de los bounded contexts, así como en la revisión	**AV2:** El equipo completó el paso de una primera versión a una aplicación completa y tuvo que aprender a usar la cámara y ML Kit, a cobrar con Stripe, a firmar y distribuir las aplicaciones y a crear una segunda aplicación con Flutter. Cada integrante aplicó lo aprendido en las tareas del Sprint 2, lo que permitió entregar un backend completo con 150 pruebas automáticas, una aplicación Android con las funciones del ganadero y del veterinario, una primera versión de Flutter y la distribución de ambas aplicaciones.

Criterio específico	Acciones realizadas	Conclusiones
	de los flujos de mensajes del dominio ganadero. **TB1:** Aprendió cómo se comunican la aplicación y el servidor y cómo comprobar que ese servicio funciona correctamente. Aplicó este conocimiento al conectar la aplicación con el servidor, añadir los corrales, documentar los servicios, proteger la sesión del usuario y crear las pruebas automáticas del servidor. **AV2:** Aprendió a hacer que el servidor decida qué información ve cada usuario, a confirmar un pago desde el servidor, a probar el servicio publicado en condiciones de uso real y a probar una aplicación Flutter con pruebas de dominio, de estado y de pantallas. Aplicó este conocimiento al filtrar los datos por usuario, integrar el retorno de Stripe, sembrar los planes de suscripción, archivar animales, generar nombres consecutivos en el alta masiva y los datos de demostración, configurar el servicio publicado y escribir las pruebas de Flutter. **Saul Ortega Muñoz:** **AV1:** Investigó la notación C4, Structurizr DSL y la representación de arquitecturas para aplicaciones móviles. Aplicó estos conocimientos al apoyar la definición de los diagramas de contexto, contenedores, despliegue y componentes de AniTec. **TB1:** Aprendió a diseñar páginas y pantallas fáciles de usar en cualquier dispositivo y en dos idiomas. Aplicó este conocimiento al construir la Landing Page adaptable al celular y las pantallas de registro, inicio de sesión y ficha del animal de la aplicación. **AV2:** Aprendió a usar la cámara del teléfono para leer códigos QR con ML Kit, a crear un proyecto Flutter con la misma estructura por contextos que la aplicación	

Criterio específico	Acciones realizadas	Conclusiones
	Android y a redactar mensajes de error claros en dos idiomas. Aplicó este conocimiento al implementar los dispositivos IoT, el escáner con búsqueda manual, la arquitectura, el diseño y los idiomas del proyecto Flutter, las pruebas de integración y BDD del servidor y los mensajes de validación, el cambio de idioma y los textos localizados. **Giuseppe Villanueva Rodriguez:** **AV1:** Revisó conceptos relacionados con desarrollo Android, aplicaciones multiplataforma, persistencia local, sincronización, uso de cámara y notificaciones. Aplicó estos conocimientos al verificar la coherencia entre los requisitos del proyecto, las funcionalidades móviles y las historias incluidas en el Product Backlog. **TB1:** Aprendió a organizar un proyecto móvil de forma ordenada, a comprobar que funciona mediante pruebas y a preparar una aplicación para distribuirla. Aplicó este conocimiento al crear la base del proyecto Android, la pantalla de inicio del ganadero y la edición de animales, al escribir las pruebas de la aplicación y al registrar las aplicaciones en Firebase. **AV2:** Aprendió a firmar y distribuir una aplicación (keystore, compilación release y Firebase App Distribution), a representar datos con gráficos propios y a probar la navegación y la base de datos local de una aplicación Android. Aplicó este conocimiento a la analítica del ganadero y del veterinario, al archivado y restauración de animales en Android, a la compilación firmada con R8, a la distribución de las dos aplicaciones y a las pruebas unitarias e instrumentadas de Android.	

Criterio específico	Acciones realizadas	Conclusiones
Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de software.	**Luciana Celeste Sanchez Silva:** **AV1:** Identificó que los supuestos sobre facilidad de uso, adopción y colaboración entre ganaderos y veterinarios deben contrastarse continuamente mediante entrevistas y validaciones. Por ello, consideró la investigación con usuarios como una fuente permanente de aprendizaje para mejorar la propuesta móvil. **TB1:** Reconoció que un diseño no está terminado hasta que se prueba en la aplicación real. Al llevar los mock-ups a pantallas, ajustó los estados sin datos, los errores y las confirmaciones, y entendió que debe seguir revisando el diseño con los usuarios para mejorarlo. **AV2:** Reconoció que mantener el mismo diseño en dos tecnologías exige revisar con cuidado cada estado de la pantalla, y que debe seguir contrastándolo con el uso real en el teléfono. **Josep Eliu Melgarejo Quiroz:** **AV1:** Reconoció la necesidad de revisar continuamente los requisitos del proyecto, el Product Backlog y los criterios de evaluación para detectar brechas entre el trabajo heredado y las exigencias del nuevo curso. Utilizó la comparación sistemática y la mejora iterativa del informe como estrategia de aprendizaje. **TB1:** Reconoció que lo aprendido debe comprobarse con el uso real. Al ver que el filtrado de la información por usuario se hacía solo en la aplicación, identificó que debe reforzarse también en el servidor, y adoptó la revisión continua del trabajo como forma de aprender. **AV2:** Reconoció que una prueba debe reproducir el problema antes de la	**AV1:** El equipo reconoció que el desarrollo de AniTec requiere aprendizaje continuo porque combina un producto web heredado con nuevas plataformas, herramientas y restricciones móviles. La revisión de documentación, el análisis de trabajos anteriores, la investigación individual, la discusión en equipo y la validación progresiva permitieron corregir supuestos y fundamentar decisiones. Estas prácticas fortalecen la capacidad del grupo para incorporar nuevos conocimientos durante los siguientes hitos y responder a cambios técnicos o de usuario sin perder coherencia en la solución. **TB1:** El equipo comprobó que implementar obliga a seguir aprendiendo: las pruebas mostraron errores, la implementación mostró aspectos del diseño que debían ajustarse y el despliegue exigió dominar nuevas herramientas. El equipo respondió revisando y corrigiendo lo hecho, y dejó identificados los puntos por aprender para el siguiente hito, como el filtrado por usuario en el servidor y la distribución firmada de la aplicación. **AV2:** El equipo comprobó que completar una aplicación obliga a seguir aprendiendo: el despliegue mostró un límite de la base de datos que no aparecía en el equipo local, la cámara virtual del emulador no permite probar la lectura de códigos QR, y las historias parciales mostraron lo que faltaba

Criterio específico	Acciones realizadas	Conclusiones
	corrección y que documentar con honestidad lo que queda parcial, como el recordatorio de las actividades o la sincronización sin repetir operaciones, también es aprender. Dejó esos puntos como aprendizaje del Sprint 3. **Sebastian Martin Beingolea Montalvo:** **AV1:** Comprendió que la delimitación del dominio puede evolucionar conforme aparecen nuevos eventos, reglas y dependencias. Adoptó la revisión de recursos anteriores, la discusión del lenguaje ubicuo y la contrastación entre diagramas como estrategias para refinar el diseño. **TB1:** Comprendió que escribir pruebas también es una forma de aprender. Gracias a ellas descubrió un error de permisos, en el que un veterinario recibía un fallo del servidor en lugar de un aviso de acceso denegado, y lo corrigió, lo que le mostró la importancia de probar también los casos de error y no solo los que funcionan. **AV2:** Reconoció que lo que funciona en el equipo local puede fallar al publicarse, como ocurrió con el límite de conexiones de la base de datos, y que conviene probar el servicio con las condiciones reales de uso. **Saul Ortega Muñoz:** **AV1:** Reconoció que las herramientas y convenciones de arquitectura requieren práctica y actualización constante. Consultó ejemplos, documentación y resultados exportados para comprobar que los diagramas representaran correctamente las relaciones entre usuarios, aplicaciones y servicios. **TB1:** Reconoció que el primer contacto del	cerrar. El equipo dejó identificados los puntos por aprender para el siguiente hito: el consentimiento del ganadero sobre el acceso veterinario, las notificaciones, la sincronización idempotente y la persistencia local en Flutter.

Criterio específico	Acciones realizadas	Conclusiones
	usuario con el producto, ya sea la Landing Page o el inicio de sesión, debe revisarse continuamente. Aprendió a mejorar los mensajes, las validaciones y la presentación de la información después de comprobar cómo se ven en pantallas pequeñas. **AV2:** Reconoció que los recursos del dispositivo, como la cámara, no pueden probarse del todo en el emulador, cuya cámara virtual no lee códigos, y que la lectura de códigos QR debe comprobarse con una cámara real. **Giuseppe Villanueva Rodriguez:** **AV1:** Identificó que las capacidades móviles plantean riesgos que deben investigarse antes de implementarse, especialmente en conectividad, permisos, almacenamiento local y uso de recursos del dispositivo. Consideró las Spike Stories, los prototipos y las pruebas en dispositivos físicos como estrategias necesarias para adquirir evidencia y ajustar decisiones. **TB1:** Reconoció que probar desde el inicio reduce el costo de los cambios y que distribuir una aplicación exige aprender pasos nuevos, como la firma y el envío a los evaluadores. Identificó esos pasos como pendientes y los asumió como aprendizaje para el siguiente hito. **AV2:** Reconoció que distribuir una aplicación exige cuidar la firma y la copia de respaldo del keystore, y que una aplicación firmada para distribuir debe comprobarse instalada en un dispositivo, no solo ejecutada desde el entorno de desarrollo.	

Objetivos SMART

Cada integrante presenta su plan profesional general incluyendo dos objetivos SMART.

Sebastian Martin Beingolea Montalvo

Durante su primer año después de graduarse, Sebastian priorizará conseguir experiencia comprobable como desarrollador backend y fortalecer su portafolio. En el segundo año profundizará en arquitectura de software y servicios en la nube mediante formación compatible con su jornada laboral y un proyecto aplicado. Revisará su avance cada tres meses para ajustar el ritmo sin descuidar su bienestar o sus responsabilidades personales.

Objetivo SMART 1. Consolidar experiencia profesional en backend

Criterio SMART	Aplicación concreta
Específico	Obtener y conservar una posición junior enfocada en desarrollo backend con C# y ASP.NET Core.
Medible	Postular a 6 vacantes pertinentes por mes, realizar 2 simulaciones de entrevista mensuales y publicar 2 proyectos backend.
Alcanzable	Utilizará su formación universitaria y proyectos de portafolio para postular a empresas peruanas, consultoras y equipos remotos de la región.
Relevante	La experiencia laboral le permitirá aplicar buenas prácticas y construir una base para asumir responsabilidades técnicas mayores.
Temporal	Conseguirá la posición dentro de los 12 meses posteriores a su graduación y completará 6 meses de experiencia.

Objetivo SMART 2. Fortalecer su perfil en arquitectura de software

Criterio SMART	Aplicación concreta
Específico	Profundizar en arquitectura, Domain-Driven Design y fundamentos de nube aplicados al backend.
Medible	Completar 120 horas de formación, desarrollar 1 proyecto documentado y presentarlo ante un docente, equipo o comunidad profesional.
Alcanzable	Estudiará 4 horas semanales y priorizará recursos gratuitos, de bajo costo o financiados por su empleador.
Relevante	La especialización complementará su experiencia backend y lo preparará para diseñar soluciones de mayor complejidad.
Temporal	Cumplirá la formación y el proyecto dentro de los 24 meses posteriores a su graduación.

Josep Eliu Melgarejo Quiroz

Durante su primer año después de graduarse, Josep construirá evidencias de su capacidad para desarrollar aplicaciones completas y utilizarlas en procesos de selección. Luego fortalecerá sus habilidades de coordinación técnica mediante una experiencia pequeña y controlada, sin asumir responsabilidades superiores a su experiencia. Mantendrá entregas quincenales y revisiones trimestrales para sostener el avance junto con sus obligaciones laborales.

Objetivo SMART 1. Mejorar su empleabilidad como desarrollador de aplicaciones

Criterio SMART	Aplicación concreta
Específico	Crear una aplicación móvil con Flutter y otra solución conectada a una API propia.
Medible	Publicar 2 aplicaciones, recibir retroalimentación de 3 profesionales e incorporar al menos 5 mejoras.
Alcanzable	Definirá alcances pequeños, trabajará en entregas quincenales y reutilizará conocimientos adquiridos durante la carrera.
Relevante	Los proyectos aportarán evidencia concreta para postular a posiciones junior de desarrollo de software.
Temporal	Ambas aplicaciones estarán terminadas dentro de los 12 meses posteriores a su graduación.

Objetivo SMART 2. Desarrollar capacidad de coordinación técnica

Criterio SMART	Aplicación concreta
Específico	Aprender metodologías ágiles y coordinar una iniciativa académica, comunitaria o laboral de 3 a 5 participantes.
Medible	Completar 60 horas de formación, entregar al menos el 80 % del alcance acordado y registrar 3 aprendizajes en una retrospectiva.
Alcanzable	Empezará con un equipo y un alcance reducidos, utilizando un backlog y reuniones breves de seguimiento.
Relevante	La experiencia mejorará su capacidad de colaborar y asumir gradualmente responsabilidades de coordinación.
Temporal	Completará la formación y la iniciativa dentro de los 18 meses posteriores a su graduación.

Saul Ortega Muñoz

Durante el primer año después de graduarse, Saul buscará incorporarse al desarrollo móvil con un portafolio que incluya Android nativo y Flutter. En los seis meses siguientes reforzará la calidad de sus aplicaciones mediante pruebas, revisión de código y seguridad básica. Probará sus soluciones en dispositivos de gama media y con conectividad limitada para aproximarse a condiciones frecuentes de los usuarios peruanos.

Objetivo SMART 1. Ingresar profesionalmente al desarrollo móvil

Criterio SMART	Aplicación concreta
Específico	Ingresar a un equipo o proyecto de desarrollo móvil para Android o Flutter.
Medible	Postular a 6 oportunidades por mes, realizar 2 simulaciones de entrevista mensuales y presentar 2 aplicaciones móviles terminadas.
Alcanzable	Preparará una aplicación con Kotlin y otra con Flutter, y postulará a empresas peruanas y equipos remotos de nivel junior.
Relevante	La experiencia le permitirá convertir los conocimientos del curso en competencias profesionales comprobables.
Temporal	Obtendrá la oportunidad dentro de los 12 meses posteriores a su graduación y completará 6 meses de experiencia o un proyecto remunerado.

Objetivo SMART 2. Especializarse en calidad de aplicaciones móviles

Criterio SMART	Aplicación concreta
Específico	Fortalecer pruebas unitarias, pruebas de interfaz, seguridad básica y generación de versiones móviles.
Medible	Completar 80 horas de formación, cubrir al menos el 70 % de la lógica crítica y corregir todos los defectos de severidad alta detectados.
Alcanzable	Dedicará 3 horas semanales, conservará reportes de prueba y solicitará revisión de código antes de cada versión importante.
Relevante	Estas prácticas mejorarán la estabilidad de sus aplicaciones y su perfil como desarrollador móvil.
Temporal	Completará la formación y las mejoras dentro de los 18 meses posteriores a su graduación.

Luciana Celeste Sanchez Silva

Durante su primer año después de graduarse, Luciana desarrollará productos pequeños y validará sus decisiones con usuarios reales para fortalecer su portafolio. Paralelamente, buscará una posición junior relacionada con desarrollo móvil, análisis funcional o producto digital. Evaluará cada trimestre si la ruta seleccionada continúa siendo compatible con sus intereses, oportunidades laborales y bienestar personal.

Objetivo SMART 1. Crear productos digitales centrados en usuarios

Criterio SMART	Aplicación concreta
Específico	Diseñar o desarrollar 2 productos pequeños vinculados con necesidades reales de educación, servicios, comercio o productividad.
Medible	Validar cada producto con al menos 5 usuarios, aplicar 3 mejoras por producto y publicar 2 casos de estudio.

Criterio SMART	Aplicación concreta
Alcanzable	Elegirá alcances reducidos y podrá trabajar en proyectos laborales, independientes o de voluntariado.
Relevante	Los casos demostrarán su capacidad de combinar desarrollo de software, análisis y experiencia de usuario.
Temporal	Terminará los 2 productos y sus casos de estudio dentro de los 12 meses posteriores a su graduación.

Objetivo SMART 2. Consolidar experiencia en desarrollo móvil o producto digital

Criterio SMART	Aplicación concreta
Específico	Incorporarse a una organización peruana o equipo remoto regional en una función vinculada con productos digitales.
Medible	Postular a 5 vacantes adecuadas por mes y participar en 1 actividad de empleabilidad o comunidad profesional cada 2 meses.
Alcanzable	Adaptará su CV y portafolio al perfil solicitado y reforzará semanalmente la competencia más recurrente en las vacantes.
Relevante	La posición le permitirá aplicar su formación y definir con experiencia real la especialidad que desea desarrollar.
Temporal	Conseguirá la posición dentro de los 15 meses posteriores a su graduación y completará 6 meses continuos de experiencia.

Giuseppe Adrian Villanueva Rodriguez

Durante el primer año después de graduarse, Giuseppe buscará experiencia en aseguramiento de calidad y automatización mediante formación práctica y proyectos demostrables. En el segundo año ampliará su perfil hacia integración y entrega continua para comprender el ciclo completo desde las pruebas hasta el despliegue. Ajustará las herramientas estudiadas según las tecnologías solicitadas en oportunidades laborales reales del mercado peruano.

Objetivo SMART 1. Obtener experiencia en aseguramiento de calidad y automatización

Criterio SMART	Aplicación concreta
Específico	Ingresar a una función junior en la que pueda diseñar, ejecutar y automatizar pruebas de software.
Medible	Postular a 6 oportunidades por mes, completar 60 horas de formación y publicar 2 proyectos que sumen al menos 20 pruebas automatizadas.
Alcanzable	Practicará con APIs y aplicaciones web o móviles, utilizando proyectos propios y herramientas accesibles para construir su portafolio.

Criterio SMART	Aplicación concreta
Relevante	La experiencia fortalecerá su capacidad para prevenir defectos y contribuir a entregas de software confiables.
Temporal	Obtendrá la oportunidad dentro de los 12 meses posteriores a su graduación y completará 6 meses de experiencia o un proyecto remunerado.

Objetivo SMART 2. Ampliar su perfil hacia integración y entrega continua

Criterio SMART	Aplicación concreta
Específico	Aprender Git, contenedores, automatización y fundamentos de nube para crear pipelines de integración continua.
Medible	Completar 80 horas de formación, implementar pipelines en 2 proyectos y recoger al menos 3 recomendaciones profesionales.
Alcanzable	Avanzará por etapas y dedicará entre 3 y 4 horas semanales, priorizando herramientas presentes en vacantes reales.
Relevante	La automatización complementará su perfil de calidad y le permitirá participar en procesos de entrega más seguros.
Temporal	Finalizará la formación y los 2 pipelines dentro de los 24 meses posteriores a su graduación.

Capítulo I: Presentación

1.1. Startup Profile

En esta sección se presenta la startup responsable de AniTec y a los integrantes del equipo que desarrollarán la solución durante el curso de Aplicaciones para Dispositivos Móviles.

1.1.1. Descripción de la Startup

Titan es una startup tecnológica que desarrolla soluciones digitales para mejorar la gestión de pequeños y medianos productores ganaderos. Su producto, AniTec, nació como una aplicación web para centralizar la información sanitaria, productiva y económica del ganado. En este curso, la propuesta evoluciona hacia una solución móvil que permita realizar las tareas principales desde el teléfono, dispositivo que acompaña al usuario durante el trabajo de campo.

AniTec contará con una aplicación nativa para Android desarrollada con Kotlin y con una experiencia multiplataforma. La tecnología de esta última se seleccionará mediante una evaluación técnica entre las alternativas permitidas por el curso. Ambas aplicaciones consumirán la API REST propia de AniTec y ofrecerán una experiencia coherente, accesible e internacionalizada.

La solución móvil se concentrará en las siguientes capacidades:

- Registrar, consultar y actualizar animales y sus datos principales.
- Consultar el historial sanitario y registrar atenciones, tratamientos y otros eventos.
- Guardar borradores o información esencial localmente para afrontar conexiones inestables.
- Programar y recibir recordatorios de vacunaciones, tratamientos y actividades.
- Identificar animales con apoyo de la cámara, manteniendo una búsqueda manual como alternativa.
- Consultar indicadores y resúmenes visuales que faciliten decisiones de campo.
- Permitir que los veterinarios consulten los animales autorizados por sus clientes y registren el seguimiento correspondiente.
- Gestionar planes mediante un servicio externo de pagos, sujeto a validación técnica y funcional.

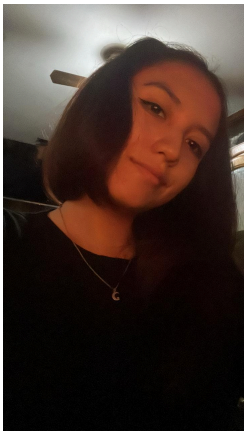
El ecosistema se complementará con la landing page existente, que comunicará la propuesta de valor y dirigirá a la descarga de las aplicaciones. El desarrollo también incluirá persistencia local, acceso a capacidades del dispositivo, manejo de errores de red, accesibilidad e internacionalización. Además, se incorporará una capacidad autónoma basada en una biblioteca o servicio no utilizado previamente en clase, cuya viabilidad se comprobará mediante una Spike Story antes de integrarla al producto.

Misión: Facilitar la gestión y trazabilidad del ganado de pequeños y medianos productores mediante una experiencia móvil simple, accesible y útil en el trabajo de campo, que conecte al ganadero con la información de sus animales y con el seguimiento veterinario.

Visión: Convertir a AniTec en una solución móvil confiable para la gestión ganadera en Latinoamérica, reconocida por adaptarse a las condiciones reales del entorno rural y por ayudar a tomar decisiones sanitarias y productivas basadas en información ordenada.

1.1.2. Perfiles de integrantes del equipo

Luciana Celeste Sanchez Silva



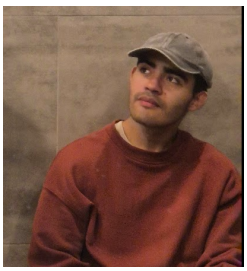
U202215979

Mi nombre es Luciana Celeste Sanchez Silva, tengo 21 años y vivo en Lima. En la actualidad, me encuentro estudiando el octavo ciclo de la carrera de ingeniería de software en la UPC debido a que desde una edad temprana tuve una fascinación relacionada con el uso de la tecnología y la programación. En mi tiempo libre trato de crecer y expandir mi conocimiento en todas las áreas posibles. De igual forma, me gusta nadar, escuchar música y tocar la guitarra. Me comprometo a colaborar en todo momento con la elaboración de esta startup y llegar a un trabajo sobresaliente. Mis habilidades son: responsabilidad, resolución de problemas y disciplina.

Josep Eliu Melgarejo Quiroz

u202315165

Mi nombre es Josep Eliu Melgarejo Quiroz, tengo 21 años y nací en Huaral, aunque actualmente vivo en San Miguel, Lima. Me encuentro cursando el 5.º ciclo de la carrera de Ingeniería de Software en la UPC. Siempre me fascinó el ámbito tecnológico y, como era un apasionado por los juegos, fui conociendo poco a poco el mundo de la programación, lo que me llevó a decidir estudiar esta carrera. Me comprometo a apoyar y motivar siempre a mis compañeros para realizar el mejor trabajo posible y dar el 100 % de mi capacidad en este proyecto.



Sebastian Martin Beingolea Montalvo

U202217853

Mi nombre es Sebastian Martin Beingolea Montalvo, tengo 21 años y vivo en Lima. Actualmente estudio la carrera de Ingeniería de Software en la UPC, motivado por el interés en crear soluciones tecnológicas que aporten valor real a las personas y a los negocios. Me gusta el desarrollo de software, el trabajo en equipo y aprender nuevas herramientas que puedan aplicar en proyectos como AniTec. En mi tiempo libre disfruto explorar tecnologías, escuchar música y seguir mejorando mis habilidades de programación. Me comprometo a colaborar de forma constante con mis compañeros, aportar ideas y contribuir a

un entregable de calidad. Mis habilidades son: responsabilidad, trabajo en equipo, capacidad de aprendizaje y orientación a resultados.

Saul Ortega Muñoz

u20231c019

Soy estudiante de Ingeniería de Software con formación en programación y desarrollo web. Tengo conocimientos en algunos lenguajes de programación. Me caracterizo por ser responsable, con capacidad de aprendizaje rápido y trabajo en equipo, aportando ideas prácticas para resolver problemas y mejorar proyectos tecnológicos.

Giuseppe Villanueva Rodriguez

u20221c554

Soy estudiante de Ingeniería de Software con habilidades en programación. Poseo conocimientos en algunos lenguajes de programación. Me caracterizo por ser empático y siempre atento cuando se trata de trabajo en equipo.

1.2. Solution Profile

1.2.1. Antecedentes y problemática

Qué (What)

¿Cuál es la situación problemática?

Los pequeños y medianos ganaderos suelen distribuir la información de sus animales entre cuadernos, hojas sueltas, archivos y mensajes. Esto dificulta mantener historiales completos, encontrar datos durante una atención, recordar actividades sanitarias y compartir información confiable con el veterinario. La aplicación web tomada como base centraliza parte de esos datos, pero no cubre por sí sola la necesidad de registrar y consultar información mientras el usuario se desplaza por el campo.

La literatura reciente respalda esta necesidad. Las soluciones digitales aplicadas a la ganadería pueden mejorar el monitoreo de la salud, la productividad y el uso de recursos, aunque su adopción todavía depende de factores como el costo, la infraestructura y la integración tecnológica (Papadopoulos et al., 2025). Asimismo, los sistemas de monitoreo en tiempo real y apoyo a decisiones permiten transformar los datos del hato en alertas y acciones oportunas para una gestión más resiliente (Nsabiyeze et al., 2025).

Cuándo (When)

¿Cuándo ocurre el problema?

El problema aparece durante todo el ciclo de manejo del animal: registro o nacimiento, vacunaciones, tratamientos, controles, reproducción, cambios productivos y venta. Se vuelve especialmente crítico cuando una actividad debe registrarse en el momento o cuando se necesita consultar el historial para decidir una atención.

Dónde (Where)

¿Dónde se manifiesta y dónde se origina?

Se manifiesta en unidades ganaderas rurales y semiurbanas, durante recorridos de campo y atenciones veterinarias. En estos entornos puede haber acceso limitado a una computadora y conectividad móvil intermitente, por lo que depender exclusivamente de una aplicación web o de una conexión constante interrumpe el flujo de trabajo.

Quién (Who)

¿Quiénes participan y quiénes usarán la solución?

Los usuarios principales serán pequeños y medianos ganaderos responsables del manejo cotidiano del hato y veterinarios de campo que necesitan consultar antecedentes y registrar atenciones. Asociaciones, cooperativas y entidades del sector son actores relacionados, pero no constituyen los segmentos de usuario priorizados para la primera versión.

Por qué (Why)

¿Cuál es la causa principal del problema?

Las herramientas disponibles no siempre se ajustan a la movilidad del trabajo ganadero, al nivel de experiencia digital de los usuarios ni a las restricciones de conectividad. Como consecuencia, el registro se posterga, la información queda incompleta o dispersa y la coordinación entre ganadero y veterinario depende de comunicaciones informales.

Cómo (How)

¿Cómo se implementará la solución?

AniTec extenderá el producto web existente con una aplicación nativa Android desarrollada con Kotlin y una aplicación multiplataforma cuya tecnología se elegirá mediante evaluación técnica. Las aplicaciones consumirán la API REST propia y permitirán registrar animales y eventos, consultar historiales, recibir notificaciones, utilizar la cámara, revisar indicadores y conservar temporalmente datos o borradores en el dispositivo.

¿Cómo se logrará una experiencia útil en campo?

Se diseñarán flujos breves, formularios con validación, estados claros de sincronización y alternativas manuales cuando una función del dispositivo no esté disponible. La solución considerará accesibilidad, internacionalización, seguridad de acceso y pruebas en dispositivos físicos bajo distintas condiciones de conectividad.

Cuánto (How much)

¿Cuál es la magnitud del contexto?

El IV Censo Nacional Agropecuario registró 2 199 243 productores agropecuarios como personas naturales y señaló que el 79,6 % tenía unidades de menos de cinco hectáreas (INEI, 2014). Además, durante el primer trimestre de 2025 solo el 20,5 % de los hogares rurales disponía de Internet en el hogar (INEI, 2025). Estas cifras no cuantifican por sí solas el mercado de AniTec, pero muestran la escala de la pequeña producción agropecuaria y la necesidad de diseñar para un entorno de conectividad restringida.

Descripción consolidada de los antecedentes y la problemática

AniTec parte de una aplicación web creada en un curso anterior. El reto actual consiste en adaptar y ampliar esa solución para el uso móvil, sin limitarse a reproducir sus pantallas. El teléfono debe convertirse en la herramienta de trabajo inmediata del ganadero y del veterinario: permitir el registro en el lugar donde ocurre el evento, presentar la información relevante con rapidez y tolerar interrupciones de red.

La propuesta comprenderá una landing page, una API REST propia, una aplicación Android nativa y una experiencia multiplataforma. El alcance inicial priorizará la gestión de animales e historiales sanitarios, el trabajo local y la posterior sincronización, las notificaciones, el uso de cámara, los indicadores de seguimiento y la colaboración autorizada con veterinarios. Las integraciones complementarias, como pagos o una capacidad autónoma basada en una tecnología nueva para el equipo, se incorporarán después de validar su viabilidad y su aporte al usuario.

El objetivo del proyecto es reducir la pérdida y fragmentación de información, facilitar el cumplimiento de actividades sanitarias y mejorar el acceso al historial del animal durante el trabajo de campo. La solución deberá mantener coherencia entre plataformas, proteger los datos, comunicar los estados de conexión y funcionar de manera comprensible para usuarios con distintos niveles de experiencia digital.

1.2.2. Lean UX Process.

1.2.2.1. Lean UX Problem Statements.

Problem Statement:

El estado actual de la gestión ganadera para pequeños y medianos productores se ha centrado principalmente en controles manuales, registros en cuadernos y herramientas digitales improvisadas para administrar la información sanitaria, reproductiva y económica del hato.

Lo que los productos y servicios existentes no abordan es la necesidad de contar con una aplicación para Android sencilla, accesible y adaptada al trabajo de campo, que permita centralizar la información del ganado y mantener el registro de las actividades principales incluso cuando la conexión a Internet sea inestable.

Nuestro producto, AniTec, abordará esta brecha mediante una aplicación móvil para Android que permitirá registrar, organizar y consultar la información del ganado desde el teléfono. La solución incluirá recordatorios sanitarios, historiales por animal, reportes y almacenamiento local de información esencial para reducir errores, evitar la pérdida de datos y facilitar la toma de decisiones.

Nuestro enfoque inicial será pequeños y medianos ganaderos que actualmente dependen de registros manuales o sistemas poco organizados para gestionar su producción, junto con veterinarios que necesitan consultar información sanitaria y dar seguimiento a los animales de sus clientes desde un dispositivo Android.

Sabremos que hemos tenido éxito cuando observemos una reducción en el uso de registros manuales, un aumento en la frecuencia y precisión de los registros realizados desde la aplicación, una mejora en el cumplimiento de vacunaciones y tratamientos, y una consulta más rápida de los historiales sanitarios por parte de ganaderos y veterinarios.

1.2.2.2. Lean UX Assumptions.

Business Assumptions:

1. **Creemos que nuestros usuarios necesitan** un método confiable y eficiente para registrar y supervisar desde el teléfono la salud, productividad y trazabilidad de su ganado.
2. **Creemos que esta necesidad puede satisfacerse** mediante una aplicación para Android que permita registrar información clave, conservar datos esenciales en el dispositivo, generar alertas y consultar reportes útiles para la toma de decisiones.
3. **Creemos que nuestros primeros usuarios serán** pequeños y medianos ganaderos que utilizan teléfonos Android, así como veterinarios y técnicos agropecuarios que asesoran directamente en el campo.
4. **Creemos que lo más importante para los clientes es** contar con un control ordenado y disponible durante sus actividades, evitando pérdidas de información y mejorando la trazabilidad del ganado.
5. **Creemos que los usuarios también recibirán** alertas sanitarias mediante notificaciones, reportes económicos, acceso al historial de cada animal y contenido educativo dentro de la aplicación.
6. **Creemos que conseguiremos clientes mediante** alianzas con asociaciones ganaderas, programas de desarrollo rural, recomendaciones de veterinarios y campañas digitales dirigidas a regiones con actividad ganadera.

7. **Creemos que los ingresos se generarán mediante** un modelo de suscripción con planes ajustados al tamaño del hato y licencias institucionales para asociaciones y entidades del sector agropecuario.
8. **Creemos que nuestra competencia incluye** aplicaciones genéricas de gestión ganadera, hojas de cálculo y métodos tradicionales de registro manual.
9. **Creemos que nuestra ventaja competitiva radica en** ofrecer una aplicación Android adaptada al contexto rural, fácil de usar y preparada para conservar el progreso cuando la conectividad sea limitada.
10. **Creemos que un riesgo importante es** que algunos ganaderos no adopten fácilmente la aplicación por falta de experiencia digital, conectividad inestable o limitaciones de sus dispositivos.
11. **Creemos que lo mitigaremos mediante** una interfaz intuitiva, tutoriales paso a paso, formularios breves, almacenamiento local y mensajes claros sobre el estado de sincronización.

User Assumptions:

- **Creemos que** los principales usuarios son pequeños y medianos ganaderos, veterinarios y técnicos agropecuarios que utilizan dispositivos Android durante sus actividades. En etapas posteriores, la aplicación también podría apoyar a asociaciones, cooperativas y entidades vinculadas con la sanidad y trazabilidad del sector.

- **Creemos que** AniTec ayuda a organizar la información del hato desde el teléfono, evitando la pérdida de datos importantes y la falta de seguimiento de vacunas, partos, tratamientos y movimientos. Para los veterinarios, facilita la consulta de clientes, pacientes e historiales sanitarios durante una atención.

- **Creemos que** los usuarios valoran el registro individual de cada animal, las notificaciones sobre actividades pendientes, los reportes simples, el historial del hato y la posibilidad de conservar registros cuando la conexión sea inestable. También creemos que la facilidad de uso es esencial para su adopción.

- **Creemos que** AniTec se integra en la rutina del ganadero porque puede utilizarse en el campo cada vez que se registra un animal, tratamiento, parto, vacunación o cambio productivo. Los veterinarios pueden utilizarla antes, durante y después de una visita para consultar antecedentes y registrar la atención.

- **Creemos que** AniTec debe ofrecer una experiencia Android intuitiva, legible y estable, pensada para usuarios con distintos niveles de experiencia tecnológica. También debe proteger los datos, comunicar los errores de conexión y permitir recuperar el trabajo pendiente.

Feature Assumptions:

Feature Assumption 01

Creemos que una funcionalidad de registro móvil centralizado permitirá a los pequeños y medianos ganaderos gestionar la información sanitaria, reproductiva y económica de sus animales desde un dispositivo Android.

Sabremos que esta funcionalidad es valiosa cuando la mayoría de los usuarios registre y actualice periódicamente la información de sus animales mediante la aplicación.

Feature Assumption 02

Creemos que un sistema de notificaciones para vacunaciones, tratamientos y eventos reproductivos ayudará a los ganaderos a recordar actividades importantes y reducir los descuidos en el manejo del hato.

Sabremos que esta funcionalidad es valiosa cuando los usuarios atiendan o reprogramen los recordatorios recibidos en sus dispositivos Android.

Feature Assumption 03

Creemos que un módulo de reportes visuales e historial de cada animal facilitará la interpretación de la información y permitirá tomar mejores decisiones productivas, reproductivas y económicas.

Sabremos que esta funcionalidad es valiosa cuando los usuarios consulten los reportes y antecedentes antes de realizar acciones relacionadas con la gestión del ganado.

Feature Assumption 04

Creemos que el almacenamiento local de información esencial y borradores permitirá que los usuarios continúen registrando actividades cuando la conexión a Internet sea inestable.

Sabremos que esta funcionalidad es valiosa cuando los usuarios puedan recuperar y sincronizar sus registros sin repetir la información ingresada.

Feature Assumption 05

Creemos que un módulo para veterinarios con acceso autorizado a clientes, historiales y eventos sanitarios facilitará el seguimiento de los animales y mejorará la colaboración con los ganaderos.

Sabremos que esta funcionalidad es valiosa cuando los veterinarios utilicen la aplicación Android para registrar y consultar la información sanitaria de los animales atendidos.

1.2.2.3. Lean UX Hypothesis Statements.

- **Hypothesis Statement 01:**

  Creemos que lograremos una mayor adopción y un uso continuo de AniTec

  Si los pequeños y medianos ganaderos

  Obtienen una forma sencilla de registrar y consultar la información sanitaria, reproductiva y económica de sus animales desde el teléfono

  Con una aplicación para Android que centralice y organice la información del hato.

- **Hypothesis Statement 02:**

  Creemos que lograremos mejorar la gestión sanitaria del hato y reducir actividades olvidadas

  Si los pequeños y medianos ganaderos

  Obtienen recordatorios oportunos sobre vacunaciones, tratamientos y eventos reproductivos

Con un sistema de alertas y notificaciones integrado en la aplicación Android.

- **Hypothesis Statement 03:**

  Creemos que lograremos una mejor toma de decisiones sobre el manejo del ganado

  Si los pequeños y medianos ganaderos

  Obtienen acceso desde el teléfono a reportes visuales y al historial de cada animal

  Con un módulo móvil de reportes e historiales organizado y fácil de interpretar.

- **Hypothesis Statement 04:**

  Creemos que lograremos reducir la pérdida de información causada por problemas de conectividad

  Si los pequeños y medianos ganaderos y veterinarios

  Obtienen la posibilidad de conservar el progreso cuando no dispongan de una conexión estable

  Con almacenamiento local y sincronización posterior dentro de la aplicación Android.

- **Hypothesis Statement 05:**

  Creemos que lograremos una mejor colaboración entre ganaderos y veterinarios y un seguimiento sanitario más eficiente

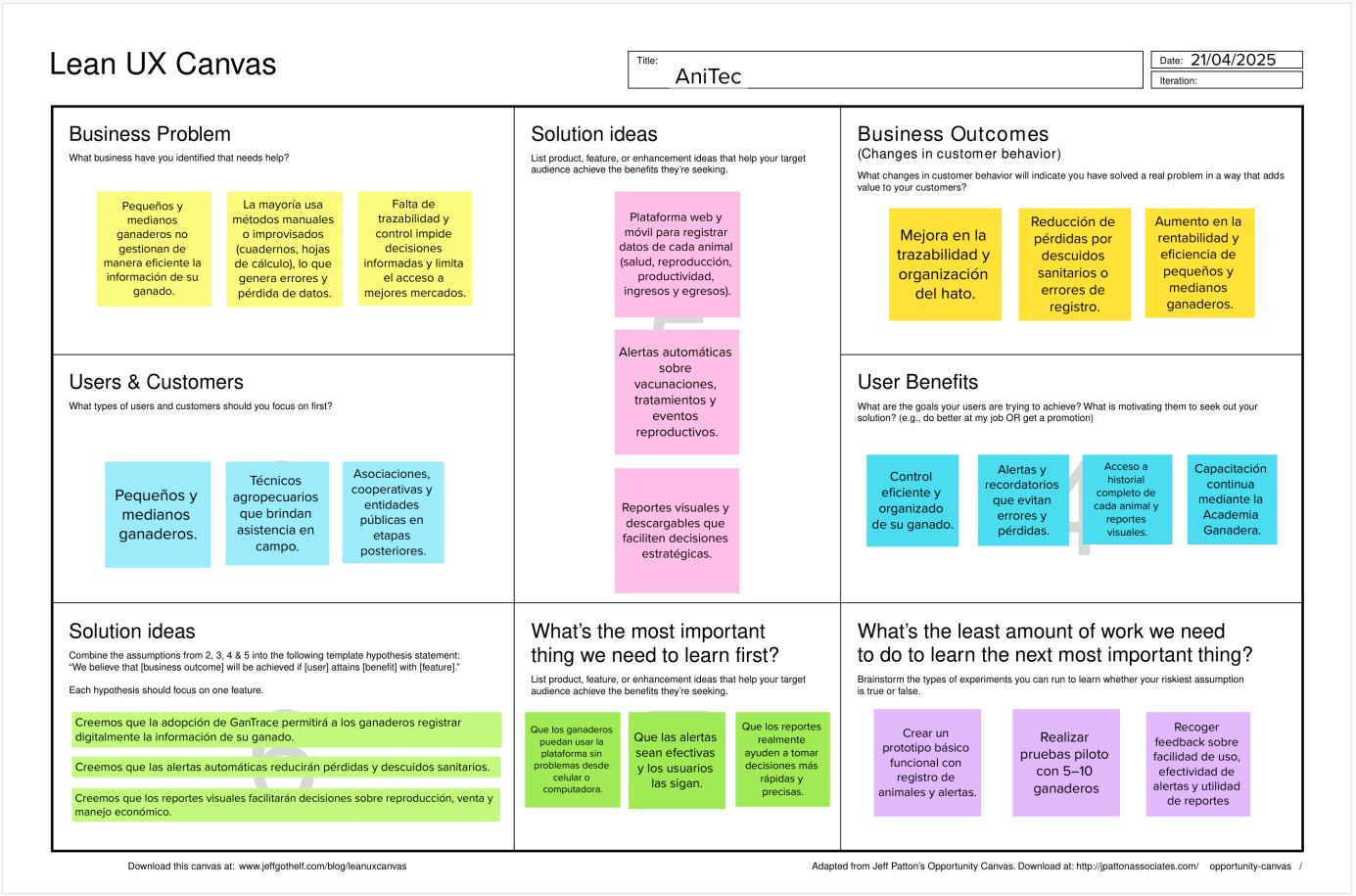
  Si los veterinarios

  Obtienen acceso autorizado a los clientes asignados, al historial de los animales y al registro de eventos sanitarios

  Con un módulo para veterinarios integrado en la aplicación Android de AniTec.

1.2.2.4. Lean UX Canvas.

El Lean UX Canvas es una herramienta utilizada en el marco del diseño centrado en el usuario (UX) y la metodología Lean, cuyo objetivo es apoyar la creación y mejora de productos de manera ágil y eficiente. Su propósito principal es proporcionar una estructura organizada que fomente la colaboración entre equipos multidisciplinarios. A continuación, se presenta el Lean UX Canvas elaborado por el equipo utilizando la plataforma digital Mural.



Enlace para acceder al Lean UX Canvas en Mural: https://tinyurl.com/LeanUxCanvasMural

1.3. Segmentos objetivo

AniTec prioriza dos segmentos que participan directamente en el registro y uso de la información ganadera: pequeños y medianos ganaderos y veterinarios de campo. Esta delimitación permite diseñar los flujos móviles alrededor de tareas concretas y evita considerar como usuarios principales a organizaciones que solo cumplen un rol de apoyo o influencia.

El IV Censo Nacional Agropecuario registró 2 199 243 productores agropecuarios como personas naturales y encontró que el 79,6 % tenía unidades de menos de cinco hectáreas (INEI, 2014). Aunque estas cifras incluyen distintas actividades agropecuarias y no representan por sí solas el mercado de AniTec, evidencian la relevancia de la pequeña producción. El diseño también debe considerar la brecha de conectividad: en el primer trimestre de 2025, el 20,5 % de los hogares rurales disponía de Internet en el hogar (INEI, 2025). A la vez, el teléfono móvil es el principal medio de acceso: en el cuarto trimestre de 2025, el 89,2 % de las personas usuarias de Internet se conectó mediante un celular y, en el área rural, la proporción fue de 88,4 % (INEI, 2026). Estas condiciones respaldan una experiencia centrada en el teléfono, con persistencia local y tolerancia a conexiones intermitentes.

Pequeños y medianos ganaderos

Son productores responsables del manejo cotidiano de un hato pequeño o mediano en zonas rurales o semiurbanas. Registran nacimientos, vacunaciones, tratamientos, reproducción, producción y movimientos de animales. Cuando esta información se conserva en cuadernos, mensajes o archivos separados, necesitan invertir más tiempo para encontrarla y corren el riesgo de perder continuidad en el seguimiento.

Las entrevistas exploratorias disponibles comprenden tres ganaderos de entre 54 y 65 años que utilizan teléfonos Android y aplicaciones de comunicación como WhatsApp. Esta muestra ayuda a formular supuestos iniciales, pero no permite generalizar sus resultados al conjunto del segmento; deberá ampliarse y contrastarse durante la validación.

Característica	**Descripción**
Contexto de uso	Campo, establo o vivienda, con acceso variable a Internet y uso frecuente del teléfono.
Necesidades principales	Registrar animales y eventos en el momento, consultar historiales, recordar actividades y revisar el estado general del hato.
Dificultades actuales	Información dispersa, pérdida u omisión de datos, demora al buscar antecedentes y conectividad inestable.
Valor esperado	Menos tiempo de registro y búsqueda, mayor continuidad sanitaria y decisiones sustentadas en información organizada.
Consideraciones de diseño	Flujos cortos, lenguaje simple, controles legibles, ayuda contextual, almacenamiento local y estados de sincronización comprensibles.

Veterinarios de campo

Son profesionales que atienden animales de distintos productores y necesitan revisar antecedentes, registrar diagnósticos, tratamientos y recomendaciones, y dar seguimiento a los casos. Requieren acceso únicamente a los clientes y animales que les hayan concedido autorización, además de una forma clara de comunicar lo registrado al ganadero.

Las entrevistas exploratorias disponibles comprenden dos veterinarios de entre 24 y 27 años. Sus respuestas orientan los primeros supuestos sobre movilidad y consulta de historiales, pero la muestra aún no satisface el mínimo de tres a cinco entrevistas exigido para cada segmento. Por ello, será necesario entrevistar al menos a un veterinario adicional y actualizar el análisis antes de cerrar la validación del problema.

Característica	**Descripción**
Contexto de uso	Atención en campo, desplazamiento entre clientes y consulta desde el teléfono antes, durante o después de una visita.
Necesidades principales	Acceder con autorización al historial del animal, registrar atenciones y revisar pendientes de seguimiento.
Dificultades actuales	Antecedentes incompletos, información repartida entre conversaciones y documentos, y demora para coordinar con el productor.
Valor esperado	Consulta rápida de antecedentes, registros sanitarios consistentes y mejor continuidad entre visitas.
Consideraciones de diseño	Permisos explícitos, identificación clara del cliente y del animal, formularios breves, trazabilidad de cambios y funcionamiento ante fallas de red.

Las asociaciones ganaderas, cooperativas, técnicos agropecuarios y entidades públicas son stakeholders externos que pueden facilitar adopción, capacitación o articulación. El equipo de desarrollo es un stakeholder interno. Estos actores se documentarán en el análisis correspondiente, pero no sustituyen a los dos segmentos objetivo priorizados.

Capítulo II: Requirements Development and Software Solution Design

2.1. Competidores

El análisis competitivo permite conocer cómo otras soluciones digitales atienden la gestión ganadera y qué aspectos debe validar AniTec para diferenciarse. Se seleccionaron tres competidores directos con productos móviles vigentes: Livestock Manager, AgriWebb e iLivestock. La comparación se concentra en las capacidades relevantes para la aplicación Android de AniTec y utiliza información publicada en los sitios oficiales de cada producto, consultada el 13 de septiembre de 2026.

2.1.1. Análisis competitivo

Pregunta que orienta el análisis

¿Cómo puede AniTec diferenciar su aplicación Android frente a otras soluciones móviles de gestión ganadera en facilidad de uso, continuidad ante conexiones inestables, seguimiento veterinario y adaptación a pequeños y medianos ganaderos peruanos?

Competitive Analysis Landscape

Aspecto	AniTec	Livestock Manager	AgriWebb	iLivestock
		Perfil		
Clasificación	Startup analizada.	Competidor directo.	Competidor directo.	Competidor directo.
Overview	Solución de gestión ganadera orientada a pequeños y medianos ganaderos y veterinarios de campo. Parte de una landing page, una API REST y una aplicación web desarrolladas previamente. En el curso se desarrollará una aplicación Android nativa y se definirá la experiencia multiplataforma requerida.	Producto de Pila para gestionar animales y lotes de distintas especies desde el teléfono y la web. Integra registros de salud, alimentación, reproducción y finanzas. Su sitio declara más de 50 000 descargas, 24 especies y 15 idiomas.	Plataforma de gestión ganadera que trabaja con animales individuales o grupos. Permite registrar actividades, movimientos, tratamientos, pesos, alimentación, inventarios y tareas desde aplicaciones móviles y una plataforma web.	Plataforma para productores de ganado bovino y ovino. Reúne registros de salud, reproducción, movimientos, pesos, tratamientos y rendimiento, y se integra con equipos de identificación electrónica y pesaje.

Aspecto	AniTec	Livestock Manager	AgriWebb	iLivestock
Ventaja competitiva	Propuesta de experiencia simple en español, adaptada al contexto de pequeños productores peruanos y con un flujo de colaboración autorizado entre ganadero y veterinario. Esta ventaja todavía debe comprobarse con usuarios.	Barrera de entrada baja mediante un plan gratuito, cobertura de múltiples especies y disponibilidad tanto móvil como web.	Amplitud funcional para operaciones ganaderas, mapas interactivos, trabajo sin conexión e integración con lectores EID, clima y dispositivos IoT.	Integración de software y hardware para identificación EID y pesaje, funcionamiento sin conexión y herramientas especializadas para ganado bovino y ovino.
Valor que ofrece a los clientes	Centralizar animales e historiales, registrar actividades desde el campo, conservar datos esenciales ante interrupciones de red, recibir recordatorios y facilitar el seguimiento sanitario compartido.	Reunir registros productivos y económicos de distintas especies en una herramienta económica y accesible desde varios dispositivos.	Registrar las operaciones del campo en el momento, mantener inventarios y movimientos actualizados y sincronizar los datos cuando se recupera la conexión.	Reducir el trabajo manual de identificación y pesaje, mantener la trazabilidad y convertir los registros del ganado en reportes de desempeño.
	Perfil de Marketing			
Mercado objetivo	Pequeños y medianos ganaderos peruanos y veterinarios de campo que emplean teléfonos Android y gestionan información dispersa.	Desde propietarios de granjas pequeñas y aficionados hasta productores profesionales y organizaciones con múltiples granjas.	Productores y equipos ganaderos que administran bovinos u ovinos de forma individual o por grupos y requieren coordinación operativa.	Productores de bovinos y ovinos, desde hatos pequeños hasta operaciones comerciales que utilizan registros, identificación EID o sistemas de pesaje.
Estrategias de marketing observables	Propuesta preliminar basada en la landing page, demostraciones con usuarios, recomendaciones de veterinarios y alianzas con	Modelo freemium sin tarjeta para iniciar, distribución en tiendas de aplicaciones, prueba inmediata del	Prueba gratuita, demostración interactiva, contenido educativo y contacto con ventas para operaciones que	Prueba gratuita de 14 días, presencia en tiendas de aplicaciones, distribuidores regionales y venta

Aspecto	AniTec	Livestock Manager	AgriWebb	iLivestock
	asociaciones ganaderas. Todavía no existe evidencia de adquisición o retención.	producto y uso de testimonios y cifras de descargas como prueba social.	requieren configuración o integraciones.	combinada de software, lectores, balanzas y etiquetas.
		Perfil de Producto		
Productos y servicios	Landing page y API REST existentes; aplicación Android nativa en desarrollo. El alcance móvil prioriza animales, historiales sanitarios, notificaciones, almacenamiento local, reportes y acceso veterinario autorizado. La solución multiplataforma se definirá durante el curso.	Registro de animales y lotes, salud, medicamentos, alimentación, reproducción, finanzas y reportes. Los planes superiores incorporan colaboración, facturación, certificados, integraciones y acceso mediante API.	Gestión individual o por grupos, mapas, movimientos, tratamientos, pesos, alimentación, reproducción, inventarios, tareas, reportes e integraciones con hardware y servicios externos.	Registros de bovinos y ovinos, grupos, tratamientos, medicamentos, movimientos, reproducción, pesos, rendimiento, reportes e integración con lectores EID y equipos de pesaje.
Precios y costos	Precio aún no definido. El modelo de suscripción y la disposición de pago son hipótesis que deben validarse. Las cifras heredadas de la landing page y del backend no se consideran precios comerciales aprobados.	Starter: gratuito. Growth: desde USD 3,99 y Scale: desde USD 9,99, mostrados como importes mensuales con facturación anual. Enterprise: cotización personalizada.	Precio calculado según la cantidad de animales y la región. El sitio solicita ingresar el número de bovinos u ovinos o contactar con ventas; también ofrece complementos con costo adicional.	Software: GBP 27,50 mensuales o GBP 300 anuales. Ofrece una prueba gratuita de 14 días; el hardware y las etiquetas se comercializan por separado.
Canales de distribución	Landing page web y, cuando se implemente y publique, Google Play para la aplicación Android. El canal de la aplicación multiplataforma permanece por definir.	Aplicaciones para Android e iOS y acceso web o de escritorio.	Aplicaciones para Android e iOS y aplicación web.	Aplicaciones para Android e iOS y acceso desde smartphone, tableta, PC o Mac.

Los precios corresponden a la información visible en los sitios oficiales en la fecha de consulta y pueden cambiar según país, cantidad de animales, modalidad de facturación o configuración contratada. En el caso de AniTec, cualquier precio deberá presentarse como una propuesta hasta que sea validado con los segmentos objetivo y exista una decisión comercial consistente.

Análisis FODA

El siguiente análisis interpreta la oferta observable de cada producto en relación con el objetivo de AniTec. Las fortalezas y debilidades describen condiciones internas de cada solución; las oportunidades y amenazas consideran su posición frente al mercado y los demás competidores.

Dimensión	AniTec	Livestock Manager	AgriWebb	iLivestock
Fortalezas	Conocimiento inicial de los segmentos locales; propuesta en español; reutilización de una API y producto web existentes; flujo diferenciado entre ganadero y veterinario.	Plan gratuito; precio inicial bajo; soporte para numerosas especies e idiomas; disponibilidad móvil y web; base pública de descargas.	Trabajo online y offline; gestión ganadera amplia; sincronización; mapas; roles de equipo e integraciones con EID e IoT.	Especialización en bovinos y ovinos; trabajo offline; integración propia con lectores y pesaje; trazabilidad y reportes de rendimiento.
Debilidades	Aplicación móvil aún no implementada; propuesta de precio sin validar; ausencia de usuarios activos demostrables; reconocimiento de marca limitado y capacidades offline aún por comprobar.	Su cobertura de muchas especies puede ofrecer menos profundidad para un flujo veterinario especializado; la propuesta pública no destaca adaptación específica al contexto peruano.	La amplitud de funciones e integraciones puede elevar el esfuerzo de configuración y aprendizaje; el precio final no es visible sin introducir datos del hato o contactar con ventas.	Parte importante de su diferenciación depende del ecosistema de hardware y EID; el precio se publica en libras esterlinas y varias integraciones regulatorias dependen del país.
Oportunidades	Atender a pequeños productores peruanos con lenguaje, soporte y flujos validados localmente; simplificar la colaboración con veterinarios; diseñar para teléfonos Android y conectividad intermitente.	Convertir usuarios del plan gratuito a funciones avanzadas y ampliar sus capacidades de colaboración y automatización.	Extender sus integraciones, análisis de rendimiento y servicios para operaciones ganaderas de mayor complejidad.	Expandir su red de distribuidores y alianzas en Latinoamérica, y aumentar la adopción de identificación y pesaje digital.
Amenazas	Competidores consolidados ya	Soluciones especializadas por	Aplicaciones más simples y económicas	Alternativas que funcionan sin

Dimensión	AniTec	Livestock Manager	AgriWebb	iLivestock
	ofrecen trabajo offline, aplicaciones Android y precios de entrada bajos; resistencia a la adopción, problemas de conectividad y desconfianza sobre el manejo de datos.	especie o región pueden ofrecer flujos más profundos; plataformas integrales pueden competir mediante mapas, hardware y cumplimiento.	pueden resultar suficientes para productores pequeños que no necesitan todas sus capacidades.	hardware adicional o con planes gratuitos pueden reducir la adopción entre productores sensibles al precio.

El Landscape muestra que el funcionamiento sin conexión ya está presente en AgriWebb e iLivestock, mientras que Livestock Manager compite con un plan gratuito y precios de entrada bajos. Por ello, AniTec no debe presentar el almacenamiento local ni la accesibilidad económica como ventajas únicas sin validación. Su diferenciación deberá comprobarse principalmente en la adaptación al contexto peruano, la simplicidad de los flujos Android y la colaboración autorizada entre ganadero y veterinario.

Fuentes del análisis competitivo

- Livestock Manager: producto, funciones y precios.
- AgriWebb: aplicación móvil y funcionamiento offline.
- AgriWebb: planes e integraciones.
- iLivestock: software, funciones y precio.
- iLivestock: presencia y alianzas en Latinoamérica.

2.1.2. Estrategias y tácticas frente a competidores

Las siguientes estrategias son preliminares y se derivan de los hallazgos del Landscape y del análisis FODA. Sus indicadores permitirán comprobar si las tácticas producen el resultado esperado.

Hallazgo competitivo	Estrategia de AniTec	Táctica preliminar	Indicador de validación
AgriWebb e iLivestock ya permiten trabajar sin conexión.	Alcanzar una continuidad de uso comparable y ofrecer estados de sincronización comprensibles.	Permitir guardar datos esenciales y borradores localmente, mostrar si existen cambios pendientes y reintentar la sincronización al recuperar la conexión.	Porcentaje de registros recuperados y sincronizados sin duplicación; cantidad de tareas abandonadas por errores de red.
Livestock Manager reduce la barrera de entrada con un plan gratuito y precios bajos.	Definir una propuesta de planes ajustada al valor percibido por pequeños productores.	Probar con los segmentos una versión gratuita o de evaluación, límites por cantidad de animales y funciones pagadas antes de fijar precios comerciales.	Comprensión de los planes, disposición de pago y porcentaje de participantes que elegiría cada alternativa.

Hallazgo competitivo	Estrategia de AniTec	Táctica preliminar	Indicador de validación
Los competidores ofrecen productos amplios para mercados internacionales.	Adaptar AniTec al contexto de los ganaderos peruanos.	Validar con usuarios el lenguaje, los formularios, las categorías sanitarias, el proceso de incorporación y las condiciones reales de uso en teléfonos Android.	Porcentaje de tareas completadas sin ayuda, tiempo por tarea y problemas de comprensión encontrados.
Las ofertas revisadas gestionan equipos o usuarios, pero no presentan como eje principal una relación ganadero-veterinario adaptada al mercado local.	Diferenciar AniTec mediante seguimiento sanitario autorizado.	Implementar permisos explícitos para asignar veterinarios, consultar historiales y registrar atenciones, mostrando al ganadero quién puede acceder a cada animal.	Porcentaje de asignaciones completadas, atenciones registradas y percepción de confianza sobre los permisos.
AniTec aún no tiene la trayectoria ni la base pública de usuarios de sus competidores.	Construir confianza mediante acompañamiento y evidencia de uso.	Realizar demostraciones con ganaderos y veterinarios, ofrecer tutorial inicial y soporte por canales conocidos, y publicar únicamente resultados obtenidos en validaciones reales.	Activación del primer animal, retención semanal, solicitudes de soporte y recomendaciones obtenidas.
AgriWebb e iLivestock cuentan con integraciones avanzadas de hardware y EID.	Priorizar primero los problemas centrales del segmento y evaluar integraciones mediante aprendizaje técnico.	Desarrollar una Spike Story para la capacidad autónoma elegida y probar su utilidad antes de convertirla en una función permanente del producto.	Viabilidad técnica documentada, tasa de éxito en pruebas y beneficio percibido por los usuarios.

Las campañas de referidos, alianzas con asociaciones, participación en ferias y acercamiento a veterinarios podrán utilizarse como tácticas de adquisición cuando exista una versión Android demostrable. Cada actividad deberá registrar participantes, usuarios activados y continuidad de uso; la asistencia a una actividad, por sí sola, no demostrará adopción del producto.

2.2. Entrevistas.

Las entrevistas son una herramienta esencial para comprender a fondo a nuestro público objetivo. Para que sean efectivas, deben seguir una estructura clara y directa, utilizando preguntas específicas que permitan recolectar información de valor y datos precisos de los participantes.

2.2.1. Diseño de entrevistas.

Objetivo: Identificar frustraciones, necesidades, dispositivos disponibles, grado de digitalización y percepción sobre el registro de información ganadera.

2.2.1. Diseño de entrevistas.

Segmentos entrevistados:

- Ganaderos
- Veterinarios

Formato: Entrevistas semiestructuradas, de 25-30 minutos, registradas en video con consentimiento.

Preguntas dirigidas al personal de **Ganaderos**.

Preguntas principales:

- ¿Podría indicarnos su nombre completo y su edad?
- ¿Cuánto tiempo lleva dedicado a la ganadería? ¿Qué tipo de ganado maneja actualmente?
- ¿Cuál es el tamaño aproximado de su ganado? ¿Y cuántas personas trabajan en su unidad ganadera?
- ¿Qué herramientas utiliza actualmente para llevar el control de sus animales y sus actividades?
- ¿Lleva algún registro sobre la salud, alimentación o reproducción de su ganado? ¿Cómo lo hace?
- ¿Cuáles son las principales dificultades que enfrenta en la gestión diaria del ganado?
- ¿Cómo monitorea actualmente la productividad y salud de su ganado?
- ¿Qué tan importante considera llevar un control digital del historial veterinario y productivo de cada animal?
- ¿Ha enfrentado problemas por no tener registros claros (por ejemplo, en ventas, enfermedades o reproducción)?
- ¿Confía en herramientas digitales o ha probado alguna aplicación para el manejo ganadero?
- ¿Cuánto tiempo promedio dedica al registro manual de datos (si lo realiza)?
- ¿Qué tipo de información considera más importante tener a la mano sobre su ganado?
- ¿Estaría dispuesto a usar una aplicación móvil/web para llevar el control del ganado si fuera sencilla y funcional?

- ¿Qué funcionalidades le gustaría que tenga esta herramienta (alertas, historial médico, reproductivo, reportes, etc.)?
- ¿Qué beneficios espera al adoptar una herramienta digital para su ganadería?

Preguntas dirigidas a los **Veterinarios**

Preguntas principales:

- ¿Podría proporcionarnos su nombre completo y su edad?
- ¿Cuánto tiempo lleva ejerciendo como veterinario y en qué región trabaja principalmente?
- ¿Está especializado en atención ganadera? ¿Qué tipo de ganado atiende con más frecuencia?
- ¿Cómo realiza el seguimiento del historial médico de los animales que atiende?
- ¿Utiliza actualmente alguna herramienta digital para llevar registros veterinarios?
- ¿Qué información considera fundamental registrar tras una consulta o intervención (vacunas, tratamientos, diagnóstico)?
- ¿Cómo se comunica con los ganaderos respecto al seguimiento o tratamientos posteriores?
- ¿Con qué frecuencia atiende emergencias ganaderas? ¿Cómo coordina este tipo de intervenciones?
- ¿Ha tenido casos donde la falta de información del animal haya afectado la efectividad del tratamiento?
- ¿Qué retos encuentra en su trabajo relacionado con el registro o gestión de información?
- ¿Le resultaría útil tener acceso al historial médico del animal antes de una consulta?
- ¿Qué tan dispuesto estaría a utilizar una aplicación móvil/web para registrar y acceder al historial de sus pacientes?
- ¿Qué funcionalidades considera clave en una herramienta digital veterinaria (calendario, historial, recordatorios, fichas clínicas)?
- ¿Cómo podría mejorar su trabajo con una solución que conecte a veterinarios con ganaderos en tiempo real?
- ¿Qué tan importante considera el análisis de datos (estadísticas de salud, tratamientos más comunes, etc.) en su labor?

Preguntas complementarias (para ambos segmentos):

- ¿Qué expectativas tendría sobre una plataforma digital que centralice la información ganadera y veterinaria?
- ¿Qué dispositivos usa con más frecuencia para sus actividades laborales (celular, laptop, tablet)? ¿Está familiarizado con el uso de apps?

- ¿Qué es lo que más valora en una herramienta digital: rapidez, facilidad de uso, seguridad de datos u otro aspecto?

Preguntas principales (comunes):

1. ¿Cómo lleva actualmente el registro de su ganado (peso, salud, vacunas)?
2. ¿Qué desafíos ha enfrentado por llevar registros manuales?
3. ¿Qué tan cómodo se siente utilizando un celular o computadora?
4. ¿Le sería útil recibir alertas de vacunación o reproducción?
5. ¿Ha perdido información relevante alguna vez?
6. ¿Qué contenido educativo le interesaría tener en una app?
7. ¿Qué canales digitales usa actualmente (WhatsApp, redes sociales, etc.)?

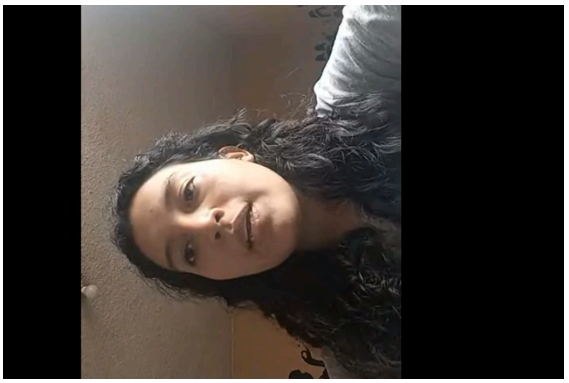
Variables demográficas a recolectar: Edad, género, distrito de residencia, educación, tipo de hacienda, frecuencia de registros, ocupación alterna, herramientas digitales que maneja, tipo de celular, acceso a internet, objetivos personales, frustraciones, marcas preferidas, influencia de técnicos o asociaciones.

2.2.2. Registro de entrevistas.

Entrevistas al segmento de ganaderos

Entrevista 1: Caterina Rodriguez

Campo	Información
Segmento	Ganadero
Nombres y apellidos	Caterina Rodriguez
Edad	33 años
Distrito	San miguel, Lima
Ocupación	Ganadera con más de 9 años de experiencia
Tipo de ganado	Ganado porcino
Tamaño aproximado del ganado	16 cabezas de ganado
Inicio de la entrevista	00:00
Duración	00:07:54
URL del video	https://tinyurl.com/caterinarodri



Resumen de la entrevista:

Caterina Rodríguez es una ganadera de 33 años con aproximadamente ocho años de experiencia en el manejo, principalmente, de ganado vacuno. Actualmente gestiona alrededor de 45 animales junto a un equipo de tres personas (dos familiares y un cuidador). Durante la entrevista, explicó que su gestión de datos se realiza de manera fragmentada: utiliza un cuaderno físico, un cuadro de Excel y su teléfono celular (tomando fotos y notas) para registrar información sobre salud, alimentación y reproducción. Esta forma de trabajo, a la que dedica entre 20 y 30 minutos diarios, le genera dificultades para mantener la información organizada. La entrevistada señaló que los datos suelen traspapelarse o perderse al estar en distintos formatos, o resultar incomprensibles por la mala caligrafía, lo que complica el seguimiento preciso de vacunas y tratamientos.

En cuanto a su perfil tecnológico y forma de trabajo, Caterina se muestra como una persona consciente de la necesidad de optimizar su tiempo y mejorar la toma de decisiones en su ganadería. Valora tener a la mano información vital de cada animal, como edad, peso, historial de enfermedades y tratamientos. Respecto a la tecnología, confía en las herramientas digitales, aunque hasta ahora no ha utilizado ninguna aplicación especializada en el manejo ganadero. Su dispositivo principal es el celular, siendo WhatsApp la aplicación que más utiliza para registrar información y comunicarse, además de usar redes sociales como Facebook y TikTok. El uso de la computadora es mínimo y lo reserva casi exclusivamente para revisar Excel.

La entrevista permitió identificar que Caterina está totalmente dispuesta a utilizar una aplicación móvil o web para el control de su ganado. Destacó que la herramienta ideal debe centralizar la información (usando códigos por animal), contar con un historial médico y reproductivo, y emitir alertas de vacunación y atención. Un punto crucial para ella es que la plataforma sea muy sencilla e intuitiva de usar, ya que sus familiares, que son personas mayores, también deberán interactuar con ella. Además, mostró interés en recibir contenido educativo sobre prevención de enfermedades, alimentación y cuidado de terneros. Esta información sustenta el arquetipo de una ganadera joven que reconoce las limitaciones de los métodos tradicionales y busca una digitalización práctica y accesible que unifique el trabajo de su equipo familiar.

Entrevista 2: Rebeca Noemi Quiroz Roldan

Campo	Información
Segmento	Ganadera
Nombres y apellidos	Rebeca Noemi Quiroz Roldan
Edad	54 años

Campo	Información
Distrito	Lima
Ocupación	Productora ganadera
Tipo de ganado	Ganado vacuno
Tamaño aproximado del ganado	Hato pequeño familiar
Inicio de la entrevista	00:00
Duración	00:06:53
URL del video	https://tinyurl.com/entrevista2RebecaQuiroz

Resumen de la entrevista:

Rebeca Noemi Quiroz Roldan comentó que su actividad ganadera se enfoca en el cuidado de ganado vacuno y que una de sus mayores preocupaciones aparece cuando se presentan enfermedades como mastitis o cuando los terneros se enferman. Actualmente registra la información mediante cuadernos, apuntes y hojas sueltas. Aunque mencionó que logra manejar su actividad con estos medios, también reconoció que una aplicación podría ayudarle a tener mayor orden y a recordar eventos importantes, como vacunaciones, dosis de medicamentos y controles sanitarios.

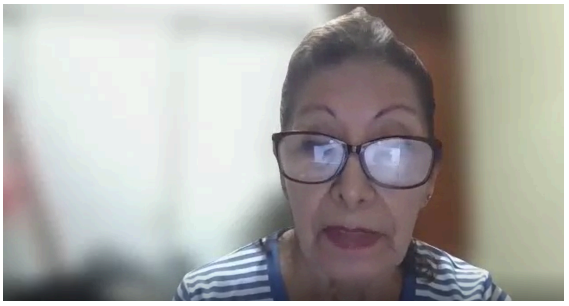
En sus respuestas se observa una personalidad responsable y preventiva. Rebeca valora la seguridad y la confianza antes de adoptar una herramienta digital, por lo que una solución como AniTec debe transmitir protección de datos, facilidad de uso y utilidad concreta. Su principal canal de interacción es el celular, especialmente WhatsApp, ya que lo usa para comunicarse con familiares, trabajadores y profesionales que la asesoran. Además, suele apoyarse en un ingeniero o especialista para atender problemas sanitarios del ganado, lo que muestra que sus decisiones están influenciadas por personal técnico de confianza.

Respecto a tecnología, utiliza smartphone y navegación básica desde el celular. No manifestó rechazo hacia las aplicaciones, pero sí dejó claro que no desea una herramienta complicada. Esta entrevista ayuda a justificar funciones como recordatorios, alertas sanitarias, registro simple de enfermedades y acceso rápido a datos importantes. Sus respuestas aportan evidencia para construir un arquetipo de ganadera que tiene disposición a digitalizarse siempre que la plataforma sea segura, clara y útil para resolver problemas reales del manejo diario.

Entrevista 3: Ursula Trigozo Salcedo

Campo	Información
Segmento	Ganadero

Campo	Información
Nombres y apellidos	Ursula trigozo Salcedo
Edad	64 años
Distrito	Lurin, Lima
Ocupación	Ganadero artesanal
Tipo de ganado	Ganado vacuno
Tamaño aproximado del ganado	Hato pequeño familiar
Inicio de la entrevista	00:00
Duración	00:10:31
URL del video	https://upcedupe-my.sharepoint.com/✌🏻/g/personal/u20231c019_upc_edu_pe/IQDw1Fks32sxQroa9TMJp nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJTdHJlYW1XZWJBcHAiLCJyZWZlcnJhbFZpZXciOiJ



Resumen de la entrevista:

Úrsula Trigozo Salcedo, de 64 años, cuenta con 25 años de experiencia en la ganadería y actualmente maneja aproximadamente 35 cabezas de ganado vacuno junto con tres familiares. Su gestión se basa principalmente en cuadernos, papel y celular, donde registra vacunas, tratamientos, reproducción, partos y observaciones.

Su principal dificultad es mantener la información ordenada y disponible, ya que en ocasiones pierde tiempo buscando registros o estos se deterioran. Considera importante contar con un historial digital por animal y recibir alertas de vacunas, controles veterinarios y reproducción.

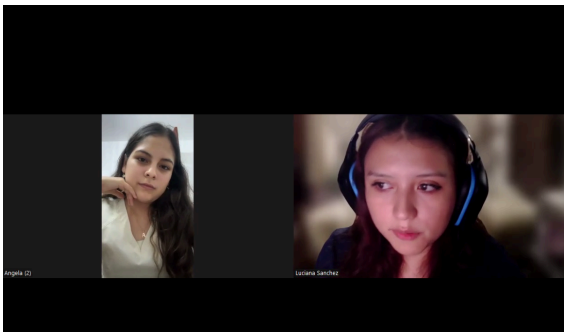
Úrsula está familiarizada con aplicaciones como WhatsApp, Facebook y YouTube, por lo que estaría dispuesta a utilizar AniTec si es simple, rápida, segura y funciona desde el celular. También valora el contenido educativo sobre alimentación, enfermedades y manejo del ganado.

En este perfil, AniTec debe enfocarse en centralizar la información, ahorrar tiempo, evitar la pérdida de datos y facilitar el seguimiento individual de cada animal. Sus respuestas representan el arquetipo de una ganadera experimentada y abierta a la tecnología, siempre que sea sencilla y útil para su trabajo diario.

Entrevistas al segmento de veterinarios

Entrevista 4: Angela Mendoza

Campo	Información
Segmento	Veterinaria
Nombres y apellidos	Angela Mendoza
Edad	24 años
Distrito	Camaná, Arequipa
Ocupación	Médica veterinaria
Zona de trabajo	Sierra sur del Perú
Tipo de atención	Atención sanitaria de ganado
Inicio de la entrevista	00:00
Duración	00:06:55
URL del video	https://tinyurl.com/entrevista4AngelaMendoza



Resumen de la entrevista:

Angela Mendoza explicó que en su trabajo como veterinaria observa que muchos productores todavía gestionan la información sanitaria con cuadernos físicos, archivos simples de Excel, fotografías y conversaciones de WhatsApp. Esta dispersión de datos afecta el seguimiento de vacunas, tratamientos, diagnósticos y antecedentes clínicos. También mencionó que, en visitas de emergencia o campañas sanitarias, la falta de información ordenada puede retrasar la atención y complicar la toma de decisiones.

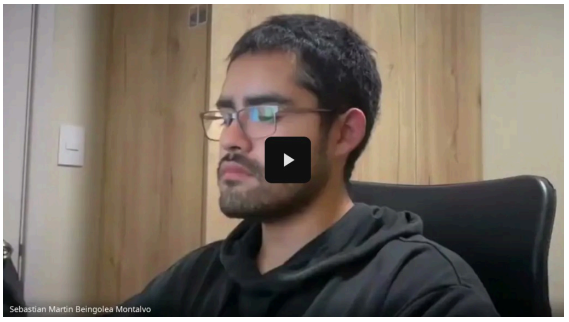
Su perfil evidencia una personalidad organizada, técnica y orientada al servicio. Angela valora la rapidez, la trazabilidad y la claridad de la información porque su trabajo depende de revisar antecedentes antes de indicar tratamientos. Sus canales principales de interacción son WhatsApp, llamadas telefónicas y archivos digitales básicos que comparte con productores. Usa smartphone y laptop para su labor diaria, y se siente cómoda navegando en Google Chrome o usando herramientas web cuando necesita consultar información técnica. Sus influencias profesionales provienen de su formación veterinaria, colegas del sector, campañas sanitarias y necesidades reales observadas en campo.

La entrevistada considera que una herramienta como AniTec sería útil si permite centralizar el historial clínico de cada animal, registrar tratamientos, programar seguimientos y mejorar la comunicación con los ganaderos. Sin embargo,

remarcó que la plataforma debe ser rápida y fácil de usar, porque el trabajo veterinario de campo no permite perder tiempo en procesos largos. Esta entrevista aporta evidencia para el arquetipo de veterinaria joven que usa tecnología, pero necesita una solución enfocada en eficiencia clínica y coordinación con productores.

Entrevista 5: Jeferson Castro Pariona

Campo	Información
Segmento	Veterinario
Nombres y apellidos	Jeferson Castro Pariona
Edad	27 años
Distrito	Ventanilla, Lima (Residencia)
Ocupación	Médico veterinario
Zona de trabajo	Selva central del Perú
Tipo de atención	Atención clínica y sanitaria de ganado en campo
Inicio de la entrevista	00:00
Duración	00:08:09
URL del video	https://tinyurl.com/b3txtyh3



Resumen de la entrevista:

Jeferson Castro Pariona comentó que el reto más grande y frecuente en la atención veterinaria ganadera es la falta de trazabilidad y la pérdida de continuidad en el historial médico de los animales. Explicó que la información suele estar dispersa en cuadernos, notas físicas, hojas de cálculo o chats de WhatsApp, y que cuando se enfrentan emergencias o se incorporan nuevos animales, debe tomar decisiones clínicas con datos limitados. Esto afecta directamente la planificación sanitaria, la efectividad de los tratamientos y el seguimiento de las intervenciones.

El entrevistado muestra un perfil analítico, práctico y orientado a la tecnología como herramienta de trabajo diario. Utiliza su smartphone para la comunicación y el registro rápido en campo, y su laptop para consultas técnicas más detalladas. Considera fundamental contar con datos concretos antes de intervenir y valora la rapidez, la facilidad de uso y la intuición en cualquier herramienta digital, especialmente considerando los retos de conectividad en el entorno rural.

Según sus respuestas, una plataforma digital (como AniTec) le aportaría un valor significativo si logra centralizar la información sin ser un sistema pesado. Sus funcionalidades clave ideales incluyen: acceso inmediato al historial sanitario del animal, un módulo rápido para registrar diagnósticos y tratamientos, alertas de vacunación o

reproducción para programar rutas de campo, y un canal claro de comunicación con el ganadero. También valoró la inclusión de contenido educativo rápido, como fichas técnicas o información sobre brotes en la región. En conclusión, está totalmente dispuesto a digitalizar su labor para reducir los errores derivados de la información dispersa, siempre y cuando la herramienta se integre de forma ágil a su flujo real de trabajo en el campo.

2.2.3. Análisis de entrevistas.

Análisis del segmento de Ganaderos

Para el segmento de ganaderos se analizaron 3 entrevistas: Vicente Huamán Alacutte, Rebeca Noemi Quiroz Roldan y Porfirio Salazar Rodriguez. La muestra evidencia un perfil de productores con experiencia práctica, uso frecuente de métodos tradicionales y apertura moderada hacia herramientas digitales siempre que sean simples, útiles y accesibles.

Característica identificada	**Resultado estadístico**	**Sustento en entrevistas**	**Relación con el arquetipo**
Edad adulta y experiencia en ganadería	3 de 3 entrevistados (100%) tienen entre 54 y 65 años y experiencia directa en actividades ganaderas.	Vicente tiene 62 años y más de 30 años de experiencia; Rebeca tiene 54 años y gestiona ganado vacuno; Porfirio tiene 65 años y trabaja en ganadería artesanal.	El arquetipo debe representar a un usuario con conocimiento práctico del campo, pero no necesariamente familiarizado con sistemas digitales complejos.
Uso de registros manuales	3 de 3 entrevistados (100%) usan cuadernos, apuntes, hojas o registros básicos para controlar información del ganado.	Vicente usa cuaderno físico y Excel; Rebeca usa cuadernos y hojas sueltas; Porfirio trabaja con una gestión artesanal y poco digitalizada.	AniTec debe priorizar registro rápido, ordenado y fácil de consultar para reemplazar gradualmente el control manual.
Necesidad de controlar salud y tratamientos	2 de 3 entrevistados (66.7%) mencionan directamente problemas relacionados con enfermedades, tratamientos o control sanitario.	Vicente menciona control de enfermedades y vacunación; Rebeca menciona mastitis, terneros enfermos y dosis de medicamentos.	El arquetipo necesita alertas sanitarias, historial médico y recordatorios de vacunación o tratamiento.
Uso del celular como canal principal	3 de 3 entrevistados (100%) muestran familiaridad básica con el celular como medio de comunicación o consulta.	Vicente usa celular y WhatsApp; Rebeca usa principalmente su celular; Porfirio utiliza celular y comunicación por WhatsApp.	La solución debe funcionar bien en pantallas pequeñas y permitir una interacción directa desde dispositivos móviles.
Interés en adoptar	3 de 3 entrevistados (100%) reconocen que la	Vicente afirma que la tecnología puede profesionalizar el sector;	El arquetipo no rechaza la tecnología, pero necesita

Característica identificada	Resultado estadístico	Sustento en entrevistas	Relación con el arquetipo
tecnología	tecnología podría ayudar a mejorar su gestión.	Rebeca estaría dispuesta a usar una aplicación; Porfirio considera que la tecnología podría mejorar su productividad.	percibir un beneficio claro antes de usarla.
Necesidad de simplicidad	3 de 3 entrevistados (100%) requieren que la herramienta sea sencilla, clara o fácil de justificar en el trabajo diario.	Vicente pide una app sencilla para el hombre de campo; Rebeca no desea una herramienta complicada; Porfirio evalúa si el beneficio justifica la inversión.	La interfaz debe usar lenguaje simple, formularios directos y pocos pasos para completar tareas.
Influencia de terceros en decisiones	3 de 3 entrevistados (100%) dependen o se ven influenciados por otras personas del entorno ganadero.	Vicente considera compradores y técnicos; Rebeca se apoya en un especialista; Porfirio aprende de experiencia familiar y otros productores.	El arquetipo toma decisiones con apoyo de redes de confianza, por lo que la app debe transmitir seguridad y utilidad comprobable.
Sensibilidad al costo	1 de 3 entrevistados (33.3%) menciona directamente el costo como barrera.	Porfirio indica que necesitaría capital suficiente para pagar una solución digital.	El modelo de adopción debe considerar planes accesibles, prueba gratuita o beneficios económicos visibles.

En conjunto, el análisis muestra que el ganadero objetivo de AniTec es un usuario con experiencia práctica y responsabilidad directa sobre el cuidado del ganado, pero con procesos todavía manuales. La característica más fuerte del segmento es la necesidad de orden y recordatorios, ya que el 100% de entrevistados depende de registros tradicionales y el 66.7% menciona problemas sanitarios específicos. También se observa una oportunidad clara para una solución móvil, debido a que el 100% usa el celular como canal principal o familiar.

Desde el punto de vista subjetivo, los ganaderos entrevistados valoran la confianza, la utilidad concreta, la seguridad de la información y la facilidad de uso. Estas características provienen directamente de los resúmenes: Vicente prioriza una herramienta simple para el campo, Rebeca valora la seguridad y Porfirio evalúa el beneficio frente al costo. Por ello, el arquetipo de ganadero debe construirse como una persona práctica, cuidadosa, sensible al esfuerzo de aprendizaje y dispuesta a digitalizarse si la solución demuestra valor real.

Análisis del segmento de Veterinarios

Para el segmento de veterinarios se analizaron 2 entrevistas: Angela Mendoza y Jeferson Castro Pariona. La muestra evidencia un perfil profesional joven, con experiencia en atención de campo, necesidad de información clínica ordenada y uso de herramientas digitales básicas para comunicarse con productores.

Característica identificada	Resultado estadístico	Sustento en entrevistas	Relación con el arquetipo
Profesionales jóvenes	2 de 2 entrevistados (100%) tienen entre 24 y 27 años.	Angela tiene 24 años; Jeferson tiene 27 años.	El arquetipo puede representarse como un veterinario joven, con mayor disposición a usar herramientas digitales.
Trabajo en campo o zonas descentralizadas	2 de 2 entrevistados (100%) trabajan o han trabajado en zonas fuera del entorno urbano principal.	Angela trabaja en la sierra sur; Jeferson tiene experiencia en la selva central.	La solución debe considerar rapidez, movilidad y uso en contextos con posible conectividad limitada.
Problemas por registros incompletos o dispersos	2 de 2 entrevistados (100%) mencionan información sanitaria desordenada o incompleta.	Angela menciona cuadernos, Excel, fotos y WhatsApp; Jeferson menciona cuadernos, notas físicas, hojas de cálculo y chats de WhatsApp.	El arquetipo necesita acceso centralizado al historial clínico y sanitario de los animales.
Uso de WhatsApp y herramientas digitales básicas	2 de 2 entrevistados (100%) usan canales digitales simples para comunicarse o complementar su trabajo.	Angela usa WhatsApp, llamadas y archivos digitales; Jeferson usa smartphone en campo y laptop para consultas técnicas.	AniTec debe integrarse al flujo real del veterinario y reducir la dependencia de información dispersa.
Necesidad de historial clínico previo	2 de 2 entrevistados (100%) señalan que conocer antecedentes mejora la atención.	Angela necesita revisar antecedentes antes de tratamientos; Jeferson indica que la falta de trazabilidad obliga a decidir con datos limitados.	El arquetipo requiere fichas clínicas, historial sanitario y acceso rápido a tratamientos previos.
Valoración de rapidez y facilidad de uso	2 de 2 entrevistados (100%) indican que la herramienta debe ser rápida, intuitiva y adaptada al campo.	Angela remarca que no puede perder tiempo en procesos largos; Jeferson pide rapidez, facilidad de uso e integración ágil al trabajo rural.	La interfaz veterinaria debe permitir registrar atenciones con pocos pasos y consultar datos al instante.
Necesidad de mejorar comunicación con ganaderos	2 de 2 entrevistados (100%) consideran importante coordinar mejor con productores.	Angela menciona mejorar comunicación con ganaderos; Jeferson valora un canal claro de comunicación con el productor.	El arquetipo necesita funciones de seguimiento, coordinación y comunicación clara con el productor.
Actitud favorable hacia herramientas tecnológicas	2 de 2 entrevistados (100%) muestran apertura a soluciones	Angela considera útil centralizar historial y seguimiento; Jeferson está dispuesto a digitalizar su labor	El arquetipo es un usuario con mayor predisposición digital que el ganadero,

Característica identificada	Resultado estadístico	Sustento en entrevistas	Relación con el arquetipo
	digitales para mejorar su trabajo.	si la herramienta no es pesada y aporta alertas, registro rápido y contenido educativo.	pero exige eficiencia profesional.

El análisis del segmento veterinario muestra que las principales necesidades están relacionadas con trazabilidad clínica, acceso rápido a información y coordinación con productores. El 100% de los entrevistados menciona problemas por información dispersa y el 100% considera importante contar con antecedentes del animal antes o durante la atención. Esto evidencia que el arquetipo veterinario debe estar construido alrededor de la toma de decisiones clínicas, el seguimiento sanitario y la necesidad de reducir incertidumbre durante el trabajo de campo.

En cuanto a características subjetivas, ambos veterinarios se muestran más familiarizados con la tecnología que el segmento ganadero. Angela representa un perfil organizado, técnico y orientado al servicio, mientras que Jeferson muestra un perfil analítico, práctico y orientado a la tecnología. Ambos valoran la rapidez, la trazabilidad y la utilidad real. Por ello, el arquetipo de veterinario debe representar a un profesional joven, móvil, acostumbrado a usar herramientas digitales básicas, pero que necesita una plataforma más integrada para evitar pérdida de información y mejorar la continuidad de tratamientos.

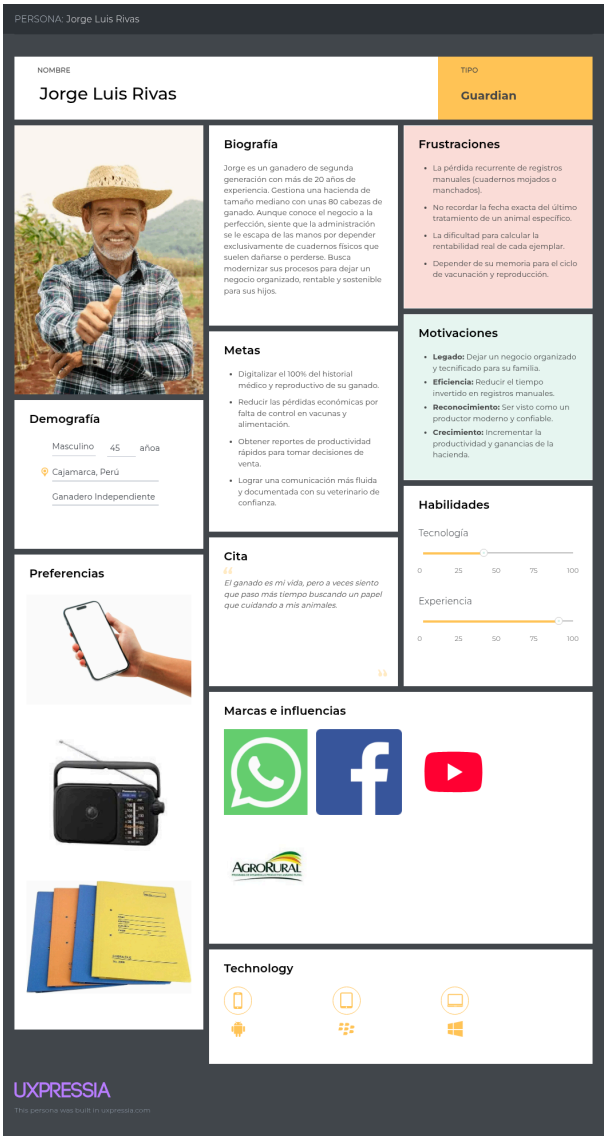
2.3. Needfinding.

En esta sección se presentarán los artefactos resultantes del proceso de análisis de la información recolectada de los segmentos objetivos. Aquí se incluyen secciones para User Personas, User Task Matrix, User Journey Maps, Empathy Mapping y As-is Scenario Mapping.

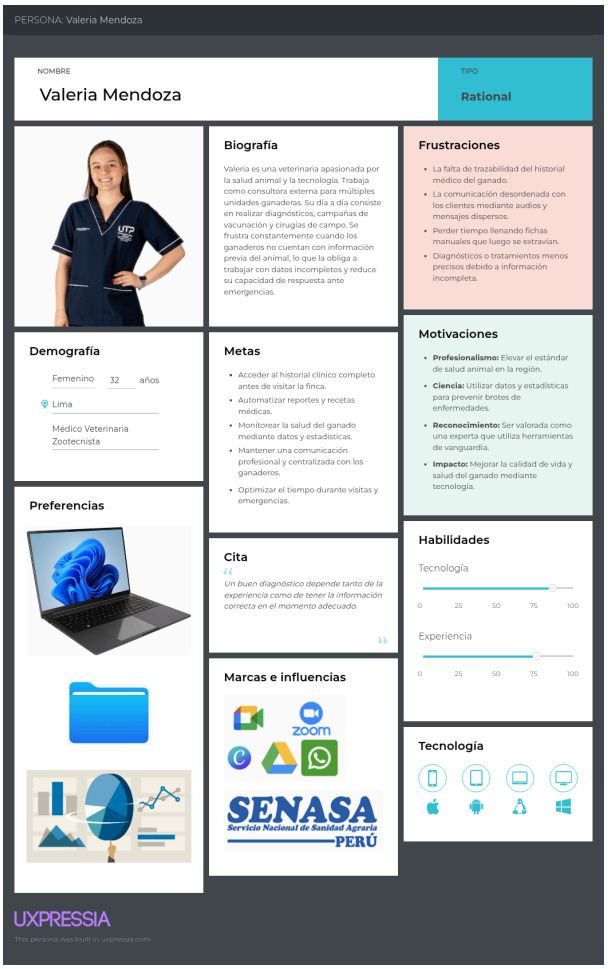
2.3.1. User Personas.

A continuación, se presentan los User Personas diseñados para representar a los segmentos objetivo identificados durante la fase de investigación. Estos arquetipos detallan variables demográficas, rasgos psicográficos, motivaciones y comportamientos, así como los pains (frustraciones) y gains (objetivos) que enfrentan en su gestión diaria. Asimismo, se analiza su nivel digital y su interacción con soluciones tecnológicas del sector agropecuario. Toda la información ha sido sintetizada a partir de los insights recolectados en las entrevistas y estructurada mediante la plataforma UXPressia para garantizar una representación fiel de las necesidades del usuario.

User Persona: Ganaderos



User Persona: Veterinarios



2.3.2. User Task Matrix.

A través de la User Task Matrix, es posible identificar y organizar las principales actividades que los usuarios realizan actualmente dentro de su contexto de trabajo. Al categorizar estas tareas según su frecuencia e importancia, se logra comprender cuáles representan mayores dificultades y necesidades para cada perfil de usuario, permitiendo detectar oportunidades de mejora en la gestión ganadera y veterinaria.

User Task	**Jorge Luis Rivas (Frecuencia)**	**Jorge Luis Rivas (Importancia)**	**Valeria Mendoza (Frecuencia)**	**Valeria Mendoza (Importancia)**
Anotar el nacimiento o compra de un nuevo animal en cuadernos físicos	Sometimes	High	Rarely	Medium
Registrar manualmente vacunas y tratamientos del ganado	Often	High	Always	High
Revisar fechas de vacunación en notas, calendarios o cuadernos	Often	High	Often	High

User Task	Jorge Luis Rivas (Frecuencia)	Jorge Luis Rivas (Importancia)	Valeria Mendoza (Frecuencia)	Valeria Mendoza (Importancia)
Recordar manualmente vacunas o controles pendientes	Sometimes	High	Sometimes	High
Anotar peso y crecimiento del ganado durante controles	Often	Medium	Often	Medium
Revisar manualmente información sobre productividad y rendimiento	Sometimes	Medium	Often	Medium
Compartir documentos físicos o fotografías de registros con asociaciones o compradores	Rarely	Medium	Rarely	Low
Buscar información o capacitaciones ganaderas en internet y redes sociales	Sometimes	Low	Sometimes	Medium
Llevar el control reproductivo mediante anotaciones manuales	Rarely	Medium	Rarely	Medium
Buscar antecedentes médicos y sanitarios en cuadernos o archivos físicos	Sometimes	High	Sometimes	High

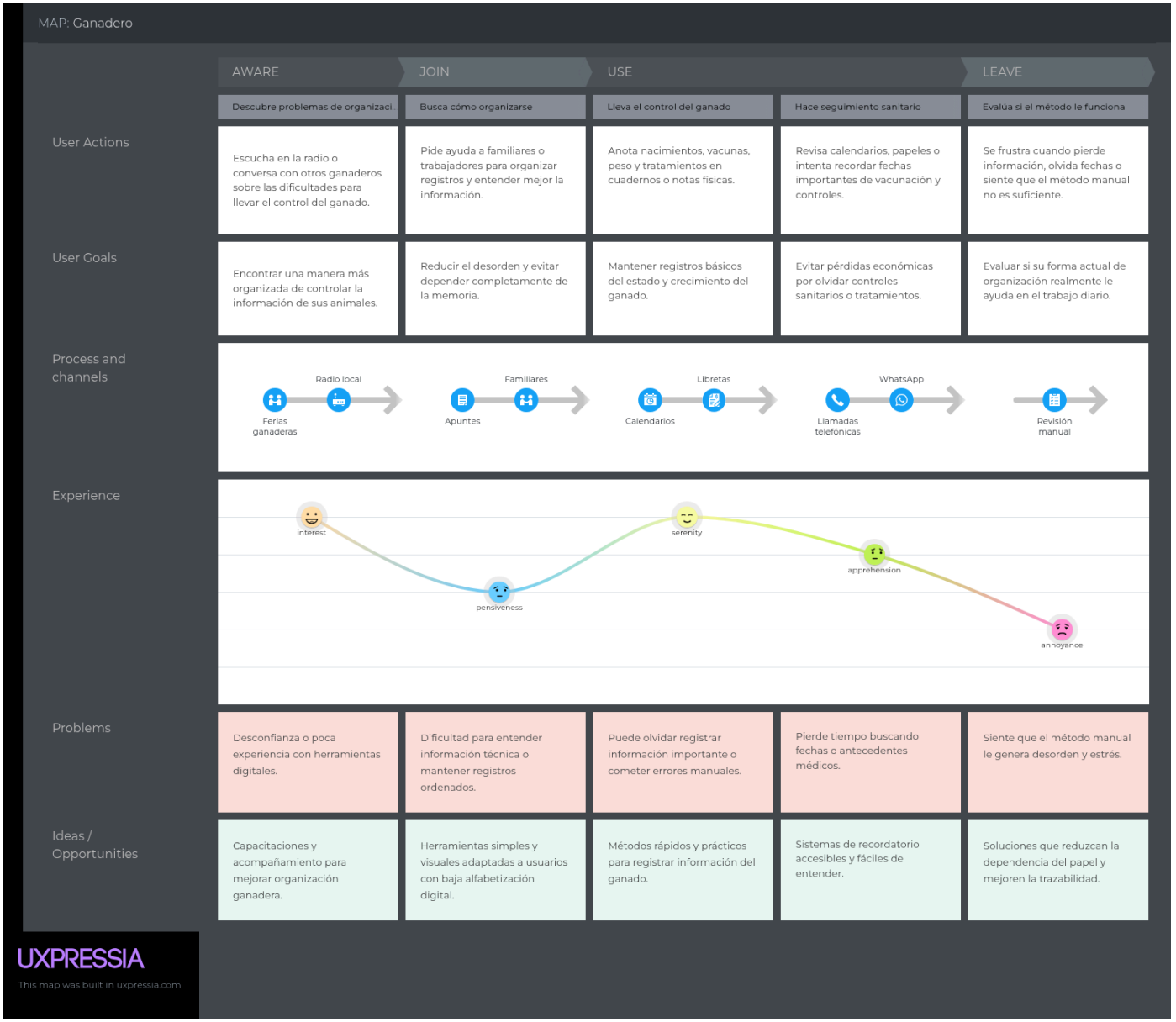
La User Task Matrix evidencia que tanto Jorge Luis como Valeria realizan constantemente actividades relacionadas con el control sanitario y el seguimiento del ganado. Mientras Jorge Luis depende principalmente de registros físicos y de su memoria para organizar la información de sus animales, Valeria necesita acceder rápidamente a datos precisos durante sus visitas de campo y procedimientos veterinarios. Asimismo, ambos perfiles presentan dificultades relacionadas con la organización, trazabilidad y acceso oportuno a la información, especialmente en procesos de vacunación, historial clínico y monitoreo del ganado. Estas tareas permiten comprender mejor el contexto actual de los usuarios e identificar necesidades reales dentro del entorno ganadero y veterinario.

2.3.3. User Journey Mapping.

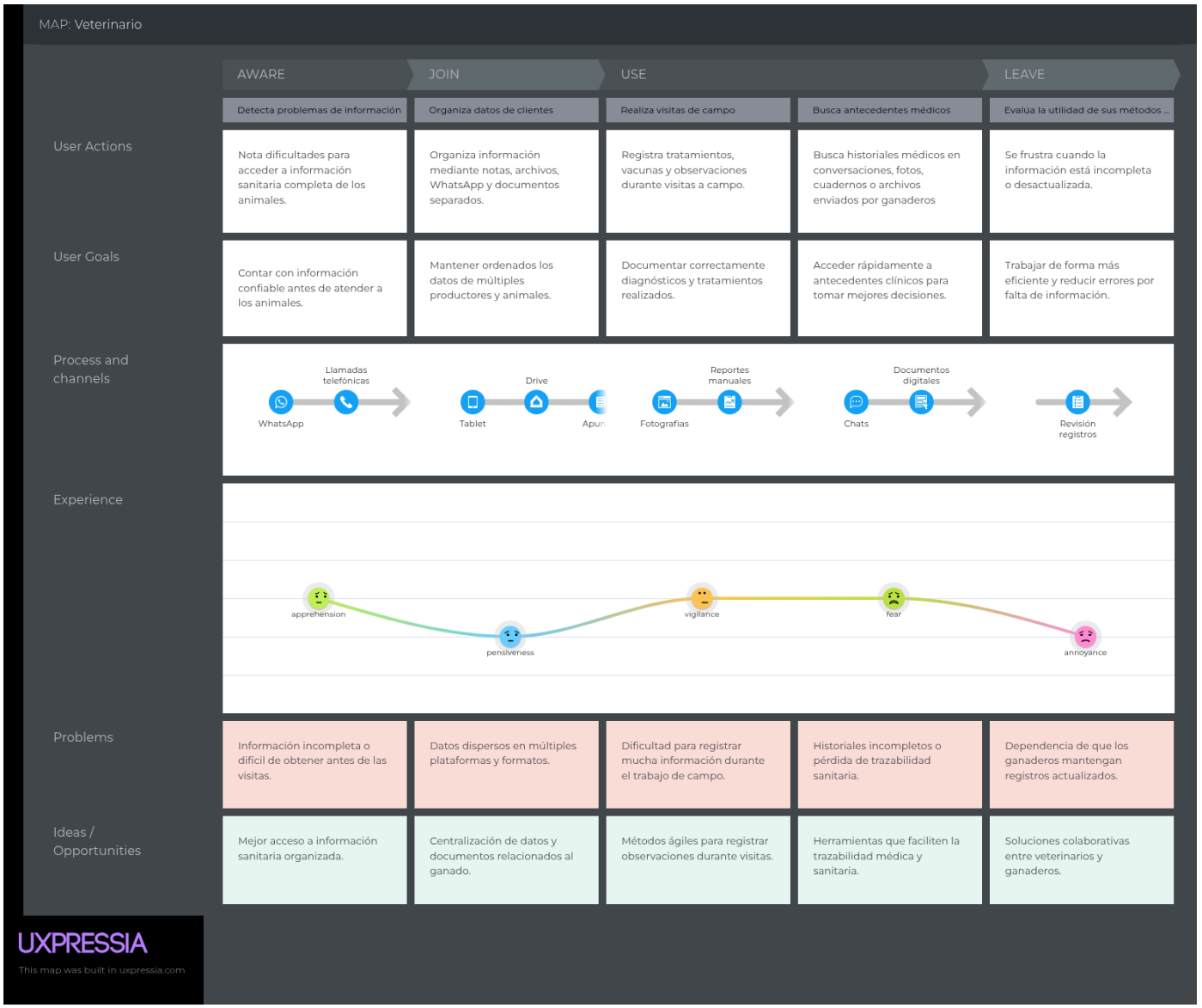
En este apartado se describe de manera detallada el recorrido y la experiencia actual de los usuarios dentro de su contexto de trabajo cotidiano, enfocándose específicamente en los dos perfiles clave identificados: productores ganaderos y médicos veterinarios. A través de los User Journey Maps As-Is, se analizan las actividades, emociones, problemas y necesidades que experimentan los usuarios durante la gestión y seguimiento del ganado, sin considerar aún la existencia de una solución tecnológica implementada.

El mapeo de este recorrido inicia desde las primeras dificultades relacionadas con la organización y acceso a la información, avanzando a través de las tareas diarias de registro, seguimiento sanitario y búsqueda de antecedentes médicos. Asimismo, se identifican los principales puntos de contacto, canales utilizados y frustraciones presentes en el proceso actual, permitiendo comprender de forma integral cómo los usuarios realizan actualmente sus actividades y cuáles son las oportunidades de mejora dentro del entorno ganadero y veterinario.

User Ganadero:



User Veterinario:



2.3.4. Empathy Mapping.

En esta sección se presenta el proceso de elaboración de los Empathy Maps correspondientes a los User Personas identificados para el proyecto: el productor ganadero y la médica veterinaria. Para el desarrollo de estos mapas de empatía, el equipo analizó la información obtenida durante la etapa de investigación y needfinding, considerando las necesidades, comportamientos, preocupaciones y motivaciones de cada perfil dentro de su contexto laboral cotidiano.

El proceso de elaboración inició colocando al User Persona en el centro del análisis, permitiendo identificar de manera estructurada qué piensa, siente, observa, escucha, dice y hace cada usuario en relación con la gestión sanitaria y administrativa del ganado. Asimismo, se identificaron los principales esfuerzos (Pains) y ganancias (Gains) presentes en sus actividades diarias, con el objetivo de comprender mejor sus frustraciones, expectativas y oportunidades de mejora dentro del entorno ganadero y veterinario.

Los Empathy Maps permitieron visualizar de forma más profunda la experiencia actual de los usuarios, evidenciando problemáticas relacionadas con la organización de información, trazabilidad sanitaria, dependencia de registros físicos y dificultades en la comunicación y acceso a datos confiables.

User Ganadero:

PERSONA: Veterinario

1. ¿Con quién estamos empatizando?

- **Quién:** César, 62 años, ganadero de Cajamarca.
- **Situación:** Lleva más de 25 años registrando información en cuadernos y suele olvidar fechas importantes.
- **Rol:** Dueño y gestor de su ganado; responsable de la salud, alimentación y ventas.

2. ¿Qué necesita hacer?

- Mantener organizados los registros de raza, edad, peso y eventos sanitarios.
- Tomar decisiones sobre vacunación, reproducción y ventas.
- Tener acceso rápido a información importante del ganado.
- **Éxito:** Reducir pérdidas y mantener un mejor control sanitario de sus animales.

3. ¿Qué está viendo?

- Cuadernos desordenados y registros difíciles de encontrar.
- Otros ganaderos enfrentando problemas similares con papeles y controles manuales.
- Herramientas digitales complejas o poco adaptadas al trabajo de campo.

4. ¿Qué está diciendo?

- *"Yo sé todo de mi ganado, pero a veces me falla la memoria."*
- *"Necesito algo sencillo y práctico para organizarme mejor."*

5. ¿Qué está haciendo?

- Anota información manualmente en cuadernos o calendarios.
- Toma fotos de registros y las comparte por WhatsApp.
- Consulta fechas importantes revisando papeles y notas antiguas.

ESFUERZOS

- Pérdida o deterioro de cuadernos físicos.
- Dificultad para encontrar información rápidamente.
- Dependencia de la memoria para recordar controles y tratamientos.
- Señal limitada o poca experiencia con herramientas digitales.

GANANCIAS

- Acceder fácilmente a información organizada del ganado.
- Reducir olvidos relacionados con vacunas y tratamientos.
- Tener mayor confianza y control sobre la salud de sus animales.
- Ahorrar tiempo en la búsqueda de registros.

7. ¿Cómo se siente y qué piensa?

- ***Piensa:*** *"¿Olvidaré otra vacuna?" "¿Dónde anoté ese tratamiento?"*
- ***Siente:*** *preocupación por perder información importante y desconfianza hacia la tecnología.*

6. ¿Qué está escuchando?

- Recomendaciones de familiares y veterinarios sobre nuevas formas de organización.
- Conversaciones sobre problemas relacionados con pérdida de registros y olvidos.
- Comentarios sobre herramientas digitales difíciles de usar.

UXPRESSIA

This persona was built in uxpressia.com

User Veterinario:

PERSONA: Veterinario

1. ¿Con quién estamos empatizando?

- **Quién:** Carla, 28 años, veterinaria de campo en Apurímac.
- **Situación:** Atiende múltiples fincas semanalmente utilizando notas físicas y herramientas digitales separadas.
- **Rol:** Responsable de diagnósticos, tratamientos y asesoría sanitaria para productores ganaderos.

2. ¿Qué necesita hacer?

- Documentar diagnósticos, vacunas y tratamientos realizados durante visitas.
- Acceder rápidamente a antecedentes clínicos confiables.
- Organizar información de diferentes productores y animales.
- **Éxito:** Mejorar el seguimiento sanitario y reducir errores por información incompleta.

3. ¿Qué está viendo?

- Información dispersa en cuadernos, fotos y chats de WhatsApp.
- Productores con registros incompletos o desactualizados.
- Colegas utilizando múltiples herramientas sin integración.

4. ¿Qué está diciendo?

- *"Si los productores registraran mejor la información, evitaríamos muchos problemas."*
- *"Necesito toda la información organizada y accesible rápidamente."*

5. ¿Qué está haciendo?

- Registra observaciones y tratamientos durante visitas de campo.
- Toma fotografías de registros y comparte información por WhatsApp.
- Utiliza documentos digitales, notas y archivos para organizar información.

ESFUERZOS

- Información sanitaria incompleta o desactualizada.
- Datos dispersos en diferentes plataformas y formatos.
- Dificultad para registrar información durante visitas de campo.
- Dependencia de que los productores mantengan registros organizados.

GANANCIAS

- Acceder rápidamente a historiales clínicos organizados.
- Reducir tiempo buscando información de animales.
- Mejorar el seguimiento sanitario y la toma de decisiones.
- Trabajar de manera más eficiente y profesional.

7. ¿Cómo se siente y qué piensa?

- ***Piensa:*** *"¿Cómo puedo tomar mejores decisiones si la información está incompleta?"*
- ***Siente:*** *frustración por la falta de organización y preocupación por cometer errores clínicos.*

6. ¿Qué está escuchando?

- Quejas de productores sobre pérdida de información o dificultad para organizar registros.
- Comentarios de colegas sobre problemas de trazabilidad sanitaria.
- Recomendaciones de herramientas digitales poco adaptadas al trabajo rural.

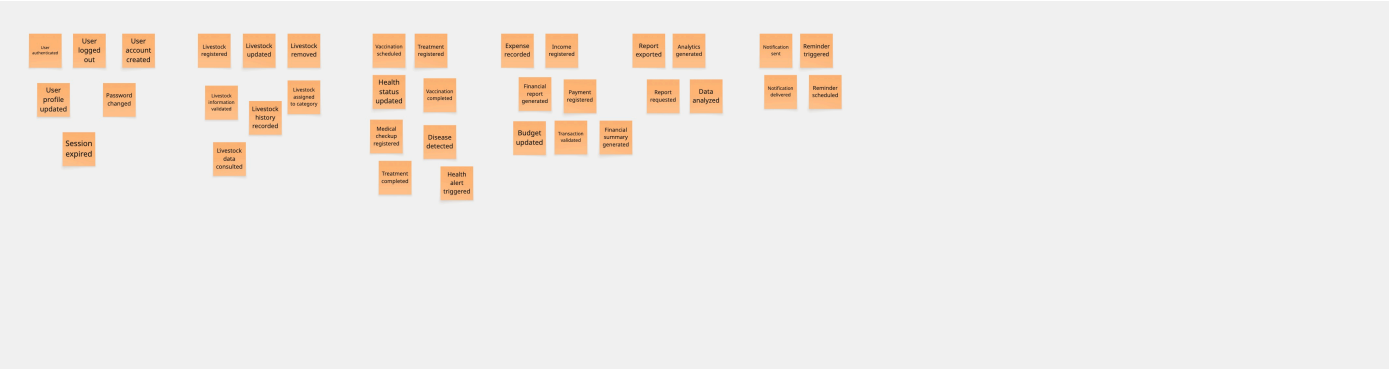
UXPRESSIA

This persona was built in uxpressia.com

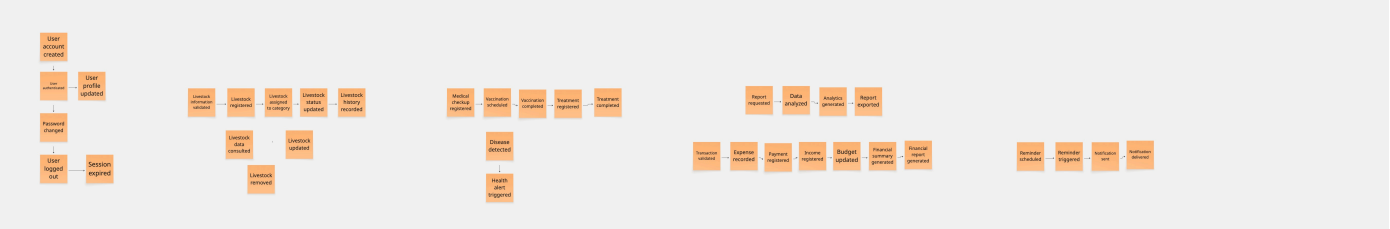
2.3.5. Big Picture EventStorming.

El presente Big Picture Event Storming se ha desarrollado de manera colaborativa utilizando la plataforma Miro, siguiendo la metodología de Philippe Bourgau para explorar el dominio del negocio de forma holística y establecer un entendimiento compartido. A través de un proceso iterativo en este entorno digital, que incluyó la generación de eventos de dominio, el ordenamiento cronológico y la identificación de puntos críticos dentro de los procesos, se ha logrado mapear la complejidad del sector ganadero en una narrativa visual coherente. Este artefacto no solo permitió identificar riesgos y oportunidades de mejora en la gestión de AniTec, sino que también sentó las bases para la comprensión del dominio y el diseño posterior de la arquitectura del sistema alineada con la realidad operativa de los ganaderos y veterinarios.

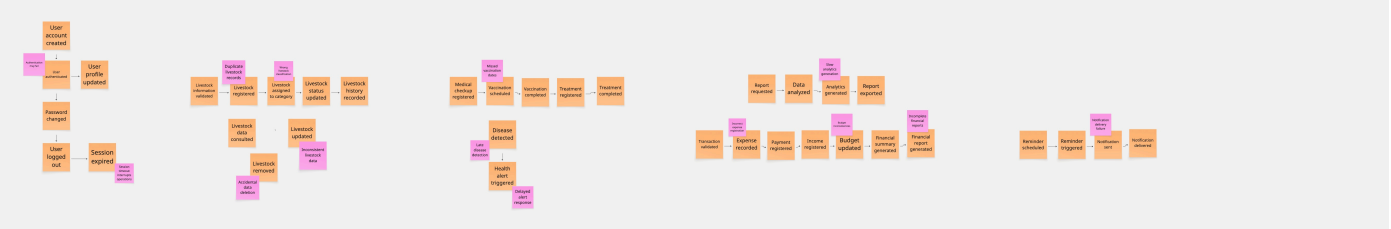
Paso 1: Unstructured Exploration (Exploración no estructurada) consiste en una lluvia de ideas colaborativa donde los participantes identifican y registran domain events, que son sucesos relevantes ocurridos dentro del negocio. Estos eventos deben redactarse obligatoriamente en tiempo pasado (por ejemplo, "Livestock registered") y se colocan en notas adhesivas de color naranja sobre la superficie de modelado. En esta etapa inicial, se prioriza el descubrimiento de conceptos y procesos importantes del dominio sobre el orden o la jerarquía, permitiendo identificar eventos relacionados con la autenticación de usuarios, gestión de ganado, procesos sanitarios, operaciones financieras, generación de reportes y envío de notificaciones.



Paso 2: Timelines, los participantes revisan los eventos de dominio generados y los organizan cronológicamente para reflejar la secuencia real de los procesos empresariales. La construcción inicia organizando el flujo principal del negocio y posteriormente incorporando relaciones, ramificaciones y dependencias entre eventos. Este paso permitió construir una narrativa visual coherente del comportamiento del sistema AniTec, facilitando la identificación de secuencias operativas, conexiones entre procesos y posibles inconsistencias dentro del dominio.



Paso 3: Pain Points, los participantes utilizan la línea de tiempo recién organizada para identificar los puntos críticos, riesgos o ineficiencias presentes dentro de los procesos modelados. Estos problemas se representan mediante notas adhesivas rosadas colocadas cerca de los eventos relacionados, permitiendo visualizar dificultades operativas y oportunidades de mejora dentro del sistema. Entre los principales pain points identificados destacan posibles fallos de autenticación, registros duplicados de ganado, detección tardía de enfermedades, inconsistencias presupuestarias y fallos en la entrega de notificaciones. La identificación explícita de estas debilidades permitió comprender mejor los desafíos del dominio y considerar posibles automatizaciones o mejoras futuras en el diseño del sistema.



Enlace para acceder al miro: https://tinyurl.com/EventSorming

2.3.6. Ubiquitous Language.

Siguiendo los conceptos de **Ubiquitous Language** definidos por **Eric Evans (2003)** en su obra *Domain-Driven Design: Tackling Complexity in the Heart of Software*, se presenta el siguiente glosario. Este conjunto de términos constituye el lenguaje común del proyecto, eliminando ambigüedades entre el equipo de ingeniería y los expertos del dominio ganadero.

Term (English)	Término (Español)	Definition (Definición)
Livestock Owner	Ganadero	Usuario responsable de la administración operativa y financiera de la hacienda, encargado de registrar eventos diarios y tomar decisiones de producción.
Veterinarian	Veterinario	Profesional especializado encargado de la salud animal, responsable de emitir diagnósticos, prescribir tratamientos y validar historiales clínicos.
Herd	Hato	Conjunto de animales agrupados dentro de una unidad productiva o finca para su control sanitario, productivo y administrativo.
Farm	Finca	Unidad productiva donde el ganadero organiza animales, actividades, registros sanitarios y movimientos financieros.
Animal	Animal	Ejemplar individual registrado por el ganadero para mantener su trazabilidad sanitaria, productiva y económica.
Health Event	Evento Sanitario	Registro de una vacuna, enfermedad, diagnóstico, tratamiento, incidencia o seguimiento relacionado con la salud de un animal.
Veterinary Client	Cliente Veterinario	Ganadero asignado a un veterinario para que este pueda revisar sus animales, consultar historiales y registrar atenciones sanitarias.
Veterinary History	Historial Veterinario	Expediente clínico consolidado que permite al especialista revisar antecedentes médicos antes de realizar una intervención.
Treatment Protocol	Protocolo de Tratamiento	Conjunto de instrucciones médicas y fármacos asignados a un animal para tratar una afección diagnosticada por el veterinario.
Traceability	Trazabilidad	Capacidad de reconstruir el historial completo de un animal, incluyendo origen, salud, peso, actividades y cambios relevantes.

Term (English)	Término (Español)	Definition (Definición)
IoT Device	Dispositivo IoT	Equipo asociado a una finca o animal para apoyar el monitoreo de datos como peso, temperatura, humedad, ubicación o actividad.
Metric	Métrica	Lectura generada por un dispositivo o registro del sistema que permite analizar el estado de un animal, finca o proceso ganadero.
Farm Activity	Actividad Ganadera	Tarea productiva, sanitaria, financiera o veterinaria programada para organizar el trabajo dentro de la operación ganadera.
Financial Movement	Movimiento Financiero	Ingreso o egreso registrado por el ganadero para controlar la rentabilidad de su operación.
Subscription Plan	Plan de Suscripción	Opción de uso de la plataforma que define condiciones, beneficios y precio para el usuario.
Payment	Pago	Registro económico asociado a la activación o renovación de un plan de suscripción dentro de AniTec.
Farm Management	Gestión de Hacienda	Administración de los recursos, personal, animales, actividades y datos que ocurren dentro de la unidad productiva ganadera.
Mobile Application	Aplicación Móvil	Software instalado en un smartphone o tablet que permite al ganadero o veterinario gestionar información del ganado desde el campo.
Native App	Aplicación Nativa	Aplicación desarrollada específicamente para un sistema operativo móvil, como Android con Kotlin, aprovechando capacidades del dispositivo.
Cross-Platform App	Aplicación Multiplataforma	Aplicación móvil que se ejecuta en más de un sistema operativo a partir de una base de código compartida, manteniendo una experiencia coherente.
Push Notification	Notificación Push	Mensaje enviado al dispositivo del usuario para alertar sobre vacunaciones, tratamientos, recordatorios o eventos sanitarios pendientes.
Offline Mode	Modo Sin Conexión	Capacidad de la aplicación móvil para registrar o consultar información esencial aunque no exista conexión a internet estable.
Local Persistence	Persistencia Local	Almacenamiento de datos en el dispositivo móvil para conservar borradores, historiales básicos o registros pendientes de sincronización.
Synchronization	Sincronización	Proceso mediante el cual los datos guardados en el dispositivo se actualizan con el servidor cuando se recupera la conectividad.
Camera Capture	Captura con Cámara	Uso de la cámara del dispositivo móvil para identificar animales, registrar evidencias o apoyar la búsqueda visual dentro de la aplicación.

Term (English)	Término (Español)	Definition (Definición)
Device Capability	Capacidad del Dispositivo	Función del hardware o sistema móvil utilizada por la app, como cámara, almacenamiento local, notificaciones o sensores.
Mobile Session	Sesión Móvil	Estado de autenticación del usuario dentro de la aplicación móvil que permite acceder a rutas protegidas y consumir la API de AniTec.
Mobile Dashboard	Panel Móvil	Vista resumida en la aplicación móvil que muestra indicadores, alertas y accesos rápidos para facilitar decisiones en campo.
Responsive Navigation	Navegación Móvil	Estructura de pantallas y menús adaptada a pantallas pequeñas para que el usuario complete tareas con pocos pasos desde el teléfono.
API Consumption	Consumo de API	Comunicación de la aplicación móvil con el backend mediante servicios REST para enviar o recibir datos del dominio ganadero.
Draft Record	Borrador	Registro temporal guardado en el dispositivo cuando el usuario aún no puede completar o sincronizar una operación por falta de red o tiempo.
Accessibility	Accesibilidad	Conjunto de prácticas de diseño móvil que facilitan el uso de la aplicación a personas con distintos niveles de experiencia digital o limitaciones.
Internationalization	Internacionalización	Capacidad de la aplicación móvil para adaptar idioma, formatos y textos a diferentes contextos regionales sin cambiar la lógica del producto.

2.4. Requirements specification

Esta sección especifica los requisitos de la solución móvil de AniTec a partir de las entrevistas, los User Personas, el Needfinding y los supuestos Lean UX. El alcance comprende la landing page estática, la API REST propia, una aplicación Android nativa desarrollada con Kotlin y Jetpack Compose, y una aplicación multiplataforma desarrollada con Flutter y Dart. Los requisitos también incluyen almacenamiento local, uso de la cámara, notificaciones, integración con Stripe como servicio externo y un feature de aprendizaje autónomo basado en Google ML Kit Barcode Scanning.

La priorización utiliza MoSCoW: **Must Have** identifica capacidades necesarias para cumplir el alcance o completar los flujos core; **Should Have** corresponde a capacidades importantes que admiten una implementación posterior dentro del proyecto; **Could Have** representa capacidades complementarias cuya inclusión depende de la capacidad disponible. Todo elemento incluido en el Product Backlog deberá implementarse antes de la entrega final o retirarse formalmente del alcance.

To-Be Scenario Mapping

El To-Be Scenario Mapping representa cómo realizarán sus tareas Jorge Luis Rivas, del segmento ganadero, y Valeria Mendoza, del segmento veterinario, cuando utilicen AniTec. Los escenarios incorporan el uso en campo, la conectividad intermitente, la identificación mediante cámara, los recordatorios y el acceso veterinario autorizado.

To-Be Scenario Mapping: pequeño o mediano ganadero

User Persona: Jorge Luis Rivas **Escenario:** Registro y seguimiento móvil de un animal durante el trabajo de campo.

FASES	Revisar actividades y seleccionar la finca	Identificar al animal	Registrar información en campo	Programar el seguimiento	Sincronizar y compartir el historial
DOING	Consulta sus actividades pendientes y los animales de la finca desde el teléfono.	Escanea el código QR del animal o lo busca manualmente.	Registra una incidencia, control o actualización; si no hay conexión, guarda el trabajo localmente.	Define la próxima actividad y activa el recordatorio correspondiente.	Revisa los cambios pendientes y permite que se sincronicen cuando vuelve la conexión.
THINKING	“Puedo saber qué debo atender antes de comenzar el recorrido”.	“Necesito encontrar la ficha correcta sin perder tiempo”.	“Quiero dejar el registro ahora para no olvidarlo después”.	“El teléfono debe ayudarme a recordar la siguiente acción”.	“Quiero confirmar que la información quedó guardada y disponible para el veterinario autorizado”.
FEELING	Orientado al tener una lista	Confiado cuando identifica al	Tranquilo al conservar el	Aliviado al delegar el recordatorio en la	Seguro cuando distingue los datos

	clara de pendientes.	animal correcto.	trabajo pese a la falta de conexión.	aplicación.	sincronizados de los pendientes.

To-Be Scenario Mapping: veterinario de campo

User Persona: Valeria Mendoza **Escenario:** Consulta de antecedentes, atención y seguimiento de un paciente autorizado.

FASES	Seleccionar un cliente autorizado	Identificar al paciente	Consultar antecedentes	Registrar la atención	Programar y sincronizar el seguimiento
DOING	Consulta los ganaderos que mantienen una autorización vigente.	Escanea el código QR o busca al animal dentro de los pacientes del cliente.	Revisa el historial sanitario disponible y su última sincronización.	Registra diagnóstico, tratamiento y recomendaciones, incluso si la conexión se interrumpe.	Programa el próximo control y verifica la sincronización de la atención.
THINKING	“Debo consultar únicamente los clientes que me autorizaron”.	“Necesito confirmar que atenderé al paciente correcto”.	“Los antecedentes me ayudarán a sustentar la decisión clínica”.	“La atención debe quedar registrada con mi autoría”.	“El ganadero debe poder consultar las indicaciones y la próxima fecha”.
FEELING	Seguro al reconocer el alcance de su autorización.	Confiado al verificar la identidad del animal.	Preparado al disponer de antecedentes ordenados.	Responsable al dejar un registro trazable.	Satisfecho al mantener continuidad entre visitas.

2.4.1. User Stories

Las Epics y las historias siguientes describen resultados esperados para la landing page, las aplicaciones móviles y los servicios. Las User Stories evitan decisiones de interfaz en sus criterios de aceptación; las Technical Stories describen capacidades sin interacción directa y utilizan el rol Developer. Por otro lado, las Spike Stories delimitan investigaciones técnicas breves para reducir incertidumbre y comprobar la viabilidad de una alternativa antes de implementarla.

Epics

Epic ID	Title	Description
EP-001	Landing Page	Comunica la propuesta de valor, los segmentos atendidos y los canales de acceso a las aplicaciones móviles de AniTec.
EP-002	Identity and Access Management	Gestiona el registro, la autenticación, la sesión y la autorización de ganaderos y veterinarios.

Epic ID	Title	Description
EP-003	Farm and Livestock Management	Permite organizar fincas y corrales y registrar, consultar, actualizar y archivar animales, con su fotografía, desde las aplicaciones móviles.
EP-004	Sanitary Management	Centraliza incidencias, diagnósticos, tratamientos, controles e historiales sanitarios de los animales.
EP-005	Veterinary Collaboration	Gestiona la relación autorizada entre ganaderos y veterinarios para el seguimiento de clientes y pacientes.
EP-006	Activities and Notifications	Organiza actividades ganaderas y sanitarias y genera recordatorios en los dispositivos móviles.
EP-007	Offline Storage and Synchronization	Mantiene información esencial y trabajo pendiente en el dispositivo y lo sincroniza cuando vuelve la conexión.
EP-008	Animal Identification and Autonomous Feature	Identifica animales mediante códigos QR y la cámara, incorporando una tecnología investigada de forma autónoma.
EP-009	Analytics and Reports	Presenta indicadores sanitarios y operativos relevantes para ganaderos y veterinarios.
EP-010	Financial and Subscription Management	Permite consultar finanzas, planes, pagos y el estado de la suscripción mediante un servicio externo.
EP-011	Accessibility, Internationalization and Resilience	Asegura una experiencia comprensible, accesible, internacionalizada y tolerante a errores.
EP-012	Mobile Platforms and Services	Agrupa la arquitectura y las integraciones técnicas de Android nativo, Flutter, API REST, almacenamiento y distribución.

User Stories

Story ID	User	Priority	Epic
US-001	Visitante	Must Have	EP-001

Title

Comprender la propuesta de valor de AniTec

Description

Como visitante, quiero conocer el problema que resuelve AniTec y sus beneficios para determinar si la solución se relaciona con mis necesidades.

Acceptance Criteria

Scenario 1: Propuesta disponible

Given el contenido público de AniTec está publicado

When el visitante accede a la dirección de la landing page

Then el sistema presenta la propuesta de valor y los beneficios principales para la gestión ganadera

Scenario 2: Contenido verificable

Given la startup todavía no dispone de resultados comerciales validados

When el visitante consulta cifras o testimonios

Then el sistema diferencia las metas y testimonios reales de cualquier información todavía no validada

Story ID	User	Priority	Epic
US-002	Visitante	Must Have	EP-001

Title Conocer las soluciones para cada segmento

Description

Como visitante, quiero conocer cómo AniTec ayuda a ganaderos y veterinarios para identificar la aplicación móvil adecuada para mi perfil.

Acceptance Criteria

Scenario 1: Información para ganaderos

Given el visitante pertenece al segmento ganadero

When el visitante consulta la información dirigida a su segmento

Then el sistema explica las capacidades móviles destinadas a la gestión del hato

Scenario 2: Información para veterinarios

Given el visitante pertenece al segmento veterinario

When el visitante consulta la información dirigida a su segmento

Then el sistema explica las capacidades de seguimiento de clientes y pacientes

Story ID	User	Priority	Epic
US-003	Visitante	Should Have	EP-001

Title Acceder a una landing page adaptable e internacionalizada

Description

Como visitante, quiero consultar la landing page desde distintos dispositivos y en un idioma disponible para comprender la información y acceder a los canales de contacto o descarga.

Acceptance Criteria

Scenario 1: Adaptación al dispositivo

Given el visitante utiliza un teléfono, una tableta o una computadora

When el visitante accede a la landing page

Then el contenido conserva su legibilidad y permite completar las acciones principales

Scenario 2: Idioma disponible

Given la landing page ofrece más de un idioma

When el visitante selecciona un idioma

Then el sistema muestra el contenido traducido y conserva la preferencia durante la navegación

Story ID	User	Priority	Epic
US-004	Ganadero o veterinario	Must Have	EP-002

Title	Registrar una cuenta según el rol

Description

Como ganadero o veterinario, quiero crear una cuenta con mi rol para acceder a las capacidades que corresponden a mi actividad.

Acceptance Criteria

Scenario 1: Registro válido

Given la persona proporciona datos válidos y selecciona un rol permitido

When la persona solicita crear su cuenta

Then el sistema registra la cuenta y asocia el rol seleccionado

Scenario 2: Datos duplicados o inválidos

Given ya existe una cuenta con el mismo correo o los datos incumplen una regla

When la persona solicita crear su cuenta

Then el sistema rechaza el registro e informa la causa sin exponer información sensible

Story ID	User	Priority	Epic
US-005	Usuario registrado	Must Have	EP-002

Title	Iniciar sesión

Description

Como usuario registrado, quiero autenticarme con mis credenciales para consultar de manera segura mi información en AniTec.

Acceptance Criteria

Scenario 1: Credenciales válidas

Given existe una cuenta activa con las credenciales proporcionadas

When el usuario solicita iniciar sesión

Then el sistema autentica al usuario y habilita las capacidades de su rol

Scenario 2: Credenciales inválidas

Given las credenciales no corresponden a una cuenta activa

When el usuario solicita iniciar sesión

Then el sistema rechaza el acceso mediante un mensaje que no revela qué dato es incorrecto

Story ID	User	Priority	Epic
US-006	Usuario autenticado	Must Have	EP-002

Title	Mantener y finalizar la sesión móvil

Description

Como usuario autenticado, quiero conservar mi sesión de manera segura y poder finalizarla para evitar accesos no autorizados a mis datos.

Acceptance Criteria

Scenario 1: Sesión vigente

Given el usuario tiene una sesión válida almacenada de forma segura

When el usuario vuelve a abrir la aplicación

Then el sistema recupera la sesión y valida su vigencia antes de entregar información protegida

Scenario 2: Cierre de sesión

Given el usuario mantiene una sesión activa

When el usuario solicita finalizarla

Then el sistema elimina las credenciales locales y bloquea el acceso a la información protegida

Story ID	User	Priority	Epic
US-007	Usuario autenticado	Must Have	EP-002

Title Acceder únicamente a información autorizada

Description

Como usuario autenticado, quiero acceder solamente a los datos permitidos para mi rol y relaciones vigentes para proteger la información ganadera y clínica.

Acceptance Criteria

Scenario 1: Acceso permitido

Given el usuario posee el rol y la relación requeridos por una operación

When el usuario solicita consultar o modificar información

Then el sistema procesa la operación dentro del alcance autorizado

Scenario 2: Acceso denegado

Given el usuario no posee el rol o la relación requeridos

When el usuario solicita consultar o modificar información protegida

Then el sistema rechaza la operación y registra el resultado de autorización

Story ID	User	Priority	Epic
US-008	Ganadero	Must Have	EP-003

Title Consultar las fincas registradas

Description

Como ganadero, quiero consultar mis fincas para organizar los animales según su unidad productiva.

Acceptance Criteria

Scenario 1: Fincas existentes

Given el ganadero tiene una o más fincas registradas

When el ganadero consulta sus fincas

Then el sistema devuelve únicamente las fincas que le pertenecen

Scenario 2: Sin fincas

Given el ganadero todavía no registra fincas

When el ganadero consulta sus fincas

Then el sistema informa que no existen unidades productivas registradas

Story ID	User	Priority	Epic
US-009	Ganadero	Must Have	EP-003

Title Registrar y actualizar una finca

Description

Como ganadero, quiero registrar y mantener los datos de una finca para asociar correctamente mis animales y actividades.

Acceptance Criteria

Scenario 1: Registro válido

Given el ganadero proporciona los datos obligatorios de una nueva finca

When el ganadero solicita registrarla

Then el sistema crea la finca y la deja disponible para asociar animales

Scenario 2: Actualización válida

Given existe una finca perteneciente al ganadero

When el ganadero modifica información permitida

Then el sistema conserva los cambios y la relación con sus animales

Story ID	User	Priority	Epic
US-010	Ganadero	Must Have	EP-003

Title Consultar y buscar animales

Description

Como ganadero, quiero consultar y buscar los animales de mis fincas para localizar rápidamente el registro que necesito.

Acceptance Criteria

Scenario 1: Consulta autorizada

Given el ganadero tiene animales registrados

When el ganadero consulta sus animales

Then el sistema devuelve únicamente animales asociados a sus fincas

Scenario 2: Búsqueda

Given existen animales que coinciden con un código, nombre, especie o raza

When el ganadero realiza una búsqueda

Then el sistema devuelve los animales coincidentes

Story ID	User	Priority	Epic
US-011	Ganadero	Must Have	EP-003

Title Registrar un animal

Description

Como ganadero, quiero registrar un animal en una de mis fincas para iniciar su trazabilidad sanitaria y productiva.

Acceptance Criteria

Scenario 1: Registro válido

Given el ganadero tiene una finca y proporciona los datos obligatorios del animal

When el ganadero solicita registrar el animal

Then el sistema crea un identificador único y asocia el animal con la finca

Scenario 2: Código duplicado

Given ya existe un animal del ganadero con el mismo código de identificación

When el ganadero solicita registrar otro animal con ese código

Then el sistema rechaza la operación e informa la duplicidad

Story ID	User	Priority	Epic
US-012	Ganadero	Must Have	EP-003
Title	Actualizar o archivar un animal		

Description

Como ganadero, quiero actualizar o archivar un animal para mantener vigente el inventario sin perder su historial.

Acceptance Criteria

Scenario 1: Actualización válida

Given el animal pertenece al ganadero

When el ganadero modifica datos permitidos

Then el sistema conserva los cambios y mantiene el historial asociado

Scenario 2: Archivado

Given el animal ya no forma parte del hato activo

When el ganadero solicita archivarlo

Then el sistema lo excluye del inventario activo y conserva su historial para consulta

Story ID	User	Priority	Epic
US-013	Usuario autorizado	Must Have	EP-003
Title	Consultar el detalle de un animal		

Description

Como usuario autorizado, quiero consultar la ficha de un animal para conocer sus datos e historial relevante antes de realizar una acción.

Acceptance Criteria

Scenario 1: Detalle autorizado

Given el usuario está autorizado para consultar el animal

When el usuario solicita su detalle

Then el sistema devuelve los datos generales y las referencias a su historial

Scenario 2: Animal no autorizado

Given el usuario no tiene relación autorizada con el animal

When el usuario solicita su detalle

Then el sistema rechaza la consulta sin revelar información del animal

Story ID	User	Priority	Epic
US-014	Usuario autorizado	Must Have	EP-004

Title Consultar eventos sanitarios

Description

Como usuario autorizado, quiero consultar los eventos sanitarios de los animales a mi alcance para conocer su situación de salud.

Acceptance Criteria

Scenario 1: Eventos existentes

Given existen eventos sanitarios de animales autorizados

When el usuario consulta los eventos

Then el sistema devuelve la fecha, el tipo, el animal y el responsable de cada registro

Scenario 2: Sin eventos

Given no existen eventos sanitarios dentro del alcance autorizado

When el usuario realiza la consulta

Then el sistema informa que no existen registros disponibles

Story ID	User	Priority	Epic
US-015	Ganadero	Must Have	EP-004

Title Registrar una incidencia sanitaria

Description

Como ganadero, quiero registrar una incidencia observada en uno de mis animales para dejar evidencia y solicitar seguimiento oportuno.

Acceptance Criteria

Scenario 1: Incidencia válida

Given el animal pertenece al ganadero y los datos obligatorios están completos

When el ganadero registra la incidencia

Then el sistema incorpora el evento al historial del animal

Scenario 2: Animal ajeno

Given el animal no pertenece al ganadero

When el ganadero intenta registrar una incidencia

Then el sistema rechaza la operación

Story ID	User	Priority	Epic
US-016	Veterinario	Must Have	EP-004

Title Registrar diagnóstico y tratamiento

Description

Como veterinario, quiero registrar el diagnóstico, tratamiento y recomendaciones de un paciente autorizado para documentar la atención realizada.

Acceptance Criteria

Scenario 1: Atención autorizada

Given el paciente pertenece a un cliente que autorizó al veterinario

When el veterinario registra una atención con datos válidos

Then el sistema incorpora el registro al historial e identifica al profesional responsable

Scenario 2: Atención no autorizada

Given el veterinario no tiene acceso vigente al paciente

When el veterinario intenta registrar una atención

Then el sistema rechaza la operación

Story ID	User	Priority	Epic
US-017	Usuario autorizado	Must Have	EP-004

Title Consultar el historial sanitario de un animal

Description

Como usuario autorizado, quiero consultar cronológicamente el historial sanitario de un animal para tomar decisiones con base en sus antecedentes.

Acceptance Criteria

Scenario 1: Historial disponible

Given el animal tiene eventos sanitarios y el usuario posee acceso

When el usuario consulta el historial

Then el sistema devuelve los eventos ordenados y con su responsable

Scenario 2: Conectividad interrumpida

Given existe una copia local vigente del historial autorizado

When el usuario consulta el historial sin conexión

Then el sistema entrega la información disponible e indica cuándo fue actualizada

Story ID	User	Priority	Epic
US-018	Autor de un registro sanitario	Should Have	EP-004

Title Corregir un registro sanitario con trazabilidad

Description

Como autor de un registro sanitario, quiero corregir información incorrecta dejando constancia del cambio para preservar la confiabilidad del historial.

Acceptance Criteria

Scenario 1: Corrección permitida

Given el usuario es autor del registro y proporciona el motivo

When el usuario solicita corregir información permitida

Then el sistema guarda la nueva versión e identifica la fecha, el autor y el motivo

Scenario 2: Eliminación no permitida

Given el registro clínico ya forma parte del historial

When el usuario intenta eliminarlo definitivamente

Then el sistema conserva el registro y ofrece el mecanismo de corrección o anulación trazable

Story ID	User	Priority	Epic
US-019	Veterinario	Should Have	EP-004

Title Programar un control sanitario posterior

Description

Como veterinario, quiero programar el próximo control de un paciente para dar continuidad al tratamiento indicado.

Acceptance Criteria

Scenario 1: Control programado

Given el veterinario registra una atención autorizada

When el veterinario establece una fecha futura de control

Then el sistema crea la actividad asociada al paciente y al cliente

Scenario 2: Fecha inválida

Given la fecha propuesta no es posterior a la atención

When el veterinario solicita programar el control

Then el sistema rechaza la programación e informa la regla incumplida

Story ID	User	Priority	Epic
US-020	Ganadero	Must Have	EP-005

Title Recibir una solicitud de seguimiento veterinario

Description

Como ganadero, quiero recibir solicitudes de veterinarios para decidir quién puede consultar y registrar información sanitaria de mis animales.

Acceptance Criteria

Scenario 1: Solicitud válida

Given un veterinario registrado solicita relacionarse con el ganadero

When el sistema procesa la solicitud

Then el ganadero recibe la solicitud con la identidad del profesional y su estado pendiente

Scenario 2: Solicitud duplicada

Given ya existe una solicitud pendiente o una relación vigente

When el mismo veterinario envía otra solicitud

Then el sistema evita crear una relación duplicada

Story ID	User	Priority	Epic
US-021	Ganadero	Must Have	EP-005

Title	Aceptar o rechazar acceso veterinario

Description

Como ganadero, quiero aceptar o rechazar una solicitud veterinaria para controlar el acceso a la información de mis animales.

Acceptance Criteria

Scenario 1: Solicitud aceptada

Given existe una solicitud pendiente

When el ganadero la acepta

Then el sistema activa la relación y autoriza al veterinario dentro del alcance definido

Scenario 2: Solicitud rechazada

Given existe una solicitud pendiente

When el ganadero la rechaza

Then el sistema cierra la solicitud sin conceder acceso

Story ID	User	Priority	Epic
US-022	Ganadero	Must Have	EP-005

Title	Revocar el acceso de un veterinario

Description

Como ganadero, quiero revocar una autorización veterinaria para impedir futuras consultas o registros sobre mis animales.

Acceptance Criteria

Scenario 1: Revocación válida

Given existe una relación veterinaria vigente

When el ganadero revoca la autorización

Then el sistema impide nuevas operaciones del veterinario y conserva la autoría de registros anteriores

Scenario 2: Relación inexistente

Given no existe una relación vigente con el veterinario

When el ganadero solicita revocarla

Then el sistema informa que no existe acceso activo

Story ID	User	Priority	Epic
US-023	Veterinario	Must Have	EP-005

Title Consultar clientes y pacientes autorizados

Description

Como veterinario, quiero consultar mis clientes y sus pacientes autorizados para organizar las atenciones de campo.

Acceptance Criteria

Scenario 1: Clientes autorizados

Given el veterinario tiene relaciones vigentes

When el veterinario consulta sus clientes

Then el sistema devuelve únicamente los ganaderos y pacientes autorizados

Scenario 2: Sin clientes

Given el veterinario no tiene relaciones vigentes

When el veterinario consulta sus clientes

Then el sistema informa que no existen clientes autorizados

Story ID	User	Priority	Epic
US-024	Veterinario	Must Have	EP-005

Title Consultar antecedentes de un paciente autorizado

Description

Como veterinario, quiero consultar los antecedentes de un paciente autorizado para sustentar el diagnóstico y seguimiento.

Acceptance Criteria

Scenario 1: Acceso vigente

Given el ganadero mantiene autorización vigente

When el veterinario consulta los antecedentes del paciente

Then el sistema devuelve la información sanitaria permitida

Scenario 2: Acceso revocado

Given el ganadero revocó la autorización

When el veterinario vuelve a solicitar los antecedentes

Then el sistema rechaza la consulta

Story ID	User	Priority	Epic
US-025	Usuario autenticado	Must Have	EP-006

Title Consultar actividades programadas

Description

Como usuario autenticado, quiero consultar mis actividades ganaderas o sanitarias para organizar el trabajo pendiente.

Acceptance Criteria

Scenario 1: Actividades existentes

Given el usuario tiene actividades dentro de un periodo

When el usuario consulta su programación

Then el sistema devuelve las actividades ordenadas por fecha y prioridad

Scenario 2: Actividades por rol

Given existen actividades de distintos propietarios o profesionales

When el usuario realiza la consulta

Then el sistema devuelve solamente las actividades dentro de su alcance

Story ID	User	Priority	Epic
US-026	Usuario autenticado	Must Have	EP-006

Title Gestionar una actividad o recordatorio

Description

Como usuario autenticado, quiero crear, actualizar o cancelar una actividad para mantener vigente mi planificación.

Acceptance Criteria

Scenario 1: Actividad válida

Given el usuario proporciona una fecha y datos válidos

When el usuario guarda una actividad

Then el sistema registra la actividad y programa el recordatorio cuando corresponde

Scenario 2: Cancelación

Given existe una actividad pendiente perteneciente al usuario

When el usuario solicita cancelarla

Then el sistema cambia su estado y evita recordatorios posteriores

Story ID	User	Priority	Epic
US-027	Usuario autenticado	Must Have	EP-006

Title Recibir una notificación de actividad

Description

Como usuario autenticado, quiero recibir notificaciones de actividades próximas para reducir olvidos de controles y tareas importantes.

Acceptance Criteria

Scenario 1: Permiso concedido

Given existe una actividad pendiente y el dispositivo permite notificaciones

When llega el momento configurado

Then el sistema entrega una notificación relacionada con la actividad

Scenario 2: Permiso denegado

Given el dispositivo no permite notificaciones

When se aproxima una actividad

Then el sistema conserva la actividad para consulta y comunica que las notificaciones están deshabilitadas

Story ID	User	Priority	Epic
US-028	Usuario autenticado	Should Have	EP-006

Title Atender o reprogramar una actividad

Description

Como usuario autenticado, quiero marcar una actividad como atendida o reprogramarla para mantener actualizado su seguimiento.

Acceptance Criteria

Scenario 1: Actividad atendida

Given existe una actividad pendiente

When el usuario registra que fue atendida

Then el sistema actualiza su estado y conserva la fecha de atención

Scenario 2: Actividad reprogramada

Given existe una actividad pendiente

When el usuario establece una nueva fecha válida

Then el sistema actualiza la programación y reemplaza el recordatorio anterior

Story ID	User	Priority	Epic
US-029	Usuario autenticado	Must Have	EP-007

Title Consultar información esencial sin conexión

Description

Como usuario autenticado, quiero consultar información esencial previamente sincronizada cuando no tengo conexión para continuar mi trabajo en campo.

Acceptance Criteria

Scenario 1: Datos locales disponibles

Given el dispositivo conserva datos autorizados y no existe conexión

When el usuario consulta fincas, animales, actividades o historiales disponibles

Then el sistema entrega la copia local e informa su última sincronización

Scenario 2: Datos locales inexistentes

Given no existe conexión ni una copia local de la información solicitada

When el usuario realiza la consulta

Then el sistema informa que los datos requieren una sincronización inicial

Story ID	User	Priority	Epic
US-030	Usuario autenticado	Must Have	EP-007

Title	Guardar trabajo pendiente sin conexión

Description

Como usuario autenticado, quiero guardar localmente un registro pendiente cuando no tengo conexión para evitar perder la información ingresada.

Acceptance Criteria

Scenario 1: Guardado local

Given la conexión no está disponible y los datos cumplen las reglas locales

When el usuario guarda un animal, incidencia, atención o actividad

Then el sistema conserva la operación como pendiente de sincronización

Scenario 2: Datos inválidos

Given los datos incumplen una regla que puede validarse localmente

When el usuario intenta guardar la operación

Then el sistema no la incorpora a la cola y explica la corrección necesaria

Story ID	User	Priority	Epic
US-031	Usuario autenticado	Must Have	EP-007

Title	Sincronizar operaciones pendientes

Description

Como usuario autenticado, quiero sincronizar el trabajo pendiente al recuperar la conexión para mantener consistentes el dispositivo y el servidor.

Acceptance Criteria

Scenario 1: Sincronización exitosa

Given existen operaciones locales pendientes y la conexión vuelve a estar disponible

When el sistema inicia la sincronización

Then el servidor procesa cada operación una sola vez y el dispositivo actualiza su estado

Scenario 2: Sincronización parcial

Given una operación es rechazada y otras son válidas

When el sistema procesa la cola

Then el sistema conserva como pendiente solo la operación rechazada e informa su causa

Story ID	User	Priority	Epic
US-032	Usuario autenticado	Should Have	EP-007

Title	Resolver errores o conflictos de sincronización

Description

Como usuario autenticado, quiero conocer y resolver los conflictos de sincronización para evitar sobrescribir información válida.

Acceptance Criteria

Scenario 1: Conflicto detectado

Given el registro local y el remoto fueron modificados después de la última sincronización

When el sistema intenta sincronizarlos

Then el sistema conserva ambas referencias y solicita aplicar una regla de resolución válida

Scenario 2: Reintento seguro

Given una operación falló por una causa temporal

When el usuario o el sistema reintenta la sincronización

Then el servidor evita duplicar la operación ya procesada

Story ID	User	Priority	Epic
US-033	Usuario autorizado	Must Have	EP-008

Title Identificar un animal mediante código QR

Description

Como usuario autorizado, quiero escanear el código QR de un animal con la cámara para abrir rápidamente su ficha.

Acceptance Criteria

Scenario 1: Código reconocido

Given el dispositivo concede acceso a la cámara y el código pertenece a un animal autorizado

When el usuario realiza el escaneo

Then el sistema identifica al animal y recupera su información disponible

Scenario 2: Código no autorizado

Given el código corresponde a un animal fuera del alcance del usuario

When el usuario realiza el escaneo

Then el sistema rechaza la consulta sin revelar datos del animal

Story ID	User	Priority	Epic
US-034	Usuario autorizado	Must Have	EP-008

Title Identificar un animal sin utilizar la cámara

Description

Como usuario autorizado, quiero buscar manualmente un animal cuando la cámara o el código QR no estén disponibles para continuar la tarea.

Acceptance Criteria

Scenario 1: Permiso de cámara denegado

Given el dispositivo no concede acceso a la cámara

When el usuario intenta iniciar la identificación

Then el sistema mantiene disponible la búsqueda por código o nombre

Scenario 2: Código ilegible

Given la cámara no reconoce un código válido

When el usuario concluye el intento de escaneo

Then el sistema informa el resultado y permite identificar el animal manualmente

Story ID	User	Priority	Epic
US-035	Ganadero	Should Have	EP-009

Title Consultar indicadores del hato

Description

Como ganadero, quiero consultar indicadores de mis animales, actividades y eventos sanitarios para priorizar decisiones de manejo.

Acceptance Criteria

Scenario 1: Indicadores con datos

Given el ganadero tiene información registrada

When el ganadero consulta sus indicadores

Then el sistema calcula resultados únicamente con datos de sus fincas y comunica el periodo considerado

Scenario 2: Indicadores sin datos suficientes

Given no existen datos suficientes para calcular un indicador

When el ganadero realiza la consulta

Then el sistema informa la falta de información sin presentar resultados engañosos

Story ID	User	Priority	Epic
US-036	Veterinario	Should Have	EP-009

Title Consultar indicadores sanitarios de clientes

Description

Como veterinario, quiero consultar indicadores sanitarios de mis clientes autorizados para priorizar pacientes y controles pendientes.

Acceptance Criteria

Scenario 1: Indicadores autorizados

Given el veterinario tiene clientes vigentes con información sanitaria

When el veterinario consulta los indicadores

Then el sistema calcula resultados únicamente con los datos autorizados

Scenario 2: Autorización revocada

Given un cliente deja de autorizar al veterinario

When el veterinario actualiza la consulta

Then el sistema excluye los datos de ese cliente

Story ID	User	Priority	Epic
US-037	Ganadero	Could Have	EP-010

Title	Registrar y consultar movimientos financieros

Description

Como ganadero, quiero registrar ingresos y egresos y consultar mi balance para conocer el resultado económico de mi operación.

Acceptance Criteria

Scenario 1: Movimiento válido

Given el ganadero proporciona tipo, monto, categoría y fecha válidos

When el ganadero registra el movimiento

Then el sistema incorpora el movimiento y actualiza el balance correspondiente

Scenario 2: Consulta por periodo

Given existen movimientos del ganadero en distintos periodos

When el ganadero consulta un periodo

Then el sistema calcula ingresos, egresos y balance únicamente con sus movimientos

Story ID	User	Priority	Epic
US-038	Usuario autenticado	Must Have	EP-010

Title	Consultar planes de suscripción

Description

Como usuario autenticado, quiero comparar los planes vigentes para seleccionar una alternativa acorde con mi operación.

Acceptance Criteria

Scenario 1: Planes disponibles

Given existen planes activos definidos por AniTec

When el usuario consulta las alternativas

Then el sistema devuelve precio, moneda, periodicidad, límites y capacidades de cada plan

Scenario 2: Precio por validar

Given la startup todavía no aprueba un precio comercial

When el usuario consulta un entorno de validación

Then el sistema identifica claramente la información como propuesta o prueba

Story ID	User	Priority	Epic
US-039	Usuario autenticado	Must Have	EP-010

Title	Iniciar un pago mediante un proveedor externo

Description

Como usuario autenticado, quiero iniciar el pago de un plan mediante un proveedor externo para contratar una suscripción sin entregar datos bancarios directamente a AniTec.

Acceptance Criteria

Scenario 1: Sesión de pago creada

Given el usuario selecciona un plan activo

When el usuario solicita continuar con el pago

Then el sistema crea una sesión asociada al usuario y transfiere el proceso al proveedor externo

Scenario 2: Solicitud repetida

Given ya existe una operación en proceso para la misma referencia

When el usuario repite la solicitud

Then el sistema evita registrar cobros duplicados

Story ID	User	Priority	Epic
US-040	Usuario autenticado	Must Have	EP-010
Title	Consultar el resultado del pago y la suscripción		

Description

Como usuario autenticado, quiero consultar el resultado de mis pagos y el estado de mi suscripción para conocer las capacidades disponibles en mi cuenta.

Acceptance Criteria

Scenario 1: Pago confirmado

Given el proveedor externo confirma una operación válida

When el sistema procesa la confirmación

Then el sistema activa o actualiza la suscripción una sola vez y conserva la referencia del pago

Scenario 2: Pago cancelado o fallido

Given el proveedor externo no confirma la operación

When el usuario vuelve a AniTec

Then el sistema conserva la suscripción anterior e informa el estado sin registrar un pago exitoso

Story ID	User	Priority	Epic
US-041	Usuario	Should Have	EP-011
Title	Cambiar el idioma de la aplicación		

Description

Como usuario, quiero elegir un idioma disponible para comprender la información y realizar mis tareas con términos consistentes.

Acceptance Criteria

Scenario 1: Cambio de idioma

Given la aplicación dispone de las traducciones requeridas

When el usuario selecciona otro idioma

Then el sistema actualiza los textos y formatos localizables

Scenario 2: Preferencia conservada

Given el usuario eligió un idioma

When el usuario vuelve a abrir la aplicación

Then el sistema aplica la preferencia almacenada

Story ID	User	Priority	Epic
US-042	Usuario	Must Have	EP-011

Title Utilizar la aplicación con necesidades de accesibilidad

Description

Como usuario con necesidades de accesibilidad, quiero percibir y comprender el contenido para completar las tareas principales de manera autónoma.

Acceptance Criteria

Scenario 1: Escalado de texto

Given el dispositivo utiliza un tamaño de fuente mayor

When el usuario accede a una tarea principal

Then el contenido mantiene su legibilidad y no oculta información necesaria

Scenario 2: Tecnología de asistencia

Given el usuario utiliza un lector de pantalla

When el usuario recorre la información y las acciones principales

Then el sistema comunica nombres, estados y propósitos comprensibles

Story ID	User	Priority	Epic
US-043	Usuario	Must Have	EP-011

Title Comprender errores y estados de conectividad

Description

Como usuario, quiero recibir información clara ante errores o cambios de conectividad para decidir cómo continuar sin perder mi trabajo.

Acceptance Criteria

Scenario 1: Error recuperable

Given una operación falla por una causa temporal

When el sistema recibe el error

Then el sistema conserva el trabajo posible y comunica una acción de recuperación

Scenario 2: Cambio de conectividad

Given el dispositivo pierde o recupera la conexión

When el sistema detecta el cambio

Then el sistema actualiza el estado de trabajo local y sincronización sin afirmar que una operación terminó antes de confirmarla

Story ID	User	Priority	Epic
US-044	Ganadero	Must Have	EP-003

Title	Gestionar los corrales de una finca

Description

Como ganadero, quiero crear, editar y eliminar los corrales de mis fincas para organizar mis animales según el lugar donde se encuentran.

Acceptance Criteria

Scenario 1: Alta de un corral

Given el ganadero tiene una finca registrada y proporciona el nombre del corral

When el ganadero solicita crear el corral

Then el sistema asocia el corral con la finca y lo ofrece para ubicar animales

Scenario 2: Datos incompletos

Given el ganadero no proporciona el nombre del corral

When el ganadero solicita crearlo

Then el sistema rechaza la operación e indica el dato obligatorio

Scenario 3: Eliminación de un corral

Given el ganadero tiene un corral registrado

When el ganadero confirma su eliminación

Then el sistema elimina el corral y deja de ofrecerlo en la finca

Story ID	User	Priority	Epic
US-045	Ganadero	Should Have	EP-003

Title	Adjuntar una fotografía al animal

Description

Como ganadero, quiero adjuntar una fotografía a un animal para reconocerlo visualmente en su ficha.

Acceptance Criteria

Scenario 1: Fotografía válida

Given el ganadero registra o edita un animal y proporciona una imagen válida

When el ganadero guarda el animal

Then el sistema almacena la imagen y la asocia con el animal para mostrarla en su ficha

Scenario 2: Imagen no admitida

Given la imagen supera el tamaño máximo o no tiene un formato admitido

When el ganadero intenta adjuntarla

Then el sistema rechaza la imagen e informa el motivo sin perder los demás datos del animal

Technical Stories

La estrategia técnica de AniTec considera Android nativo y Flutter porque la selección de un enfoque multiplataforma debe evaluar su adopción, mantenibilidad y consistencia entre plataformas (Jošt & Taneski, 2025). Además, las

historias técnicas incorporan pruebas unitarias, de interfaz e integración en distintas configuraciones, debido a que la evidencia reciente identifica una adopción y cobertura todavía limitadas de las pruebas automatizadas en proyectos Android reales (Mahmud et al., 2025).

Story ID	User	Priority	Epic
TS-001	Developer	Must Have	EP-012

Title Configurar la aplicación Android nativa

Description

Como Developer, quiero configurar el proyecto Android con Kotlin y Jetpack Compose para implementar una aplicación nativa mantenible y ejecutable en dispositivos físicos.

Acceptance Criteria

Scenario 1: Proyecto reproducible

Given el repositorio contiene la configuración documentada

When el Developer compila el proyecto en un ambiente limpio

Then el sistema genera una aplicación instalable sin depender de archivos locales no versionados

Scenario 2: Dispositivo físico

Given existe una compilación de desarrollo válida

When el Developer instala y ejecuta la aplicación en un dispositivo Android compatible

Then las funciones base inician sin errores bloqueantes

Story ID	User	Priority	Epic
TS-002	Developer	Must Have	EP-012

Title Configurar la aplicación multiplataforma con Flutter

Description

Como Developer, quiero configurar un proyecto Flutter con Dart para entregar una experiencia multiplataforma coherente con la aplicación Android nativa.

Acceptance Criteria

Scenario 1: Proyecto reproducible

Given el repositorio contiene las versiones y dependencias documentadas

When el Developer ejecuta la compilación desde un ambiente limpio

Then el sistema genera una aplicación para las plataformas objetivo definidas

Scenario 2: Consistencia funcional

Given existe un flujo core implementado en Android nativo

When el Developer implementa el mismo flujo en Flutter

Then ambas aplicaciones aplican las mismas reglas de negocio y contrato de servicio

Story ID	User	Priority	Epic
TS-003	Developer	Must Have	EP-012

Title	Definir la arquitectura móvil por capas y bounded contexts

Description

Como Developer, quiero definir una arquitectura móvil alineada con Domain-Driven Design para separar presentación, aplicación, dominio e infraestructura.

Acceptance Criteria

Scenario 1: Límites definidos

Given se conocen los bounded contexts y dependencias permitidas

When el equipo documenta la arquitectura móvil

Then cada módulo identifica sus responsabilidades y evita dependencias contrarias a la dirección acordada

Scenario 2: Regla de dominio aislada

Given existe una regla que no depende del sistema operativo

When el Developer implementa y prueba la regla

Then la regla se ejecuta sin requerir componentes de interfaz o infraestructura

Story ID	User	Priority	Epic
TS-004	Developer	Must Have	EP-012

Title	Integrar las aplicaciones con la API REST interna

Description

Como Developer, quiero implementar clientes HTTP autenticados en Android y Flutter para consumir de forma consistente la API REST de AniTec.

Acceptance Criteria

Scenario 1: Respuesta exitosa

Given la API devuelve una respuesta válida

When la aplicación solicita un recurso

Then el cliente transforma la respuesta al modelo correspondiente sin exponer el formato de transporte al dominio

Scenario 2: Respuesta de error

Given la API devuelve un código de error o no responde

When la aplicación procesa la solicitud

Then el cliente clasifica el resultado para que la aplicación pueda informar o reintentar de forma segura

Story ID	User	Priority	Epic
TS-005	Developer	Must Have	EP-007

Title	Implementar persistencia local segura en Android

Description

Como Developer, quiero implementar persistencia local con Room para almacenar información esencial y operaciones pendientes en la aplicación Android.

Acceptance Criteria

Scenario 1: Persistencia

Given la aplicación recibe información autorizada

When el repositorio local la almacena

Then los datos permanecen disponibles después de reiniciar la aplicación

Scenario 2: Protección y limpieza

Given el usuario finaliza la sesión o cambia de cuenta

When la aplicación procesa el cambio

Then los datos protegidos que no deben compartirse se eliminan o aíslan según la política definida

Story ID	User	Priority	Epic
TS-006	Developer	Must Have	EP-007

Title Implementar persistencia local en Flutter

Description

Como Developer, quiero implementar una base local compatible con Flutter para ofrecer el mismo comportamiento offline en la aplicación multiplataforma.

Acceptance Criteria

Scenario 1: Esquema equivalente

Given Android y Flutter comparten las reglas de datos offline

When el Developer configura el almacenamiento local de Flutter

Then los registros esenciales y operaciones pendientes conservan los campos necesarios para sincronizarse

Scenario 2: Migración local

Given existe una versión anterior del esquema en el dispositivo

When la aplicación se actualiza

Then la migración conserva los datos compatibles sin bloquear el inicio

Story ID	User	Priority	Epic
TS-007	Developer	Must Have	EP-007

Title Implementar sincronización idempotente

Description

Como Developer, quiero implementar una cola de sincronización idempotente para enviar operaciones pendientes sin duplicarlas ni perderlas.

Acceptance Criteria

Scenario 1: Operación identificada

Given una operación local pendiente posee un identificador único

When el cliente la envía más de una vez por un fallo temporal

Then el servidor produce un único efecto y devuelve el resultado correspondiente

Scenario 2: Conflicto

Given el servidor detecta una versión incompatible

When el cliente intenta sincronizar

Then el sistema conserva el conflicto y aplica la estrategia documentada antes de reemplazar información

Story ID	User	Priority	Epic
TS-008	Developer	Must Have	EP-002

Title	Proteger credenciales y datos de sesión

Description

Como Developer, quiero almacenar tokens mediante mecanismos seguros de cada plataforma para reducir la exposición de credenciales.

Acceptance Criteria

Scenario 1: Almacenamiento seguro

Given el servidor entrega un token válido

When la aplicación conserva la sesión

Then el token se almacena mediante el mecanismo seguro definido para la plataforma

Scenario 2: Token vencido

Given el servidor rechaza un token por expiración

When el cliente recibe la respuesta

Then la aplicación invalida la sesión o ejecuta la renovación autorizada sin reutilizar credenciales inválidas

Story ID	User	Priority	Epic
TS-009	Developer	Must Have	EP-006

Title	Implementar notificaciones móviles

Description

Como Developer, quiero integrar notificaciones locales o push para entregar recordatorios sanitarios respetando los permisos del sistema operativo.

Acceptance Criteria

Scenario 1: Programación

Given existe una actividad válida y el permiso correspondiente

When la aplicación programa el recordatorio

Then el sistema operativo recibe una notificación con una referencia válida a la actividad

Scenario 2: Permiso ausente

Given el permiso no fue concedido

When la aplicación procesa una actividad con recordatorio

Then la actividad permanece registrada y la aplicación informa que no puede entregar la notificación

Story ID	User	Priority	Epic
TS-010	Developer	Must Have	EP-008

Title	Integrar identificación QR mediante Google ML Kit

Description

Como Developer, quiero integrar Google ML Kit Barcode Scanning y una alternativa compatible en Flutter para identificar animales mediante códigos QR usando la cámara.

Acceptance Criteria

Scenario 1: Código compatible

Given la cámara recibe un código QR válido

When la biblioteca procesa la imagen

Then la aplicación obtiene el identificador sin almacenar imágenes innecesarias

Scenario 2: Lectura fallida

Given la biblioteca no reconoce un código válido

When finaliza el intento configurado

Then la aplicación devuelve un resultado controlado y conserva la alternativa manual

Story ID	User	Priority	Epic
TS-011	Developer	Must Have	EP-010

Title	Integrar el checkout externo de Stripe

Description

Como Developer, quiero integrar el checkout de Stripe mediante el backend y enlaces seguros para procesar pagos de prueba desde las aplicaciones móviles.

Acceptance Criteria

Scenario 1: Creación de checkout

Given el cliente solicita pagar un plan activo

When el backend valida la solicitud

Then el servicio crea una sesión de Stripe y devuelve una URL permitida

Scenario 2: Confirmación idempotente

Given Stripe informa el resultado mediante el mecanismo configurado

When el backend procesa el evento

Then el pago y la suscripción se actualizan una sola vez

Story ID	User	Priority	Epic
TS-012	Developer	Must Have	EP-011

Title	Aplicar internacionalización y accesibilidad móvil

Description

Como Developer, quiero centralizar recursos traducibles y aplicar prácticas de accesibilidad para mantener una experiencia coherente en Android y Flutter.

Acceptance Criteria

Scenario 1: Recursos traducibles

Given existen idiomas soportados

When el Developer agrega o modifica contenido

Then los textos visibles provienen de recursos localizables y no quedan cadenas funcionales sin traducir

Scenario 2: Verificación accesible

Given existen flujos core implementados

When el equipo ejecuta verificaciones con escalado de texto y lector de pantalla

Then los problemas encontrados se documentan y los bloqueantes se corrigen

Story ID	User	Priority	Epic
TS-013	Developer	Must Have	EP-012

Title Adaptar y documentar los servicios backend para móviles

Description

Como Developer, quiero adaptar los endpoints existentes y documentarlos con OpenAPI para soportar los flujos móviles y las reglas de autorización.

Acceptance Criteria

Scenario 1: Contrato documentado

Given un endpoint forma parte del alcance móvil

When el Developer publica la documentación OpenAPI

Then el contrato describe autenticación, parámetros, respuestas exitosas y errores

Scenario 2: Autorización

Given una solicitud autenticada intenta acceder a un recurso ajeno

When la API evalúa el rol y la relación

Then la API rechaza la operación mediante el código HTTP correspondiente

Story ID	User	Priority	Epic
TS-014	Developer	Should Have	EP-012

Title Automatizar pruebas de los flujos móviles críticos

Description

Como Developer, quiero automatizar pruebas unitarias y de integración para detectar regresiones en dominio, persistencia, sincronización y consumo de servicios.

Acceptance Criteria

Scenario 1: Pruebas de dominio

Given existen reglas críticas de autorización, estado o sincronización

When el equipo ejecuta la suite

Then las pruebas verifican resultados exitosos y alternativos de manera determinista

Scenario 2: Integración controlada

Given existen repositorios locales y clientes de servicio

When el equipo ejecuta pruebas con dobles o ambientes controlados

Then la suite comprueba persistencia, mapeo y tratamiento de errores

Story ID	User	Priority	Epic
TS-015	Developer	Must Have	EP-012
Title	Configurar compilación y distribución de versiones móviles		

Description

Como Developer, quiero automatizar compilaciones firmadas y distribuir versiones mediante Firebase App Distribution para probarlas en dispositivos físicos.

Acceptance Criteria

Scenario 1: Compilación versionada

Given el repositorio alcanza un hito de entrega

When el pipeline genera la versión

Then el artefacto incluye un número de versión trazable y una configuración válida

Scenario 2: Distribución

Given existe un artefacto aprobado para pruebas

When el equipo publica la versión en Firebase App Distribution

Then los evaluadores autorizados pueden instalarla y se registra la versión distribuida

Matriz de coherencia con el alcance móvil y los requisitos del curso

La siguiente matriz hace trazable cada condición tecnológica obligatoria con una decisión verificable y con los elementos del backlog que permiten implementarla. Ambas aplicaciones consumen el mismo dominio y la misma API REST, pero conservan implementaciones y pruebas propias para demostrar el desarrollo Android nativo y el desarrollo multiplataforma.

Aspecto requerido	Decisión para AniTec	Evidencia esperada	Historias relacionadas
Aplicación Android nativa	Kotlin y Jetpack Compose, con arquitectura por capas y funcionalidades core de ganaderos y veterinarios.	Proyecto Android compilable, ejecución y pruebas en un dispositivo Android físico.	TS-001, TS-003, TS-004 y US-004 a US-043.
Aplicación multiplataforma	Flutter y Dart, con el mismo alcance funcional y contratos de servicio que la aplicación Android.	Proyecto Flutter compilable y comparación de los flujos core en las plataformas objetivo definidas por el equipo.	TS-002, TS-003, TS-004, TS-006 y US-004 a US-043.
Almacenamiento local	Room en Android y una base de datos local compatible en Flutter para caché, operaciones	Consulta y registro de información esencial sin	US-029 a US-032 y TS-005 a TS-007.

Aspecto requerido	Decisión para AniTec	Evidencia esperada	Historias relacionadas
	pendientes y estado de sincronización.	conexión, seguidos por una sincronización idempotente.	
Recurso interno del dispositivo	Cámara del teléfono para leer el código QR que identifica al animal, con alternativa de búsqueda manual.	Prueba con permiso concedido, permiso denegado, código válido, código inválido y condiciones reales de iluminación.	US-033, US-034, TS-010 y SP-001.
Servicio REST interno	API REST propia de AniTec, documentada con OpenAPI y protegida mediante autenticación y autorización por rol y relación.	Backend público al 70 % en Sprint 1 y al 100 % desde Sprint 2, con documentación accesible.	US-004 a US-040, TS-004, TS-008 y TS-013.
Servicio externo	Stripe Checkout para iniciar pagos y confirmar el estado de la suscripción sin almacenar datos de tarjeta en AniTec.	Flujos de pago aprobado, rechazado o cancelado y validación del resultado desde el backend.	US-038 a US-040, SP-002 y TS-011.
Feature de aprendizaje autónomo	Investigación, prototipo e integración de Google ML Kit Barcode Scanning para reconocer códigos QR en el dispositivo.	Informe del Spike, fuentes consultadas, alternativas, riesgos, prueba física y conclusión de viabilidad.	US-033, TS-010 y SP-001.
Notificaciones móviles	Recordatorios locales o remotos asociados con actividades ganaderas y sanitarias.	Recepción, atención y reprogramación de recordatorios con permisos habilitados o denegados.	US-025 a US-028 y TS-009.
Despliegue y validación física	Versiones firmadas y distribuidas mediante Firebase App Distribution o un servicio equivalente.	Instalación trazable en dispositivos físicos y registro de versión para la Release Review.	TS-014 y TS-015.

Spike Stories

Story ID	User	Priority	Epic
SP-001	Developer	Must Have	EP-008
Title	Investigar identificación de animales con Google ML Kit		

Description

Como equipo de desarrollo móvil, queremos investigar y prototipar Google ML Kit Barcode Scanning en Android y una alternativa compatible en Flutter para comprender su viabilidad, riesgos y esfuerzo antes de implementar la identificación de animales mediante cámara.

Research Objective

Determinar si la lectura de códigos QR puede funcionar de manera confiable en Android y Flutter, incluso sin conexión, respetando permisos, privacidad y restricciones de dispositivos de gama media utilizados en el contexto peruano.

Timebox Entre 8 y 16 horas dentro del Sprint 2, antes de iniciar TS-010.

Acceptance Criteria

Scenario 1: Investigación documentada

Given el equipo necesita identificar animales mediante códigos QR en Android y Flutter

When el Developer revisa documentación oficial y compara compatibilidad, permisos, operación offline, licencias y privacidad

Then el informe del spike registra fuentes, alternativas, riesgos y criterios de selección

Scenario 2: Prototipo en dispositivo físico

Given existen códigos válidos e inválidos y al menos un dispositivo Android físico de gama media

When el Developer ejecuta el prototipo con distintas condiciones de iluminación y conectividad

Then el prototipo registra resultados de lectura, fallos, tiempos observados y comportamiento sin conexión

Scenario 3: Comparación entre plataformas

Given Android utiliza Kotlin y la estrategia multiplataforma utiliza Flutter

When el Developer evalúa la integración prevista para ambas aplicaciones

Then el informe identifica dependencias, diferencias de implementación y una alternativa manual para los casos no compatibles

Scenario 4: Conclusión técnica

Given la investigación y las pruebas han terminado

When el equipo revisa los hallazgos

Then el informe concluye si la solución es viable, recomienda un enfoque y estima el esfuerzo de implementación

Definition of Done

El informe del spike contiene fuentes, resultados, riesgos, comparación entre plataformas y conclusión técnica; el código del prototipo está registrado en una rama o repositorio identificable; y los hallazgos han sido revisados por el equipo para refinar TS-010 y las historias relacionadas.

Story ID	User	Priority	Epic
SP-002	Developer	Must Have	EP-010

Title Investigar la integración de Stripe para suscripciones móviles

Description

Como equipo de desarrollo móvil y backend, queremos investigar y prototipar la integración de Stripe con Android, Flutter y la API ASP.NET Core de AniTec para comprender el flujo recomendado, los riesgos de seguridad y el esfuerzo requerido antes de implementar pagos y suscripciones.

Research Objective

Determinar si Stripe Checkout en modo de prueba permite iniciar pagos desde ambas aplicaciones móviles, confirmar el resultado en el backend mediante webhooks e impedir cobros o activaciones duplicadas sin almacenar datos de tarjeta en AniTec.

Timebox Entre 8 y 16 horas durante el Sprint 2, antes de iniciar TS-011.

Acceptance Criteria

Scenario 1: Evaluación del flujo y dependencias
Given AniTec necesita procesar suscripciones desde Android y Flutter
When el Developer revisa la documentación oficial y compara Stripe Checkout con un flujo personalizado
Then el informe identifica el enfoque recomendado, dependencias, configuración, costos de referencia y restricciones para las aplicaciones y el backend

Scenario 2: Seguridad y cumplimiento
Given AniTec no debe almacenar datos de tarjeta
When el Developer analiza redirecciones, claves, firmas de webhooks, idempotencia y responsabilidades PCI
Then el informe documenta riesgos, medidas de mitigación y datos que pueden conservarse de forma segura

Scenario 3: Prototipo de pago
Given existe una cuenta de prueba de Stripe y un plan de prueba
When una aplicación móvil solicita una sesión al backend y completa o cancela el checkout
Then el prototipo registra el resultado, valida la firma del webhook y evita procesar dos veces el mismo evento

Scenario 4: Conclusión y estimación
Given la investigación y el prototipo han terminado
When el equipo revisa los resultados
Then el informe presenta ventajas, limitaciones, bloqueadores, enfoque recomendado y una estimación para implementar US-039, US-040 y TS-011

Definition of Done

El prototipo funciona únicamente en modo de prueba y su código está registrado; el informe documenta arquitectura, seguridad, dependencias, costos de referencia, resultados y recomendación; y los hallazgos han sido revisados por el equipo para refinar las historias de pago sin afirmar que Stripe ya está implementado en producción.

2.4.2. Impact Mapping

El Impact Mapping conecta objetivos SMART del piloto con los User Personas, los cambios de comportamiento esperados, los entregables y las historias que los habilitan. Los objetivos se medirán con analítica de uso, registros de sincronización y pruebas moderadas antes de concluir el Sprint 3.

Business Goal	Actor / User Persona	Impact	Deliverable	User Stories
BG-01: Lograr que al menos 8 de 10 ganaderos participantes registren tres animales y dos eventos desde una aplicación móvil durante un piloto de cuatro semanas, antes del cierre del Sprint 3.	Jorge Luis Rivas — Ganadero	Registra la información en el lugar donde ocurre y deja de postergarla por falta de una computadora.	Gestión móvil de fincas y animales.	Como ganadero, deseo registrar y consultar mis animales desde el teléfono para mantener su trazabilidad. US-008 a US-013, US-044 y US-045.
	Jorge Luis Rivas — Ganadero	Continúa trabajando durante interrupciones de red y confirma después la sincronización.	Persistencia local, cola de operaciones y estados de sincronización.	Como ganadero, deseo guardar y sincronizar registros sin repetirlos para trabajar con conectividad intermitente. US-029 a US-032.
BG-02: Conseguir que al menos el 70 % de los recordatorios sanitarios creados durante el piloto se marquen como atendidos o reprogramados dentro de su plazo, antes del cierre del Sprint 3.	Jorge Luis Rivas — Ganadero	Registra incidencias y consulta actividades pendientes con mayor frecuencia.	Historial sanitario, calendario y notificaciones móviles.	Como ganadero, deseo registrar incidencias y recibir recordatorios para cumplir actividades sanitarias. US-014, US-015 y US-025 a US-028.
	Valeria Mendoza — Veterinaria	Programa el control posterior como parte de la atención.	Registro clínico trazable y actividad de seguimiento.	Como veterinaria, deseo registrar tratamientos y próximos controles para mantener continuidad clínica. US-016 a US-019.
BG-03: Lograr que al menos 4 de 5 veterinarios participantes consulten antecedentes y registren una atención autorizada en menos de cinco minutos y sin ayuda, durante la validación previa al cierre del Sprint 3.	Valeria Mendoza — Veterinaria	Consulta únicamente clientes y pacientes autorizados antes de atenderlos.	Gestión de relaciones, clientes, pacientes e historiales.	Como veterinaria, deseo consultar clientes y pacientes autorizados para preparar y documentar la atención. US-023 y US-024.

Business Goal	Actor / User Persona	Impact	Deliverable	User Stories
	Jorge Luis Rivas — Ganadero	Controla quién puede consultar y registrar información sanitaria.	Solicitudes, aceptación y revocación de acceso.	Como ganadero, deseo conceder o revocar acceso veterinario para proteger los datos de mis animales. US-020 a US-022.
BG-04: Conseguir que al menos el 80 % de los participantes identifique correctamente un animal mediante QR o la alternativa manual en menos de 30 segundos, durante las pruebas del Sprint 2.	Jorge Luis Rivas y Valeria Mendoza	Localizan la ficha correcta con menos tiempo y mantienen una alternativa cuando la cámara falla.	Escaneo QR con Google ML Kit y búsqueda manual.	Como usuario autorizado, deseo identificar un animal mediante cámara o búsqueda para acceder rápidamente a su ficha. US-033 y US-034.

2.4.3. Product Backlog

El Product Backlog reúne las User Stories, Technical Stories y Spike Stories del producto, priorizadas según su valor para el negocio. Primero se presentan las historias del core y luego los procesos de soporte. Las historias técnicas y los spikes se ubican antes de las funcionalidades que habilitan. Las estimaciones utilizan la escala 1, 2, 3, 5 u 8.

Durante la planificación de los Sprints 1 y 2 (secciones 4.2.1.1 y 4.2.2.1), el backlog se ajustó según el avance real del producto. La distribución final, detallada en el siguiente cuadro, dejó 24 historias y 114 Story Points en el Sprint 2, y 14 historias y 69 Story Points en el Sprint 3.

Producto mínimo viable (MVP). El MVP comprende la gestión de fincas y animales (BG-01), el seguimiento sanitario (BG-02), la colaboración con veterinarios (BG-03), la identificación de animales (BG-04) y la base técnica necesaria. Debido a los ajustes de planificación, su alcance queda completo al cierre del Sprint 3 (TB2).

La distribución considera los hitos del curso:

- **Sprint 1 — TB1, semana 7:** landing page desplegada, backend al 70 %, base de la aplicación Android y pantallas core de acceso, gestión de fincas, corrales y animales, y registros sanitarios básicos.
- **Sprint 2 — AV2, semana 12:** backend al 100 %, primera versión de la aplicación Flutter (inicio del ganadero) y principales funciones core de sanidad, vista del veterinario, actividades, finanzas, analítica, trabajo offline, identificación mediante cámara, suscripciones con Stripe y distribución en Firebase App Distribution.
- **Sprint 3 — TB2, semana 15:** consentimiento veterinario, notificaciones, cierre de trazabilidad sanitaria y sincronización idempotente, persistencia local en Flutter, accesibilidad, pruebas automatizadas y aplicación completa según el backlog.

# Orden	User Story Id	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
1	US-011	Registrar un animal	5	Sprint 1
2	US-010	Consultar y buscar animales	5	Sprint 1
3	US-013	Consultar el detalle de un animal	3	Sprint 1
4	US-009	Registrar y actualizar una finca	5	Sprint 1
5	US-008	Consultar las fincas registradas	3	Sprint 1
6	US-044	Gestionar los corrales de una finca	3	Sprint 1
7	US-012	Actualizar o archivar un animal	5	Sprint 1
8	US-015	Registrar una incidencia sanitaria	5	Sprint 1
9	US-016	Registrar diagnóstico y tratamiento	5	Sprint 1
10	US-014	Consultar eventos sanitarios	3	Sprint 1
11	US-045	Adjuntar una fotografía al animal	3	Sprint 1
12	US-001	Comprender la propuesta de valor de AniTec	3	Sprint 1
13	US-002	Conocer las soluciones para cada segmento	3	Sprint 1
14	TS-013	Adaptar y documentar los servicios backend para móviles	8	Sprint 1
15	TS-004	Integrar las aplicaciones con la API REST interna	5	Sprint 1
16	TS-003	Definir la arquitectura móvil por capas y bounded contexts	5	Sprint 1
17	TS-001	Configurar la aplicación Android nativa	5	Sprint 1

# Orden	User Story Id	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
18	TS-005	Implementar persistencia local segura en Android	5	Sprint 1
19	US-004	Registrar una cuenta según el rol	5	Sprint 1
20	US-005	Iniciar sesión	3	Sprint 1
21	US-007	Acceder únicamente a información autorizada	5	Sprint 1
22	US-006	Mantener y finalizar la sesión móvil	3	Sprint 1
23	TS-008	Proteger credenciales y datos de sesión	5	Sprint 1
24	US-003	Acceder a una landing page adaptable e internacionalizada	5	Sprint 1
25	US-017	Consultar el historial sanitario de un animal	5	Sprint 2
26	US-024	Consultar antecedentes de un paciente autorizado	3	Sprint 2
27	US-023	Consultar clientes y pacientes autorizados	5	Sprint 2
28	US-026	Gestionar una actividad o recordatorio	5	Sprint 2
29	US-025	Consultar actividades programadas	3	Sprint 2
30	US-037	Registrar y consultar movimientos financieros	5	Sprint 2
31	US-035	Consultar indicadores del hato	5	Sprint 2
32	US-036	Consultar indicadores sanitarios de clientes	5	Sprint 2
33	TS-002	Configurar la aplicación multiplataforma con Flutter	5	Sprint 2
34	US-029	Consultar información esencial sin conexión	5	Sprint 2

# Orden	User Story Id	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
35	US-030	Guardar trabajo pendiente sin conexión	5	Sprint 2
36	US-031	Sincronizar operaciones pendientes	8	Sprint 2
37	SP-001	Investigar identificación de animales con Google ML Kit	5	Sprint 2
38	TS-010	Integrar identificación QR mediante Google ML Kit	5	Sprint 2
39	US-033	Identificar un animal mediante código QR	5	Sprint 2
40	US-034	Identificar un animal sin utilizar la cámara	3	Sprint 2
41	SP-002	Investigar la integración de Stripe para suscripciones móviles	5	Sprint 2
42	US-038	Consultar planes de suscripción	3	Sprint 2
43	TS-011	Integrar el checkout externo de Stripe	8	Sprint 2
44	US-039	Iniciar un pago mediante un proveedor externo	5	Sprint 2
45	US-040	Consultar el resultado del pago y la suscripción	5	Sprint 2
46	US-041	Cambiar el idioma de la aplicación	3	Sprint 2
47	US-043	Comprender errores y estados de conectividad	3	Sprint 2
48	TS-015	Configurar compilación y distribución de versiones móviles	5	Sprint 2
49	US-020	Recibir una solicitud de seguimiento veterinario	3	Sprint 3
50	US-021	Aceptar o rechazar acceso veterinario	3	Sprint 3
51	US-022	Revocar el acceso de un veterinario	3	Sprint 3

# Orden	User Story Id	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
52	TS-009	Implementar notificaciones móviles	5	Sprint 3
53	US-027	Recibir una notificación de actividad	5	Sprint 3
54	US-019	Programar un control sanitario posterior	3	Sprint 3
55	US-028	Atender o reprogramar una actividad	3	Sprint 3
56	TS-006	Implementar persistencia local en Flutter	5	Sprint 3
57	TS-007	Implementar sincronización idempotente	8	Sprint 3
58	US-032	Resolver errores o conflictos de sincronización	8	Sprint 3
59	US-018	Corregir un registro sanitario con trazabilidad	5	Sprint 3
60	TS-012	Aplicar internacionalización y accesibilidad móvil	5	Sprint 3
61	US-042	Utilizar la aplicación con necesidades de accesibilidad	5	Sprint 3
62	TS-014	Automatizar pruebas de los flujos móviles críticos	8	Sprint 3

2.5. Strategic-Level Domain-Driven Design

El diseño estratégico de AniTec utiliza Domain-Driven Design para separar el dominio ganadero en modelos con responsabilidades, reglas y lenguaje propios. El proceso parte del Big Picture EventStorming de la sección 2.3.5, lo contrasta con los User Personas, entrevistas y requisitos de la sección 2.4, y concluye con bounded contexts y relaciones que pueden mantenerse de forma coherente en el backend y en las aplicaciones móviles.

La revisión recupera la evidencia del trabajo anterior —especialmente los pasos 4 al 10 del EventStorming, los canvases de Livestock, Sanitary, Financial y Analytics, y la estructura modular del backend— y la adapta al alcance actual: una aplicación Android nativa, una aplicación multiplataforma, operación offline, identificación QR, colaboración veterinaria, notificaciones y pagos mediante Stripe.

2.5.1. EventStorming

El equipo utilizó Miro para construir una narrativa compartida del dominio de AniTec. La sesión identificó actores, comandos, eventos de dominio, políticas, read models, sistemas externos, agregados, puntos problemáticos y eventos pivote. El análisis se organizó en bloques de trabajo acotados para evitar que la exploración se convirtiera en un inventario de funcionalidades sin secuencia de negocio.

Dato de la sesión	**Registro**
Herramienta	Miro
Participantes	Equipo de AniTec
Fecha real	**Placeholder:** completar con la fecha de la sesión registrada por el equipo.
Duración real	**Placeholder:** completar con la duración de la sesión; el time-box recomendado es de una a dos horas.
Entradas	Entrevistas, User Personas, Needfinding, Ubiquitous Language y Product Backlog móvil.
Resultados	Flujo actualizado del dominio, eventos pivote, nueve bounded contexts candidatos, Domain Stories y dependencias.

Convención visual

Elemento	**Color utilizado**	**Propósito**
Domain Event	Naranja	Hecho relevante del negocio redactado en pasado.
Command	Azul	Intención que puede producir un evento.
Actor	Amarillo	Persona o rol que ejecuta una acción.
Policy	Morado	Regla que reacciona ante un evento y puede generar otro comando.
Read Model	Verde	Información que un actor consulta antes de tomar una decisión.
External System	Rosado	Sistema ajeno a AniTec que participa en el flujo.

Elemento	Color utilizado	Propósito
Hotspot	Rojo o rosado intenso	Duda, riesgo, conflicto o decisión todavía abierta.
Aggregate	Amarillo amplio	Frontera de consistencia que recibe comandos y produce eventos.

Proceso de exploración y refinamiento

Paso 1 — Unstructured Exploration. Los participantes registraron hechos relevantes sin imponer inicialmente una estructura. Se identificaron eventos relacionados con autenticación, fincas, animales, sanidad, finanzas, analítica, actividades y suscripciones.

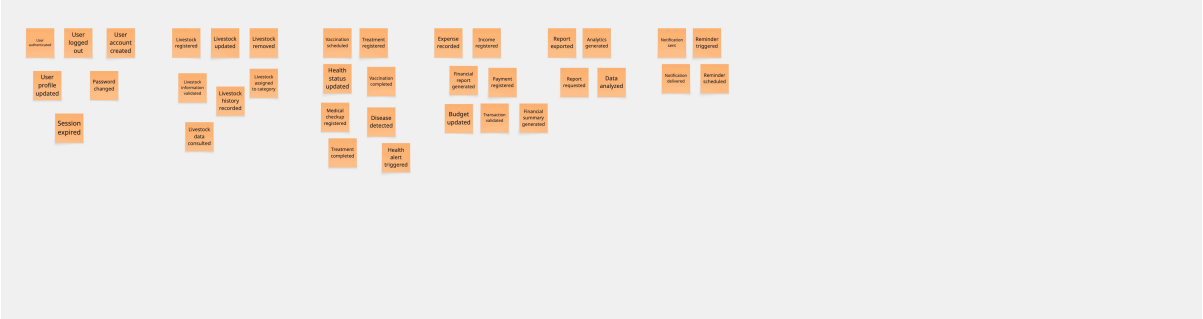


Figura 2.5.1. Exploración no estructurada del dominio. Fuente: elaboración propia.

Paso 2 — Timeline. Los eventos fueron ordenados para formar secuencias de negocio. El orden permitió distinguir el acceso al sistema, la gestión del hato, la atención sanitaria, el trabajo de campo y la obtención de indicadores.

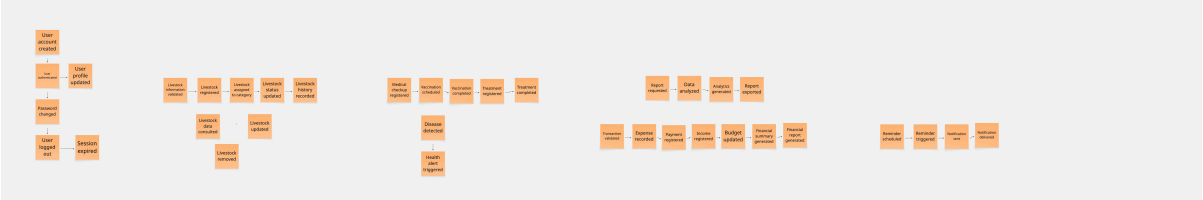


Figura 2.5.2. Línea de tiempo inicial. Fuente: elaboración propia.

Paso 3 — Pain Points. Se marcaron problemas como credenciales inválidas, animales duplicados, detección tardía de enfermedades, registros financieros incompletos y fallos de notificación. Para el alcance móvil se añadieron conectividad intermitente, conflictos de sincronización, cámara no disponible, QR ilegible, autorización veterinaria vencida y pago no confirmado.

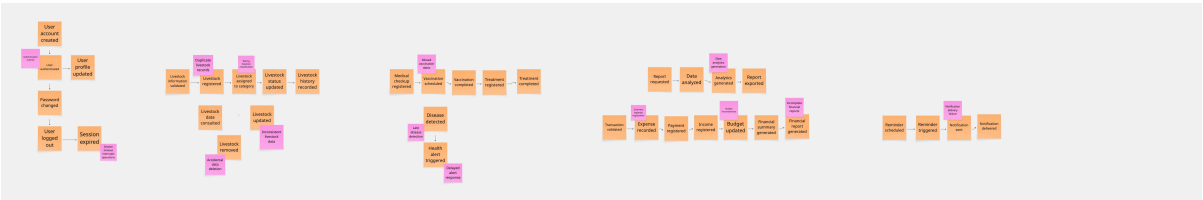


Figura 2.5.3. Pain points identificados. Fuente: elaboración propia.

Paso 4 — Pivotal Events. Se buscaron hechos que cambian el estado o la responsabilidad del proceso. Los principales eventos pivote son UserRegistered, AnimalRegistered, AnimalIdentified, VeterinaryAccessGranted,

HealthEventRecorded, DataSynchronized, PaymentConfirmed y SubscriptionActivated. Estos eventos ayudaron a proponer límites entre modelos.

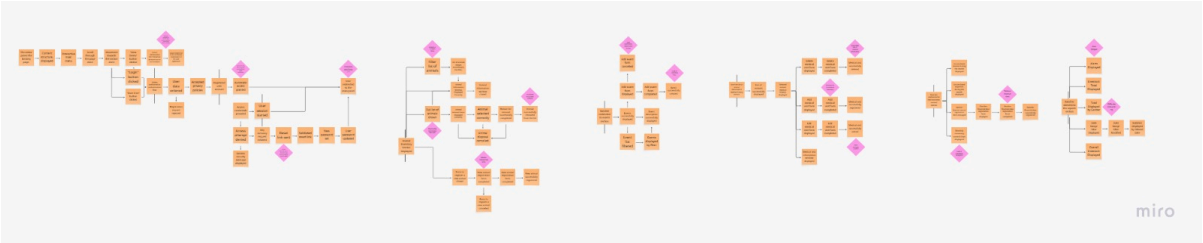


Figura 2.5.4. Identificación de eventos pivote. Fuente: elaboración propia.

Paso 5 — Commands and Actors. Cada evento se relacionó con el comando que lo origina y con el actor correspondiente. Por ejemplo, el ganadero ejecuta RegisterAnimal, mientras que el veterinario ejecuta RecordVeterinaryCare únicamente cuando existe una autorización vigente.

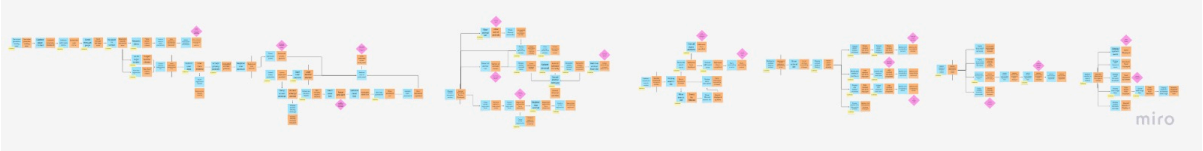


Figura 2.5.5. Incorporación de comandos y actores. Fuente: elaboración propia.

Paso 6 — Policies. Se representaron reglas reactivas. Cuando una atención requiere seguimiento, se solicita programar una actividad; cuando la conexión vuelve, se solicita sincronizar operaciones pendientes; cuando Stripe confirma un pago válido, se solicita activar la suscripción.

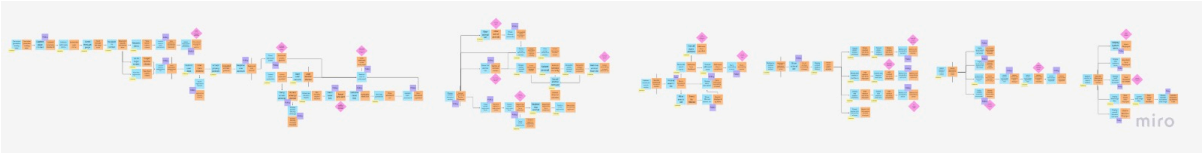


Figura 2.5.6. Políticas y automatizaciones. Fuente: elaboración propia.

Paso 7 — Read Models. Se incorporó la información necesaria para decidir: lista de fincas, ficha del animal, historial sanitario, clientes autorizados, actividades pendientes, resumen financiero, indicadores y estado de suscripción. En la aplicación móvil, cada read model debe indicar si procede del servidor o de una copia local y cuándo fue actualizado.

Figura 2.5.7. Read models del dominio. Fuente: elaboración propia.

Paso 8 — External Systems. Stripe participa en el checkout y la confirmación de pagos. El proveedor de notificaciones entrega recordatorios cuando se utiliza notificación remota. Google ML Kit se trata como una dependencia técnica ejecutada en el dispositivo para decodificar el QR y no como parte del modelo de negocio.

Figura 2.5.8. Sistemas externos. Fuente: elaboración propia.

Paso 9 — Aggregates. Los comandos y eventos se agruparon alrededor de agregados con consistencia propia: User, Profile, Farm, Animal, HealthHistory, VeterinaryAuthorization, FarmActivity, FinancialRecord, Subscription y las proyecciones de analítica.

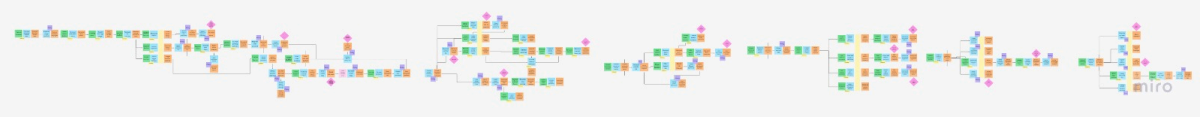


Figura 2.5.9. Agregados candidatos. Fuente: elaboración propia.

La siguiente síntesis actualiza el resultado anterior con las capacidades móviles del Product Backlog. Su propósito es mantener visibles los cambios de estado del negocio sin convertir Room, Flutter, Android, sincronización o ML Kit en conceptos del dominio.

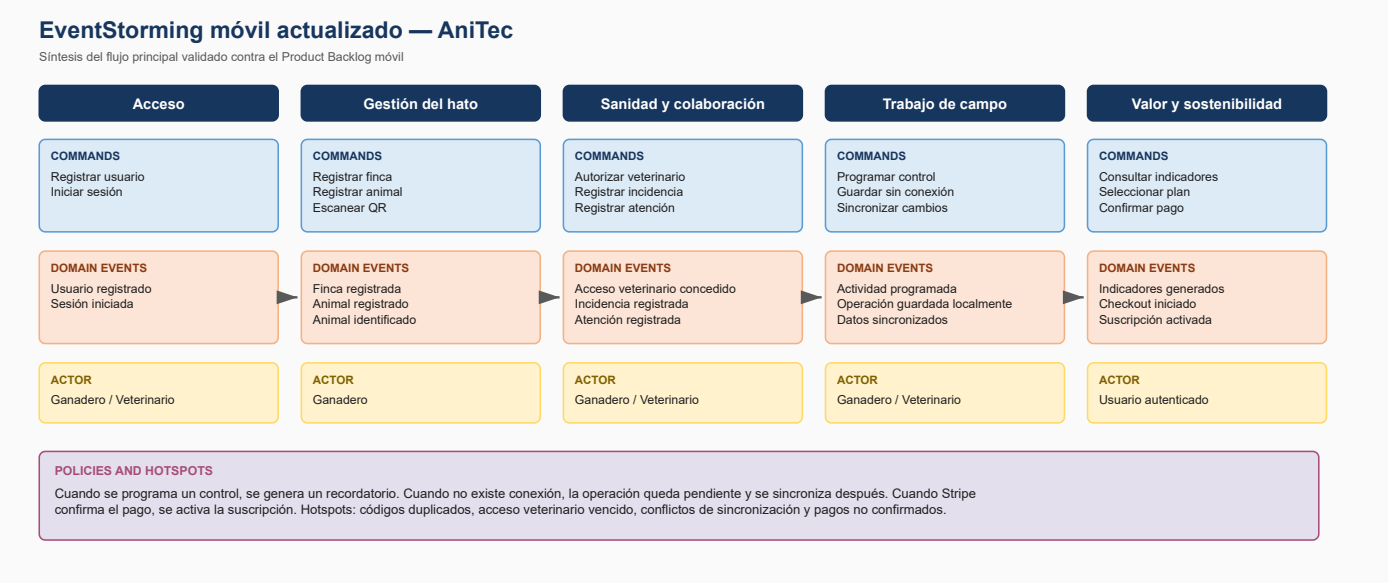


Figura 2.5.10. Síntesis actualizada del EventStorming móvil. Fuente: elaboración propia.

Enlace del tablero de trabajo: https://tinyurl.com/EventSorming

El equipo debe actualizar en ese tablero las notas correspondientes a QR, autorización veterinaria, offline, sincronización, Stripe y aplicaciones móviles para que la evidencia pública coincida con esta versión del informe.

Decisiones derivadas del EventStorming

- La gestión del hato, la sanidad y la colaboración veterinaria concentran el valor diferencial y se clasifican como Core Domain.
- La autorización veterinaria posee reglas y ciclo de vida propios, por lo que no se modela como un simple atributo del perfil.

- La suscripción y los movimientos financieros representan conceptos distintos: una suscripción controla el acceso a un plan; un movimiento financiero describe la operación económica del ganadero.
- La analítica consume información de otros contextos mediante proyecciones y no modifica sus agregados.
- La sincronización offline pertenece a la infraestructura móvil. El contexto propietario mantiene las reglas para aceptar, rechazar o detectar conflictos.
- La lectura QR es una forma de identificar un animal; el dominio continúa trabajando con AnimalId y AnimalCode aunque cambie la tecnología de escaneo.

2.5.1.1. Candidate Context Discovery

El Candidate Context Discovery aplicó dos técnicas complementarias. **Start-with-Value** permitió separar las capacidades que generan el valor principal para ganaderos y veterinarios. **Look-for-Pivotal-Events** permitió ubicar cambios de estado y de responsabilidad en la línea de tiempo. Después se contrastaron los candidatos con el Ubiquitous Language, el Product Backlog y los módulos existentes del backend.

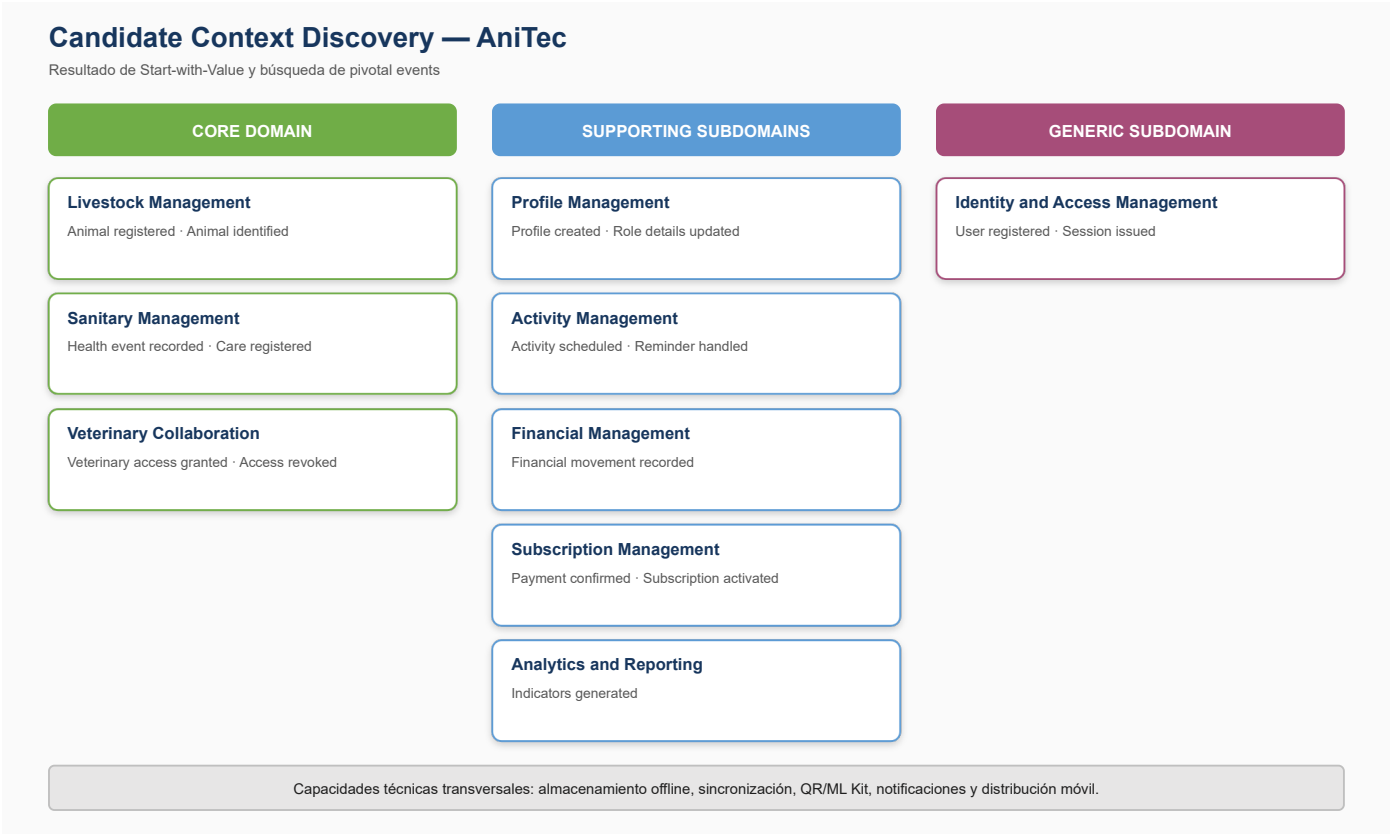


Figura 2.5.11. Clasificación de los bounded contexts candidatos. Fuente: elaboración propia.

Bounded Context	Clasificación	Responsabilidad y eventos pivote	Correspondencia técnica actual
Identity and Access Management	Generic	Identidad, credenciales, sesiones y roles. UserRegistered, SessionIssued.	Iam
Profile Management	Supporting	Datos personales y profesionales. ProfileCreated, ProfileUpdated.	Profiles

Bounded Context	Clasificación	Responsabilidad y eventos pivote	Correspondencia técnica actual
Livestock Management	Core	Fincas, hatos, animales e identificación. AnimalRegistered, AnimalIdentified.	Livestock
Sanitary Management	Core	Incidencias, diagnósticos, tratamientos e historial. HealthEventRecorded, VeterinaryCareRecorded.	Sanitary
Veterinary Collaboration	Core	Solicitudes y autorizaciones entre ganaderos y veterinarios. VeterinaryAccessGranted, VeterinaryAccessRevoked.	Clients, que deberá adoptar el lenguaje de colaboración.
Activity Management	Supporting	Actividades, controles y recordatorios. ActivityScheduled, ReminderDue.	Activities
Financial Management	Supporting	Ingresos, egresos y resumen financiero. FinancialMovementRecorded.	Financial
Subscription Management	Supporting	Planes, checkout, pago confirmado y suscripción. PaymentConfirmed, SubscriptionActivated.	Subscriptions
Analytics and Reporting	Supporting	Indicadores y read models para cada rol. IndicatorsGenerated.	Analytics

Decisiones sobre los límites

Elemento analizado	Decisión	Justificación
Offline Storage and Synchronization	No constituye bounded context.	Es una capacidad técnica de Android y Flutter. Cada operación sigue perteneciendo al contexto que define sus reglas.
Animal Identification con QR	Capability de Livestock Management.	ML Kit traduce una imagen a un código; Livestock resuelve el animal y verifica autorización.
Notifications	Capability técnica de Activity Management.	Activities decide cuándo existe un recordatorio; el proveedor solo lo entrega.
Landing Page	Producto informativo, no bounded context.	No contiene reglas relevantes del dominio ganadero.
Shared	Infraestructura compartida, no subdominio.	Repositorio base, Unit of Work y manejo de errores son mecanismos técnicos.
Devices y Metrics	Fuera del alcance actual.	Pertenecen a la propuesta IoT anterior y no están respaldados por el Product Backlog móvil vigente.

El resultado corrige la diferencia existente entre la lista anterior de cinco contextos, los cuatro canvases históricos y los contextos adicionales utilizados en los Domain Stories. Los nueve contextos seleccionados serán la referencia única para Context Mapping, Software Architecture y Tactical-Level DDD.

2.5.1.2. Domain Message Flows Modeling

Los Domain Message Flows aplican Domain Storytelling para mostrar cómo los actores, aplicaciones móviles y bounded contexts colaboran en los escenarios principales. Cada historia identifica quién inicia el flujo, qué comando se transmite, quién aplica las reglas y qué evento informa el resultado. Los términos Android, Flutter, base local y ML Kit aparecen únicamente cuando explican el transporte o la interacción con el dispositivo.

Domain Story	Objetivo	Requisitos relacionados
DS-01	Registrar un animal con conectividad disponible o interrumpida.	US-008 a US-013, US-029 a US-031, TS-005 a TS-007.
DS-02	Identificar un animal mediante QR y conservar búsqueda manual.	US-033, US-034, TS-010 y SP-001.
DS-03	Registrar una incidencia y programar seguimiento.	US-014 a US-019 y US-025 a US-028.
DS-04	Autorizar al veterinario y registrar una atención trazable.	US-016 a US-024.
DS-05	Sincronizar operaciones idempotentes y comunicar conflictos.	US-029 a US-032 y TS-007.
DS-06	Procesar una suscripción mediante Stripe.	US-038 a US-040 y TS-011.

DS-01. Registro móvil de un animal

El ganadero registra un animal en la finca seleccionada. La aplicación guarda un comando identificado de forma única. Con conexión, el comando se envía inmediatamente; sin conexión, permanece pendiente. Livestock Management valida la propiedad de la finca y la unicidad del código antes de producir AnimalRegistered.

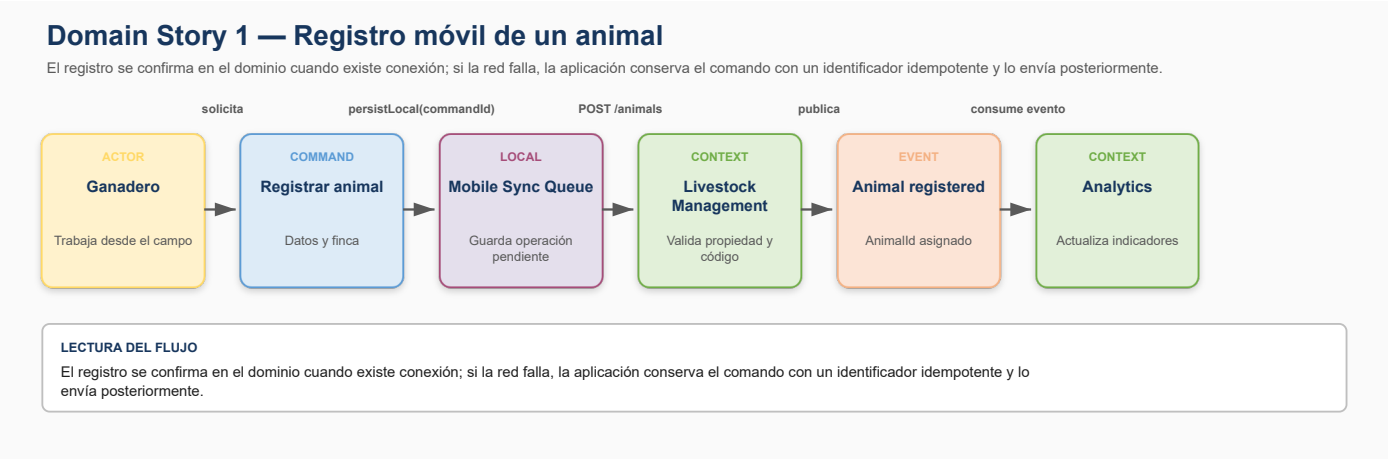


Figura 2.5.12. Domain Story del registro móvil de un animal. Fuente: elaboración propia.

DS-02. Identificación mediante QR

La cámara proporciona la imagen y ML Kit decodifica el QR dentro del dispositivo. El código resultante se envía a Livestock Management, que resuelve AnimalId y comprueba que el usuario pueda consultar la ficha. Si no se obtiene

un código válido, la búsqueda manual mantiene el resultado de negocio disponible.

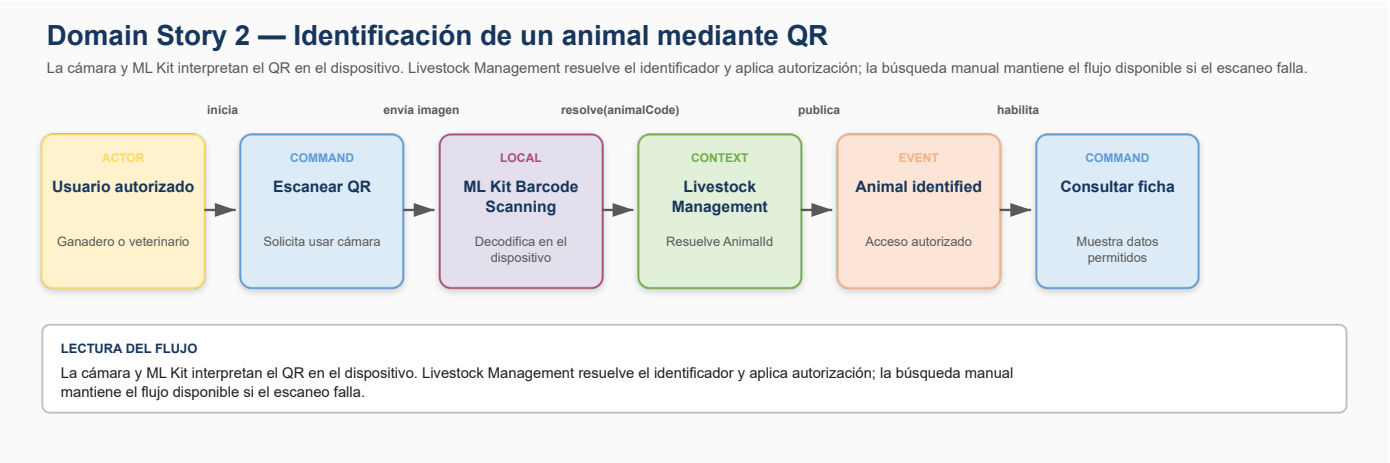


Figura 2.5.13. Domain Story de identificación mediante QR. Fuente: elaboración propia.

DS-03. Incidencia y seguimiento sanitario

Sanitary Management incorpora la incidencia al historial del animal. Cuando el evento requiere seguimiento, publica FollowUpRequired; Activity Management lo transforma en una actividad con fecha y produce ReminderScheduled para que el mecanismo móvil correspondiente programe la notificación.

Figura 2.5.14. Domain Story de incidencia y seguimiento sanitario. Fuente: elaboración propia.

DS-04. Autorización y atención veterinaria

Veterinary Collaboration administra la relación entre el ganadero y el veterinario. Sanitary Management consulta ese contrato antes de exponer antecedentes o registrar una atención. El evento final conserva la identidad profesional para asegurar trazabilidad.

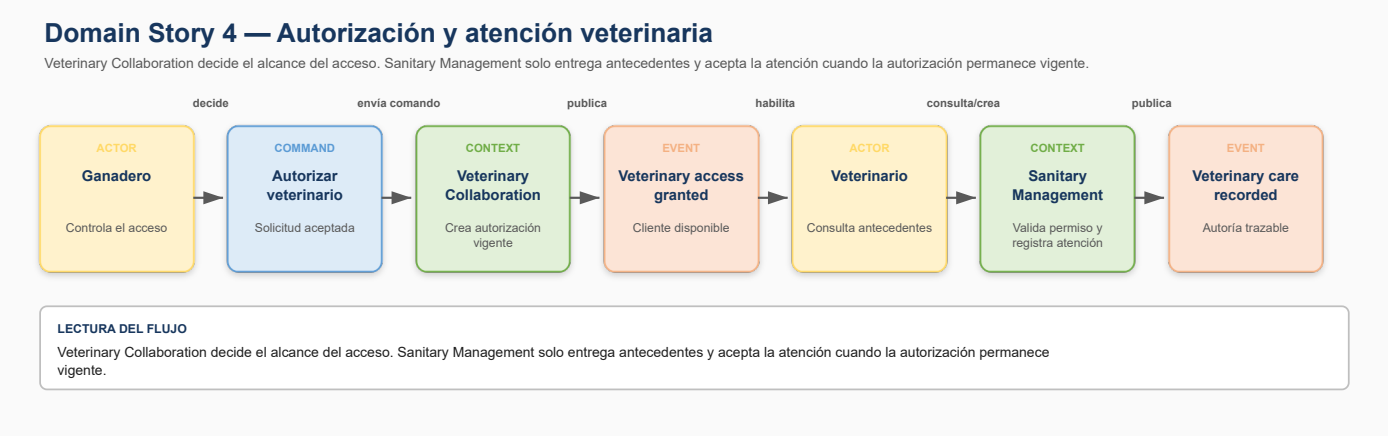


Figura 2.5.15. Domain Story de autorización y atención veterinaria. Fuente: elaboración propia.

DS-05. Sincronización y resolución de conflictos

La base local y el Sync Worker pertenecen a la infraestructura móvil. El comando conserva CommandId, versión y fecha. El contexto propietario comprueba idempotencia y concurrencia y responde con Synced, Rejected o Conflict; la aplicación muestra el estado sin duplicar la operación.

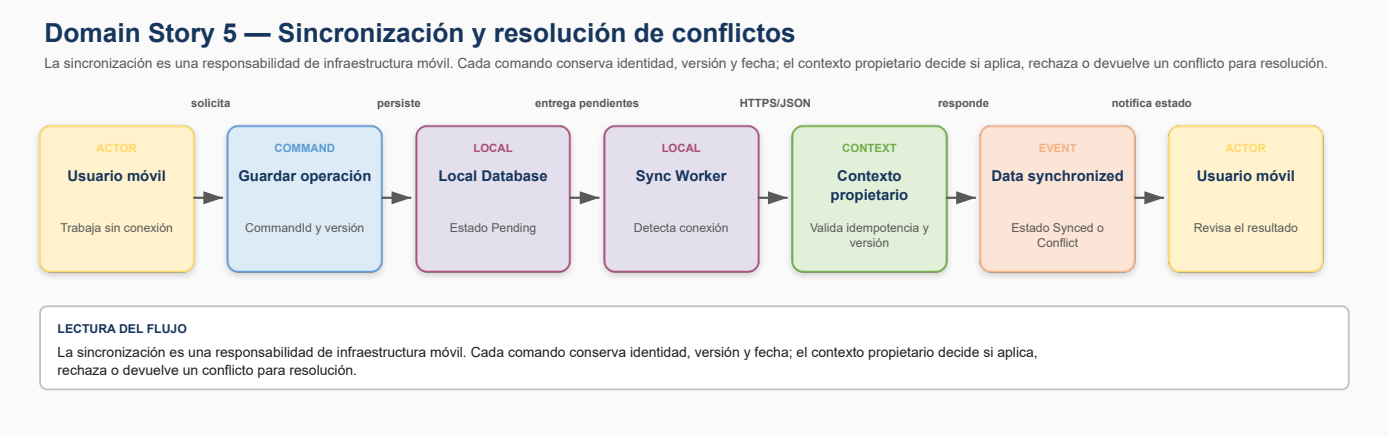


Figura 2.5.16. Domain Story de sincronización y resolución de conflictos. Fuente: elaboración propia.

DS-06. Suscripción mediante Stripe

Subscription Management crea el checkout para el plan seleccionado. La Anti-Corruption Layer transforma el modelo interno al contrato de Stripe y valida la respuesta. La suscripción solo cambia a activa después de que el backend comprueba el pago; una redirección del cliente por sí sola no confirma la operación.

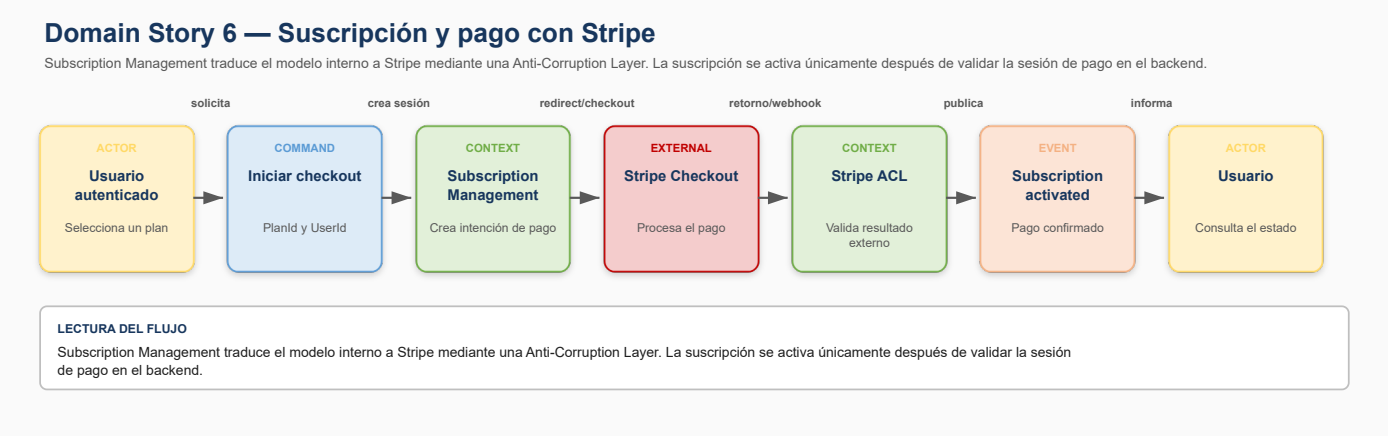


Figura 2.5.17. Domain Story de suscripción mediante Stripe. Fuente: elaboración propia.

Tipos de colaboración identificados

- Las consultas directas desde Android y Flutter utilizan JSON sobre HTTPS mediante la API REST.
- Los eventos como AnimalRegistered, HealthEventRecorded o FollowUpRequired representan mensajes del dominio y permiten desacoplar consumidores.
- Los comandos offline se procesan con idempotencia para que una retransmisión no genere duplicados.
- Analytics utiliza read models y no consulta ni modifica directamente los agregados de otros contextos.
- Stripe y el proveedor de notificaciones se aíslan mediante Anti-Corruption Layers.

2.5.1.3. Bounded Context Canvases

Los canvases de Livestock Management, Animal Health, Financial Control y Analytics del trabajo anterior se utilizaron como punto de partida. La revisión actual normaliza sus nombres, actualiza reglas según el Product Backlog y agrega los contextos ausentes. Cada canvas se elaboró siguiendo este ciclo:

1. **Context Overview Definition:** propósito y clasificación estratégica.
2. **Business Rules Distillation and Ubiquitous Language Capture:** reglas y términos exclusivos.
3. **Capability Analysis:** comandos, consultas y eventos principales.
4. **Capability Layering:** distinción entre dominio, soporte e infraestructura.
5. **Dependencies Capture:** comunicaciones entrantes, salientes y contextos relacionados.
6. **Design Critique:** supuestos, métricas, preguntas abiertas y revisión de límites.

Bounded Context	Resultado de la revisión
IAM	Se separaron credenciales y sesiones de los datos del perfil.
Profiles	Se delimitó la identidad descriptiva de ganaderos y veterinarios.
Livestock	Se incorporaron finca, identificación QR y alternativa manual.
Sanitary	Se añadieron autorización, autoría y correcciones trazables.
Veterinary Collaboration	Se formalizó como modelo con ciclo de vida propio.
Activities	Se relacionaron controles, estados y recordatorios.
Financial	Se separó la operación financiera de los pagos de suscripción.

Bounded Context	Resultado de la revisión
Subscriptions	Se reemplazó el pago simulado por Stripe y una ACL.
Analytics	Se limitó a indicadores respaldados por historias vigentes.

Capability Analysis and Layering

La revisión separa capacidades del dominio de mecanismos de aplicación e infraestructura. Esta separación evita crear bounded contexts para cada librería, pantalla o recurso del dispositivo.

Bounded Context	Capacidades del dominio	Capacidades de aplicación o infraestructura
IAM	Registro, autenticación, sesión y autorización.	JWT, BCrypt y almacenamiento seguro del token.
Profiles	Creación y actualización de datos personales y profesionales.	Validación, localización y exposición REST del perfil.
Livestock	Fincas, animales, códigos e identificación.	Cámara, ML Kit, caché local y adaptadores REST.
Sanitary	Historial, incidencia, atención, tratamiento y corrección.	Persistencia local, sincronización y control de versiones.
Veterinary Collaboration	Solicitud, concesión, alcance y revocación del acceso.	Consulta REST y aplicación de políticas de autorización.
Activities	Programación, atención y reprogramación.	Scheduler y adaptador del proveedor de notificaciones.
Financial	Ingresos, egresos, categorías y balance.	Caché local y exposición REST.
Subscriptions	Planes, estado de suscripción y confirmación de pago.	Stripe Checkout y Anti-Corruption Layer.
Analytics	Definición de indicadores por rol y periodo.	Proyecciones, agregación y caché de read models.

Identity and Access Management

Gestiona cuentas, credenciales, sesiones y autorización. Publica un identificador estable y el rol; no administra los datos personales del perfil.

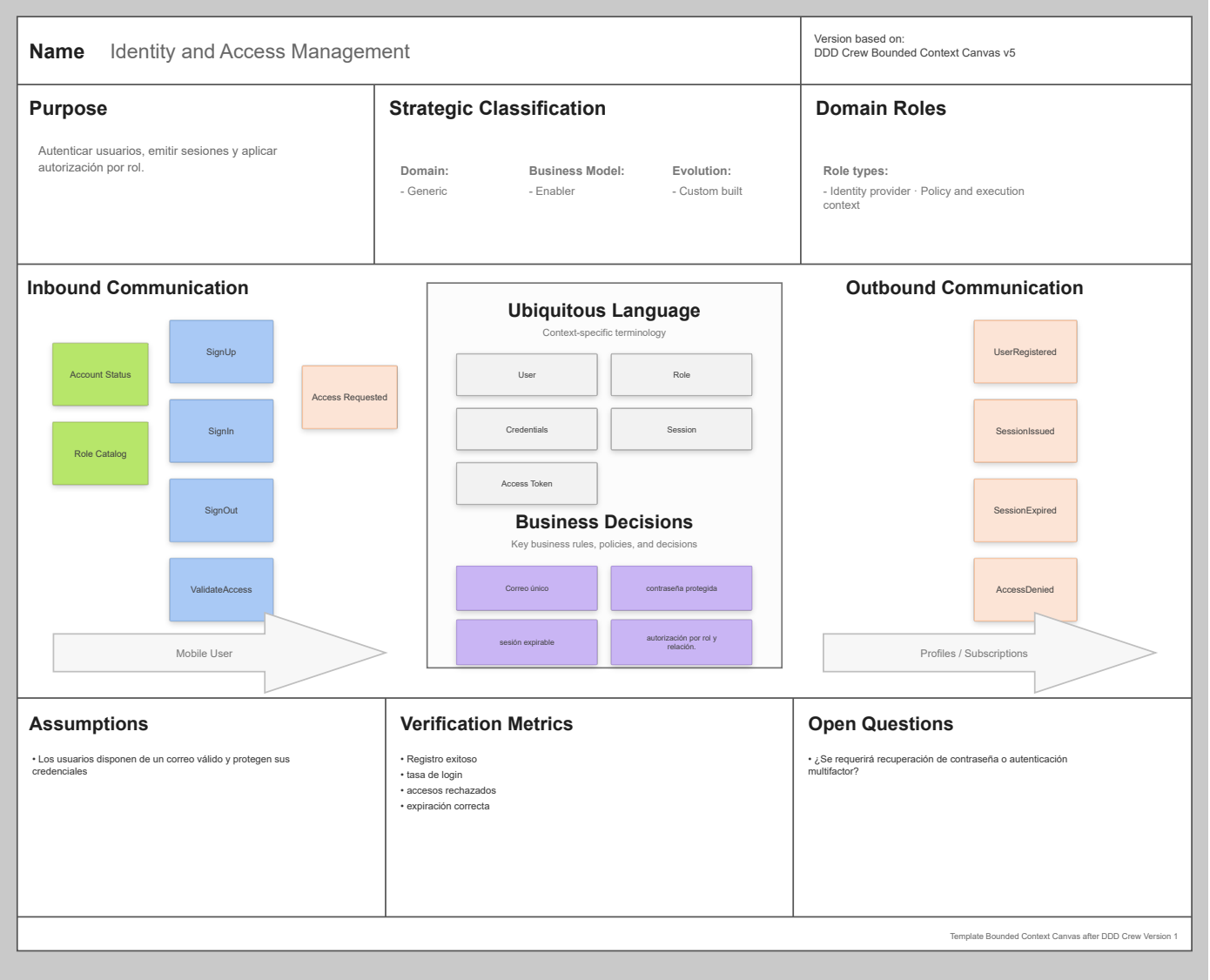


Figura 2.5.18. Bounded Context Canvas de IAM. Fuente: elaboración propia.

Profile Management

Mantiene la información personal y profesional asociada con la identidad. Su separación evita que los cambios del perfil alteren el modelo de autenticación.

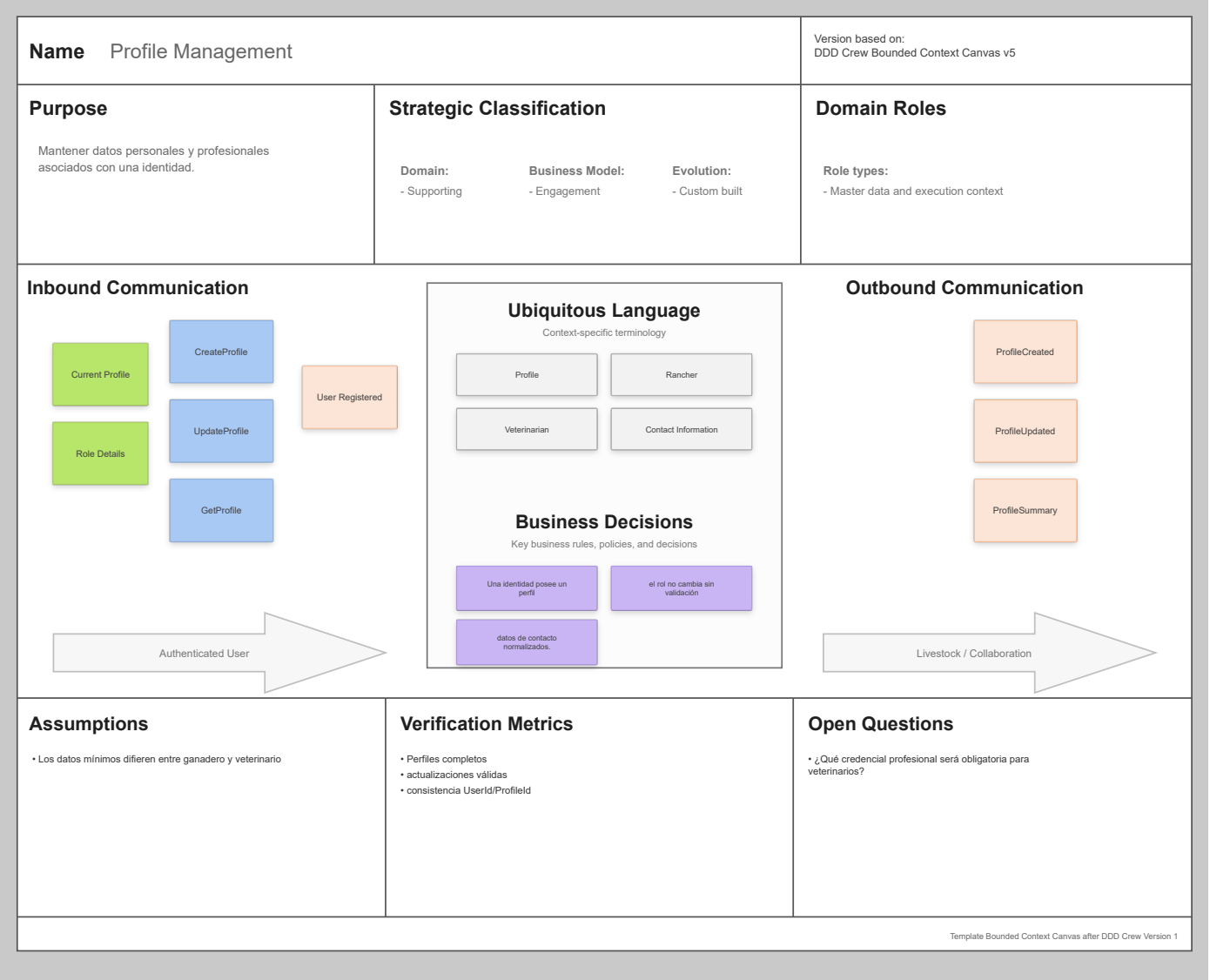


Figura 2.5.19. Bounded Context Canvas de Profile Management. Fuente: elaboración propia.

Livestock Management

Concentra el registro de fincas, hatos y animales. El QR es un medio de identificación y no sustituye el identificador estable del animal.

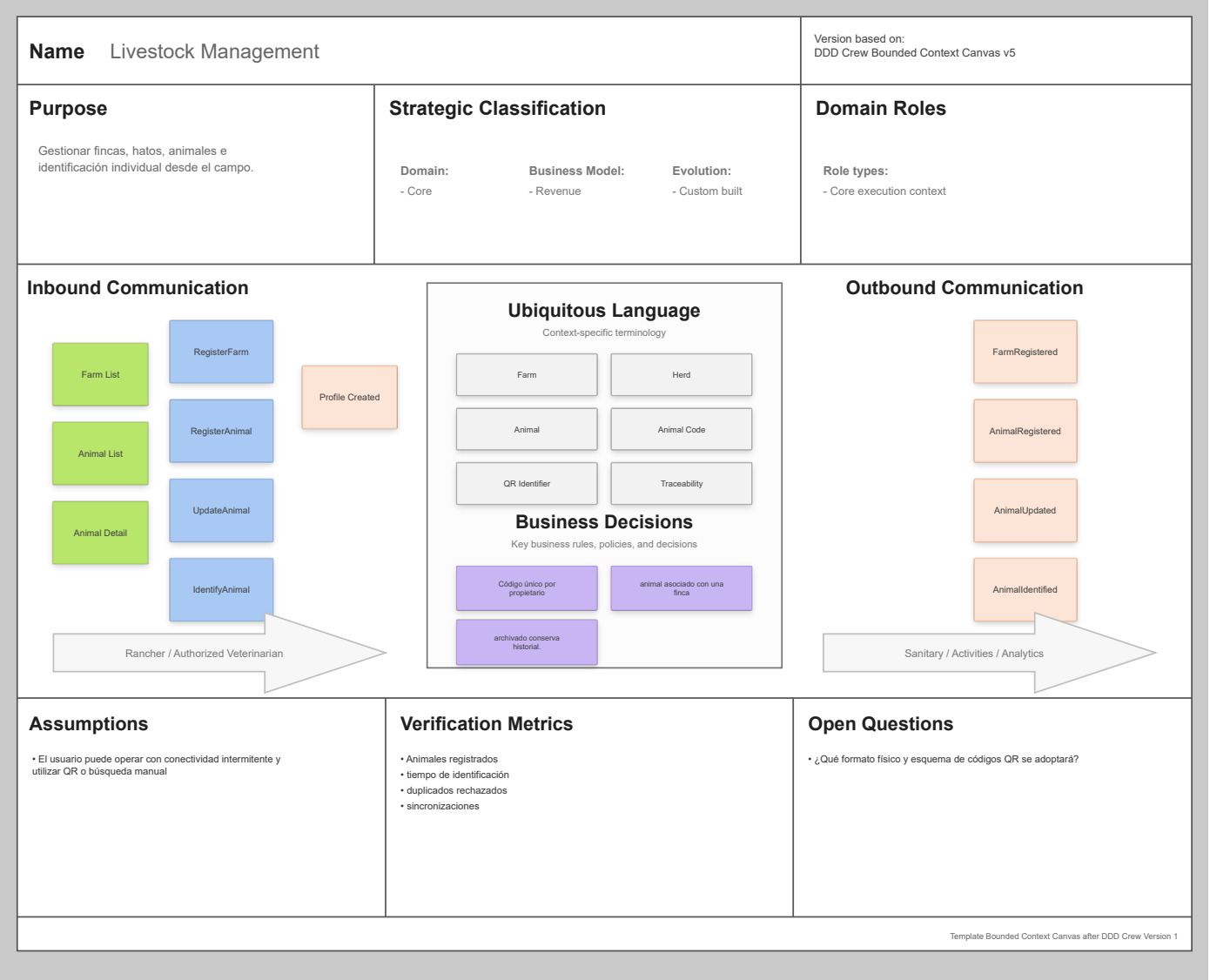


Figura 2.5.20. Bounded Context Canvas de Livestock Management. Fuente: elaboración propia.

Sanitary Management

Mantiene el historial sanitario, las atenciones y la trazabilidad clínica. Depende de Livestock para reconocer el animal y de Veterinary Collaboration para validar al profesional.

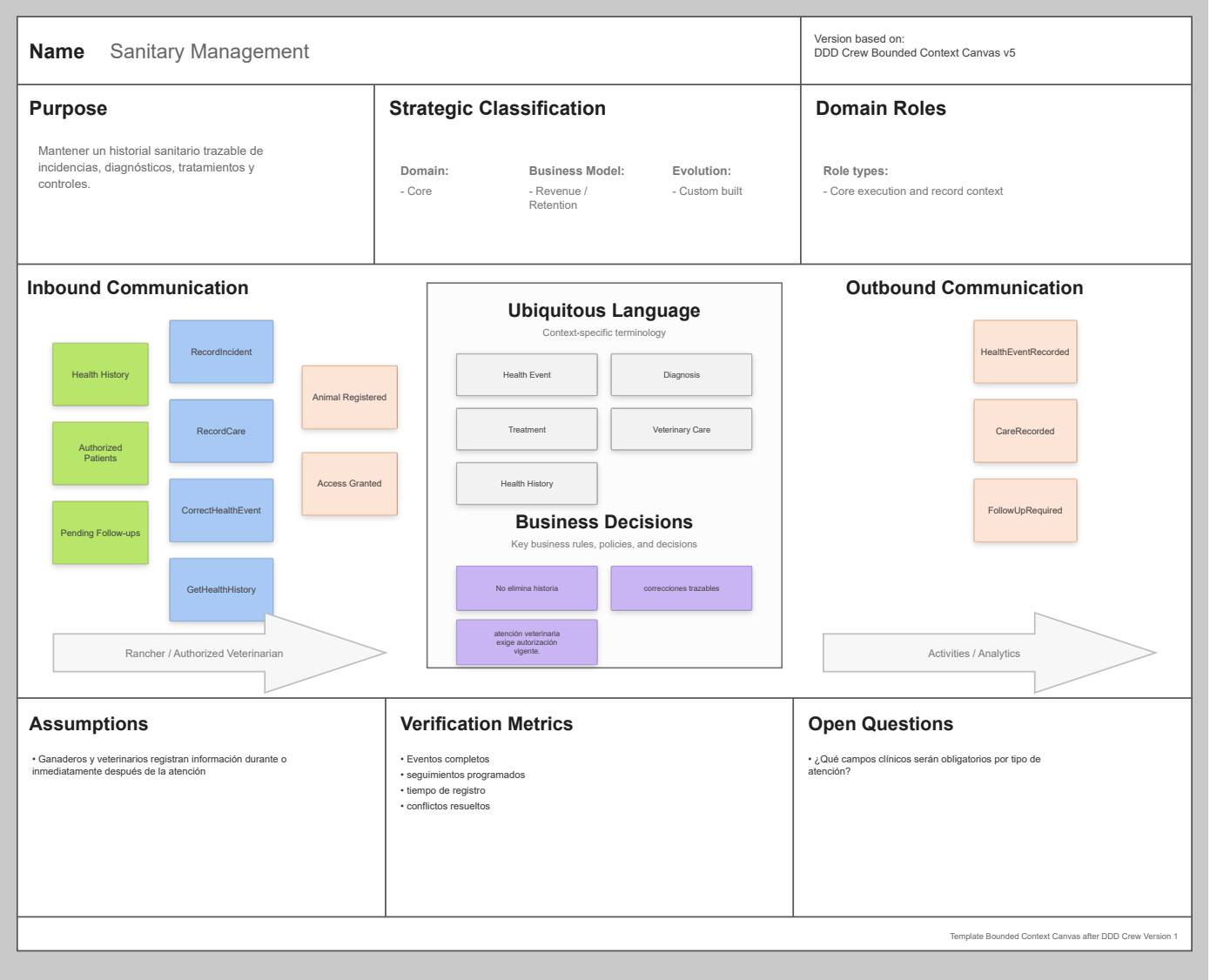


Figura 2.5.21. Bounded Context Canvas de Sanitary Management. Fuente: elaboración propia.

Veterinary Collaboration

Define solicitudes, autorizaciones, revocaciones, clientes y pacientes. La autorización es temporal y delimita qué información puede consultar o modificar el veterinario.

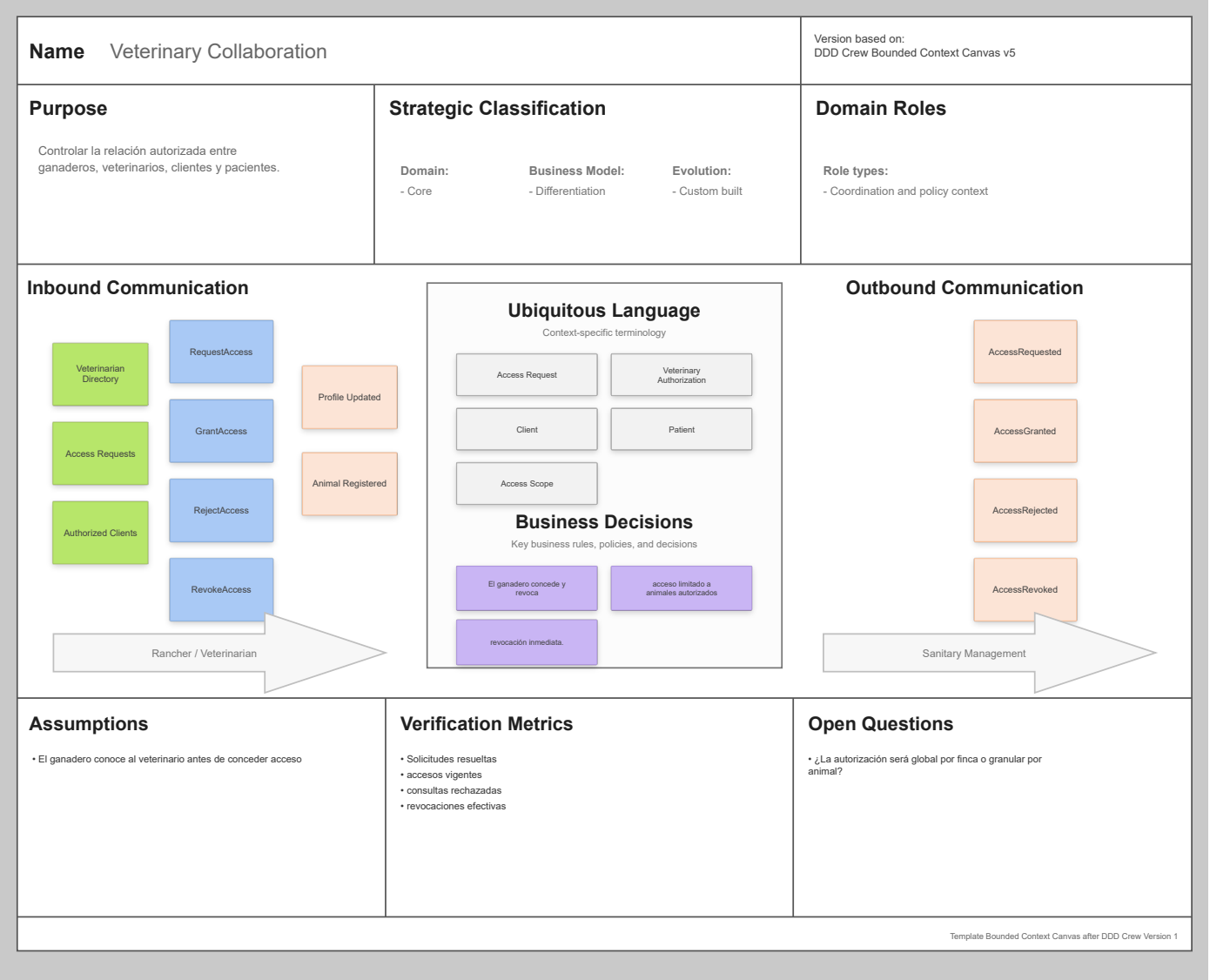


Figura 2.5.22. Bounded Context Canvas de Veterinary Collaboration. Fuente: elaboración propia.

Activity Management

Administra actividades, fechas, responsables, estados y reprogramaciones. El mecanismo de notificación ejecuta la entrega, mientras Activity Management conserva la decisión de negocio sobre cuándo corresponde recordar.

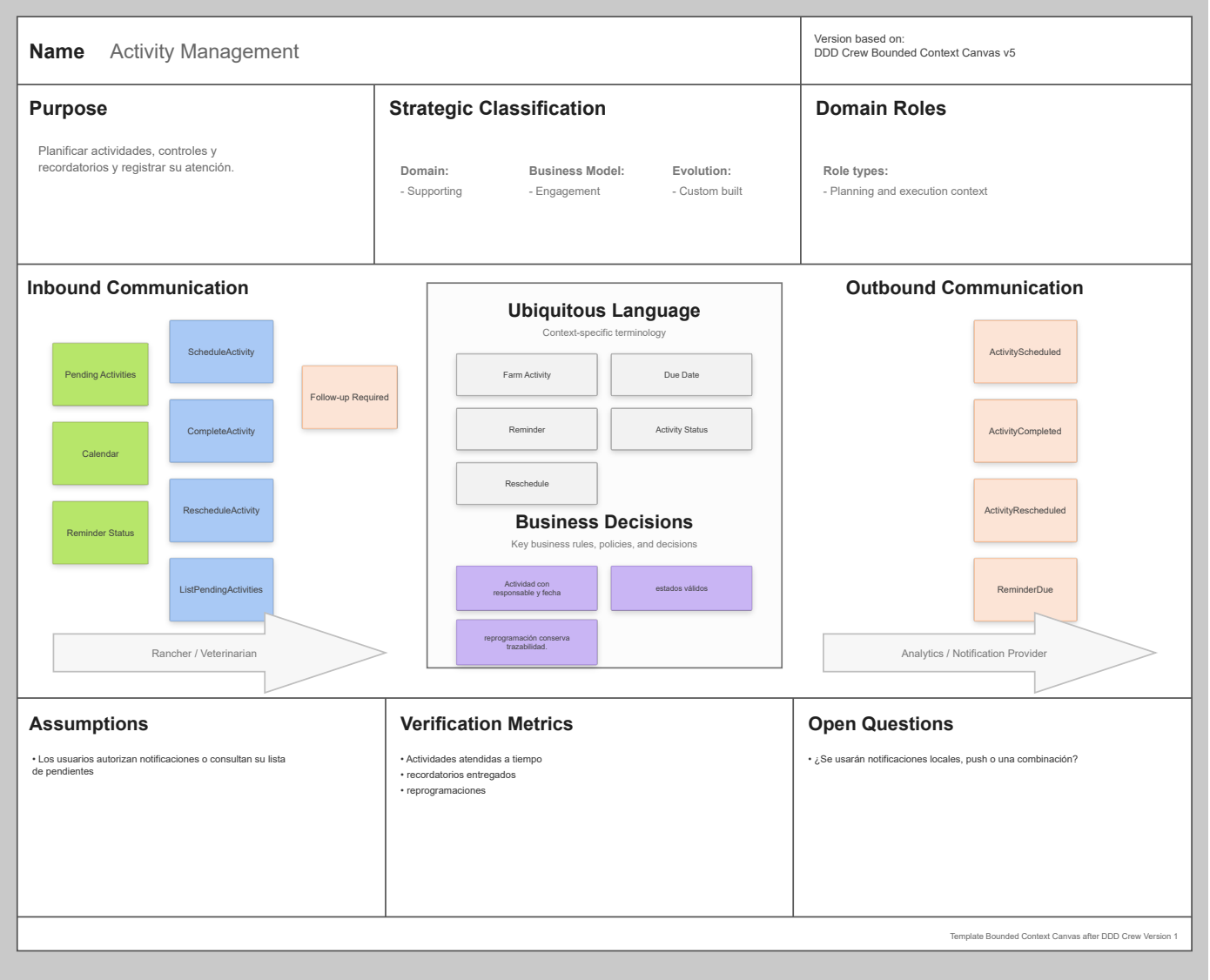


Figura 2.5.23. Bounded Context Canvas de Activity Management. Fuente: elaboración propia.

Financial Management

Registra ingresos y egresos operativos del ganadero. Su alcance inicial proporciona visibilidad económica básica y no reemplaza un sistema contable o tributario.

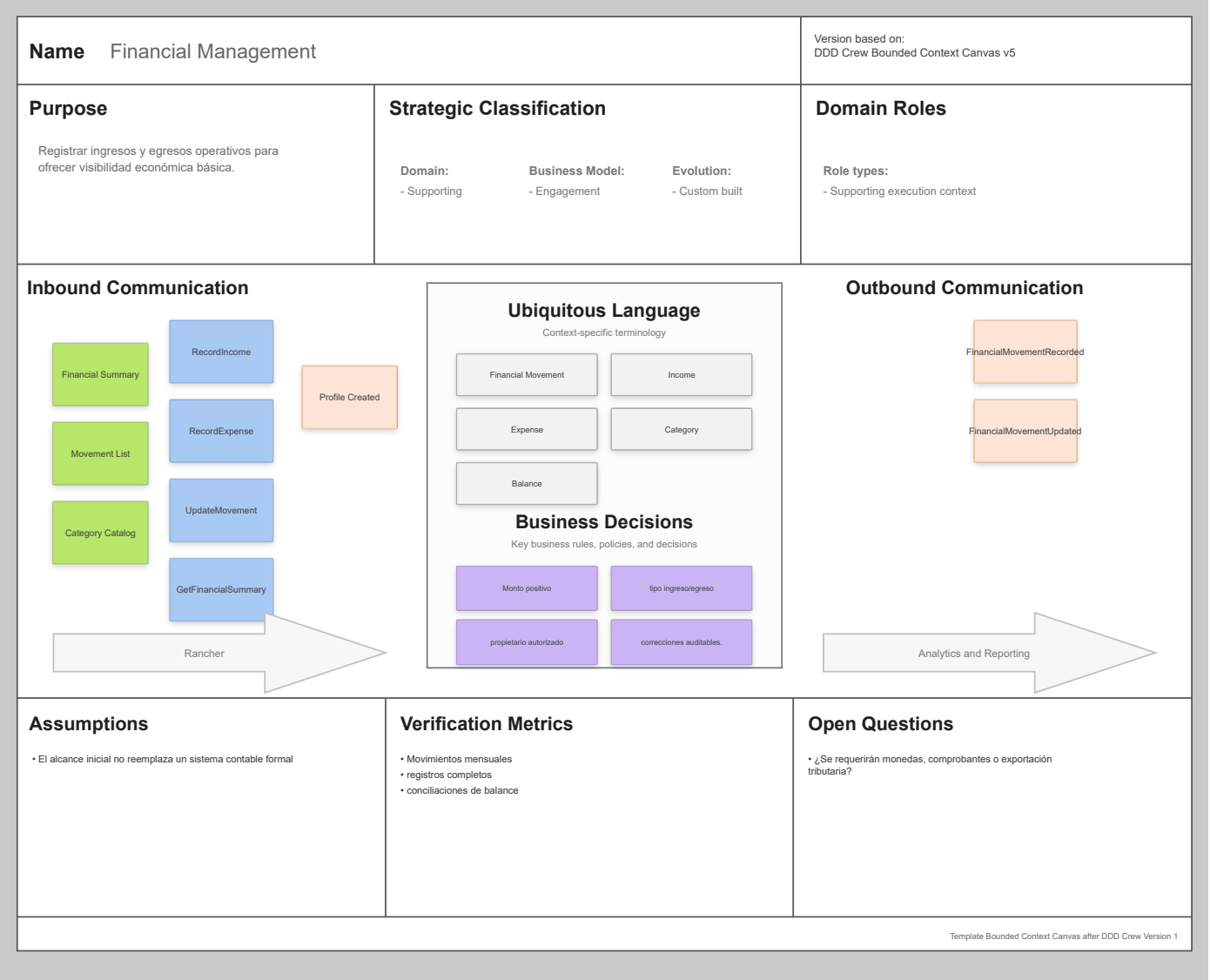


Figura 2.5.24. Bounded Context Canvas de Financial Management. Fuente: elaboración propia.

Subscription Management

Administra planes, checkout, pagos confirmados y estado de suscripción. Stripe permanece fuera del modelo y se traduce mediante una Anti-Corruption Layer.

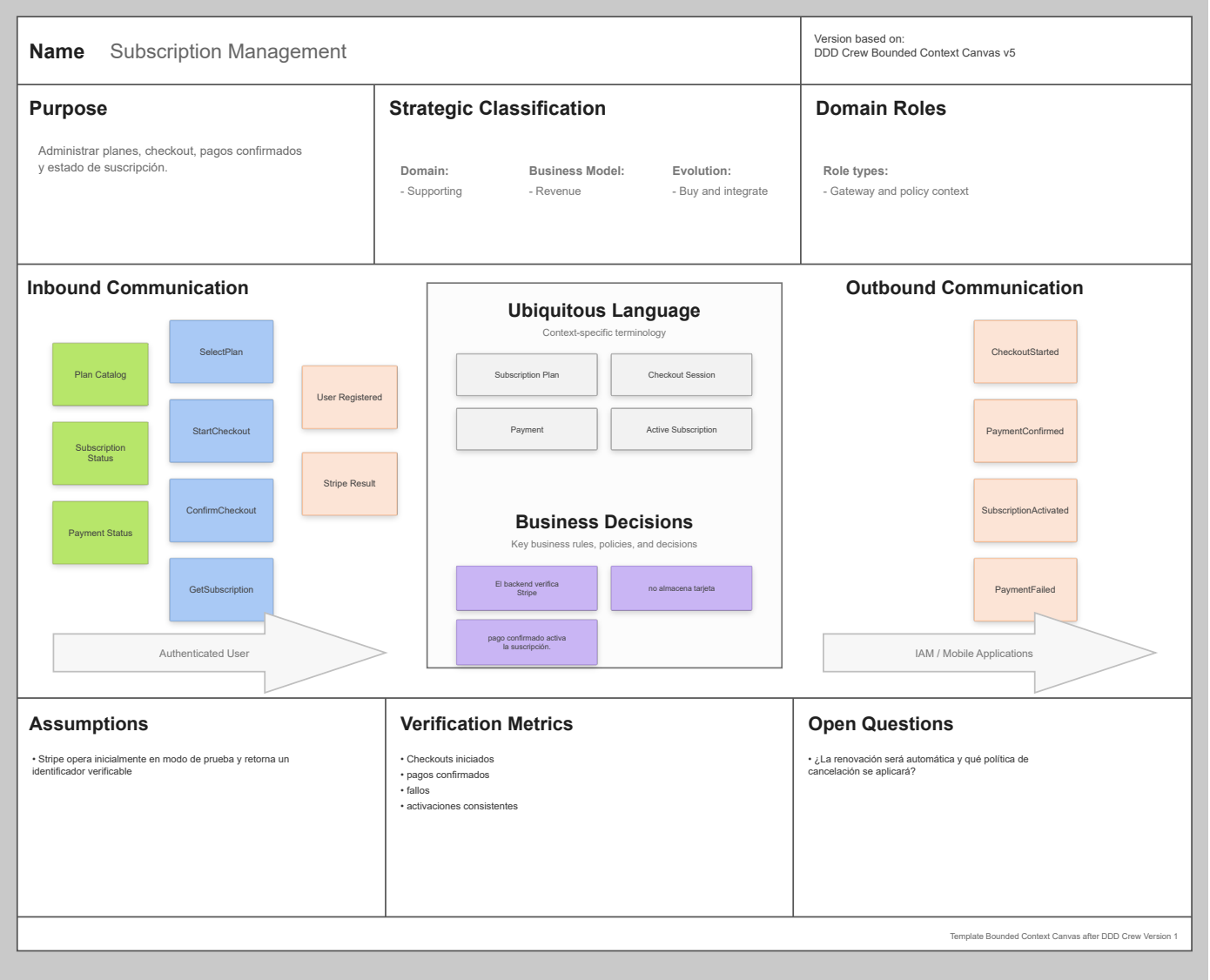


Figura 2.5.25. Bounded Context Canvas de Subscription Management. Fuente: elaboración propia.

Analytics and Reporting

Construye read models para indicadores de ganaderos y veterinarios. Consume información publicada por los contextos fuente y no adquiere propiedad sobre sus reglas o registros.

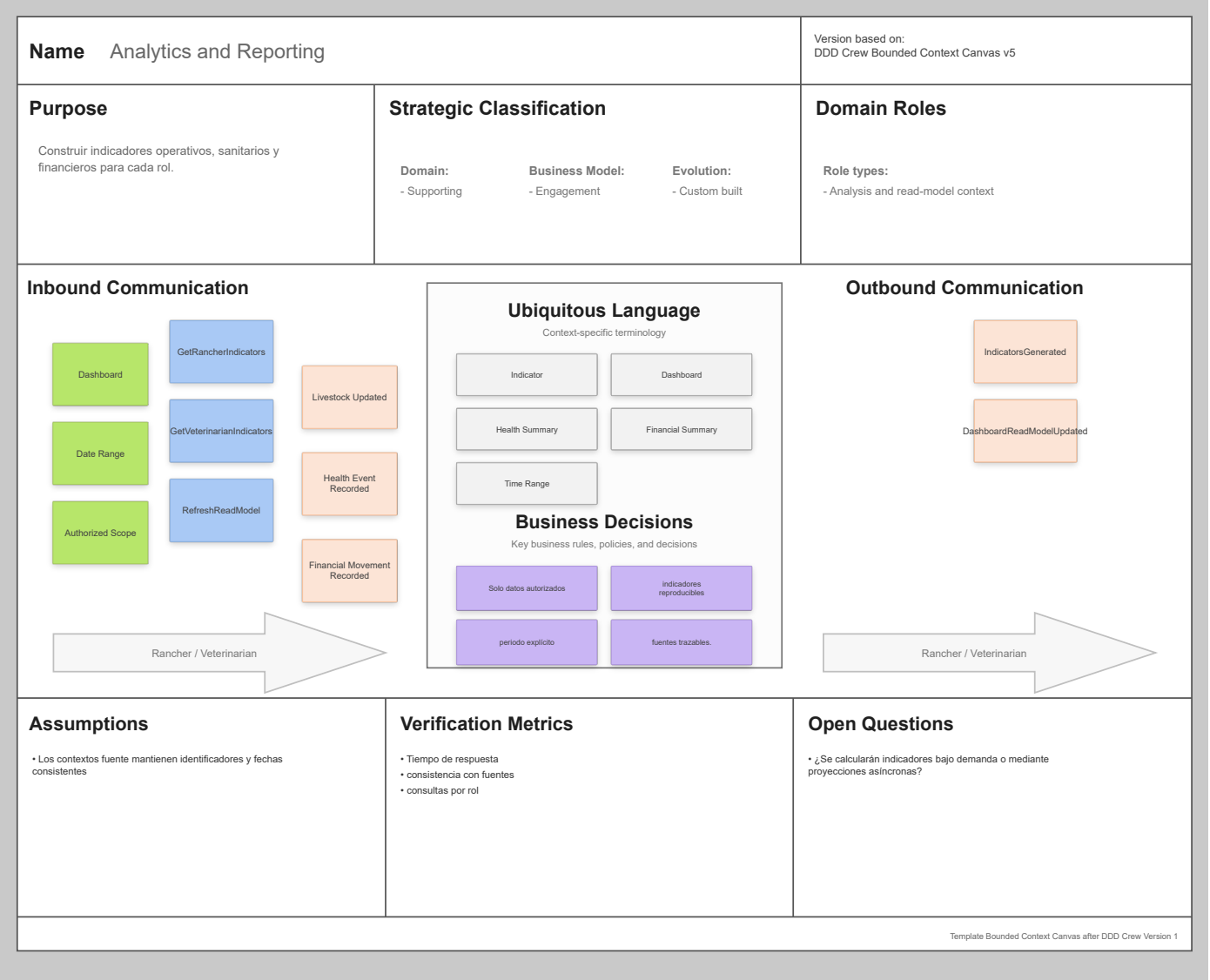


Figura 2.5.26. Bounded Context Canvas de Analytics and Reporting. Fuente: elaboración propia.

Las preguntas abiertas de cada canvas forman parte del Design Critique y deberán resolverse durante los sprints sin modificar silenciosamente el lenguaje o los límites. Cualquier decisión que cambie responsabilidades deberá reflejarse primero en el Context Map y después en el diseño táctico.

2.5.2. Context Mapping

El Context Mapping compara las relaciones estructurales entre los bounded contexts y explicita qué modelo actúa como proveedor, qué modelo consume sus contratos y qué patrón reduce el acoplamiento. Antes de seleccionar el mapa final se evaluaron tres alternativas.

Alternativa	Descripción	Evaluación
A. Contextos amplios	Unir perfiles, colaboración, sanidad y actividades dentro de un único contexto de gestión ganadera.	Rechazada porque mezcla ciclos de vida, vocabularios y reglas de autorización; también dificulta la evolución independiente.
B. Replicar todos los módulos	Considerar como contextos todos los módulos del backend, incluidos Devices,	Rechazada porque confunde infraestructura con subdominios y conserva capacidades IoT fuera

Alternativa	Descripción	Evaluación
anteriores	Metrics y Shared.	del backlog actual.
C. Nueve contextos alineados con valor	Separar IAM, Profiles, Livestock, Sanitary, Veterinary Collaboration, Activities, Financial, Subscriptions y Analytics.	Seleccionada porque mantiene reglas cohesivas, coincide con el alcance móvil y aprovecha la modularidad existente del backend.

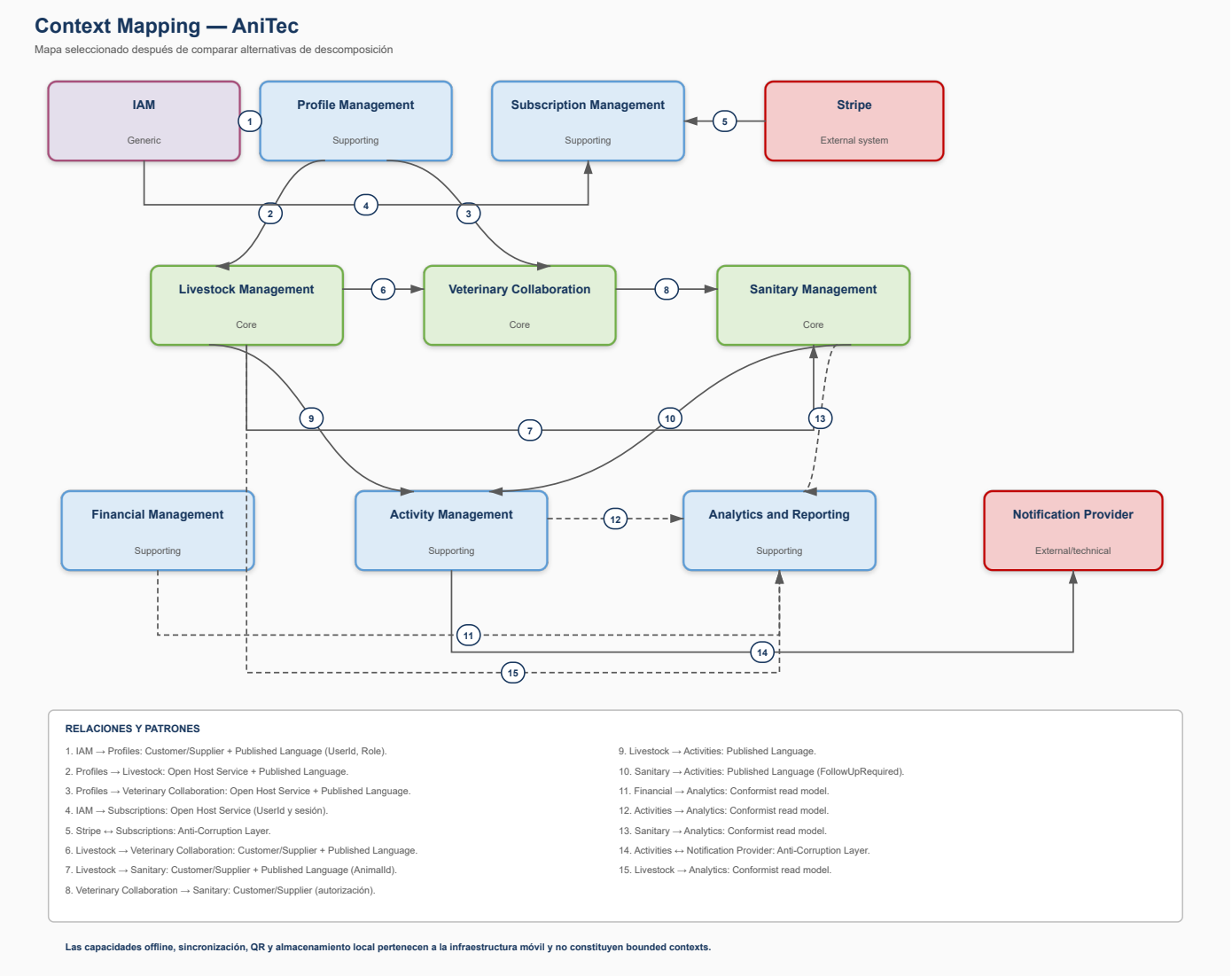


Figura 2.5.27. Context Map seleccionado para AniTec. Fuente: elaboración propia.

Relaciones entre bounded contexts

Upstream / proveedor	Downstream / consumidor	Patrón	Contrato principal
IAM	Profiles	Customer/Supplier y Published Language	UserId, Role, UserRegistered.
IAM	Subscription Management	Open Host Service	Identidad y sesión autenticada.
Profile Management	Livestock Management	Open Host Service y Published Language	Resumen del propietario.

Upstream / proveedor	Downstream / consumidor	Patrón	Contrato principal
Profile Management	Veterinary Collaboration	Open Host Service y Published Language	Identidad y rol de las partes.
Livestock Management	Veterinary Collaboration	Customer/Supplier y Published Language	Propietario, finca y animales.
Livestock Management	Sanitary Management	Customer/Supplier y Published Language	AnimalId, estado y propiedad.
Veterinary Collaboration	Sanitary Management	Customer/Supplier	Estado y alcance de la autorización.
Livestock Management	Activity Management	Published Language	Referencia de animal y finca.
Sanitary Management	Activity Management	Published Language	FollowUpRequired.
Livestock, Sanitary, Activities y Financial	Analytics and Reporting	Conformist read models	Eventos y resúmenes publicados para proyecciones.
Stripe	Subscription Management	Anti-Corruption Layer	Checkout Session y resultado de pago traducidos al modelo interno.
Notification Provider	Activity Management	Anti-Corruption Layer	Solicitud de entrega y resultado de notificación.

Design Critique del Context Map

- **Coherencia con el negocio:** cada contexto contiene conceptos que cambian por las mismas reglas y razones.
- **Protección del Core Domain:** Livestock, Sanitary y Veterinary Collaboration no dependen del modelo externo de Stripe ni del proveedor de notificaciones.
- **Autonomía de Analytics:** sus proyecciones pueden reconstruirse sin modificar los datos fuente.
- **Movilidad:** Android y Flutter consumen contratos publicados por la API; no introducen un segundo modelo del dominio.
- **Offline:** los clientes almacenan comandos y read models locales, mientras las reglas de aceptación siguen en el contexto propietario.
- **Evolución:** reemplazar ML Kit, Stripe o el proveedor de notificaciones no obliga a cambiar el Ubiquitous Language.
- **Trazabilidad:** los mismos nueve contextos deberán utilizarse en el capítulo 2.6 y en los futuros diagramas C4.

El mapa seleccionado será la base de 2.5.3 Software Architecture. En esa sección se representarán la landing page, Android, Flutter, almacenamiento local, API REST, MySQL, Stripe, ML Kit, notificaciones y Firebase App Distribution sin convertir los contenedores técnicos en bounded contexts adicionales.

2.5.3. Software Architecture

La arquitectura de AniTec se documenta con el modelo C4 para representar la solución con distintos niveles de detalle. Las tres vistas se construyen desde un único modelo en Structurizr DSL, de modo que los actores, dependencias, tecnologías y relaciones conserven el mismo significado. El archivo fuente se encuentra en anitec-software-architecture.dsl.

La solución reutiliza la API REST y la persistencia desarrolladas en el proyecto anterior, pero reemplaza el frontend web como canal operativo por una aplicación Android nativa y una aplicación multiplataforma. La landing page mantiene una función informativa. Los nueve bounded contexts definidos en las secciones anteriores pertenecen al backend y no se modelan como contenedores independientes en este nivel; su estructura interna se desarrollará en el diseño táctico del punto 2.6.

2.5.3.1. Software Architecture Context Level Diagrams

El Context Level Diagram delimita a AniTec como un único sistema de software y muestra quién lo usa y de qué servicios externos depende. El **Rancher** administra su operación ganadera; el **Veterinarian** consulta pacientes autorizados y registra atenciones; el **Visitor** conoce la propuesta mediante la landing page; y el **Evaluator / Tester** instala las versiones móviles distribuidas para la validación del curso.

Stripe procesa el checkout de las suscripciones, Firebase Cloud Messaging entrega notificaciones push, Google ML Kit reconoce códigos QR en el dispositivo y Firebase App Distribution distribuye compilaciones firmadas. Estos servicios permanecen fuera del límite de AniTec y se integran mediante contratos que evitan trasladar sus modelos al dominio.

Elemento externo	Relación con AniTec	Resultado esperado
Visitor	Consulta la landing page.	Comprende la propuesta de valor y encuentra el canal de acceso al producto.
Rancher	Usa las aplicaciones móviles.	Gestiona fincas, animales, sanidad, actividades y finanzas.
Veterinarian	Usa las aplicaciones móviles con autorización del ganadero.	Consulta clientes y pacientes y mantiene la trazabilidad sanitaria permitida.
Evaluator / Tester	Recibe e instala builds mediante Firebase App Distribution.	Valida la aplicación en un dispositivo físico y una versión identificable.
Stripe	Recibe solicitudes de checkout y devuelve resultados de pago.	AniTec activa o conserva la suscripción según una confirmación válida.
Firebase Cloud Messaging	Recibe solicitudes de entrega de notificaciones.	El dispositivo presenta recordatorios definidos por Activity Management.
Google ML Kit	Procesa imágenes de la cámara localmente.	La aplicación obtiene el código que Livestock Management resuelve como un animal.
Firebase App Distribution	Publica builds firmadas para usuarios autorizados.	Las versiones de prueba y entrega se distribuyen con trazabilidad.

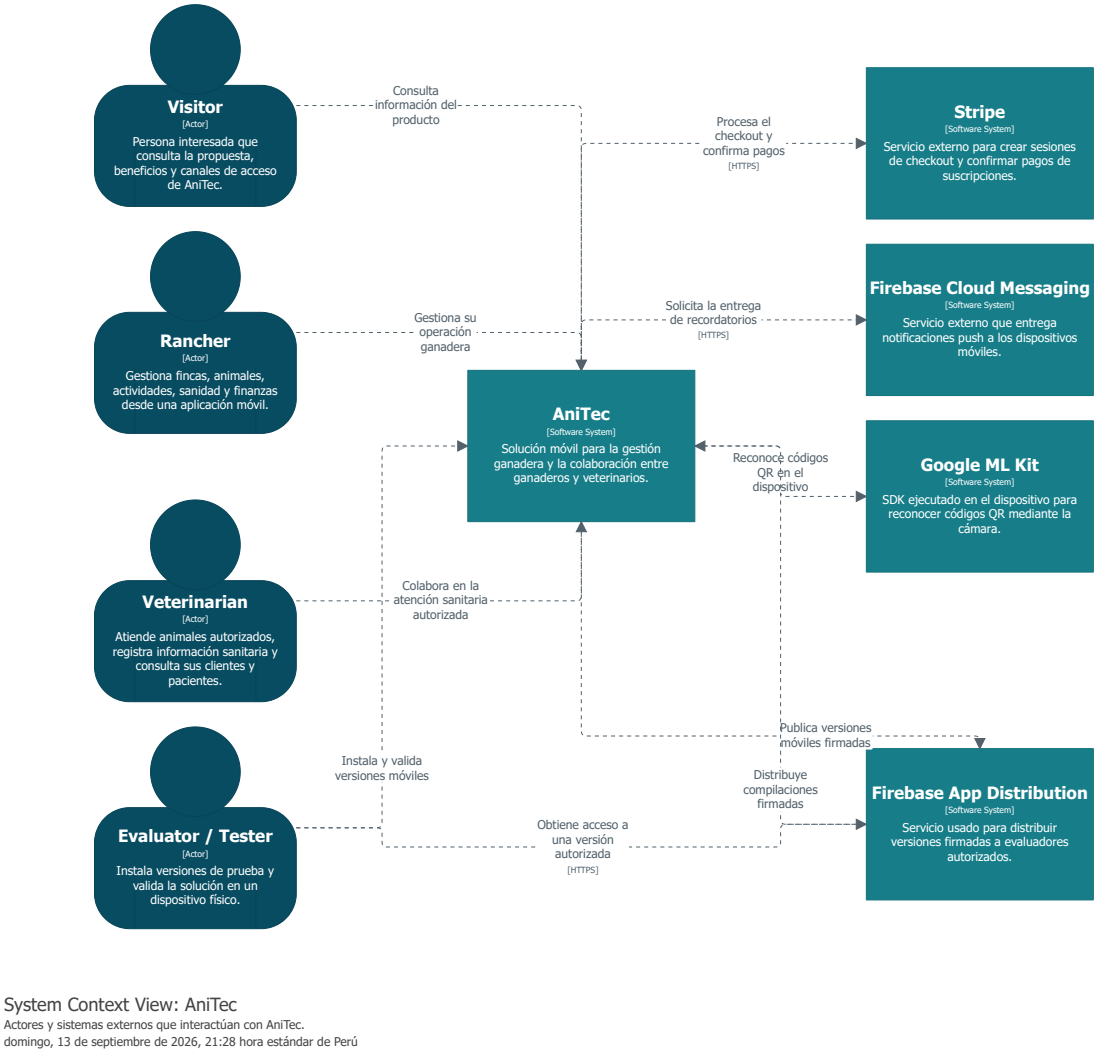


Figura 2.5.28. Software Architecture Context Level Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

2.5.3.2. Software Architecture Container Level Diagrams

El Container Level Diagram descompone AniTec en unidades ejecutables o de almacenamiento. La landing page comunica el producto; las dos aplicaciones móviles implementan la experiencia de los roles; cada cliente mantiene almacenamiento local para caché y operaciones pendientes; la API centraliza autenticación, autorización y reglas de negocio; y MySQL conserva la fuente de verdad compartida.

Android y Flutter consumen los mismos contratos JSON sobre HTTPS y se autentican con JWT. La persistencia local mejora la continuidad ante conectividad limitada, pero las reglas de aceptación continúan bajo el bounded context propietario en la API. La cámara y ML Kit actúan como capacidades del dispositivo: decodifican un QR, mientras Livestock Management determina qué animal representa y si el usuario puede consultarlo.

Contenedor	**Responsabilidad**	**Tecnología**
Landing Page	Presenta la propuesta de valor, segmentos, características y acceso a versiones autorizadas.	HTML, CSS y JavaScript.
Native Android Application	Implementa los flujos móviles nativos, utiliza cámara y notificaciones y sincroniza datos.	Kotlin y Jetpack Compose.

Contenedor	Responsabilidad	Tecnología
Android Local Database	Almacena caché, read models y operaciones pendientes de sincronización.	Room sobre SQLite.
Cross-Platform Mobile Application	Implementa el mismo alcance funcional mediante una base de código multiplataforma.	Flutter y Dart.
Flutter Local Database	Proporciona persistencia local equivalente para la aplicación multiplataforma.	SQLite mediante una biblioteca compatible con Flutter.
AniTec REST API	Expone casos de uso, aplica reglas de negocio y autorización y coordina integraciones externas.	C#, ASP.NET Core, Entity Framework Core, JWT y OpenAPI.
AniTec Database	Persiste la información transaccional y las proyecciones de los bounded contexts.	MySQL.

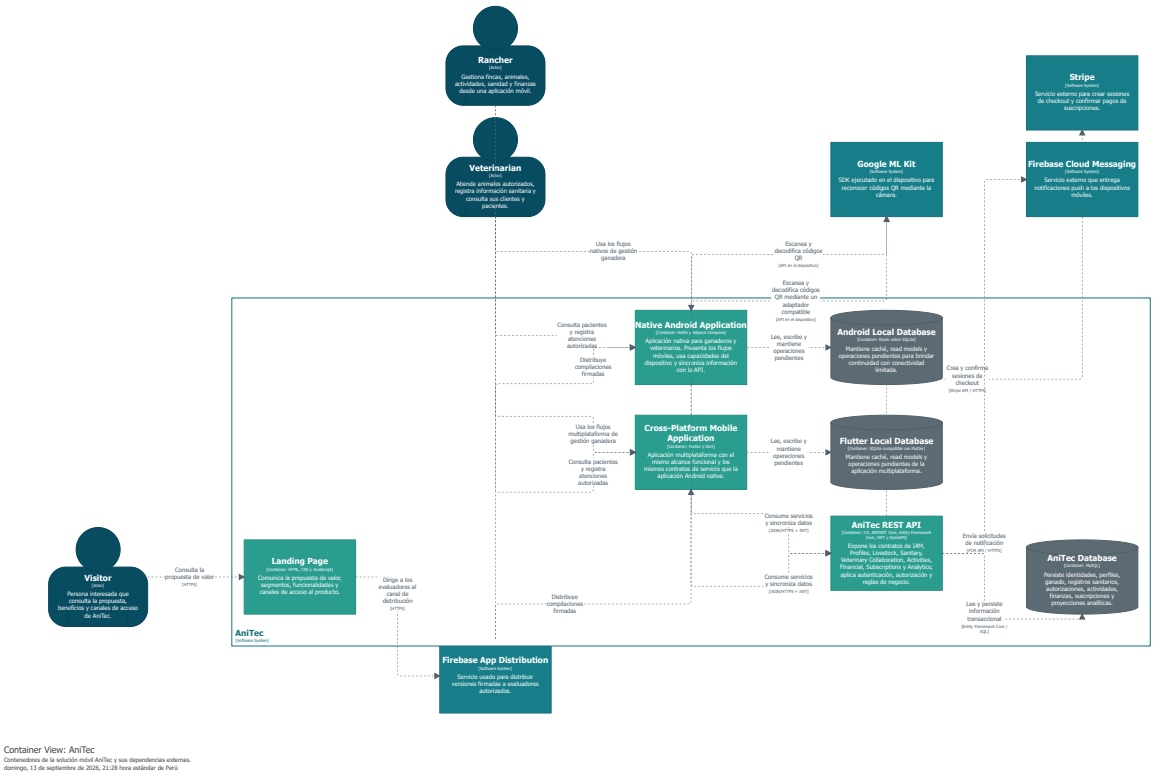


Figura 2.5.29. Software Architecture Container Level Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

2.5.3.3. Software Architecture Deployment Diagrams

El Deployment Diagram representa la topología objetivo para las entregas del curso. La aplicación Android nativa se instala en un dispositivo Android físico y conserva su base Room dentro del mismo dispositivo. La aplicación Flutter se instala en el dispositivo físico correspondiente a la plataforma objetivo definida por el equipo y mantiene su propia base SQLite local. En ambos casos, la cámara y el reconocimiento QR se ejecutan dentro del dispositivo.

La landing page se publica como contenido estático en GitHub Pages. La API ASP.NET Core se ejecuta como Web Service en Render y accede a MySQL administrado en Filess.io, destinos recuperados del despliegue funcional del proyecto anterior. Stripe, Firebase Cloud Messaging y Firebase App Distribution se mantienen como servicios

externos. Esta topología deberá actualizarse si durante los sprints cambia algún proveedor, sin alterar las responsabilidades de los contenedores.

Nodo de despliegue	Artefacto o instancia	Comunicación principal
Physical Android Device	Native Android Application, Room/SQLite, cámara y ML Kit.	HTTPS con la API; almacenamiento y reconocimiento local.
Cross-Platform Target Device	Flutter Application, SQLite, cámara y adaptador de reconocimiento QR.	HTTPS con la API; almacenamiento y reconocimiento local.
GitHub Pages	Landing Page.	HTTPS con visitantes y acceso al canal de distribución.
Render	AniTec REST API sobre el runtime de .NET.	JSON/HTTPS con clientes móviles; SQL con MySQL; HTTPS con servicios externos.
Filess.io	AniTec Database sobre MySQL.	Conexión restringida desde la API.
Firebase App Distribution	Builds móviles firmadas y versionadas.	Distribución a evaluadores autorizados.
Firebase Cloud Messaging	Servicio de entrega de notificaciones.	Solicitudes desde el backend y entrega al dispositivo.
Stripe Cloud	Checkout y confirmación de pago.	Stripe API sobre HTTPS.

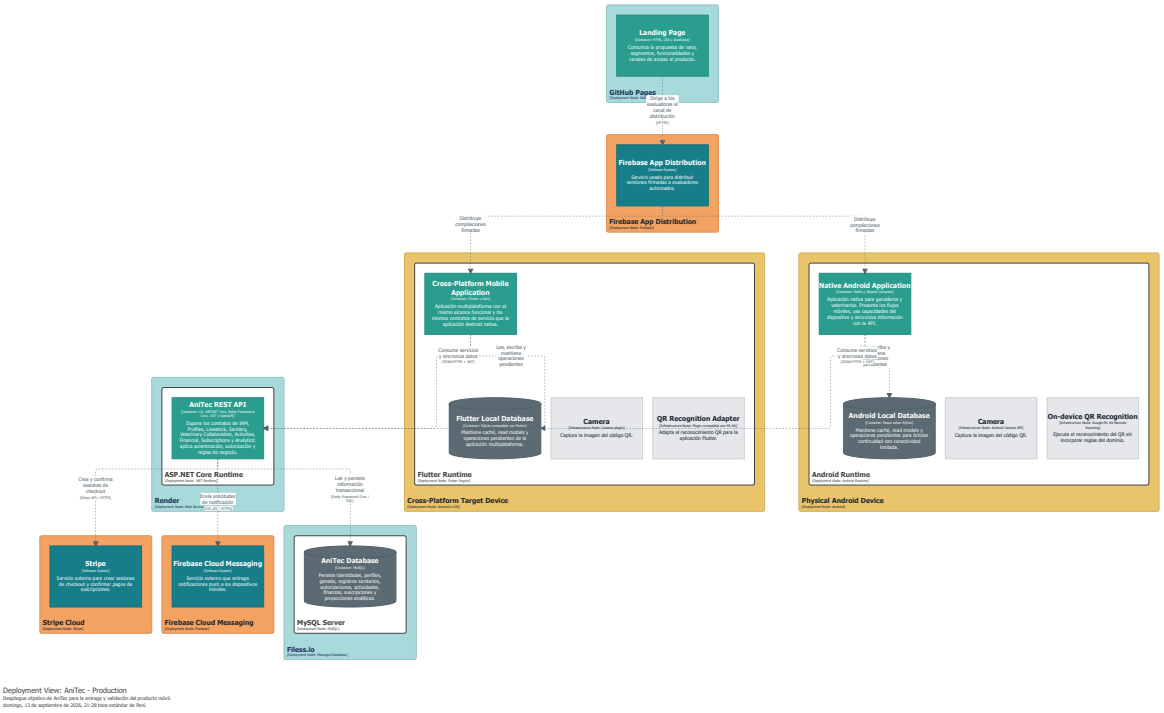


Figura 2.5.30. Software Architecture Deployment Diagram de AniTec. Fuente: elaboración propia con Structurizr DSL.

Las tres imágenes deben exportarse desde el mismo archivo DSL. Los nombres y rutas requeridos se detallan en codefordiagrams/README.md, de modo que al copiar los PNG exportados los placeholders del informe se resuelvan sin modificar nuevamente el contenido.

2.6. Tactical-Level Domain-Driven Design

Esta sección describe cómo los nueve bounded contexts identificados en el diseño estratégico se materializan en clases, capas, componentes y estructuras de persistencia. La propuesta utiliza un enfoque híbrido trazable: conserva nombres y responsabilidades del backend heredado y marca como no implementados los elementos requeridos para Android, Flutter y la evolución del dominio que todavía no existen en el código.

Para todos los contextos se aplican las siguientes convenciones:

- La **Domain Layer** mantiene entidades, aggregates, value objects, servicios de dominio e interfaces de repositorio.
- La **Application Layer** orquesta comandos, consultas, eventos y casos de uso sin depender de frameworks de presentación.
- La **Interface Layer** traduce HTTP o acciones de la interfaz móvil hacia los casos de uso.
- La **Infrastructure Layer** implementa persistencia, red, seguridad, sincronización e integraciones externas.
- MySQL es la fuente autoritativa; Room y SQLite mantienen caché y un outbox idempotente.
- Las reglas de negocio permanecen en la API y el dominio; Android y Flutter reutilizan los mismos contratos y lenguaje ubicuo.

Para mantener el informe legible, las fichas individuales se reservan para aggregates, entidades, servicios, handlers, controladores y adaptadores que explican decisiones relevantes de arquitectura. Los DTO, resources, mappers, comandos simples, configuraciones y adaptadores repetitivos se omiten del diccionario detallado y permanecen representados en el código o en los diagramas correspondientes cuando resulta necesario.

Sección	Bounded Context	Módulo heredado	Propósito
2.6.1	Identity and Access Management	Iam	Administrar identidades, credenciales, roles y sesiones para que cada operación de AniTec se ejecute con una identidad autenticada y autorizada.
2.6.2	Profile Management	Profiles	Mantener la información personal y de contacto asociada con una identidad sin mezclarla con credenciales o reglas de autenticación.
2.6.3	Livestock Management	Livestock	Gestionar fincas, hatos, animales e identificadores QR como fuente de referencia para los demás procesos ganaderos.
2.6.4	Sanitary Management	Sanitary	Conservar la historia sanitaria del animal y controlar el registro de diagnósticos, tratamientos, prescripciones y seguimientos autorizados.
2.6.5	Veterinary Collaboration	Clients	Administrar solicitudes y autorizaciones entre ganaderos y veterinarios, delimitando clientes, pacientes y alcance de acceso.
2.6.6	Activity Management	Activities	Planificar actividades ganaderas y sanitarias, controlar su estado y decidir cuándo corresponde generar un recordatorio.

Sección	Bounded Context	Módulo heredado	Propósito
2.6.7	Financial Management	Financial	Registrar ingresos y egresos operativos y producir resúmenes económicos básicos para apoyar decisiones del ganadero.
2.6.8	Subscription Management	Subscriptions	Administrar planes, vigencia de suscripciones y pagos confirmados sin introducir conceptos propios de Stripe en el dominio.
2.6.9	Analytics and Reporting	Analytics	Construir proyecciones y métricas de consulta para los dashboards sin adquirir propiedad sobre los registros de los contextos fuente.

2.6.1. Bounded Context: Identity and Access Management

Administrar identidades, credenciales, roles y sesiones para que cada operación de AniTec se ejecute con una identidad autenticada y autorizada.

La base implementada se encuentra en el módulo Iam de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.1.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Identity and Access Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: User

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Mantener identidad, credenciales protegidas y rol del usuario.
Relaciones	User se relaciona con UserRole : has; AuthenticatedSession se relaciona con User : belongs to; CredentialPolicy depende de User : validates; IUserRepository depende de User : persists

Atributos o dependencias

Nombre	Tipo
id	int
username	string
passwordHash	string
fullName	string
role	UserRole

Métodos u operaciones

Firma	Retorno
UpdateUsername(username: string)	User
UpdatePasswordHash(hash: string)	User
UpdateProfile(fullName: string, role: UserRole)	User

Enumeration: UserRole

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Definir los valores válidos de UserRole.
Relaciones	User se relaciona con UserRole : has

Atributos o dependencias

Nombre	Tipo
Rancher	No especificado
Veterinarian	No especificado
Administrator	No especificado

Métodos u operaciones

No aplica.

Entity: AuthenticatedSession

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Controlar vigencia y revocación de una sesión autenticada.
Relaciones	AuthenticatedSession se relaciona con User : belongs to

Atributos o dependencias

Nombre	Tipo
userId	int
tokenReference	string
expiresAt	DateTime

Métodos u operaciones

Firma	Retorno
IsExpired(now: DateTime)	bool
Revoke()	void

Domain Service: CredentialPolicy

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Validar reglas de nombre de usuario y contraseña.
Relaciones	CredentialPolicy depende de User : validates

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
ValidateUsername(username: string)	bool
ValidatePassword(password: string)	bool

Repository Interface: IUserRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de User.
Relaciones	IUserRepository depende de User : persists

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	User?
FindByUsername(username: string)	User?
Add(user: User)	void

2.6.1.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro de cuentas, autenticación, consulta de identidad y control de sesión** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: AuthenticationController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Identity and Access Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
userCommandService	IUserCommandService
errorLocalizer	IStringLocalizer
iamLocalizer	IStringLocalizer
problemDetailsFactory	ProblemDetailsFactory

Métodos u operaciones

Firma	Retorno
SignIn(SignInResource signInResource, CancellationToken cancellationToken)	No especificado
SignUp(SignUpResource signUpResource, CancellationToken cancellationToken)	No especificado

Presentation Model: UserViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: UserController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.1.3. Application Layer

Esta capa coordina las capacidades de **registro de cuentas, autenticación, consulta de identidad y control de sesión**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: UserCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar registro de cuentas, autenticación, consulta de identidad y control de sesión sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
userRepository	IUserRepository

Nombre	Tipo
tokenService	ITokenService
hashingService	IHashingService
unitOfWork	IUnitOfWork
localizer	IStringLocalizer

Métodos u operaciones

Firma	Retorno
Handle(SignInCommand command, CancellationToken cancellationToken)	No especificado
Handle(SignUpCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveUserUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: SignUpCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: UserAuthenticatedEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.1.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Identity and Access Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: UserRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IUserRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindByUsernameAsync(string username, CancellationToken cancellationToken)	No especificado
ExistsByUsernameAsync(string username, CancellationToken cancellationToken)	No especificado

Room Adapter: UserDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: UserLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Token Adapter: TokenService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de tokens y utiliza JWT.

Atributos o dependencias

Nombre	Tipo
expiration	issuer, audience, signingKey,

Métodos u operaciones

Firma	Retorno
GenerateToken(user)	string
ValidateToken(token)	Task<int?>

Secure Storage Adapter: SecureSessionStore

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto local de sesión sobre Keystore/EncryptedSharedPreferences o Secure Storage.

Atributos o dependencias

Nombre	Tipo
expiresAt	tokenReference,

Métodos u operaciones

Firma	Retorno
Save(session)	No especificado
Read()	No especificado
Clear()	No especificado

2.6.1.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC1-ApiComponents, BC1-AndroidComponents y BC1-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

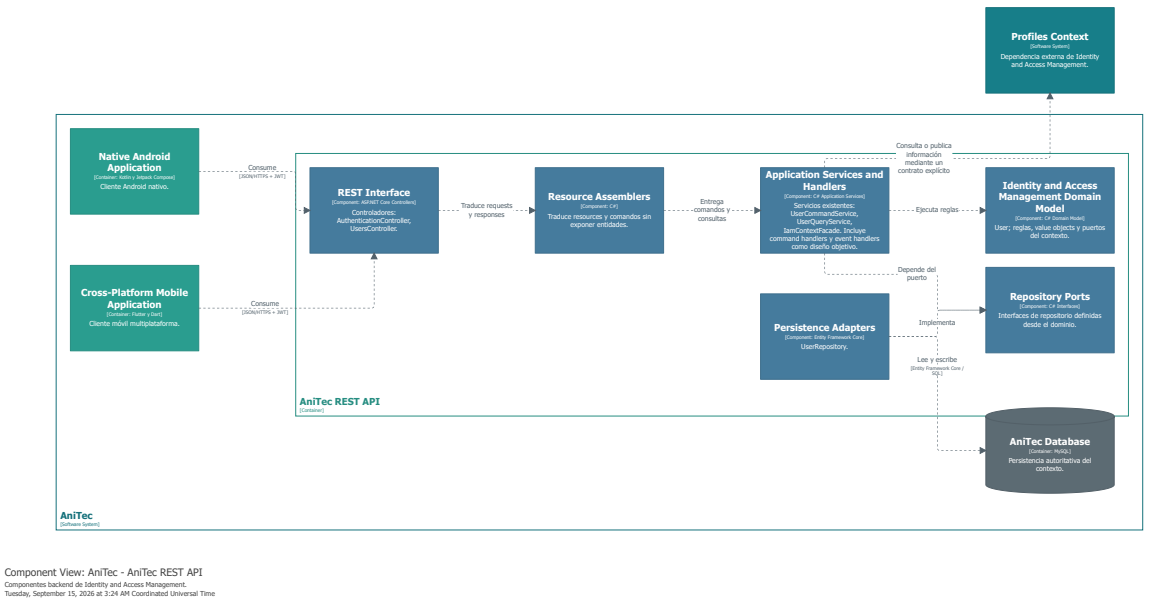


Figura 2.6.1.1. Componentes de la API para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

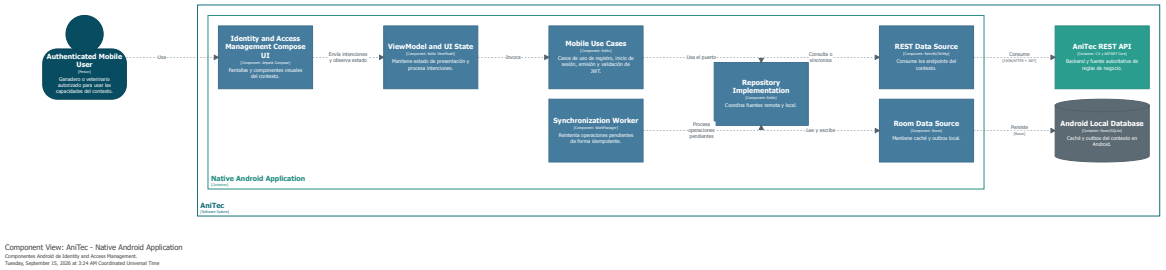


Figura 2.6.1.2. Componentes Android para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

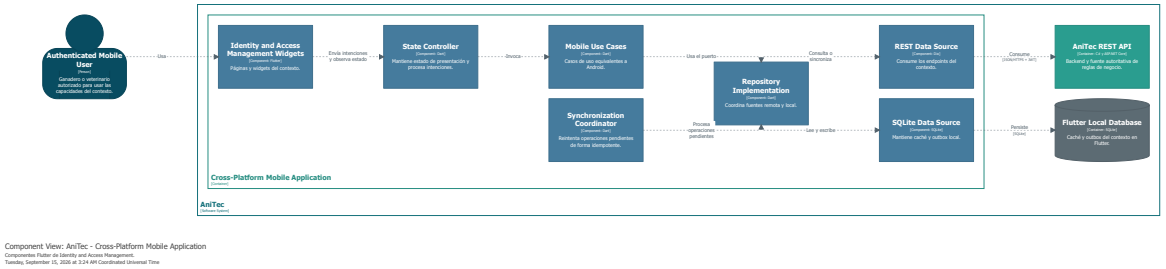


Figura 2.6.1.3. Componentes Flutter para Identity and Access Management. Fuente: elaboración propia con Structurizr DSL.

2.6.1.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.1.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

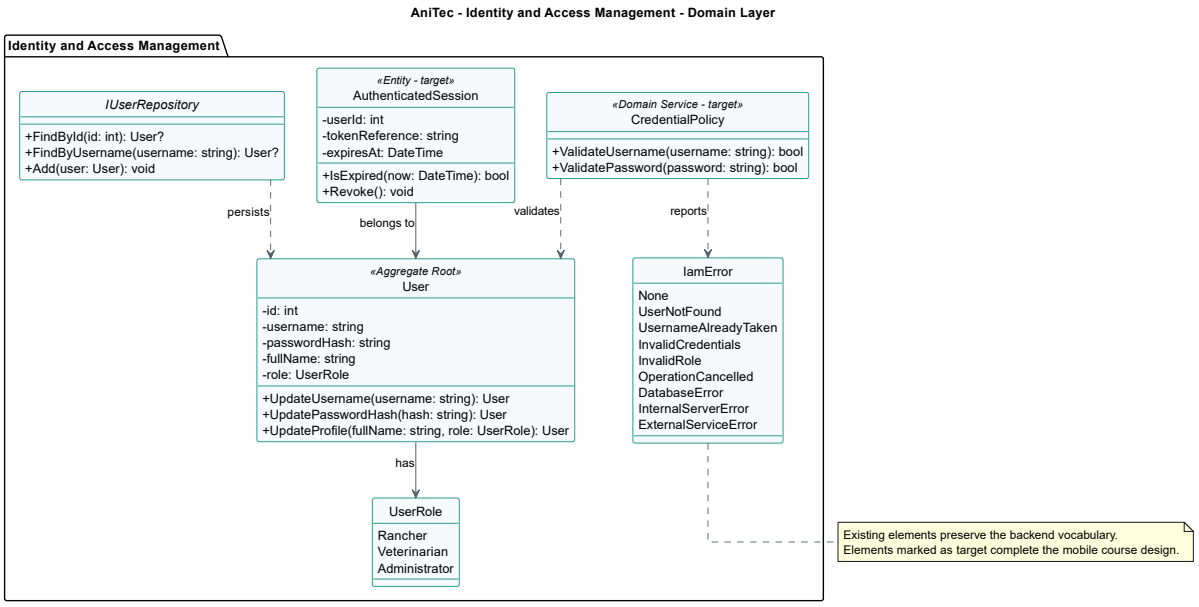


Figura 2.6.1.4. Domain Layer Class Diagram de Identity and Access Management. Fuente: elaboración propia con PlantUML.

2.6.1.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

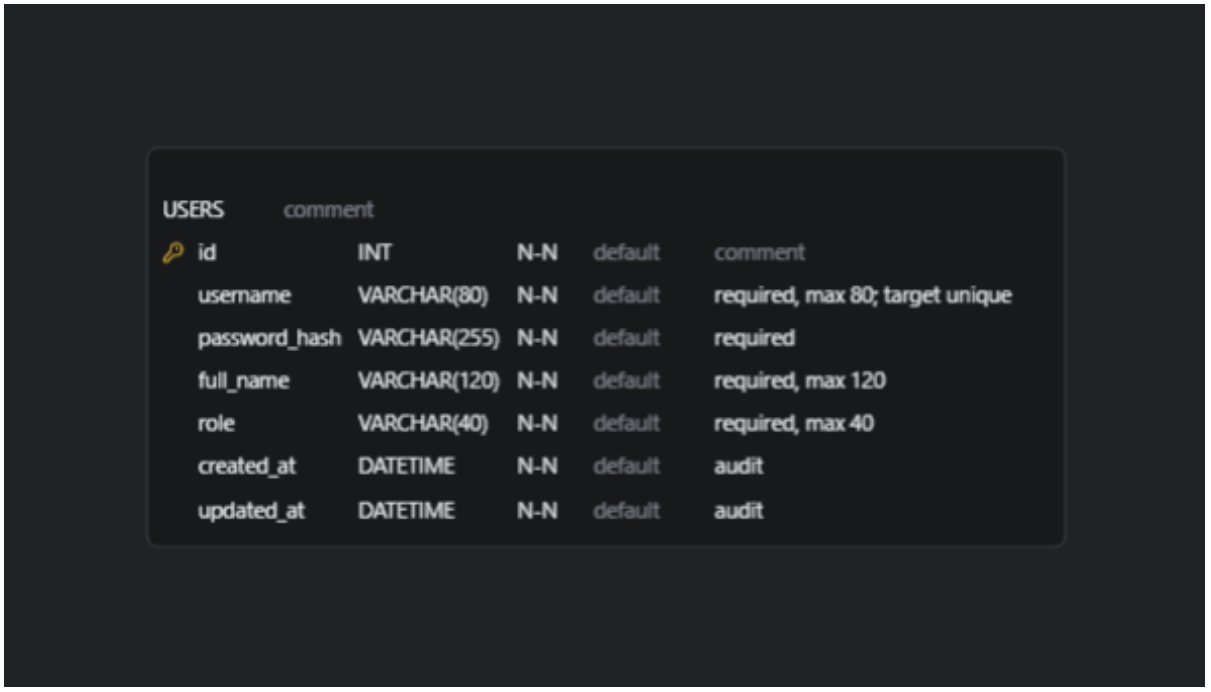


Figura 2.6.1.5. MySQL Database Design de Identity and Access Management. Fuente: elaboración propia a partir del esquema SQL.

Figura 2.6.1.6. Android Room Database Design de Identity and Access Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.2. Bounded Context: Profile Management

Mantener la información personal y de contacto asociada con una identidad sin mezclarla con credenciales o reglas de autenticación.

La base implementada se encuentra en el módulo Profiles de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.2.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Profile Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: Profile

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Mantener la información personal y de contacto de un propietario.
Relaciones	Profile compone PersonName; Profile compone EmailAddress; Profile compone StreetAddress; IProfileRepository depende de Profile : persists

Atributos o dependencias

Nombre	Tipo
id	int
ownerId	int
name	PersonName
email	EmailAddress
address	StreetAddress

Métodos u operaciones

Firma	Retorno
FullName()	string
ChangeEmail(email: EmailAddress)	void
ChangeAddress(address: StreetAddress)	void

Value Object: PersonName

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Representar un nombre personal válido.
Relaciones	Profile compone PersonName

Atributos o dependencias

Nombre	Tipo
firstName	string
lastName	string

Métodos u operaciones

Firma	Retorno
FullName()	string

Value Object: EmailAddress

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Representar y validar un correo electrónico.
Relaciones	Profile compone EmailAddress

Atributos o dependencias

Nombre	Tipo
address	string

Métodos u operaciones

Firma	Retorno
IsValid()	bool

Value Object: StreetAddress

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Representar una dirección postal completa.
Relaciones	Profile compone StreetAddress

Atributos o dependencias

Nombre	Tipo
street	string
number	string
city	string
postalCode	string
country	string

Métodos u operaciones

Firma	Retorno
FullAddress()	string

Repository Interface: IProfileRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de Profile.
Relaciones	IProfileRepository depende de Profile : persists

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	Profile?
FindByOwnerId(ownerId: int)	Profile?
Add(profile: Profile)	void
Update(profile: Profile)	void

2.6.2.2. Interface Layer

Esta capa recibe las acciones relacionadas con **creación, actualización y consulta de perfiles** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: ProfilesController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Profile Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
profileCommandService	IProfileCommandService
profileQueryService	IProfileQueryService
errorLocalizer	IStringLocalizer
problemDetailsFactory	ProblemDetailsFactory

Métodos u operaciones

Firma	Retorno
GetProfileById(int profileId, CancellationToken cancellationToken)	No especificado
CreateProfile(CreateProfileResource resource, CancellationToken cancellationToken)	No especificado
GetAllProfiles(CancellationToken cancellationToken)	No especificado

Presentation Model: ProfileViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado

Firma	Retorno
RetrySync()	No especificado

State Controller: ProfileController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.2.3. Application Layer

Esta capa coordina las capacidades de **creación, actualización y consulta de perfiles**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: ProfileCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar creación, actualización y consulta de perfiles sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
profileRepository	IProfileRepository
unitOfWork	IUnitOfWork
localizer	IStringLocalizer

Métodos u operaciones

Firma	Retorno
Handle(CreateProfileCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveProfileUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreateProfileCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: ProfileUpdatedEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.2.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Profile Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: ProfileRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**

Campo	Detalle
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IProfileRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindProfileByEmailAsync(EmailAddress email, CancellationToken cancellationToken)	No especificado

Room Adapter: ProfileDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: ProfileLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.

Campo	Detalle
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Context Adapter: IamProfileOwnerAdapter

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de verificación del propietario y consume IAM Context Facade.

Atributos o dependencias

Nombre	Tipo
iamFacade	No especificado

Métodos u operaciones

Firma	Retorno
Exists(ownerId)	bool

2.6.2.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC2-ApiComponents, BC2-AndroidComponents y BC2-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

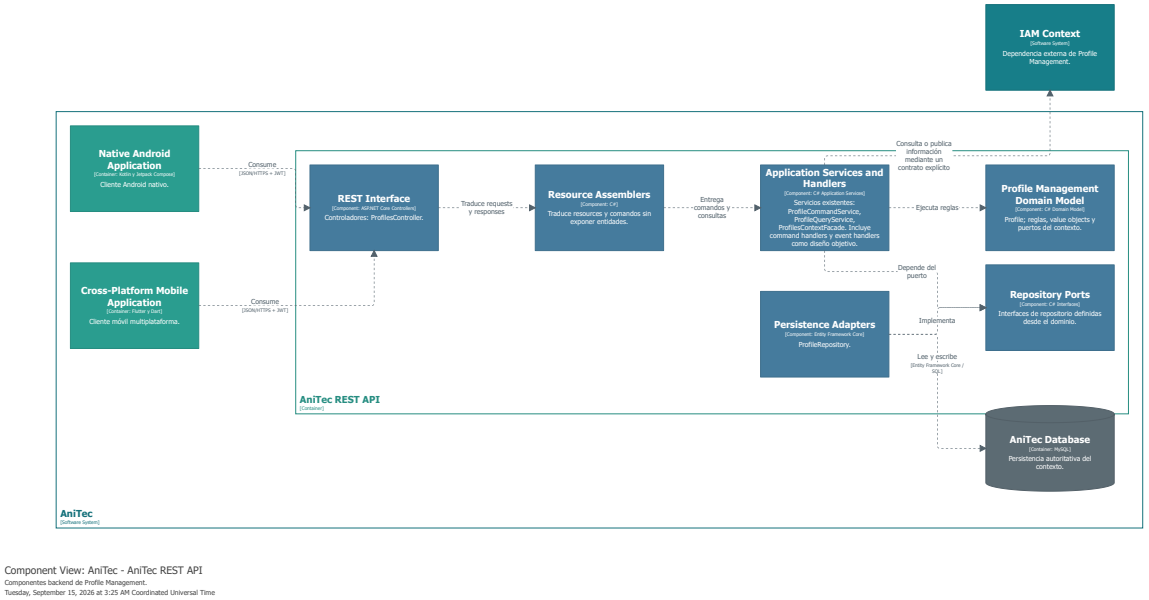


Figura 2.6.2.1. Componentes de la API para Profile Management. Fuente: elaboración propia con Structurizr DSL.

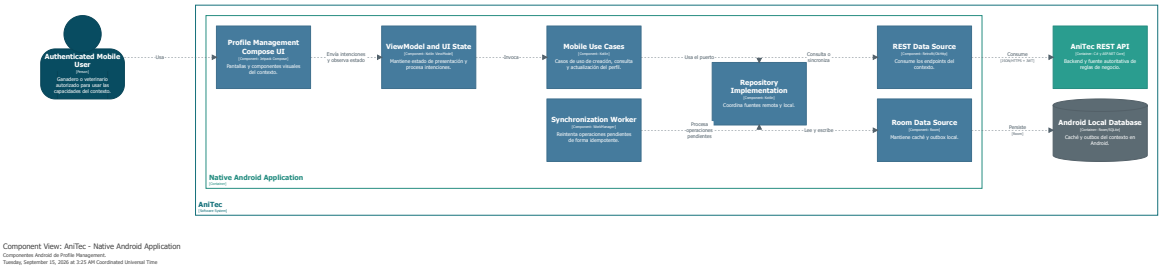


Figura 2.6.2.2. Componentes Android para Profile Management. Fuente: elaboración propia con Structurizr DSL.

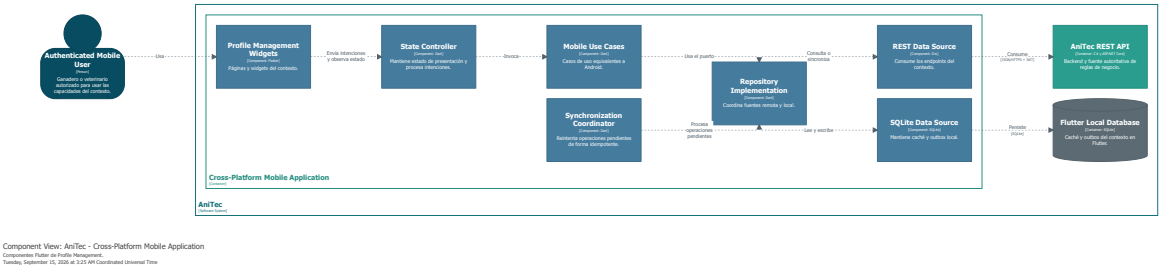


Figura 2.6.2.3. Componentes Flutter para Profile Management. Fuente: elaboración propia con Structurizr DSL.

2.6.2.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.2.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

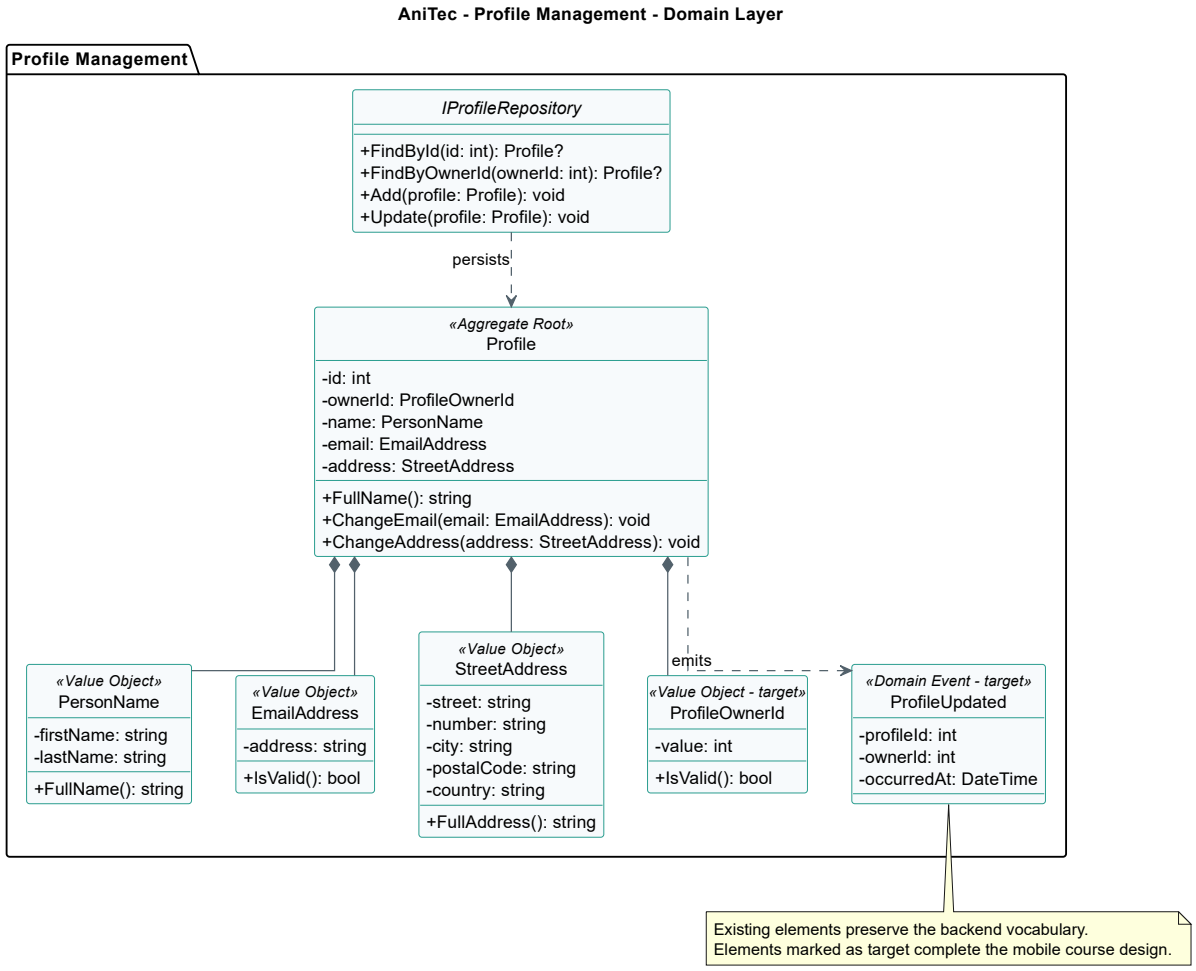


Figura 2.6.2.4. Domain Layer Class Diagram de Profile Management. Fuente: elaboración propia con PlantUML.

2.6.2.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

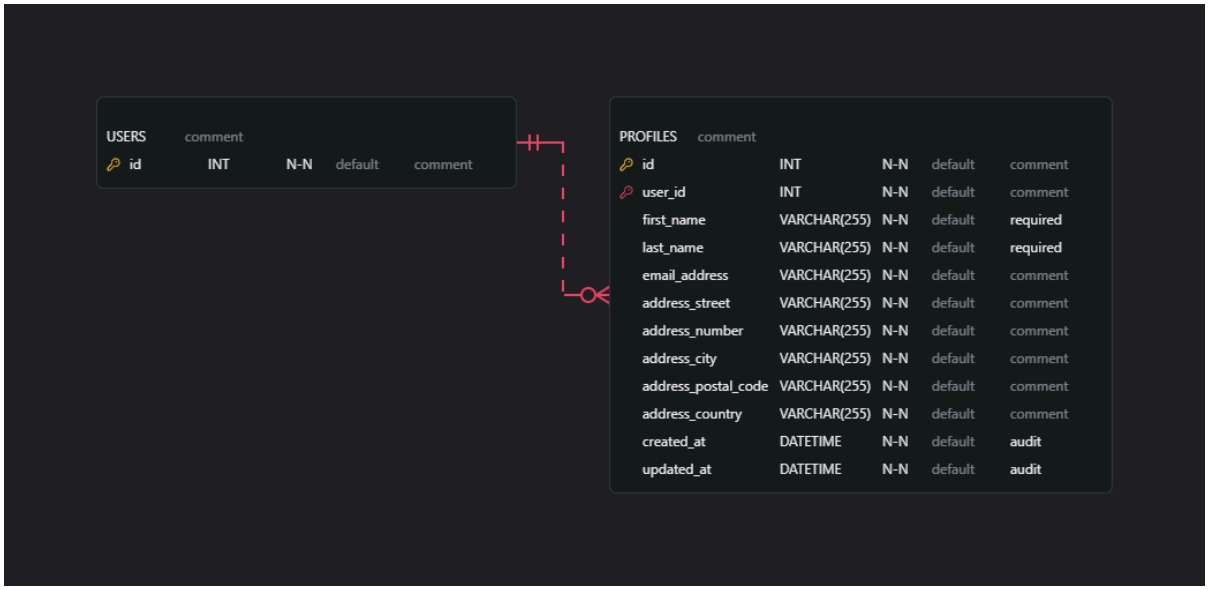


Figura 2.6.2.5. MySQL Database Design de Profile Management. Fuente: elaboración propia a partir del esquema SQL.

Figura 2.6.2.6. Android Room Database Design de Profile Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.3. Bounded Context: Livestock Management

Gestionar fincas, hatos, animales e identificadores QR como fuente de referencia para los demás procesos ganaderos.

La base implementada se encuentra en el módulo Livestock de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.3.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Livestock Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: Farm

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Agrupar los hatos de un propietario.
Relaciones	Farm compone Herd

Atributos o dependencias

Nombre	Tipo
id	int
ownerId	int
name	string
location	string

Métodos u operaciones

Firma	Retorno
RegisterHerd(herd: Herd)	void
BelongsTo(ownerId: int)	bool

Entity: Herd

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Administrar un conjunto de animales dentro de una finca.

Campo	Detalle
Relaciones	Farm compone Herd; Herd compone Animal; IHerdRepository depende de Herd

Atributos o dependencias

Nombre	Tipo
id	int
farmId	int
name	string
mainType	string
veterinarianId	int?

Métodos u operaciones

Firma	Retorno
AssignVeterinarian(id: int)	void
AddAnimal(animal: Animal)	void

Entity: Animal

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Mantener identificación y estado productivo de un animal.
Relaciones	Herd compone Animal; Animal compone AnimalTag; Animal compone QrIdentifier; Animal se relaciona con AnimalStatus; IAnimalRepository depende de Animal

Atributos o dependencias

Nombre	Tipo
id	int
herdId	int
tag	AnimalTag
qrIdentifier	QrIdentifier
name	string
species	string
breed	string
status	AnimalStatus

Métodos u operaciones

Firma	Retorno
UpdateWeight(weight: decimal)	void
ChangeStatus(status: AnimalStatus)	void

Value Object: QrIdentifier

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar la carga QR que identifica un animal.
Relaciones	Animal compone QrIdentifier

Atributos o dependencias

Nombre	Tipo
value	string

Métodos u operaciones

Firma	Retorno
AsPayload()	string

Enumeration: AnimalStatus

Campo	Detalle
Producto y estado	Backend y modelos equivalentes Android/Flutter — **Implementado**
Propósito	Definir los valores válidos de AnimalStatus.
Relaciones	Animal se relaciona con AnimalStatus

Atributos o dependencias

Nombre	Tipo
Active	No especificado
Sold	No especificado
Deceased	No especificado

Métodos u operaciones

No aplica.

Repository Interface: IAnimalRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de Animal.
Relaciones	IAnimalRepository depende de Animal

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	Animal?
FindByQr(code: string)	Animal?
Add(animal: Animal)	void

2.6.3.2. Interface Layer

Esta capa recibe las acciones relacionadas con **gestión de fincas, hatos, animales e identificación QR** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: HerdsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Livestock Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IHerdCommandService
queryService	IHerdQueryService

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateHerdResource resource, CancellationToken cancellationToken)	No especificado

Firma	Retorno
Update(int id, CreateHerdResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

REST Controller: AnimalsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Livestock Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IAnimalCommandService
queryService	IAnimalQueryService

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateAnimalResource resource, CancellationToken cancellationToken)	No especificado
Update(int id, CreateAnimalResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

Presentation Model: FarmHerdYAnimalViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: FarmHerdYAnimalController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.3.3. Application Layer

Esta capa coordina las capacidades de **gestión de fincas, hatos, animales e identificación QR**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: AnimalCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**

Campo	Detalle
Propósito	Orquestar gestión de fincas, hatos, animales e identificación QR sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IAnimalRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateAnimalCommand command, CancellationToken cancellationToken)	No especificado
Handle(UpdateAnimalCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteAnimalCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveFarmHerdYAnimalUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreateAnimalCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: AnimalRegisteredEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.3.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Livestock Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: HerdRepository / AnimalRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IHerdRepository / IAnimalRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
HerdRepository	AppDbContext context
AnimalRepository	AppDbContext context

Métodos u operaciones

Firma	Retorno
FindById(id)	No especificado
Add(entity)	No especificado
Update(entity)	No especificado
Delete(entity)	No especificado

Room Adapter: FarmHerdYAnimalDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado

Nombre	Tipo
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: FarmHerdYAnimalLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Device Adapter: MlKitQrScanner

Campo	Detalle
Producto y estado	Android / ML Kit — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de lectura QR mediante la cámara y Google ML Kit.

Atributos o dependencias

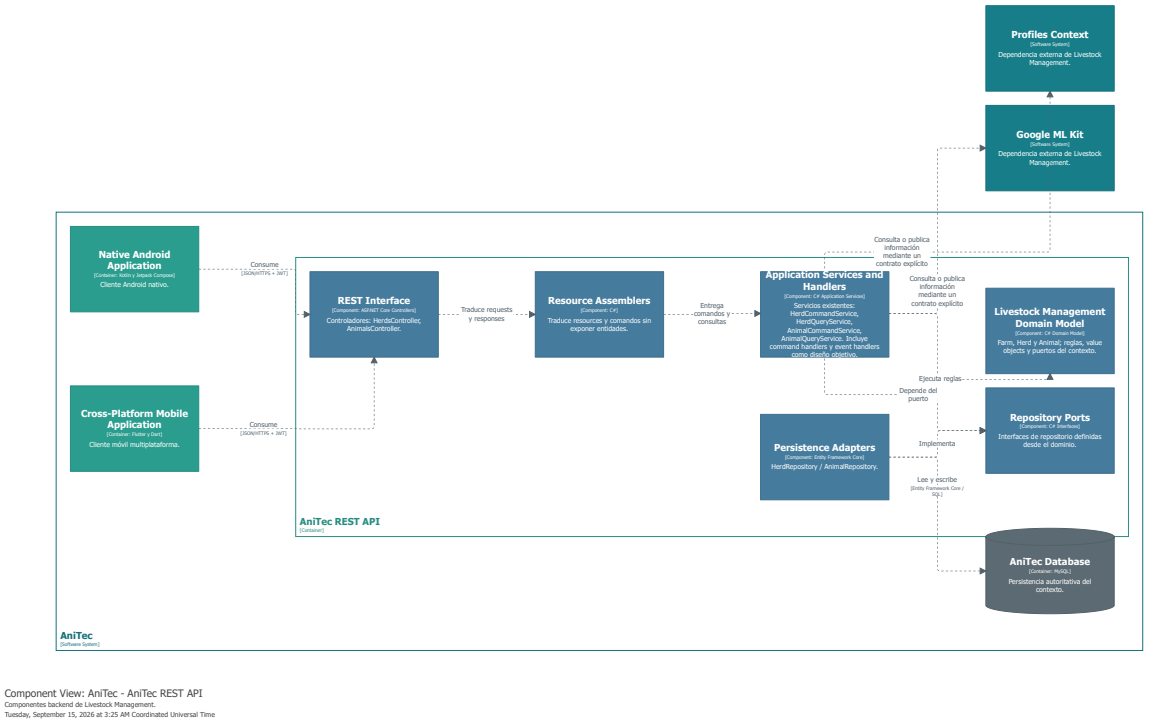
Nombre	Tipo
scannerClient	No especificado

Métodos u operaciones

Firma	Retorno
Scan(image)	QrIdentifier

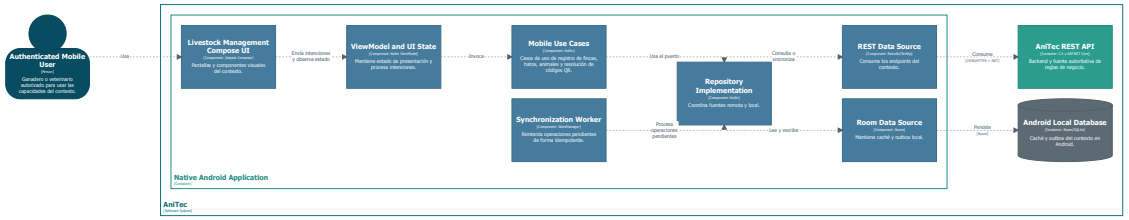
2.6.3.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC3-ApiComponents, BC3-AndroidComponents y BC3-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



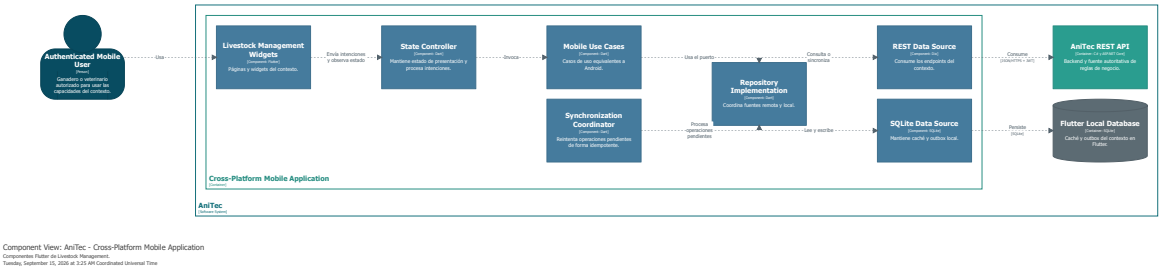
Component View: AniTec - AniTec REST API
Componentes backend de Livestock Management.
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.1. Componentes de la API para Livestock Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application
Componentes Android de Livestock Management.
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.2. Componentes Android para Livestock Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application
Componentes Flutter de Livestock Management.
Tuesday, September 15, 2026 at 3:25 AM Coordinated Universal Time

Figura 2.6.3.3. Componentes Flutter para Livestock Management. Fuente: elaboración propia con Structurizr DSL.

2.6.3.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.3.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

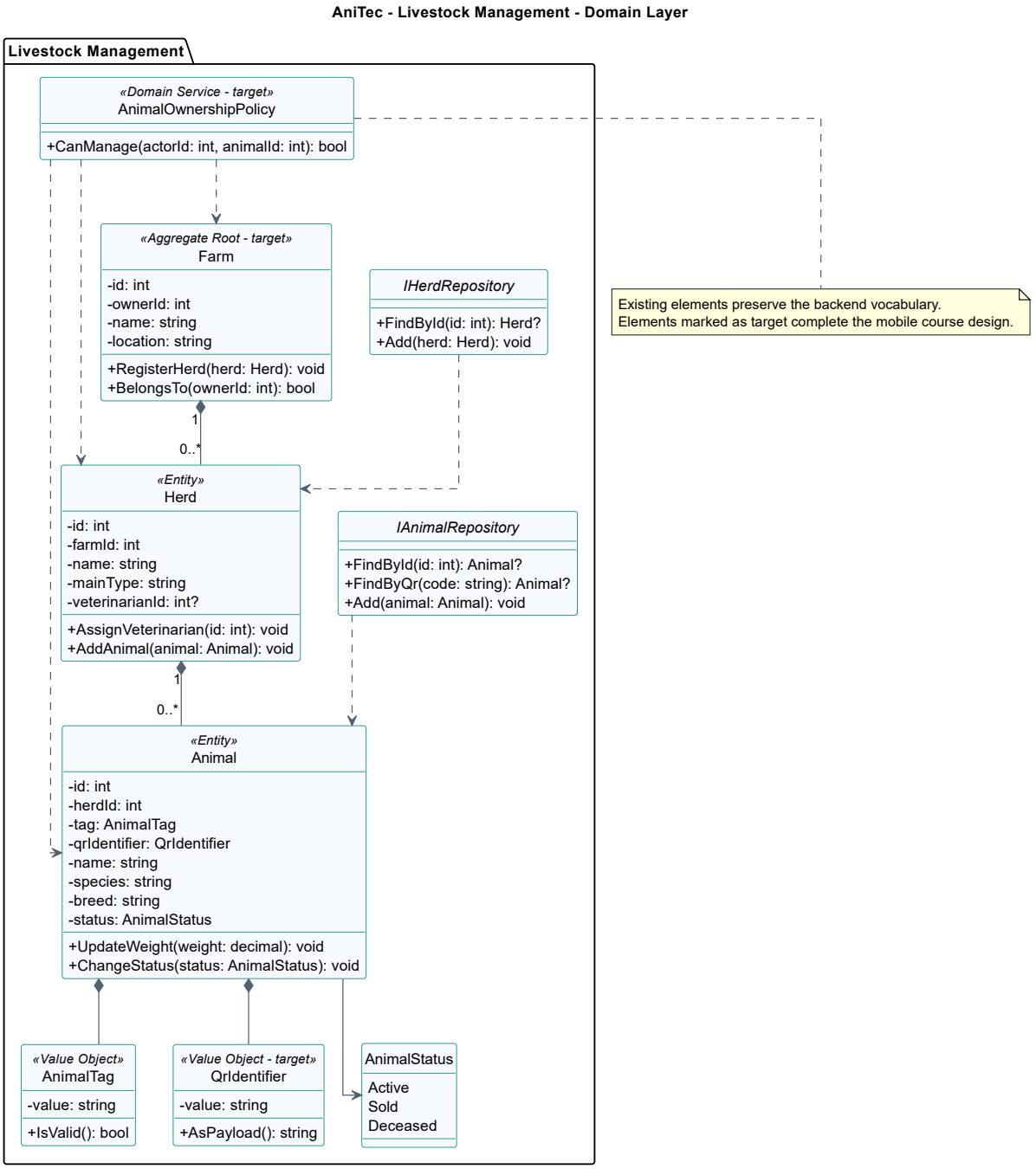


Figura 2.6.3.4. Domain Layer Class Diagram de Livestock Management. Fuente: elaboración propia con PlantUML.

2.6.3.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

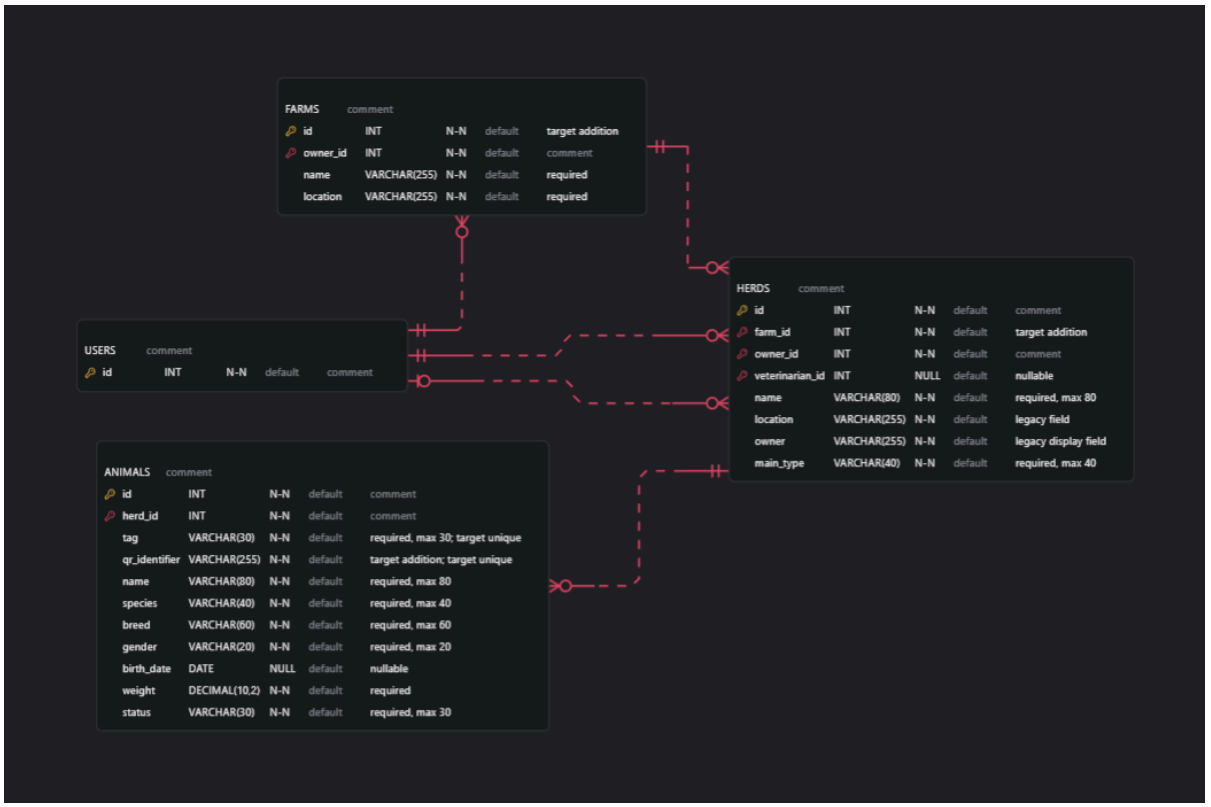


Figura 2.6.3.5. MySQL Database Design de Livestock Management. Fuente: elaboración propia a partir del esquema SQL.

Figura 2.6.3.6. Android Room Database Design de Livestock Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.4. Bounded Context: Sanitary Management

Conservar la historia sanitaria del animal y controlar el registro de diagnósticos, tratamientos, prescripciones y seguimientos autorizados.

La base implementada se encuentra en el módulo Sanitary de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.4.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Sanitary Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: HealthEvent

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Mantener un acontecimiento clínico dentro de la historia sanitaria.
Relaciones	HealthEvent compone Diagnosis; HealthEvent compone Treatment; HealthEvent compone Prescription; HealthEvent se relaciona con HealthEventType; SanitaryAuthorizationPolicy depende de HealthEvent; IHealthEventRepository depende de HealthEvent

Atributos o dependencias

Nombre	Tipo
id	int
animalId	int
type	HealthEventType
date	Date
veterinarianId	int?
description	string
diagnosis	Diagnosis
treatment	Treatment
prescription	Prescription
nextDueDate	Date?

Métodos u operaciones

Firma	Retorno
RescheduleFollowUp(date: Date)	void
UpdateClinicalData(diagnosis: Diagnosis, treatment: Treatment)	void

Value Object: Diagnosis

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar el diagnóstico clínico.
Relaciones	HealthEvent compone Diagnosis

Atributos o dependencias

Nombre	Tipo
description	string

Métodos u operaciones

Firma	Retorno
IsEmpty()	bool

Value Object: Treatment

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar instrucciones de tratamiento.
Relaciones	HealthEvent compone Treatment

Atributos o dependencias

Nombre	Tipo
instructions	string

Métodos u operaciones

Firma	Retorno
IsValid()	bool

Value Object: Prescription

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar la prescripción asociada a un evento sanitario.
Relaciones	HealthEvent compone Prescription

Atributos o dependencias

Nombre	Tipo
details	string

Métodos u operaciones

Firma	Retorno
IsRequired()	bool

Domain Service: SanitaryAuthorizationPolicy

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Decidir si un actor puede registrar información sanitaria.
Relaciones	SanitaryAuthorizationPolicy depende de HealthEvent

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
CanRegister(actorId: int, animalId: int)	bool

Repository Interface: IHealthEventRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de HealthEvent.
Relaciones	IHealthEventRepository depende de HealthEvent

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	HealthEvent?
FindByAnimal(animalId: int)	List~HealthEvent~
Add(event: HealthEvent)	void
Update(event: HealthEvent)	void

2.6.4.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: HealthEventsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Sanitary Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IHealthEventCommandService
queryService	IHealthEventQueryService

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateHealthEventResource resource, CancellationToken cancellationToken)	No especificado
Update(int id, CreateHealthEventResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

Presentation Model: HealthEventViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**

Campo	Detalle
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: HealthEventController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.4.3. Application Layer

Esta capa coordina las capacidades de **registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: HealthEventCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar registro y seguimiento de eventos sanitarios, diagnósticos y tratamientos sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IHealthEventRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateHealthEventCommand command, CancellationToken cancellationToken)	No especificado
Handle(UpdateHealthEventCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteHealthEventCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveHealthEventUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado

Nombre	Tipo
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreateHealthEventCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: HealthEventRegisteredEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.4.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Sanitary Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: HealthEventRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IHealthEventRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindById(id)	No especificado
Add(entity)	No especificado
Update(entity)	No especificado
Delete(entity)	No especificado

Room Adapter: HealthEventDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: HealthEventLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Offline Adapter: SanitaryOutboxStore

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de sincronización sanitaria sobre Room o SQLite.

Atributos o dependencias

Nombre	Tipo
serializer	database,

Métodos u operaciones

Firma	Retorno
Enqueue(event)	No especificado
Pending()	No especificado
MarkSynced(id)	No especificado

2.6.4.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC4-ApiComponents, BC4-AndroidComponents y BC4-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

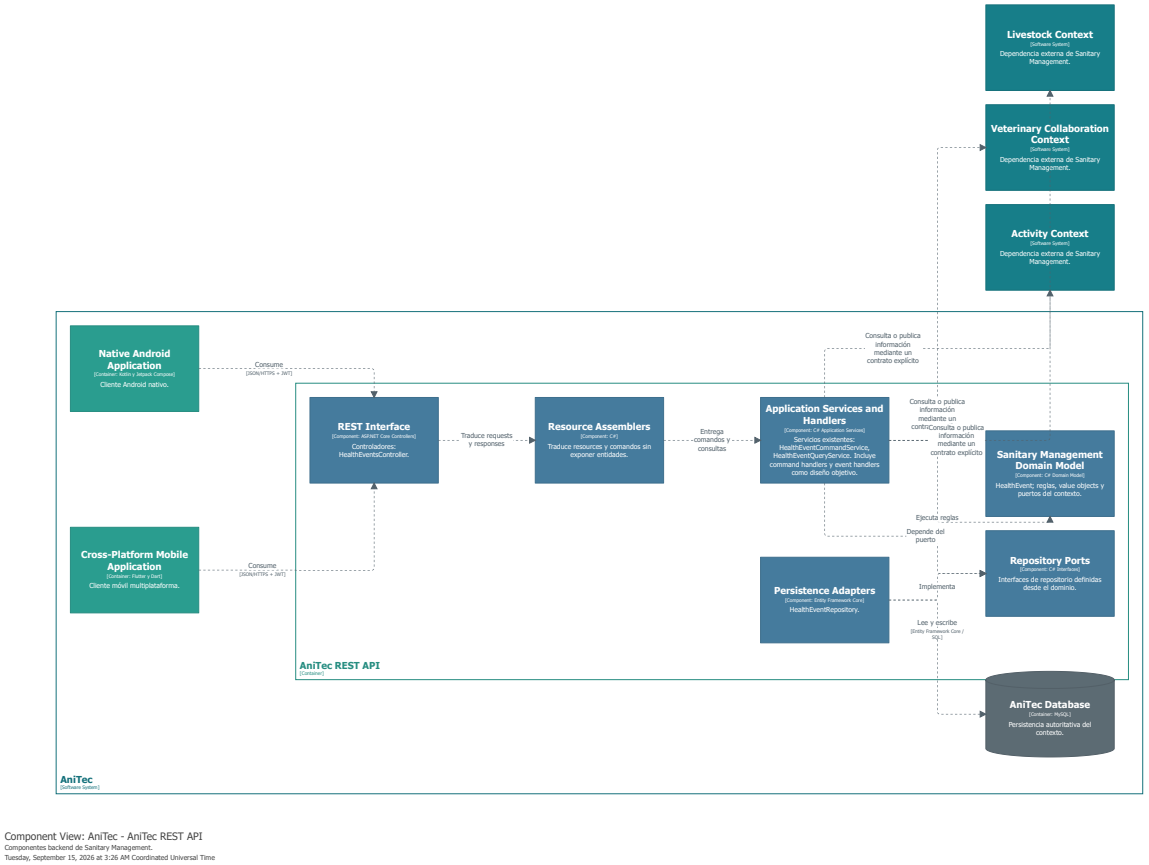


Figura 2.6.4.1. Componentes de la API para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

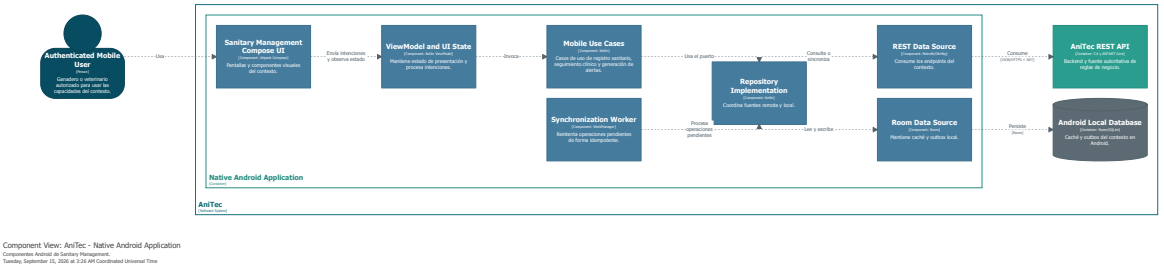


Figura 2.6.4.2. Componentes Android para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

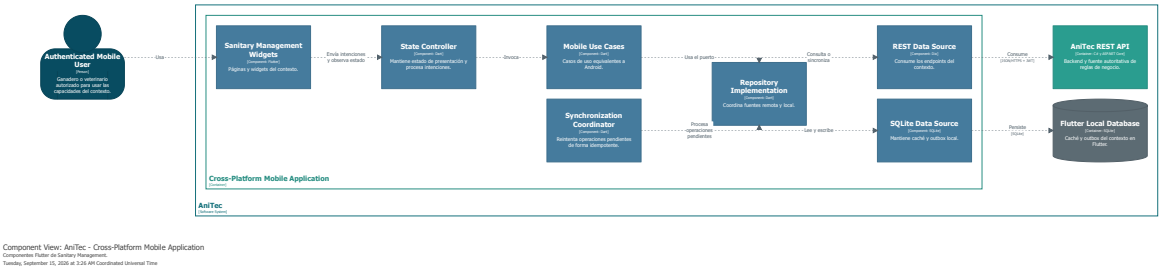


Figura 2.6.4.3. Componentes Flutter para Sanitary Management. Fuente: elaboración propia con Structurizr DSL.

2.6.4.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las

columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.4.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

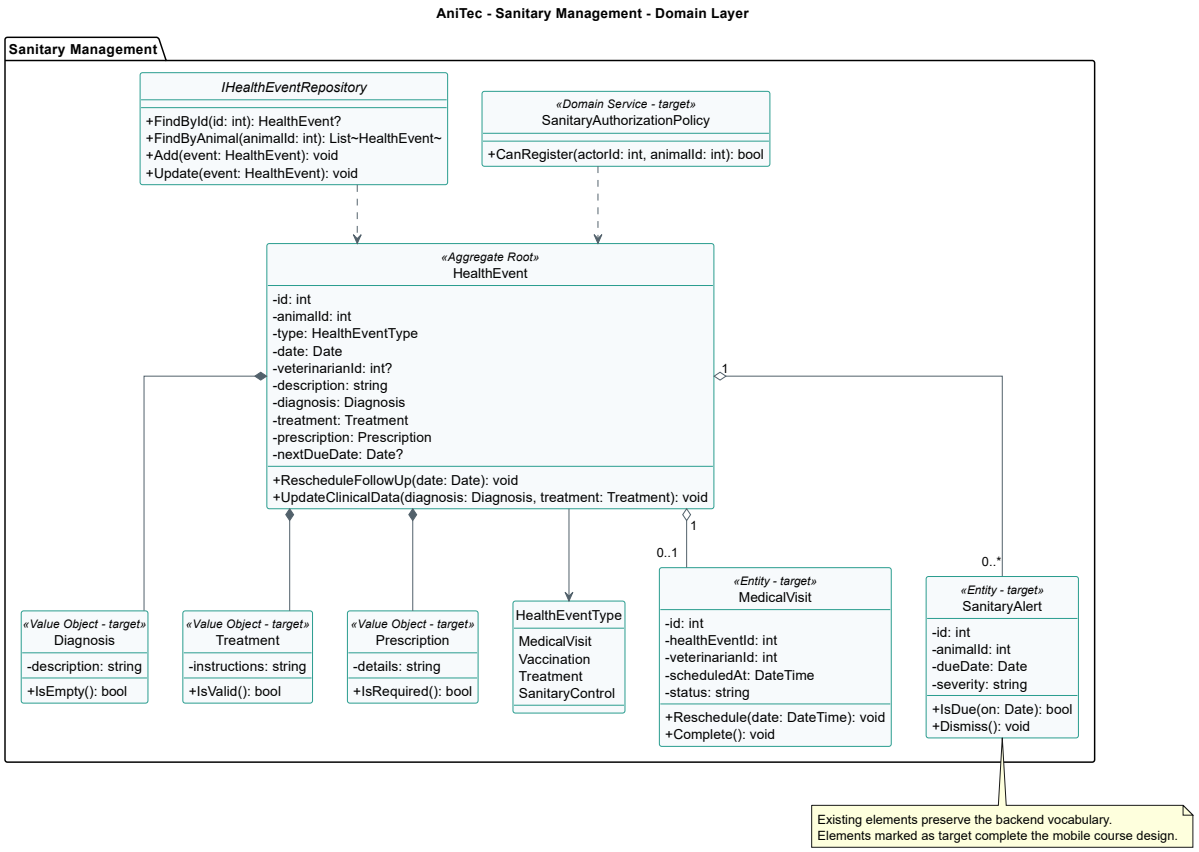


Figura 2.6.4.4. Domain Layer Class Diagram de Sanitary Management. Fuente: elaboración propia con PlantUML.

2.6.4.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

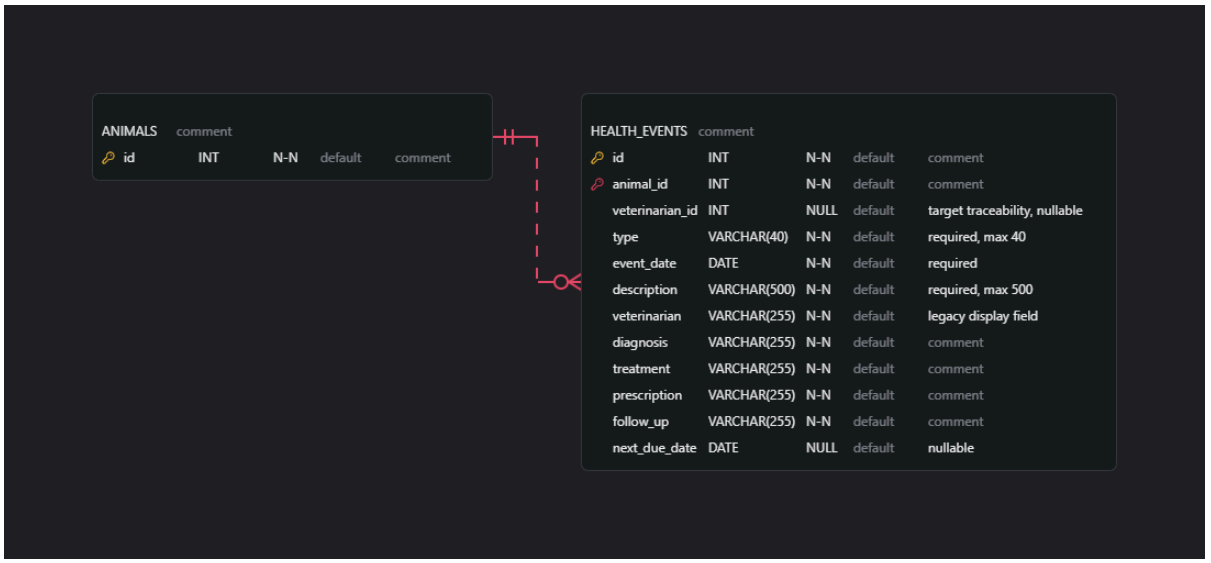


Figura 2.6.4.5. MySQL Database Design de Sanitary Management. Fuente: elaboración propia a partir del esquema SQL.

Figura 2.6.4.6. Android Room Database Design de Sanitary Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.5. Bounded Context: Veterinary Collaboration

Administrar solicitudes y autorizaciones entre ganaderos y veterinarios, delimitando clientes, pacientes y alcance de acceso.

La base implementada se encuentra en el módulo Clients de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.5.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Veterinary Collaboration**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: VeterinarianClient

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Controlar la relación y autorización entre veterinario y ganadero.
Relaciones	VeterinarianClient se relaciona con CollaborationStatus; VeterinarianClient compone AuthorizationScope; CollaborationPolicy depende de VeterinarianClient; IVeterinarianClientRepository depende de VeterinarianClient

Atributos o dependencias

Nombre	Tipo
id	int
veterinarianId	int
rancherId	int
status	CollaborationStatus
requestedAt	DateTime
acceptedAt	DateTime?
revokedAt	DateTime?
scope	AuthorizationScope

Métodos u operaciones

Firma	Retorno
Accept(at: DateTime)	void
Reject()	void
Revoke(at: DateTime)	void
IsActive()	bool

Enumeration: CollaborationStatus

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Definir los valores válidos de CollaborationStatus.
Relaciones	VeterinarianClient se relaciona con CollaborationStatus

Atributos o dependencias

Nombre	Tipo
Pending	No especificado
Accepted	No especificado
Rejected	No especificado
Revoked	No especificado

Métodos u operaciones

No aplica.

Value Object: AuthorizationScope

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Delimitar fincas, animales y operaciones autorizadas.
Relaciones	VeterinarianClient compone AuthorizationScope

Atributos o dependencias

Nombre	Tipo
farmIds	Set~int~
animalIds	Set~int~
canWriteHealthRecords	bool

Métodos u operaciones

Firma	Retorno
AllowsAnimal(animalId: int)	bool

Domain Service: CollaborationPolicy

Campo	Detalle
Producto y estado	Backend y modelos equivalentes Android/Flutter — **No implementado**
Propósito	Evaluar aceptación y acceso dentro de una colaboración.
Relaciones	CollaborationPolicy depende de VeterinarianClient

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
CanAccept(rancherId: int, relation: VeterinarianClient)	bool
CanAccess(veterinarianId: int, animalId: int)	bool

Repository Interface: IVeterinarianClientRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de VeterinarianClient.
Relaciones	IVeterinarianClientRepository depende de VeterinarianClient

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
Find(veterinarianId: int, rancherId: int)	VeterinarianClient?
FindByVeterinarian(id: int)	List~VeterinarianClient~
Add(relation: VeterinarianClient)	void
Update(relation: VeterinarianClient)	void

2.6.5.2. Interface Layer

Esta capa recibe las acciones relacionadas con **solicitud, aceptación, revocación y consulta de colaboraciones veterinarias** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: VeterinarianClientsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Veterinary Collaboration.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IVeterinarianClientCommandService
queryService	IVeterinarianClientQueryService
userQueryService	IUserQueryService
herdQueryService	IHerdQueryService
animalQueryService	IAnimalQueryService

Métodos u operaciones

Firma	Retorno
GetClients(int veterinarianId, CancellationToken cancellationToken)	No especificado
GetAvailableRanchers(int veterinarianId, CancellationToken cancellationToken)	No especificado
AddClient(int veterinarianId, int rancherId, CancellationToken cancellationToken)	No especificado
RemoveClient(int veterinarianId, int rancherId, CancellationToken cancellationToken)	No especificado

Presentation Model: VeterinarianClientViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado

Nombre	Tipo
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: VeterinarianClientController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.5.3. Application Layer

Esta capa coordina las capacidades de **solicitud, aceptación, revocación y consulta de colaboraciones veterinarias**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: VeterinarianClientCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar solicitud, aceptación, revocación y consulta de colaboraciones veterinarias sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IVeterinarianClientRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateVeterinarianClientCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteVeterinarianClientCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveVeterinarianClientUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: RequestCollaborationCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: CollaborationAcceptedEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.5.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Veterinary Collaboration** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: VeterinarianClientRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IVeterinarianClientRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindByVeterinarianIdAsync(int veterinarianId, CancellationToken cancellationToken)	No especificado
FindByVeterinarianIdAndRancherIdAsync(int veterinarianId, int rancherId, CancellationToken cancellationToken)	No especificado
ExistsByVeterinarianIdAndRancherIdAsync(int veterinarianId, int rancherId, CancellationToken cancellationToken)	No especificado

Room Adapter: VeterinarianClientDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado

Nombre	Tipo
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: VeterinarianClientLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Context Adapter: ProfilesDirectoryAdapter

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de directorio y consume Profile Management.

Atributos o dependencias

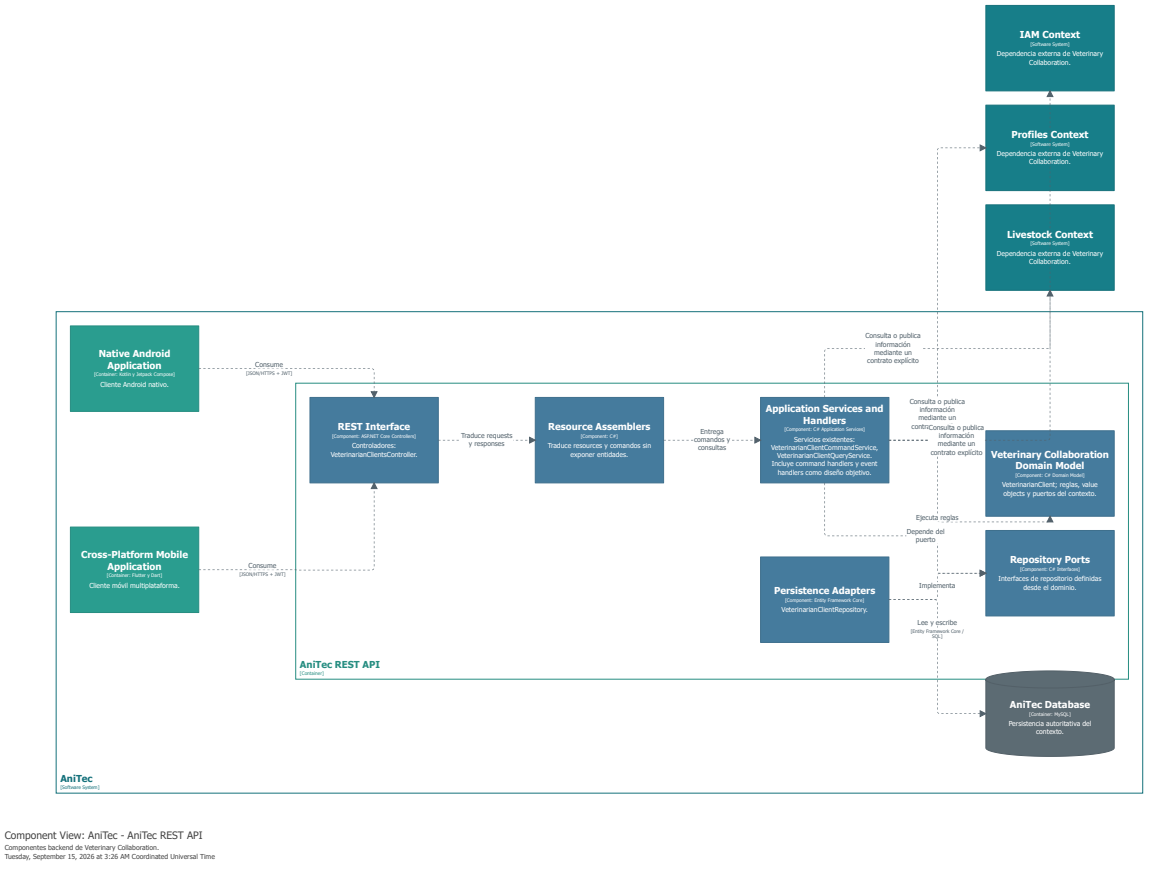
Nombre	Tipo
profilesFacade	No especificado

Métodos u operaciones

Firma	Retorno
GetDisplayName(userId)	No especificado
FindRanchers()	No especificado

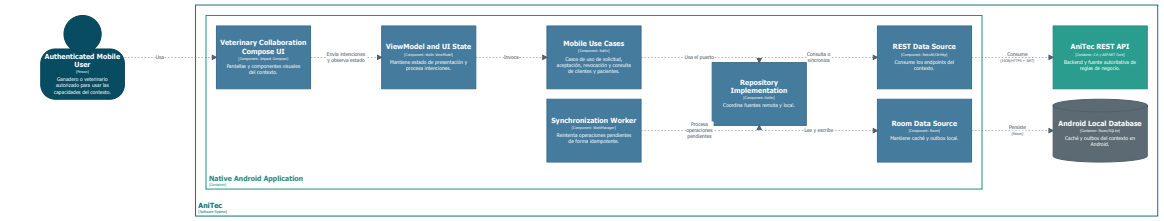
2.6.5.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC5-ApiComponents, BC5-AndroidComponents y BC5-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



Component View: AniTec - AniTec REST API
Componentes backend de Veterinary Collaboration.
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.5.1. Componentes de la API para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application
Componentes Android de Veterinary Collaboration.
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.5.2. Componentes Android para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.

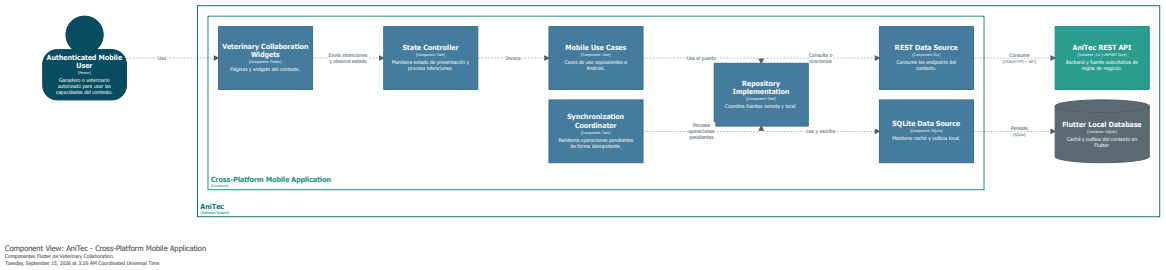


Figura 2.6.5.3. Componentes Flutter para Veterinary Collaboration. Fuente: elaboración propia con Structurizr DSL.

2.6.5.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.5.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

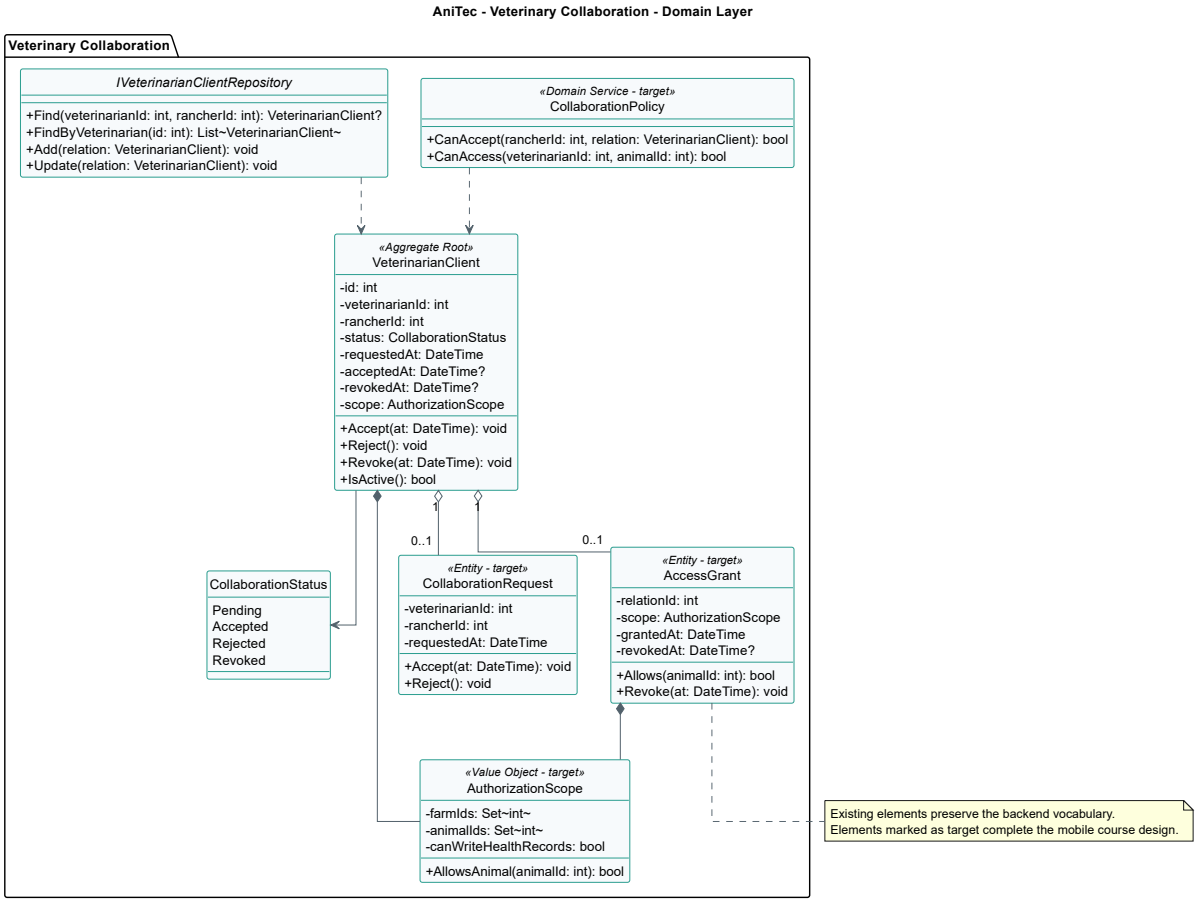


Figura 2.6.5.4. Domain Layer Class Diagram de Veterinary Collaboration. Fuente: elaboración propia con PlantUML.

2.6.5.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

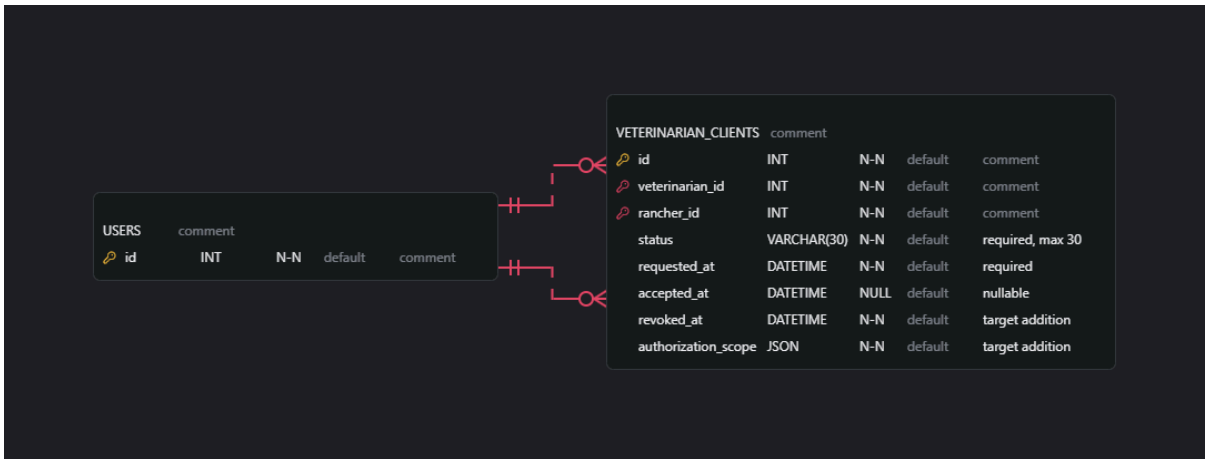


Figura 2.6.5.5. MySQL Database Design de Veterinary Collaboration. Fuente: elaboración propia a partir del esquema SQL.

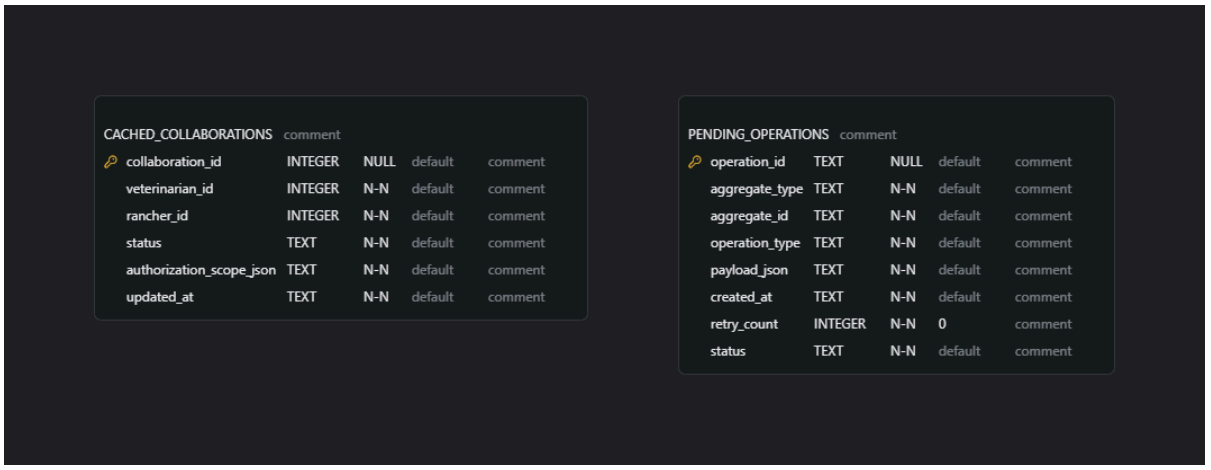


Figura 2.6.5.6. Android Room Database Design de Veterinary Collaboration. Fuente: elaboración propia a partir del esquema SQL.

2.6.6. Bounded Context: Activity Management

Planificar actividades ganaderas y sanitarias, controlar su estado y decidir cuándo corresponde generar un recordatorio.

La base implementada se encuentra en el módulo Activities de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.6.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Activity Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: FarmActivity

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Controlar el ciclo de vida de una actividad programada.
Relaciones	FarmActivity compone ActivitySchedule; FarmActivity se relaciona con ActivityPriority; FarmActivity se relaciona con ActivityStatus; ReminderPolicy depende de FarmActivity; IFarmActivityRepository depende de FarmActivity

Atributos o dependencias

Nombre	Tipo
id	int
ownerId	int?
veterinarianId	int?
animalId	int?
title	string
type	string
schedule	ActivitySchedule
priority	ActivityPriority
status	ActivityStatus

Métodos u operaciones

Firma	Retorno
Reschedule(schedule: ActivitySchedule)	void
Complete()	void
Cancel()	void

Value Object: ActivitySchedule

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar fecha programada y recordatorio.
Relaciones	FarmActivity compone ActivitySchedule

Atributos o dependencias

Nombre	Tipo
scheduledAt	DateTime
reminderAt	DateTime?

Métodos u operaciones

Firma	Retorno
IsUpcoming(now: DateTime)	bool

Enumeration: ActivityStatus

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Definir los valores válidos de ActivityStatus.
Relaciones	FarmActivity se relaciona con ActivityStatus

Atributos o dependencias

Nombre	Tipo
Pending	No especificado
InProgress	No especificado
Completed	No especificado
Cancelled	No especificado

Métodos u operaciones

No aplica.

Domain Service: ReminderPolicy

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Decidir cuándo debe emitirse una notificación.
Relaciones	ReminderPolicy depende de FarmActivity

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
ShouldNotify(activity: FarmActivity, now: DateTime)	bool

Repository Interface: IFarmActivityRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de FarmActivity.
Relaciones	IFarmActivityRepository depende de FarmActivity

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	FarmActivity?
FindUpcoming(userId: int)	List~FarmActivity~
Add(activity: FarmActivity)	void
Update(activity: FarmActivity)	void

2.6.6.2. Interface Layer

Esta capa recibe las acciones relacionadas con **programación, reprogramación, finalización y recordatorio de actividades** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: FarmActivitiesController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Activity Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IFarmActivityCommandService
queryService	IFarmActivityQueryService

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateFarmActivityResource resource, CancellationToken cancellationToken)	No especificado
Update(int id, CreateFarmActivityResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

Presentation Model: FarmActivityViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado

Firma	Retorno
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: FarmActivityController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.6.3. Application Layer

Esta capa coordina las capacidades de **programación, reprogramación, finalización y recordatorio de actividades**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: FarmActivityCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar programación, reprogramación, finalización y recordatorio de actividades sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IFarmActivityRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateFarmActivityCommand command, CancellationToken cancellationToken)	No especificado
Handle(UpdateFarmActivityCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteFarmActivityCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveFarmActivityUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreateFarmActivityCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: ActivityScheduledEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.6.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Activity Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: FarmActivityRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IFarmActivityRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindById(id)	No especificado
Add(entity)	No especificado
Update(entity)	No especificado
Delete(entity)	No especificado

Room Adapter: FarmActivityDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: FarmActivityLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Messaging Adapter: FirebaseReminderPublisher

Campo	Detalle
Producto y estado	Backend y Android — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de notificaciones mediante FCM y notificaciones locales.

Atributos o dependencias

Nombre	Tipo
localNotifier	fcmClient,

Métodos u operaciones

Firma	Retorno
Publish(reminder)	No especificado
ScheduleLocal(reminder)	No especificado

2.6.6.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC6-ApiComponents, BC6-AndroidComponents y BC6-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

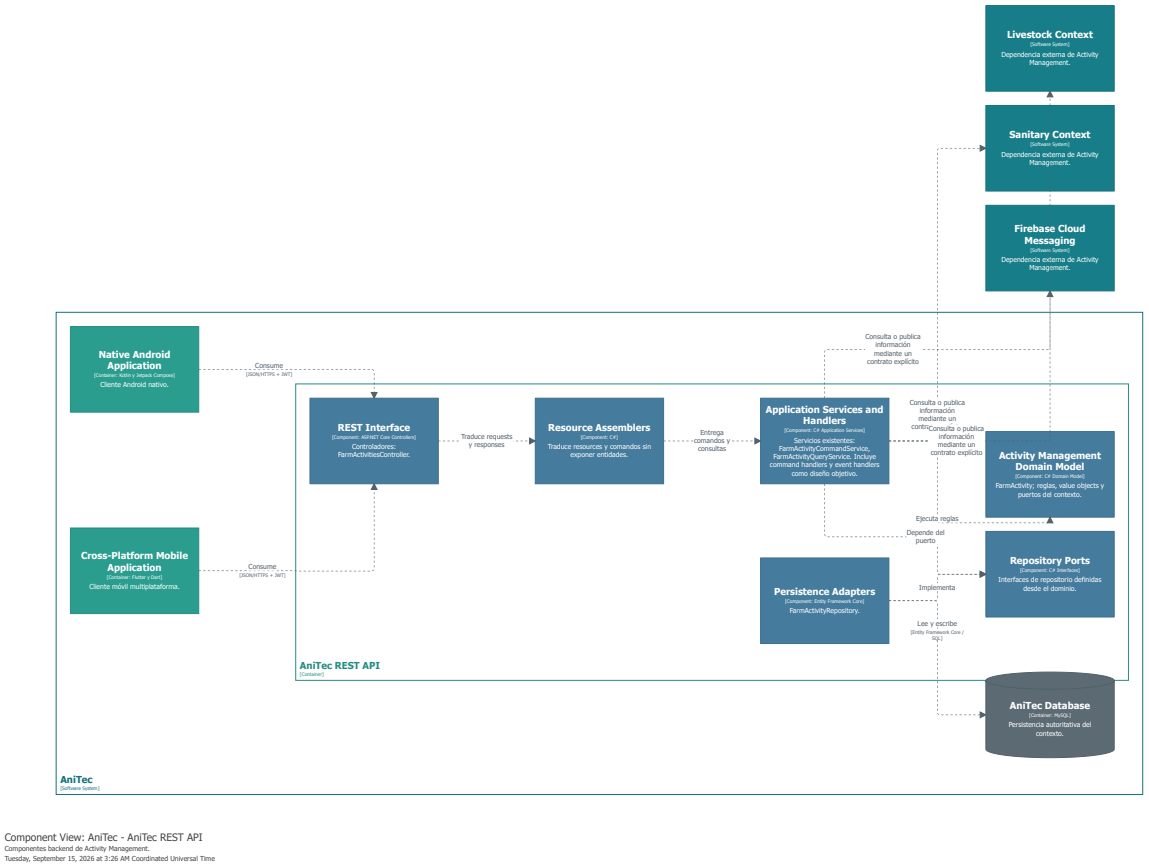


Figura 2.6.6.1. Componentes de la API para Activity Management. Fuente: elaboración propia con Structurizr DSL.

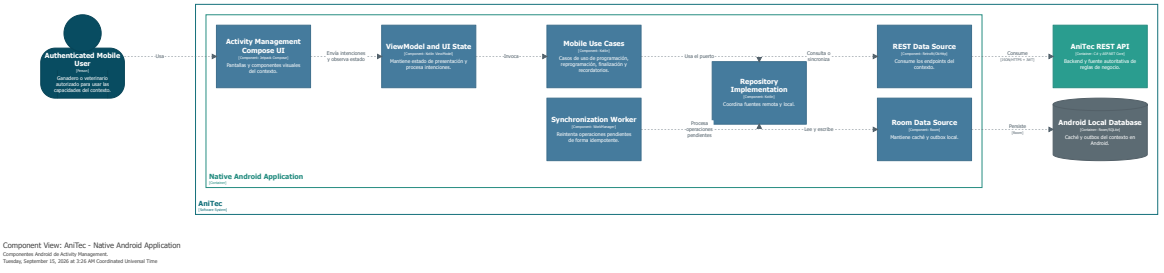


Figura 2.6.6.2. Componentes Android para Activity Management. Fuente: elaboración propia con Structurizr DSL.

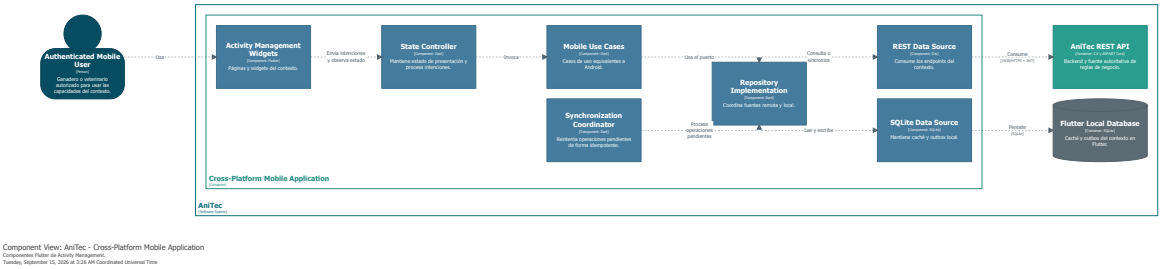


Figura 2.6.6.3. Componentes Flutter para Activity Management. Fuente: elaboración propia con Structurizr DSL.

2.6.6.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.6.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

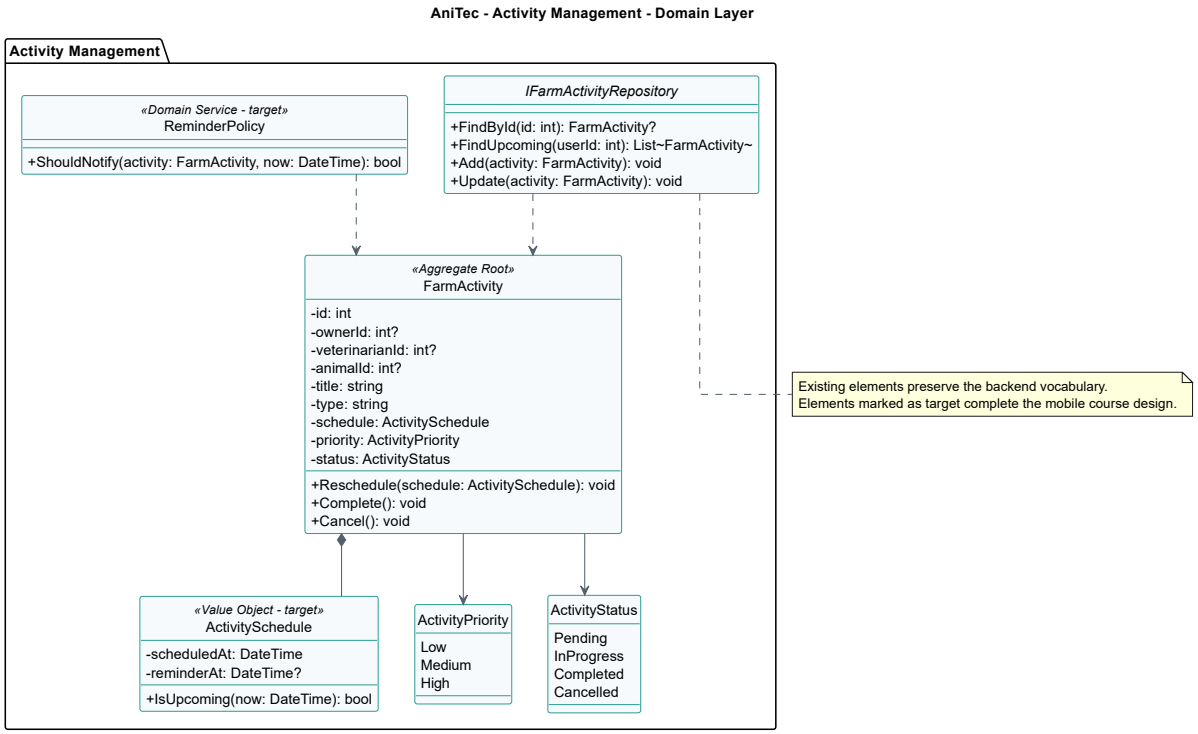


Figura 2.6.6.4. Domain Layer Class Diagram de Activity Management. Fuente: elaboración propia con PlantUML.

2.6.6.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

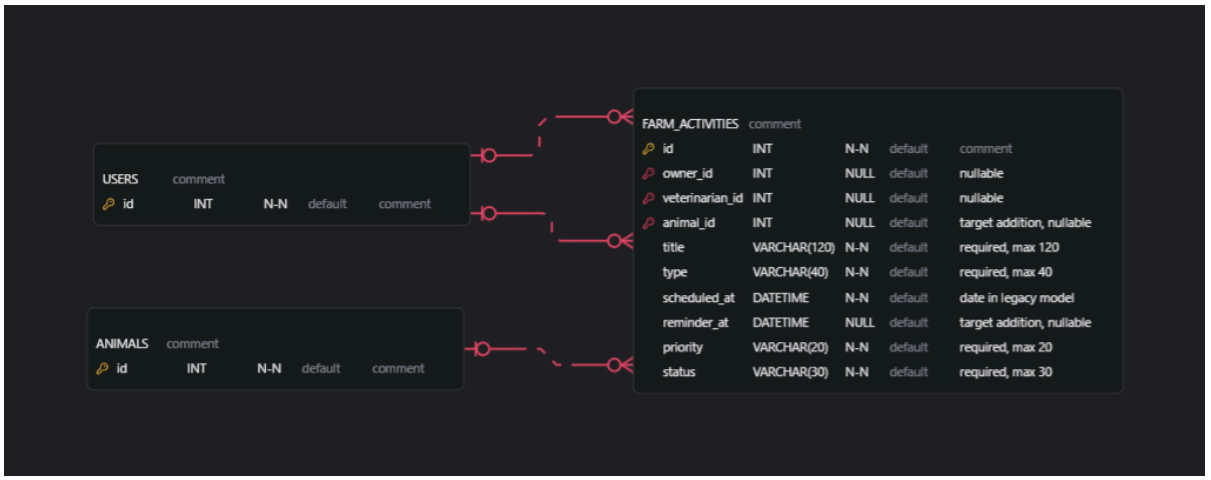


Figura 2.6.6.5. MySQL Database Design de Activity Management. Fuente: elaboración propia a partir del esquema SQL.

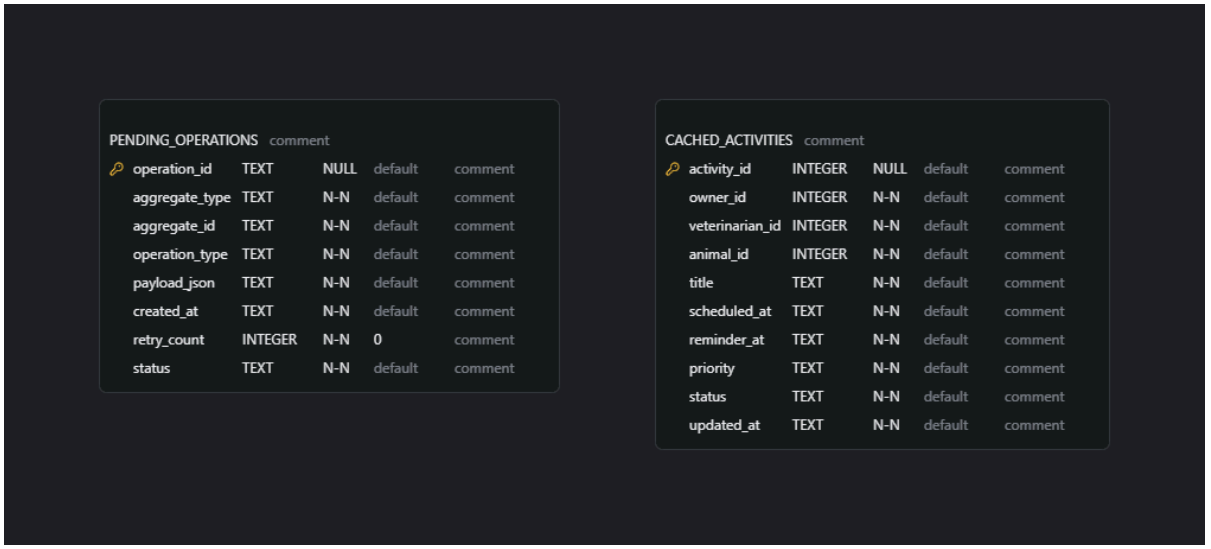


Figura 2.6.6.6. Android Room Database Design de Activity Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.7. Bounded Context: Financial Management

Registrar ingresos y egresos operativos y producir resúmenes económicos básicos para apoyar decisiones del ganadero.

La base implementada se encuentra en el módulo Financial de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.7.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Financial Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: FinancialRecord

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Mantener un ingreso o egreso del ganadero.
Relaciones	FinancialRecord compone Money; FinancialRecord se relaciona con FinancialRecordType; FinancialRecord compone FinancialCategory; FinancialSummary depende de FinancialRecord; IFinancialRecordRepository depende de FinancialRecord

Atributos o dependencias

Nombre	Tipo
id	int
ownerId	int
type	FinancialRecordType
category	FinancialCategory
amount	Money
date	Date
description	string

Métodos u operaciones

Firma	Retorno
ChangeAmount(amount: Money)	void

Firma	Retorno
ChangeCategory(category: FinancialCategory)	void

Value Object: Money

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar un importe junto con su moneda.
Relaciones	FinancialRecord compone Money

Atributos o dependencias

Nombre	Tipo
amount	decimal
currency	string

Métodos u operaciones

Firma	Retorno
Add(other: Money)	Money
IsPositive()	bool

Value Object: FinancialCategory

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Clasificar un movimiento financiero.
Relaciones	FinancialRecord compone FinancialCategory

Atributos o dependencias

Nombre	Tipo
name	string

Métodos u operaciones

Firma	Retorno
IsValid()	bool

Domain Service: FinancialSummary

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Calcular totales y balance para un conjunto de movimientos.
Relaciones	FinancialSummary depende de FinancialRecord

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
Calculate(records: List~FinancialRecord~)	Money

Repository Interface: IFinancialRecordRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de FinancialRecord.
Relaciones	IFinancialRecordRepository depende de FinancialRecord

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	FinancialRecord?
FindByOwner(ownerId: int)	List~FinancialRecord~
Add(record: FinancialRecord)	void
Update(record: FinancialRecord)	void

2.6.7.2. Interface Layer

Esta capa recibe las acciones relacionadas con **registro de ingresos y egresos, actualización y cálculo de resúmenes** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: FinancialRecordsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Financial Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	IFinancialRecordCommandService
queryService	IFinancialRecordQueryService

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateFinancialRecordResource resource, CancellationToken cancellationToken)	No especificado
Update(int id, CreateFinancialRecordResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

Presentation Model: FinancialRecordViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado

Firma	Retorno
RetrySync()	No especificado

State Controller: FinancialRecordController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.7.3. Application Layer

Esta capa coordina las capacidades de **registro de ingresos y egresos, actualización y cálculo de resúmenes**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: FinancialRecordCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar registro de ingresos y egresos, actualización y cálculo de resúmenes sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IFinancialRecordRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateFinancialRecordCommand command, CancellationToken cancellationToken)	No especificado
Handle(UpdateFinancialRecordCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteFinancialRecordCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveFinancialRecordUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreateFinancialRecordCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: FinancialRecordRegisteredEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.7.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Financial Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: FinancialRecordRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IFinancialRecordRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindById(id)	No especificado
Add(entity)	No especificado
Update(entity)	No especificado
Delete(entity)	No especificado

Room Adapter: FinancialRecordDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: FinancialRecordLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Offline Adapter: FinancialOutboxStore

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa el puerto de sincronización financiera sobre Room o SQLite.

Atributos o dependencias

Nombre	Tipo
serializer	database,

Métodos u operaciones

Firma	Retorno
Enqueue(record)	No especificado
Pending()	No especificado
MarkSynced(id)	No especificado

2.6.7.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC7-ApiComponents, BC7-AndroidComponents y BC7-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

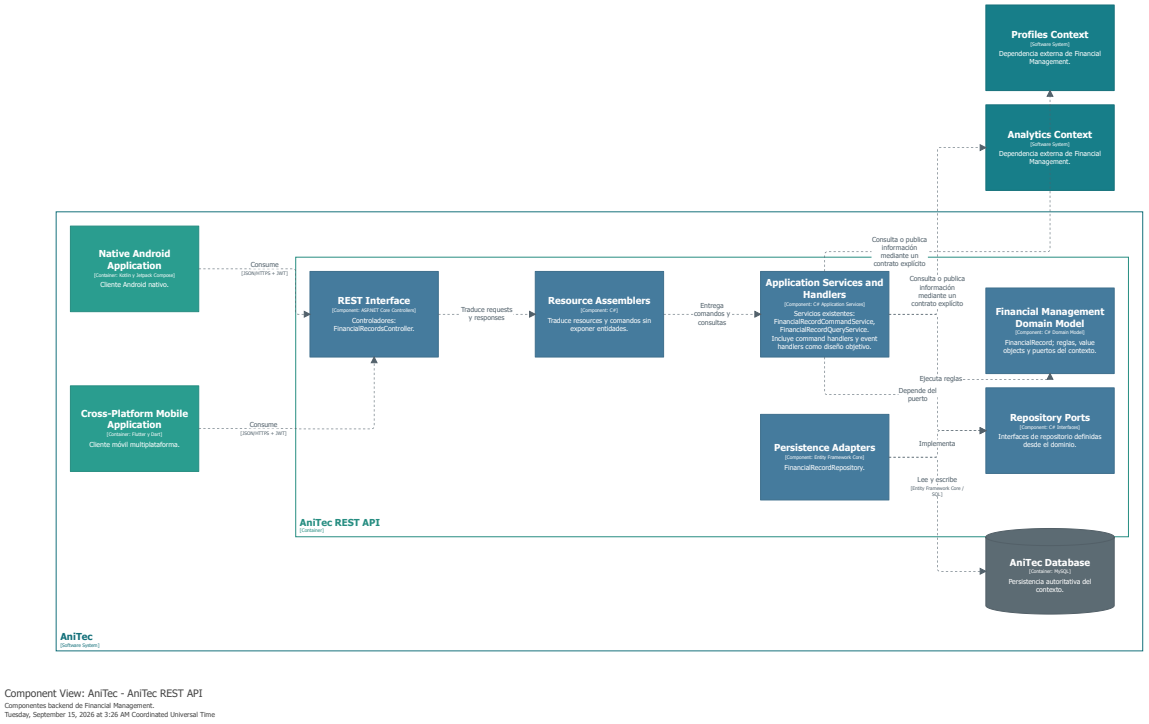


Figura 2.6.7.1. Componentes de la API para Financial Management. Fuente: elaboración propia con Structurizr DSL.

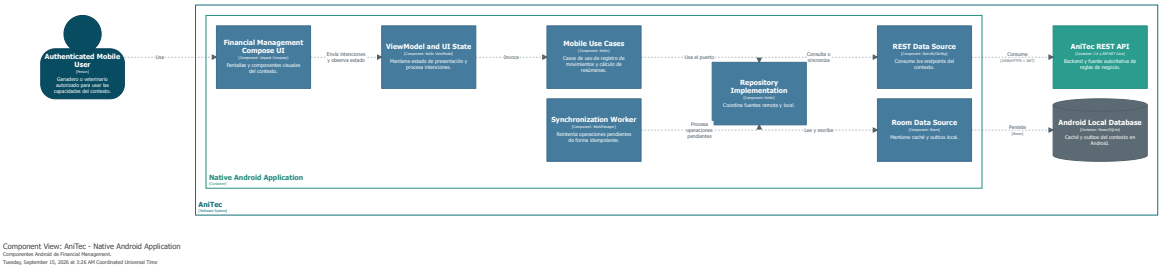


Figura 2.6.7.2. Componentes Android para Financial Management. Fuente: elaboración propia con Structurizr DSL.

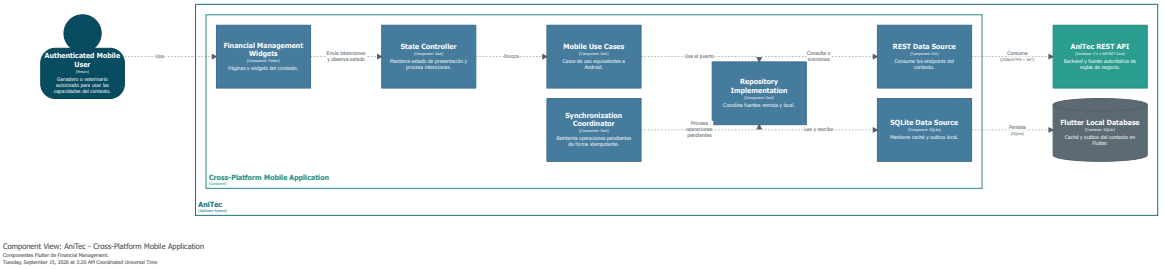


Figura 2.6.7.3. Componentes Flutter para Financial Management. Fuente: elaboración propia con Structurizr DSL.

2.6.7.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.7.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

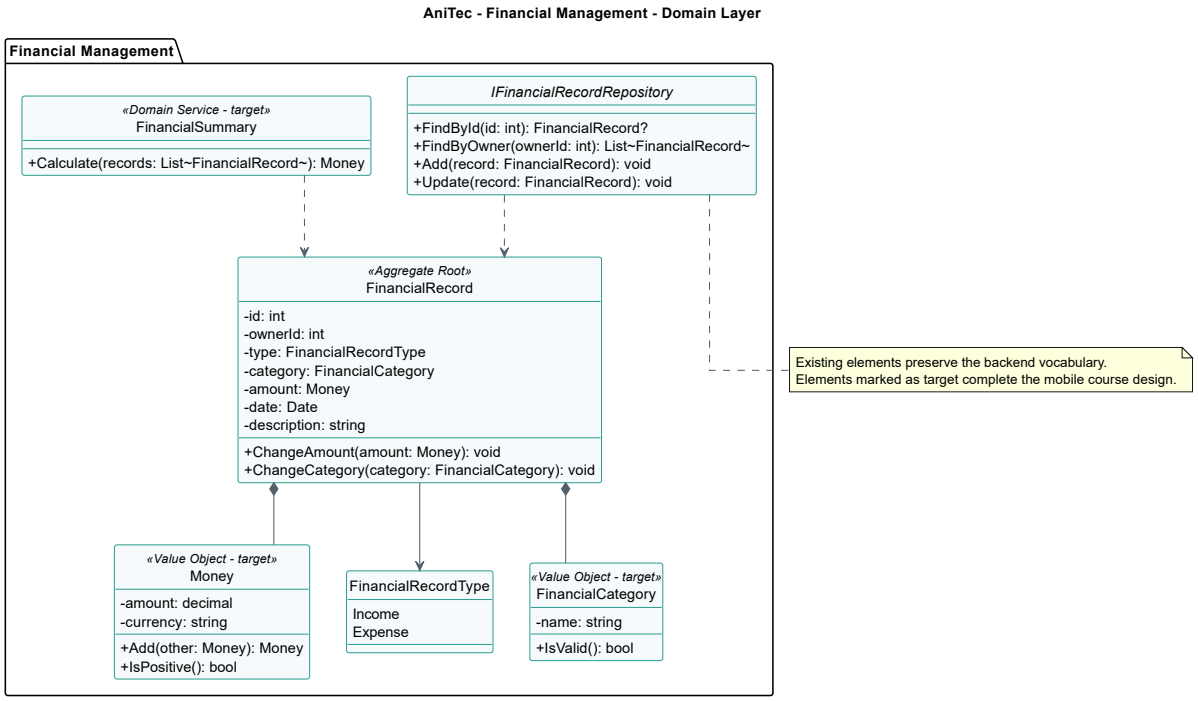


Figura 2.6.7.4. Domain Layer Class Diagram de Financial Management. Fuente: elaboración propia con PlantUML.

2.6.7.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

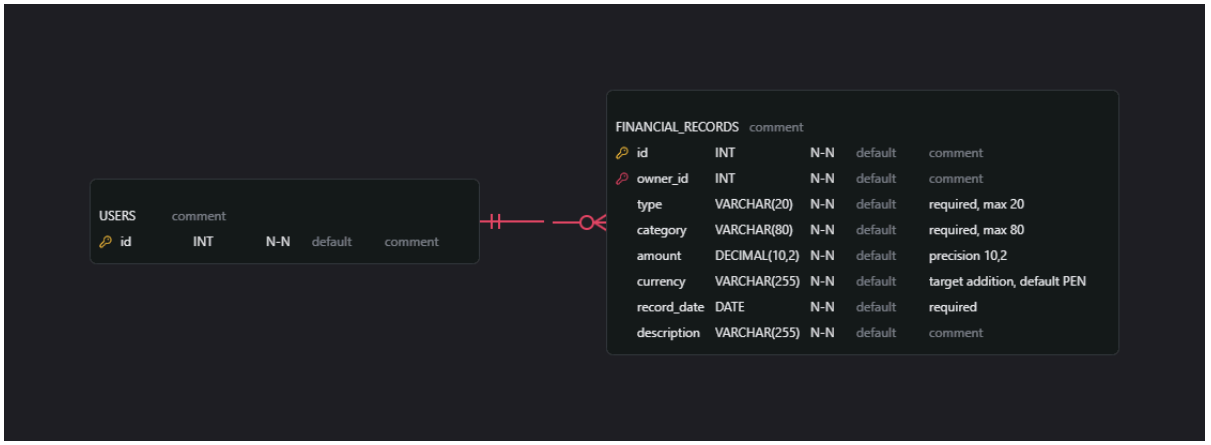


Figura 2.6.7.5. MySQL Database Design de Financial Management. Fuente: elaboración propia a partir del esquema SQL.

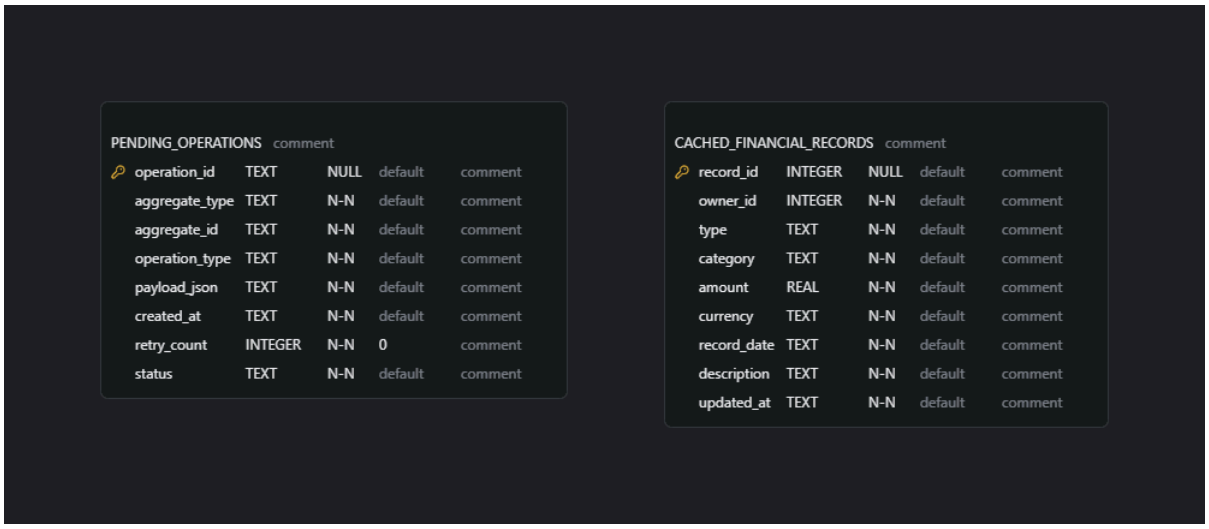


Figura 2.6.7.6. Android Room Database Design de Financial Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.8. Bounded Context: Subscription Management

Administrar planes, vigencia de suscripciones y pagos confirmados sin introducir conceptos propios de Stripe en el dominio.

La base implementada se encuentra en el módulo Subscriptions de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.8.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Subscription Management**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Entity: SubscriptionPlan

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Definir condiciones y límites de un plan comercial.
Relaciones	SubscriptionPlan compone Money

Atributos o dependencias

Nombre	Tipo
id	int
name	string
price	Money
providerPriceId	string
maxAnimals	int
isActive	bool

Métodos u operaciones

Firma	Retorno
Deactivate()	void
ChangePrice(price: Money)	void

Aggregate Root: Subscription

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Controlar vigencia y estado de la suscripción de un usuario.
Relaciones	Subscription compone Payment; Subscription se relaciona con SubscriptionStatus; ISubscriptionRepository depende de Subscription

Atributos o dependencias

Nombre	Tipo
id	int
userId	int
planId	int
status	SubscriptionStatus
startedAt	Date
endsAt	Date?

Métodos u operaciones

Firma	Retorno
Activate(start: Date, end: Date?)	void
Cancel(end: Date)	void
IsActive(on: Date)	bool

Entity: Payment

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Registrar el resultado de un pago.
Relaciones	Subscription compone Payment; Payment se relaciona con PaymentStatus; Payment compone Money

Atributos o dependencias

Nombre	Tipo
id	int
subscriptionId	int
userId	int

Nombre	Tipo
amount	Money
providerPaymentId	string
status	PaymentStatus
paidAt	DateTime

Métodos u operaciones

Firma	Retorno
Confirm(providerId: string, paidAt: DateTime)	void
Reject()	void

Value Object: Money

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar un importe junto con su moneda.
Relaciones	Payment compone Money; SubscriptionPlan compone Money

Atributos o dependencias

Nombre	Tipo
amount	decimal
currency	string

Métodos u operaciones

Firma	Retorno
IsPositive()	bool

Enumeration: SubscriptionStatus

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Definir los valores válidos de SubscriptionStatus.
Relaciones	Subscription se relaciona con SubscriptionStatus

Atributos o dependencias

Nombre	Tipo
Pending	No especificado
Active	No especificado
Cancelled	No especificado
Expired	No especificado

Métodos u operaciones

No aplica.

Repository Interface: ISubscriptionRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de Subscription.
Relaciones	ISubscriptionRepository depende de Subscription

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindById(id: int)	Subscription?
FindActiveByUser(userId: int)	Subscription?
Add(subscription: Subscription)	void
Update(subscription: Subscription)	void

Domain Port: PaymentGateway

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **No implementado**
Propósito	Abstraer el proveedor externo de cobros.
Relaciones	Es implementado por StripePaymentGateway en Infrastructure Layer.

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
CreateCheckout(subscription, plan)	No especificado
ConfirmPayment(reference)	No especificado

2.6.8.2. Interface Layer

Esta capa recibe las acciones relacionadas con **consulta de planes, contratación, pago y actualización de suscripciones** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: SubscriptionsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Subscription Management.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
commandService	ISubscriptionCommandService
queryService	ISubscriptionQueryService
planQueryService	ISubscriptionPlanQueryService
paymentCommandService	IPaymentCommandService
paymentQueryService	IPaymentQueryService
configuration	IConfiguration

Métodos u operaciones

Firma	Retorno
GetAll(CancellationToken cancellationToken)	No especificado
GetById(int id, CancellationToken cancellationToken)	No especificado
Create(CreateSubscriptionResource resource, CancellationToken cancellationToken)	No especificado
GetActiveByUser(int userId, CancellationToken cancellationToken)	No especificado
GetPaymentsByUser(int userId, CancellationToken cancellationToken)	No especificado
MockCheckout(MockCheckoutResource resource, CancellationToken cancellationToken)	No especificado
CreateStripeCheckout(StripeCheckoutResource resource, CancellationToken cancellationToken)	No especificado

Firma	Retorno
ConfirmStripeCheckout(string sessionId, CancellationToken cancellationToken)	No especificado
Update(int id, CreateSubscriptionResource resource, CancellationToken cancellationToken)	No especificado
Delete(int id, CancellationToken cancellationToken)	No especificado

Presentation Model: SubscriptionViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: SubscriptionController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.8.3. Application Layer

Esta capa coordina las capacidades de **consulta de planes, contratación, pago y actualización de suscripciones**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: SubscriptionCommandService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar consulta de planes, contratación, pago y actualización de suscripciones sin contener reglas del dominio.
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	ISubscriptionRepository
unitOfWork	IUnitOfWork

Métodos u operaciones

Firma	Retorno
Handle(CreateSubscriptionCommand command, CancellationToken cancellationToken)	No especificado
Handle(UpdateSubscriptionCommand command, CancellationToken cancellationToken)	No especificado
Handle(DeleteSubscriptionCommand command, CancellationToken cancellationToken)	No especificado

Use Case: ObserveSubscriptionUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**

Campo	Detalle
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Command Handler: CreatePaymentCommandHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Ejecutar una intención concreta, aplicar reglas del agregado y confirmar la transacción.
Relaciones	Consume un Command, carga el aggregate mediante su repository y puede publicar un Domain Event.

Atributos o dependencias

Nombre	Tipo
repository	No especificado
unitOfWork	No especificado
domainPolicy	No especificado

Métodos u operaciones

Firma	Retorno
Handle(command)	Result

Event Handler: PaymentConfirmedEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.8.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Subscription Management** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: SubscriptionPlanRepository / SubscriptionRepository / PaymentRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa ISubscriptionPlanRepository / ISubscriptionRepository / IPaymentRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
SubscriptionPlanRepository	AppDbContext context
SubscriptionRepository	AppDbContext context
PaymentRepository	AppDbContext context

Métodos u operaciones

Firma	Retorno
PaymentRepository.FindByUserIdAsync(int userId, CancellationToken cancellationToken)	No especificado
PaymentRepository.FindByProviderPaymentIdAsync(string providerPaymentId, CancellationToken cancellationToken)	No especificado

Room Adapter: SubscriptionDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: SubscriptionLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado

Nombre	Tipo
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Anti-Corruption Layer: StripePaymentGateway

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa PaymentGateway y traduce Stripe al lenguaje del dominio.

Atributos o dependencias

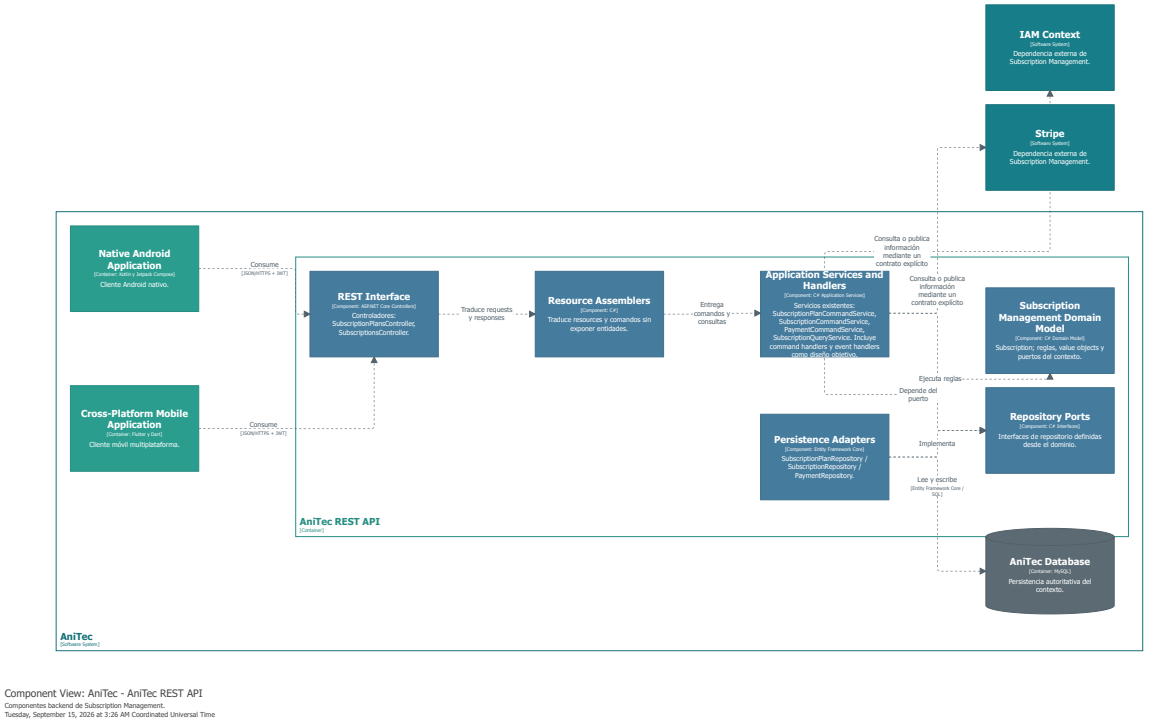
Nombre	Tipo
webhookSecret	stripeClient,

Métodos u operaciones

Firma	Retorno
CreateCheckout(command)	No especificado
ConfirmWebhook(payload)	No especificado
Refund(paymentId)	No especificado

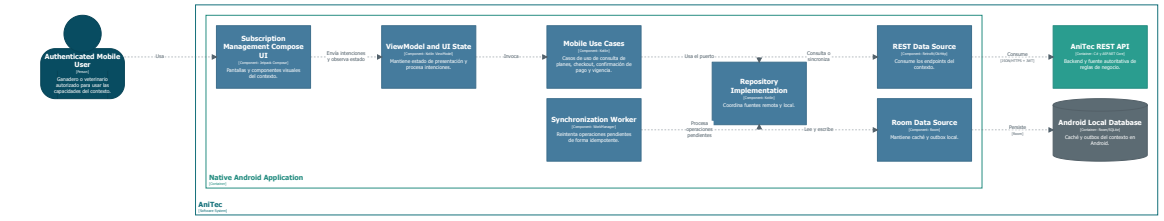
2.6.8.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC8-ApiComponents, BC8-AndroidComponents y BC8-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.



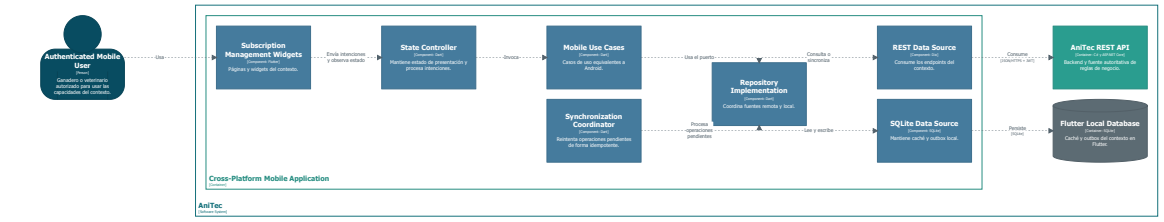
Component View: AniTec - AniTec REST API
Componentes backend de Subscription Management.
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.1. Componentes de la API para Subscription Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Native Android Application
Componentes Android de Subscription Management.
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.2. Componentes Android para Subscription Management. Fuente: elaboración propia con Structurizr DSL.



Component View: AniTec - Cross-Platform Mobile Application
Componentes Flutter de Subscription Management.
Tuesday, September 15, 2026 at 3:26 AM Coordinated Universal Time

Figura 2.6.8.3. Componentes Flutter para Subscription Management. Fuente: elaboración propia con Structurizr DSL.

2.6.8.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.8.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

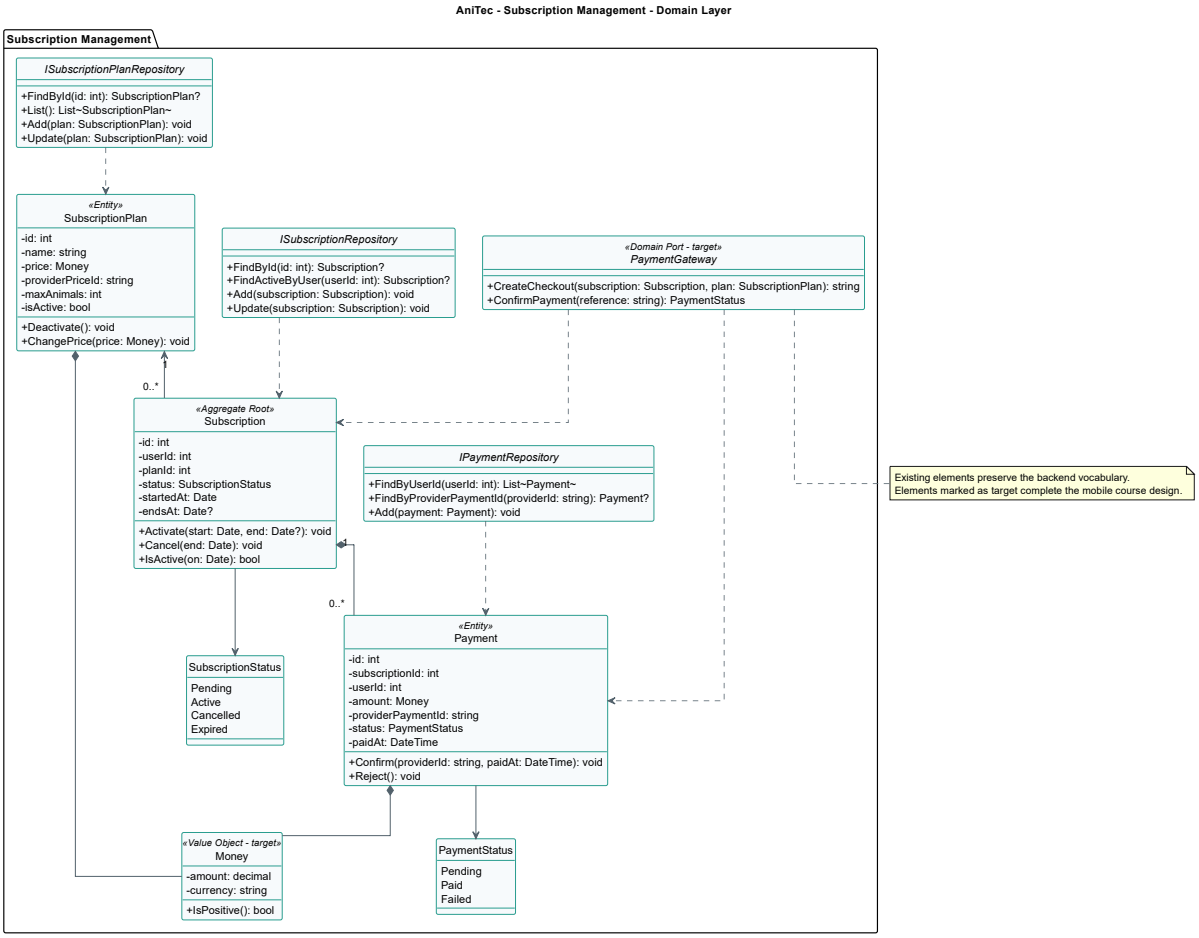


Figura 2.6.8.4. Domain Layer Class Diagram de Subscription Management. Fuente: elaboración propia con PlantUML.

2.6.8.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

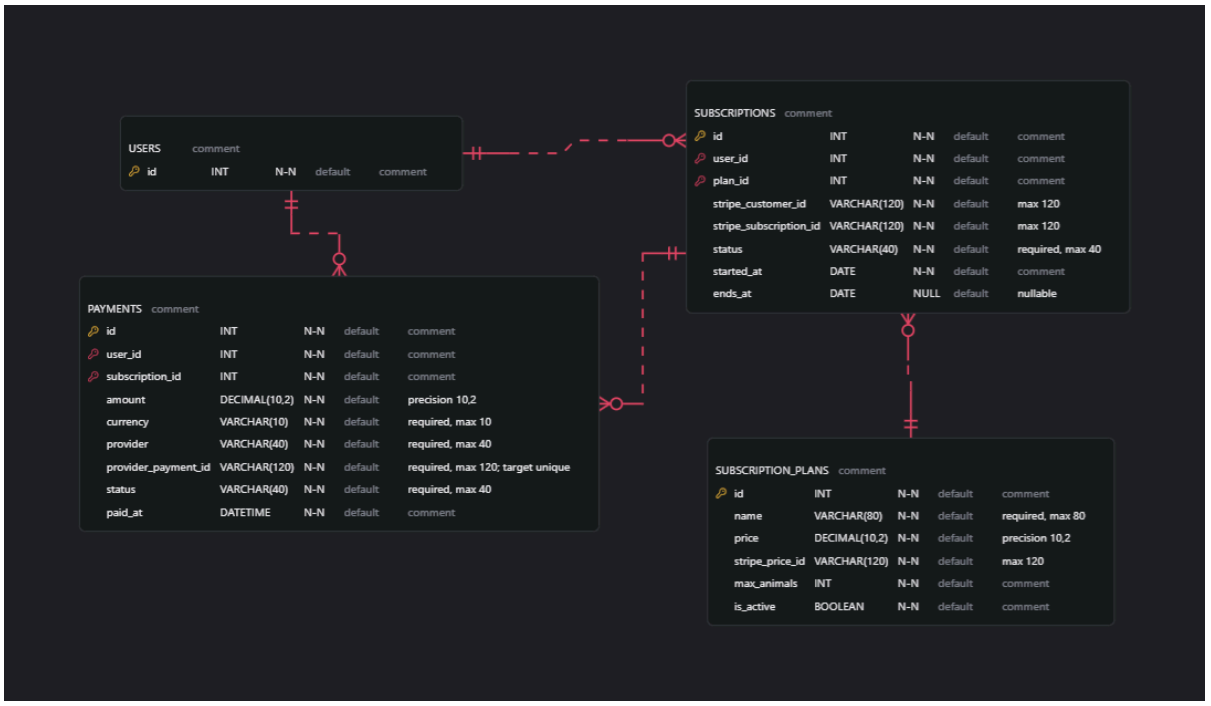


Figura 2.6.8.5. MySQL Database Design de Subscription Management. Fuente: elaboración propia a partir del esquema SQL.

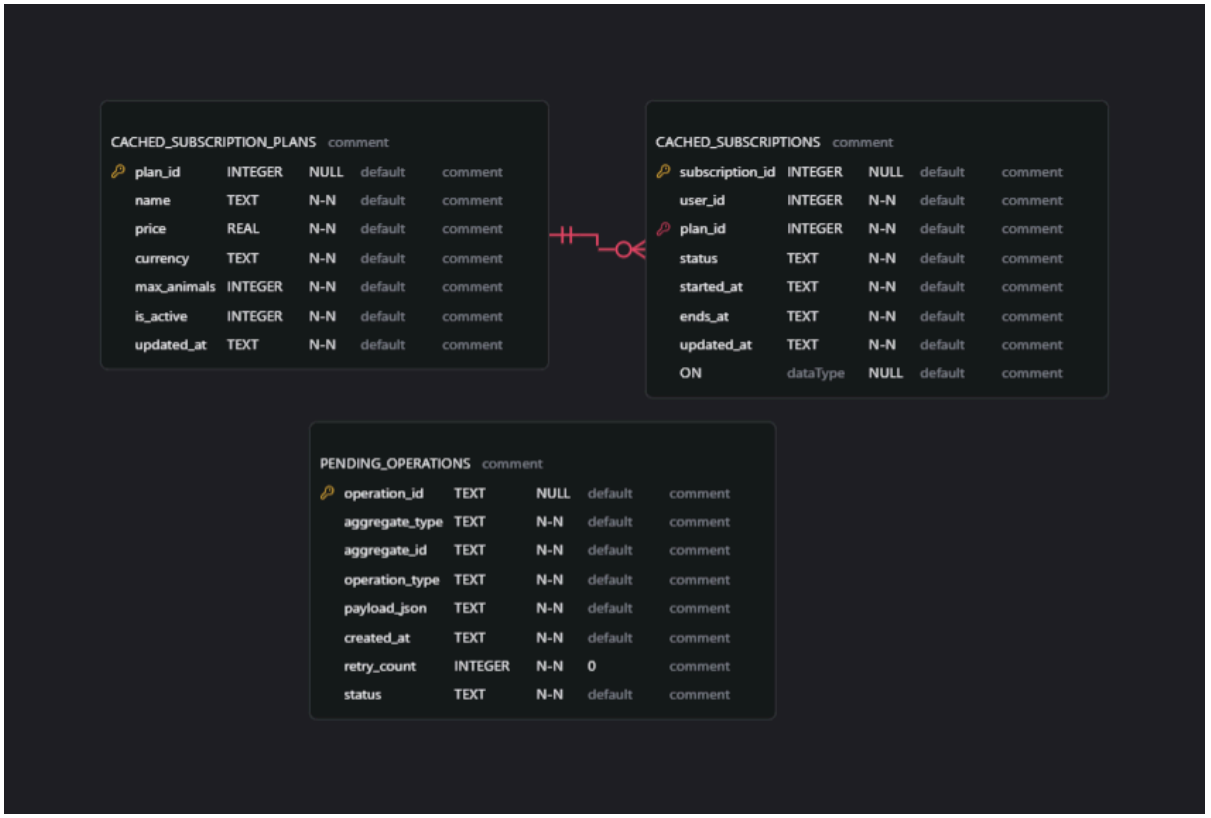


Figura 2.6.8.6. Android Room Database Design de Subscription Management. Fuente: elaboración propia a partir del esquema SQL.

2.6.9. Bounded Context: Analytics and Reporting

Construir proyecciones y métricas de consulta para los dashboards sin adquirir propiedad sobre los registros de los contextos fuente.

La base implementada se encuentra en el módulo Analytics de la API ASP.NET Core. El diseño móvil de Android y Flutter se documenta como **no implementado** porque esos clientes todavía no existen en el workspace. Los tres productos comparten contratos REST y lenguaje ubicuo, mientras la API conserva las reglas autoritativas.

2.6.9.1. Domain Layer

Esta capa documenta el modelo que representa el núcleo de **Analytics and Reporting**. Las reglas autoritativas se ejecutan en la API; Android y Flutter mantienen modelos equivalentes para presentación, validación inmediata y trabajo offline. La columna de estado distingue el código heredado de la arquitectura objetivo.

Aggregate Root: DashboardProjection

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Agrupar métricas calculadas para una audiencia y periodo.
Relaciones	DashboardProjection compone ReportMetric; DashboardProjection compone MetricPeriod; ProjectionBuilder depende de DashboardProjection; IReportMetricRepository depende de DashboardProjection

Atributos o dependencias

Nombre	Tipo
id	int
ownerId	int
audience	string
period	MetricPeriod
generatedAt	DateTime
metrics	List~ReportMetric~

Métodos u operaciones

Firma	Retorno
ReplaceMetrics(metrics: List~ReportMetric~)	void
IsStale(now: DateTime)	bool

Entity: ReportMetric

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Representar una métrica individual del dashboard.
Relaciones	DashboardProjection compone ReportMetric; ReportMetric compone MetricSlice

Atributos o dependencias

Nombre	Tipo
id	int
label	string
slice	MetricSlice
trend	string
sourceContext	string

Métodos u operaciones

Firma	Retorno
Update(slice: MetricSlice, trend: string)	void

Value Object: MetricSlice

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Representar un valor, unidad y etiqueta de una métrica.
Relaciones	ReportMetric compone MetricSlice

Atributos o dependencias

Nombre	Tipo
label	string
value	decimal
unit	string

Métodos u operaciones

Firma	Retorno
IsComparableTo(other: MetricSlice)	bool

Domain Service: ProjectionBuilder

Campo	Detalle
Producto y estado	Backend, Android y Flutter (modelo canónico) — **No implementado**
Propósito	Construir proyecciones a partir de los contextos fuente.
Relaciones	ProjectionBuilder depende de DashboardProjection

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
BuildRancher(ownerId: int, period: MetricPeriod)	DashboardProjection
BuildVeterinarian(id: int, period: MetricPeriod)	DashboardProjection

Repository Interface: IReportMetricRepository

Campo	Detalle
Producto y estado	Backend; modelo equivalente en Android y Flutter — **Implementado**
Propósito	Abstraer la persistencia de ReportMetric.
Relaciones	IReportMetricRepository depende de DashboardProjection

Atributos o dependencias

No aplica.

Métodos u operaciones

Firma	Retorno
FindProjection(ownerId: int, period: MetricPeriod)	DashboardProjection?
Save(projection: DashboardProjection)	void

2.6.9.2. Interface Layer

Esta capa recibe las acciones relacionadas con **construcción y consulta de dashboards, métricas y proyecciones** y las traduce a casos de uso. Los controllers y resources corresponden a la API; las pantallas y controladores de estado representan la presentación objetivo en Android y Flutter. Ninguna de estas clases implementa reglas de negocio.

REST Controller: DashboardAnalyticsController

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Publicar por HTTP las capacidades de Analytics and Reporting.
Relaciones	Recibe resources, invoca servicios de aplicación y devuelve resources HTTP.

Atributos o dependencias

Nombre	Tipo
herdQueryService	IHerdQueryService
animalQueryService	IAnimalQueryService
healthEventQueryService	IHealthEventQueryService
financialRecordQueryService	IFinancialRecordQueryService
farmActivityQueryService	IFarmActivityQueryService
deviceQueryService	IDeviceQueryService
veterinarianClientQueryService	IVeterinarianClientQueryService

Métodos u operaciones

Firma	Retorno
GetRancherDashboard(int rancherId, CancellationToken cancellationToken)	No especificado
GetVeterinarianDashboard(int veterinarianId, CancellationToken cancellationToken)	No especificado
GetRancherHealthSummary(int rancherId, CancellationToken cancellationToken)	No especificado
GetRancherFinancialSummary(int rancherId, CancellationToken cancellationToken)	No especificado

Presentation Model: DashboardProjectionViewModel

Campo	Detalle
Producto y estado	Android / Kotlin — **No implementado**
Propósito	Mantener el estado observable y traducir acciones de Android a casos de uso.
Relaciones	Invoca casos de uso de Application Layer y publica un UI State inmutable.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
Load()	No especificado
Submit(action)	No especificado
RetrySync()	No especificado

State Controller: DashboardProjectionController

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Mantener el estado de presentación de Flutter y coordinar casos de uso.
Relaciones	Invoca Application Layer y publica estados de carga, éxito y error.

Atributos o dependencias

Nombre	Tipo
state	No especificado
observeUseCase	No especificado
syncUseCase	No especificado

Métodos u operaciones

Firma	Retorno
load()	No especificado
submit(action)	No especificado
retrySync()	No especificado

2.6.9.3. Application Layer

Esta capa coordina las capacidades de **construcción y consulta de dashboards, métricas y proyecciones**. Los commands y queries expresan intenciones; los handlers cargan aggregates, aplican reglas, persisten cambios y reaccionan a eventos. Los casos de uso móviles coordinan lectura local, actualización remota y sincronización idempotente.

Application Service: ReportMetricQueryService

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **Implementado**
Propósito	Orquestar construcción y consulta de dashboards, métricas y proyecciones sin contener reglas del dominio.

Campo	Detalle
Relaciones	Invoca agregados y repositories; confirma la transacción mediante Unit of Work.

Atributos o dependencias

Nombre	Tipo
repository	IReportMetricRepository

Métodos u operaciones

Firma	Retorno
Handle(GetReportMetricByIdQuery query, CancellationToken cancellationToken)	No especificado
Handle(GetAllReportMetricsQuery query, CancellationToken cancellationToken)	No especificado

Use Case: ObserveDashboardProjectionUseCase

Campo	Detalle
Producto y estado	Android y Flutter — **No implementado**
Propósito	Entregar primero datos locales y actualizar la consulta cuando exista conectividad.
Relaciones	Es invocado por ViewModel/Controller y coordina repositorios móviles.

Atributos o dependencias

Nombre	Tipo
localRepository	No especificado
remoteRepository	No especificado
connectivityMonitor	No especificado

Métodos u operaciones

Firma	Retorno
Execute(criteria)	Stream

Query Handler: BuildDashboardQueryHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Resolver una consulta de aplicación y construir el modelo de lectura requerido.
Relaciones	Consume una Query y consulta repositories o proyecciones.

Atributos o dependencias

Nombre	Tipo
queryRepository	No especificado
projectionBuilder	No especificado

Métodos u operaciones

Firma	Retorno
Handle(query)	Result

Event Handler: SourceRecordChangedEventHandler

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Reaccionar al evento confirmado y actualizar proyecciones o integraciones.
Relaciones	Consume un Domain Event y utiliza puertos de infraestructura sin modificar directamente el agregado.

Atributos o dependencias

Nombre	Tipo
projectionRepository	No especificado
notificationPort	No especificado
unitOfWork	No especificado

Métodos u operaciones

Firma	Retorno
Handle(domainEvent)	Task

2.6.9.4. Infrastructure Layer

Esta capa implementa los puertos definidos hacia el interior de **Analytics and Reporting** y concentra acceso a base de datos, red, almacenamiento local e integraciones externas. Las clases de infraestructura traducen errores y contratos técnicos antes de devolver resultados a Application Layer.

Repository Adapter: ReportMetricRepository

Campo	Detalle
Producto y estado	Backend / Entity Framework Core — **Implementado**

Campo	Detalle
Propósito	Implementar el puerto de persistencia definido por Domain Layer.
Relaciones	Implementa IReportMetricRepository; utiliza AppDbContext/MySQL y reconstruye el aggregate.

Atributos o dependencias

Nombre	Tipo
context	AppDbContext

Métodos u operaciones

Firma	Retorno
FindById(id)	No especificado
Add(entity)	No especificado
Update(entity)	No especificado
Delete(entity)	No especificado

Room Adapter: DashboardProjectionDao

Campo	Detalle
Producto y estado	Android / Room — **No implementado**
Propósito	Implementar persistencia local y observación reactiva en Android.
Relaciones	Implementa el puerto local mediante Room y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
roomDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
Observe(criteria)	No especificado
Upsert(entity)	No especificado
Delete(id)	No especificado
Pending()	No especificado

SQLite Adapter: DashboardProjectionLocalDataSource

Campo	Detalle
Producto y estado	Flutter / Dart — **No implementado**
Propósito	Implementar persistencia local equivalente en Flutter.
Relaciones	Implementa el puerto local mediante SQLite y participa en la estrategia de caché/outbox.

Atributos o dependencias

Nombre	Tipo
sqliteDatabase	No especificado
entityMapper	No especificado

Métodos u operaciones

Firma	Retorno
watch(criteria)	No especificado
upsert(entity)	No especificado
delete(id)	No especificado
pending()	No especificado

Projection Source Adapter: LivestockMetricsAdapter

Campo	Detalle
Producto y estado	Backend ASP.NET Core — **No implementado**
Propósito	Aislar una dependencia externa detrás de un puerto explícito.
Relaciones	Implementa una fuente de proyección desde Livestock Management.

Atributos o dependencias

Nombre	Tipo
livestockQueries	No especificado

Métodos u operaciones

Firma	Retorno
ReadLivestockMetrics(ownerId, period)	No especificado

2.6.9.5. Bounded Context Software Architecture Component Level Diagrams

El archivo component-level.dsl contiene las vistas BC9-ApiComponents, BC9-AndroidComponents y BC9-FlutterComponents. Las tres parten del mismo modelo C4 y muestran la separación entre presentación, aplicación, dominio y adaptadores.

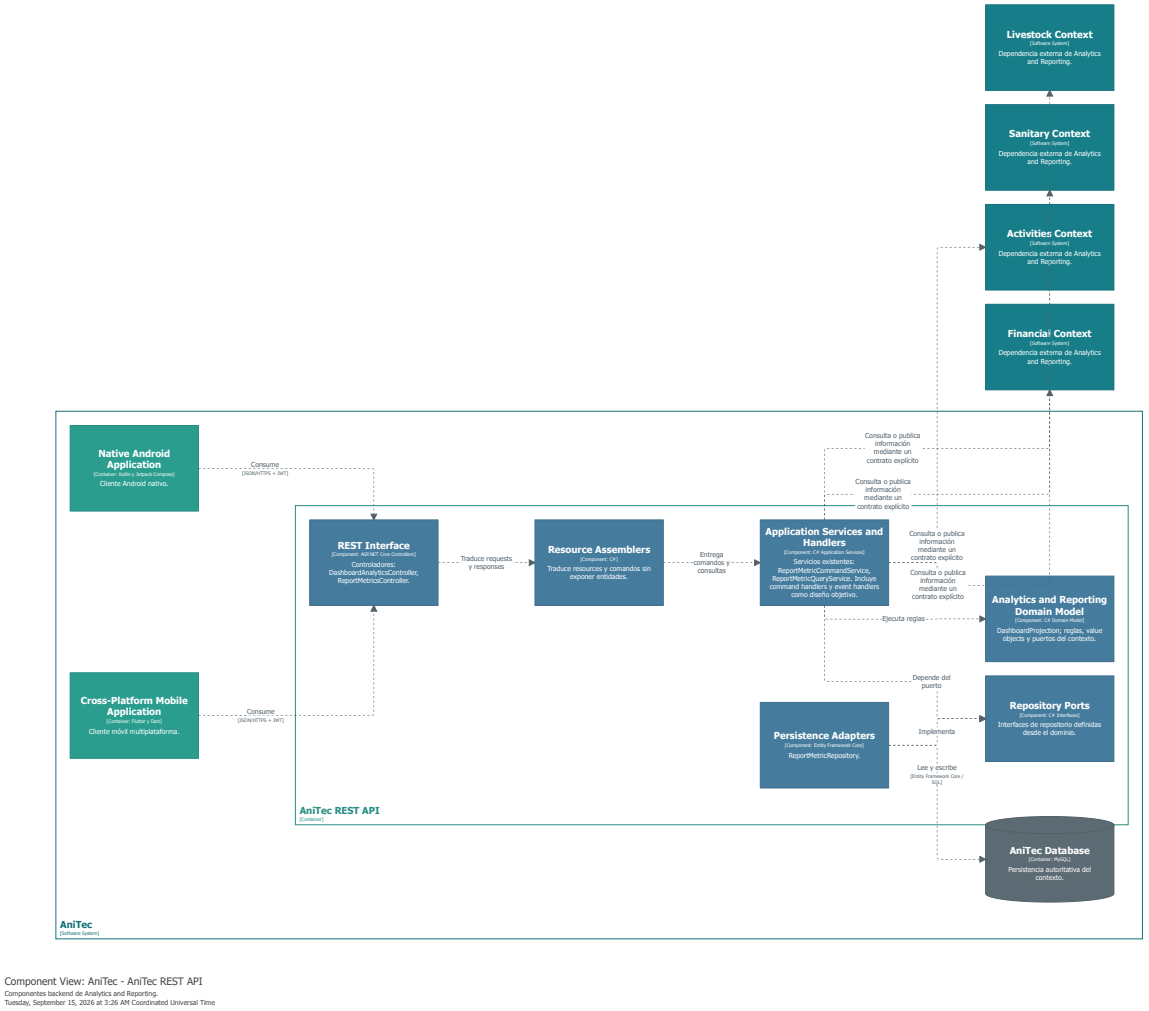


Figura 2.6.9.1. Componentes de la API para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

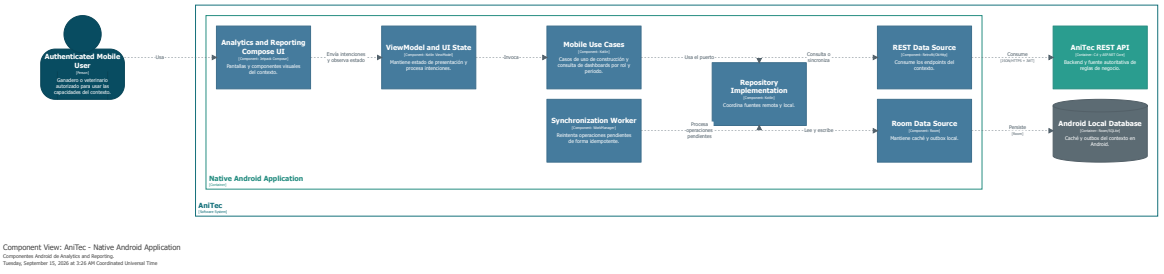


Figura 2.6.9.2. Componentes Android para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

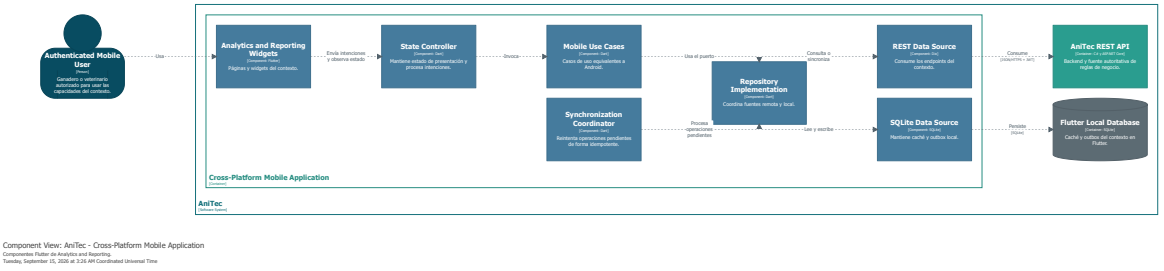


Figura 2.6.9.3. Componentes Flutter para Analytics and Reporting. Fuente: elaboración propia con Structurizr DSL.

2.6.9.6. Bounded Context Software Architecture Code Level Diagrams

Los diagramas de código detallan el modelo del dominio y los objetos de persistencia. El UML diferencia los elementos existentes de las incorporaciones objetivo, mientras los esquemas SQL señalan mediante comentarios las columnas propuestas. Los archivos ERD quedan disponibles para completar la importación manual.

2.6.9.6.1. Bounded Context Domain Layer Class Diagrams

El Class Diagram incluye agregados, entidades, value objects, enumeraciones, servicios de dominio e interfaces de repositorio con atributos, operaciones, visibilidad y multiplicidades.

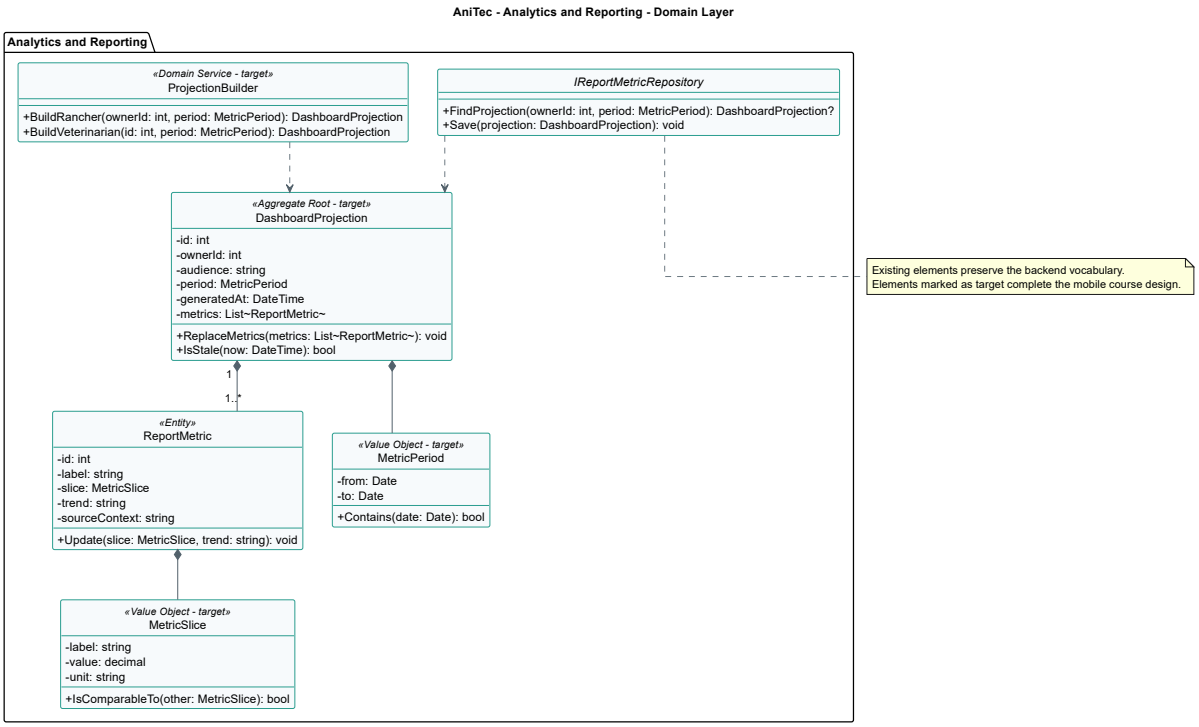


Figura 2.6.9.4. Domain Layer Class Diagram de Analytics and Reporting. Fuente: elaboración propia con PlantUML.

2.6.9.6.2. Bounded Context Database Design Diagram

MySQL mantiene la persistencia autoritativa. Room y SQLite contienen únicamente caché, metadatos de sincronización y operaciones pendientes; no sustituyen las reglas ni la fuente de verdad del backend. En IAM, las credenciales y tokens permanecen fuera de las tablas locales y se almacenan mediante mecanismos seguros del sistema operativo.

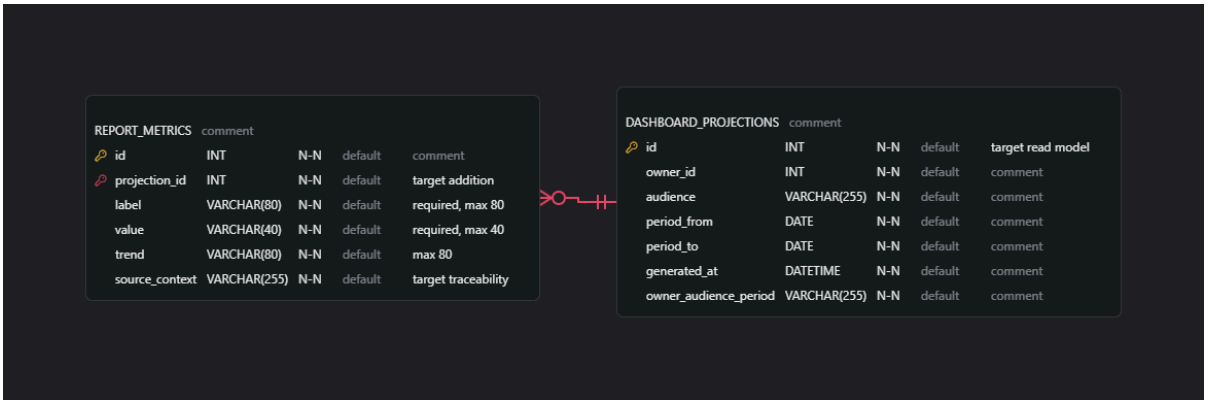


Figura 2.6.9.5. MySQL Database Design de Analytics and Reporting. Fuente: elaboración propia a partir del esquema SQL.

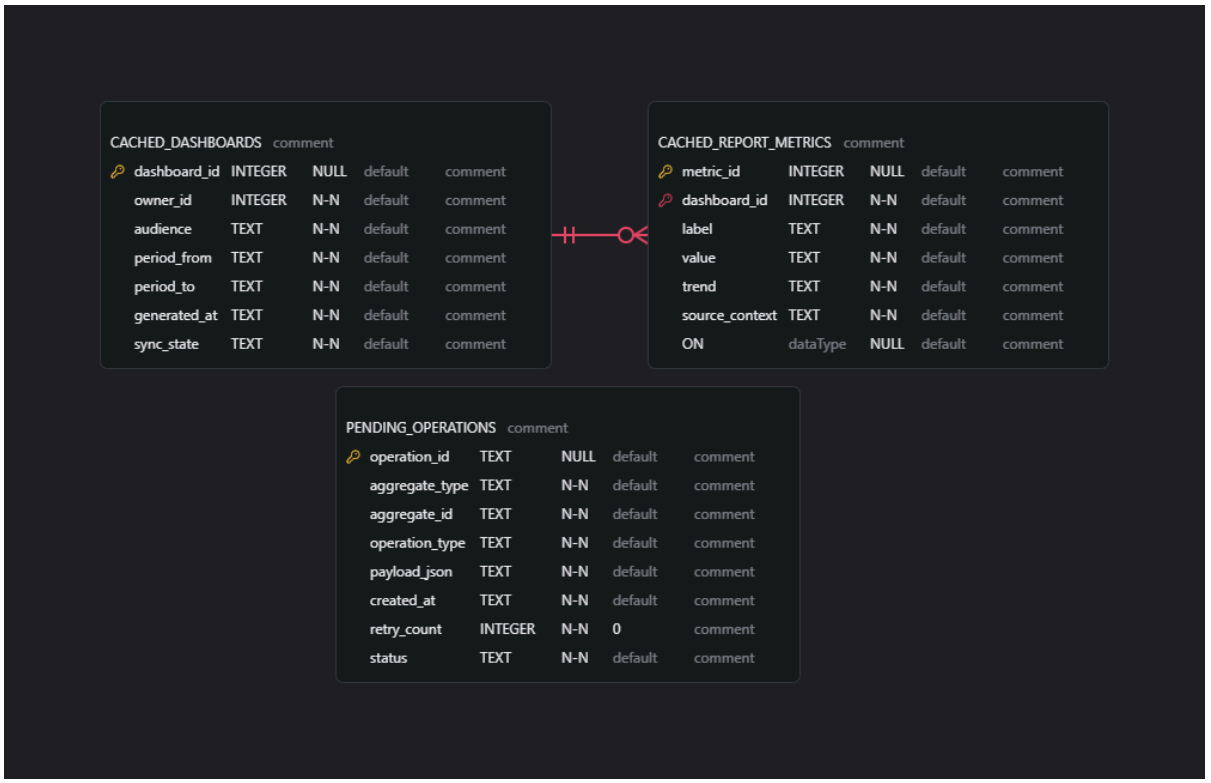


Figura 2.6.9.6. Android Room Database Design de Analytics and Reporting. Fuente: elaboración propia a partir del esquema SQL.

Capítulo III: Solution UI/UX Design

3.1. Product design

El diseño de AniTec traduce las necesidades de ganaderos y veterinarios en una experiencia coherente entre la Landing Page, la aplicación Android nativa y la aplicación multiplataforma. Se priorizan la consulta rápida en campo, el registro con pocos pasos, la legibilidad en exteriores y la continuidad ante una conexión inestable. Los artefactos mantienen trazabilidad con los User Personas, User Stories y bounded contexts del capítulo II.

3.1.1. Style Guidelines

Estas pautas establecen un lenguaje visual común para que las interfaces sean consistentes, reconocibles y accesibles en la Landing Page, Jetpack Compose y Flutter.

3.1.1.1. General Style Guidelines

Identidad visual. AniTec combina referencias al entorno ganadero con una presentación digital moderna. Los logotipos deben conservar sus proporciones, un área libre alrededor y suficiente contraste. No deben deformarse, rotarse ni recolorearse arbitrariamente.

Figura 1. Logo de la startup Titan. Fuente: elaboración propia.

Figura 2. Logo de AniTec. Fuente: elaboración propia.

Paleta de colores. Los verdes representan salud, campo y crecimiento; los marrones conectan la interfaz con la actividad ganadera; y los tonos claros producen superficies legibles. Ningún estado dependerá únicamente del color: las alertas, confirmaciones y errores también usarán texto o iconografía.

Color	Código	Uso principal
Verde principal	#79B267	Acciones principales, selección activa y marca.
Marrón oscuro	#925930	Contraste de identidad y elementos ganaderos.
Marrón medio	#A3794F	Acentos, ilustraciones y acciones secundarias.
Verde suave	#A3C4A8	Estados secundarios y fondos de apoyo.

Color	Código	Uso principal
Beige	#D1BFA5	Tarjetas, superficies y separadores.
Crema	#F5F0E6	Fondo claro principal y descanso visual.

Figura 3. Paleta cromática de AniTec. Fuente: elaboración propia.

Tipografía. AniTec utiliza Poppins por su lectura clara. Los encabezados emplean SemiBold o Bold y el cuerpo Regular o Medium. En móviles se respetará el escalado configurado por el usuario y se evitarán bloques extensos en mayúsculas.

Uso	Peso recomendado	Tamaño móvil de referencia
Título principal	Bold	28–32 sp
Título de pantalla	SemiBold	22–24 sp
Subtítulo	SemiBold	18–20 sp
Cuerpo	Regular	16 sp
Etiqueta o ayuda	Medium / Regular	14 sp
Texto auxiliar	Regular	12 sp como mínimo

Figura 4. Tipografía Poppins. Fuente: elaboración propia.

Espaciado, formas e iconografía. Se usará una cuadrícula base de 8 unidades. Las tarjetas podrán utilizar radios de 8 a 16 dp y los controles táctiles mantendrán un área mínima de 48 × 48 dp. Los iconos tendrán etiquetas o descripciones accesibles cuando su significado no sea evidente.

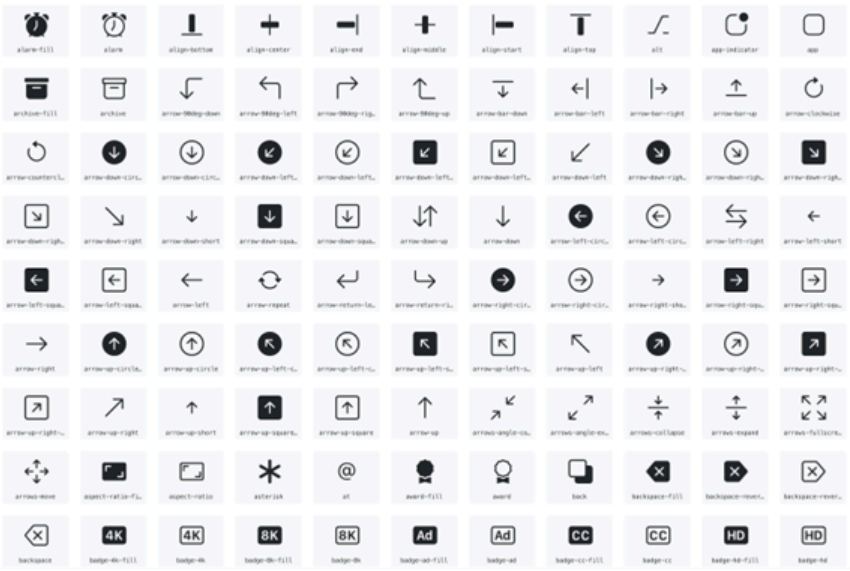


Figura 5. Referencia de iconografía. Fuente: elaboración propia.

Tono de comunicación. La comunicación será clara, respetuosa, serena y orientada a la acción. Usará términos conocidos por ganaderos y veterinarios, instrucciones breves y mensajes que indiquen cómo recuperarse de un problema.

Situación	Redacción recomendada
Acción	"Registrar animal"
Confirmación	"El animal se registró correctamente."
Error recuperable	"No pudimos guardar los cambios. Revisa tu conexión e inténtalo nuevamente."
Sin resultados	"No se encontraron animales con esos filtros."
Permiso	"AniTec necesita acceso a la cámara para leer el código del animal."

Dimensiones del tono. El tono de comunicación se ubica en las siguientes posiciones:

Dimensión	Posición	Cómo se aplica
Divertido / Serio	Serio, con cercanía	Mensajes profesionales; sin bromas en alertas sanitarias o financieras.
Formal / Casual	Casual profesional	Tuteo y frases breves, como «Revisa tu conexión»; sin jerga técnica.
Respetuoso / Irreverente	Respetuoso	Reconoce el conocimiento del ganadero y del veterinario y nunca culpa al usuario por un error.
Entusiasta / Sereno	Sereno	Confirmaciones claras y sin exceso; ante un problema se mantiene la calma e indica el siguiente paso.

Sustento de las decisiones. Las decisiones de marca, color, tipografía y espaciado se apoyan en los siguientes principios y elementos de diseño:

- **Jerarquía visual:** la escala tipográfica y el peso de la fuente ordenan la lectura de títulos, cuerpo y textos de ayuda.
- **Contraste y legibilidad:** los fondos claros y sin brillo excesivo favorecen la lectura en exteriores, y el texto y los controles recurren a las variantes más oscuras de verdes y marrones cuando se necesita contraste suficiente.
- **Proximidad y consistencia:** las tarjetas agrupan la información relacionada, y los mismos componentes, colores y espaciados se repiten en todos los productos, de modo que lo aprendido en uno sirve en los demás.
- **Significado cromático:** el verde se asocia con la salud y las acciones principales, el marrón con la actividad ganadera y los tonos beige con superficies neutras; el color nunca es la única señal de un estado.
- **Facilidad de uso táctil:** el área mínima de 48 × 48 dp y la cuadrícula de 8 unidades reducen los errores al tocar los controles.
- **Carga cognitiva:** cada nivel de navegación ofrece pocas opciones (cinco destinos principales), lo que reduce el esfuerzo de decisión.

Accesibilidad e internacionalización

- Mantener contraste suficiente y no depender solo del color.
- Permitir el aumento de texto sin pérdida de contenido.
- Incluir descripciones accesibles en imágenes y controles.
- Centralizar textos para English (en_US) y Latin American Spanish (es_419), con inglés como idioma predeterminado.
- Formatear fechas, números y monedas según la región.
- Ofrecer una alternativa manual cuando la cámara no esté disponible.
- Usar etiquetas semánticas, foco visible y atributos ARIA en la Landing Page.

Criterios por producto. La Landing Page tendrá jerarquía vertical, navegación superior y diseño responsive. Android aplicará los patrones de Material Design con Jetpack Compose. Flutter utilizará componentes equivalentes y conservará la misma identidad y orden de tareas. Ambas aplicaciones contemplarán carga, contenido, ausencia de datos, error, confirmación y operación sin conexión cuando corresponda.

3.1.1.2. Web Style Guidelines

Estas pautas aplican a la Landing Page y a la aplicación web, y complementan las pautas generales.

- **Estructura:** encabezado con navegación superior, secciones apiladas con un encabezado centrado, contenido en bloques de ancho limitado y pie de página con columnas de enlaces.
- **Diseño responsive:** la Landing Page se diseñó para escritorio y para un ancho móvil de 412 px; la aplicación web reorganiza su contenido en pantallas de hasta 720 px, por ejemplo al ocultar las columnas secundarias de las tablas.
- **Componentes:** botones principales rellenos en verde y secundarios con borde; tarjetas de esquinas redondeadas sobre fondo crema o beige; secciones que alternan fondos para separar los temas. La aplicación web utiliza los componentes de PrimeVue adaptados a la paleta de AniTec.
- **Accesibilidad:** HTML semántico, atributos ARIA, foco visible, textos alternativos en las imágenes y no depender solo del color.
- **Internacionalización:** inglés (en_US) como idioma predeterminado y español latinoamericano (es_419) como alternativa.
- **Recursos compartidos:** los logotipos, las fuentes y la iconografía se conservan en la carpeta assets de cada repositorio y en el archivo de diseño de Figma del proyecto.

3.1.1.3. Mobile Style Guidelines

Estas pautas aplican a la aplicación Android nativa y a la aplicación Flutter, que comparten identidad y orden de tareas.

- **Plataforma y componentes:** Android utiliza Material Design 3 con Jetpack Compose; Flutter emplea componentes equivalentes con la misma identidad visual. Se usa un único tema claro.
- **Navegación:** barra superior con título y acciones, barra inferior de cinco destinos por rol, botón de acción flotante para crear registros y pantallas completas con flecha de retorno para formularios y detalles.
- **Contenedores:** tarjetas con esquinas de 8 a 16 dp, fichas de detalle en hojas inferiores y etiquetas de estado que combinan color y texto.

- **Controles:** objetivos táctiles de al menos 48 × 48 dp, botón principal de ancho completo con degradado verde, teclado decimal en los campos de importes y selectores de fecha del sistema.
- **Estados:** cada pantalla de datos contempla carga, lista vacía, error con opción de reintentar y operación sin conexión, con un aviso de los cambios pendientes de sincronizar.
- **Permisos y alternativas:** la cámara se solicita al abrir el escáner o al tomar una foto; si se deniega, el usuario puede escribir el código del arete o elegir una imagen de la galería.
- **Accesibilidad e internacionalización:** descripciones accesibles en los iconos, textos que respetan el tamaño de fuente del sistema, inglés (en_US) como idioma predeterminado y español latinoamericano (es_419) con un selector dentro de la aplicación.

3.1.2. Information Architecture

La arquitectura de información organiza la Landing Page y las aplicaciones para que visitantes, ganaderos y veterinarios encuentren con rapidez las funciones relacionadas con sus objetivos. La propuesta reduce carga cognitiva mediante etiquetas breves, agrupación por tareas y navegación diferenciada por rol.

3.1.2.1. Organization Systems

Sistema	Aplicación en AniTec
Jerárquico	Los dashboards priorizan alertas, indicadores y acciones frecuentes.
Secuencial	Registro de cuenta, finca, animal y evento sanitario se resuelve en pasos ordenados.
Cronológico	Actividades, historial sanitario, pagos y operaciones pendientes se ordenan por fecha.
Alfabético	Las listas de animales, fincas y corrales se ordenan por nombre para localizar un elemento con rapidez.
Por tópicos	Animales, fincas, sanidad, actividades, finanzas, colaboración, analíticas y suscripciones.
Por audiencia	La Landing Page y la navegación diferencian a ganaderos y veterinarios.
Matricial	Los reportes cruzan indicadores por finca, animal, categoría, estado o periodo.

La Landing Page sigue un recorrido de descubrimiento: propuesta de valor, problema, beneficios, funciones por segmento, planes y llamada a la acción. En las aplicaciones, la autenticación conduce a un dashboard ajustado al rol.

3.1.2.2. Labelling Systems

Las etiquetas representan conceptos del dominio y evitan términos técnicos internos. Las etiquetas de las aplicaciones corresponden a la aplicación Android implementada y se conservan en la aplicación Flutter.

Producto o rol	Etiqueta	Significado
Landing Page	Inicio	Propuesta principal de AniTec.
Landing Page	Beneficios	Valor ofrecido por el producto.

Producto o rol	Etiqueta	Significado
Landing Page	Ganaderos	Capacidades de gestión del hato.
Landing Page	Veterinarios	Capacidades de seguimiento clínico.
Landing Page	Planes	Alternativas de suscripción.
Ganadero	Inicio	Alertas, indicadores del hato, próximas actividades y registros sanitarios recientes.
Ganadero	Animales	Búsqueda, registro individual o masivo, ficha técnica y acciones sobre varios animales.
Ganadero	Sanidad	Incidencias, vacunas, revisiones, tratamientos y diagnósticos.
Ganadero	Actividades	Tareas y recordatorios ordenados por fecha.
Ganadero	Más	Fincas, Corrales, Finanzas, Analítica, Dispositivos IoT, Suscripciones y Términos de Servicio.
Veterinario	Inicio	Resumen de clientes, pacientes que requieren atención y seguimientos pendientes.
Veterinario	Pacientes	Animales de cada cliente, con acceso a su historial clínico.
Veterinario	Sanidad	Registros sanitarios de los pacientes.
Veterinario	Actividades	Visitas y tareas programadas para los clientes.
Veterinario	Más	Clientes, Analítica, Dispositivos IoT, Suscripciones y Términos de Servicio.
Compartido	Escanear arete	Identificación del animal mediante código QR o de barras, con ingreso manual como alternativa.
Compartido	Cambios pendientes	Aviso de registros guardados sin conexión que esperan sincronizarse.

Las acciones principales serán «Registrar», «Guardar», «Editar», «Buscar», «Filtrar», «Escanear», «Reintentar» y «Cancelar». «Eliminar» se reservará para operaciones destructivas y requerirá confirmación; «Descartar» se usará solo para los cambios sin sincronizar que el servidor rechazó.

3.1.2.3. SEO Tags and Meta Tags

Landing Page. El sitio define sus etiquetas en inglés, idioma predeterminado; la tabla las muestra junto con su versión en español.

Elemento	Valor implementado (en_US)	Valor en español (es_419)
Title	AniTec - Livestock Management App for Ranchers and Veterinarians	AniTec - App de gestión ganadera para ganaderos y veterinarios
Description	AniTec is a mobile app for ranchers and veterinarians. Register farms and animals, keep	AniTec es una aplicación móvil para ganaderos y veterinarios. Registra fincas y animales, lleva los

Elemento	Valor implementado (en_US)	Valor en español (es_419)
	health records, schedule activities and keep working with limited signal.	registros sanitarios, programa actividades y sigue trabajando con poca señal.
Keywords	livestock management app, rancher app, veterinarian app, animal health records, farm management, livestock traceability, Android app, AniTec	app de gestión ganadera, app para ganaderos, app para veterinarios, registros sanitarios, gestión de fincas, trazabilidad animal, aplicación Android, AniTec
Author	AniTec	AniTec
Robots	index, follow	index, follow
Open Graph title	AniTec - Livestock Management App for Ranchers and Veterinarians	AniTec - App de gestión ganadera
Open Graph description	AniTec is a mobile app for ranchers and veterinarians. Register farms and animals, keep health records and keep working with limited signal.	AniTec es una aplicación móvil para ganaderos y veterinarios. Registra fincas y animales, lleva los registros sanitarios y sigue trabajando con poca señal.

Aplicación web. La aplicación web define hoy solo el título de la página; los demás valores son la propuesta que se incorporará.

Elemento	Valor
Title	AniTec Web App
Description	AniTec Web App helps ranchers and veterinarians manage animals, health records, activities and finances from the browser. (propuesto)
Keywords	AniTec, livestock management, animal health, ranchers, veterinarians, web application (propuesto)
Author	AniTec (propuesto)

ASO de las aplicaciones móviles

Elemento	Android nativo	Aplicación Flutter
App title	AniTec Ganadería	AniTec Ganadería
Subtitle	Gestión del ganado en campo	Gestión ganadera multiplataforma
Keywords	ganado, animales, sanidad, trazabilidad, veterinario	ganado, animales, sanidad, trazabilidad, veterinario
Short description	Registra animales, controla su salud y organiza actividades desde el teléfono.	Consulta y gestiona la información esencial de AniTec desde dispositivos compatibles.
Full description	Aplicación para ganaderos y veterinarios que necesitan información organizada, alertas y trazabilidad en campo.	Experiencia multiplataforma para acceder a los flujos de AniTec mediante la misma API y reglas del producto.

Los textos definitivos se ajustarán a las restricciones de longitud de cada tienda antes de la distribución.

3.1.2.4. Searching Systems

Datos	Búsqueda	Filtros	Presentación
Animales	Código, nombre, especie, raza, sexo, estado, finca o corral	Corral	Lista con foto, código, finca, corral y estado; la selección múltiple habilita acciones masivas.
Pacientes (veterinario)	No aplica	Cliente y finca	Lista de animales con acceso al historial clínico.
Historial sanitario	No aplica	Animal	Registros en orden cronológico.
Clientes (veterinario)	Nombre o usuario del ganadero al agregar un cliente	No aplica	Lista de ganaderos disponibles con la acción de agregar.
Actividades	No aplica	No aplica	Próximas actividades primero y luego las vencidas, con prioridad y estado.
Finanzas	No aplica	No aplica	Resumen de ingresos, egresos y balance, y lista de movimientos por fecha.
Identificación de animales	Código del arete leído por cámara o escrito a mano	No aplica	Ficha del animal encontrado o mensaje de que no existe.

Las búsquedas se realizan sobre los datos guardados en el dispositivo, por lo que también están disponibles sin conexión. Cuando no hay coincidencias se muestra un estado vacío que explica la situación.

3.1.2.5. Navigation Systems

La Landing Page navega hacia Inicio, Beneficios, Ganaderos, Veterinarios, Planes y Contacto. En pantallas pequeñas usará un menú condensado sin ocultar las llamadas a la acción.

Rol	Destinos principales	Rutas secundarias (Más)
Ganadero	Inicio, Animales, Sanidad, Actividades y Más	Fincas, Corrales, Finanzas, Analítica, Dispositivos IoT, Suscripciones, Términos de Servicio y cambio de idioma.
Veterinario	Inicio, Pacientes, Sanidad, Actividades y Más	Clientes, Analítica, Dispositivos IoT, Suscripciones, Términos de Servicio y cambio de idioma.

La barra superior de las pantallas principales da acceso al escáner de aretes. Los formularios y las pantallas secundarias se abren a pantalla completa con una flecha de retorno, y las acciones destructivas piden confirmación. Si la sesión vence, la aplicación regresa al inicio de sesión. Las notificaciones y los enlaces profundos, como el retorno del pago de la suscripción, se incorporarán en una etapa posterior y comprobarán autenticación y autorización antes de abrir un recurso.

3.1.3. Landing Page UI Design

La Landing Page comunica el problema, los beneficios para cada segmento y las opciones para conocer las aplicaciones. Aplica la identidad visual, la organización jerárquica y una estructura responsive.

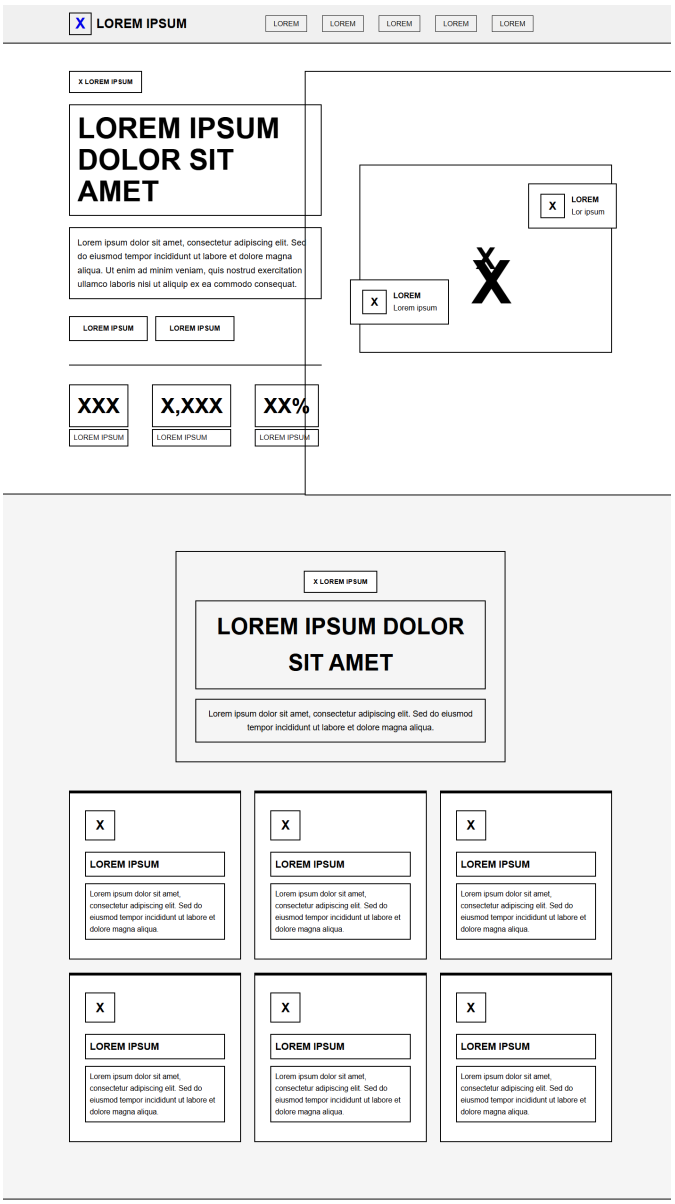
La propuesta aplica la arquitectura de información de la sección 3.1.2 y el Design System de la sección 3.1.1. El recorrido combina los sistemas de organización secuencial y por audiencia: propuesta de valor, características, pasos para comenzar, testimonios, planes y una llamada final a la acción. La navegación superior, las etiquetas breves y la repetición de colores, tipografía e iconografía permiten reconocer AniTec en cualquier punto de contacto, y el mismo recorrido se mantiene en el navegador de escritorio y en el móvil.

Enlace de diseño móvil: https://www.figma.com/design/DvQjG8GIupLP7TBQNi5Hr6/LandingMovil_MockUp?node-id=0-1&t=qJgJhyhc97VCfTMW-1
https://www.figma.com/design/q7A10f5s09GjpeGGWNdsZG/LandingMovil_wireframe?node-id=0-1&t=FDHM5xffOJs1fj7Z-1

3.1.3.1. Landing Page Wireframe

El wireframe de escritorio define la distribución del encabezado, propuesta de valor, beneficios, secciones por segmento, planes, testimonios y pie de página antes de aplicar el acabado visual.

El wireframe prescinde del color y de las imágenes para evaluar solo la estructura. De arriba hacia abajo se distinguen una cabecera con cinco enlaces de navegación; un bloque principal con titular, texto de apoyo, dos llamadas a la acción y tres indicadores; una cuadrícula de seis tarjetas de características; una secuencia de cuatro pasos; tres testimonios; tres planes, con el central destacado; una llamada final a la acción y un pie de página con columnas de enlaces. Se aplicaron los principios de jerarquía (los títulos de sección dominan visualmente), proximidad (cada tarjeta agrupa icono, título y descripción), repetición (todas las secciones usan el mismo encabezado centrado) y contraste (el plan recomendado y las llamadas a la acción se distinguen del resto). Como parte del diseño inclusivo, el orden visual coincide con el orden de lectura, de modo que la estructura puede recorrerse con teclado o lector de pantalla.


X LOREM IPSUM
LOREM
LOREM
LOREM
LOREM
LOREM
X LOREM IPSUM
LOREM IPSUM DOLOR SIT AMET
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
LOREM IPSUM
LOREM IPSUM
X
LOREM
Lor ipsum
X
LOREM
Lorem ipsum
XXX
LOREM IPSUM
X,XXX
LOREM IPSUM
XX%
LOREM IPSUM
X LOREM IPSUM
LOREM IPSUM DOLOR SIT AMET
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X
LOREM IPSUM
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X
LOREM IPSUM
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X
LOREM IPSUM
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X
LOREM IPSUM
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X
LOREM IPSUM
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
X LOREM IPSUM
LOREM IPSUM DOLOR SIT AMET?
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
X
X
X
X
LOREM IPSUM
Lorem ipsum dolor sit amet
LOREM IPSUM
Lorem ipsum dolor sit amet
LOREM IPSUM
Lorem ipsum dolor sit amet
LOREM IPSUM
Lorem ipsum dolor sit amet
X LOREM IPSUM
LOREM IPSUM DOLOR SIT AMET
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
"Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua."
XX
LOREM IPSUM
Lorem ipsum
"Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua."
XX
LOREM IPSUM
Lorem ipsum
"Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua."
XX
LOREM IPSUM
Lorem ipsum

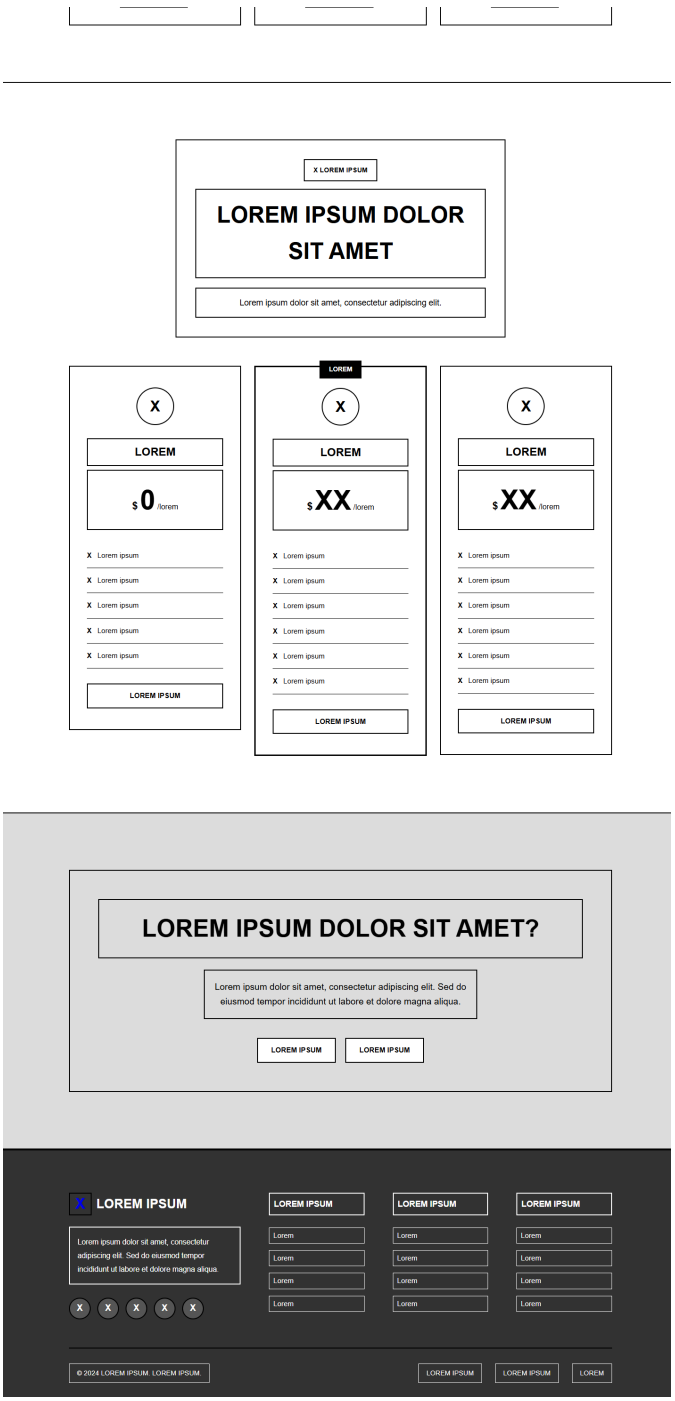
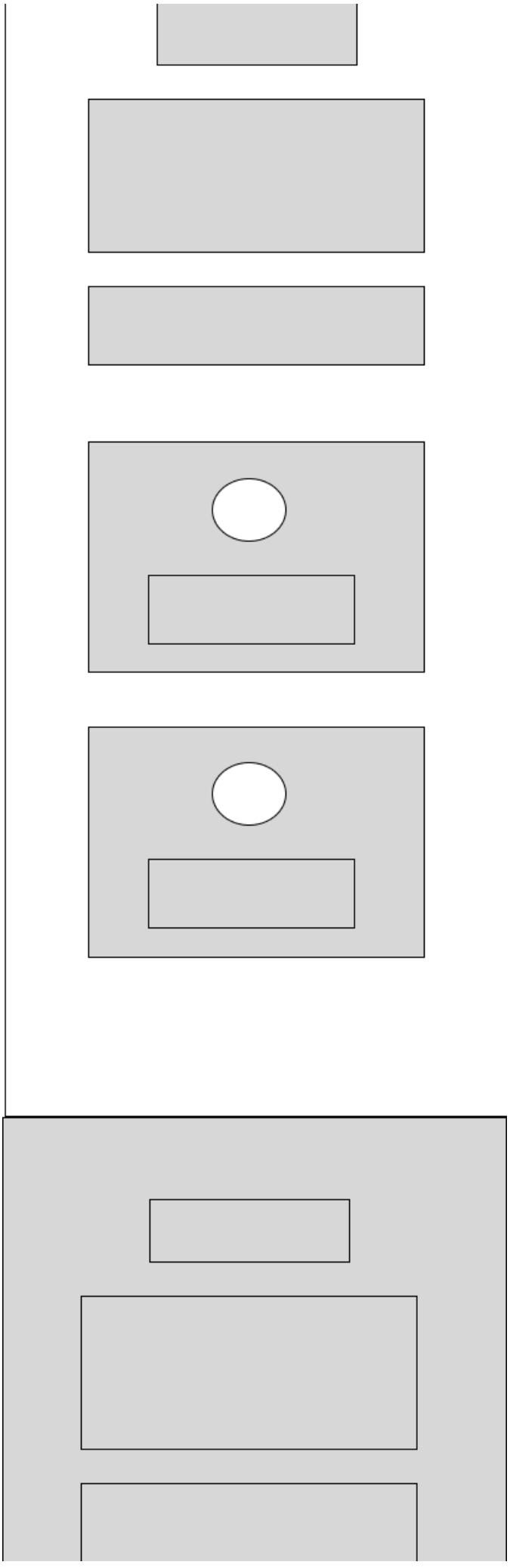
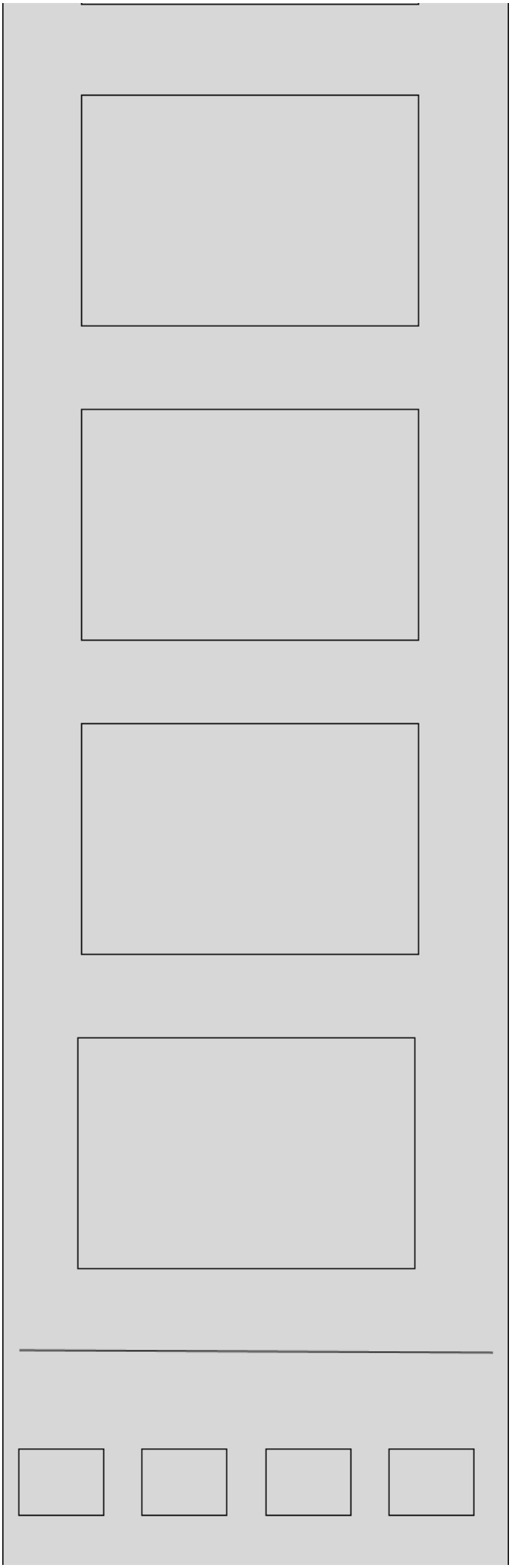


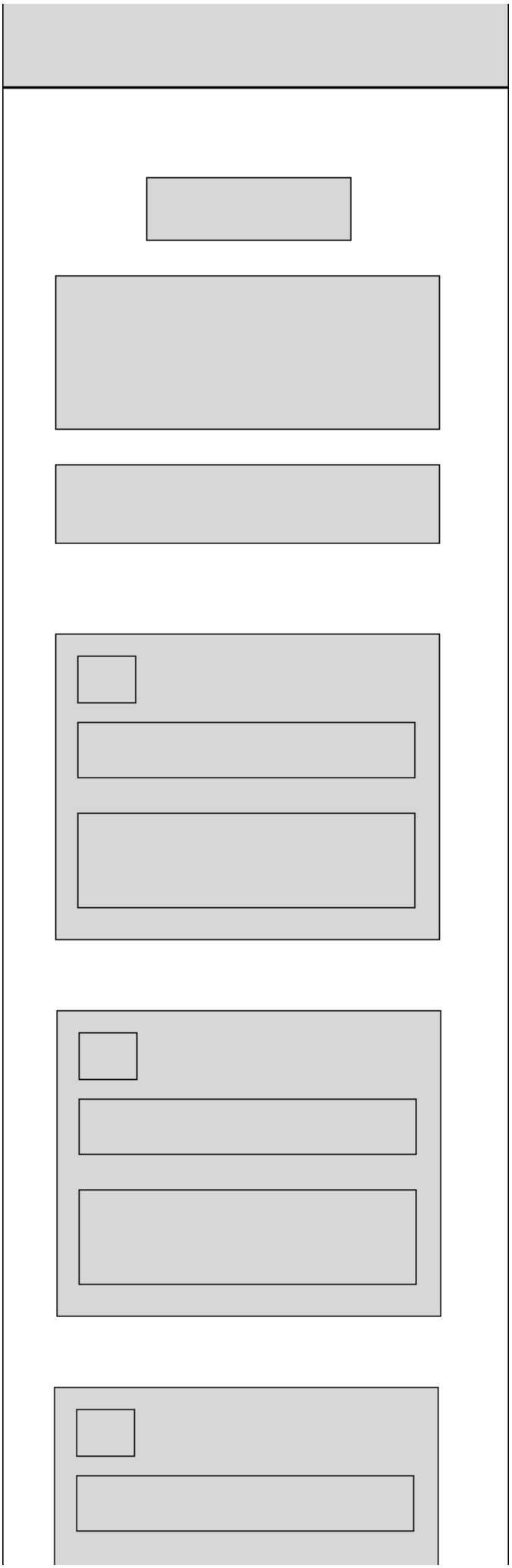
Figura 6. Wireframe de escritorio de la Landing Page. Fuente: elaboración propia.

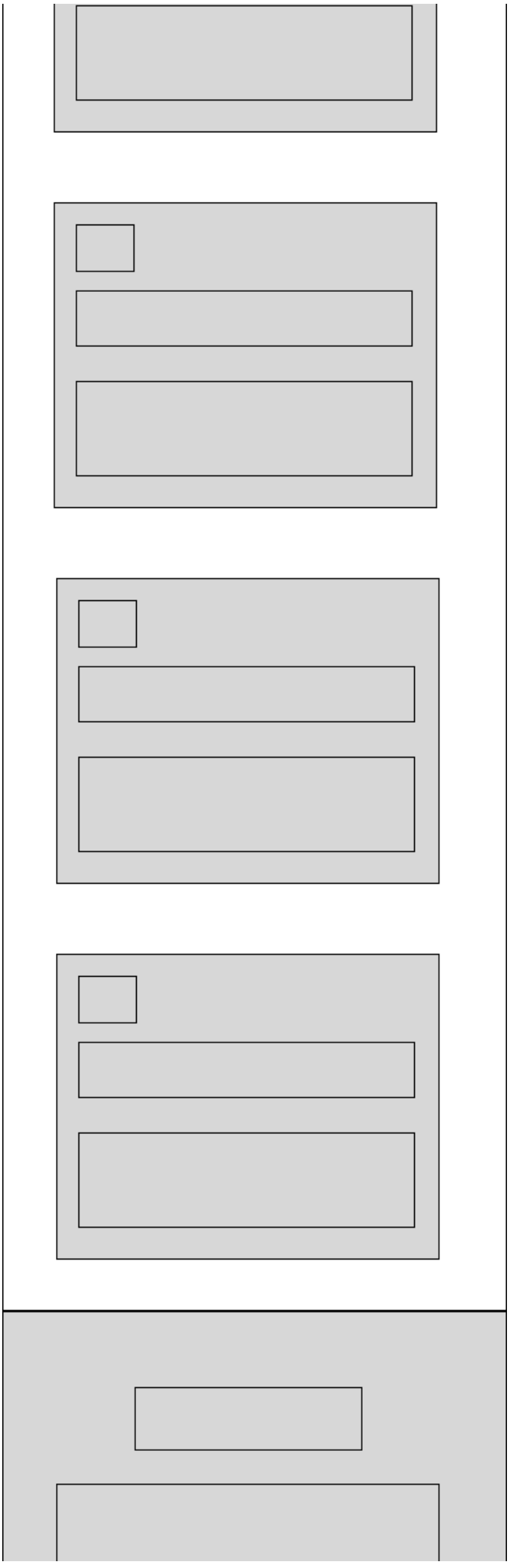
LOREM IPSUM

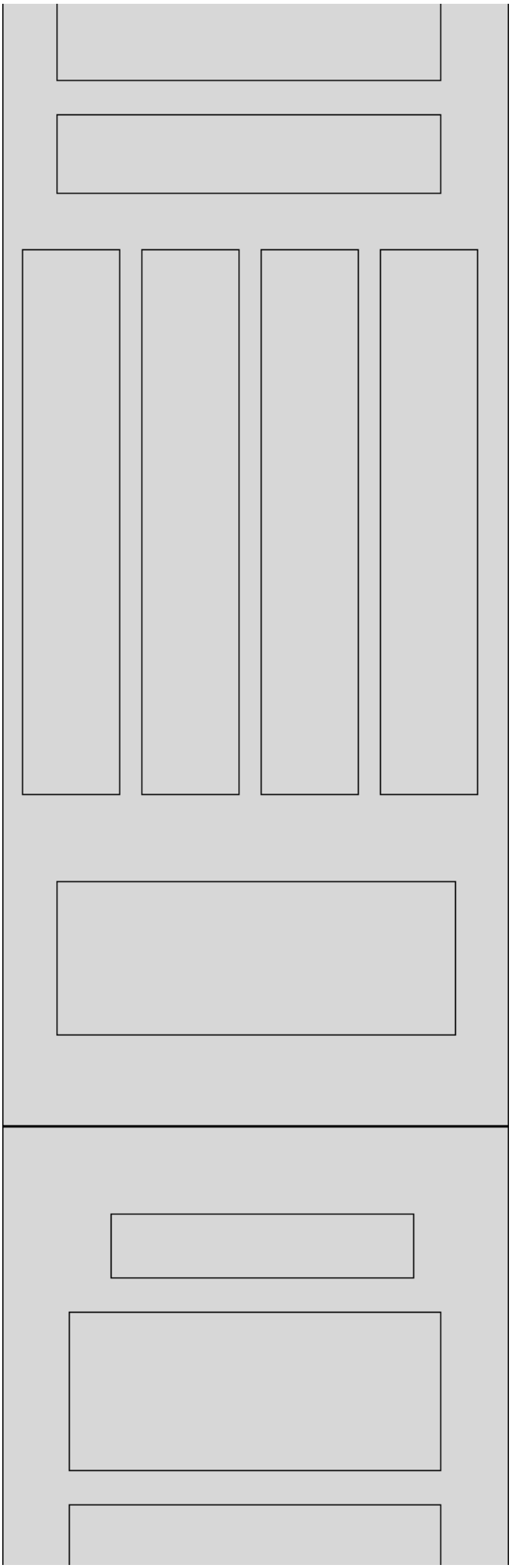
LOREM IPSUM

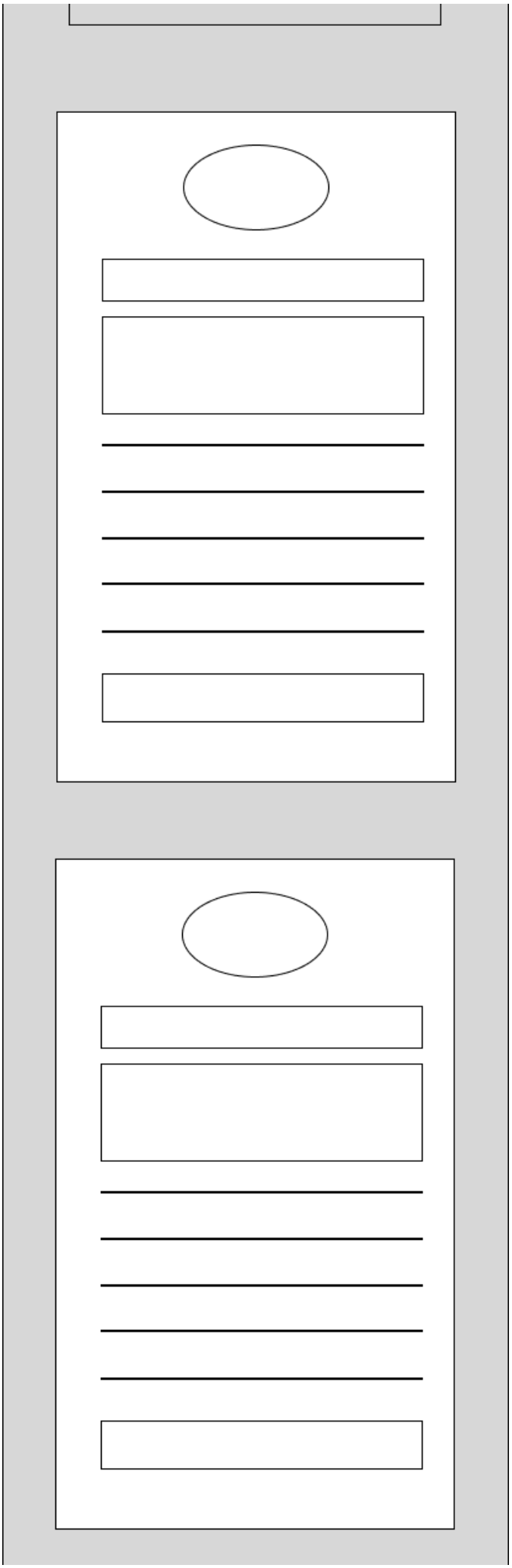


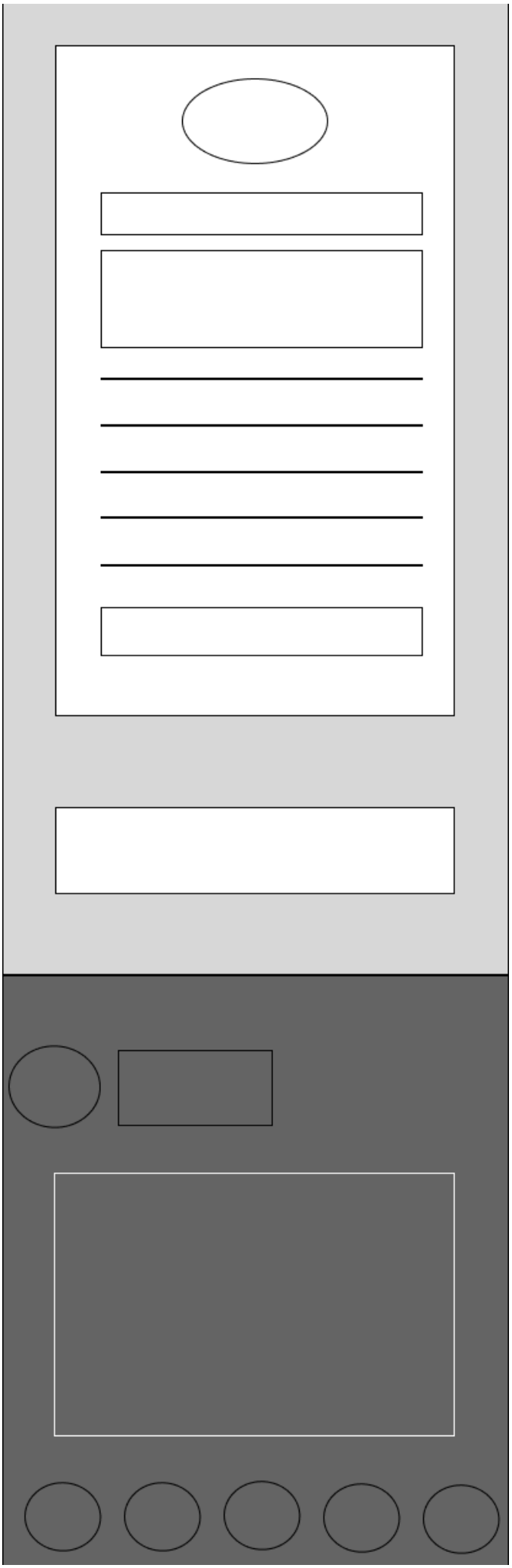


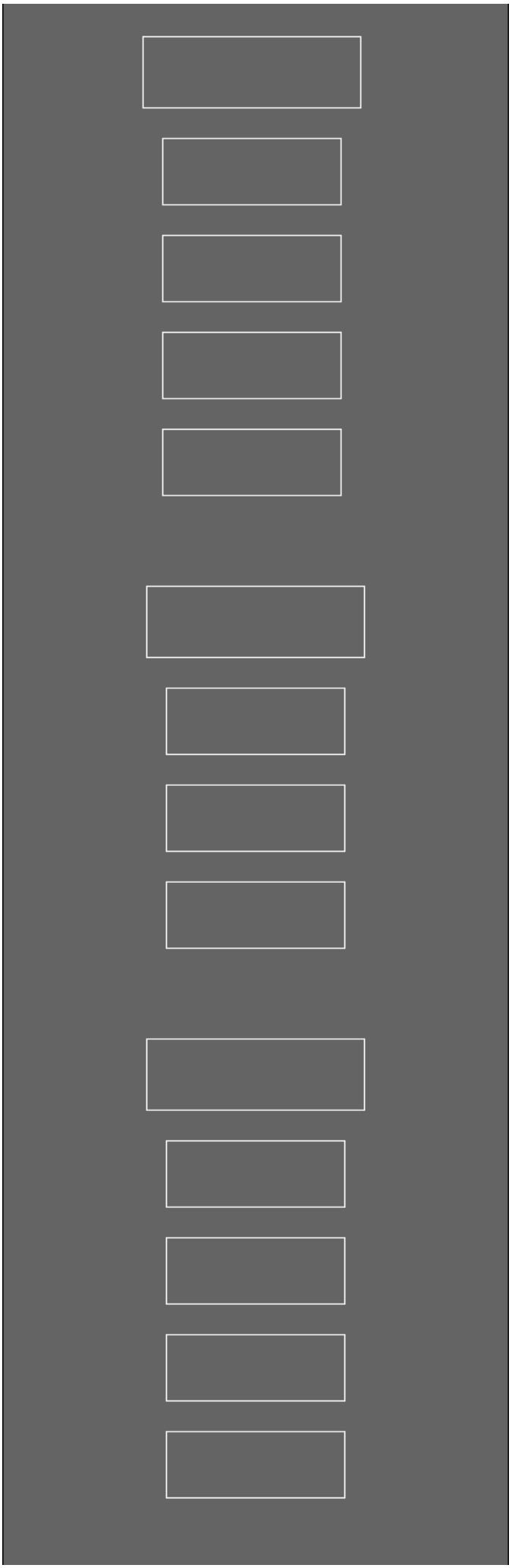












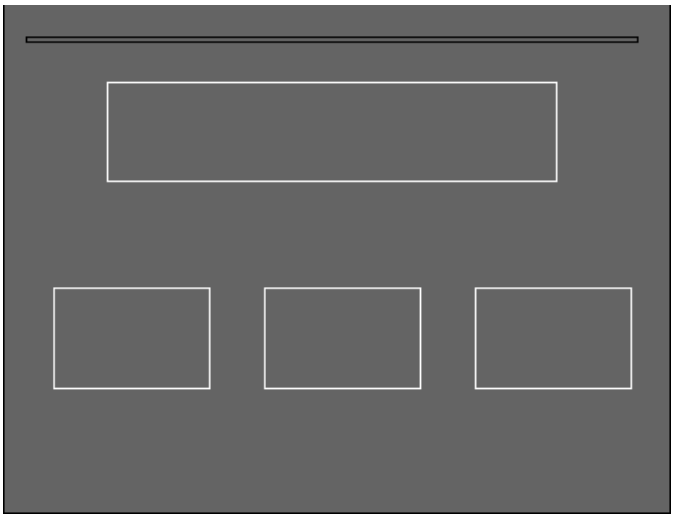


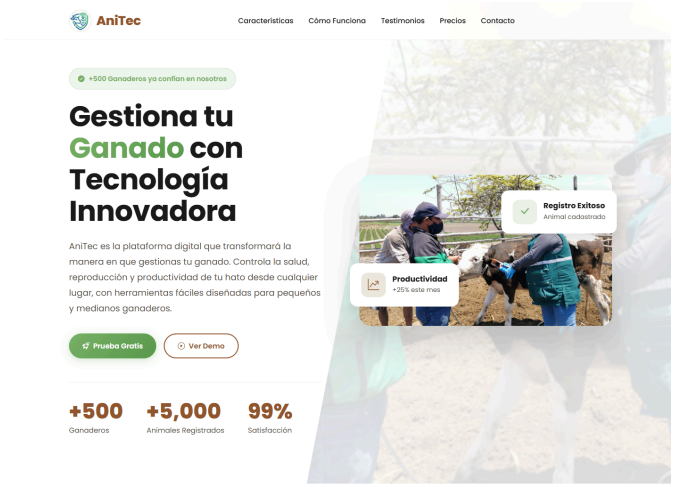
Figura 7. Wireframe de movil de la Landing Page. Fuente: elaboración propia.

Link del wireframe movil: https://www.figma.com/design/q7A10f5s09GjpeGGWNdsZG/LandingMovil_wireframe?node-id=0-1&t=FDHM5xffOJs1fj7Z-1

3.1.3.2. Landing Page Mock-up

El mock-up de escritorio incorpora la paleta, Poppins, imágenes, iconografía y llamadas a la acción para transmitir confianza y relación con el entorno agropecuario.

El mock-up traduce la estructura anterior al Design System de AniTec: fondo crema y tarjetas beige, verde para las acciones principales, marrón para las secciones destacadas, como los testimonios, y tipografía Poppins con una jerarquía clara de pesos. Cada característica usa un icono con título y descripción breve, y los tres planes comparan la oferta con la misma estructura para facilitar la elección. Las llamadas a la acción conservan el mismo estilo en el bloque principal y en el cierre, y las secciones alternan fondos para separar los temas sin depender solo del color. La versión móvil conserva el mismo orden de secciones y el mismo lenguaje visual.


AniTec
Características
Cómo Funciona
Testimonios
Precios
Contacto
+500 Ganaderos ya confían en nosotros
Gestiona tu Ganado con Tecnología Innovadora
AniTec es la plataforma digital que transformará la manera en que gestionas tu ganado. Controla la salud, reproducción y productividad de tu hato desde cualquier lugar, con herramientas fáciles diseñadas para pequeños y medianos ganaderos.
Prueba Gratis
Ver Demo
+500
Ganaderos
+5,000
Animales Registrados
99%
Satisfacción
Registro Exitoso
Animal cadastrado
Productividad
+25% este mes

Características Principales
Todo lo que necesitas para gestionar tu ganado
Una plataforma completa con herramientas diseñadas para optimizar cada aspecto de tu gestión ganadera.
Control Sanitario
Registra y controla todas las vaccaciones, tratamientos y eventos de salud de tu ganado. Recibe alertas automáticas para mantener a tu hato sano y protegido.
Reproducción
Controla el ciclo reproductivo de tu ganado, registra partos, monta y destetes. Optimiza la genética de tu hato con herramientas de seguimiento genealógico.
Gestión Económica
Controla tus gastos y ganancias. Genera reportes económicos detallados para tomar decisiones informadas sobre la rentabilidad de tu negocio ganadero.
Trazabilidad
Cada animal tiene su historial completo. Identifica mediante QR o código único para mantener una trazabilidad total desde el nacimiento hasta la venta.
Alertas Inteligentes
Recibe notificaciones automáticas sobre vaccaciones próximas, partos esperados y cualquier evento importante directamente en tu celular.
Acceso Móvil
Accede a tu plataforma desde tu celular o tablet en cualquier lugar. Funciona sin conexión donde la cobertura es limitada.

Fácil y Sencillo
¿Cómo comenzar con AniTec?
En solo 4 pasos tendrás toda tu gestión ganadera digitalizada y lista para usar.
1
Regístrate
Crea tu cuenta en minutos sin costos ocultos
2
Agrega tu Ganado
Registra cada animal con sus datos básicos
3
Registra Eventos
Anota vaccaciones, tratamientos y más
4
Toma Decisiones
Usa los reportes para mejorar tu producción

Testimonios
Lo que dicen nuestros usuarios
Descubre cómo AniTec ha transformado la gestión de cientos de ganaderos.
"Desde que uso AniTec, tengo todo el control de mi ganado en la palma de mi mano. Ya no pierdo tiempo buscando registros en papel. La alerta de vaccaciones me ha salvado de varios problemas."
MR
Miguel Rodríguez
Ganadero, Costa Rica
"AniTec me ha permitido aumentar mi productividad en un 30%. El control reproductivo es excelente y puedo planificar mejor mis crías. Totalmente recomendado."
LC
Lucía Chen
Productora Lechera, Ecuador
"Lo mejor es la simplicidad. Mi papá que tiene 70 años aprende a usarlo sin problemas. Ahora toda la familia puede consultar el estado del ganado."
GP
Gabriel Pérez
Ganadero, Colombia

Planes Flexibles
Elige el plan ideal para ti
Planes diseñados para adaptarse a cualquier tamaño de operación ganadera.
Más Popular

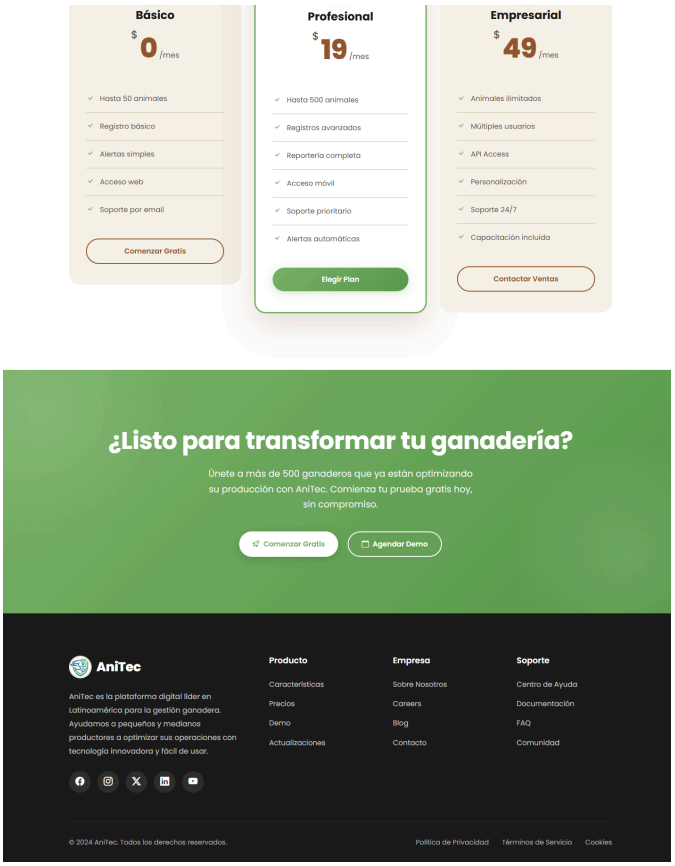


Figura 8. Mock-up de escritorio de la Landing Page. Fuente: elaboración propia.

Anitec

Comenzar

+500 Ganaderos ya confian en nosotros

Gestiona tu Ganado con Tecnología Innovadora

AniTec es la plataforma digital que transformará la manera en que gestionas tu ganado. Controla la salud, reproducción y roductividad de tu hato desde cualquier lugar, con herramienta fáciles diseñadas para pequeños y medianos ganaderos.

Ver Planes

Saber más

+500

Ganaderos

+5,000

Animales Registrados

99%

Satisfaccion

Dirigido a Dos Segmentos

Diseñado para Ganaderos y Veterinarios

AniTec aborda las necesidades específicas de cada segmento en el sector ganadero

Ganaderos

Productores pequeños y medianos que necesitan digitalizar la gestión del hato, desde el registro animal hasta el control económico.

Ver Beneficios →

Veterinarios

Profesionales de la salud animal que necesitan gestionar la historia clínica de sus pacientes y coordinar con ganaderos.

Ver Beneficios →

Resultados que hablan

El Impacto de AniTec

Únete a cientos de productores que ya están transformando su gestión ganadera.

85%
Menos Información Perdida
Digitaliza todos tus registros y nunca más pierdas datos importantes.
2h
Tiempo Ahorrado Diario
Deja de buscar en cuadernos. Encuentra todo en segundos.
30%
Más Productividad
Alertas automáticas que te ayudan a prevenir pérdidas.
100%
Trazabilidad Completa
Cumple con regulaciones y accede a mejores mercados.
+500
Ganaderos
+120
Veterinarios
+5,000
Animales
99%
Satisfacción de usuarios

Características Principales

Todo lo que necesitas para gestionar tu ganado

Una plataforma completa con herramientas diseñadas para optimizar cada aspecto de tu gestión ganadera.

Control Sanitario

Registra y controla todas las vacunaciones, tratamientos y eventos de salud de tu ganado. Recibe alertas automáticas para mantener a tu hato sano y protegido.

Reproducción

Controla el ciclo reproductivo de tu ganado, registra partos, monta y destetes. Optimiza la genética de tu hato con herramientas de seguimiento genealógico.

Gestión Económica

Controla tus gastos y ganancias. Genera informes económicos detallados para tomar decisiones informadas sobre la

tomar decisiones informadas sobre la rentabilidad de tu negocio ganadero.

Trazabilidad

Cada animal tiene su historial completo. Identifique ello mediante ID o código único para mantener trazabilidad total desde el nacimiento hasta la venta.

Alertas Inteligentes

Recibe notificaciones automáticas sobre próximas vacunaciones, nacimientos esperados y cualquier evento importante directamente en tu teléfono.

Acceso Móvil

Accede a tu plataforma desde tu teléfono o tablet en cualquier lugar. Funciona sin conexión donde la cobertura es limitada.

Comienza hoy

Empieza en minutos, sin

complicaciones

No necesitas ser un experto en tecnología. Nuestra interfaz es tan fácil que tendrás todo digitalizado en poco tiempo.

01

Crea tu cuenta

Regístrate con tu correo en menos de 2 minutos. No requiere tarjeta de crédito.

02

Agrega tu Ganado

Registra cada animal uno por uno o importa datos si ya los tienes en Excel.

03

Activa las Alertas

Configura recordatorios para vacunas, nacimientos y tratamientos.

04

¡Listo! Crecemos

Recibe informes y métricas para tomar mejores decisiones.

Comienza Ahora

Configura en minutos. Cancela cuando quieras.

Comienza Ahora

Planes Flexibles

Elige el plan ideal para ti

Sin costos ocultos. Cancela cuando quieras.

Más Popular

Profesional

Para ganaderos que buscan productividad

$19 /mes

- Hasta 500 animales
- Registros avanzados
- Reportería completa
- Acceso móvil
- Soporte prioritario
- Alertas automáticas

Elegir Plan

Básico

Perfecto para empezar a digitalizar tu ganado

$15 /mes

- Up to **50 animals**
- Basic animal registration
- Simple vaccine alerts
- Basic web access
- Email support

Comenzar

Empresarial
Para operaciones a gran escala
$49/mes
Unlimited animals
Multiple users and roles
Full API access
Field customization
24/7 dedicated support
Training included
Contactar Ventas
Todos los planes incluyen encriptación de datos y respaldo automático
AniTec
AniTec es la plataforma digital líder en Latinoamérica para la gestión ganadera. Ayudamos a productores pequeños y medianos y veterinarios a optimizar sus operaciones con tecnología innovadora y fácil de usar.

Producto

Caracterísiticas

Precios

Para Ganaderos

Para Veterinarios

Empresa

Nosotros

Blog

Contacto

Soporte

Centro de Ayuda

Documentación

Preguntas Frecuentes

Comunidad

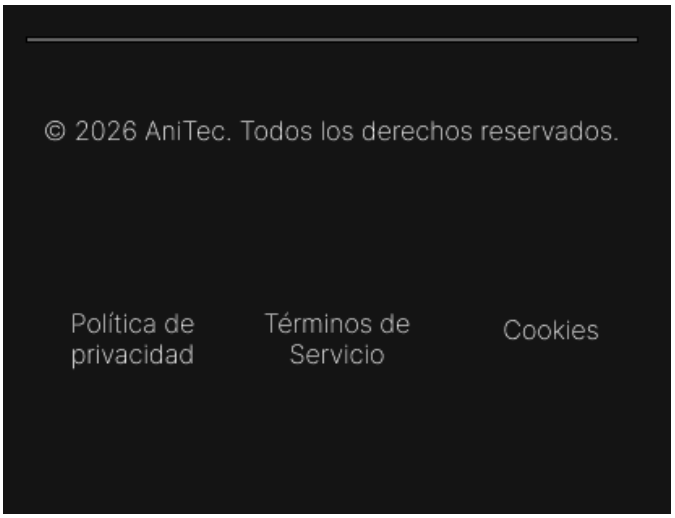


Figura 9. Mock-up de movil de la Landing Page. Fuente: elaboración propia.

Link del MockUp: https://www.figma.com/design/DvQjG8GIupLP7TBQNi5Hr6/LandingMovil_MockUp?node-id=0-1&t=qJgJhyhc97VCfTMW-1

3.1.4. Mobile Applications UX/UI Design

La propuesta comprende Android nativo con Kotlin y Jetpack Compose y una aplicación multiplataforma con Flutter y Dart. Ambas comparten objetivos, contratos, identidad y reglas de negocio, pero respetan los patrones de su plataforma. Los primeros flujos cubren autenticación, dashboard, registro y mantenimiento de animales, consulta del historial y registro sanitario.

Los diseños de ambas aplicaciones aplican la arquitectura de información de la sección 3.1.2 y el Design System de la sección 3.1.1: navegación diferenciada por rol con una barra inferior de cinco destinos, pantallas que combinan tarjetas, listas y formularios con la misma jerarquía de títulos, colores de estado acompañados de texto o icono, objetivos táctiles de al menos 48 × 48 dp y una alternativa manual a la lectura con cámara. El diseño inclusivo se refleja en el contraste, en el texto que acompaña a cada color de estado, en las descripciones accesibles de los iconos y en estados que explican qué ocurrió y cómo continuar: carga, vacío, sin resultados, error, sesión vencida, permiso de cámara denegado y trabajo sin conexión.

3.1.4.1. Mobile Applications Wireframes

Los wireframes representan la estructura funcional de cada pantalla de la aplicación móvil, definiendo la distribución de elementos y flujos de interacción básicos. Sirven como punto de partida para validar la organización visual y funcional del producto.

Los wireframes de baja fidelidad se elaboran en Figma sin color ni imágenes finales y sirven para validar la jerarquía, el orden de los controles y la navegación. Cada pantalla se documenta con su objetivo y con la User Story que satisface. Para ambos roles se aplican los siguientes criterios:

- Una acción principal por pantalla, ubicada en una zona alcanzable con el pulgar.
- Información agrupada por proximidad, por ejemplo los datos del animal, su ubicación y su estado, y ordenada por prioridad.
- Formularios por secciones, con etiquetas visibles, campos agrupados y mensajes de validación junto al campo.

- Navegación persistente con los destinos principales y acceso a las funciones secundarias desde «Más».
- Estados previstos desde el inicio: carga, sin datos, error y sin conexión.

El diseño en Figma se encuentra en: https://www.figma.com/design/uRmjCeeukXUb2AnZ2kFQsA/Anitec-2026-2?node-id=1-2&t=GnQ9UR4xNp7TYaA1-1

Aplicación Flutter

IAM

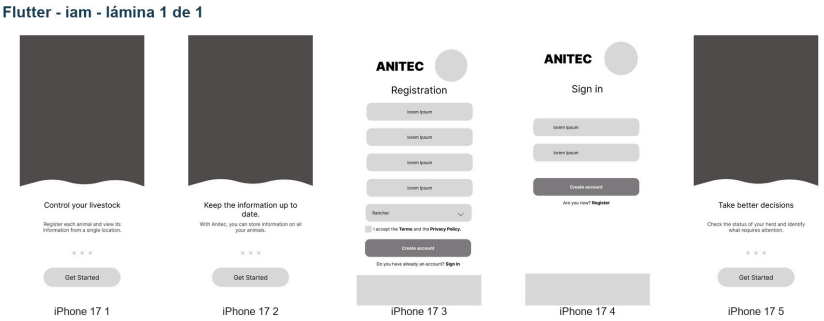


Figura 3.1.4.1.1. Wireframes Flutter del módulo IAM, lámina 1 de 1. Fuente: elaboración propia.

Usuario rancher

Figura 3.1.4.1.2. Wireframes Flutter del usuario rancher, lámina 1 de 5. Fuente: elaboración propia.

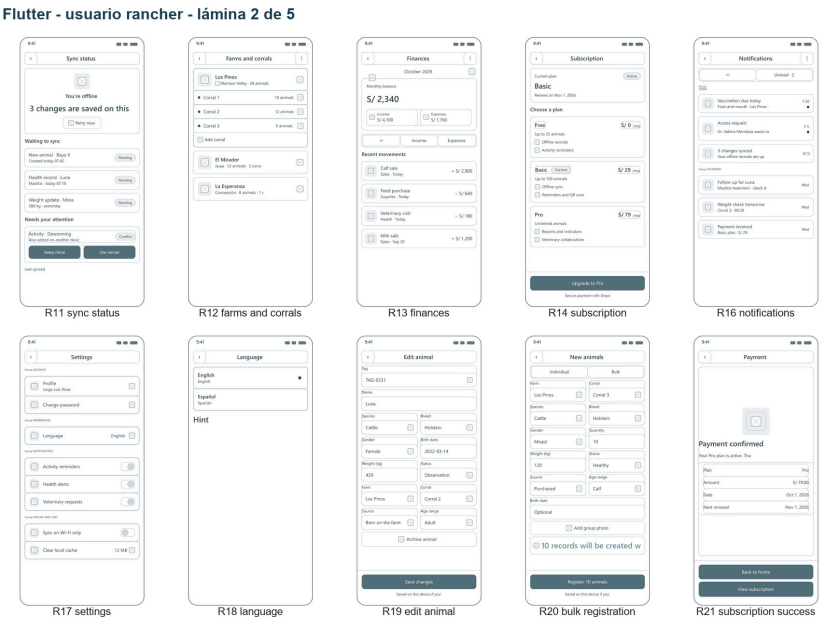


Figura 3.1.4.1.3. Wireframes Flutter del usuario rancher, lámina 2 de 5. Fuente: elaboración propia.

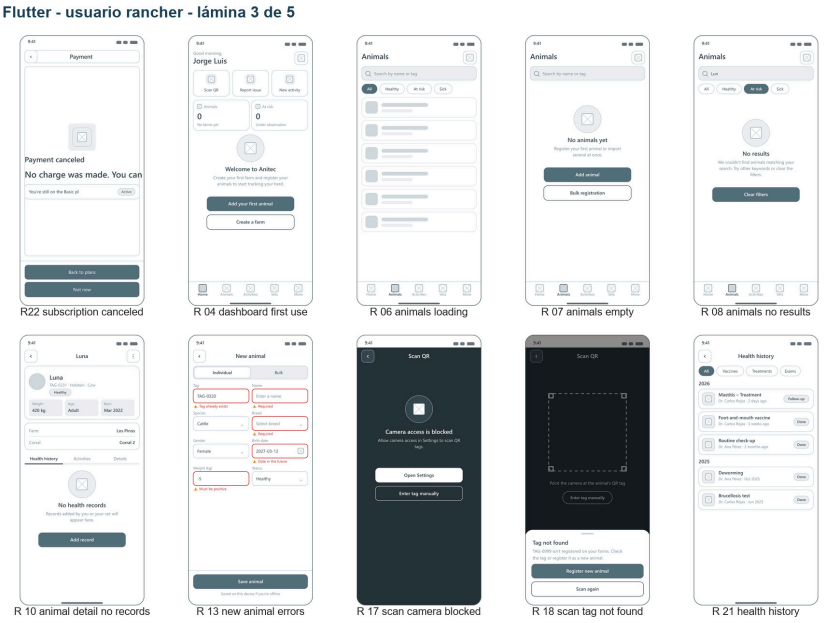


Figura 3.1.4.1.4. Wireframes Flutter del usuario rancher, lámina 3 de 5. Fuente: elaboración propia.

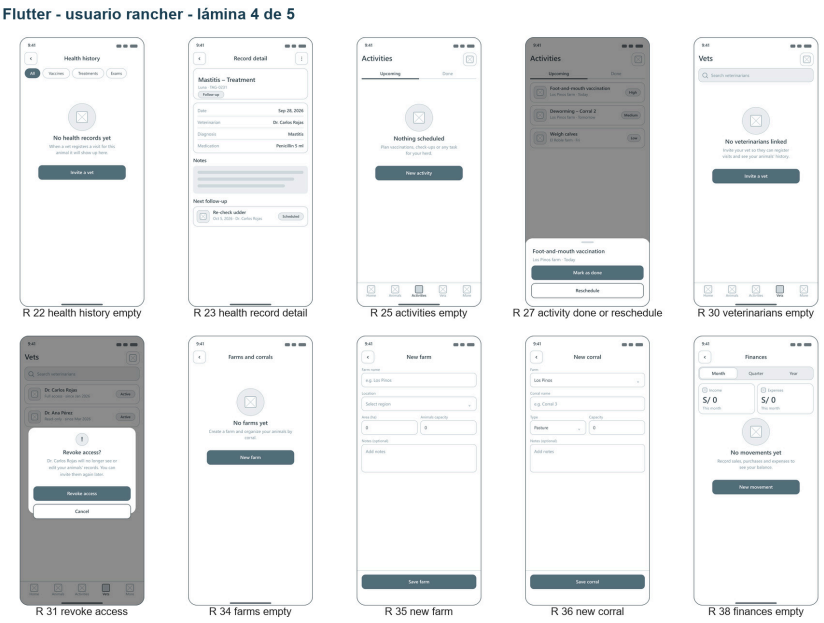


Figura 3.1.4.1.5. Wireframes Flutter del usuario rancher, lámina 4 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.6. Wireframes Flutter del usuario rancher, lámina 5 de 5. Fuente: elaboración propia.

Usuario veterinario

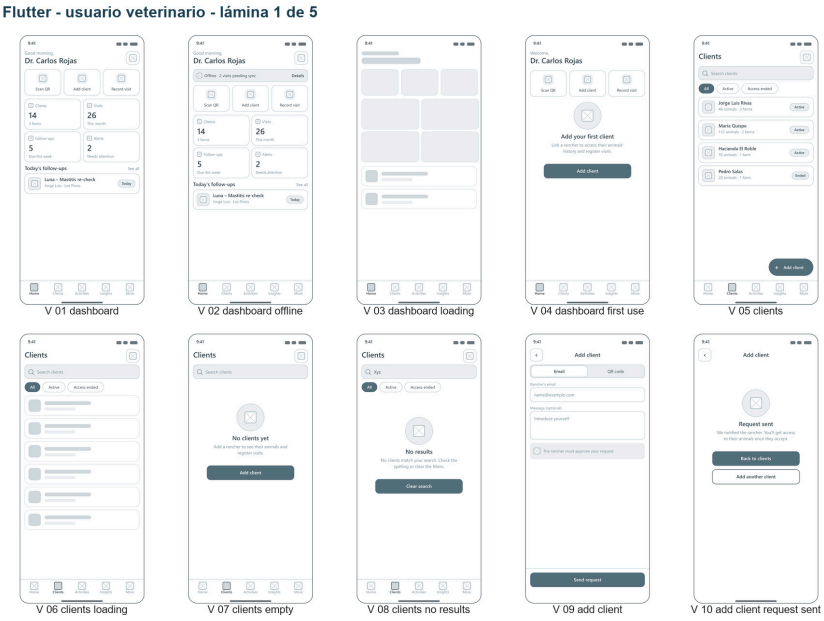


Figura 3.1.4.1.7. Wireframes Flutter del usuario vet, lámina 1 de 5. Fuente: elaboración propia.

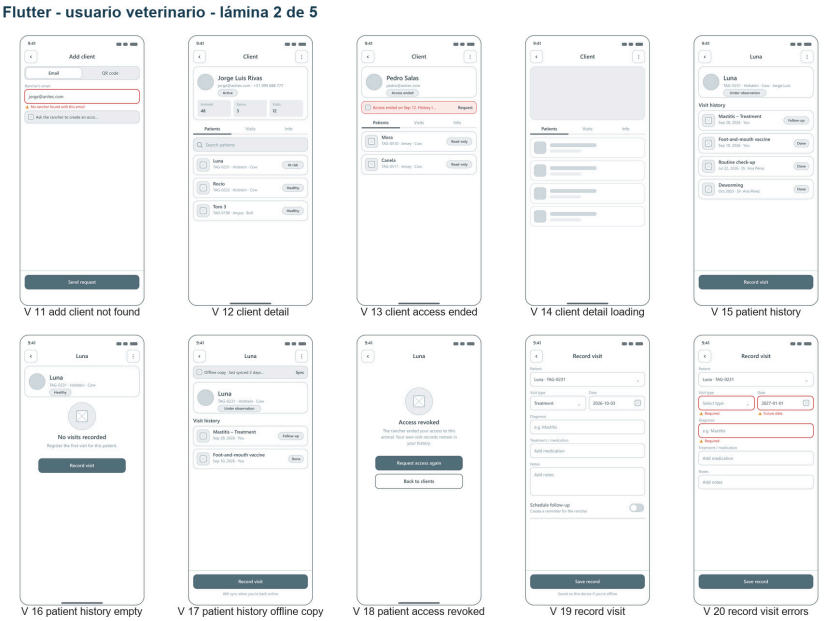


Figura 3.1.4.1.8. Wireframes Flutter del usuario vet, lámina 2 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.9. Wireframes Flutter del usuario vet, lámina 3 de 5. Fuente: elaboración propia.

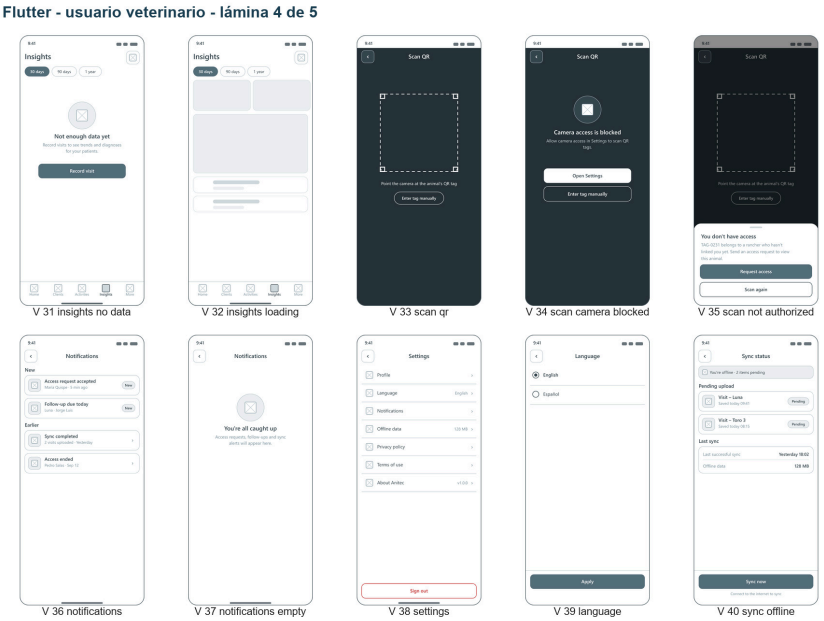


Figura 3.1.4.1.10. Wireframes Flutter del usuario vet, lámina 4 de 5. Fuente: elaboración propia.

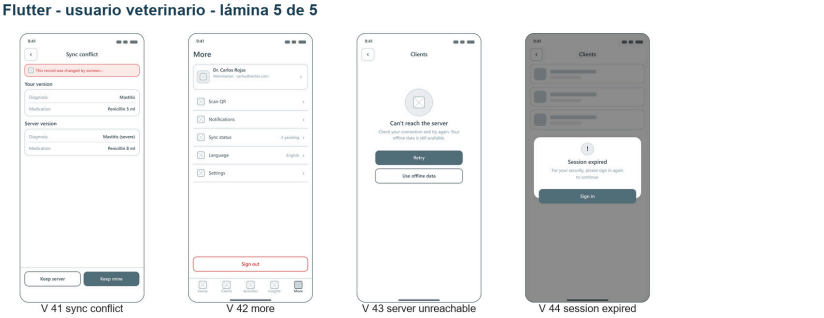


Figura 3.1.4.1.11. Wireframes Flutter del usuario vet, lámina 5 de 5. Fuente: elaboración propia.

Aplicación Android

IAM

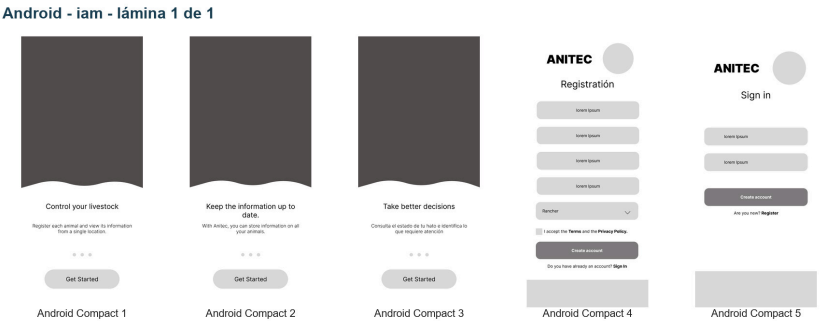


Figura 3.1.4.1.12. Wireframes Android del módulo IAM, lámina 1 de 1. Fuente: elaboración propia.

Usuario rancher

Figura 3.1.4.1.13. Wireframes Android del usuario rancher, lámina 1 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.14. Wireframes Android del usuario rancher, lámina 2 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.15. Wireframes Android del usuario rancher, lámina 3 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.16. Wireframes Android del usuario rancher, lámina 4 de 5. Fuente: elaboración propia.

Figura 3.1.4.1.17. Wireframes Android del usuario rancher, lámina 5 de 5. Fuente: elaboración propia.

Usuario veterinario

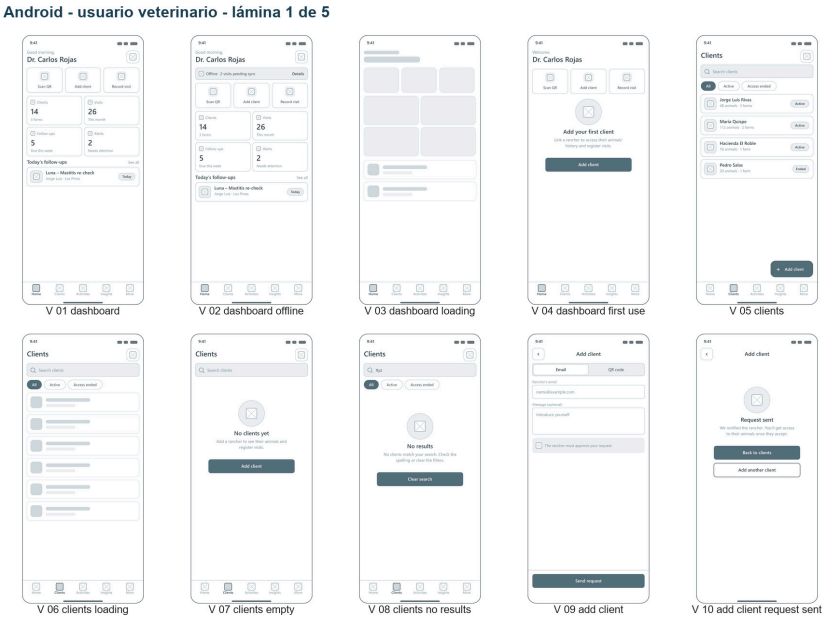


Figura 3.1.4.1.18. Wireframes Android del usuario vet, lámina 1 de 5. Fuente: elaboración propia.

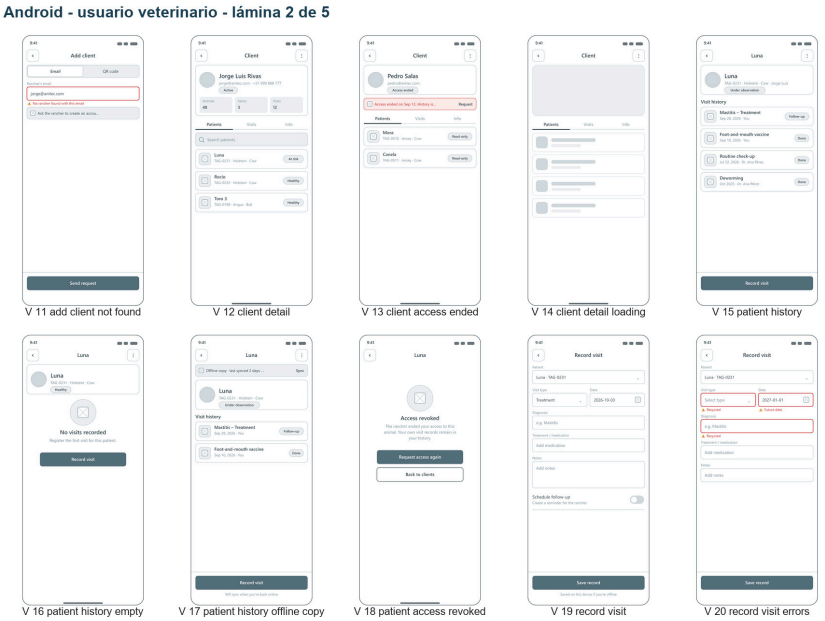


Figura 3.1.4.1.19. Wireframes Android del usuario vet, lámina 2 de 5. Fuente: elaboración propia.

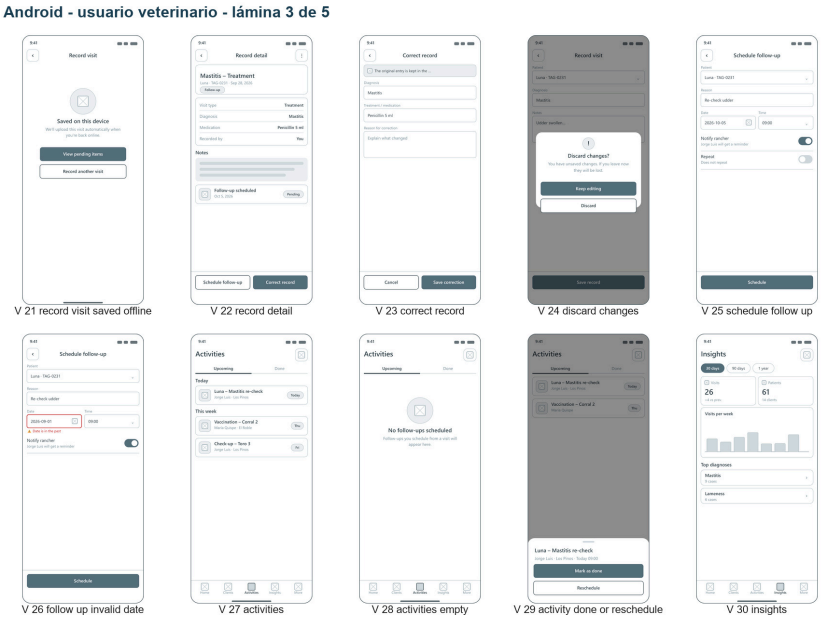


Figura 3.1.4.1.20. Wireframes Android del usuario vet, lámina 3 de 5. Fuente: elaboración propia.

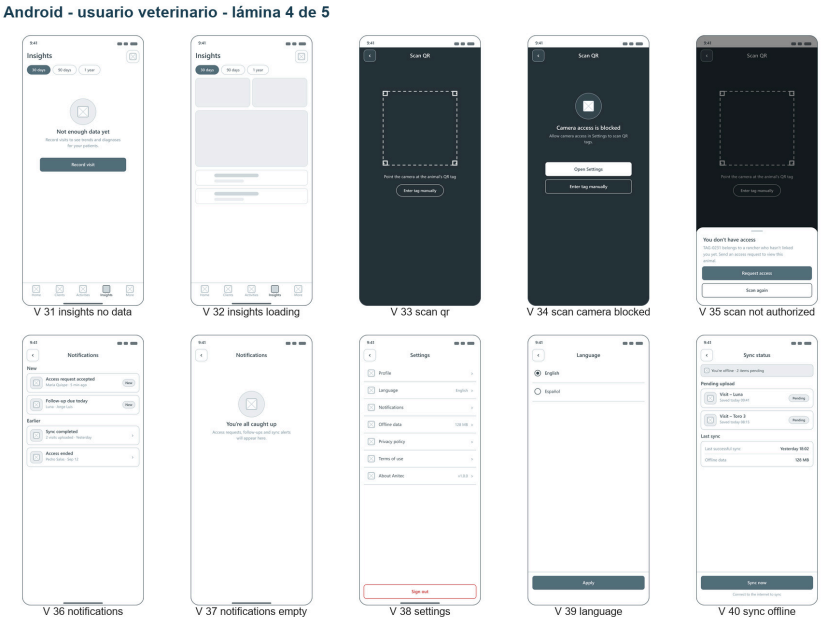


Figura 3.1.4.1.21. Wireframes Android del usuario vet, lámina 4 de 5. Fuente: elaboración propia.

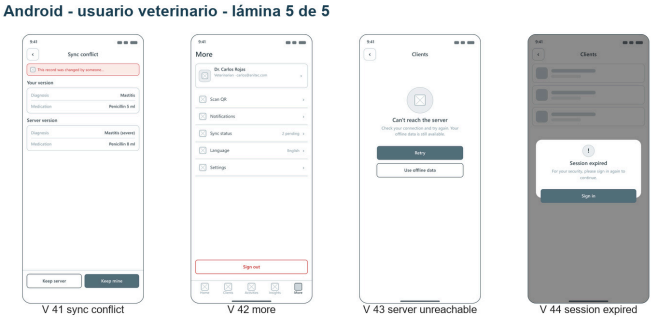


Figura 3.1.4.1.22. Wireframes Android del usuario vet, lámina 5 de 5. Fuente: elaboración propia.

3.1.4.2. Mobile Applications Wireflow Diagrams

Cada wireflow muestra cómo cambia la interfaz después de una acción: La flecha continua representa el *happy path* y las flechas discontinuas rojas, las rutas alternativas (*unhappy paths*); cuando una alternativa no cuenta con una pantalla propia, se representa con un recuadro punteado. Se presenta un wireflow por cada user goal, y por perfil cuando el objetivo lo realizan tanto el ganadero como el veterinario.

User goal	Actor	Punto inicial	Pasos principales	Alternativas	Resultado
Registrarse e iniciar sesión	Ganadero / Veterinario	Bienvenida	Seleccionar rol, ingresar datos y autenticar	Cuenta existente, datos inválidos, error de red	Dashboard del rol
Consultar dashboard	Ambos roles	Sesión autenticada	Cargar resumen y abrir módulo	Sin datos, sin conexión o acceso no autorizado	Recurso seleccionado
Registrar animal	Ganadero	Lista de animales	Completar datos y guardar	Datos incompletos, duplicado u operación offline	Animal registrado o pendiente
Consultar o actualizar animal	Ganadero	Lista o búsqueda	Abrir detalle, revisar y editar	Sin autorización, archivado o error	Información actualizada
Consultar historial sanitario	Usuario autorizado	Detalle del animal	Abrir sanidad, filtrar y revisar	Historial vacío o permiso insuficiente	Evento consultado
Registrar evento sanitario	Usuario autorizado	Historial	Seleccionar tipo, completar y guardar	Validación, falta de permiso o sin conexión	Evento registrado o pendiente

Wireflow 1. Registrarse e iniciar sesión

User goal: Crear mi cuenta con el rol que me corresponde e ingresar a AniTec para llegar a mi dashboard.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria)

El usuario recorre el onboarding de tres pantallas, llega a Registration, elige el tipo de cuenta (Rancher o Veterinarian), completa los datos y accede al dashboard de su rol. Si los datos son inválidos se muestran errores en línea y permanece en el formulario; si ya tiene cuenta, pasa a Sign in; si falla la red, se informa el error y se permite reintentar.

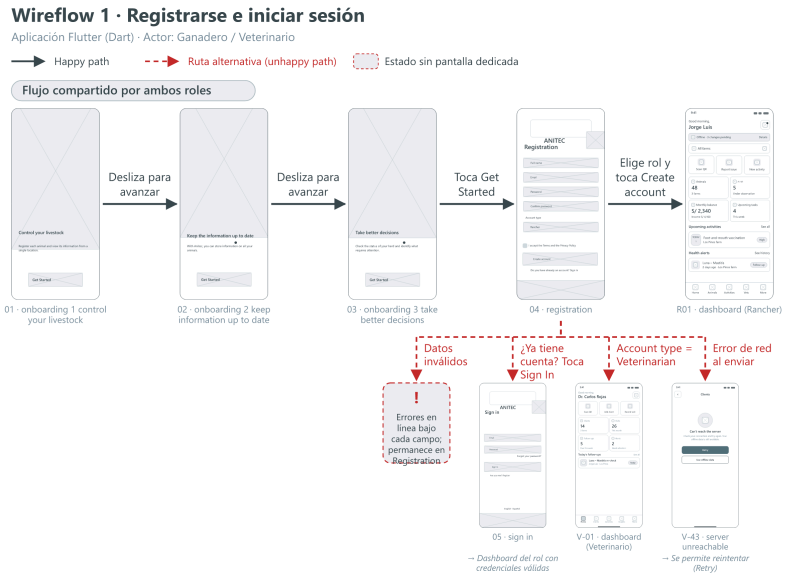


Figura 3.1.4.2.1. Wireflow del user goal «Registrarse e iniciar sesión». Fuente: elaboración propia.

Wireflow 2. Consultar dashboard

User goal: Ver de un vistazo el estado de mi hato, mis actividades y mis alertas (en el caso de la veterinaria, mis clientes y seguimientos pendientes) y entrar al módulo que necesito.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria)

Con la sesión autenticada, el ganadero llega a su dashboard (animales, actividades y alertas) y abre un módulo desde la barra inferior; el veterinario llega al suyo (clientes y seguimientos) y abre Clients. Las alternativas cubren el primer uso sin datos, la carga del resumen, el trabajo sin conexión (banner offline y acceso al estado de sincronización), la sesión vencida, el servicio no disponible y el acceso no autorizado.

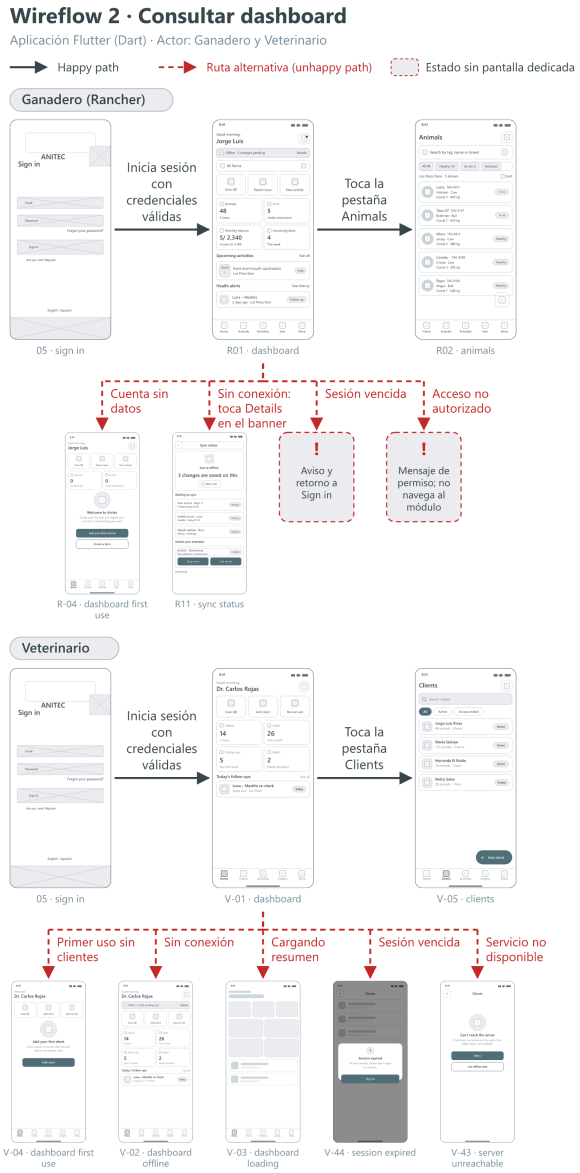


Figura 3.1.4.2.2. Wireflow del user goal «Consultar dashboard». Fuente: elaboración propia.

Wireflow 3. Registrar animal

User goal: Registrar un animal nuevo en pocos pasos, incluso cuando estoy en el campo sin conexión.
User Persona: Jorge Luis Rivas (ganadero)

Desde la lista de animales, el ganadero abre el formulario New animal, completa los datos y guarda; el resultado es el detalle del animal registrado. Si la lista está vacía se ofrece crear el primer animal; los campos incompletos o un tag duplicado se señalan en línea; sin conexión el registro se guarda en el dispositivo y queda pendiente de sincronizar; también puede cambiarse a registro masivo (Bulk).

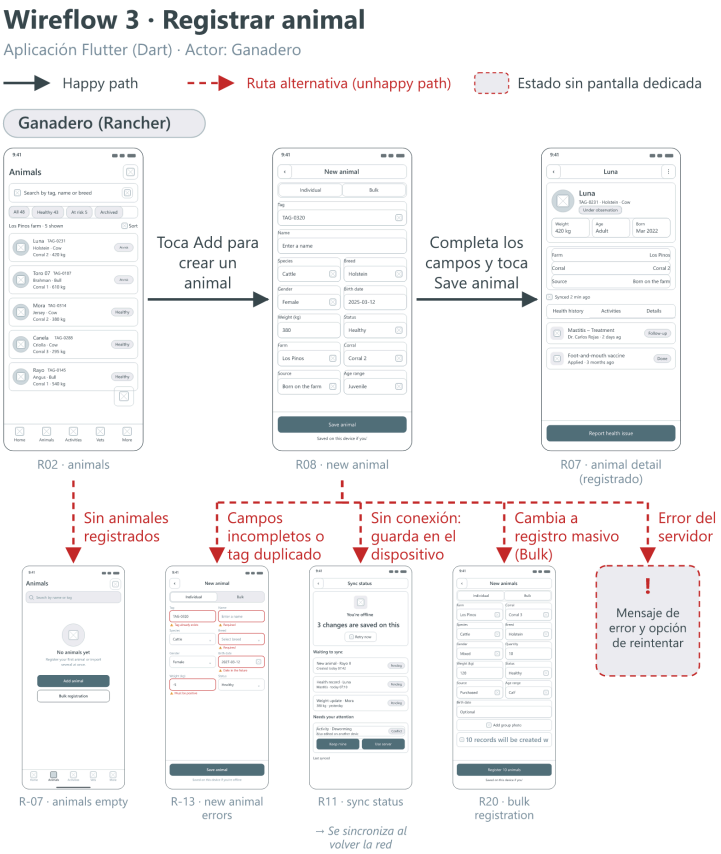


Figura 3.1.4.2.3. Wireflow del user goal «Registrar animal». Fuente: elaboración propia.

Wireflow 4. Consultar o actualizar animal

User goal: Encontrar un animal, revisar su información y mantenerla actualizada.
User Persona: Jorge Luis Rivas (ganadero)

El ganadero busca o selecciona un animal, revisa su detalle, entra a Edit animal, modifica los datos y guarda; el detalle refleja la información actualizada. Una búsqueda sin coincidencias muestra el estado sin resultados; la falta de autorización o un animal archivado dejan el detalle en solo lectura; los datos inválidos se corrigen antes de guardar y un conflicto de sincronización se resuelve en Sync status (Keep mine / Use server).

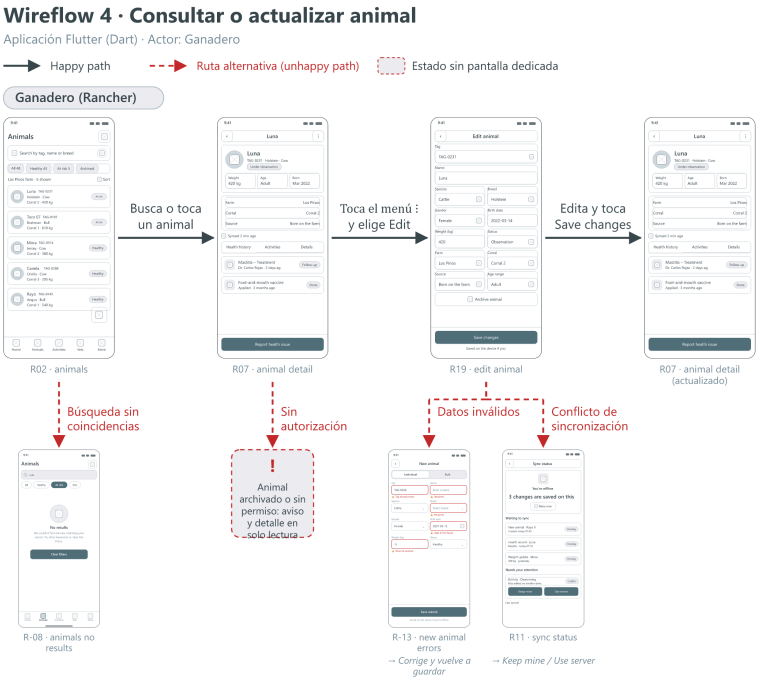


Figura 3.1.4.2.4. Wireflow del user goal «Consultar o actualizar animal». Fuente: elaboración propia.

Wireflow 5. Consultar historial sanitario

User goal: Revisar qué vacunas, tratamientos y controles ha recibido un animal para decidir qué hacer a continuación.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria)

El usuario autorizado abre el historial sanitario del animal (el ganadero desde el detalle del animal; el veterinario desde el detalle del cliente y de su paciente) y revisa el detalle de un registro. Las alternativas son el historial vacío, el animal sin registros, el permiso insuficiente, la copia local sin conexión y el acceso revocado o finalizado por el ganadero.

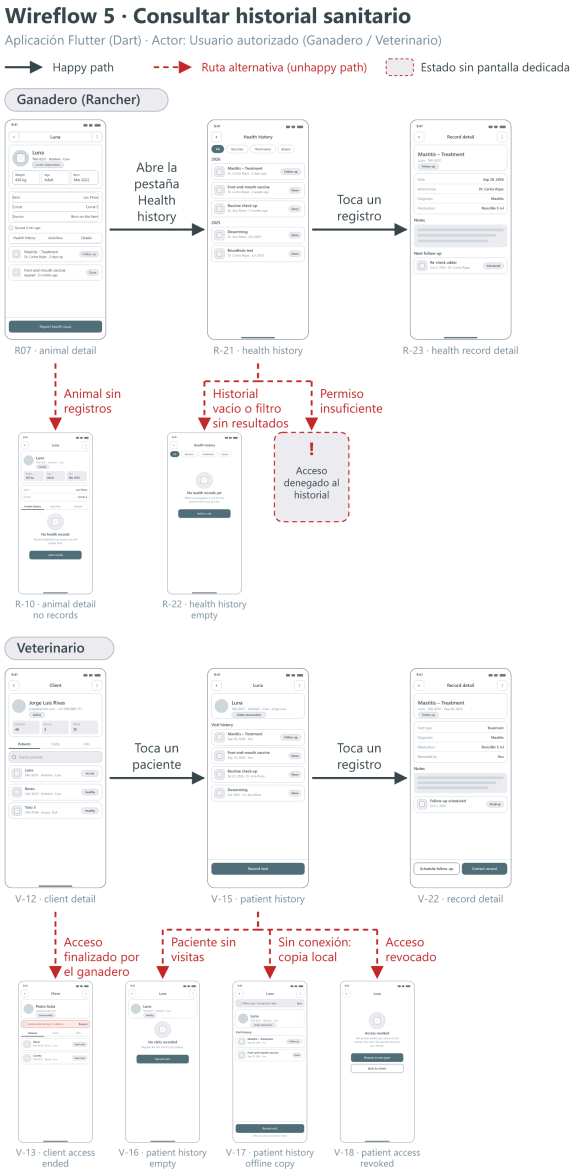


Figura 3.1.4.2.5. Wireflow del user goal «Consultar historial sanitario». Fuente: elaboración propia.

Wireflow 6. Registrar evento sanitario

User goal: Dejar constancia de un evento de salud: reportar un problema (ganadero) o registrar la visita y su seguimiento (veterinaria).
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria)

El ganadero reporta un evento desde el detalle del animal (Report health issue) y el veterinario registra una visita desde el historial del paciente (Record visit); en ambos casos se completa el formulario, se guarda y el evento queda visible en el historial. Los campos obligatorios vacíos se señalan en línea; sin conexión el evento queda guardado localmente y pendiente de sincronizar; sin permiso o con el acceso revocado no se puede registrar; el veterinario puede además programar un seguimiento o descartar los cambios sin guardar.

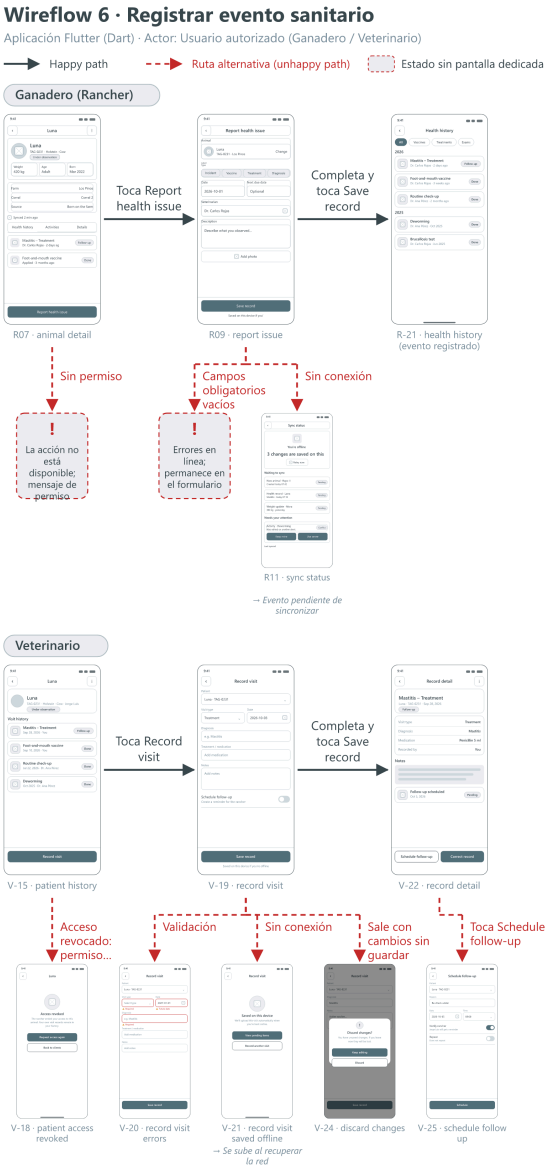


Figura 3.1.4.2.6. Wireflow del user goal «Registrar evento sanitario». Fuente: elaboración propia.

Descripción de los wireflows. Cada wireflow parte de un user goal y de un User Persona de la sección 2.3.1, y muestra cómo cambia la interfaz después de cada acción añadiendo un paso con el estado resultante.

- **Registrarse e iniciar sesión.** El ganadero o la veterinaria abre la aplicación, elige su tipo de cuenta, completa el formulario, acepta los Términos de Servicio y accede al dashboard de su rol. Si los datos son inválidos se marca el campo con el error; si la cuenta ya existe se ofrece iniciar sesión; si falla la red se permite reintentar.
- **Consultar el dashboard.** Desde la sesión autenticada se muestran los indicadores y alertas del rol, y el usuario abre un módulo desde una tarjeta o desde la barra de navegación. Sin datos se invita a registrar el primer elemento; sin conexión se muestran los datos guardados con un aviso.
- **Registrar un animal.** El ganadero abre la lista de animales, elige el registro individual o masivo, completa los datos (código, nombre, especie, finca y corral) y guarda. Si faltan campos obligatorios se resalta el primero; sin conexión el registro se guarda en el dispositivo y queda pendiente de sincronizar.
- **Consultar o actualizar un animal.** Desde la lista o la búsqueda se abre la ficha del animal con su historial y desde allí se edita y se guarda. Si no hay resultados se ofrece limpiar la búsqueda; si el recurso ya no está disponible se explica la situación y se regresa a la lista.

- **Consultar el historial sanitario.** Desde la ficha del animal, o desde un paciente en el caso del veterinario, se abre el historial en orden cronológico. Si está vacío se invita a registrar el primer evento.
- **Registrar un evento sanitario.** Desde el historial se elige el tipo de evento, se completan fecha, descripción y seguimiento, y se guarda. Si falta información se muestra la validación; sin conexión el evento queda pendiente de sincronizar.

3.1.4.3. Mobile Applications Mock-ups

Los mock-ups presentan la propuesta visual de las aplicaciones móviles de AniTec y aplican el Design System definido en la sección 3.1.1 a la estructura validada mediante los wireframes. Las pantallas muestran contenido representativo, controles táctiles, navegación y estados de la interfaz para IAM, el usuario rancher y el usuario veterinario. Para facilitar su revisión se organizan por tecnología y perfil funcional.

El diseño editable y completo de pantallas se encuentran en https://www.figma.com/design/uRmjCeeukXUb2AnZ2kFQsA/Anitec-2026-2?node-id=1-2&t=GnQ9UR4xNp7TYaA1-1

Aplicación del Design System. Los mock-ups usan la paleta, la tipografía y la iconografía de la sección 3.1.1: fondo crema, tarjetas claras con bordes suaves, verde para las acciones principales y la selección activa, y etiquetas de estado, como Saludable o En observación, que combinan color y texto. Las pantallas se organizan con patrones que se repiten (lista con búsqueda y filtros, ficha de detalle, formulario por secciones y tarjeta de resumen), de modo que el usuario reconoce cómo operar un módulo nuevo a partir de otro que ya conoce. Los formularios muestran etiquetas visibles, selectores para los valores controlados y mensajes de validación junto al campo, y la acción de guardar permanece fija al pie de la pantalla.

Estados e inclusión. Para los flujos principales se diseñaron los estados que evitan dejar al usuario sin orientación: carga con marcadores de posición, listas vacías con una acción sugerida, búsquedas sin resultados con la opción de limpiar el filtro, errores de conexión con reintento, aviso de sesión vencida, permiso de cámara denegado con alternativa de ingreso manual del código, cancelación de pago y avisos de que los registros se guardan en el dispositivo y se sincronizan después. Estos estados responden al uso en campo con conexión inestable que motiva el diseño del producto (sección 3.1).

Aplicación Flutter

IAM

Este conjunto presenta el onboarding, el registro y el inicio de sesión compartidos por los usuarios de la aplicación. La secuencia progresiva reduce la cantidad de información mostrada en cada paso; los formularios emplean etiquetas visibles, jerarquía tipográfica y una acción primaria destacada.

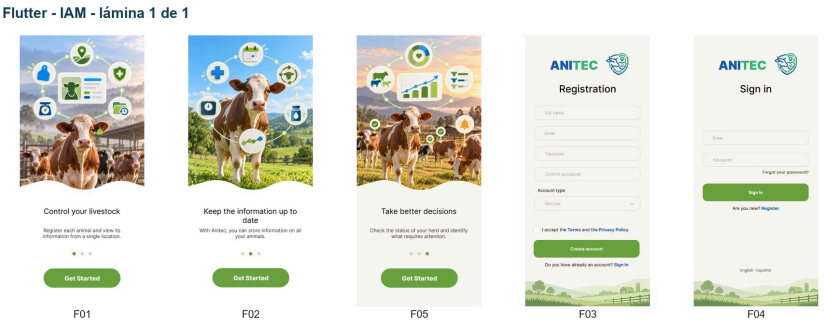


Figura 10. Mock-ups Flutter del módulo IAM. Fuente: elaboración propia.

Usuario rancher

Las siguientes láminas recorren la experiencia del ganadero: dashboard, animales, actividades, veterinarios, escaneo QR, finanzas, suscripción, configuración y los estados asociados a estas operaciones. El dashboard aplica jerarquía visual para priorizar alertas, indicadores y accesos frecuentes; la navegación agrupa las funciones por tareas del dominio; y las listas combinan etiquetas, búsqueda, filtros y estados.

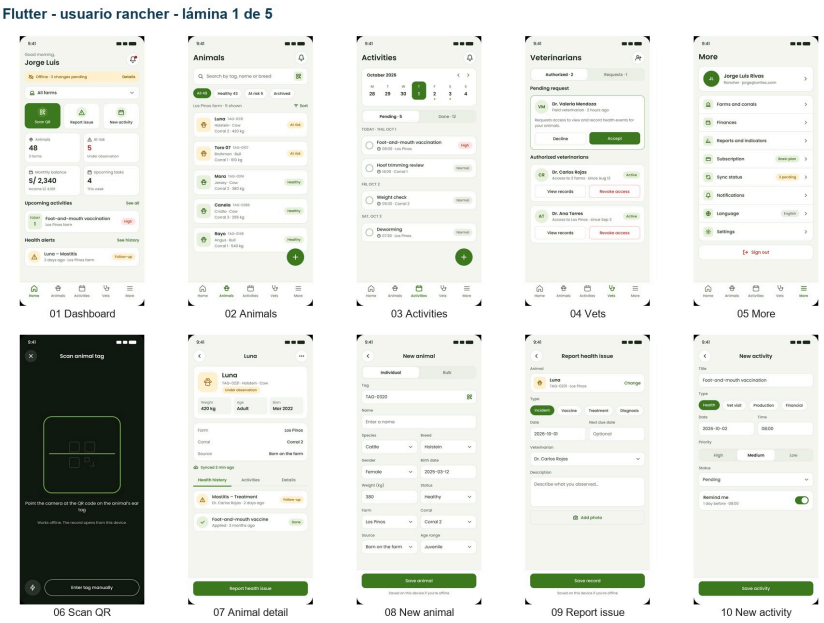


Figura 11. Mock-ups Flutter del usuario rancher, lámina 1 de 5. Fuente: elaboración propia.

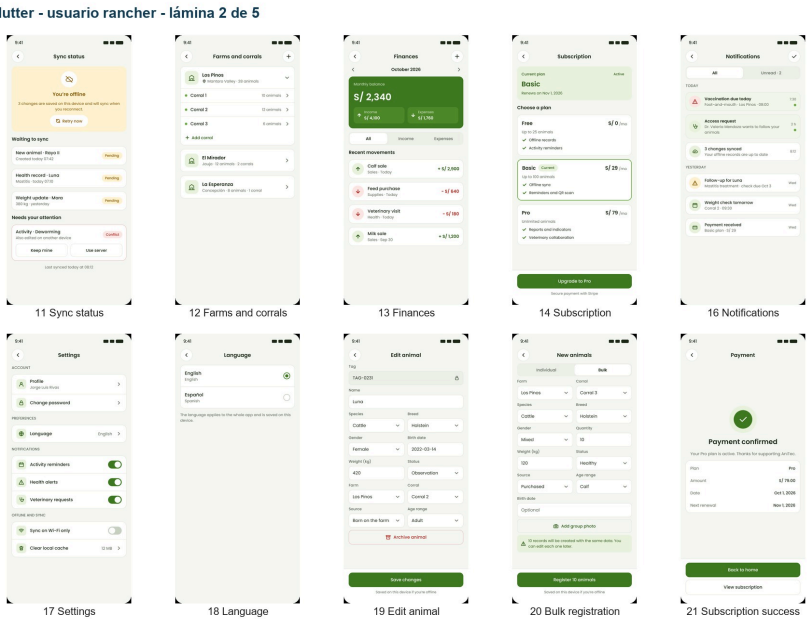


Figura 12. Mock-ups Flutter del usuario rancher, lámina 2 de 5. Fuente: elaboración propia.

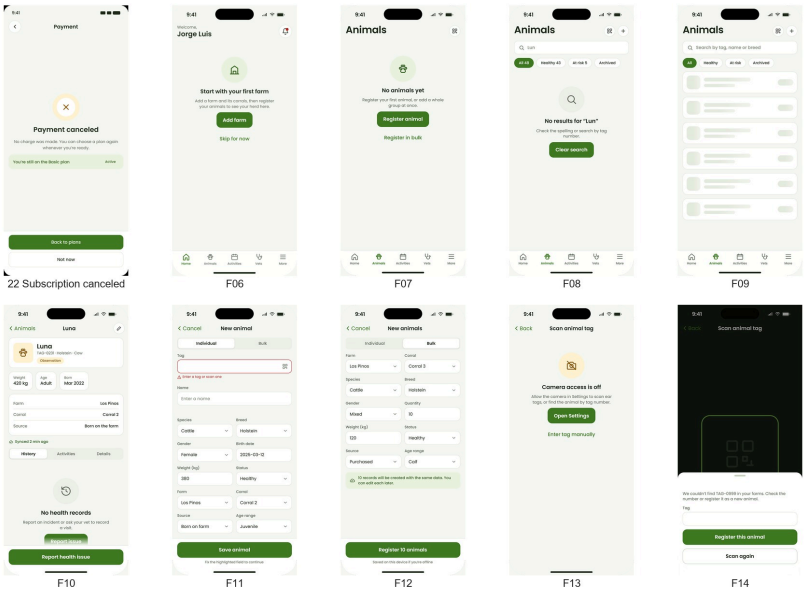


Figura 13. Mock-ups Flutter del usuario rancher, lámina 3 de 5. Fuente: elaboración propia.

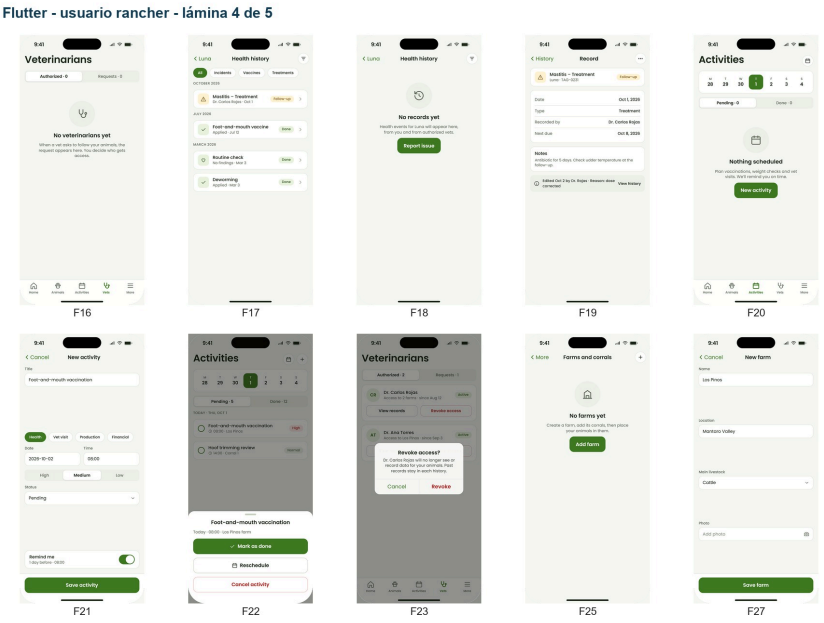


Figura 14. Mock-ups Flutter del usuario rancher, lámina 4 de 5. Fuente: elaboración propia.

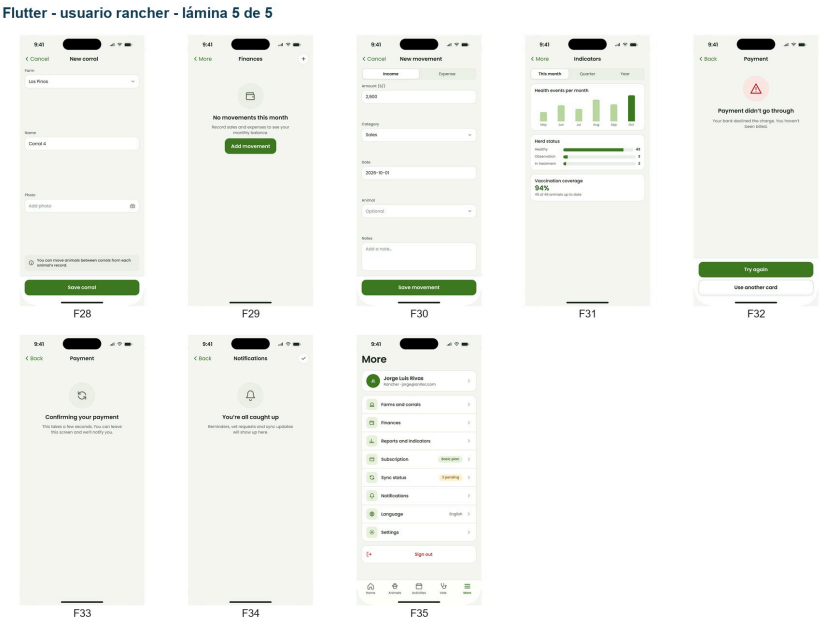


Figura 15. Mock-ups Flutter del usuario rancher, lámina 5 de 5. Fuente: elaboración propia.

Usuario veterinario

Las pantallas del veterinario cubren el dashboard profesional, la gestión de clientes, el historial clínico, el registro y corrección de visitas, el seguimiento, las actividades, los indicadores y la configuración. La arquitectura por audiencia presenta solamente la información autorizada para este rol; los clientes y pacientes se organizan mediante búsqueda, filtros y orden cronológico

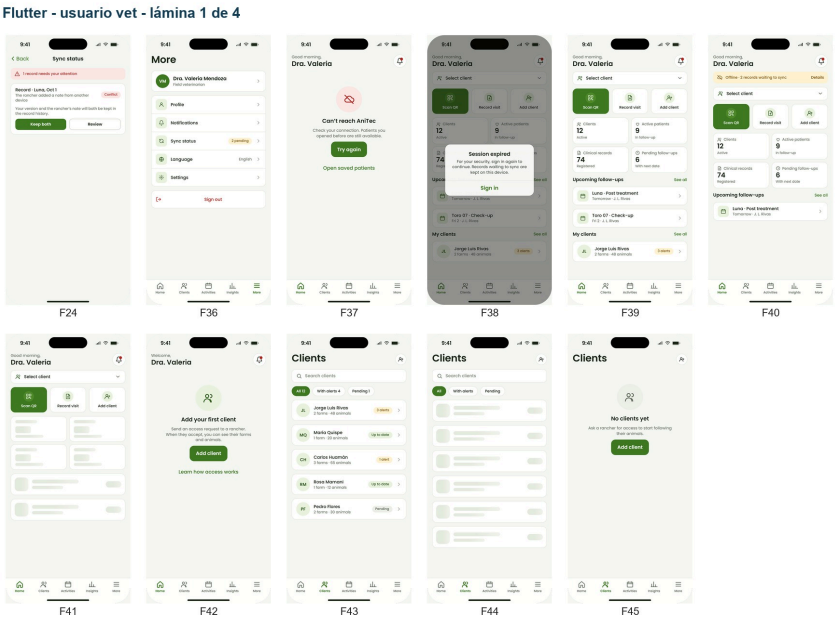


Figura 16. Mock-ups Flutter del usuario vet, lámina 1 de 4. Fuente: elaboración propia.

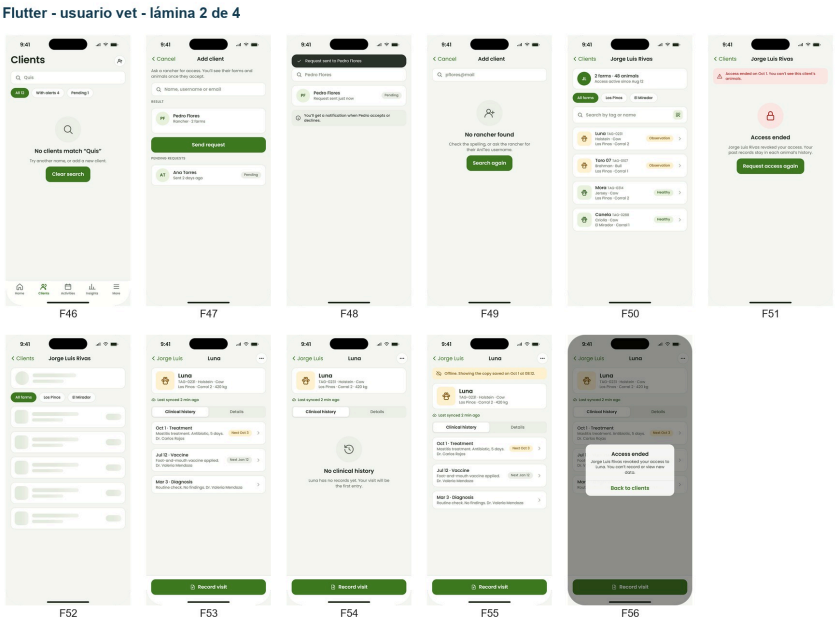


Figura 17. Mock-ups Flutter del usuario vet, lámina 2 de 4. Fuente: elaboración propia.

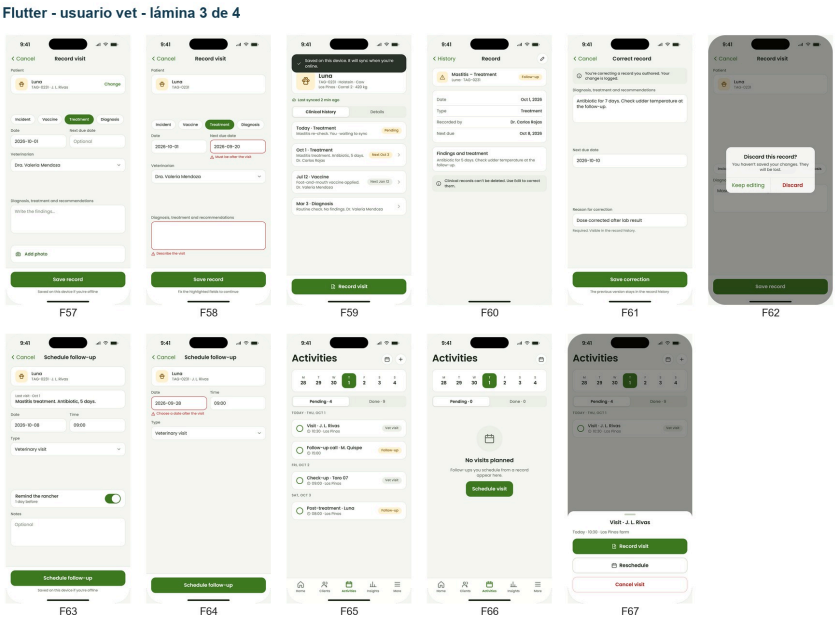


Figura 18. Mock-ups Flutter del usuario vet, lámina 3 de 4. Fuente: elaboración propia.

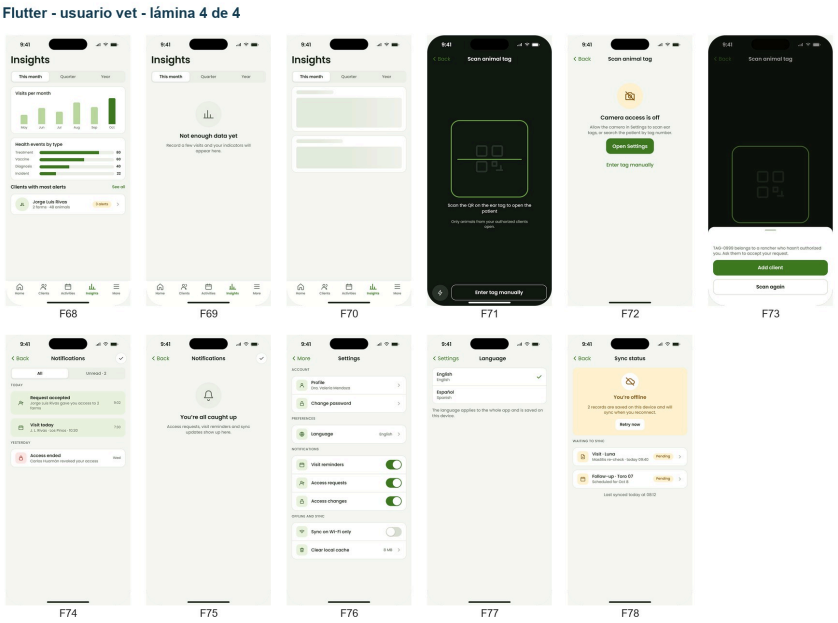


Figura 19. Mock-ups Flutter del usuario vet, lámina 4 de 4. Fuente: elaboración propia.

Aplicación Android

IAM

La versión Android conserva el mismo alcance funcional del acceso y adapta la presentación a los patrones visuales de Material Design. Mantiene la identidad de AniTec, la jerarquía y las etiquetas del flujo Flutter, pero ajusta campos, botones y controles de navegación a las convenciones de Android para conservar familiaridad y consistencia externa.

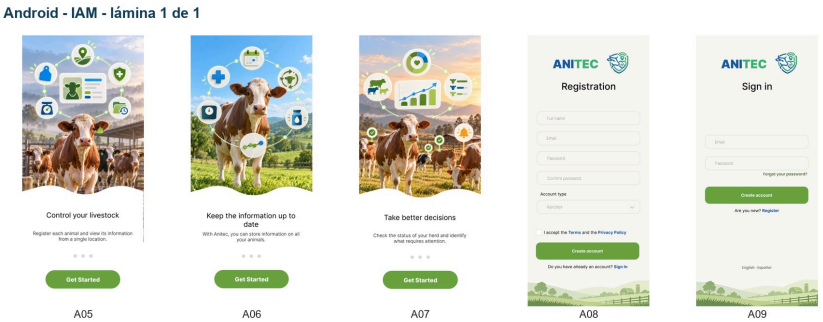


Figura 20. Mock-ups Android del módulo IAM. Fuente: elaboración propia.

Usuario rancher

Las láminas Android mantienen la secuencia funcional del usuario rancher e incluyen vistas principales, formularios, estados vacíos, confirmaciones, errores y sincronización. La misma arquitectura por tareas permite cambiar de plataforma sin reaprender el producto: Inicio resume la situación del hato, Animals concentra la gestión del ganado, Activities organiza el trabajo y More agrupa funciones de menor frecuencia. Los formularios conservan etiquetas persistentes, acciones principales visibles y mensajes que explican cómo corregir o reintentar.

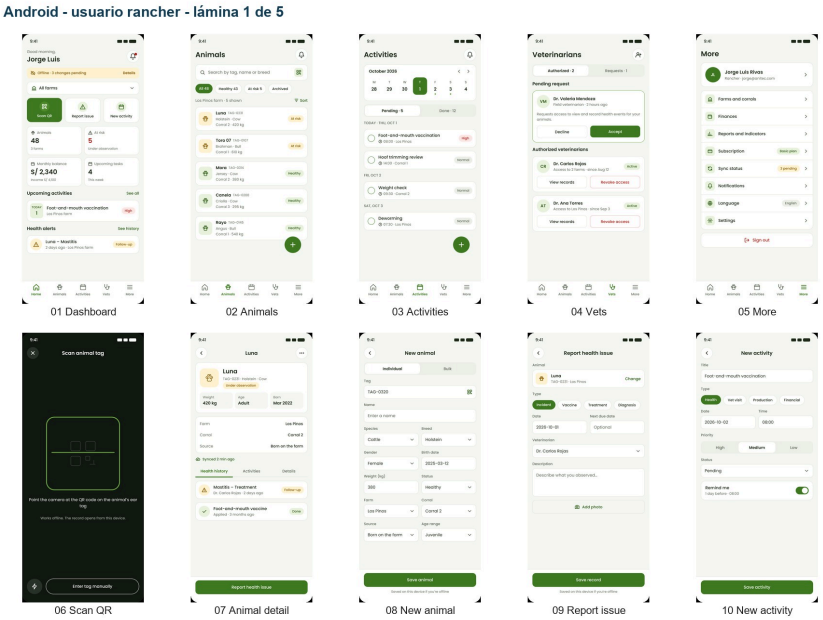


Figura 21. Mock-ups Android del usuario rancher, lámina 1 de 5. Fuente: elaboración propia.

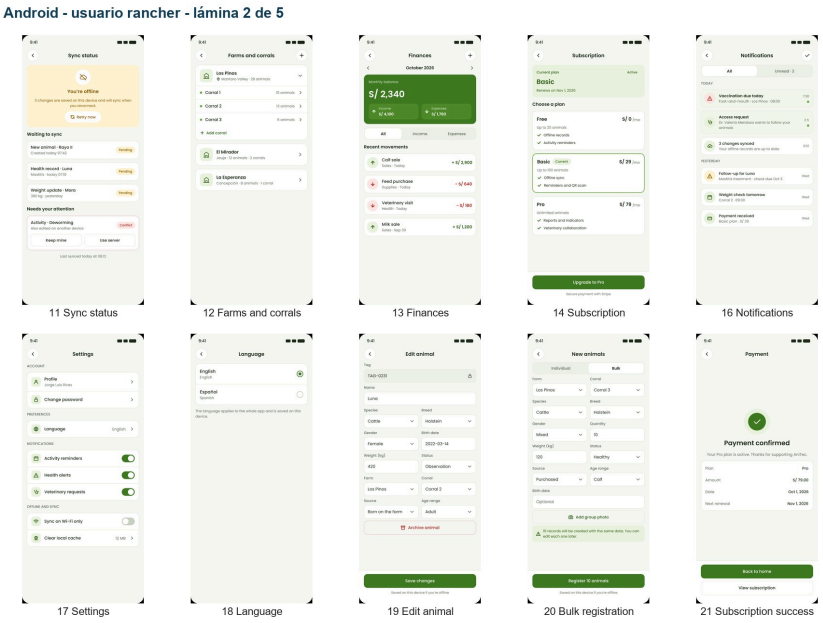


Figura 22. Mock-ups Android del usuario rancher, lámina 2 de 5. Fuente: elaboración propia.

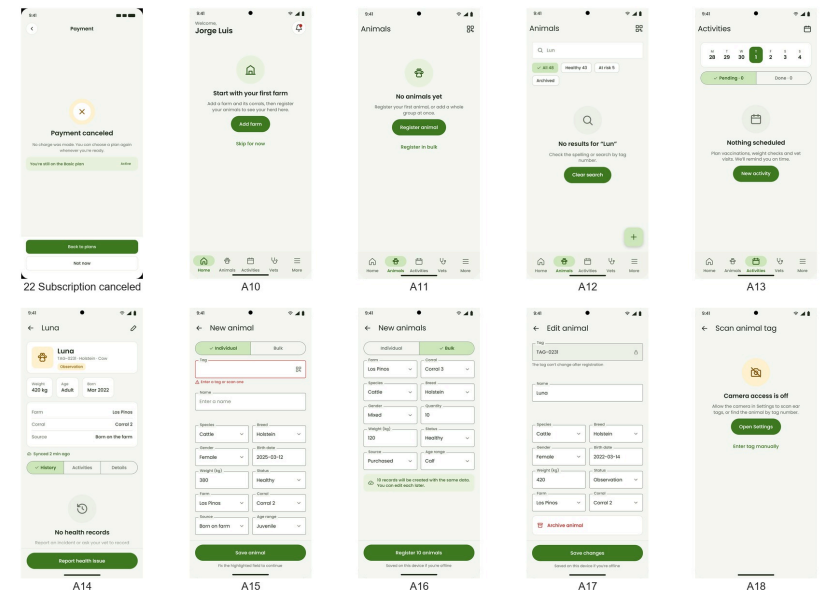


Figura 23. Mock-ups Android del usuario rancher, lámina 3 de 5. Fuente: elaboración propia.

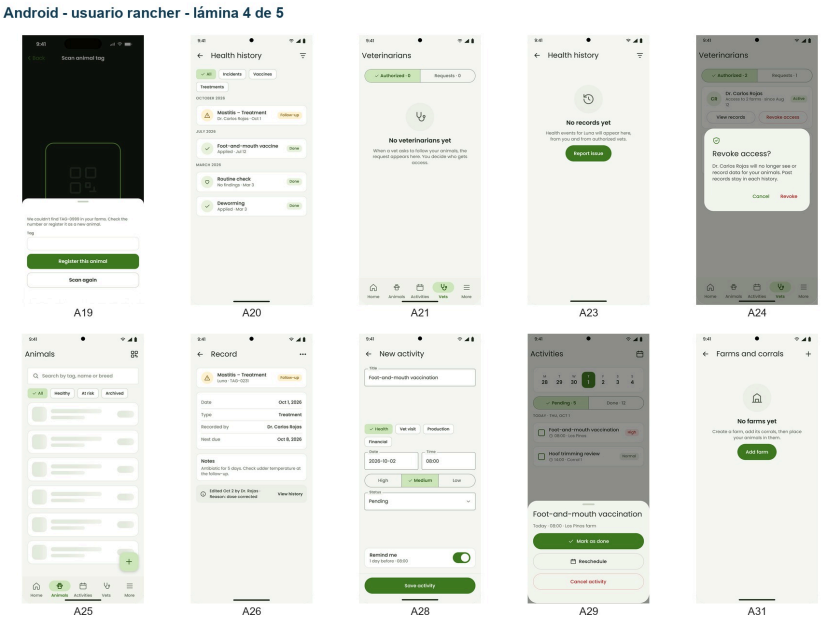


Figura 24. Mock-ups Android del usuario rancher, lámina 4 de 5. Fuente: elaboración propia.

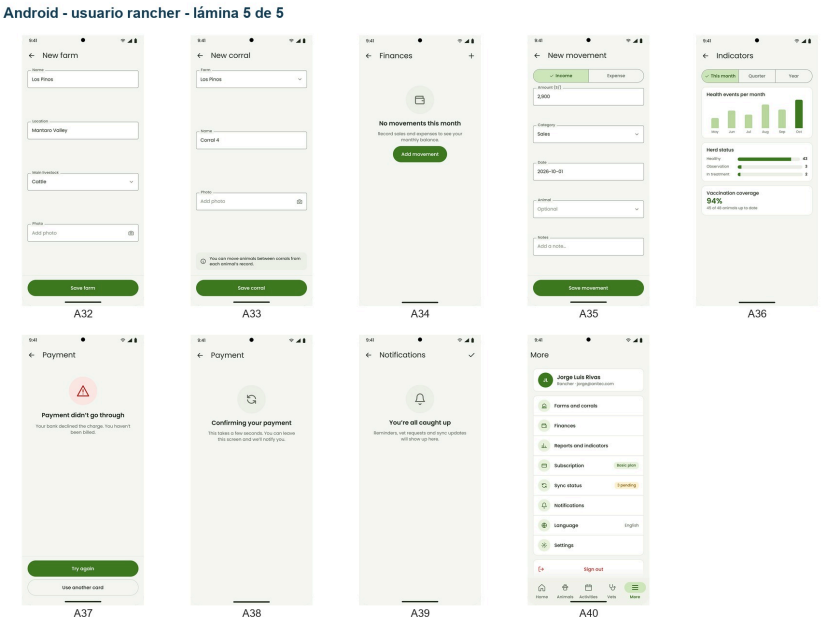


Figura 25. Mock-ups Android del usuario rancher, lámina 5 de 5. Fuente: elaboración propia.

Usuario veterinario

La versión Android del perfil veterinario documenta la gestión de clientes y visitas, los estados operativos, los indicadores, el escaneo, las notificaciones y las opciones de cuenta. El dashboard prioriza clientes, visitas y seguimientos; los historiales siguen una organización cronológica; y las pantallas de autorización, validación y estados vacíos combinan iconografía, texto y acciones de recuperación. Así, la adaptación a Android conserva tanto el Design System como los criterios de inclusión y arquitectura de información definidos para AniTec.

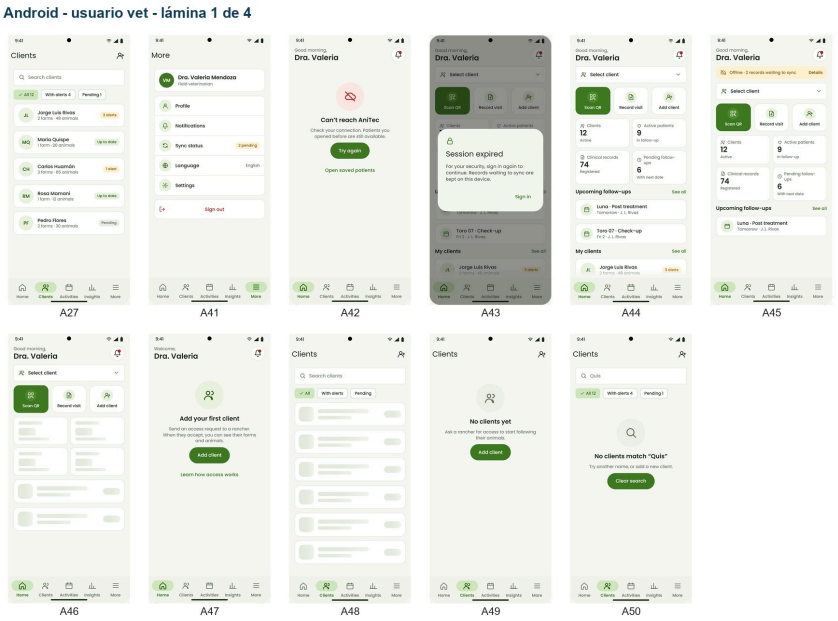


Figura 26. Mock-ups Android del usuario vet, lámina 1 de 4. Fuente: elaboración propia.

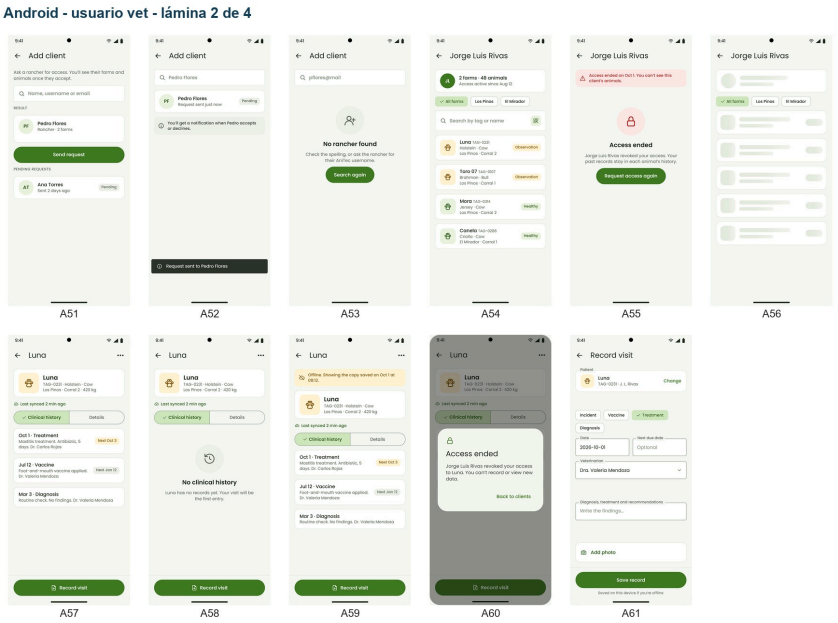


Figura 27. Mock-ups Android del usuario vet, lámina 2 de 4. Fuente: elaboración propia.

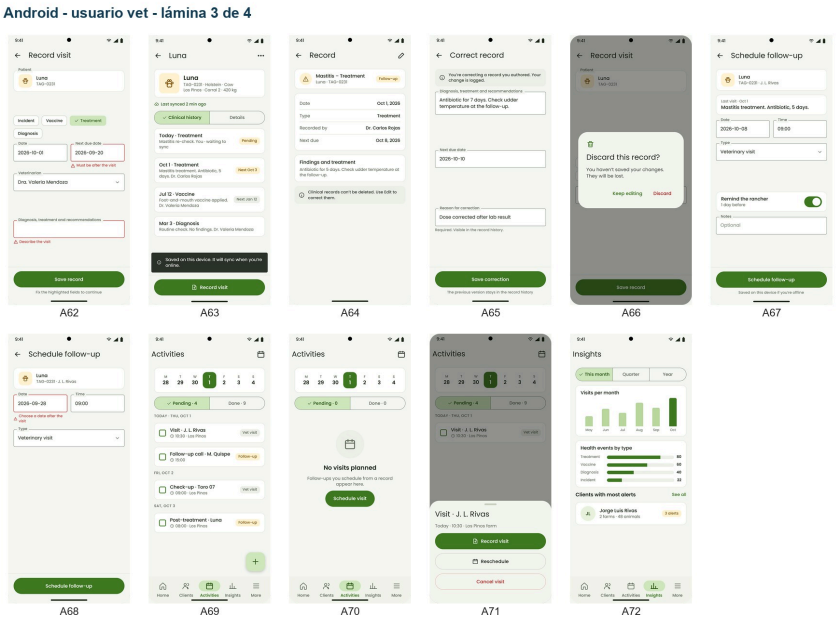


Figura 28. Mock-ups Android del usuario vet, lámina 3 de 4. Fuente: elaboración propia.

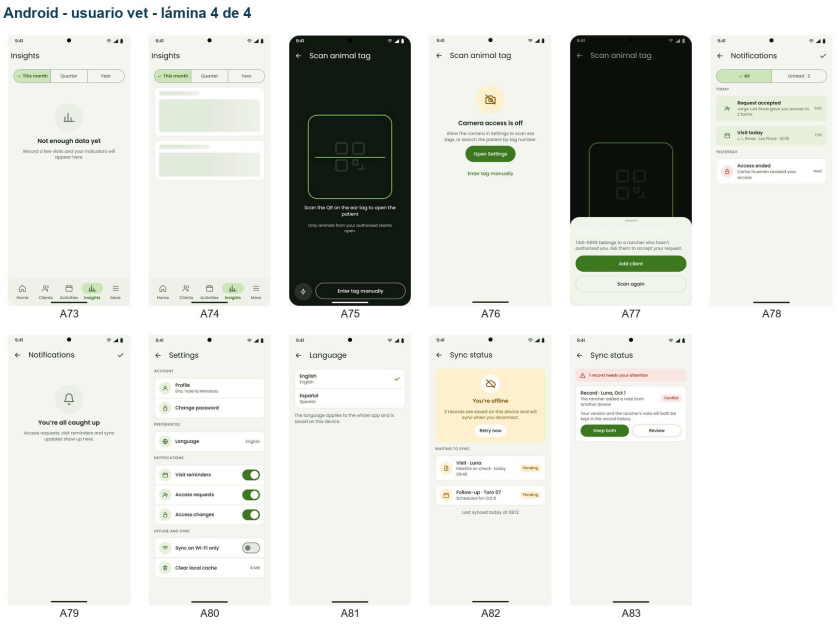


Figura 29. Mock-ups Android del usuario vet, lámina 4 de 4. Fuente: elaboración propia.

3.1.4.4. Mobile Applications User Flow Diagrams

Los User Flow Diagrams presentan la navegación propuesta para alcanzar cada user goal definido. Se establece un único flujo estándar por objetivo, aplicable tanto a Android como a Flutter, porque ambas implementaciones comparten la misma arquitectura de información, las mismas acciones esenciales y las mismas condiciones de recuperación. Los diagramas se derivan de los wireflows y emplean mock-ups representativos de las vistas involucradas. Para conservar la legibilidad, cada lámina resume el recorrido en tres pantallas principales y concentra las condiciones secundarias en bloques breves; su explicación detallada se mantiene en el texto. La línea verde continua representa el *happy path*; la línea roja discontinua representa los *unhappy paths* que requieren corrección, reintento o recuperación; y la línea ámbar discontinua identifica una ruta alternativa válida que no constituye un error.

User goal	Happy path	Unhappy paths	Relación
Registro e inicio de sesión	Datos válidos y dashboard	Datos inválidos, cuenta existente, credenciales incorrectas y sin red	US-004, US-005, US-006
Dashboard	Resumen disponible y acceso a módulo	Sin datos, sesión vencida y error del servicio	US-007 y flujos del rol
Registro de animal	Formulario válido y confirmación	Campos faltantes, duplicado, servidor y guardado offline	US-011, TS-004, TS-005
Consulta y actualización	Recurso autorizado y actualización	Sin permiso, inexistente y conflicto de sincronización	US-010, US-012, US-013
Historial sanitario	Historial disponible	Historial vacío, filtro sin resultados y acceso denegado	US-014
Registro sanitario	Datos válidos y confirmación	Validación, permiso insuficiente y sin conexión	US-015

User Flow 1. Registrarse e iniciar sesión

User goal: Crear mi cuenta con el rol que me corresponde e ingresar a AniTec para llegar a mi dashboard.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria).

Este flujo se deriva del Wireflow 1. En el *happy path*, la persona recorre el onboarding, elige crear una cuenta o iniciar sesión, completa los datos requeridos y llega al dashboard correspondiente a su rol. Si el registro contiene datos inválidos, se mantienen los valores ingresados y se señalan los campos por corregir. Una cuenta existente conduce a Sign in como ruta alternativa válida. Las credenciales incorrectas permiten reintentar y un fallo de conexión conserva el contexto antes de volver a enviar la solicitud.

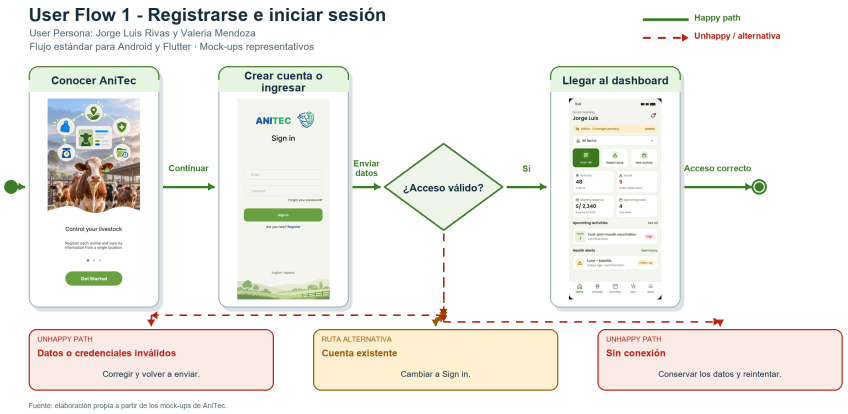


Figura 3.1.4.4.1. User Flow para registrarse e iniciar sesión. Fuente: elaboración propia.

User Flow 2. Consultar dashboard

User goal: Ver de un vistazo el estado de mi hato, mis actividades y mis alertas o, para la veterinaria, mis clientes y seguimientos pendientes, y entrar al módulo que necesito.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria).

Este flujo se deriva del Wireflow 2. El *happy path* parte de una sesión autenticada, carga el dashboard ajustado al rol y continúa al módulo elegido desde la navegación. En el primer uso se muestra una llamada a crear la primera finca o agregar el primer cliente; durante la carga se conservan estructuras que anticipan el contenido. Sin conexión se informa el uso de datos locales; una sesión vencida solicita autenticación y los errores de servicio o autorización ofrecen reintento o recuperación de acceso.

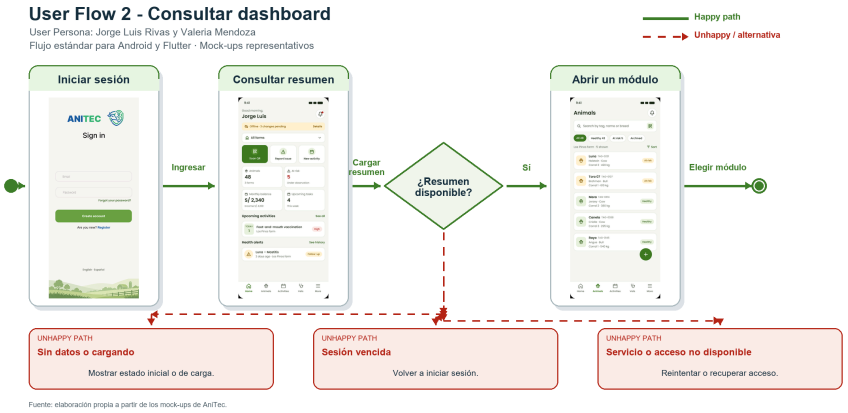


Figura 3.1.4.4.2. User Flow para consultar el dashboard. Fuente: elaboración propia.

User Flow 3. Registrar animal

User goal: Registrar un animal nuevo en pocos pasos, incluso cuando estoy en el campo sin conexión.
User Persona: Jorge Luis Rivas (ganadero).

Este flujo se deriva del Wireflow 3. En la ruta esperada, Jorge abre Animals, toca la acción de nuevo registro, completa el formulario, guarda y revisa el detalle creado. Si la lista está vacía se ofrece registrar el primer animal. Los campos incompletos o un tag duplicado mantienen el formulario abierto con mensajes de corrección; sin conexión el registro queda pendiente en Sync status. El formulario Bulk constituye una alternativa válida cuando necesita registrar varios animales.

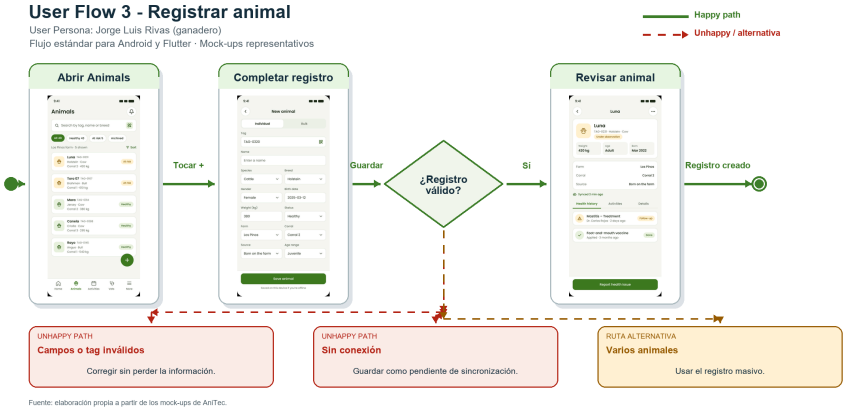


Figura 3.1.4.4.3. User Flow para registrar un animal. Fuente: elaboración propia.

User Flow 4. Consultar o actualizar animal

User goal: Encontrar un animal, revisar su información y mantenerla actualizada.
User Persona: Jorge Luis Rivas (ganadero).

Este flujo se deriva del Wireflow 4. El *happy path* comienza con la búsqueda o selección de un animal, continúa con la revisión del detalle y la edición y termina mostrando los datos actualizados. Una búsqueda sin coincidencias permite limpiar los filtros; la falta de autorización o el estado archivado mantienen el recurso en solo lectura. Los datos inválidos se corrigen antes de guardar y los conflictos de sincronización se resuelven eligiendo entre la versión local y la versión del servidor.

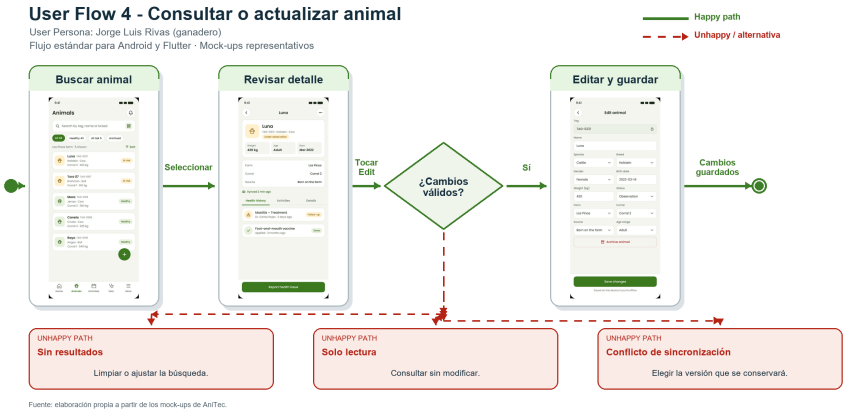


Figura 3.1.4.4.4. User Flow para consultar o actualizar un animal. Fuente: elaboración propia.

User Flow 5. Consultar historial sanitario

User goal: Revisar qué vacunas, tratamientos y controles ha recibido un animal para decidir qué hacer a continuación.
User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria).

Este flujo se deriva del Wireflow 5 y muestra los puntos de entrada de ambos perfiles. Jorge parte del detalle del animal y Valeria del cliente o paciente autorizado; ambos abren Health history y seleccionan un registro. Si no existen antecedentes, se ofrece crear el primero; si una búsqueda no devuelve resultados, se modifican sus criterios. El contenido protegido no se muestra sin permiso, la copia local se identifica como potencialmente desactualizada y un acceso revocado finaliza la consulta indicando cómo solicitar una nueva autorización.

Figura 3.1.4.4.5. User Flow para consultar el historial sanitario. Fuente: elaboración propia.

User Flow 6. Registrar evento sanitario

User goal: Dejar constancia de un evento de salud: reportar un problema como ganadero o registrar la visita y su seguimiento como veterinaria.

User Persona: Jorge Luis Rivas (ganadero) y Valeria Mendoza (médica veterinaria).

Este flujo se deriva del Wireflow 6. Jorge utiliza Report health issue desde el detalle del animal y Valeria utiliza Record visit desde el historial del paciente; ambos completan los datos, guardan y comprueban que el evento aparezca en el historial. Los campos obligatorios vacíos mantienen el formulario abierto, el trabajo sin conexión guarda una copia pendiente y un permiso revocado bloquea el registro. Antes de salir sin guardar se solicita confirmación. Como ruta alternativa válida, Valeria puede programar un seguimiento posterior a la visita.

Figura 3.1.4.4.6. User Flow para registrar un evento sanitario. Fuente: elaboración propia.

Descripción de los User Flows. Cada User Flow reutiliza los mock-ups del wireflow correspondiente y agrega las decisiones que separan el camino esperado (happy path) de los caminos alternativos (unhappy paths). Comienza en la pantalla desde la que el usuario inicia la tarea y termina en la que confirma el resultado.

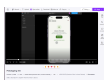
- **Registro e inicio de sesión:** bienvenida, formulario, validación y dashboard del rol. Decisiones: ¿los datos son válidos?, ¿la cuenta ya existe?, ¿hay conexión?
- **Dashboard:** sesión autenticada, carga, resumen y módulo elegido. Decisiones: ¿hay datos?, ¿la sesión sigue vigente?
- **Registro de animal:** lista, formulario individual o masivo, validación y confirmación. Decisiones: ¿los campos están completos?, ¿el código ya existe?, ¿hay conexión o debe guardarse localmente?
- **Consulta y actualización:** lista o búsqueda, ficha, edición y confirmación. Decisiones: ¿hay resultados?, ¿el usuario tiene permiso?
- **Historial sanitario:** ficha o paciente e historial. Decisiones: ¿hay eventos?, ¿el filtro devuelve resultados?, ¿el acceso está autorizado?
- **Registro sanitario:** historial, formulario, validación y confirmación. Decisiones: ¿los datos son válidos?, ¿el usuario tiene permiso?, ¿hay conexión?

3.1.4.5. Mobile Applications Prototyping

Los prototipos simularán la navegación de los User Flow Diagrams y permitirán comprobar etiquetas, acciones, retroalimentación, recuperación ante errores y consistencia.

Los criterios de interacción de los prototipos derivan de la arquitectura de información de la sección 3.1.2 y de los User Flows de la sección 3.1.4.4. La navegación principal se resuelve con una barra inferior de cinco destinos por rol y las funciones secundarias se agrupan en «Más». Las tareas de creación se inician desde un botón de acción visible en

cada lista, los formularios se abren a pantalla completa con retorno explícito y las acciones destructivas piden confirmación. Cada interacción devuelve retroalimentación inmediata, como la selección activa en la barra, los mensajes de validación junto al campo, las confirmaciones y los avisos de sincronización, y los errores ofrecen una acción de recuperación: reintentar, limpiar la búsqueda o ingresar el código manualmente. Con ello se comprueban las etiquetas, la jerarquía, la recuperación ante errores y la consistencia entre pantallas.

Usuario	Prototipo Evidencia	Captura del video	Video en Microsoft Stream
Ganadero			https://upcedupe-my.sharepoint.com/ ✌🏻 /g/personal/u202215979_upc_edu_pe/IQAoOjKIja11SZkyivBWNnFHAfqWrhmYFjAE e=nkIv3O
Veterinario			https://upcedupe-my.sharepoint.com/ ✌🏻 /g/personal/u202215979_upc_edu_pe/IQC1i57fMYVmS5YQVocXyarQAfhuAk37q3v e=KRkjwP

Enlace de diseño en Figma: https://www.figma.com/design/uRmjCeeukXUb2AnZ2kFQsA/Anitec-2026-2?node-id=1-2&t=GnQ9UR4xNp7TYaA1-1

Capítulo IV: Product Implementation & Validation

4. Product Implementation & Validation

Este capítulo describe la configuración, planificación, implementación, comprobación, despliegue y validación de los productos que forman AniTec. El alcance comprende la Landing Page, los servicios REST desarrollados con ASP.NET Core, la aplicación Android nativa y la aplicación multiplataforma. El avance se organiza en sprints y se sustenta mediante evidencias trazables a los repositorios, pruebas, documentación OpenAPI, entornos publicados y sesiones con usuarios.

Para el TB1 se documenta el Sprint 1. Los campos marcados como pendientes deberán completarse únicamente con información y evidencia producida durante la iteración vigente.

4.1. Software Configuration Management

La gestión de configuración de AniTec define las herramientas, repositorios, convenciones y procesos de publicación que permiten trabajar sobre la Landing Page, la API REST y las aplicaciones móviles de manera consistente y trazable.

4.1.1. Software Development Environment Configuration

Las herramientas se agrupan según la actividad que soportan e incluyen su propósito y su ruta de referencia (servicios en línea) o de descarga (programas instalados). Las tecnologías del proyecto son Figma, Lucidchart, Structurizr, PlantUML, HTML5/CSS3/JavaScript, ASP.NET Core con OpenAPI vía Swagger, Kotlin en Android, Flutter con Dart, Trello y Git con GitHub. Todo integrante debe utilizar versiones compatibles con los repositorios y registrar cualquier cambio de versión que afecte la compilación.

Project Management

Herramienta	Propósito	Referencia
Trello	Organizar el Sprint Backlog y los estados de las tareas (Todo, In-Process, To-Review, Done).	https://trello.com
GitHub Issues / Projects	Dar trazabilidad a incidencias, cambios y trabajo de cada repositorio.	https://github.com/features/issues

Requirements Management

Herramienta	Propósito	Referencia
UXPressia	Elaborar User Personas, Empathy Maps, Journey Maps e Impact Maps.	https://uxpressia.com
Miro	Modelar el EventStorming y otros flujos colaborativos.	https://miro.com
Trello	Mantener el Product Backlog con estimación y prioridad.	https://trello.com

Herramienta	Propósito	Referencia
Gherkin	Especificar criterios de aceptación y escenarios BDD en archivos .feature.	https://cucumber.io/docs/gherkin/

Product UX/UI Design

Herramienta	Propósito	Referencia
Figma	Elaborar wireframes, mock-ups y prototipos de la Landing Page y de las aplicaciones móviles.	https://www.figma.com
Lucidchart	Elaborar wireflows, user flows, diagramas UML y el diseño de la base de datos.	https://www.lucidchart.com
Canva	Preparar recursos visuales complementarios.	https://www.canva.com
Material Design 3	Lenguaje de diseño y componentes de la aplicación Android.	https://m3.material.io

Software Development

Control de versiones y edición

Herramienta	Propósito	Referencia	Versión
Git y GitHub	Control de versiones y colaboración.	https://git-scm.com/downloads · https://github.com	Versión estable
Visual Studio Code	Landing Page, frontend web, documentación y edición general.	https://code.visualstudio.com/download	Versión estable

Landing Page y aplicación web

Tecnología	Propósito	Referencia	Versión
HTML5, CSS3 y JavaScript	Landing Page estática (anitec-landing-page).	https://developer.mozilla.org	Sin framework
Node.js	Ejecutar las herramientas de construcción del frontend web.	https://nodejs.org/en/download	^20.19.0 o >=22.12.0
Vue, Vite, Pinia, Vue Router	Aplicación web (anitec-frontend): componentes, construcción, estado y rutas.	https://vuejs.org · https://vite.dev	Vue 3.5, Vite 8, Pinia 3, Vue Router 5
PrimeVue, vue-i18n, Axios, Chart.js	Componentes de interfaz, internacionalización, consumo de la API y gráficos.	https://primevue.org	PrimeVue 4.5, vue-i18n 11, Axios 1.16, Chart.js 4.5

Web Services (anitec-backend)

Herramienta o tecnología	Propósito	Referencia
Rider / Visual Studio	Desarrollo de la API ASP.NET Core.	https://www.jetbrains.com/rider/download
.NET SDK	Compilar y ejecutar el backend (global.json).	https://dotnet.microsoft.com/download
ASP.NET Core	Exponer servicios REST y OpenAPI.	https://learn.microsoft.com/aspnet/core
Entity Framework Core	Persistencia y migraciones.	https://learn.microsoft.com/ef/core
MySql.EntityFrameworkCore	Proveedor MySQL para EF Core.	https://www.nuget.org/packages/MySql.EntityFrameworkCore
MySQL / MySQL Workbench	Base de datos relacional y su administración.	https://dev.mysql.com/downloads
Swashbuckle.AspNetCore	Documentación OpenAPI y Swagger UI.	https://github.com/domaindrivendev/Swashbuckle.AspNetCore
System.IdentityModel.Tokens.Jwt / JwtBearer	Emisión y validación de JWT.	https://www.nuget.org/packages/Microsoft.AspNetCore.Authentic
BCrypt.Net-Next	Hash de contraseñas.	https://www.nuget.org/packages/BCrypt.Net-Next
Stripe.net	Cliente de Stripe para el módulo de suscripciones.	https://docs.stripe.com/api
Cortex.Mediator	Despacho de comandos y consultas.	https://www.nuget.org/packages/Cortex.Mediator

Herramienta o tecnología	Propósito	Referencia
Docker	Empaquetar la API para su publicación (Dockerfile).	https://www.docker.com/products/docker-desktop
Postman / Swagger UI	Probar los contratos HTTP de la API.	https://www.postman.com/downloads

Aplicación Android (anitec-android)

Herramienta o tecnología	Propósito	Referencia	Versión
Android Studio	Desarrollo, emulación y depuración.	https://developer.android.com/studio	Build AI-261.26222.65.2613.1594802
Kotlin	Lenguaje de la aplicación nativa.	https://kotlinlang.org/docs/home.html	2.4.20
Android Gradle Plugin / Gradle	Construcción del proyecto.	https://developer.android.com/build	AGP 9.3.3 / Gradle 9.5.0 (wrapper)
JDK	Compilación y ejecución de Gradle.	https://adoptium.net	Código para Java 17; Gradle ejecuta con un toolchain JVM 25
KSP	Procesamiento de símbolos para Hilt y Room.	https://github.com/google/ksp	2.3.12
Jetpack Compose y Material 3	Interfaz de usuario declarativa.	https://developer.android.com/compose	Compose BOM 2026.09.00
SDK de Android	Plataforma de compilación y destino.	https://developer.android.com/tools/releases/platforms	minSdk 26, compileSdk y targetSdk 37
Android Emulator	Ejecutar y depurar la aplicación.	https://developer.android.com/studio/run/emulator	Sprint 1: dispositivo Pixel_7_sem2, API 33. Sprint 2: dispositivo

Herramienta o tecnología	Propósito	Referencia	Versión
			Medium_Phone_API_37.0, API 37 (Android 17)
R8	Reducir y ofuscar el código de la compilación release (app/proguard-rules.pro).	https://developer.android.com/build/shrink-code	Integrado en AGP 9.3.3
keytool y apksigner	Crear el keystore de firma y comprobar la firma del APK.	https://developer.android.com/tools/apksigner	JDK de Android Studio y build-tools 36.0.0
Dispositivo físico	Instalación del APK release distribuido por Firebase y demostración final.	—	**Pendiente:** modelo y versión de Android

Librerías de la aplicación Android

Librería	Propósito	Versión
Navigation Compose	Navegación con rutas tipadas.	2.10.2
Hilt (Dagger) y AndroidX Hilt	Inyección de dependencias y workers.	2.60.1 y 1.4.0
Room	Base de datos local (SQLite) con caché y cola de cambios pendientes.	2.8.5
DataStore y Tink	Almacenamiento cifrado de la sesión (AES-256-GCM con Android Keystore).	1.2.1 y 1.23.0
Retrofit, OkHttp y kotlinx.serialization	Consumo del API REST propio.	3.0.0, 5.5.0 y 1.11.0
Kotlin Coroutines	Concurrencia y flujos de estado.	1.11.0
Coil	Carga de imágenes de los animales.	3.6.3
CameraX	Acceso a la cámara (recurso interno del dispositivo).	1.6.2
Google ML Kit Barcode Scanning	Lectura de códigos QR y de barras (feature de aprendizaje autónomo).	17.3.0

Librería	Propósito	Versión
WorkManager	Envío en segundo plano de los cambios hechos sin conexión.	2.12.0
AppCompat	Cambio de idioma dentro de la aplicación (English y Español).	1.8.0
AndroidX Browser (Custom Tabs)	Abre el Checkout de Stripe en una pestaña personalizada del navegador.	1.10.0
Navigation Testing	Pruebas instrumentadas de la navegación inferior.	2.10.2

Aplicación Flutter (anitec-flutter)

Herramienta o paquete	Propósito	Referencia	Versión
Flutter SDK y Dart	Aplicación multiplataforma.	https://docs.flutter.dev/get-started/install	Flutter 3.47.7 (stable) y Dart 3.13.5
flutter_riverpod	Estado de las pantallas, separado de la interfaz.	https://pub.dev/packages/flutter_riverpod	^3.4.3
go_router	Navegación y redirección según la sesión.	https://pub.dev/packages/go_router	^18.1.1
dio	Cliente HTTP con interceptor Bearer y manejo de 401.	https://pub.dev/packages/dio	^5.11.1
flutter_secure_storage	Almacenamiento cifrado del token de sesión.	https://pub.dev/packages/flutter_secure_storage	^11.2.0
shared_preferences	Idioma elegido por el usuario.	https://pub.dev/packages/shared_preferences	^2.5.6
flutter_localizations e intl	Textos en inglés y español (es-419) con archivos ARB.	https://docs.flutter.dev/ui/accessibility-and-localization/internationalization	intl ^0.20.3
mocktail y flutter_lints	Dobles de prueba y reglas de análisis estático.	https://pub.dev/packages/mocktail	^1.0.5 y ^6.0.0

Software Testing

Producto	Herramientas	Tipo de comprobación
Backend	xUnit, NSubstitute, WebApplicationFactory con SQLite en memoria, Reqnroll (Gherkin), Swagger UI y cliente HTTP.	Unitarias, integración y aceptación BDD (proyecto Anitec.Platform.Tests); contratos HTTP y ejecución exploratoria.
Android	JUnit 4, MockK, Turbine, KotlinX Coroutines Test, MockWebServer, Room Testing, Navigation Testing y Android Emulator.	Unitarias de dominio, casos de uso, repositorios y ViewModels; instrumentadas de DAO de Room, carga de imágenes y navegación inferior.
Flutter	flutter_test y mocktail.	Unitarias y de widgets (dominio, estado y pantallas); análisis con flutter analyze.
Landing Page	DevTools, Lighthouse y validadores web.	Responsive, accesibilidad y desempeño.

Software Deployment

Herramienta	Propósito	Referencia
GitHub Pages	Publicar la Landing Page.	https://pages.github.com
Render	Publicar la API REST.	https://render.com
Firebase App Distribution	Distribuir las aplicaciones Android y Flutter a los evaluadores.	https://firebase.google.com/docs/app-distribution
Firebase CLI	Subir compilaciones a App Distribution desde la terminal (opcional).	https://firebase.google.com/docs/cli
Stripe (modo de prueba)	Servicio externo de pagos del módulo de suscripciones.	https://stripe.com

Software Documentation

Herramienta	Propósito	Referencia
Markdown y Visual Studio Code	Redactar el informe en el repositorio Informe y exportarlo a PDF con una extensión de VS Code.	https://code.visualstudio.com/download
Python	Ejecutar generar_reporte_completo.py, que reúne los archivos de markdown/content en el informe completo.	https://www.python.org/downloads
Structurizr	Mantener los diagramas C4 (Structurizr DSL).	https://structurizr.com
PlantUML	Mantener los diagramas de clases.	https://plantuml.com
Swagger / OpenAPI	Documentar y probar los endpoints.	https://swagger.io/specification/

4.1.2. Source Code Management

GitHub es la plataforma central de versionado. Los repositorios vigentes son:

Producto	Repositorio
Informe	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/Informe
Landing Page	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-landing-page
Web Services	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend
Aplicación web (frontend)	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-frontend
Android nativo	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android
Aplicación Flutter	https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-flutter

GitFlow

- main: versiones estables y entregables.
- develop: integración del trabajo aprobado para la siguiente versión.
- feature/-: desarrollo de una funcionalidad o artefacto.
- release/: estabilización de una versión candidata.
- hotfix/: corrección urgente originada desde main.

Cada Pull Request indicará propósito, cambios, evidencia de verificación y User Story relacionada. La integración requerirá revisión y comprobaciones correspondientes al producto.

Semantic Versioning. Las versiones seguirán MAJOR.MINOR.PATCH: MAJOR para cambios incompatibles, MINOR para funcionalidad compatible y PATCH para correcciones compatibles.

Conventional Commits. Se utilizará type(scope): description, con descripciones breves en inglés. Tipos principales: feat, fix, docs, style, refactor, test, build, ci y chore.

4.1.3. Source Code Style Guide & Conventions

El código fuente, los nombres técnicos, las rutas y los mensajes de commit se redactarán en inglés. Las clases y funciones tendrán responsabilidades claras y se mantendrá la separación por capas y bounded contexts definida en el capítulo II.

Tecnología	Convenciones principales
HTML y CSS	HTML semántico, atributos de accesibilidad, indentación de dos espacios y clases kebab-case.
JavaScript	Variables y funciones camelCase, constantes descriptivas, módulos pequeños y uso de async/await.
Kotlin (Android)	**Google Kotlin Style Guide** y convenciones de Kotlin de JetBrains. Estructura por bounded contexts (iam, livestock, sanitary, veterinary, activities, financial, devices, analytics, scanner) y capas domain, application, infrastructure (con remote y local) e interfaces (con ui y viewmodel). Tipos y clases en PascalCase; funciones y variables en camelCase. **Sufijos:** *UseCase para la lógica de aplicación, *Entity y *Dao para Room, *Api y *Dto para Retrofit, *RepositoryImpl para los adaptadores de

Tecnología	Convenciones principales
	infraestructura, *ViewModel y *UiState para la presentación y *Module para Hilt. Las capas domain y application no importan clases de Android.
Jetpack Compose	Flujo de datos unidireccional: el ViewModel expone un estado inmutable (StateFlow<*UiState>) y la interfaz emite eventos. Las funciones Composable usan PascalCase y se nombran como sustantivos. El tema se centraliza en Theme.kt, Color.kt y Type.kt con Material Design 3; la fuente configurada actualmente es la sans-serif del sistema, mientras que Poppins es la fuente definida por el Design System. Los componentes reutilizables viven en core/designsystem.
Recursos de Android	Todo texto de interfaz se define en strings*.xml por bounded context. El inglés es el idioma predeterminado (values) y el español latinoamericano está en values-b+es+419. Las cantidades usan plurals.
Room (SQLite)	Tablas en snake_case y plural; entidades *Entity; esquemas exportados en app/schemas para cada versión de la base de datos.
Dart	**Effective Dart**. Archivos en lowercase_with_underscores. Clases, enums y typedefs en UpperCamelCase. Miembros de clases y variables en lowerCamelCase.
Flutter	Separación entre interfaz y lógica de estado (notifiers de Riverpod). Carpetas por bounded context (iam, livestock, sanitary, activities) y un core con el diseño, el cliente HTTP, la sesión y el idioma. Widgets pequeños y componibles. Textos centralizados en archivos ARB (lib/l10n) y localizables; los tokens de color y tipografía siguen los de la aplicación Android.
C#	Convenciones Microsoft (C# Coding Conventions); tipos, métodos y propiedades PascalCase, parámetros camelCase y campos privados _camelCase. Controllers: *Controller en Interfaces/Rest. Resources (DTOs): *Resource y assemblers *FromResourceAssembler / FromEntityAssembler. Commands y Queries: CreateCommand, Get*Query en Domain/Model. Rutas en minúsculas bajo api/v1/.... Estructura por bounded context. Errores HTTP vía Problem Details.
REST / OpenAPI	Sustantivos plurales en rutas, verbos HTTP correctos, resources/DTOs, códigos de estado y respuestas de error consistentes. Documentación con Swashbuckle (AddSwaggerGen, anotaciones [SwaggerOperation]) y esquema Bearer JWT.
Pruebas	Clases *Test junto a la clase probada, con nombres de prueba que describen el comportamiento esperado (por ejemplo, «a refused change is marked failed and the next one is still sent»). Las pruebas se organizan por bounded context.
Gherkin	Features y escenarios ligados a User Stories, pasos declarativos y estructura Given–When–Then. Los archivos .feature están en el proyecto de pruebas del backend (Anitec.Platform.Tests/Bdd/Features).

Referencias adoptadas. Las convenciones anteriores se basan en las siguientes guías estándar:

Tecnología	Referencia
HTML y CSS	HTML Style Guide and Coding Conventions: https://www.w3schools.com/html/html5_syntax.asp. Google HTML/CSS Style Guide: https://google.github.io/styleguide/htmlcssguide.html.

Tecnología	Referencia
JavaScript y Vue	Google JavaScript Style Guide: https://google.github.io/styleguide/jsguide.html. Guía de estilo de Vue: https://vuejs.org/style-guide/.
Kotlin	Android Kotlin Style Guide: https://developer.android.com/kotlin/style-guide. Kotlin Coding Conventions: https://kotlinlang.org/docs/coding-conventions.html.
Jetpack Compose	API Guidelines for Jetpack Compose: https://github.com/androidx/androidx/blob/androidx-main/compose/docs/compose-api-guidelines.md.
Dart y Flutter	Effective Dart: https://dart.dev/effective-dart.
C#	C# Coding Conventions: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions.
REST / OpenAPI	OpenAPI Specification: https://swagger.io/specification/.
Gherkin	Gherkin Conventions for Readable Specifications: https://specflow.org/gherkin/gherkin-conventions-for-readable-specifications/.

Reglas compartidas

- **Caché local y cambios pendientes:** las pantallas de la aplicación Android leen siempre de la base de datos local (Room), que se actualiza desde la API; por eso la información ya consultada está disponible sin conexión. Las escrituras se envían primero al servidor. Solo la **creación** de animales, registros sanitarios, actividades y movimientos financieros, si no hay conexión, se guarda en el dispositivo con un identificador temporal negativo, se muestra como «Pendiente de sincronizar» y se envía en segundo plano cuando vuelve la red. Editar y eliminar requieren conexión. La aplicación Flutter no tiene caché local: consulta siempre al servidor.
- **Inyección de dependencias:** uso centralizado de Hilt para proveer repositorios, casos de uso y servicios, organizado mediante módulos (*Module).
- **Secretos:** no incluir secretos, tokens ni cadenas de conexión en el repositorio. El token de sesión se guarda cifrado con Tink (AES-256-GCM) sobre Android Keystore; las claves de firma (*.jks, *.keystore), los archivos con sus contraseñas (keystore.properties en Android y key.properties en Flutter) y google-services.json están excluidos por .gitignore; los secretos del backend se configuran como variables de entorno en Render.
- **Internacionalización:** centralizar los textos en recursos en lugar de escribirlos en las vistas.
- **Reglas de negocio:** evitar duplicarlas entre la interfaz y la API; el backend conserva las reglas autoritativas y la validación final. Los clientes solo aplican validaciones de formato para dar respuesta inmediata.
- **Análisis estático:** en Android no se emplean ktlint ni detekt; la calidad se comprueba con las inspecciones de Android Studio, las advertencias del compilador y la revisión de los cambios. En Flutter se ejecutan dart format --set-exit-if-changed lib test y flutter analyze con las reglas de flutter_lints antes de cada entrega.

4.1.4. Software Deployment Configuration

Cada producto se configura y publica de manera independiente, pero las aplicaciones móviles consumen la misma API mediante HTTPS y contratos documentados con OpenAPI. Esta sección describe los pasos para que, a partir de

los repositorios de código fuente, se publique cada producto: la Landing Page en GitHub Pages, los Web Services en Render y las aplicaciones Android y Flutter en Firebase App Distribution.

Producto	Repositorio	Plataforma	URL o identificador	Estado
Landing Page	anitec-landing-page	GitHub Pages	https://adm-1acc0238-2620-13975-grupo01.github.io/anitec-landing-page/	Publicada
Web Services	anitec-backend	Render (Docker)	API: https://anitec-backend-android.onrender.com. Swagger: https://anitec-backend-android.onrender.com/swagger/index.html	Publicado
Android	anitec-android	Firebase App Distribution	Proyecto Firebase «AniTec» (anitec-17af9), App ID 1:630071935433:android:d5f011b7f718ab5c0fe818 (paquete com.anitec.platform). Vínculos de invitación del grupo «Equipo AniTec»: https://appdistribution.firebase.dev/i/417dc4e6d8ae3e32 y https://appdistribution.firebase.dev/i/862507cca1d4abc6	Publicada: versión 0.2.0 (2)
Flutter	anitec-flutter	Firebase App Distribution	Mismo proyecto Firebase, paquete com.anitec.platform.flutter, App ID 1:630071935433:android:45b987300751a4a90fe818. Los mismos vínculos de invitación del grupo «Equipo AniTec»	Publicada: versión 1.0.0 (1)

Diagrama de despliegue

El diagrama de despliegue muestra los dispositivos, productos, servicios externos y relaciones necesarias para ejecutar AniTec.

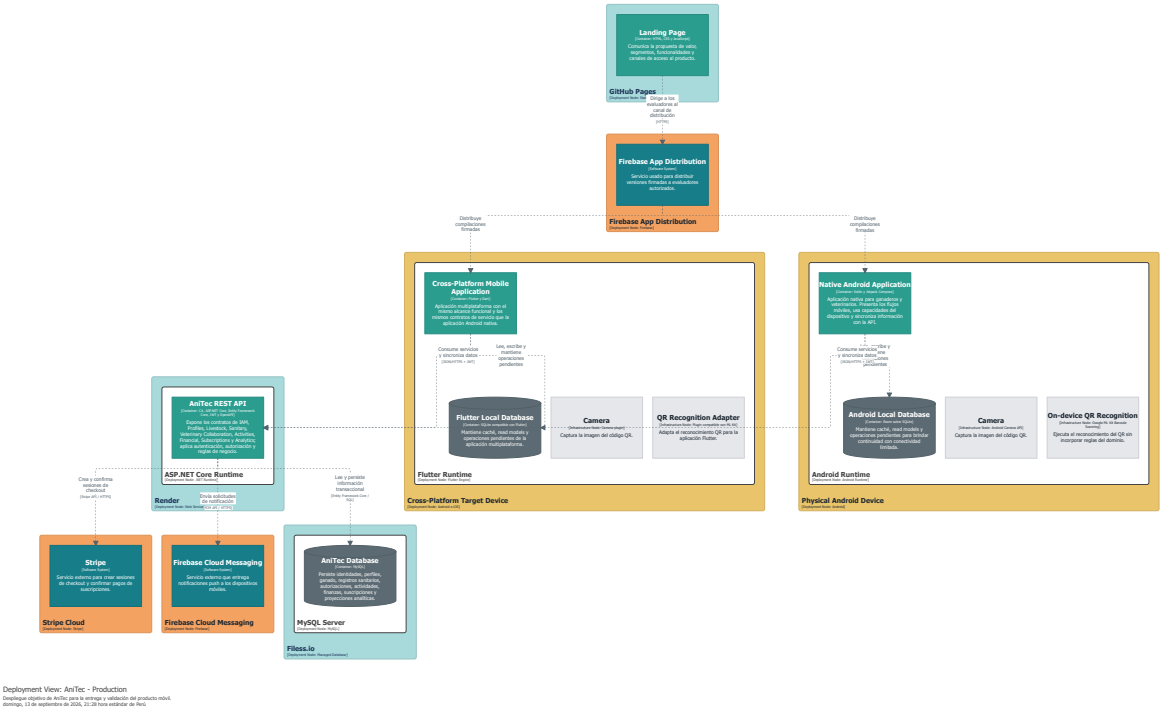


Figura 4.1.4.1. Software Architecture Deployment Diagram. Fuente: elaboración propia con Structurizr DSL.

Landing Page: GitHub Pages

La Landing Page es un sitio estático (index.html y carpeta assets), por lo que no requiere compilación.

1. Integrar el contenido aprobado en la rama main del repositorio anitec-landing-page.
2. En GitHub, abrir **Settings** → **Pages** del repositorio.
3. En **Build and deployment**, elegir **Deploy from a branch**, seleccionar la rama main y la carpeta / (root), y guardar.
4. Esperar a que la acción de publicación termine. La URL tiene la forma https://<organización>.github.io/anitec-landing-page/; la del proyecto es https://adm-1acc0238-2620-13975-grupo01.github.io/anitec-landing-page/.
5. Verificar desde la URL pública la navegación entre secciones, la carga de recursos, el diseño responsive en escritorio y móvil, el cambio de idioma, la accesibilidad básica y el enlace a los Términos de Servicio en el pie de página.
6. Registrar la URL en esta sección y en 4.2.1.8. Ya está registrada en ambas.

Web Services: Render

El backend se publica desde la rama main del repositorio anitec-backend (https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend/tree/main) con Docker, como el servicio anitec-backend-android. Requiere una base de datos MySQL accesible por Internet, porque Render no ofrece MySQL administrado.

1. **Base de datos.** Crear una base MySQL 8 en un proveedor externo y anotar servidor, puerto, usuario, contraseña y nombre de la base. No es necesario crear las tablas: la API aplica las migraciones de Entity Framework Core al iniciar (Database.Migrate()), incluidas las de corrales y del correo del usuario.
2. **Servicio.** En Render, crear un **New** → **Web Service**, conectar el repositorio anitec-backend, elegir la rama main, el entorno **Docker** (usa el Dockerfile de la raíz) y el plan Free.
3. **Imagen.** El Dockerfile compila en una primera etapa con mcr.microsoft.com/dotnet/sdk:10.0 (restaura y publica Anitec.Platform/Anitec.Platform.csproj en Release) y ejecuta en una segunda con mcr.microsoft.com/dotnet/aspnet:10.0, expone el puerto 8080 y define ASPNETCORE_URLS=http://0.0.0.0:8080.
4. **Variables de entorno.** Configurarlas en el panel **Environment** de Render; los valores no se publican en el informe ni en el repositorio:

Variable	Contenido
ASPNETCORE_ENVIRONMENT	Production para la operación normal. Con Development la API carga además los datos de demostración (paso 6).
ConnectionStrings__DefaultConnection	Cadena de conexión MySQL del paso 1. El plan gratuito del proveedor limita las conexiones simultáneas y la aplicación pide varias listas a la vez al abrir el inicio; por eso la cadena termina con ;Maximum Pool Size=2, que hace que las peticiones esperen turno en lugar de fallar con 500.
TokenSettings__Secret	Cadena aleatoria larga (al menos 32 caracteres) para firmar los JWT.
StripeSettings__SecretKey	Clave secreta de Stripe en modo de prueba.
StripeSettings__WebhookSecret	Secreto del webhook de Stripe.

Variable	Contenido
StripeSettings__SuccessUrl y StripeSettings__CancelUrl	URLs de retorno del pago para la aplicación web. Por defecto apuntan a http://localhost:5173; deben cambiarse a las URLs finales.
StripeSettings__MobileReturnBaseUrl	Opcional. Dirección HTTPS pública del backend usada como retorno del pago iniciado desde la aplicación móvil; si no se define, el backend la deduce de la solicitud.
SeedDemoData	Opcional. Con el valor true, la API carga los datos de demostración al iniciar aunque el entorno no sea Development; solo actúa sobre una base sin usuarios (paso 6). Se retira o se pone en false una vez cargados.

5. **Publicación.** Pulsar **Create Web Service**. Render construye la imagen y publica el servicio. Con la rama main conectada, cada push posterior inicia un despliegue automático.
6. **Datos iniciales.** Los planes de suscripción se cargan en cualquier entorno al iniciar, y solo si la tabla está vacía. Los usuarios, fincas, corrales y animales de demostración se cargan cuando ASPNETCORE_ENVIRONMENT es Development o cuando la variable SeedDemoData vale true, y únicamente si la base no tiene ningún usuario; si ya hay usuarios, el cargador no modifica nada. Para cargar la demostración en una base ya usada, se vacían primero sus tablas desde el cliente web del proveedor (con DROP TABLE de cada tabla; no se elimina la base, para conservar sus credenciales), se define SeedDemoData=true en Render y, al redesplegar, la API crea las tablas con las migraciones y carga los datos. Una vez cargados, se retira la variable. Las cuentas de demostración (ganadero, maria, jose y rosa como ganaderos; veterinaria, vetpedro y vetlucia como veterinarios) comparten la contraseña de demostración documentada en el README del backend; con datos reales deben cambiarse o eliminarse.
7. **Verificación.** Abrir /swagger/index.html en la URL pública, registrar un usuario, iniciar sesión, autorizar con el token en Swagger y consultar GET /api/v1/animals.
8. **Consideraciones del plan gratuito.** El servicio se suspende tras un periodo de inactividad y la primera solicitud posterior puede tardar casi un minuto. La base de datos externa limita las conexiones simultáneas: sin el ajuste Maximum Pool Size=2 de la cadena de conexión, la mayoría de las ocho listas que pide la aplicación al abrir el inicio fallaban con 500 cuando se pedían a la vez, y con el ajuste las peticiones se atienden por turno. El disco es efímero: las fotos de animales guardadas en wwwroot/uploads/animals se pierden al reiniciar el servicio, por lo que, para producción, se requiere un almacenamiento externo.

El servicio quedó publicado en https://anitec-backend-android.onrender.com y su documentación Swagger en https://anitec-backend-android.onrender.com/swagger/index.html. Al verificarlo se comprobó que Swagger responde y que los endpoints protegidos devuelven 401 sin token. La evidencia siguiente muestra el servicio en Render y la configuración de sus variables, con los valores ocultos.

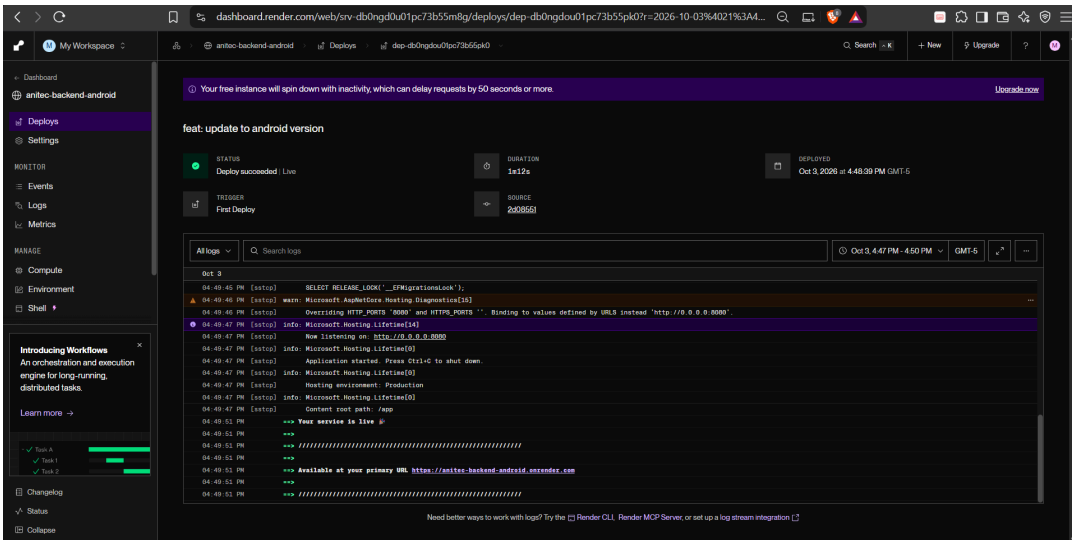


Figura 4.1.4.2. Servicio anitec-backend-android en Render con estado Live y URL pública. Fuente: elaboración propia (captura de Render).

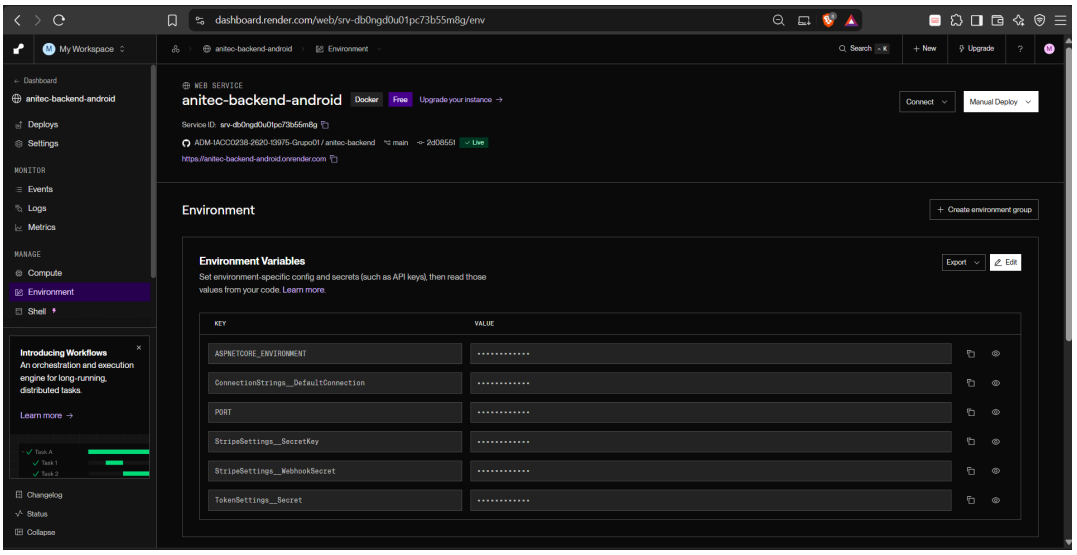


Figura 4.1.4.3. Variables de entorno del backend en Render (valores ocultos): ASPNETCORE_ENVIRONMENT, ConnectionStrings__DefaultConnection, TokenSettings__Secret y StripeSettings. Fuente: elaboración propia (captura de Render).

Android: Firebase App Distribution

La distribución usa un proyecto de Firebase llamado «AniTec» (plan Spark, sin costo) con dos aplicaciones de Android registradas: «AniTec» (com.anitec.platform) y «AniTec Flutter» (com.anitec.platform.flutter). Las dos usan el mismo grupo de evaluadores, «Equipo AniTec». Para publicar una versión de la aplicación nativa:

1. **URL de la API.** La compilación release usa la dirección configurada en app/build.gradle.kts, que por defecto apunta a https://anitec-backend-android.onrender.com (SERVER_URL y API_BASE_URL). La propiedad anitec.releaseServerUrl de local.properties, que no se versiona, permite cambiarla para una prueba. La compilación debug usa http://10.0.2.2:5191, el backend local visto desde el emulador.
2. **Versión.** Incrementar versionCode y asignar versionName según Semantic Versioning antes de cada subida (la versión distribuida en el Sprint 2 es 0.2.0, con versionCode 2).

3. **Firma.** Crear el keystore una sola vez con keytool -genkeypair -v -keystore /anitec-release.jks -alias anitec -keyalg RSA -keysize 2048 -validity 10000, fuera del repositorio, y guardar una copia de respaldo con sus contraseñas, porque sin ellas no se puede actualizar una aplicación ya instalada. Copiar keystore.properties.example a keystore.properties (ignorado por git) y completar storeFile, storePassword, keyAlias y keyPassword. Si el archivo no existe, la compilación avisa y firma con la llave de depuración, que no debe distribuirse. Las contraseñas no se escriben en el repositorio ni en el informe.
4. **Compilación.** Desde la carpeta del proyecto y con el JDK de Android Studio, ejecutar ./gradlew --stop (cierra daemons iniciados con otro JDK, como el de la extensión de Java de VS Code, que no incluye jlink y hace fallar la compilación) y luego ./gradlew assembleRelease. R8 reduce el código en esta compilación. El archivo resultante es app/build/outputs/apk/release/app-release.apk.
5. **Prueba previa.** Instalar el APK en un dispositivo físico, iniciar sesión y comprobar que se cargan los datos del backend publicado, antes de distribuirlo.
6. **Proyecto y aplicaciones en Firebase.** En la consola de Firebase, crear el proyecto (sin Gemini ni Google Analytics, que no se necesitan) y registrar la aplicación con **Agregar app** → **Android**, el nombre de paquete y un alias, sin huella SHA-1. No se descarga google-services.json, porque la aplicación no usa el SDK de Firebase y App Distribution solo necesita que la aplicación esté registrada.
7. **Subida.** En **App Distribution**, elegir la aplicación en el selector superior y pulsar **Comenzar**. En **Versiones**, arrastrar el archivo .apk al recuadro de carga. La consola también permite subirlo desde la terminal con firebase appdistribution:distribute.
8. **Evaluadores.** En **Verificadores y grupos** se crea el grupo «Equipo AniTec». Al distribuir la versión se eligen los destinatarios (correos de Google o el grupo) y se escriben las notas; el botón **Distribuir** solo se habilita si hay al menos un destinatario con miembros. Cada evaluador recibe un correo con el enlace de instalación.
9. **Vínculo de invitación.** Para que otras personas se unan sin que se escriba su correo, en **Vínculos de invitación** → **Crear vínculo de invitación** se elige el grupo y se deja vacía la restricción de dominio. Quien abre el vínculo con una cuenta de Google se une al grupo; el grupo debe estar entre los destinatarios de la versión para que reciba la compilación. Los vínculos pertenecen al grupo y no a una aplicación: la consola muestra los mismos vínculos desde cualquiera de las dos aplicaciones (Figura 4.1.4.7), y el grupo tiene acceso a las versiones que se le distribuyen en ambas. Un vínculo puede desactivarse en cualquier momento.
10. **Instalación.** El evaluador abre el correo o el vínculo con su cuenta de Google, acepta la invitación, descarga la versión (Firebase puede sugerir su aplicación App Tester) y permite una vez la instalación desde ese origen. Si ya hay instalado un APK firmado con otra llave, por ejemplo uno de depuración, debe desinstalarse antes.
11. **Registro.** Anotar versión, commit y fecha de cada release en la sección de despliegue del sprint (4.2.2.8).

Flutter: Firebase App Distribution

La aplicación Flutter sigue el mismo proceso, con estas diferencias. El proyecto (anitec-flutter) usa el paquete com.anitec.platform.flutter en android/app/build.gradle.kts y su versión sale de pubspec.yaml (version: 1.0.0+1; el número después del + es el versionCode y debe aumentarse en cada subida). La firma usa android/key.properties, que tiene el mismo formato que keystore.properties y por tanto admite el mismo keystore; el archivo está ignorado por git y, si no existe, la compilación firma con la llave de depuración. La compilación es flutter build apk --release, que genera build/app/outputs/flutter-apk/app-release.apk y apunta al backend publicado (la URL puede cambiarse con --dart-define=API_BASE_URL=...). La firma del APK se comprobó con apksigner verify --print-certs. En Firebase se registra una segunda aplicación **Android**, no la opción «Flutter» del asistente, que instala herramientas y modifica el

proyecto. En App Distribution se cambia el selector a «AniTec Flutter» y se repiten los pasos de subida y destinatarios; la versión 1.0.0 (1) se distribuyó con el grupo «Equipo AniTec», que ya tenía sus vínculos de invitación.

Las capturas siguientes muestran el proyecto de Firebase, el registro de las aplicaciones y la distribución de las dos versiones.

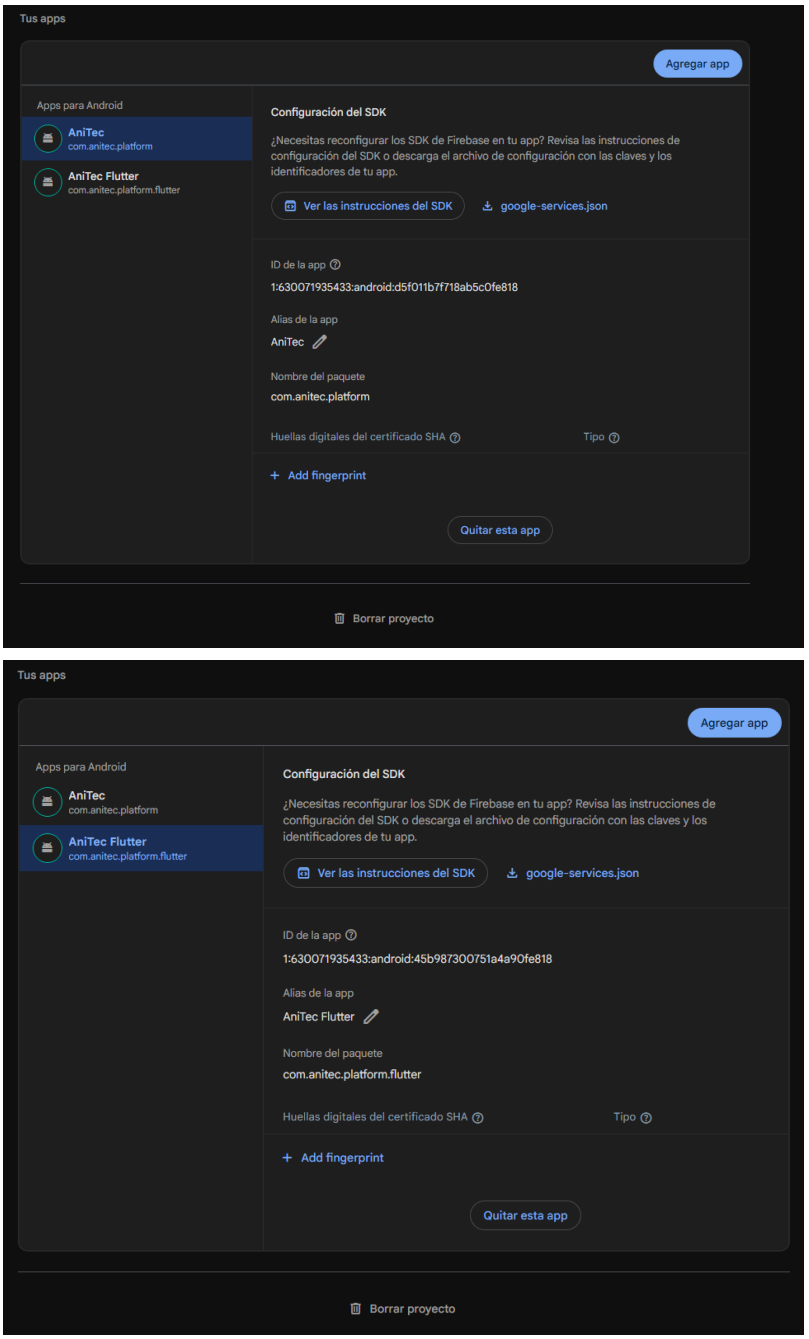


Figura 4.1.4.4. Aplicaciones «AniTec» y «AniTec Flutter» registradas en el proyecto Firebase AniTec (nombre de paquete y App ID). Fuente: elaboración propia (captura de Firebase).

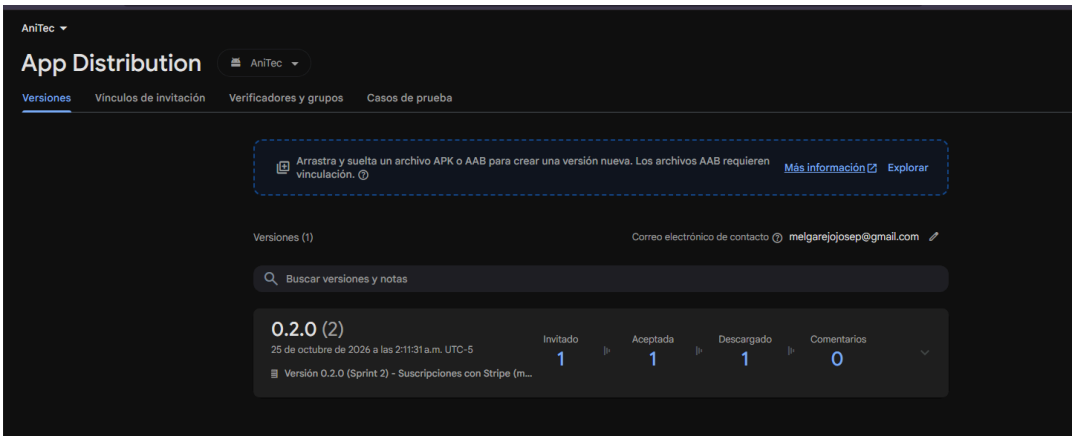


Figura 4.1.4.5. Versión 0.2.0 de la aplicación Android nativa en Firebase App Distribution. Fuente: elaboración propia (captura de Firebase).

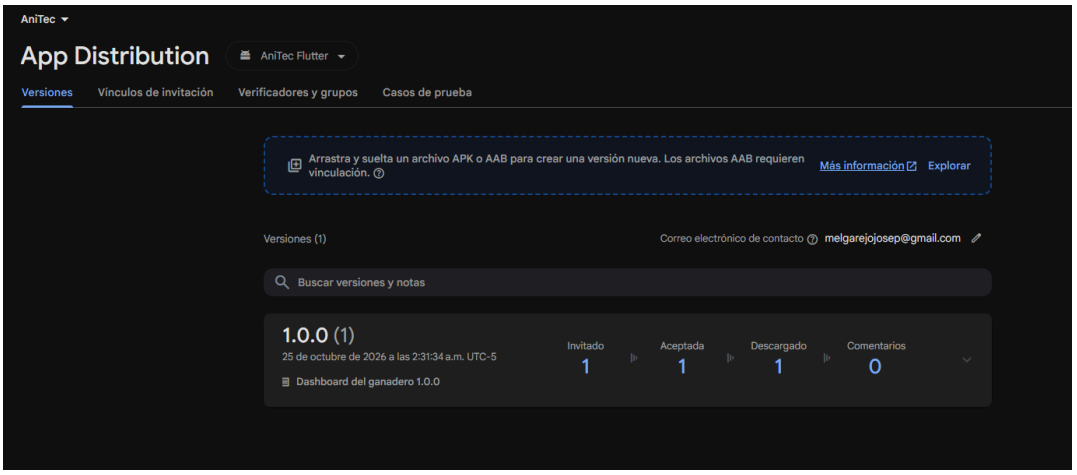


Figura 4.1.4.6. Versión 1.0.0 de la aplicación Flutter en Firebase App Distribution. Fuente: elaboración propia (captura de Firebase).

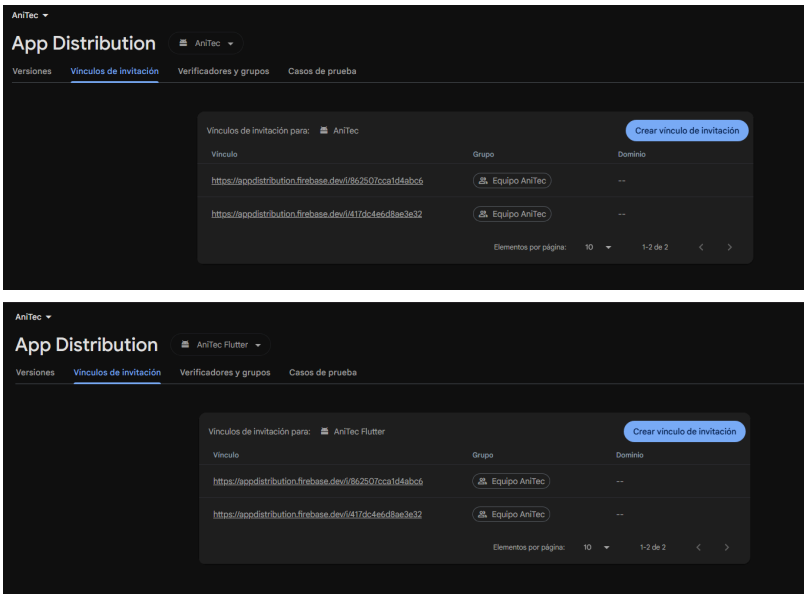


Figura 4.1.4.7. Vínculos de invitación del grupo «Equipo AniTec», que se listan igual desde la aplicación AniTec (arriba) y desde AniTec Flutter (abajo). Fuente: elaboración propia (captura de Firebase).

La configuración de secretos se mantiene fuera de los repositorios de GitHub y se administra en el panel Environment de Render y en el equipo de cada desarrollador.

4.2. Landing Page & Mobile Application Implementation

Esta sección registra el avance de la Landing Page, los Web Services y las aplicaciones móviles mediante sprints. Cada iteración relaciona el Product Backlog con tareas, responsables, commits, pruebas, documentación, ejecución y despliegue verificable.

4.2.1. Sprint 1

El Sprint 1 corresponde al TB1 y establece la base ejecutable de los productos de AniTec. Su alcance es el publicado en la rama main del repositorio de la aplicación Android (https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android): la Landing Page, la adaptación del backend para el uso móvil y la aplicación Android nativa con autenticación, gestión de fincas, corrales y animales, y registros sanitarios básicos. La aplicación Flutter queda como placeholder para un sprint posterior.

4.2.1.1. Sprint Planning 1

La reunión de planificación definió el Sprint Goal, la capacidad del equipo y el conjunto de historias que podía completarse con evidencia verificable. El Product Backlog del AV1 asignaba al Sprint 1 veintiuna historias (96 Story Points); al contrastarlas con lo realmente publicado en main, el compromiso quedó ajustado a las historias de la tabla siguiente y el Product Backlog de la sección 2.4.3 se actualizó con ese ajuste: 24 historias y 105 Story Points.

Sprint #	Sprint 1
	Sprint Planning Background
Date	2026-09-25
Time	10:53 PM
Location	Discord (reunión virtual)
Prepared By	Melgarejo Quiroz, Josep Eliu
Attendees	Beingolea Montalvo, Sebastian Martin / Melgarejo Quiroz, Josep Eliu / Ortega Muñoz, Saul / Sanchez Silva, Luciana Celeste / Villanueva Rodriguez, Giuseppe
Sprint 0 Review Summary	No existe un sprint de implementación previo. La línea base del Sprint 1 es el AV1: el informe hasta el capítulo II (requisitos, Product Backlog y diseño estratégico y táctico), la Landing Page inicial y el backend ASP.NET Core con sus bounded contexts.
Sprint 0 Retrospective Summary	Como aciertos del AV1, el equipo valoró haber cerrado los capítulos I y II con los requisitos y el diseño estratégico y táctico, y contar con una Landing Page inicial y un backend organizado por bounded contexts. Como oportunidades de mejora, identificó definir el alcance móvil (Android nativo y Flutter para una etapa posterior) antes de empezar a implementar, mantener alineados el informe y el código, y incorporar pruebas automatizadas al backend desde el inicio.
	Sprint Goal & User Stories

Sprint 1 Goal	**Nuestro foco** es ofrecer una primera aplicación Android ejecutable para ganaderos y veterinarios. **Creemos que** entrega una forma rápida de registrar y consultar sus fincas, animales y registros sanitarios desde el teléfono a los ganaderos y veterinarios de pequeñas y medianas explotaciones. **Lo confirmaremos cuando** un usuario pueda registrarse, iniciar sesión y registrar y consultar un animal desde la aplicación instalada, usando la API del backend, con la Landing Page publicada.
Sprint 1 Velocity	105 Story Points, igual a la suma de las historias comprometidas. Es la capacidad acordada por el equipo para el sprint; al no existir un sprint previo, no hay una velocidad histórica de referencia.
Sum of Story Points	105

La tabla siguiente lista las 24 historias comprometidas, sus Story Points y su estado al cierre del sprint. Las historias US-014, US-015 y US-016 se adelantaron desde el Sprint 2; TS-002 (configurar Flutter) y SP-001 (investigar ML Kit) pasan al Sprint 2, y US-044 y US-045 son historias nuevas incorporadas al Product Backlog (sección 2.4).

ID	Título	Story Points	Estado al cierre
US-011	Registrar un animal	5	Completa: registro individual y masivo.
US-010	Consultar y buscar animales	5	Completa.
US-013	Consultar el detalle de un animal	3	Completa: ficha técnica.
US-009	Registrar y actualizar una finca	5	Completa.
US-008	Consultar las fincas registradas	3	Completa.
US-044	Gestionar los corrales de una finca	3	Completa.
US-012	Actualizar o archivar un animal	5	Parcial: se actualiza y se elimina; el archivado no está implementado.
US-015	Registrar una incidencia sanitaria	5	Completa.
US-016	Registrar diagnóstico y tratamiento	5	Completa.
US-014	Consultar eventos sanitarios	3	Completa.

ID	Título	Story Points	Estado al cierre
US-045	Adjuntar una fotografía al animal	3	Completa.
US-001	Comprender la propuesta de valor de AniTec	3	Completa.
US-002	Conocer las soluciones para cada segmento	3	Completa.
TS-013	Adaptar y documentar los servicios backend para móviles	8	Completa: corrales, operaciones masivas, OpenAPI y pruebas automatizadas del backend (sección 4.2.1.5).
TS-004	Integrar las aplicaciones con la API REST interna	5	Completa para Android.
TS-003	Definir la arquitectura móvil por capas y bounded contexts	5	Completa.
TS-001	Configurar la aplicación Android nativa	5	Completa.
TS-005	Implementar persistencia local segura en Android	5	Completa: Room y sesión cifrada.
US-004	Registrar una cuenta según el rol	5	Completa.
US-005	Iniciar sesión	3	Completa.
US-007	Acceder únicamente a información autorizada	5	Parcial: el filtrado se hace en la aplicación; el backend aún no filtra por usuario.
US-006	Mantener y finalizar la sesión móvil	3	Completa.
TS-008	Proteger credenciales y datos de sesión	5	Completa.
US-003	Acceder a una landing page adaptable e internacionalizada	5	Completa.

4.2.1.2. Aspect Leaders and Collaborators

La matriz LACX indica un líder (L) y los colaboradores (C) de cada aspecto del sprint. Los aspectos corresponden a los productos y actividades del Sprint 1: diseño UX/UI, aplicación Android, aplicación Flutter, backend, Landing Page, pruebas, y documentación y despliegue. La matriz se deriva de las tareas asignadas en el Sprint Backlog (sección 4.2.1.3) y de las contribuciones al Informe; la aplicación Flutter no tiene responsables porque su desarrollo no forma parte de este sprint.

Team Member	GitHub Username	UX/UI	Android	Flutter	Backend	Landing Page	Testing	Documentation & Deployment
Beingolea Montalvo, Sebastian Martin	smbmontalvo	—	C	—	L	—	C	C
Melgarejo Quiroz, Josep Eliu	Melga1502	—	L	—	—	—	—	C
Ortega Muñoz, Saul	Ss1lent10	C	C	—	—	L	—	C
Sanchez Silva, Luciana Celeste	luccsss	L	C	—	—	—	—	C
Villanueva Rodriguez, Giuseppe	Giuseppe152004	—	C	—	—	—	L	L

4.2.1.3. Sprint Backlog 1

El Sprint Backlog descompone las historias comprometidas en tareas comprobables y refleja el objetivo del sprint: una primera aplicación Android ejecutable, respaldada por la API del backend y por la Landing Page publicada. El tablero utiliza los estados Todo, In-Process, To-Review y Done.

- **Sprint Goal:** ofrecer una primera aplicación Android ejecutable para ganaderos y veterinarios (ver 4.2.1.1).
- **Board URL:** URL pública del tablero.

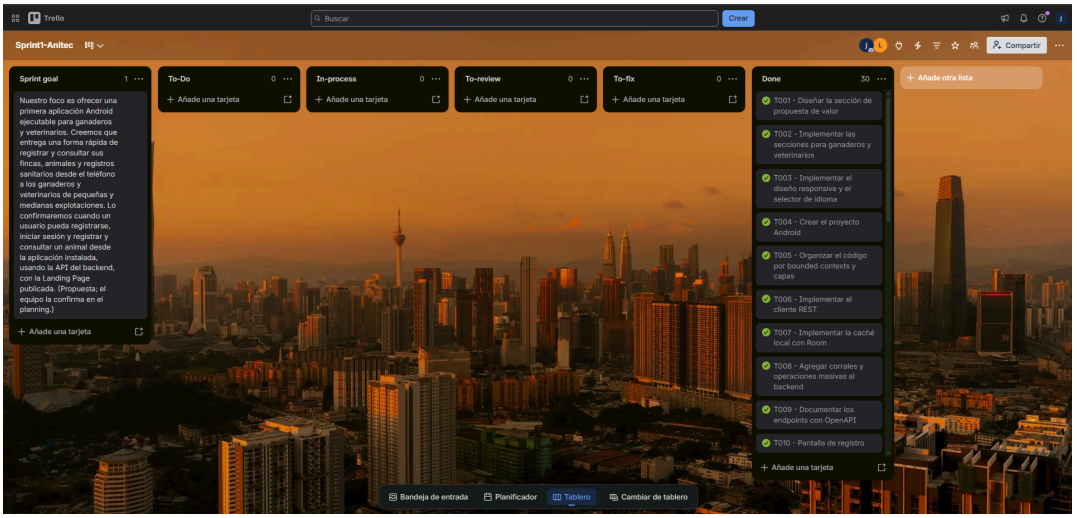


Figura 4.2.1.3.1 Tablero de Trello Sprint 1 Fuente: elaboración propia.

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
US-001	Comprender la propuesta de valor de AniTec	T-001	Diseñar la sección de propuesta de valor	Encabezado, mensaje principal e indicadores de la Landing Page.	4	Ortega Muñoz, Saul	Done
US-002	Conocer las soluciones para cada segmento	T-002	Implementar las secciones para ganaderos y veterinarios	Páginas por segmento con sus funcionalidades.	6	Ortega Muñoz, Saul	Done
US-003	Acceder a una landing page adaptable e internacionalizada	T-003	Implementar el diseño responsive y el selector de idioma	Adaptación a móvil y textos en inglés y español.	8	Ortega Muñoz, Saul	Done
TS-001	Configurar la aplicación Android nativa	T-004	Crear el proyecto Android	Proyecto Kotlin con Jetpack Compose, Hilt y la configuración de compilación.	6	Villanueva Rodriguez, Giuseppe	Done
TS-003	Definir la arquitectura móvil por capas y bounded contexts	T-005	Organizar el código por bounded contexts y capas	Paquetes domain, application, infrastructure e interfaces por contexto.	6	Villanueva Rodriguez, Giuseppe	Done
TS-004	Integrar las aplicaciones con la API REST interna	T-006	Implementar el cliente REST	Retrofit y OkHttp con interceptor de token y mapeo de errores HTTP.	8	Beingolea Montalvo, Sebastian Martin	Done
TS-005	Implementar persistencia local segura en Android	T-007	Implementar la caché local con Room	Entidades y DAO por contexto; limpieza de la caché al cerrar sesión.	8	Melgarejo Quiroz, Josep Eliu	Done
TS-013	Adaptar y documentar los servicios backend para móviles	T-008	Agregar corrales y operaciones masivas al backend	Entidad Corral, relación animal-corral y endpoints de alta, estado y baja masiva.	10	Beingolea Montalvo, Sebastian Martin	Done
TS-013	Adaptar y documentar los servicios backend para móviles	T-009	Documentar los endpoints con OpenAPI	Swagger UI con esquema Bearer JWT para los endpoints del Sprint.	4	Beingolea Montalvo, Sebastian Martin	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
US-004	Registrar una cuenta según el rol	T-010	Pantalla de registro	Formulario con selección de rol y aceptación de los Términos de Servicio.	6	Ortega Muñoz, Saul	Done
US-005	Iniciar sesión	T-011	Pantalla de inicio de sesión	Formulario, validaciones y mensajes de error recuperables.	4	Ortega Muñoz, Saul	Done
US-006	Mantener y finalizar la sesión móvil	T-012	Mantener y finalizar la sesión	Sesión persistente, cierre de sesión y retorno al inicio al vencer el token.	6	Ortega Muñoz, Saul	Done
US-007	Acceder únicamente a información autorizada	T-013	Navegación por rol y filtrado de datos	Barra de navegación por rol y filtrado de fincas, animales y registros del usuario en la aplicación.	8	Melgarejo Quiroz, Josep Eliu	To-Review
TS-008	Proteger credenciales y datos de sesión	T-014	Cifrar el token de sesión	Almacenamiento con DataStore, Tink (AES-256-GCM) y Android Keystore.	6	Beingolea Montalvo, Sebastian Martin	Done
US-008	Consultar las fincas registradas	T-015	Lista de fincas	Tarjetas con ubicación, propietario y conteo de corrales y animales.	4	Sanchez Silva, Luciana Celeste	Done
US-009	Registrar y actualizar una finca	T-016	Formulario de fincas	Alta y edición con validaciones y eliminación con confirmación.	5	Sanchez Silva, Luciana Celeste	Done
US-010	Consultar y buscar animales	T-017	Lista de animales con búsqueda y filtro	Búsqueda por varios campos y filtro por corral.	8	Melgarejo Quiroz, Josep Eliu	Done
US-011	Registrar un animal	T-018	Formulario de registro de animales	Registro individual y masivo de 1 a 500 animales.	10	Melgarejo Quiroz, Josep Eliu	Done
US-012	Actualizar o archivar un animal	T-019	Edición y eliminación de animales	Edición, eliminación individual y acciones sobre varios animales con confirmación.	6	Villanueva Rodriguez, Giuseppe	To-Review
US-013	Consultar el detalle de un animal	T-020	Ficha técnica del animal	Hoja de detalle con todos los datos y la	6	Ortega Muñoz,	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
				fotografía.		Saul	
US-014	Consultar eventos sanitarios	T-021	Lista de registros sanitarios	Tarjetas con tipo, fecha y seguimiento.	5	Sanchez Silva, Luciana Celeste	Done
US-015	Registrar una incidencia sanitaria	T-022	Formulario de registro sanitario	Tipo, fecha, descripción y fecha de seguimiento.	6	Sanchez Silva, Luciana Celeste	Done
US-016	Registrar diagnóstico y tratamiento	T-023	Diagnóstico y tratamiento	Campos de diagnóstico, tratamiento y prescripción del registro sanitario.	3	Sanchez Silva, Luciana Celeste	Done
US-044	Gestionar los corrales de una finca	T-024	Lista y formulario de corrales	Alta, edición y eliminación de corrales por finca.	6	Sanchez Silva, Luciana Celeste	Done
US-045	Adjuntar una fotografía al animal	T-025	Fotografía del animal	Captura con cámara o elección de galería, reducción de tamaño y subida al servidor.	8	Melgarejo Quiroz, Josep Eliu	Done
Sin historia	Tarea técnica sin historia asociada	T-026	Pruebas unitarias e instrumentadas de Android	JUnit, MockK y pruebas de DAO de Room sobre dominio, casos de uso, repositorios y ViewModels.	12	Villanueva Rodriguez, Giuseppe	Done
Sin historia	Tarea técnica sin historia asociada	T-027	Pantalla de inicio del ganadero	Contadores del hato, alertas y registros sanitarios recientes.	6	Villanueva Rodriguez, Giuseppe	Done
TS-013	Adaptar y documentar los servicios backend para móviles	T-028	Crear la suite de pruebas automatizadas del backend	Proyecto Anitec.Platform.Tests con pruebas unitarias, de integración y BDD (xUnit, NSubstitute y Reqnroll) de Iam, Livestock y Sanitary.	12	Beingolea Montalvo, Sebastian Martin	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
Sin historia	Tarea de diseño sin historia asociada	T-029	Elaborar los mock-ups móviles	Mock-ups de Android y Flutter organizados por aplicación y rol, base de las pantallas de la aplicación.	12	Sanchez Silva, Luciana Celeste	Done
Sin historia	Tarea de despliegue sin historia asociada	T-030	Registrar las aplicaciones en Firebase y documentar su despliegue	Aplicaciones Android y Flutter registradas en Firebase App Distribution y documentación del proceso de publicación.	6	Villanueva Rodriguez, Giuseppe	Done

Métrica	Valor
Historias comprometidas	24
Story Points comprometidos	105
Tareas	30
Horas estimadas	205
Tareas completadas (Done)	28 de 30; las 2 restantes están en To-Review

4.2.1.4. Development Evidence for Sprint Review

Esta sección registra los commits que contribuyen al alcance comprometido. La aplicación Android se desarrolló durante el sprint sobre la rama main; el backend y la aplicación web parten de proyectos creados antes del sprint, por lo que se registra su commit de creación, y el backend suma el commit de sus pruebas automatizadas. El Informe documenta el diseño, la configuración y las evidencias del sprint.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
anitec-android	main	2f3b903	feat: add Android project foundation and authentication	Aplicación Compose con el sistema de diseño de AniTec y localización es-419/en; red con Retrofit/OkHttp, sesión cifrada y manejo de 401; inicio de sesión, registro, términos de servicio y navegación por rol; pruebas unitarias del mapeo de errores y de los ViewModels de autenticación.	2026-10-01
anitec-android	main	8ade432	feat: add livestock management, health records and rancher dashboard	Fincas, corrales y animales con búsqueda, filtro por corral, selección múltiple con acciones masivas, ficha técnica y subida de fotografía; lista y formulario de registros	2026-10-01

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
				sanitarios; inicio del ganadero con contadores y registros recientes; caché Room limitada al usuario y limpiada al cerrar sesión; pruebas instrumentadas.	
anitec-backend	main	125d53e	chore: add initial commit with all project files	Solución Anitec.Platform con sus bounded contexts (Iam, Profiles, Livestock, Sanitary, Financial, Activities, Analytics, Devices, Metrics, Subscriptions, Clients, Shared), controllers REST, Entity Framework Core con MySQL, JWT y Swagger.	2026-09-04
anitec-backend	main	888f1b9	chore: add tests for Anitec.Platform	Proyecto Anitec.Platform.Tests con 52 pruebas unitarias, 32 de integración y 20 escenarios BDD (4 archivos .feature con sus pasos); respuesta 403 en lugar de 500 para un rol sin permiso y entorno Testing para ejecutar la API con SQLite en memoria.	2026-10-04
anitec-frontend	main	f12794d	chore: add initial project files	Estructura inicial de la aplicación web en Vue.	2026-09-04
Informe	main	c8b8be7	docs: update report complete generation	Estructura del informe con sus capítulos, títulos y diagramas de arquitectura C4.	2026-09-16
Informe	main	d09a7e2	docs(chapter-4): add backend evidence	Evidencias del despliegue y de la documentación del backend.	2026-10-01
Informe	main	cd83263	docs(update): mockups organized	Mock-ups móviles organizados por aplicación y rol.	2026-10-02

4.2.1.5. Testing Suite Evidence for Sprint Review

Los Web Services cuentan con una suite de pruebas automatizadas en el proyecto Anitec.Platform.Tests (xUnit), dentro del repositorio anitec-backend: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend/tree/main/Anitec.Platform.Tests. La suite reúne tres tipos de pruebas, todas relacionadas con las historias del Sprint 1:

Tipo	Herramientas	Carpeta del proyecto	Pruebas
Unit Tests	xUnit y NSubstitute (repositorios simulados)	Unit/	52
Integration Tests	xUnit y WebApplicationFactory	Integration/	32

Tipo	Herramientas	Carpeta del proyecto	Pruebas
Acceptance Tests (BDD)	Reqnroll sobre xUnit, archivos .feature en Gherkin	Bdd/Features/ y Bdd/Steps/	20 escenarios
Total			**104**

Las pruebas de integración y de BDD arrancan la API real en memoria con WebApplicationFactory. Para no depender de un servidor MySQL, la base de datos es SQLite en memoria y el entorno es Testing: en él la API crea el esquema con EnsureCreated() en lugar de aplicar las migraciones de MySQL. Las pruebas se ejecutan con dotnet test desde la carpeta del repositorio.

Unit Tests. Cada clase valida un servicio de la capa de aplicación o de infraestructura con sus dependencias simuladas.

Test ID	Clase probada	Clase de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resu
TEST-BE-U01	UserCommandService	UserCommandServiceTests	Registro con la contraseña cifrada; nombre de usuario repetido; rol inválido y normalización del rol; correo mal formado, repetido y normalizado a minúsculas; nombre completo vacío; error de base de datos; inicio de sesión con credenciales válidas, usuario inexistente y contraseña incorrecta.	US-004, US-005	18	Comp
TEST-BE-U02	TokenService	TokenServiceTests	El token incluye usuario y rol y vence a los 7 días; un token válido devuelve el id del usuario; un token ausente, mal formado, firmado	US-005, US-007	9	Comp

Test ID	Clase probada	Clase de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resu
			con otra clave, alterado o vencido se rechaza.			
TEST-BE-U03	HashingService	HashingServiceTests	La contraseña no se guarda en claro, cada cifrado produce un hash distinto y la verificación acepta solo la contraseña original.	US-004, US-005	3	Com
TEST-BE-U04	AnimalCommandService	AnimalCommandServiceTests	Alta, edición y baja de un animal con y sin registro existente; alta masiva de 1 a 500 animales, rechazo de cantidades fuera de rango y de un corral inexistente, y códigos consecutivos por corral; cambio de estado y baja de varios animales ignorando los inexistentes.	US-011, US-012, TS-013	17	Com
TEST-BE-U05	CorralCommandService	CorralCommandServiceTests	Alta, edición y baja de un corral, y respuesta de «no encontrado» cuando no existe.	US-044, TS-013	5	Com

Integration Tests. Cada prueba llama a los endpoints REST de la API en memoria con usuarios registrados durante la prueba.

Test ID	Clase de pruebas	Endpoints y comportamientos verificados	Historias relacionadas	Pruebas	Resultado
TEST-BE-I01	AuthenticationApiTests	POST /authentication/sign-up (alta, usuario repetido 409, rol inválido 400), POST /authentication/sign-in (token, contraseña incorrecta 400) y 401 en /animals, /herds, /corrals y /health-events sin token o con un token inválido.	US-004, US-005, US-007	11	Completado
TEST-BE-I02	LivestockApiTests	Fincas (POST, GET, validación 400, no encontrada 404), corrales (POST, listado, DELETE), animales (POST con corral obligatorio, GET, PUT, DELETE, POST /animals/bulk, PATCH /animals/bulk-status y DELETE /animals/bulk con cuerpo JSON) y permisos por rol: el veterinario consulta animales, pero no los crea (403).	US-007, US-008, US-009, US-010, US-011, US-012, US-044, TS-013	16	Completado
TEST-BE-I03	HealthEventsApiTests	POST, GET y DELETE de /health-events, evento inexistente 404 y registro de un evento por un veterinario.	US-014, US-015, US-016	5	Completado

Acceptance Tests (BDD). Los escenarios se escriben en Gherkin (Given-When-Then) en cuatro archivos .feature dentro de Anitec.Platform.Tests/Bdd/Features. Los pasos se implementan en C# en Anitec.Platform.Tests/Bdd/Steps y ejecutan los escenarios contra la API en memoria.

Archivo .feature	Escenarios	Historias relacionadas
Authentication.feature	7	US-004, US-005, US-007
AnimalManagement.feature	7	US-010, US-011, US-012, US-044, TS-013
HealthRecords.feature	3	US-014, US-015, US-016
RolePermissions.feature	3	US-007

Código de Authentication.feature:

```
Feature: Authentication
  As a rancher or a veterinarian
  I want to create an account and sign in
  So that I can work with my own information in AniTec

  Scenario: A new rancher signs up and signs in
    Given I am a new visitor
    When I sign up as a "Rancher"
    And I sign in with my credentials
    Then I receive a session token
```

```
    And my role is "Rancher"

  Scenario: A new veterinarian signs up and signs in
    Given I am a new visitor
    When I sign up as a "Veterinarian"
    And I sign in with my credentials
    Then I receive a session token
    And my role is "Veterinarian"

  Scenario: Signing up with an unknown role is rejected
    Given I am a new visitor
    When I sign up as a "Administrator"
    Then the request is rejected with status 400

  Scenario: Signing up twice with the same username is rejected
    Given I have an account as a "Rancher"
    When I sign up as a "Rancher"
    Then the request is rejected with status 409

  Scenario: Signing in with a wrong password is rejected
    Given I have an account as a "Rancher"
    When I sign in with the password "wrong-password"
    Then the request is rejected with status 400
    And I do not receive a session token

  Scenario: Protected information requires a session
    Given I am a new visitor
    When I request the list of animals
    Then the request is rejected with status 401

  Scenario: A signed-in user can request protected information
    Given I am signed in as a "Rancher"
    When I request the list of animals
    Then the request succeeds
```

Código de AnimalManagement.feature:

```
Feature: Animal management
  As a rancher
  I want to register and organize my animals by farm and corral
  So that I always know what I have and in what condition

  Background:
    Given I am signed in as a "Rancher"
    And I have a farm called "La Esperanza"
    And the farm has a corral called "Corral A"

  Scenario: Register an animal in a corral
    When I register an animal with the code "COW-001" in "Corral A"
    Then the animal "COW-001" appears in the animal list

  Scenario: An animal must belong to a corral
    When I register an animal without a corral
    Then the request is rejected with status 400

  Scenario: Register several animals at once
    When I register 3 animals in bulk in "Corral A"
    Then the codes "CorralA-001, CorralA-002, CorralA-003" are assigned

  Scenario: Bulk registration is limited to 500 animals
    When I register 501 animals in bulk in "Corral A"
    Then the request is rejected with status 400

  Scenario: Mark several animals as sold
    Given the corral "Corral A" has the animal "COW-001"
    And the corral "Corral A" has the animal "COW-002"
    When I mark the animals "COW-001, COW-002" as "Vendido"
    Then the animals "COW-001, COW-002" have the status "Vendido"

  Scenario: Remove an animal
    Given the corral "Corral A" has the animal "COW-003"
    When I delete the animal "COW-003"
    Then the animal "COW-003" no longer appears in the animal list

  Scenario: Remove several animals at once
    Given the corral "Corral A" has the animal "COW-004"
    And the corral "Corral A" has the animal "COW-005"
```

```
    When I delete the animals "COW-004, COW-005"
    Then the animal "COW-004" no longer appears in the animal list
    And the animal "COW-005" no longer appears in the animal list
```

Código de HealthRecords.feature:

```
Feature: Health records
  As a rancher or a veterinarian
  I want to keep the health history of each animal
  So that incidents, vaccines and treatments are never forgotten

  Background:
    Given a rancher has a farm called "La Esperanza"
    And the farm has a corral called "Corral A"
    And the corral "Corral A" has the animal "COW-001"

  Scenario: A rancher records a health incident
    Given I am signed in as a "Rancher"
    When I record a "Incidencia" health event for the animal "COW-001" with the
description "Cojera en la pata trasera"
    Then the health event is created
    And the health record of the animal "COW-001" includes the description "Cojera en la
pata trasera"

  Scenario: A veterinarian records a treatment
    Given I am signed in as a "Veterinarian"
    When I record a "Tratamiento" health event for the animal "COW-001" with the
description "Antibiótico por 5 días"
    Then the health event is created
    And the health record of the animal "COW-001" includes the description "Antibiótico
por 5 días"

  Scenario: Health records require a session
    Given I am a new visitor
    When I request the health records
    Then the request is rejected with status 401
```

Código de RolePermissions.feature:

```
Feature: Role permissions
  As the owner of the information
  I want each role to do only what it is meant to do
  So that a veterinarian can follow animals without changing the herd

  Background:
    Given a rancher has a farm called "La Esperanza"
    And the farm has a corral called "Corral A"
    And the corral "Corral A" has the animal "COW-001"

  Scenario: A veterinarian can see the animals
    Given I am signed in as a "Veterinarian"
    When I request the list of animals
    Then the request succeeds
    And the animal "COW-001" appears in the animal list

  Scenario: A veterinarian cannot register animals
    Given I am signed in as a "Veterinarian"
    When I register an animal with the code "COW-009" in "Corral A"
    Then the request is rejected with status 403

  Scenario: A veterinarian cannot delete animals
    Given I am signed in as a "Veterinarian"
    When I delete the animal "COW-001"
    Then the request is rejected with status 403
```

Resultado de la ejecución. Las 104 pruebas del backend (52 unitarias, 32 de integración y 20 escenarios BDD) pasan sin errores. Las pruebas de integración detectaron un defecto: un veterinario que intentaba crear un animal recibía 500 en lugar de 403. Se corrigió en AuthorizeAttribute, que ahora responde 403 Forbidden, y quedó cubierto por la prueba AVeterinarian_CannotCreateAnimals y por el escenario «A veterinarian cannot register animals».

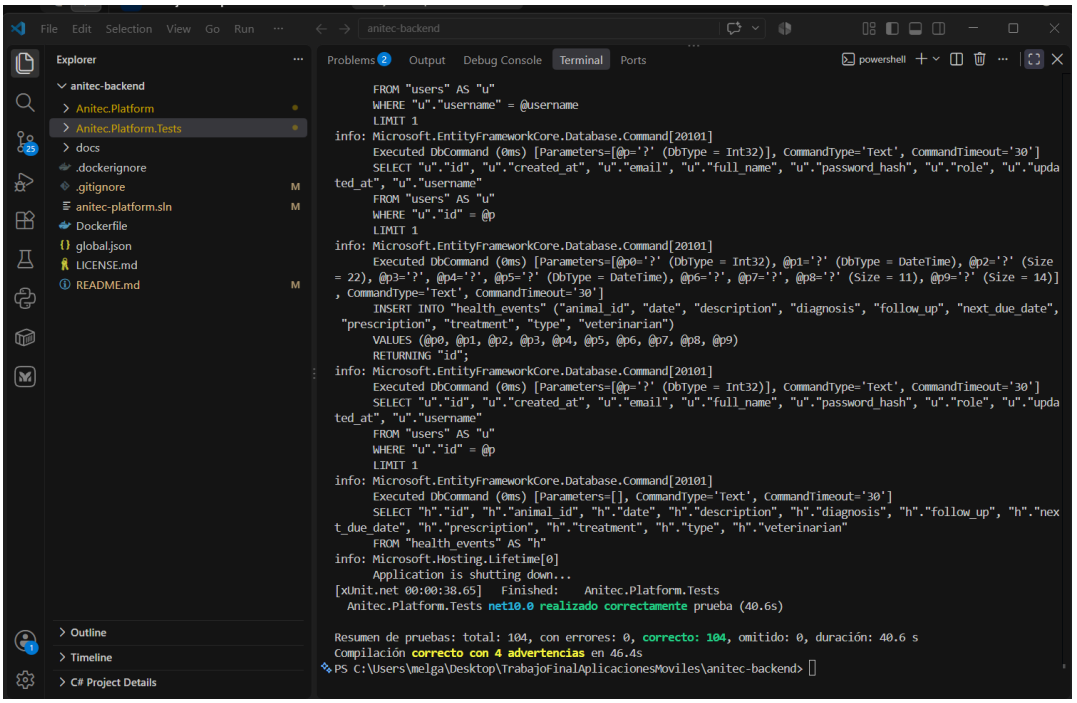


Figura 4.2.1.5.1. Captura de los 104 tests exitosos. Fuente: elaboración propia.

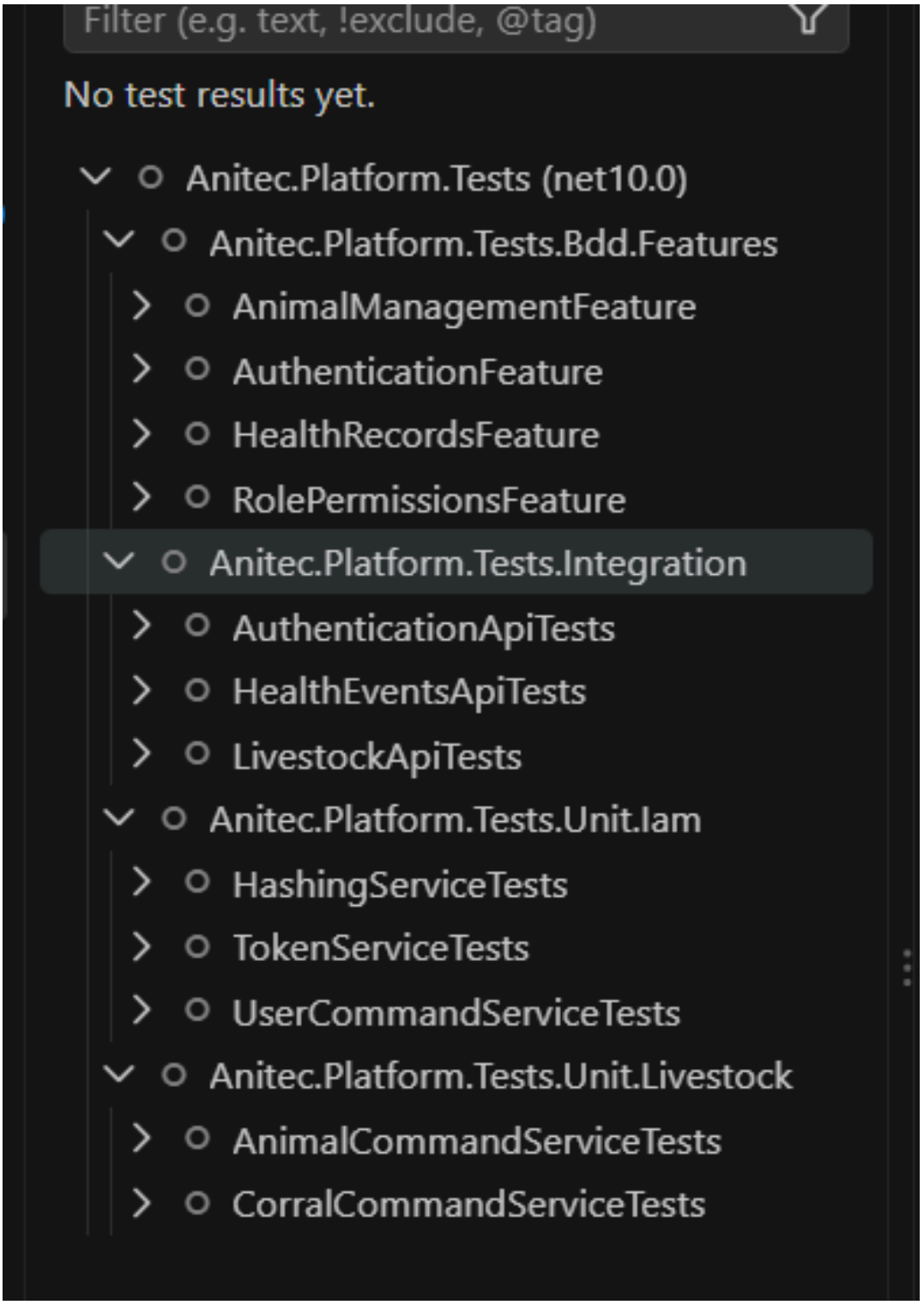


Figura 4.2.1.5.2. Captura de la lista de tests. Fuente: elaboración propia.

Otras pruebas del sprint. Pruebas de la documentación publicada, de la Landing Page y de la aplicación Android.

Test ID	Product	Type	Class / Feature	Behavior	Related Story	Result
TEST-BE-001	Web Services	Exploratoria / contrato	Swagger UI (/swagger)	Validar disponibilidad de OpenAPI y endpoints Animals /	Autenticación y registro esencial de animales / sanidad	Completado — Figura 4.2.1.6.1

Test ID	Product	Type	Class / Feature	Behavior	Related Story	Result
				Authentication en el entorno publicado		
TEST-LP-003	Landing Page	Funcional	Navigation	Validar la navegación entre las distintas secciones de la Landing Page.	Navegación y experiencia de usuario	Completado
TEST-LP-004	Landing Page	Responsive Testing	Responsive Design	Verificar la correcta adaptación de la interfaz en dispositivos móviles y escritorio.	Accesibilidad multiplataforma	Completado
TEST-AN-001	Android	Automatizada (JUnit)	AuthViewModelsTest, HomeViewModelTest	Validar flujos de estado de autenticación (Login) y carga del dashboard inicial.	Autenticación de usuario	Completado
TEST-AN-002	Android	Automatizada (JUnit)	LivestockDomainTest, LivestockUseCasesTest, SanitaryTest	Validar reglas de negocio del dominio de ganado, sanidad y casos de uso.	Gestión de ganado y Sanidad	Completado
TEST-AN-003	Android	Automatizada (Instrumentada)	RoomDaoTest	Verificar operaciones CRUD y persistencia local de la base de datos con Room (Offline-First).	Gestión de ganado	Completado

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
anitec-backend	main	888f1b9	chore: add tests for Anitec.Platform	Agrega el proyecto Anitec.Platform.Tests con las pruebas	2026-10-04

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
				unitarias, de integración y BDD, y la corrección de AuthorizeAttribute (403 en lugar de 500).	
anitec-landing-page	main	2a88184	chore: add initial project files	Agrega la estructura inicial del proyecto de la Landing Page, incluyendo configuración base y componentes principales para el desarrollo de la interfaz	2026-09-04
anitec-android	main	2f3b903	feat: add Android project foundation and authentication	Configura la arquitectura base e incluye pruebas unitarias iniciales para los flujos de autenticación (AuthViewModelsTest, HomeViewModelTest).	2026-10-01
anitec-android	main	8ade432	feat: add livestock management, health records and rancher dashboard	Implementa la lógica de ganado/sanidad y adjunta las suites de pruebas unitarias e instrumentadas (LivestockDomainTest, SanitaryTest, RoomDaoTest).	2026-10-01

4.2.1.6. Execution Evidence for Sprint Review

El Sprint 1 permite recorrer la Landing Page publicada, consultar la documentación de la API y ejecutar la aplicación Android con autenticación, fincas, corrales, animales y registros sanitarios. La evidencia de ejecución muestra el resultado mediante capturas identificables y un video que explica el recorrido implementado.

Producto	Vista o flujo	Entorno / dispositivo	Evidencia	Estado
Landing Page	Página principal y responsive	Navegador de escritorio y móvil	Figura 4.2.1.6.2 (landing_despliegue.png)	Completado
Android	Registro, inicio de sesión, inicio del ganadero, fincas, corrales, animales (lista, ficha y formulario) y registros sanitarios	Emulador y dispositivo físico	Figuras 4.2.1.6.3 a 4.2.1.6.10	Parcial: captura del inicio de sesión; el resto pendiente
Flutter	Autenticación y funciones core comprometidas	Dispositivo o emulador objetivo	Pendiente	Pendiente (placeholder)
Web Services	Swagger UI: documentación OpenAPI de la API publicada	Navegador contra el Swagger del	Figura 4.2.1.6.1 (swagger-ui.png)	Completado

Producto	Vista o flujo	Entorno / dispositivo	Evidencia	Estado
		servicio publicado en Render		
Integración	Consumo de la API y manejo de errores desde la aplicación Android	Aplicación contra el backend vigente	Figura 4.2.1.6.11	Pendiente de captura

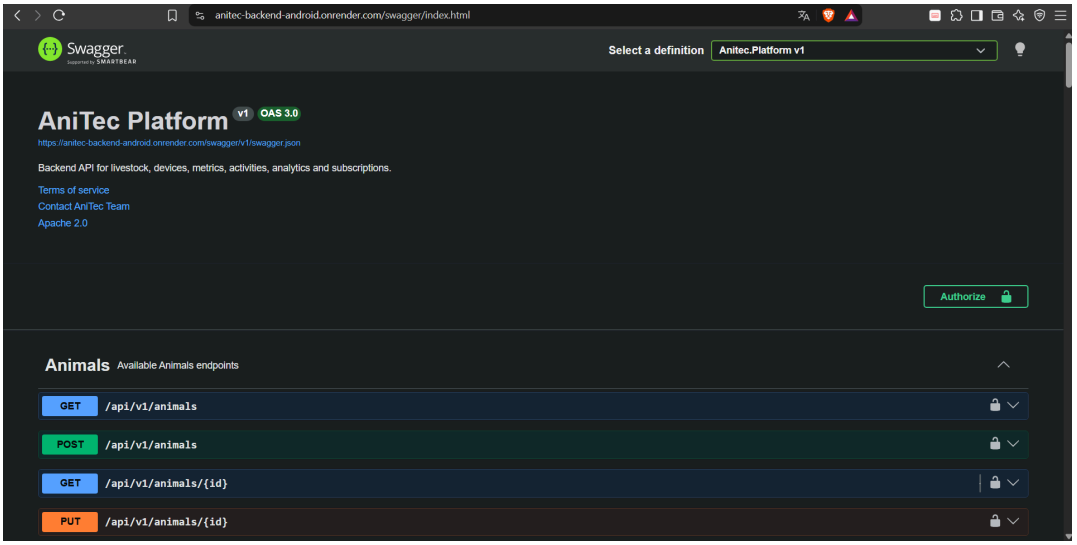


Figura 4.2.1.6.1. Swagger UI de AniTec Platform en Render, con endpoints de Animals y Authentication. Fuente: elaboración propia.

La captura demuestra que la documentación interactiva carga desde el entorno publicado y expone los contratos REST usados por el Sprint 1.

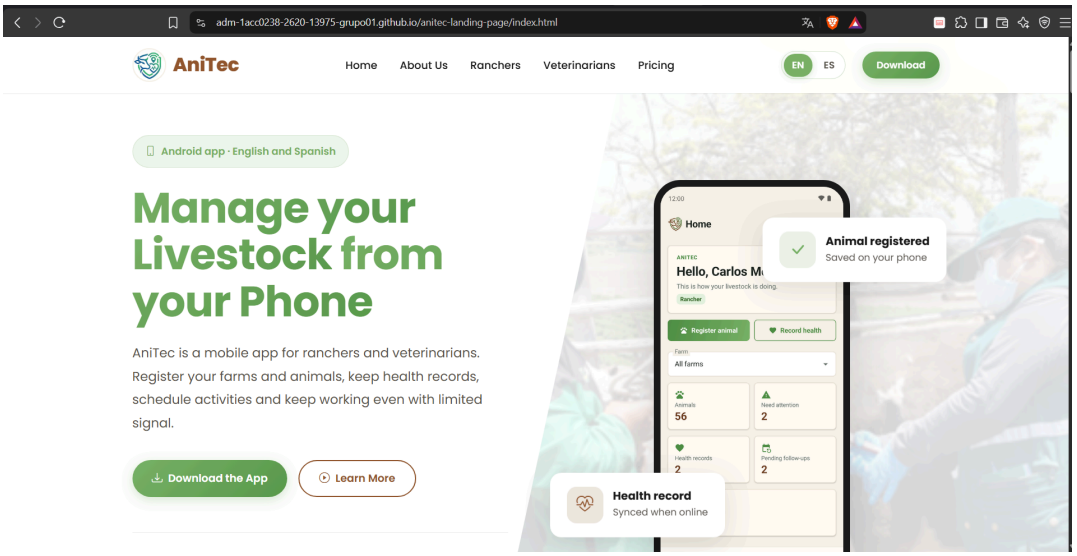


Figura 4.2.1.6.2. Landing de AniTec desplegado, Fuente: elaboración propia.

Las capturas muestran la implementación de la Landing Page de AniTec en navegadores. Se verificó la correcta visualización del contenido, la navegación entre secciones y la adaptación responsive de la interfaz para distintos tamaños de pantalla.

Aplicación Android. La captura del inicio de sesión ya está incorporada; las demás se añadirán desde el emulador y el dispositivo físico.

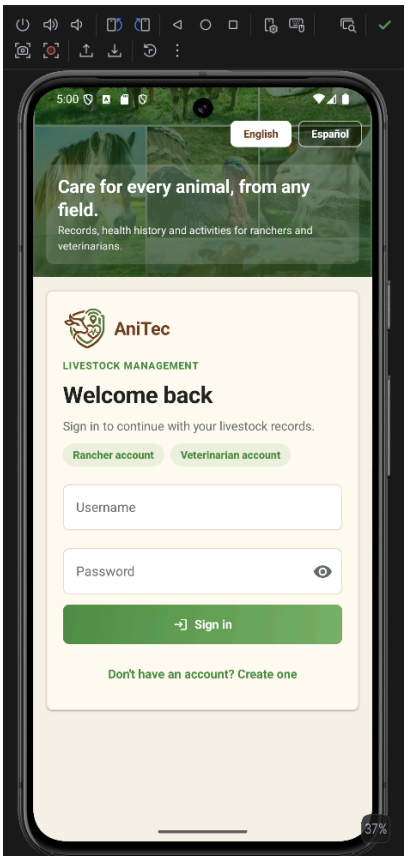


Figura 4.2.1.6.3. Ejecución de la aplicación nativa Android mostrando el flujo de Autenticación. Fuente: elaboración propia.

La captura evidencia el correcto funcionamiento de la aplicación Android instalada en el emulador, mostrando la interfaz nativa para el inicio de sesión de ganaderos y veterinarios.

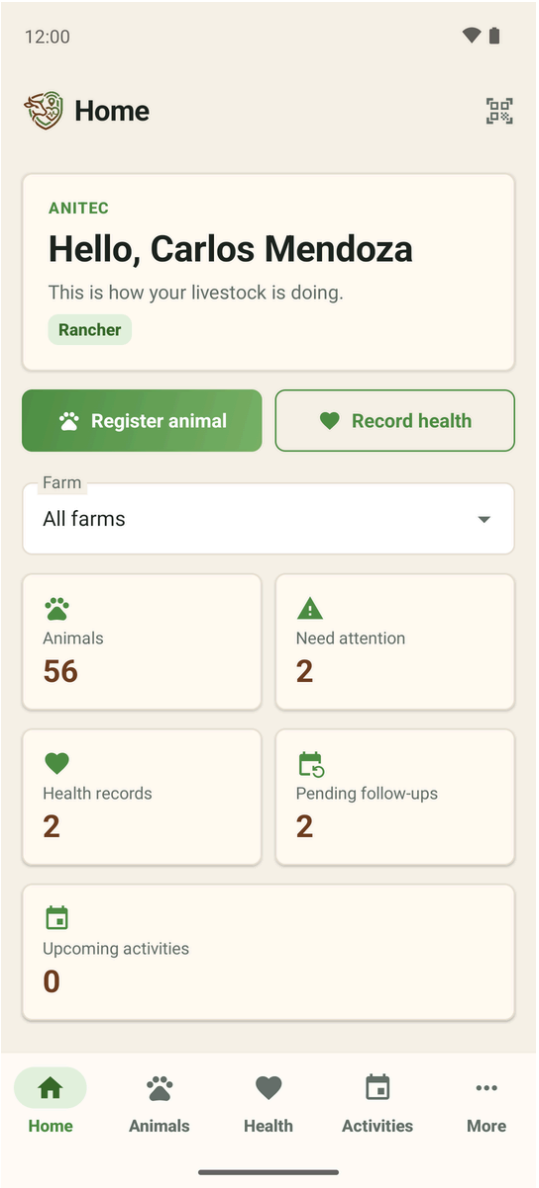


Figura 4.2.1.6.4. Inicio del ganadero. Fuente: elaboración propia.

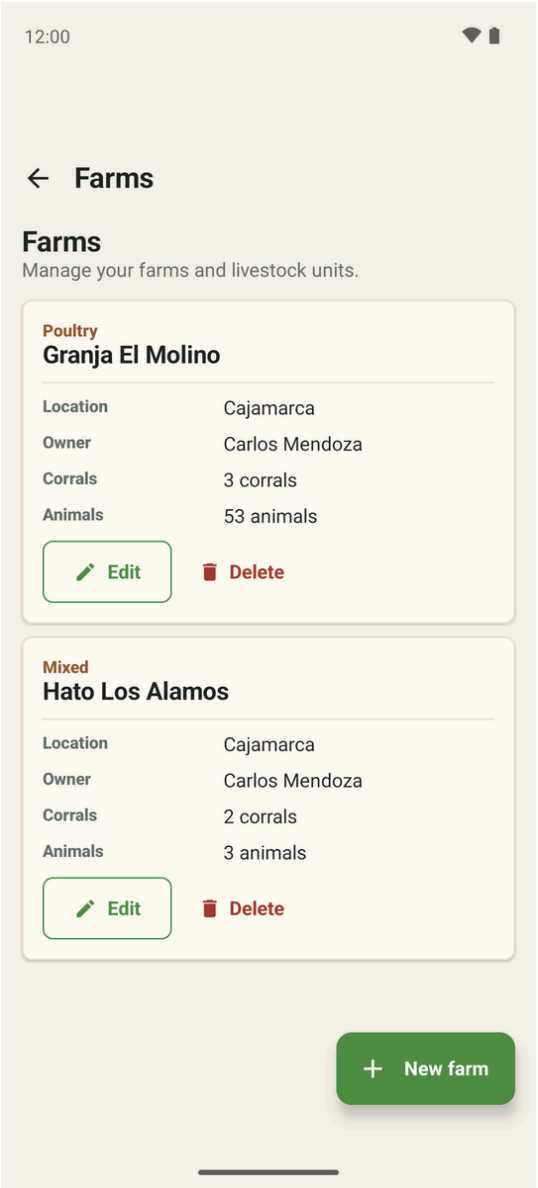


Figura 4.2.1.6.5. Lista de fincas. Fuente: elaboración propia.

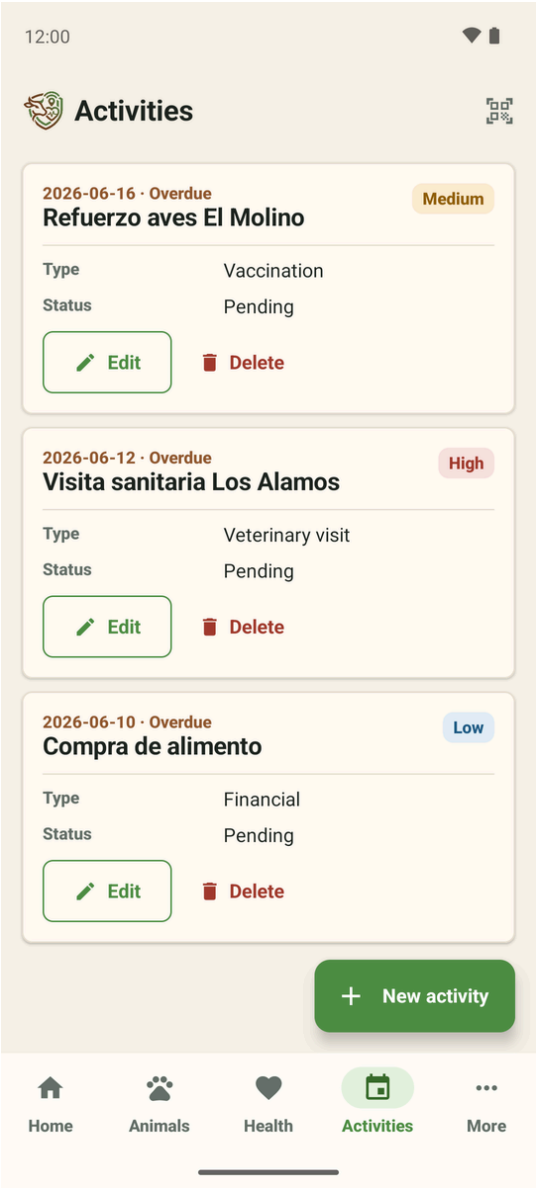


Figura 4.2.1.6.6. Lista de actividades. Fuente: elaboración propia.

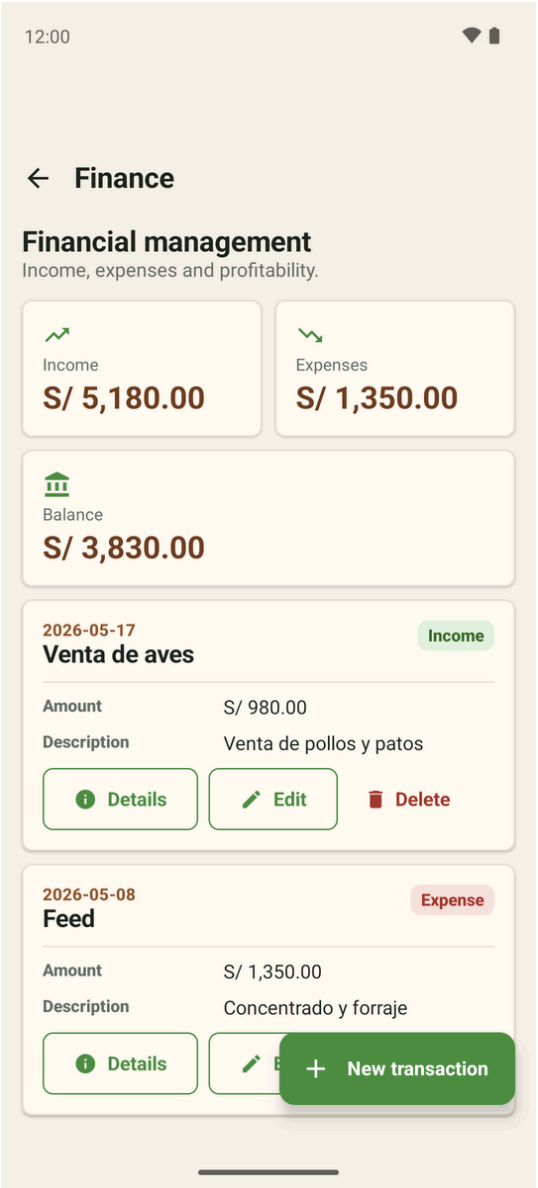


Figura 4.2.1.6.7. Pantalla de fiananzas Fuente: elaboración propia.

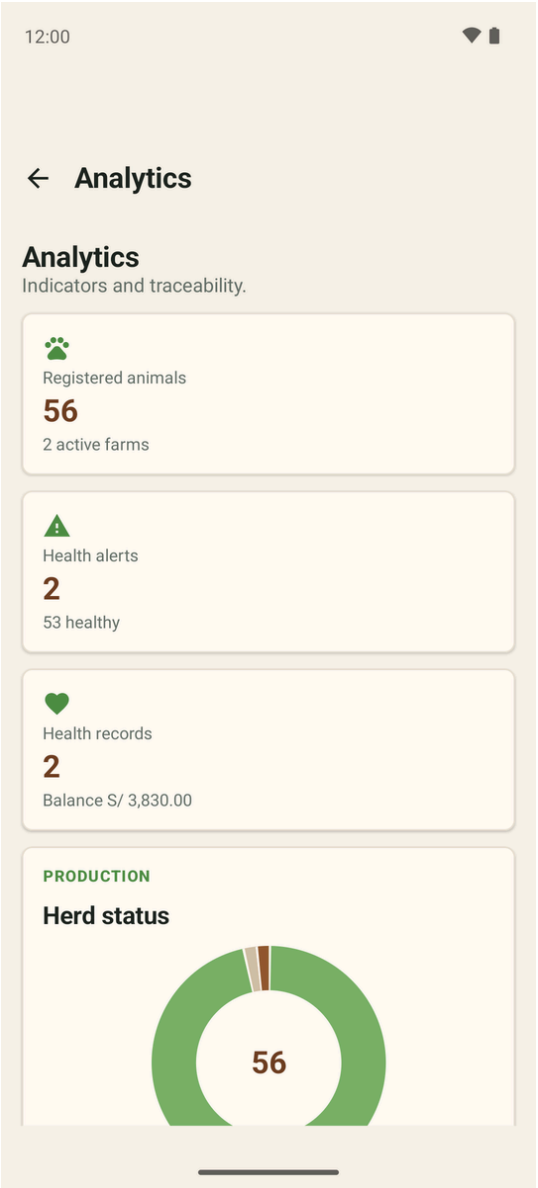


Figura 4.2.1.6.8. Pantalla de analiticas. Fuente: elaboración propia.

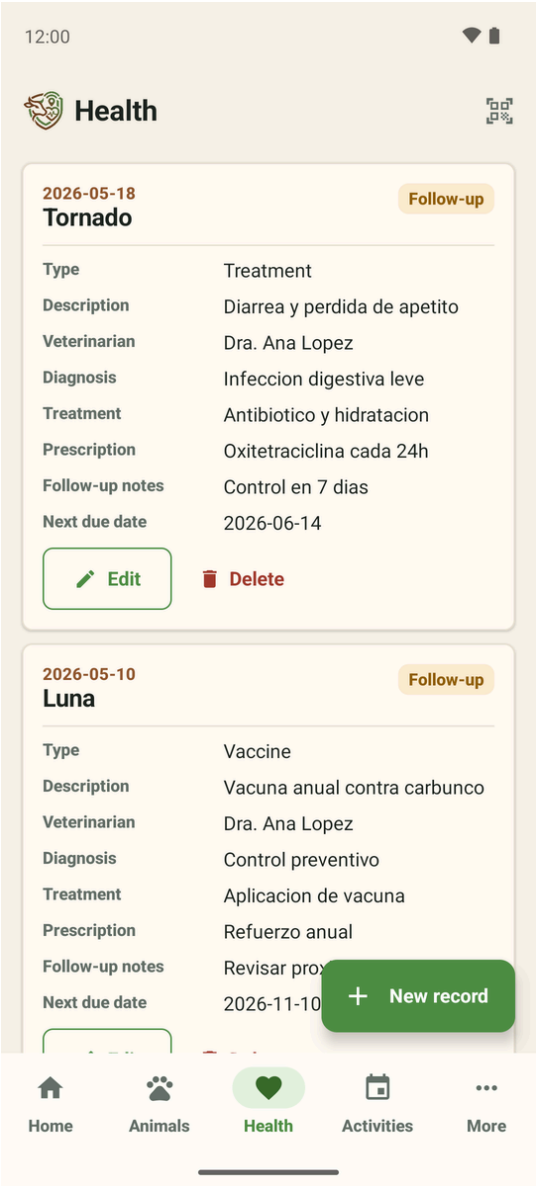


Figura 4.2.1.6.9. Pantalla de sanidad Fuente: elaboración propia.

- **Landing Page:** Ver video de ejecución (0:00 - 2:04)
- **Backend:** Ver video de ejecución (0:00 - 4:24)
- **App Android:** Ver video de ejecución (0:00 - 2:59)

4.2.1.7. Services Documentation Evidence for Sprint Review

El backend documenta su API con OpenAPI mediante Swashbuckle (AddSwaggerGen, UseSwagger y UseSwaggerUI en Program.cs) e incluye el esquema Bearer JWT para autorizar las llamadas desde la interfaz de Swagger. Durante el Sprint 1 quedaron documentados los endpoints de autenticación, fincas, corrales, animales y registros sanitarios que consume la aplicación Android, incluidos los de corrales y las operaciones masivas añadidos para el uso móvil.

Convención de llamada. Todas las rutas parten de {BASE_URL}/api/v1, donde {BASE_URL} es la URL del servicio (local o publicado). Las solicitudes con cuerpo usan Content-Type: application/json. Salvo el registro y el inicio de sesión, cada llamada envía el token en la cabecera Authorization: Bearer y recibe 401 si falta o no es válido. Las fechas usan el

formato yyyy-MM-dd. Los valores de estado, tipo y especie son texto en español, por ejemplo Saludable o Vacuna. En los ejemplos, y representan los valores reales.

Autenticación (/authentication)

Related story	HTTP	Call syntax	Parameters / body	Success resp
US-004	POST	curl -X POST "{BASE_URL}/api/v1/authentication/sign-up" -H "Content-Type: application/json" -d '{"fullName":"Carlos Mendoza","username":"cmendoza","password":"","role":"Rancher"}'	Body SignUpResource: username, password, fullName y role (Rancher o Veterinarian).	200 con {"me registro no d sesión a cont
US-005	POST	curl -X POST "{BASE_URL}/api/v1/authentication/sign-in" -H "Content-Type: application/json" -d '{"username":"ganadero","password":""}'	Body SignInResource: username y password.	200 con {"id":1,"usern Mendoza","r el token JWT envía en las c

Fincas (/herds; lectura para Rancher y Veterinarian, escritura solo para Rancher)

Ejemplo de HerdResource: {"id":1,"name":"Hato Los Alamos","location":"Cajamarca","owner":"Carlos Mendoza","ownerId":1,"veterinarianId":2,"mainType":"Mixto"}.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-008	GET	curl "{BASE_URL}/api/v1/herds" -H "Authorization: Bearer "	Ninguno.	200 con la lista de fin (un HerdResou por elemento)
US-008	GET	curl "{BASE_URL}/api/v1/herds/1" -H "Authorization: Bearer "	Ruta: id.	200 con el HerdResou de la finca
US-009	POST	curl -X POST "{BASE_URL}/api/v1/herds" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"name":"Hato Los Alamos","location":"Cajamarca","owner":"Carlos Mendoza","ownerId":1,"veterinarianId":null,"mainType":"Mixto"}'	Body CreateHerdResource: name, location, owner, ownerId, veterinarianId (opcional) y mainType.	201 con el HerdResou creado.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-009	PUT	curl -X PUT "{BASE_URL}/api/v1/herds/1" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{...}'	Ruta: id. Body CreateHerdResource.	200 con la finca actualizad
US-009	DELETE	curl -X DELETE "{BASE_URL}/api/v1/herds/1" -H "Authorization: Bearer "	Ruta: id.	204 sin contenido

Corrales (/corrals; lectura para Rancher y Veterinarian, escritura solo para Rancher)

Ejemplo de CorralResource: {"id":1,"name":"Corral ALAMOS 1","herdId":1}.

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
US-044	GET	curl "{BASE_URL}/api/v1/corrals" -H "Authorization: Bearer "	Ninguno.	200 con la lista de corrales.	401; 403.
US-044	GET	curl "{BASE_URL}/api/v1/corrals/1" -H "Authorization: Bearer "	Ruta: id.	200 con el CorralResource.	404; 401.
US-044	POST	curl -X POST "{BASE_URL}/api/v1/corrals" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"name":"Corral ALAMOS 1","herdId":1}'	Body CreateCorralResource: name y herdId.	201 con el corral creado.	400; 401; 403.
US-044	PUT	curl -X PUT "{BASE_URL}/api/v1/corrals/1" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{...}'	Ruta: id. Body CreateCorralResource.	200 con el corral actualizado.	404; 400; 401; 403.
US-044	DELETE	curl -X DELETE "{BASE_URL}/api/v1/corrals/1" -H "Authorization: Bearer "	Ruta: id.	204 sin contenido.	404; 401; 403.

Animales (/animals; lectura para Rancher y Veterinarian, escritura solo para Rancher)

Ejemplo de AnimalResource: {"id":1,"tag":"BOV-001","name":"Luna","species":"Bovino","breed":"Brown Swiss","gender":"Hembra","birthDate":"2021-03-14","weight":410.0,"status":"Saludable","herdId":1,"corralId":1,"source":"Comprado","ageRange":"Adulto","imageUrl":"/uploads/animals/.jpg"}.

Related story	HTTP	Call syntax
US-010	GET	curl "{BASE_URL}/api/v1/animals" -H "Authorization: Bearer "
US-013	GET	curl "{BASE_URL}/api/v1/animals/1" -H "Authorization: Bearer "
US-011	POST	curl -X POST "{BASE_URL}/api/v1/animals" -H "Authorization: Bearer " -H "Content-Type: applicatior 900","name":"Nube","species":"Bovino","breed":"Holstein","gender":"Hembra","birthDate":"2024-02- 01","weight":280.0,"status":"Saludable","herdId":1,"corralId":1}'
US-012	PUT	curl -X PUT "{BASE_URL}/api/v1/animals/1" -H "Authorization: Bearer " -H "Content-Type: applicatio
US-012	DELETE	curl -X DELETE "{BASE_URL}/api/v1/animals/1" -H "Authorization: Bearer "
US-011	POST	curl -X POST "{BASE_URL}/api/v1/animals/bulk" -H "Authorization: Bearer " -H "Content-Type: applic '{"species":"Bovino","breed":"Holstein","gender":"Hembra","weight":120.0,"status":"Saludable","herdI
US-012	PATCH	curl -X PATCH "{BASE_URL}/api/v1/animals/bulk-status" -H "Authorization: Bearer " -H "Content-Typ '{"animalIds":[1,2],"status":"Observacion"}'
US-012	DELETE	curl -X DELETE "{BASE_URL}/api/v1/animals/bulk" -H "Authorization: Bearer " -H "Content-Type: app [1,2]}'
US-045	POST	curl -X POST "{BASE_URL}/api/v1/animals/upload-image" -H "Authorization: Bearer " -F "file=@foto

Registros sanitarios (/health-events; Rancher y Veterinarian)

Ejemplo de HealthEventResource: {"id":1,"animalId":1,"type":"Vacuna","date":"2026-05-10","description":"Vacuna anual contra carbunco","veterinarian":"Dra. Ana Lopez","diagnosis":"Control preventivo","treatment":"Aplicacion de

vacuna","prescription":"Refuerzo anual","followUp":"Revisar proxima campana","nextDueDate":"2026-11-10"}.

Related story	HTTP	Call syntax	Parameter
US-014	GET	curl "{BASE_URL}/api/v1/health-events" -H "Authorization: Bearer "	Ninguno.
US-014	GET	curl "{BASE_URL}/api/v1/health-events/1" -H "Authorization: Bearer "	Ruta: id.
US-015, US-016	POST	curl -X POST "{BASE_URL}/api/v1/health-events" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"animalId":1,"type":"Incidencia","date":"2026-10-01","description":"Cojera en la pata trasera","veterinarian":"Dra. Ana Lopez","diagnosis":"","treatment":"","prescription":"","followUp":"","nextDueDate":null}'	Body CreateHeal animalId, ty description diagnosis, t prescriptio nextDueDa
US-015	PUT	curl -X PUT "{BASE_URL}/api/v1/health-events/1" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{...}'	Ruta: id. Bo CreateHeal
US-015	DELETE	curl -X DELETE "{BASE_URL}/api/v1/health-events/1" -H "Authorization: Bearer "	Ruta: id.

Una solicitud con un rol no autorizado responde 403: las pruebas de integración (sección 4.2.1.5) detectaron que antes respondía 500 y se corrigió en AuthorizeAttribute. El servidor aún devuelve todos los registros sin filtrarlos por usuario; ese punto se corregirá en el backend y mientras tanto la aplicación filtra en el cliente.

- **Web Services repository:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend/tree/main
- **Swagger:** https://anitec-backend-android.onrender.com/swagger/index.html
- **API publicada:** https://anitec-backend-android.onrender.com
- **Documentation commits:** 125d53e (configuración inicial de Swagger/OpenAPI y controllers) y 9831844 (endpoints de corrales y operaciones masivas de animales).
- **Interaction screenshots:** las capturas siguientes muestran llamadas con datos de muestra desde Swagger.

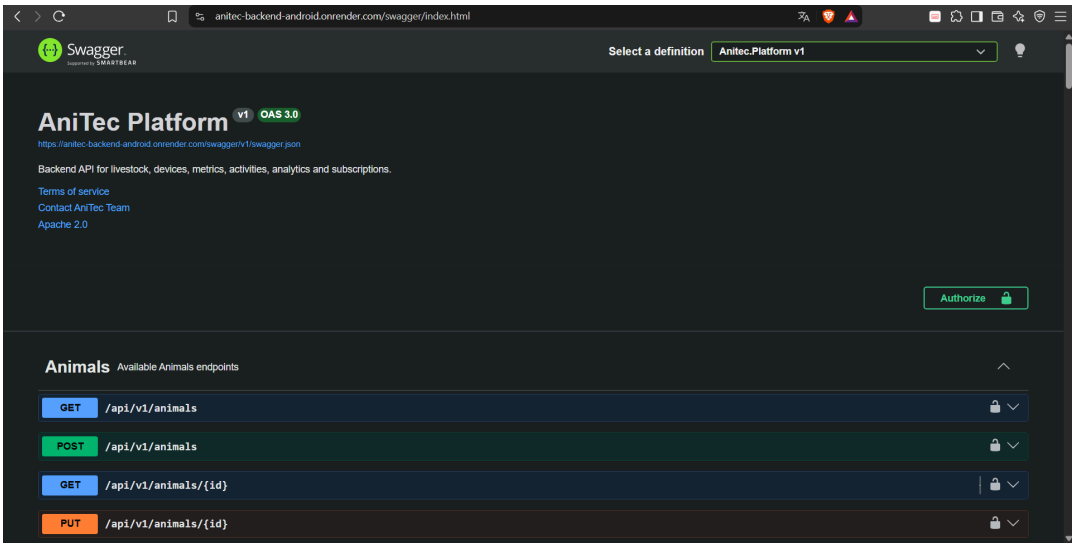


Figura 4.2.1.7.1. Evidencia de documentación de servicios: Swagger UI con Authorize y catálogo de endpoints Animals / Authentication. Fuente: elaboración propia.

La figura respalda que OpenAPI está disponible públicamente y que los endpoints del Sprint 1 pueden ejercitarse desde Swagger.

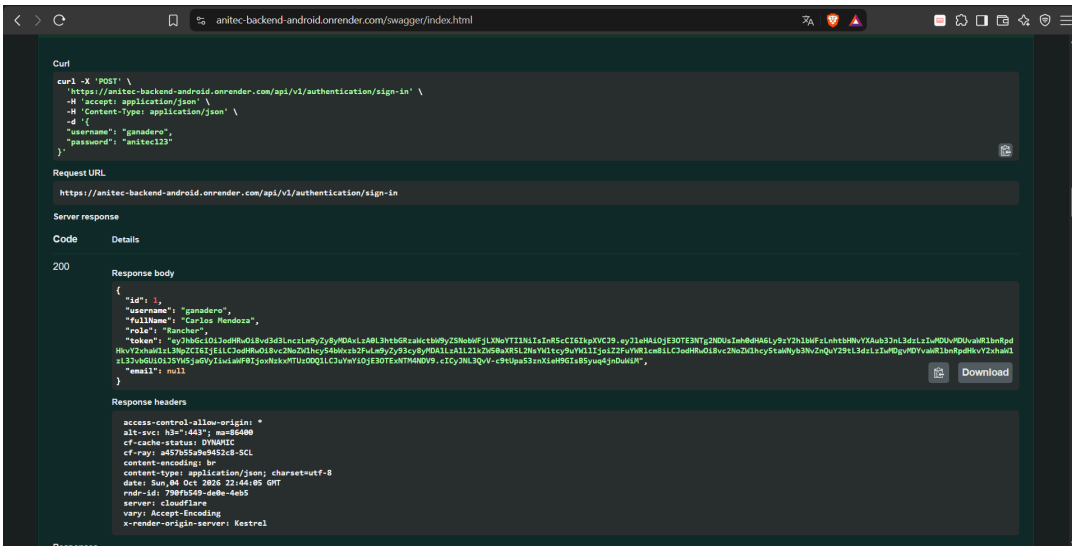


Figura 4.2.1.7.2. POST /authentication/sign-in con datos de muestra y su respuesta con token. Fuente: elaboración propia.

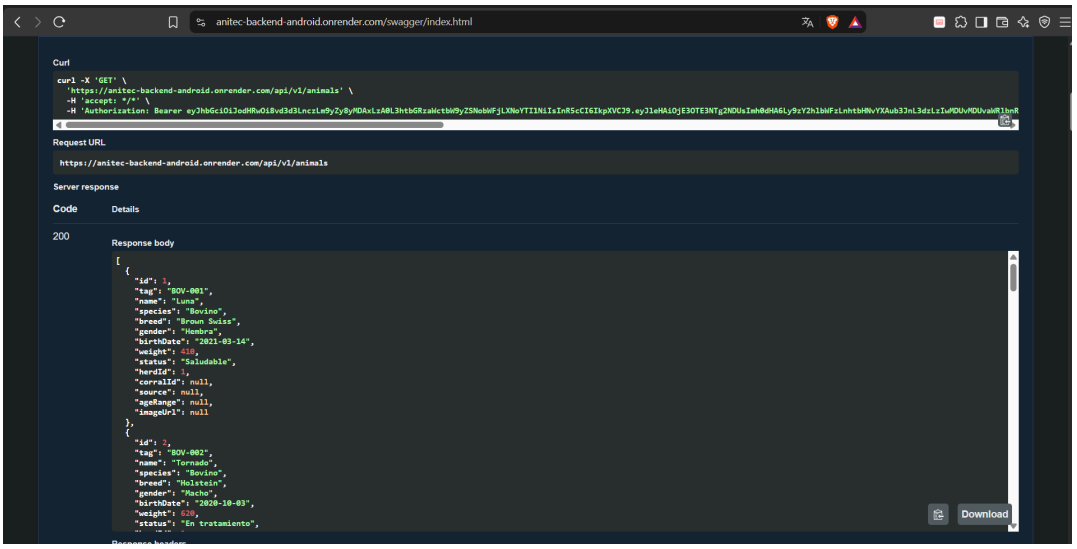


Figura 4.2.1.7.3. GET /animals autorizado con el token. Fuente: elaboración propia.

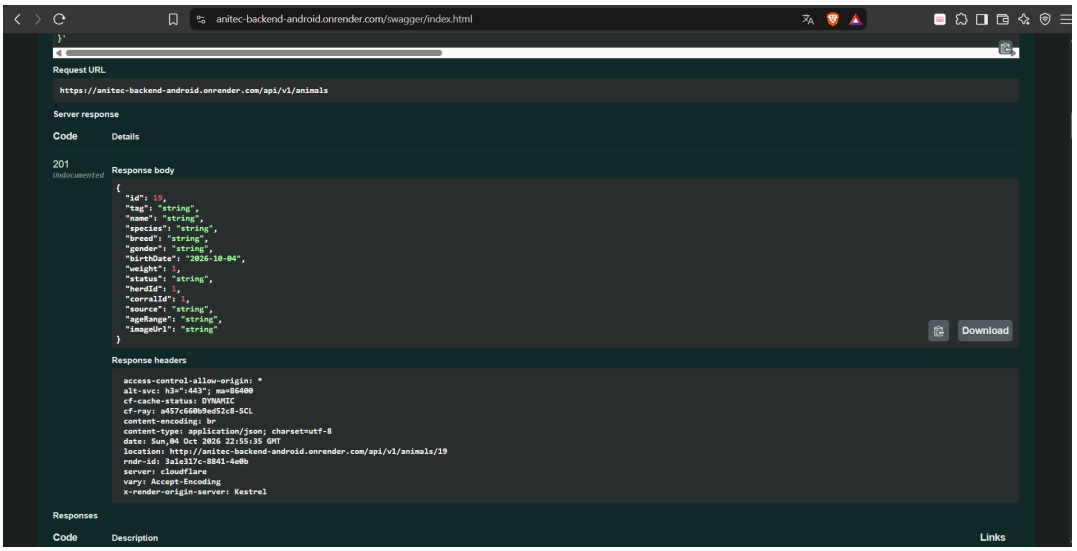


Figura 4.2.1.7.4. POST /animals con un animal de muestra y su respuesta 201. Fuente: elaboración propia.

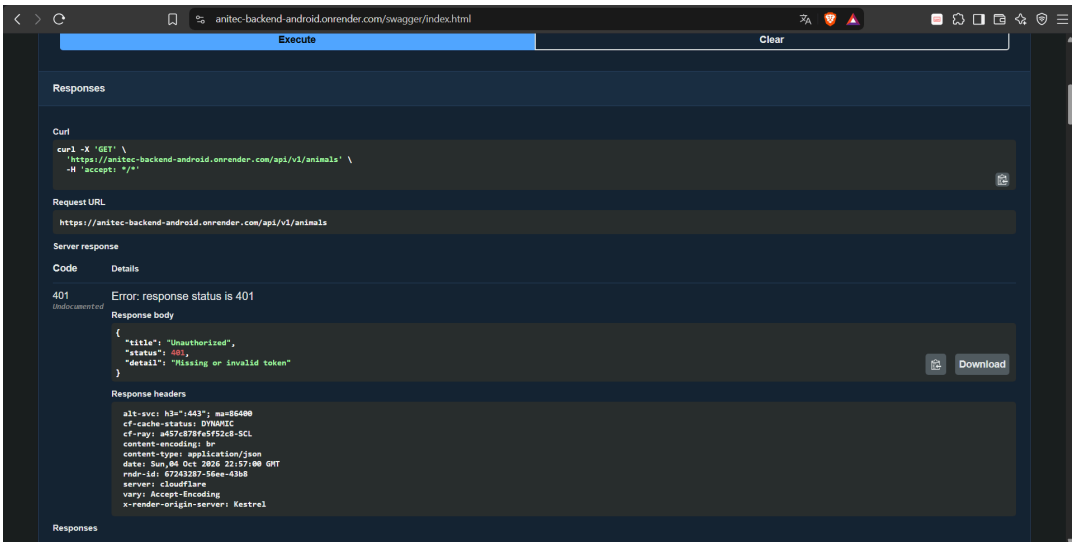


Figura 4.2.1.7.5. Respuesta 401 al llamar un endpoint sin token. Fuente: elaboración propia.

4.2.1.8. Software Deployment Evidence for Sprint Review

Durante el Sprint 1 se prepararon los entornos de publicación de cada producto: el backend se configuró como servicio Docker en Render con una base de datos MySQL externa, la Landing Page se publicó en GitHub Pages y el proyecto de Firebase tiene registradas las aplicaciones Android y Flutter para su distribución. Las cuentas utilizadas son la organización de GitHub del equipo, Render, Firebase (plan Spark) y el proveedor de MySQL. Los pasos detallados de cada despliegue se describen en la sección 4.1.4.

Product	Platform	Configuration performed	Version / Commit	Public URL or Release
Landing Page	GitHub Pages	Repositorio anitec-landing-page; publicación desde la rama main y la carpeta raíz.	Commit d774fb1	https://adm-1acc0238-26 grupo01.github.io/anitec-
Web Services	Render	Docker con el Dockerfile de la raíz; variables ASPNETCORE_ENVIRONMENT, ConnectionStrings__DefaultConnection,	Commit 2d08551	API: https://anitec-backer android.onrender.com. Sv

Product	Platform	Configuration performed	Version / Commit	Public URL or Release
		TokenSettings__Secret y StripeSettings__*; MySQL externo; migraciones automáticas al iniciar.		https://anitec-backend-android.onrender.com/sw
Android	Firebase App Distribution	Proyecto «Anitec» y aplicación «AniTec Android» registrada con el paquete com.anitec.platform (App ID 1:969068830564:android:5a2dafc1bf9a7f44652471). Firma, grupo de testers y release pendientes.	versionName 0.1.0; commit 8ade432	Release pendiente
Flutter	Firebase App Distribution	Aplicación «AniTec Flutter» registrada como aplicación de Apple en el proyecto. El desarrollo y la distribución están pendientes.	Pendiente	Release pendiente

Pasos ejecutados (Web Services):

1. Publicar el Web Service anitec-backend-android en Render con runtime Docker desde la rama main de anitec-backend.
2. Configurar los secretos de conexión MySQL, JWT y Stripe en Environment (los valores no se exponen en el informe).
3. Verificar el estado **Live** en Events y la disponibilidad de la URL primaria (https://anitec-backend-android.onrender.com).
4. Comprobar que Swagger UI responde en /swagger/index.html (https://anitec-backend-android.onrender.com/swagger/index.html). Verificado: responde 200, y los endpoints protegidos responden 401 sin token.

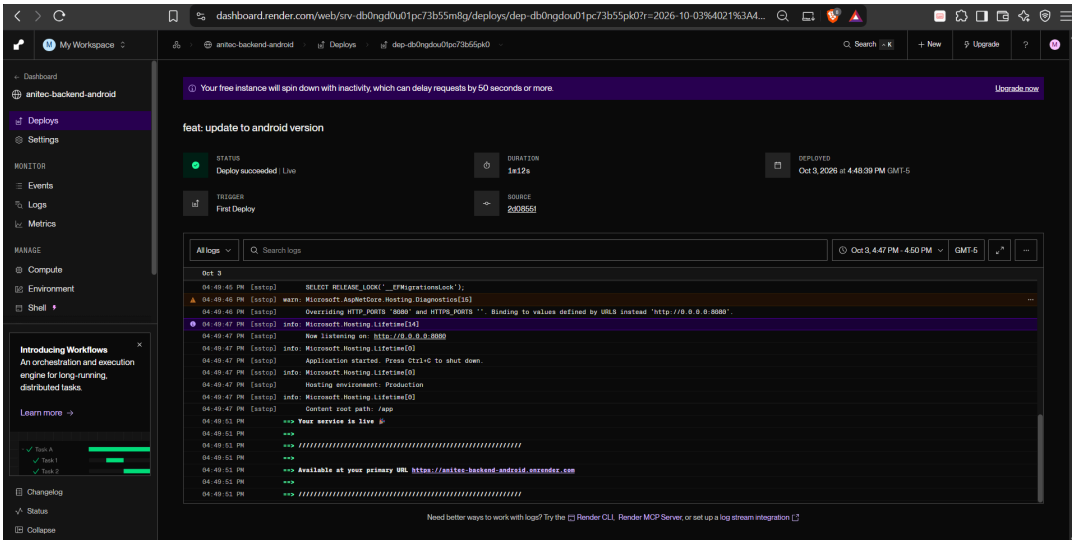


Figura 4.2.1.8.1. Evidencia de despliegue: anitec-backend-android en estado Live en Render. Fuente: elaboración propia.

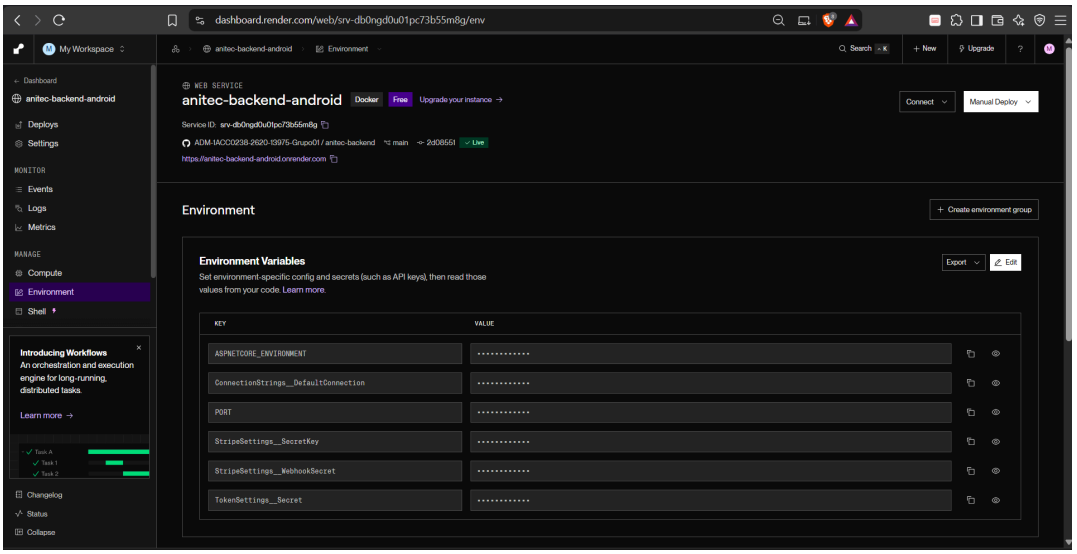


Figura 4.2.1.8.2. Evidencia de configuración de despliegue: variables de entorno del backend con valores ocultos. Fuente: elaboración propia.

Las capturas demuestran la publicación del servicio y la administración de secretos fuera del código fuente.

Pasos ejecutados (aplicaciones móviles):

1. Crear el proyecto «Anitec» en Firebase Console.
2. Registrar la aplicación Android con el nombre de paquete com.anitec.platform y la aplicación Flutter (Figuras 4.2.1.8.5 y 4.2.1.8.6).
3. Generar la compilación firmada de la aplicación Android y subirla a App Distribution. Quedó pendiente y se realizó en el Sprint 2 (sección 4.2.2.8).
4. Crear el grupo de testers (equipo y profesores) e invitarlos. Quedó pendiente y se realizó en el Sprint 2 (sección 4.2.2.8).

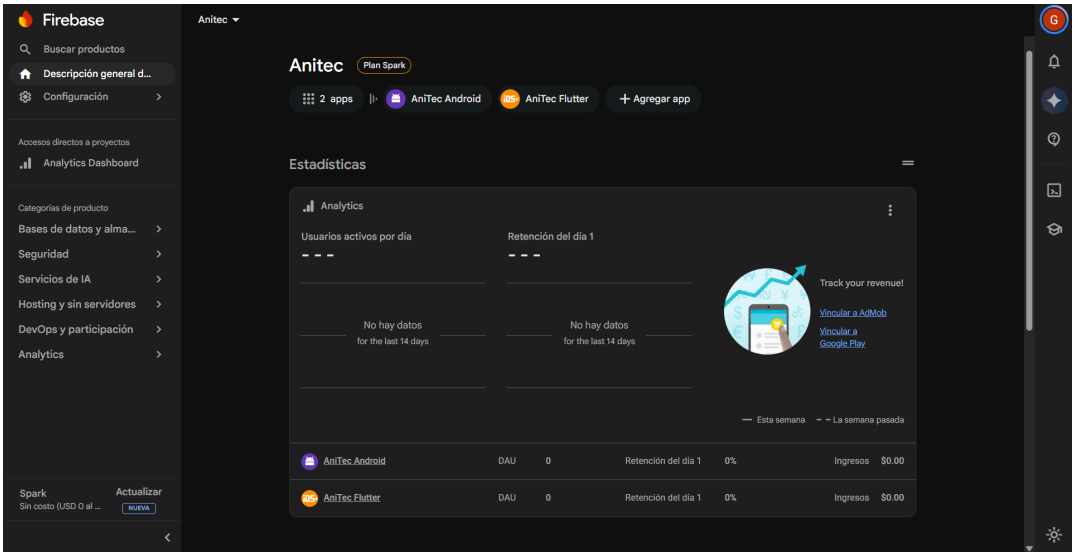


Figura 4.2.1.8.5. Aplicación «AniTec Android» registrada en el proyecto Firebase inicial del Sprint 1 (App ID y nombre de paquete). Fuente: elaboración propia (captura de Firebase).

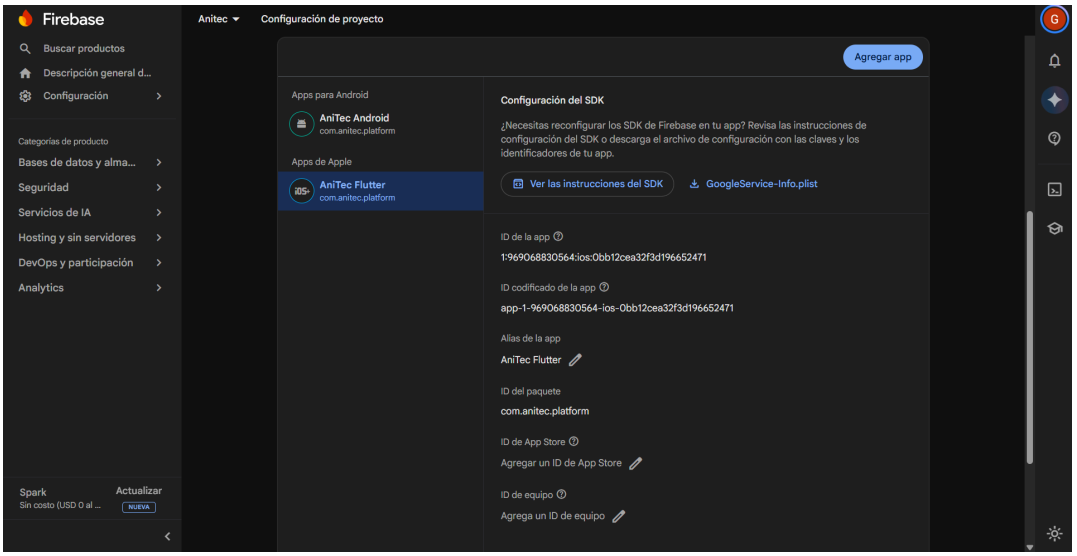


Figura 4.2.1.8.6. Aplicación «AniTec Flutter» registrada como aplicación de Apple en el proyecto Firebase inicial del Sprint 1. Fuente: elaboración propia (captura de Firebase).

Pasos ejecutados (Landing Page):

1. Integrar el contenido aprobado en la rama main.
2. Activar GitHub Pages en **Settings** → **Pages**, con la rama main y la carpeta raíz.
3. Verificar la URL pública (https://adm-1acc0238-2620-13975-grupo01.github.io/anitec-landing-page/). Verificado: carga la versión de la aplicación móvil y sus imágenes responden 200.

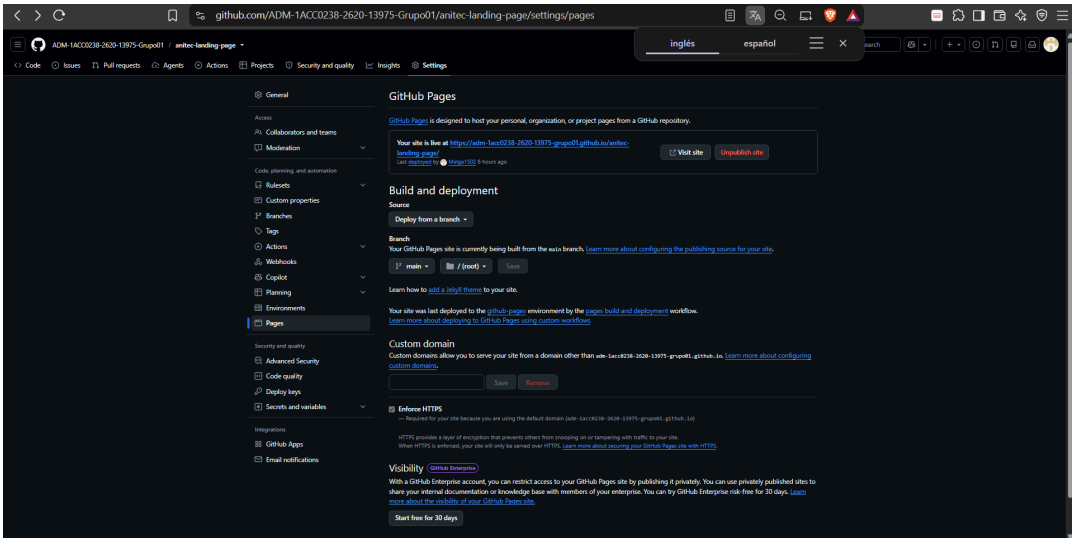


Figura 4.2.1.8.3. Configuración de GitHub Pages del repositorio de la Landing Page. Fuente: elaboración propia.

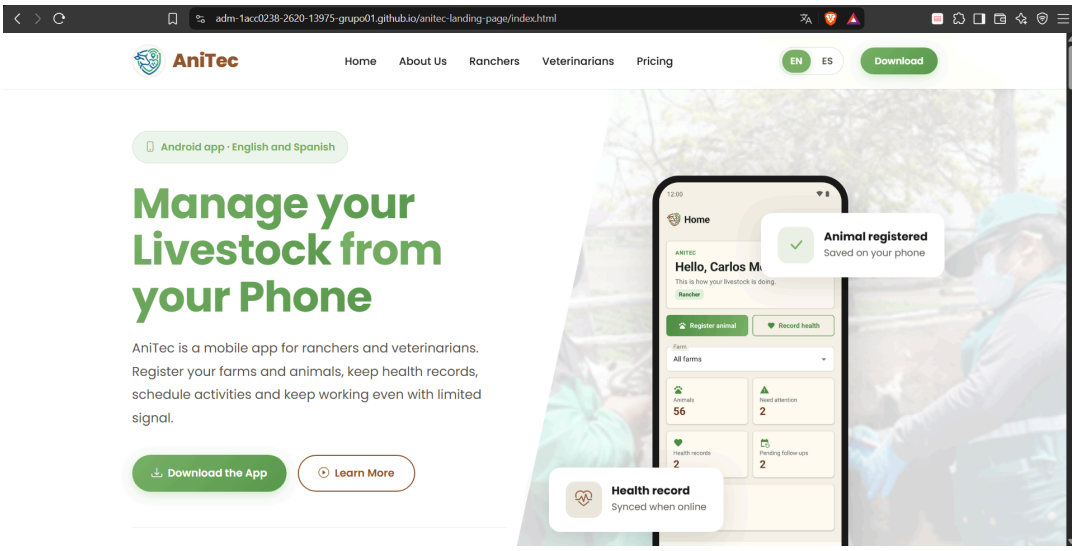


Figura 4.2.1.8.4. Landing Page publicada en la URL de la organización. Fuente: elaboración propia.

4.2.1.9. Team Collaboration Insights during Sprint

Esta sección interpreta la participación del equipo a partir de commits, Pull Requests, revisiones y colaboración por producto. Las métricas se analizan en contexto y no se usan de forma aislada para medir aporte. La tabla registra la contribución de cada integrante a partir de las tareas del Sprint Backlog (sección 4.2.1.3) y de los commits de los repositorios del proyecto, y se complementa con los analíticos de colaboración de GitHub.

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
Sebastian Martin Beingolea Montalvo	Backend, aplicación Android (red y seguridad) y documentación	Implementó el cliente REST de la aplicación, agregó los corrales y las operaciones masivas al backend, documentó los endpoints con OpenAPI, cifró el token de sesión y creó la suite de pruebas automatizadas del backend. Documentó la evidencia del despliegue del backend en Render y de Swagger.	Tareas T-006, T-008, T-009, T-014 y T-028. Informe: d09a7e2	Trabajar el cliente REST, los corrales del backend y las pruebas me mostró que un contrato de API estable permite que la aplicación y el servicio avancen por separado. Aprendí a proteger el token de sesión con Tink y Android Keystore, a documentar con OpenAPI y a comprobar con pruebas unitarias, de integración y BDD que cada endpoint responde lo acordado. Me llevo la importancia de probar también los casos de error, como un rol sin permiso, y no solo el camino feliz.

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
Josep Eliu Melgarejo Quiroz	Aplicación Android e informe	Implementó la caché local con Room, la navegación por rol con el filtrado de datos del usuario, la lista de animales con búsqueda y filtro, el registro individual y masivo de animales y la fotografía del animal. Mantiene la estructura del informe y su generación.	Tareas T-007, T-013, T-017, T-018 y T-025. anitec-android: 2f3b903, 8ade432. anitec-backend: 125d53e, 9831844. anitec-frontend: f12794d. anitec-landing-page: 2a88184. Informe: cd00045	Diseñar la caché con Room, limpiarla al cerrar sesión y filtrar los datos del usuario en la aplicación me enseñó que la seguridad de la información también depende de cómo se guarda y se muestra en el teléfono. El registro masivo y la fotografía me obligaron a cuidar los límites, como de 1 a 500 animales o el tamaño de la imagen, y a mostrar errores que el usuario pueda corregir. Queda pendiente que el backend filtre por usuario para no depender solo de la aplicación.
Saul Ortega Muñoz	Landing Page, aplicación Android (acceso y ficha del animal) y documentación	Diseñó el wireframe y el mock-up móvil de la Landing Page, implementó sus secciones, el diseño responsive y el selector de idioma, las pantallas de registro e inicio de sesión, el mantenimiento de la sesión y la ficha técnica del animal. Documentó las evidencias de desarrollo, ejecución, pruebas y despliegue de la Landing Page.	Tareas T-001, T-002, T-003, T-010, T-011, T-012 y T-020. Informe: e2d2cd6, 9beefe6, bbff341, c150062	Diseñar una Landing Page adaptable y bilingüe, y las pantallas de acceso, me enseñó a cuidar el primer contacto del usuario con mensajes claros, validaciones y formas de recuperarse de un error. Implementar la sesión persistente y su cierre al vencer el token me mostró que la seguridad también forma parte de la experiencia. La ficha técnica me dejó la lección de presentar mucha información de forma legible en pantallas pequeñas.
Luciana Celeste Sanchez Silva	Diseño UX/UI y aplicación Android (fincas, corrales y sanidad)	Elaboró los mock-ups móviles de Android y Flutter, los organizó por aplicación y rol, y construyó las pantallas de fincas, corrales y registros	Tareas T-015, T-016, T-021, T-022, T-023, T-024 y T-029. Informe: da15998,	Llevar los mock-ups de Figma a pantallas reales me mostró que un diseño debe prever los estados vacíos, los errores y las confirmaciones antes de implementarse. Reutilizar los

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
		sanitarios con sus formularios y validaciones. Actualizó el Product Backlog.	cd83263, 5f57a44	mismos componentes en fincas, corrales y sanidad dio coherencia visual y ahorró trabajo. Aprendí que el diseño y el desarrollo deben revisarse juntos para que lo prototipado sea lo que usa el ganadero.
Giuseppe Villanueva Rodriguez	Aplicación Android (base y pruebas), configuración y despliegue móvil	Creó el proyecto Android y organizó el código por bounded contexts y capas, implementó la edición y eliminación de animales y la pantalla de inicio del ganadero, escribió las pruebas unitarias e instrumentadas de Android y registró las aplicaciones en Firebase. Actualizó la configuración del entorno y del despliegue (4.1).	Tareas T-004, T-005, T-019, T-026, T-027 y T-030. Informe: 0cf6d49, 66de467, e01ac6f, a5835de	Crear el proyecto y organizar el código por capas me enseñó que una estructura clara facilita el trabajo de todos y hace posibles las pruebas. Escribir pruebas del dominio, de los casos de uso y de los ViewModels me ayudó a detectar errores antes de añadir nuevas pantallas, y registrar las aplicaciones en Firebase me dio una visión completa del camino hasta la distribución. Me llevo que probar desde el inicio reduce el costo de los cambios.

Los repositorios del proyecto son:

- Informe: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/Informe
- Landing Page: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-landing-page
- Web Services: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend
- Aplicación web: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-frontend
- Android: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android

Las capturas siguientes muestran los analíticos de colaboración y de commits de GitHub (**Insights** → **Contributors** y **Insights** → **Commits**) de cada repositorio.

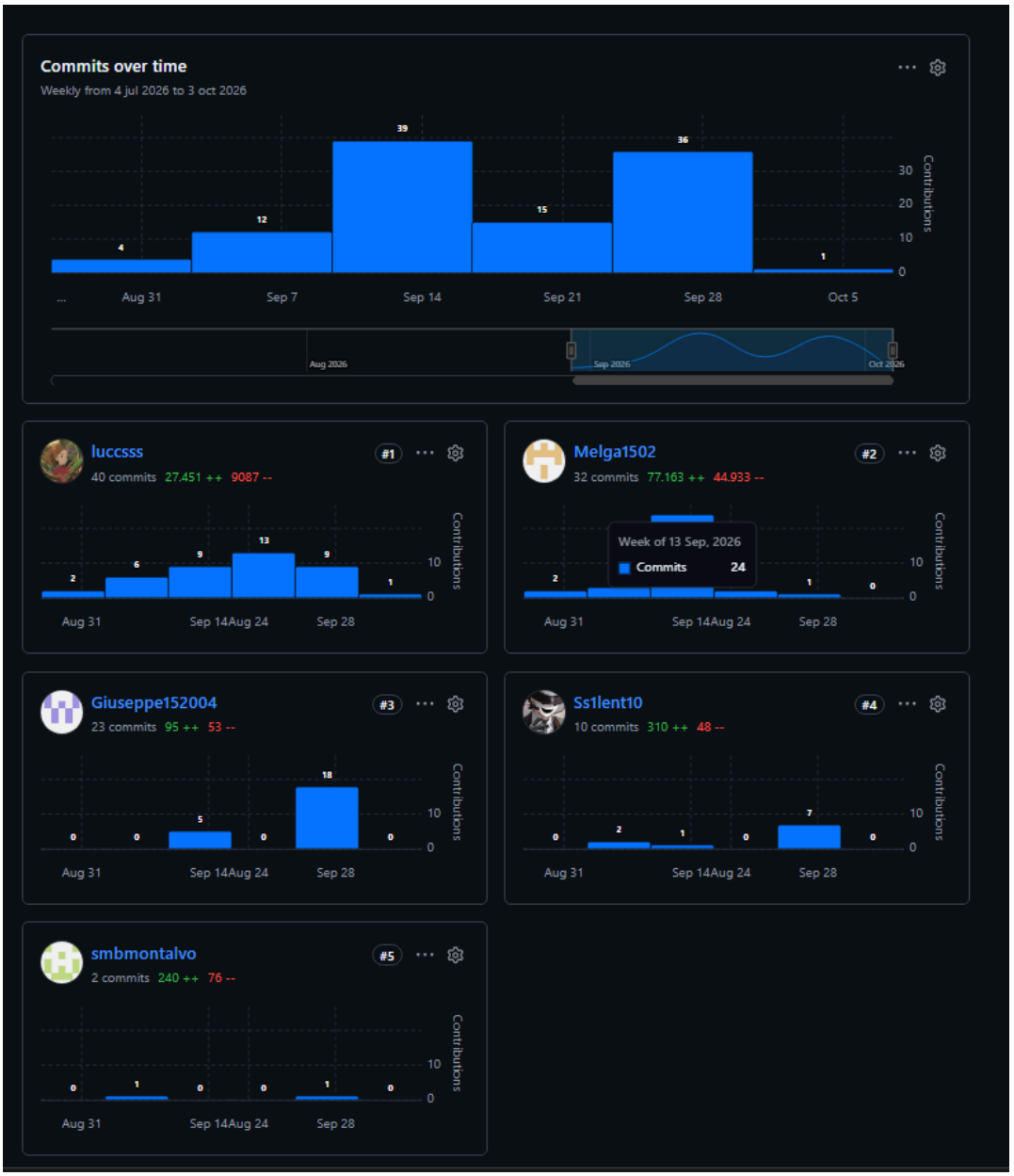


Figura 4.2.1.9.1. Analíticos de colaboración del repositorio Informe. Fuente: elaboración propia.

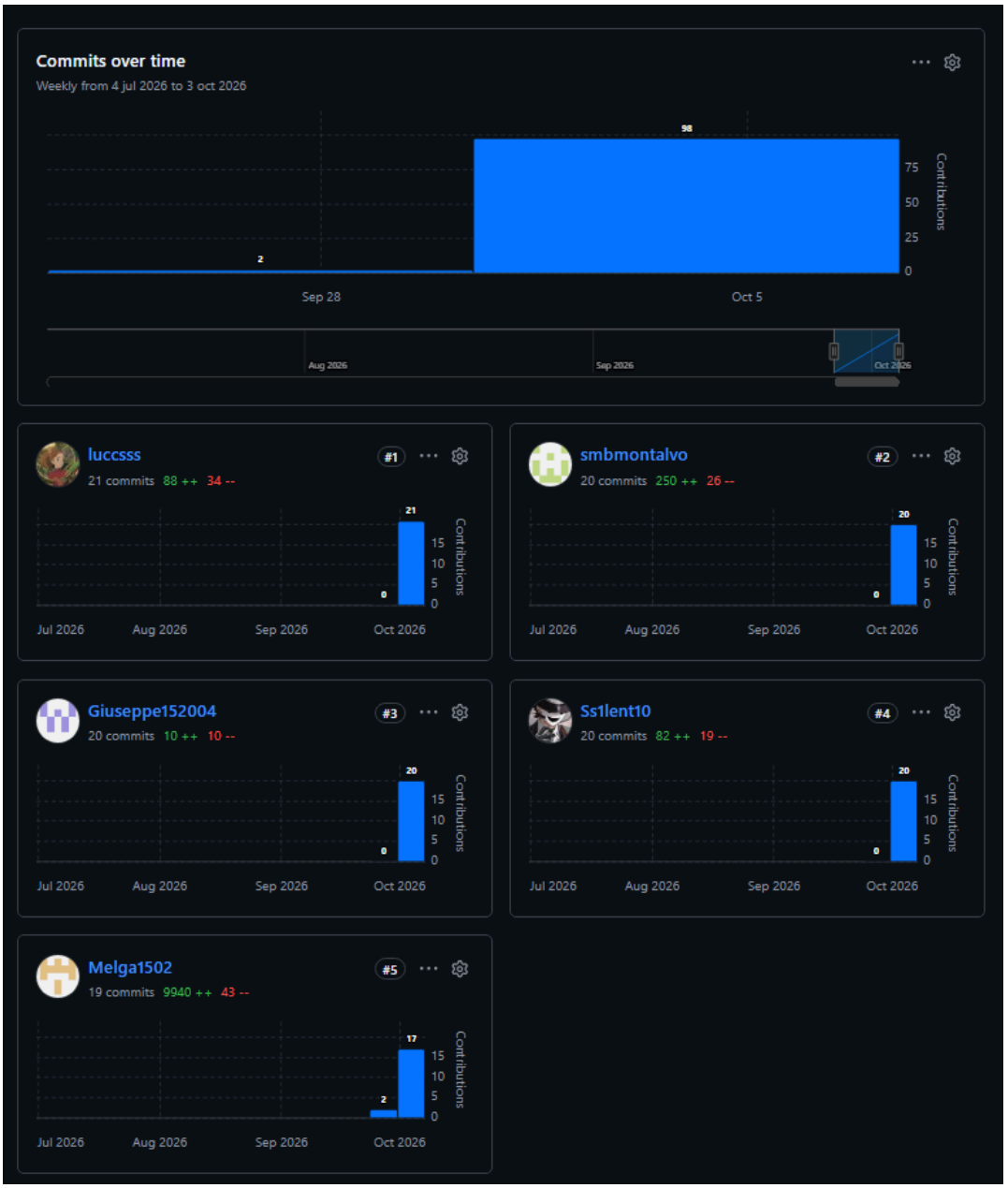


Figura 4.2.1.9.2. Analíticos de colaboración del repositorio anitec-android. Fuente: elaboración propia.

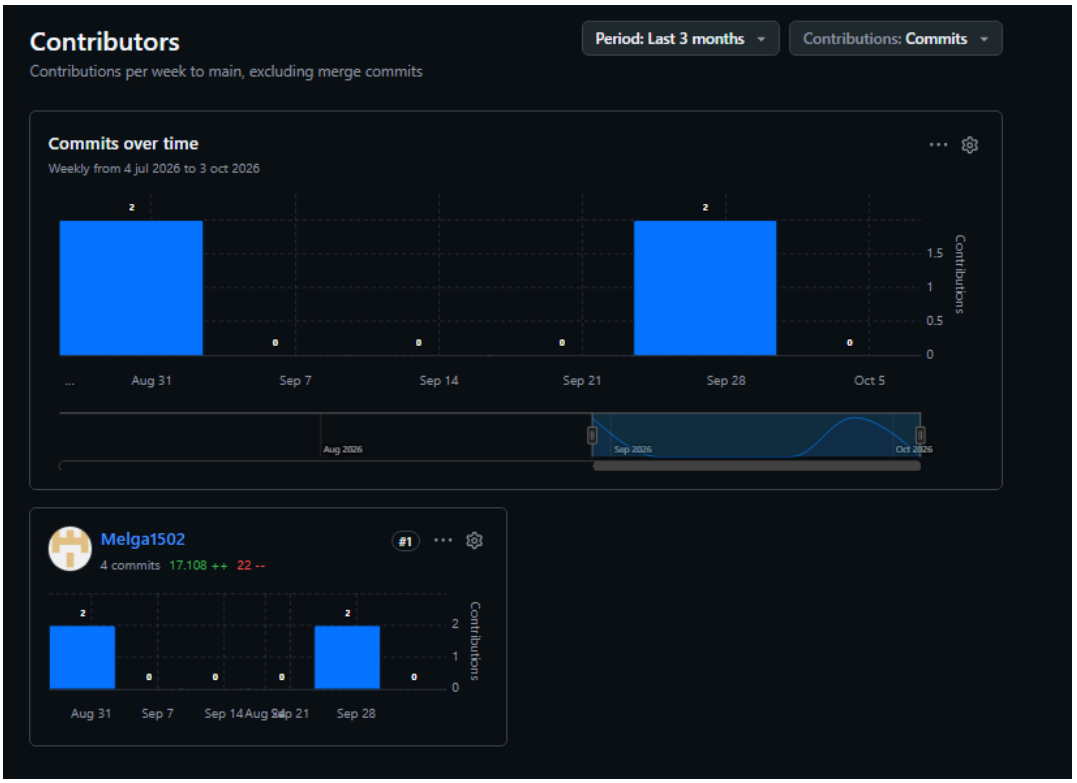


Figura 4.2.1.9.3. Analíticos de colaboración del repositorio anitec-backend. Fuente: elaboración propia.

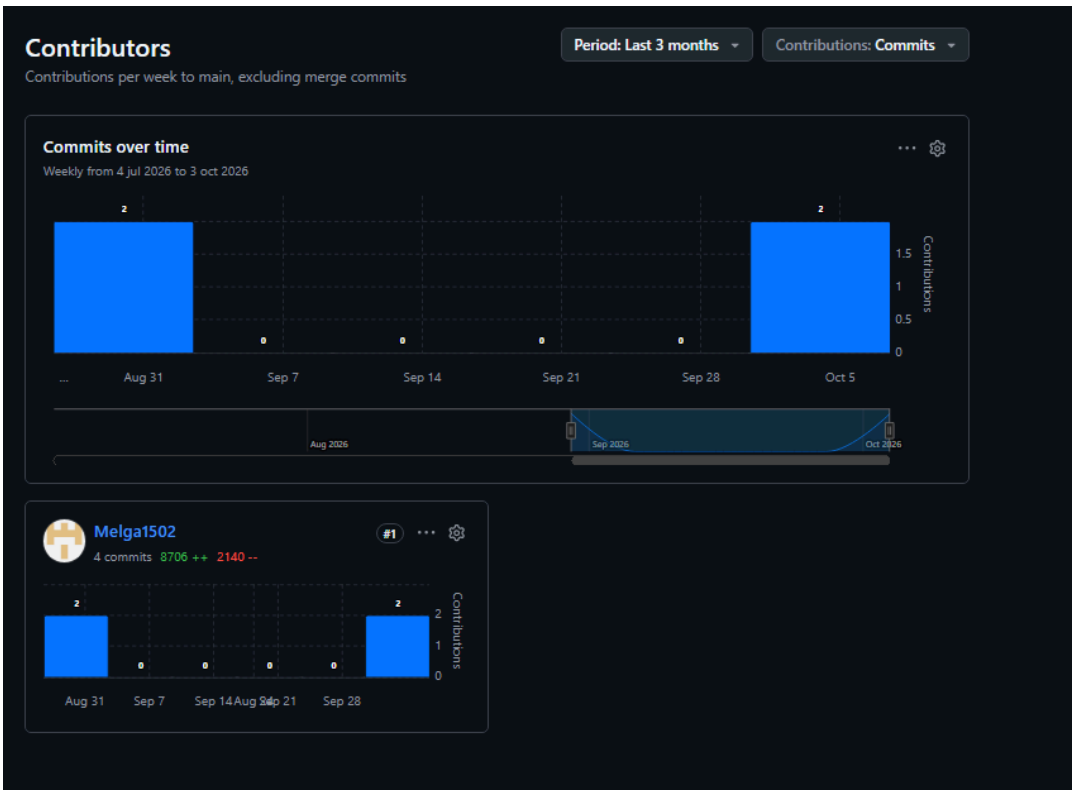


Figura 4.2.1.9.4. Analíticos de colaboración del repositorio anitec-landing-page. Fuente: elaboración propia.

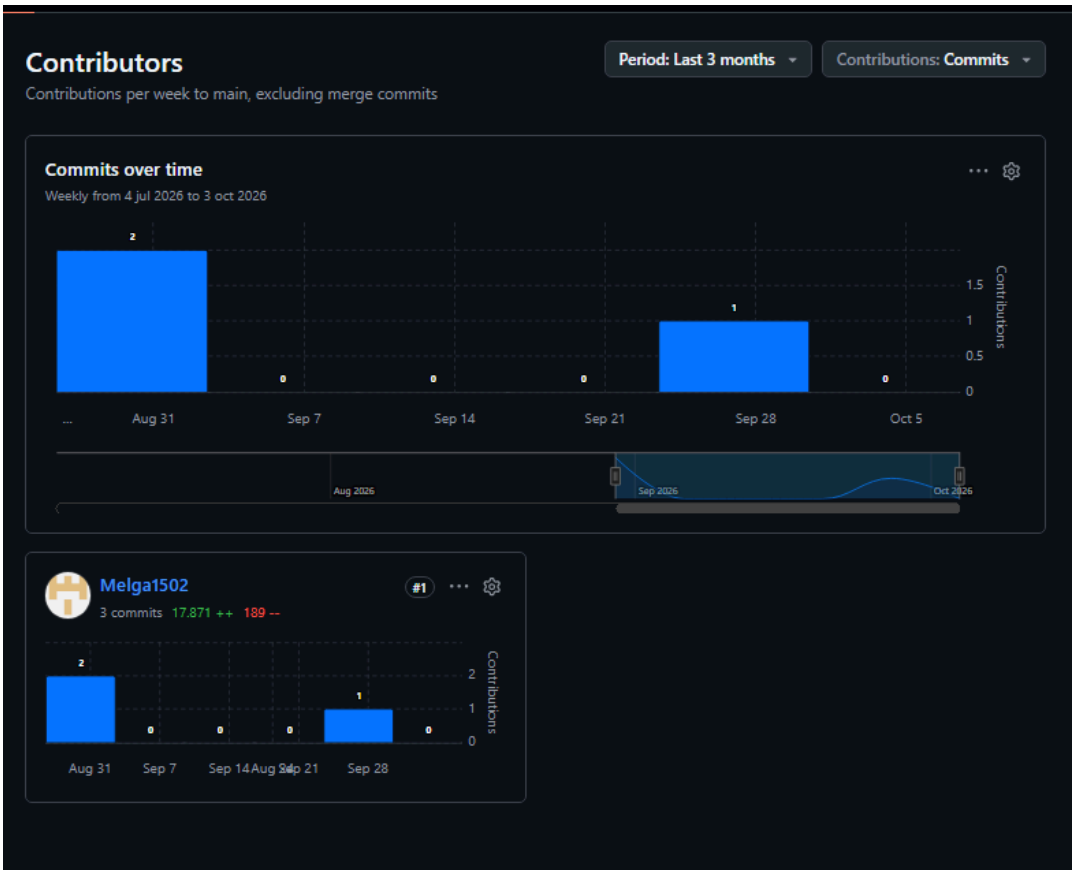


Figura 4.2.1.9.5. Analíticos de colaboración del repositorio anitec-frontend. Fuente: elaboración propia.

Interpretación de los analíticos. Los analíticos de GitHub muestran la actividad de cada repositorio durante el sprint y se leen junto con las tareas del Sprint Backlog (sección 4.2.1.3), porque el número de commits por autor no mide por sí solo el aporte de cada integrante. El trabajo se concentró en la aplicación Android, que reúne 143 de las 205 horas estimadas del sprint (cerca del 70 %) y en la que participaron los cinco integrantes: Josep Melgarejo con 42 horas, Giuseppe Villanueva con 36, Luciana Sanchez con 29, Saul Ortega con 22 y Sebastian Beingolea con 14. El resto se repartió por producto: el backend, con 26 horas a cargo de Sebastian Beingolea (corrales, documentación OpenAPI y pruebas automatizadas); la Landing Page, con 18 horas a cargo de Saul Ortega; los mock-ups móviles, con 12 horas de Luciana Sanchez; y el registro de las aplicaciones en Firebase, con 6 horas de Giuseppe Villanueva. Cada integrante lideró un aspecto y colaboró en al menos otro, como indica la matriz de la sección 4.2.1.2, y el reparto por persona quedó entre 40 y 42 horas.

Conclusión del trabajo en equipo. Al cierre del Sprint 1 el equipo entregó la Landing Page publicada, el backend desplegado y respaldado por 104 pruebas automatizadas, y una aplicación Android que permite registrarse, iniciar sesión y gestionar fincas, corrales, animales y registros sanitarios. De las 30 tareas del sprint, 28 quedaron en Done y 2 en To-Review, y las historias US-007 y US-012 quedaron parciales: falta que el backend filtre la información por usuario y que se implemente el archivado de animales. El trabajo por productos con un líder por aspecto permitió avanzar en paralelo en Android, backend, Landing Page y diseño. Como aprendizajes, el equipo destaca definir el alcance móvil antes de implementar, probar desde el inicio y mantener alineados el informe y el código. Para el Sprint 2, el foco será la vista del veterinario, la colaboración con el ganadero, las actividades, el trabajo sin conexión y la identificación de animales con la cámara.

4.2.2. Sprint 2

El Sprint 2 corresponde al AV2 y completa la base ejecutable del Sprint 1. Su alcance es el publicado en la rama main de los repositorios del backend (https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend), de la aplicación Android (https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android) y de la aplicación Flutter (https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-flutter): el backend completo, que filtra la información por usuario, gestiona las suscripciones con Stripe y archiva animales; la aplicación Android con la vista del veterinario, las actividades, las finanzas, la analítica, los dispositivos IoT, el escáner de códigos QR, el trabajo sin conexión y las suscripciones; una primera versión de la aplicación Flutter con el inicio de sesión y el inicio del ganadero; y la distribución de las dos aplicaciones en Firebase App Distribution. El consentimiento del ganadero sobre el acceso veterinario, las notificaciones, la persistencia local en Flutter y la sincronización idempotente quedan para el Sprint 3.

4.2.2.1. Sprint Planning 2

La planificación del Sprint 2 parte del Product Backlog del AV1, que asignaba al Sprint 2 veintiuna historias (99 Story Points). Al contrastarlas con lo realmente construido, el compromiso se ajustó como en el Sprint 1: siete historias pasaron al Sprint 3 (US-020, US-021, US-022, TS-009, US-027, TS-006 y TS-007) y diez historias del Sprint 3 se adelantaron porque ya estaban construidas (US-035, US-036, US-037, US-038, TS-011, US-039, US-040, US-041, US-043 y TS-015). El Product Backlog de la sección 2.4.3 se actualizó con ese ajuste: el Sprint 2 queda con 24 historias y 114 Story Points.

Sprint #	Sprint 2
	Sprint Planning Background
Date	2026-10-20
Time	06:00 PM
Location	Discord (reunión virtual)
Prepared By	Melgarejo Quiroz, Josep Eliu
Attendees	Beingolea Montalvo, Sebastian Martin / Melgarejo Quiroz, Josep Eliu / Ortega Muñoz, Saul / Sanchez Silva, Luciana Celeste / Villanueva Rodriguez, Giuseppe
Sprint 1 Review Summary	El Sprint 1 entregó la Landing Page publicada, el backend desplegado con 104 pruebas automatizadas y la aplicación Android con autenticación, fincas, corrales, animales y registros sanitarios. De las 30 tareas, 28 quedaron en Done y 2 en To-Review; de las 24 historias, 22 se completaron y 2 quedaron parciales (US-007, porque el backend aún no filtraba por usuario, y US-012, porque faltaba archivar animales), por lo que se completaron 95 de los 105 Story Points comprometidos. La retroalimentación recibida en la revisión del TB1: pendiente de completar.
Sprint 1 Retrospective Summary	Como aciertos, el equipo valoró repartir el trabajo por aspecto con un líder por producto, haber escrito pruebas automatizadas del backend que descubrieron un error de permisos y mantener el informe alineado con el código. Como oportunidades de mejora, identificó cerrar antes las historias parciales (filtrado por usuario en el servidor y archivado), preparar la firma y la distribución de la aplicación desde el inicio del sprint y comprobar en un dispositivo físico lo que el emulador no permite probar, como la cámara y la lectura de códigos QR.

Sprint Goal & User Stories	
Sprint 2 Goal	**Nuestro foco** es completar la aplicación Android para ganaderos y veterinarios, con el backend completo y publicado, y ofrecer una primera versión de la aplicación Flutter. **Creemos que** entrega al ganadero el control de su hato (finanzas, analítica, trabajo sin conexión, identificación de animales con el QR y suscripción) y al veterinario la atención de sus clientes y pacientes, desde aplicaciones que los evaluadores pueden instalar. **Lo confirmaremos cuando** los evaluadores instalen las dos aplicaciones desde Firebase App Distribution con el vínculo de invitación, inicien sesión contra el backend publicado y recorran los flujos del ganadero y del veterinario con los datos de demostración.
Sprint 2 Velocity	114 Story Points, igual a la suma de las historias comprometidas tras contrastar el backlog con lo construido. Como referencia, el Sprint 1 comprometió 105 Story Points y completó 95.
Sum of Story Points	114

La tabla siguiente lista las 24 historias comprometidas, sus Story Points y su estado al cierre del sprint. Se señalan como parciales las historias que no cumplen todos sus criterios de aceptación.

ID	Título	Story Points	Estado al cierre
US-017	Consultar el historial sanitario de un animal	5	Parcial: el veterinario consulta el historial clínico de cada paciente y el ganadero su lista de registros; la copia local no indica cuándo se actualizó.
US-024	Consultar antecedentes de un paciente autorizado	3	Completa: el servidor solo entrega la información de los clientes del veterinario.
US-023	Consultar clientes y pacientes autorizados	5	Completa: clientes, fincas y pacientes del veterinario.
US-026	Gestionar una actividad o recordatorio	5	Parcial: se crean, editan y eliminan actividades; no se programa un recordatorio ni existe el estado cancelada.
US-025	Consultar actividades programadas	3	Completa.
US-037	Registrar y consultar movimientos financieros	5	Completa: ingresos y gastos, también sin conexión.
US-035	Consultar indicadores del hato	5	Completa: pantalla de analítica del ganadero.
US-036	Consultar indicadores sanitarios de clientes	5	Completa: pantalla de analítica del veterinario.
TS-002	Configurar la aplicación multiplataforma con Flutter	5	Completa: proyecto, arquitectura, diseño, idiomas, inicio de sesión e inicio del ganadero.

ID	Título	Story Points	Estado al cierre
US-029	Consultar información esencial sin conexión	5	Parcial: la caché local de Room permite consultar sin conexión; no se muestra la última sincronización.
US-030	Guardar trabajo pendiente sin conexión	5	Completa: animales, registros sanitarios, actividades y movimientos financieros.
US-031	Sincronizar operaciones pendientes	8	Parcial: la cola se envía en segundo plano y conserva solo lo rechazado; el servidor aún no garantiza procesar cada operación una sola vez.
SP-001	Investigar identificación de animales con Google ML Kit	5	Completa: spike resuelto con un prototipo funcional, probado con la cámara de un dispositivo físico.
TS-010	Integrar identificación QR mediante Google ML Kit	5	Completa: integrada con CameraX y ML Kit y probada con la cámara de un dispositivo físico.
US-033	Identificar un animal mediante código QR	5	Completa: escaneo y rechazo de códigos fuera del alcance del usuario, probados con la cámara de un dispositivo físico.
US-034	Identificar un animal sin utilizar la cámara	3	Completa: búsqueda por código o nombre y mensaje si se deniega el permiso.
SP-002	Investigar la integración de Stripe para suscripciones móviles	5	Completa: Checkout en una Custom Tab y retorno por enlace a la aplicación.
US-038	Consultar planes de suscripción	3	Completa.
TS-011	Integrar el checkout externo de Stripe	8	Completa: probada de forma automática y con un pago de prueba real en un dispositivo físico.
US-039	Iniciar un pago mediante un proveedor externo	5	Completa: el pago se inicia en el Checkout de Stripe y se probó con una tarjeta de prueba.
US-040	Consultar el resultado del pago y la suscripción	5	Completa: el resultado, el plan activo y el historial de pagos se comprobaron tras un pago de prueba.
US-041	Cambiar el idioma de la aplicación	3	Completa: English y español (es-419) en Android y en Flutter.
US-043	Comprender errores y estados de conectividad	3	Completa: mensajes por tipo de error y aviso de sincronización pendiente.
TS-015	Configurar compilación y distribución de versiones móviles	5	Completa: compilación firmada con R8 y distribución en Firebase App Distribution.

Además, el sprint cierra dos historias que habían quedado parciales en el Sprint 1. Sus Story Points ya se contaron en ese sprint y no se suman de nuevo.

ID	Título	Estado al cierre del Sprint 2
US-007	Acceder únicamente a información autorizada	Completa: el backend filtra fincas, corrales, animales, registros sanitarios, actividades, finanzas, dispositivos, suscripciones y paneles por usuario, y responde 403 ante recursos ajenos.
US-012	Actualizar o archivar un animal	Completa: el animal puede archivarse y restaurarse (estado «Archivado») de forma individual o múltiple, y la lista activa excluye los archivados.

Las historias US-020, US-021 y US-022 (solicitud, aceptación y revocación del acceso veterinario por parte del ganadero) pasan al Sprint 3: en este sprint el veterinario agrega y retira a sus clientes directamente. Las notificaciones (TS-009 y US-027), la persistencia local en Flutter (TS-006) y la sincronización idempotente (TS-007) también pasan al Sprint 3.

4.2.2.2. Aspect Leaders and Collaborators

La matriz LACX indica un líder (L) y los colaboradores (C) de cada aspecto del Sprint 2: diseño UX/UI, aplicación Android, aplicación Flutter, backend, Landing Page, pruebas, y documentación y despliegue. La matriz se deriva de las tareas asignadas en el Sprint Backlog (sección 4.2.2.3) y de las contribuciones al Informe. A diferencia del Sprint 1, la aplicación Flutter ya tiene responsables porque su primera versión forma parte de este sprint.

Team Member	GitHub Username	UX/UI	Android	Flutter	Backend	Landing Page	Testing	Documentation & Deployment
Beingolea Montalvo, Sebastian Martin	smbmontalvo	—	—	C	L	—	C	C
Melgarejo Quiroz, Josep Eliu	Melga1502	—	L	—	—	—	—	C
Ortega Muñoz, Saul	Ss1lent10	C	C	C	C	L	C	—
Sanchez Silva, Luciana Celeste	luccsss	L	C	L	—	—	—	C
Villanueva Rodriguez, Giuseppe	Giuseppe152004	—	C	—	—	—	L	L

4.2.2.3. Sprint Backlog 2

El Sprint Backlog descompone las historias comprometidas en tareas comprobables y refleja el objetivo del sprint: una aplicación Android completa y una primera versión de Flutter, respaldadas por el backend publicado y distribuidas en

Firebase. El tablero utiliza los estados Todo, In-Process, To-Review y Done. Las tareas se repartieron en partes iguales entre los cinco integrantes, 40 horas estimadas por persona.

- **Sprint Goal:** completar la aplicación Android y ofrecer una primera versión de Flutter, con el backend publicado (ver 4.2.2.1).
- **Board URL:** URL pública del tablero.

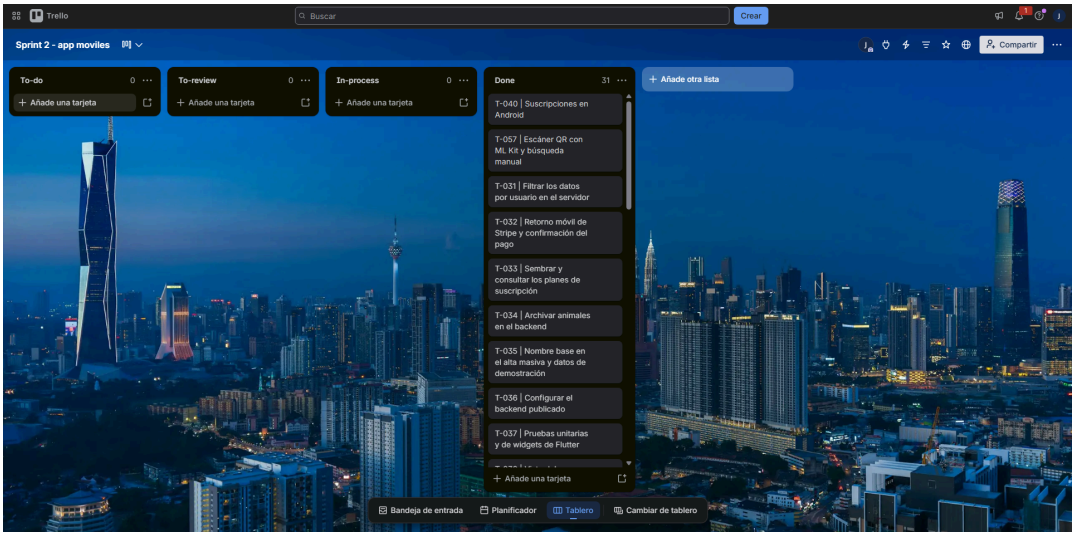


Figura 4.2.2.3.1. Tablero de Trello del Sprint 2. Fuente: elaboración propia.

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
US-007	Acceder únicamente a información autorizada	T-031	Filtrar los datos por usuario en el servidor	Servicio de alcance de acceso y su uso en los controladores de fincas, corrales, animales, registros sanitarios, actividades, finanzas, dispositivos, suscripciones y paneles; 403 ante recursos ajenos.	10	Beingolea Montalvo, Sebastian Martin	Done
TS-011, SP-002	Integrar el checkout externo de Stripe; Investigar Stripe	T-032	Retorno móvil de Stripe y confirmación del pago	Páginas HTTPS de retorno que abren la aplicación por enlace, parámetro de plataforma en el checkout, confirmación del pago y cierre del pago simulado en producción.	8	Beingolea Montalvo, Sebastian Martin	Done
US-038	Consultar planes de suscripción	T-033	Sembrar y consultar los planes de suscripción	Carga de los tres planes en cualquier entorno cuando la tabla está vacía	3	Beingolea Montalvo, Sebastian Martin	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
				y consulta desde la aplicación.			
US-012	Actualizar o archivar un animal	T-034	Archivar animales en el backend	Estado «Archivado», PATCH animals/{id}/status y filtro archived en la lista de animales.	3	Beingolea Montalvo, Sebastian Martin	Done
Sin historia	Mejora de US-011 (registro de animales)	T-035	Nombre base en el alta masiva y datos de demostración	Nombres consecutivos base-001, base-002… por finca y datos de demostración con corrales y animales para tres ganaderos y sus veterinarios.	5	Beingolea Montalvo, Sebastian Martin	Done
TS-015	Configurar compilación y distribución de versiones móviles	T-036	Configurar el backend publicado	Variable SeedDemoData para cargar los datos de demostración, límite de conexiones de la cadena de conexión y redespliegue en Render.	3	Beingolea Montalvo, Sebastian Martin	Done
Sin historia	Tarea técnica sin historia asociada	T-037	Pruebas unitarias y de widgets de Flutter	Pruebas del dominio, el estado, el cliente HTTP y las pantallas de la aplicación Flutter con flutter_test y mocktail.	8	Beingolea Montalvo, Sebastian Martin	Done
US-023, US-024, US-017	Clientes, pacientes y antecedentes del veterinario	T-038	Vista del veterinario	Inicio, clientes (alta y baja), pacientes por finca e historial clínico de cada paciente.	12	Melgarejo Quiroz, Josep Eliu	Done
US-029, US-030, US-031	Trabajo sin conexión	T-039	Trabajo sin conexión y sincronización	Cola de operaciones con WorkManager para animales, registros sanitarios, actividades y movimientos financieros, aviso de sincronización y reintentos.	10	Melgarejo Quiroz, Josep Eliu	Done
US-039, US-040	Iniciar un pago; Consultar el	T-040	Suscripciones en Android	Planes, plan activo, historial de pagos, checkout de Stripe en una	8	Melgarejo Quiroz, Josep Eliu	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
	resultado del pago			Custom Tab, enlace de retorno y confirmación del resultado.			
Sin historia	Tarea técnica sin historia asociada	T-041	Corregir la navegación inferior	Volver a Inicio desde Pacientes o Sanidad tras abrir los pacientes de un cliente, con una prueba instrumentada que reproduce el defecto.	3	Melgarejo Quiroz, Josep Eliu	Done
Sin historia	Tarea técnica sin historia asociada	T-042	Permiso de red local en la compilación de depuración	Solicitud del permiso de red local de Android 17 solo en depuración, para llegar al backend del equipo.	2	Melgarejo Quiroz, Josep Eliu	Done
Sin historia	Documentación	T-043	Documentar el Sprint 2 en el informe	Capítulos 4.1 y 4.2.2, ajuste del Product Backlog, registro de versiones, student outcomes y collaboration insights.	5	Melgarejo Quiroz, Josep Eliu	Done
US-025, US-026	Consultar actividades; Gestionar una actividad	T-044	Actividades	Lista por fecha, formulario de alta y edición y eliminación con confirmación.	8	Sanchez Silva, Luciana Celeste	Done
US-037	Registrar y consultar movimientos financieros	T-045	Finanzas	Lista, resumen de ingresos, gastos y saldo, y formulario de ingresos y gastos.	8	Sanchez Silva, Luciana Celeste	Done
Sin historia	Mejora de US-011 (registro de animales)	T-046	Nombre base en el formulario de lotes	Campo de nombre base opcional con vista previa de los nombres y textos en inglés y español.	3	Sanchez Silva, Luciana Celeste	Done
TS-002	Configurar la aplicación multiplataforma con Flutter	T-047	Inicio de sesión y sesión segura en Flutter	Pantalla de inicio de sesión, mensaje para el rol veterinario, sesión cifrada, cierre de sesión y cierre automático ante un 401.	7	Sanchez Silva, Luciana Celeste	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
US-035	Consultar indicadores del hato	T-048	Inicio del ganadero en Flutter	Saludo, selector de finca, contadores, próximas actividades, registros sanitarios recientes y cambio de idioma.	10	Sanchez Silva, Luciana Celeste	Done
Sin historia	Tarea de documentación sin historia asociada	T-049	Elaborar los videos del AV2	Videos de validación de la aplicación, About-the-Product y About-the-Team.	4	Sanchez Silva, Luciana Celeste	Done
US-035, US-036	Indicadores del hato y de los clientes	T-050	Analítica en Android	Gráficos dibujados con Canvas del hato del ganadero y de los clientes del veterinario.	9	Villanueva Rodriguez, Giuseppe	Done
US-012	Actualizar o archivar un animal	T-051	Archivar animales en Android	Archivado y restauración individual y múltiple con confirmación y filtro de archivados.	5	Villanueva Rodriguez, Giuseppe	Done
TS-015	Configurar compilación y distribución de versiones móviles	T-052	Compilación firmada con R8	Firma desde keystore.properties, R8 activado, versión 0.2.0 y comprobación del APK.	7	Villanueva Rodriguez, Giuseppe	Done
TS-015	Configurar compilación y distribución de versiones móviles	T-053	Distribución en Firebase de las dos aplicaciones	Proyecto «AniTec», registro de las dos aplicaciones, subida de los APK, grupo «Equipo AniTec» y vínculos de invitación.	6	Villanueva Rodriguez, Giuseppe	Done
Sin historia	Tarea técnica sin historia asociada	T-054	Pruebas unitarias e instrumentadas de Android	Pruebas de dominio, casos de uso, repositorios y ViewModels de los contextos nuevos y pruebas de DAO y de navegación.	10	Villanueva Rodriguez, Giuseppe	Done
Sin historia	Tarea de verificación sin historia asociada	T-055	Verificar en un dispositivo físico el escáner QR y el pago de prueba	Lectura de un código QR con la cámara real y un pago de prueba de Stripe con una tarjeta de prueba.	3	Villanueva Rodriguez, Giuseppe	Done

Story ID	Story Title	Task ID	Task Title	Description	Hours	Assigned To	Status
Sin historia	Tarea técnica sin historia asociada	T-056	Dispositivos IoT	Lista y formulario de dispositivos asignados a una finca o a un animal, con su última lectura y el número de lecturas.	7	Ortega Muñoz, Saul	Done
SP-001, TS-010, US-033, US-034	Identificación de animales con ML Kit	T-057	Escáner QR con ML Kit y búsqueda manual	CameraX con ML Kit Barcode Scanning, resolución del código a un animal del usuario, permiso de cámara y búsqueda manual.	10	Ortega Muñoz, Saul	Done
TS-002	Configurar la aplicación multiplataforma con Flutter	T-058	Proyecto Flutter: arquitectura, diseño e idiomas	Proyecto anitec-flutter, carpetas por contexto, tema con los tokens de AniTec, enrutamiento y textos en inglés y español.	9	Ortega Muñoz, Saul	Done
Sin historia	Tarea técnica sin historia asociada	T-059	Pruebas de integración y BDD del backend	Pruebas de alcance por usuario, suscripciones, archivado, nombre de lote y datos de demostración, y escenarios Gherkin ampliados.	8	Ortega Muñoz, Saul	Done
US-041, US-043	Cambiar el idioma; Comprender errores y conectividad	T-060	Idioma y mensajes de error y conectividad	Mensajes de validación específicos, cambio de idioma y textos localizados.	4	Ortega Muñoz, Saul	Done
Sin historia	Tarea de despliegue sin historia asociada	T-061	Actualizar la Landing Page con el vínculo de descarga	Reemplazar el botón de descarga de Android por el vínculo de invitación de Firebase.	2	Ortega Muñoz, Saul	Done

Métrica	Valor
Historias comprometidas	24 (más el cierre de US-007 y US-012)
Story Points comprometidos	114
Tareas	31
Horas estimadas	200

Métrica	Valor
Tareas completadas (Done)	31 de 31

4.2.2.4. Development Evidence for Sprint Review

Esta sección registra los commits que contribuyen al alcance del Sprint 2. Los avances por producto son los siguientes.

- **Backend (anitec-backend).** Un servicio de alcance de acceso filtra por usuario las fincas, los corrales, los animales, los registros sanitarios, las actividades, los movimientos financieros, los dispositivos, las suscripciones y los paneles, y responde 403 ante recursos ajenos. El módulo de suscripciones carga los planes en cualquier entorno, inicia el Checkout de Stripe para Android, devuelve al usuario a la aplicación mediante páginas HTTPS de retorno y confirma el pago; el pago simulado deja de estar disponible en producción. Los animales pueden archivarse y restaurarse (PATCH animals/{id}/status), el alta masiva admite un nombre base y la semilla de demostración incluye tres ganaderos con fincas, corrales y animales, y los veterinarios que los atienden.
- **Aplicación Android (anitec-android).** Se completan la vista del veterinario (clientes, pacientes e historial clínico), las actividades, las finanzas, la analítica del ganadero y de los clientes, los dispositivos IoT, el escáner de códigos QR con CameraX y ML Kit, y la cola de operaciones sin conexión que ahora incluye los movimientos financieros. Se añaden las suscripciones con el Checkout de Stripe en una Custom Tab y el retorno por enlace, el archivado de animales, el nombre base de los lotes, la compilación release firmada con R8 y la corrección de la navegación inferior.
- **Aplicación Flutter (anitec-flutter).** Primera versión con el inicio de sesión del ganadero, el inicio con selector de finca, contadores, próximas actividades y registros sanitarios recientes, cambio de idioma, sesión cifrada y compilación release firmada.
- **Informe.** Documentación del Sprint 2, ajuste del Product Backlog y actualización de la configuración de despliegue.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
anitec-backend	main	642e129	feat: scope data by user, seed plans, mobile Stripe return and archive animals	Servicio de alcance de acceso aplicado a los controladores de fincas, corrales, animales, registros sanitarios, actividades, finanzas, dispositivos, métricas, clientes y paneles, con 403 ante recursos ajenos; carga de los planes de suscripción en cualquier entorno; páginas HTTPS de retorno de Stripe y parámetro platform en el checkout; estado «Archivado» y PATCH animals/{id}/status; pruebas de integración y escenarios BDD nuevos.	2026-10-23
anitec-backend	main	0c4f7cc	feat: name animals in a batch and	Nombre base opcional en el alta masiva con numeración consecutiva por finca; semilla de	2026-10-24

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
			seed three ranchers with corrals and many animals	demostración con tres ganaderos con fincas, corrales y animales, y los veterinarios que los atienden; pruebas SeedDataTests y de alta masiva.	
anitec-backend	main	309c3b4	feat: let a hosted environment load the demo data with SeedDemoData	La variable SeedDemoData permite cargar los datos de demostración en un entorno publicado cuando la tabla de usuarios está vacía; es el commit que Render muestra como desplegado.	2026-10-25
anitec-backend	main	88265b2	docs: explain how to set the Stripe test key for local payments	Sección del README con el comando para definir StripeSettings__SecretKey solo en la terminal y probar los pagos de prueba con Stripe.	2026-11-02
anitec-android	main	959defc	feat: AV2 - activities, finance, IoT, analytics, QR scanner and offline outbox	Vista del veterinario (clientes, pacientes e historial), actividades, finanzas, dispositivos IoT, analítica con gráficos propios, escáner QR con CameraX y ML Kit y cola de operaciones sin conexión con WorkManager; esquemas de Room de la versión 3 a la 7 y pruebas unitarias de cada contexto.	2026-10-02
anitec-android	main	b5bbb72	Merge branch 'feature/av2' into main	Integra en main el alcance del AV2 (vista del veterinario, actividades, finanzas, dispositivos, analítica, escáner y cola sin conexión) y resuelve los conflictos de AniTecApplication, AuthRepository y AndroidManifest.	2026-10-23
anitec-android	main	000a6f4	feat: improve validation errors, accessibility and tablet layout	Mensajes de validación específicos, semántica de accesibilidad en los componentes y las listas, disposición con barra lateral para pantallas anchas, historial de lecturas por dispositivo IoT y pruebas.	2026-10-23
anitec-android	main	5553a38	feat: add Stripe subscriptions, archive animals, offline finance records and release build setup	Contexto subscriptions con planes, plan activo, historial de pagos, checkout en Custom Tab y enlace de retorno anitec://subscriptions; archivado y restauración de animales con filtro; movimientos financieros en la cola sin	2026-10-23

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
				conexión; compilación release con firma desde keystore.properties, R8 y versión 0.2.0; pruebas.	
anitec-android	main	7d34023	feat: ask for local network access when the debug build opens	Solicitud del permiso de red local de Android 17 solo en la compilación de depuración, con su prueba y la nota en el README.	2026-10-24
anitec-android	main	85a687f	feat: name the animals of a batch with a base name and number	Campo de nombre base opcional en el formulario de lotes con vista previa de los nombres, textos en inglés y español y BatchNamingTest.	2026-10-24
anitec-android	main	2e63643	fix: return to Home from the bottom bar after opening a client's patients	Corrige la navegación inferior para volver a Inicio desde Pacientes o Sanidad, con BottomNavigationTest (prueba instrumentada) que reproduce el defecto.	2026-10-24
anitec-android	main	1c2e451	docs: add release build and Firebase App Distribution steps	Pasos del README para crear el keystore, compilar el APK release, registrar la aplicación en Firebase, subirla y enviarla a los testers.	2026-10-25
anitec-android	main	3a404f4	docs: explain how to switch between the local and the hosted backend	Explica cómo elegir la variante debug (backend local) o release (backend publicado) y los avisos de la firma y del paquete compartido.	2026-11-02
anitec-android	main	3716e53	fix: always send the platform in the Stripe checkout request	El campo platform del checkout de Stripe deja de omitirse al serializar, de modo que el pago vuelve a la aplicación; incluye una prueba que comprueba el cuerpo serializado.	2026-11-02
anitec-flutter	main	bc56f7e	feat: add rancher dashboard with sign-in and language switch	Proyecto Flutter com.anitec.platform.flutter con carpetas por contexto, tema con los tokens de AniTec, enrutamiento, inicio de sesión del ganadero con sesión cifrada, inicio con selector de finca, contadores, próximas actividades y registros recientes, cambio de idioma inglés y español, y pruebas unitarias y de widgets.	2026-10-23
anitec-flutter	main	721fe36	fix: leave archived animals out of the	Los animales archivados no cuentan en el inicio y una respuesta 401 tardía ya no cierra	2026-10-23

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
			dashboard and keep a new session on a late 401	la sesión nueva; incluye pruebas de ambos casos.	
anitec-flutter	main	e59fe06	feat: sign the release build with a keystore and document the release APK	Firma de la compilación release desde android/key.properties (con ejemplo) y pasos del README para generar el APK.	2026-10-25
anitec-flutter	main	bf89e95	fix: show the farm selector choice in the language of the app	El selector de finca se reconstruye al cambiar el idioma, con la comprobación en la prueba del flujo de la aplicación.	2026-10-29
anitec-flutter	main	b71a597	docs: explain how to switch between the local and the hosted backend	Explica que flutter run usa el backend local y flutter run --release el publicado, y los avisos de la firma y del paquete.	2026-11-02
Informe	main	eea6574	docs: document sprint 2 and update deployment configuration	Primera versión de la documentación del Sprint 2, ajuste del Product Backlog y actualización de la configuración de despliegue.	2026-10-25
Informe	main	36d8c8c	docs: add Swagger screenshots of the Sprint 2 endpoints	Capturas de Swagger con los endpoints nuevos y la respuesta 403 ante un recurso ajeno.	2026-11-01
Informe	main	af41f37	docs: add Sprint 2 phone captures and update the AV2 student outcomes	Capturas del pago con Stripe, del escáner y de la distribución en Firebase, y student outcomes del AV2 según las tareas del sprint.	2026-11-02

4.2.2.5. Testing Suite Evidence for Sprint Review

La suite de pruebas del backend (Anitec.Platform.Tests, https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend/tree/main/Anitec.Platform.Tests) crece de 104 a 150 pruebas. Además, la aplicación Android cuenta con 214 pruebas unitarias y 16 instrumentadas, y la aplicación Flutter con 88 pruebas unitarias y de widgets. Las pruebas del backend siguen el mismo esquema que en el Sprint 1 (sección 4.2.1.5): una API en memoria con SQLite y el entorno Testing.

Tipo	Herramientas	Carpeta del proyecto	Sprint 1	Sprint 2
Unit Tests	xUnit y NSubstitute	Unit/	52	52
Integration Tests	xUnit y WebApplicationFactory	Integration/	32	74
Acceptance Tests (BDD)	Reqnroll sobre xUnit, archivos .feature en Gherkin	Bdd/Features/ y Bdd/Steps/	20	24
Total			**104**	**150**

Unit Tests. Las 52 pruebas unitarias (UserCommandService, TokenService, HashingService, AnimalCommandService y CorralCommandService) no cambiaron respecto al Sprint 1 y se describen en la sección 4.2.1.5; el servicio de animales mantiene cubierta el alta masiva, ahora con nombre base.

Integration Tests. Cada prueba llama a los endpoints REST de la API en memoria. Las clases nuevas del sprint verifican que cada usuario vea y modifique solo lo suyo, la gestión de suscripciones y la semilla de demostración.

Test ID	Clase de pruebas	Endpoints y comportamientos verificados	Historias relacionadas	Pruebas	Resultado
TEST-BE-I01	AuthenticationApiTests	Registro, inicio de sesión y 401 sin token (sin cambios).	US-004, US-005	11	Completado
TEST-BE-I02	LivestockApiTests	Fincas, corrales y animales (sin cambios) y, nuevas: alta masiva con nombre base y continuación de la numeración, alta masiva sin nombre base, archivar y restaurar un animal con su efecto en la lista activa, validación del cambio de estado y permisos por rol del veterinario.	US-011, US-012, US-007	22	Completado
TEST-BE-I03	HealthEventsApiTests	POST, GET y DELETE de registros sanitarios, y un veterinario que no puede registrar un evento sobre el animal de un ganadero que no es su cliente.	US-014, US-015, US-016, US-024	6	Completado
TEST-BE-I04	AccessScopeApiTests	Fincas, corrales, animales, registros sanitarios, actividades, movimientos financieros, dispositivos y paneles ocultos para otros ganaderos; 403 al crear, modificar o eliminar recursos ajenos, incluidas las operaciones masivas; el veterinario ve solo las fincas de sus clientes, programa	US-007, US-023, US-024, US-025, US-026, US-037, TS-013	23	Completado

Test ID	Clase de pruebas	Endpoints y comportamientos verificados	Historias relacionadas	Pruebas	Resultado
		actividades para ellos y no gestiona los clientes de otro veterinario; un dispositivo requiere una finca o un animal visibles para el usuario.			
TEST-BE-I05	SubscriptionsApiTests	Los planes se cargan solo cuando no existen; una suscripción, la suscripción activa y los pagos solo son visibles para su dueño; un ganadero no puede suscribir a otra persona; las páginas de retorno de Stripe abren la aplicación con la sesión de pago, descartan un identificador con caracteres inesperados y manejan la cancelación.	US-038, US-040, TS-011	7	Completado
TEST-BE-I06	SeedDataTests	Todo animal de la semilla pertenece a un corral de su finca, un ganadero nunca tiene dos animales con el mismo nombre, tres ganaderos tienen varias fincas, corrales y animales, la veterinaria tiene a los tres como clientes y la semilla incluye animales archivados, aves numeradas y seguimientos próximos.	Datos de demostración	5	Completado

Acceptance Tests (BDD). Los escenarios siguen escritos en Gherkin en cuatro archivos .feature dentro de Anitec.Platform.Tests/Bdd/Features, con sus pasos en C#. En este sprint se amplían los escenarios de sanidad y de permisos, que ahora distinguen a los clientes del veterinario.

Archivo .feature	Escenarios	Historias relacionadas
Authentication.feature	7	US-004, US-005, US-007
AnimalManagement.feature	7	US-010, US-011, US-012, US-044, TS-013
HealthRecords.feature	4	US-014, US-015, US-016, US-024
RolePermissions.feature	6	US-007, US-023, US-024

Los archivos Authentication.feature y AnimalManagement.feature no cambiaron y su código está en la sección 4.2.1.5. Código de HealthRecords.feature:

```
Feature: Health records
  As a rancher or a veterinarian
  I want to keep the health history of each animal
  So that incidents, vaccines and treatments are never forgotten

  Background:
```

```
    Given a rancher has a farm called "La Esperanza"
    And the farm has a corral called "Corral A"
    And the corral "Corral A" has the animal "COW-001"

  Scenario: A rancher records a health incident
    Given I am signed in as a "Rancher"
    When I record a "Incidencia" health event for the animal "COW-001" with the
description "Cojera en la pata trasera"
    Then the health event is created
    And the health record of the animal "COW-001" includes the description "Cojera en la
pata trasera"

  Scenario: A veterinarian records a treatment for a client
    Given I am signed in as a "Veterinarian"
    And I am a client of the rancher
    When I record a "Tratamiento" health event for the animal "COW-001" with the
description "Antibiótico por 5 días"
    Then the health event is created
    And the health record of the animal "COW-001" includes the description "Antibiótico
por 5 días"

  Scenario: A veterinarian cannot record a treatment for a rancher who is not their client
    Given I am signed in as a "Veterinarian"
    When I record a "Tratamiento" health event for the animal "COW-001" with the
description "Antibiótico por 5 días"
    Then the request is rejected with status 403

  Scenario: Health records require a session
    Given I am a new visitor
    When I request the health records
    Then the request is rejected with status 401
```

Código de RolePermissions.feature:

```
Feature: Role permissions
  As the owner of the information
  I want each role to do only what it is meant to do
  So that a veterinarian can follow animals without changing the herd

  Background:
    Given a rancher has a farm called "La Esperanza"
    And the farm has a corral called "Corral A"
    And the corral "Corral A" has the animal "COW-001"

  Scenario: A veterinarian can see the animals of their clients
    Given I am signed in as a "Veterinarian"
    And I am a client of the rancher
    When I request the list of animals
    Then the request succeeds
    And the animal "COW-001" appears in the animal list

  Scenario: A veterinarian does not see the animals of ranchers who are not their clients
    Given I am signed in as a "Veterinarian"
    When I request the list of animals
    Then the request succeeds
    And the animal "COW-001" no longer appears in the animal list

  Scenario: A rancher does not see the animals of another rancher
    Given I am signed in as another "Rancher"
    When I request the list of animals
    Then the request succeeds
    And the animal "COW-001" no longer appears in the animal list

  Scenario: A rancher cannot delete the animals of another rancher
    Given I am signed in as another "Rancher"
    When I delete the animal "COW-001"
    Then the request is rejected with status 403

  Scenario: A veterinarian cannot register animals
    Given I am signed in as a "Veterinarian"
    When I register an animal with the code "COW-009" in "Corral A"
    Then the request is rejected with status 403

  Scenario: A veterinarian cannot delete animals
    Given I am signed in as a "Veterinarian"
```

```
    When I delete the animal "COW-001"
    Then the request is rejected with status 403
```

Pruebas de la aplicación Android. Las 214 pruebas unitarias se ejecutan con JUnit 4, MockK, Turbine, Coroutines Test y MockWebServer, y se agrupan por bounded context.

Test ID	Contexto	Clases de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resultado
TEST-AN-U01	iam	AuthViewModelsTest, EmailRegistrationTest, EmailValidatorTest, SignUpRequestTest, SignUpResponseTest	Estado de inicio de sesión y registro, validación del correo, registro con correo opcional y lectura de las respuestas del servidor.	US-004, US-005	24	Completado
TEST-AN-U02	core	SafeApiCallTest, AuthInterceptorTest, ErrorMessagesTest, StringResourcesTest, LocalNetworkAccessTest	Mapeo de errores HTTP, cierre de sesión ante un 401 solo si el token sigue vigente, mensajes de validación específicos, textos en los dos idiomas y solicitud del permiso de red local en depuración.	US-043, US-041	23	Completado
TEST-AN-U03	livestock	LivestockUseCasesTest, LivestockRepositoryTest, LivestockScopeTest, AnimalSearchTest, AnimalPlacementPolicyTest, AnimalStatusTest, ArchivedStatusTest, ArchiveAnimalRepositoryTest, ArchiveAnimalsUseCasesTest, BatchNamingTest, MediaUrlTest	Alcance de los datos del usuario, búsqueda, reglas de ubicación en corrales, estados del animal, archivado y restauración individual y múltiple, nombres base-001 del lote y rutas de las fotos.	US-010, US-011, US-012, US-045	50	Completado
TEST-AN-U04	sanitary	SanitaryRepositoryTest, SanitaryScopeTest	Lectura y escritura de registros sanitarios y alcance por usuario.	US-014, US-015, US-016	6	Completado
TEST-AN-	veterinary	VeterinaryRepositoryTest, VeterinaryHelpersTest,	Clientes del veterinario (alta y	US-017, US-023, US-024	12	Completado

Test ID	Contexto	Clases de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resultado
U05		VetHomeViewModelTest	baja tras la confirmación del servidor, conservación de la caché ante un fallo), ganaderos disponibles, resumen de los animales y registros de cada cliente y totales del inicio de todos los clientes o del cliente elegido.			
TEST-AN-U06	activities	ActivityRulesTest, ActivitiesRepositoryTest	Actividades próximas primero, alcance por rol (el ganadero ve lo suyo; el veterinario lo que creó y lo de sus clientes), propietario al crear y eliminación solo cuando el servidor lo confirma.	US-025, US-026	9	Completado
TEST-AN-U07	financial	FinancialRulesTest, FinancialRepositoryTest	Totales de ingresos, gastos y saldo sin errores de centavos, montos positivos con dos decimales y coma decimal, formato en soles y registro sin conexión con identificador negativo.	US-037	14	Completado
TEST-AN-U08	devices	DeviceRulesTest, DevicesRepositoryTest	Solo dispositivos de fincas o animales visibles, última lectura y conteo por dispositivo, historial de lecturas más reciente primero y	Sin historia (dispositivos IoT)	13	Completado

Test ID	Contexto	Clases de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resultado
			dispositivo sin lecturas.			
TEST-AN-U09	analytics	AnalyticsCalculatorTest	Animales por estado de salud, exclusión de fincas no visibles, registros por tipo y por finca, caché vacía y resumen en palabras de los gráficos para lectores de pantalla.	US-035, US-036	6	Completado
TEST-AN-U10	scanner	ScannerViewModelTest, TagResolverTest	Código exacto o dentro de un enlace que encuentra al animal sin distinguir mayúsculas, código desconocido, resultado que ignora nuevas lecturas, nuevo escaneo y código manual vacío.	US-033, US-034	8	Completado
TEST-AN-U11	sync	OutboxProcessorTest, OfflineCreationTest	Envío de la cola de animales, registros sanitarios, actividades y movimientos financieros, registros que siguen a su animal, reintento sin conexión, animal no duplicado tras perder una respuesta, operación rechazada marcada como fallida sin detener las siguientes, sesión vencida, reintento y descarte.	US-029, US-030, US-031	14	Completado

Test ID	Contexto	Clases de pruebas	Comportamientos verificados	Historias relacionadas	Pruebas	Resultado
TEST-AN-U12	subscriptions	SubscriptionsRepositoryTest, SubscriptionsViewModelTest, CheckoutReturnParserTest, CheckoutReturnsTest, PlanAudienceTest	Planes según el rol y solo los activos, plan activo y pagos, dirección de pago que debe ser HTTPS, mensajes cuando no se puede iniciar el pago, lectura de los enlaces de retorno anitec://subscriptions y confirmación del resultado, incluido un pago que Stripe no completó.	US-038, US-039, US-040, TS-011	28	Completado
TEST-AN-U13	app	HomeViewModelTest, NavigationLayoutTest	Contadores del inicio de todas las fincas o de la finca elegida, máximo de cuatro registros recientes, barra inferior en teléfonos y riel de navegación en pantallas anchas.	US-007, US-035	7	Completado

Las 16 pruebas instrumentadas se ejecutan en el emulador (Medium_Phone_API_37.0, Android 17):

Test ID	Clase de pruebas	Comportamientos verificados	Pruebas	Resultado
TEST-AN-I01	RoomDaoTest	Operaciones de la base de datos local por contexto y limpieza de la caché.	7	Completado
TEST-AN-I02	AnimalImageUploaderTest	Reducción y subida de la fotografía del animal.	5	Completado
TEST-AN-I03	BottomNavigationTest	La navegación inferior vuelve a Inicio desde Pacientes o Sanidad aunque se hayan abierto los pacientes de un cliente, permite repetir el recorrido y cambia entre pestañas en cualquier orden.	4	Completado

Pruebas de la aplicación Flutter. Las 88 pruebas usan flutter_test y mocktail.

Test ID	Archivos de pruebas	Comportamientos verificados	Pruebas	Resultado
TEST-FL-001	iam/sign_in_controller_test.dart, core/session_controller_test.dart, app/router_test.dart	Campos vacíos, usuario sin espacios, inicio de sesión del ganadero, mensaje para el veterinario sin conservar la sesión, credenciales incorrectas, sesión guardada y restaurada, cierre de sesión, cierre ante un 401 y redirección según la sesión.	17	Completado
TEST-FL-002	core/api_client_test.dart, core/language_controller_test.dart, data/repositories_test.dart	Cabecera Bearer, manejo de 401 que no cierra una sesión nueva por una respuesta tardía, lectura de los recursos del backend y cambio de idioma.	21	Completado
TEST-FL-003	livestock/livestock_domain_test.dart, livestock/archived_animals_test.dart, activities/activity_test.dart, app/home_summary_test.dart	Reglas del dominio, exclusión de los animales archivados y cálculo de los contadores, las próximas actividades y los registros recientes.	24	Completado
TEST-FL-004	app/home_controller_test.dart, app/home_screen_test.dart, app/app_flow_test.dart	Estado del inicio, pantalla de inicio con sus contadores y recorrido completo desde el inicio de sesión hasta el inicio del ganadero, con estados de carga, vacío y error.	26	Completado

Resultado de la ejecución. Las 150 pruebas del backend, las 214 pruebas unitarias de Android, las 16 instrumentadas y las 88 de Flutter pasan sin errores. Las pruebas ayudaron a detectar defectos durante el sprint. La prueba BottomNavigationTest reprodujo el defecto de la navegación inferior reportado al usar la aplicación: antes de la corrección fallaban 3 de sus 4 casos, porque la barra restauraba la pila guardada con los pacientes de un cliente en lugar de volver a Inicio; la corrección, que vuelve directamente a Inicio, hizo que pasaran los 4. Al verificar la pantalla de suscripciones en el emulador apareció un cierre inesperado por claves duplicadas en una lista (un plan y un pago con el mismo identificador), que se corrigió con claves distintas por tipo de elemento. El filtrado por usuario del backend quedó cubierto por las 23 pruebas de AccessScopeApiTests.

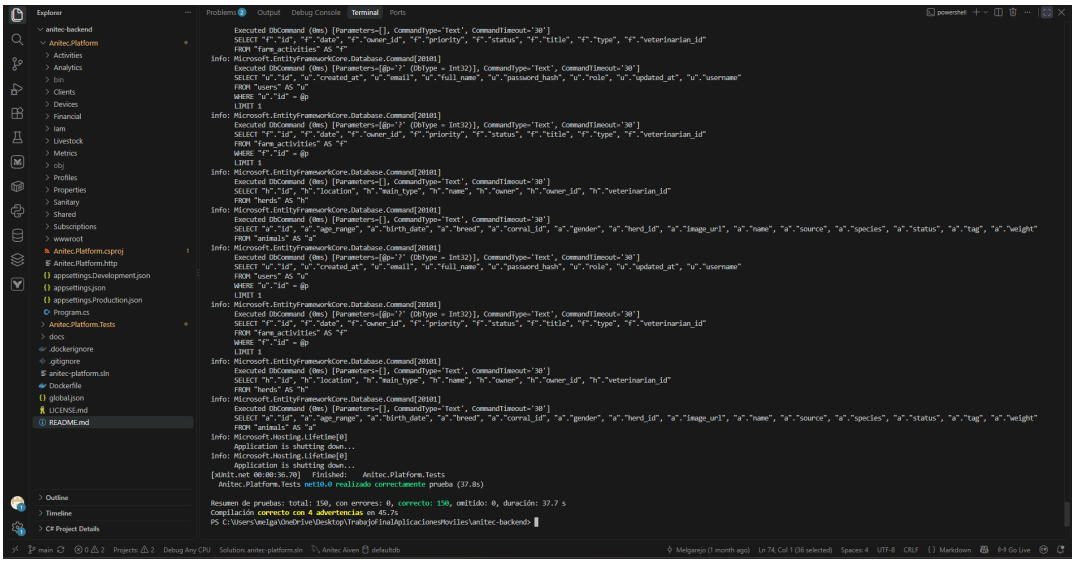


Figura 4.2.2.5.1. Captura de las 150 pruebas exitosas del backend. Fuente: elaboración propia.

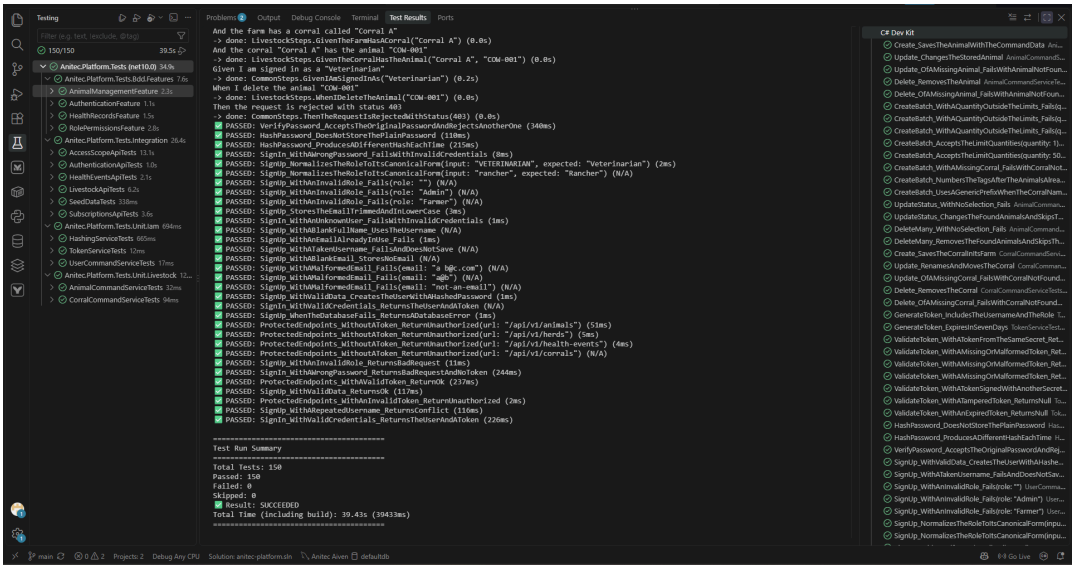
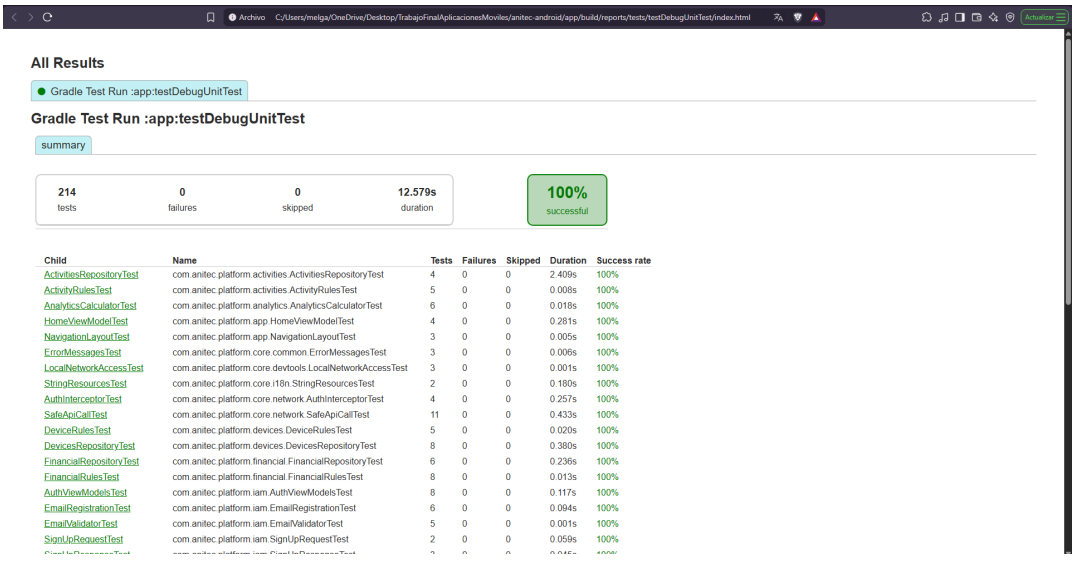


Figura 4.2.2.5.2. Lista de pruebas del backend por tipo. Fuente: elaboración propia.



All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

214	0	0	12.579s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
ActivitiesRepositoryTest	com.anitec.platform.activities.ActivitiesRepositoryTest	4	0	0	2.409s	100%
ActivityRulesTest	com.anitec.platform.activities.ActivityRulesTest	5	0	0	0.008s	100%
AnalyticsCalculatorTest	com.anitec.platform.analytics.AnalyticsCalculatorTest	6	0	0	0.018s	100%
HomeViewModelTest	com.anitec.platform.app.HomeViewModelTest	4	0	0	0.281s	100%
NavigationLayoutTest	com.anitec.platform.app.NavigationLayoutTest	3	0	0	0.005s	100%
ErrorMessagesTest	com.anitec.platform.core.common.ErrorMessagesTest	3	0	0	0.006s	100%
LocalNetworkAccessTest	com.anitec.platform.core.devtools.LocalNetworkAccessTest	3	0	0	0.001s	100%
StringResourcesTest	com.anitec.platform.core.i18n.StringResourcesTest	2	0	0	0.180s	100%
AuthInterceptorTest	com.anitec.platform.core.network.AuthInterceptorTest	4	0	0	0.257s	100%
SafeApiCallTest	com.anitec.platform.core.network.SafeApiCallTest	11	0	0	0.433s	100%
DeviceRulesTest	com.anitec.platform.devices.DeviceRulesTest	5	0	0	0.020s	100%
DevicesRepositoryTest	com.anitec.platform.devices.DevicesRepositoryTest	8	0	0	0.380s	100%
FinancialRepositoryTest	com.anitec.platform.financial.FinancialRepositoryTest	6	0	0	0.236s	100%
FinancialRulesTest	com.anitec.platform.financial.FinancialRulesTest	8	0	0	0.013s	100%
AuthViewModelsTest	com.anitec.platform.iam.AuthViewModelsTest	8	0	0	0.117s	100%
EmailRegistrationTest	com.anitec.platform.iam.EmailRegistrationTest	6	0	0	0.094s	100%
EmailValidatorTest	com.anitec.platform.iam.EmailValidatorTest	5	0	0	0.001s	100%
SignUpRequestTest	com.anitec.platform.iam.SignUpRequestTest	2	0	0	0.059s	100%

Figura 4.2.2.5.3. Resultado de las pruebas unitarias de Android. Fuente: elaboración propia.

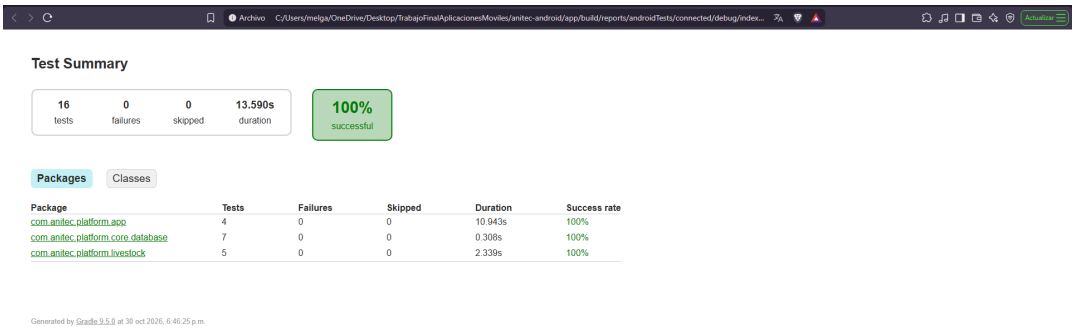


Figura 4.2.2.5.4. Resultado de las pruebas instrumentadas de Android (4 pruebas del paquete app, que incluye BottomNavigationTest). Fuente: elaboración propia.

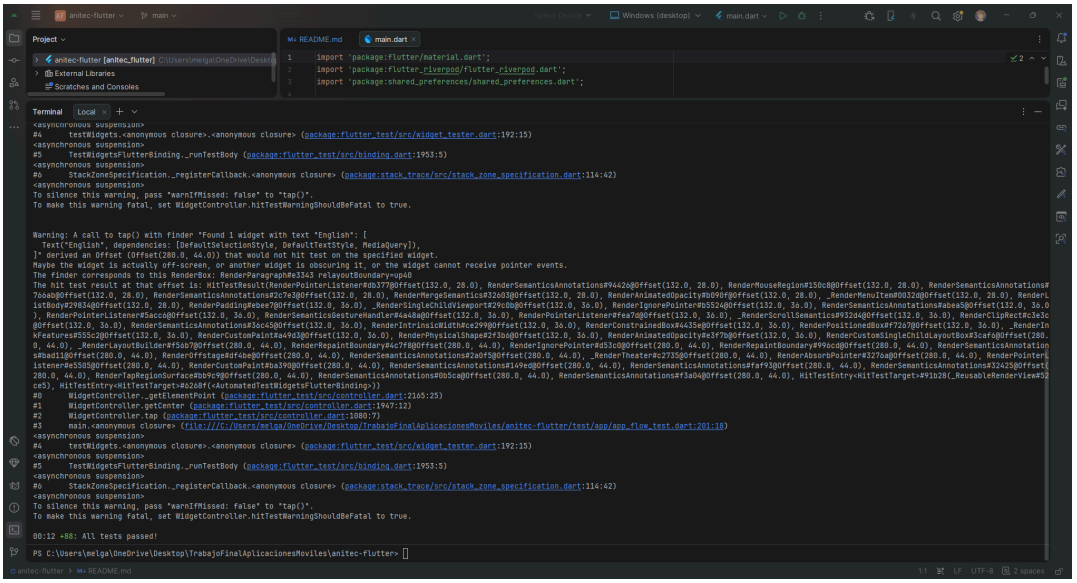


Figura 4.2.2.5.5. Resultado de las pruebas de la aplicación Flutter. Fuente: elaboración propia.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
anitec-backend	main	642e129	feat: scope data by user, seed plans, mobile Stripe return and archive animals	Agrega AccessScopeApiTests y SubscriptionsApiTests, amplía LivestockApiTests y HealthEventsApiTests, y actualiza los escenarios RolePermissions.feature y HealthRecords.feature.	2026-10-23
anitec-backend	main	0c4f7cc	feat: name animals in a batch and seed three ranchers with corrals and many animals	Agrega SeedDataTests y las pruebas del alta masiva con nombre base en LivestockApiTests.	2026-10-24
anitec-android	main	959defc	feat: AV2 - activities, finance, IoT,	Pruebas unitarias de dominio, casos de uso, repositorios y ViewModels de actividades, analítica, dispositivos,	2026-10-02

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
			analytics, QR scanner and offline outbox	finanzas, registro con correo, escáner, cola sin conexión y vista del veterinario.	
anitec-android	main	000a6f4	feat: improve validation errors, accessibility and tablet layout	Agrega ErrorMessagesTest, StringResourcesTest y NavigationLayoutTest, y amplía DevicesTest y FinancialTest.	2026-10-23
anitec-android	main	5553a38	feat: add Stripe subscriptions, archive animals, offline finance records and release build setup	Agrega SubscriptionsTest, ArchiveAnimalsTest y AuthInterceptorTest, y amplía OutboxTest, FinancialTest y ScannerTest.	2026-10-23
anitec-android	main	7d34023	feat: ask for local network access when the debug build opens	Agrega LocalNetworkAccessTest.	2026-10-24
anitec-android	main	85a687f	feat: name the animals of a batch with a base name and number	Agrega BatchNamingTest.	2026-10-24
anitec-android	main	2e63643	fix: return to Home from the bottom bar after opening a client's patients	Agrega BottomNavigationTest, prueba instrumentada con cuatro casos que reproduce el defecto de la navegación inferior.	2026-10-24
anitec-android	main	3716e53	fix: always send the platform in the Stripe checkout request	Agrega a SubscriptionsTest una prueba que serializa el cuerpo del checkout con la configuración de la aplicación.	2026-11-02
anitec-flutter	main	bc56f7e	feat: add rancher dashboard with sign-in and language switch	Pruebas del dominio, del estado, del cliente HTTP, de los repositorios y de las pantallas con flutter_test y mocktail.	2026-10-23
anitec-flutter	main	721fe36	fix: leave archived animals out of the dashboard and keep a new session on a late 401	Agrega archived_animals_test.dart y amplía api_client_test.dart.	2026-10-23
anitec-flutter	main	bf89e95	fix: show the farm selector choice in the	Amplía app_flow_test.dart con la comprobación del selector de finca al	2026-10-29

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on
			language of the app	cambiar de idioma.	

4.2.2.6. Execution Evidence for Sprint Review

El Sprint 2 permite instalar las aplicaciones Android y Flutter desde Firebase App Distribution, iniciar sesión contra el backend publicado y recorrer los flujos del ganadero y del veterinario con los datos de demostración: inicio, fincas, corrales, animales (con archivado y registro masivo con nombre base), registros sanitarios, actividades, finanzas, analítica, dispositivos IoT, escáner de códigos QR, trabajo sin conexión, suscripciones y, para el veterinario, clientes, pacientes e historial clínico. La aplicación Flutter muestra el inicio del ganadero. Las capturas de las figuras 4.2.2.6.1 a 4.2.2.6.8 y 4.2.2.6.13 a 4.2.2.6.22 se tomaron en el emulador Medium_Phone_API_37.0 (Android 17): las de Android con la compilación de depuración y las de Flutter con la compilación de perfil (la pantalla de inicio de sesión, con la compilación release), conectadas al backend local con los datos de demostración, e identifican el usuario de demostración usado. Las pantallas del pago con Stripe, del escáner de códigos QR y de la instalación desde Firebase se capturaron en un dispositivo físico.

Producto	Vista o flujo	Entorno / dispositivo	Evidencia	Estado
Android (ganadero)	Inicio, animales y archivado, registro masivo con nombre base, actividades, finanzas, analítica, dispositivos IoT	Emulador, usuario ganadero	Figuras 4.2.2.6.1 a 4.2.2.6.7	Completado
Android (ganadero)	Suscripciones: planes, pago con Stripe, resultado e historial	Planes e historial en el emulador; pago en un dispositivo físico con tarjeta de prueba de Stripe	Figuras 4.2.2.6.8 a 4.2.2.6.10	Completado
Android (ganadero)	Escáner de códigos QR y búsqueda manual	Dispositivo físico	Figuras 4.2.2.6.11 y 4.2.2.6.12	Completado
Android (ganadero)	Trabajo sin conexión y aviso de sincronización	Emulador con el modo avión activado	Figura 4.2.2.6.13	Completado
Android (veterinario)	Inicio, clientes, pacientes, historial clínico y analítica	Emulador, usuario veterinaria	Figuras 4.2.2.6.14 a 4.2.2.6.18	Completado
Flutter	Inicio de sesión, inicio del ganadero, cambio de idioma y mensaje para el veterinario	Emulador, usuarios ganadero y veterinaria	Figuras 4.2.2.6.19 a 4.2.2.6.22	Completado
Distribución	Instalación desde Firebase App Distribution	Dispositivo físico	Figura 4.2.2.6.23	Pendiente de captura

Producto	Vista o flujo	Entorno / dispositivo	Evidencia	Estado
Web Services	Swagger UI con los endpoints del Sprint 2	Navegador contra el Swagger del servicio publicado	Figuras de la sección 4.2.2.7	Pendiente de captura

Aplicación Android: ganadero. Se inicia sesión con el usuario ganadero (los datos de demostración se describen en 4.1.4).

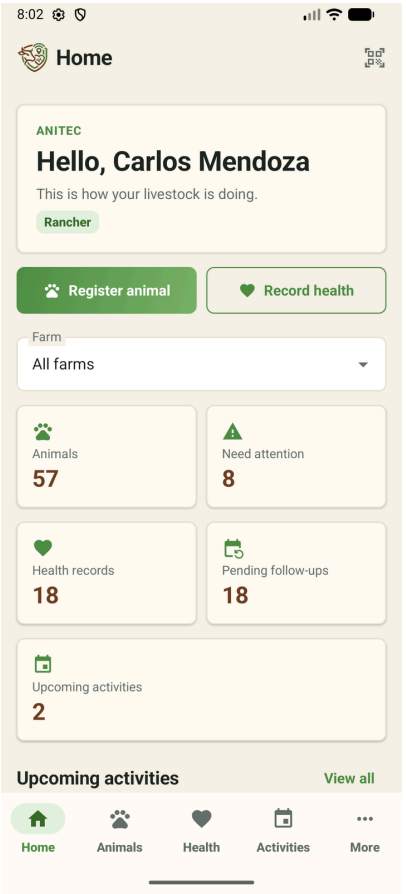


Figura 4.2.2.6.1. Inicio del ganadero con los datos de demostración. Fuente: elaboración propia.

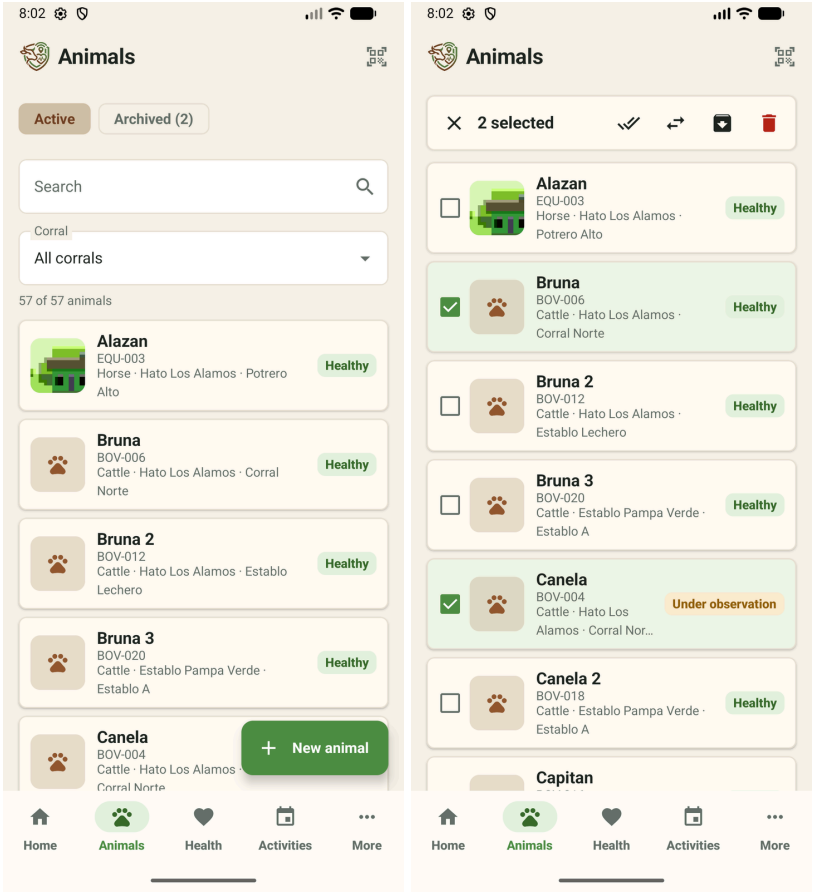


Figura 4.2.2.6.2. Lista de animales con selección múltiple, acción de archivar y filtro de archivados. Fuente: elaboración propia.

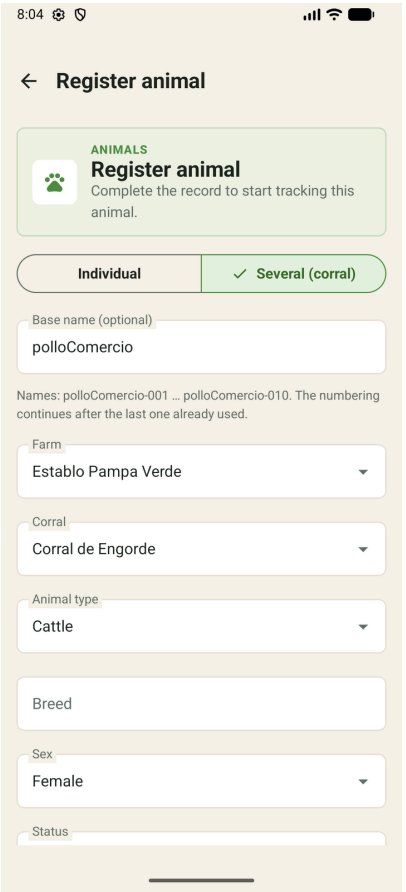


Figura 4.2.2.6.3. Registro masivo de animales con nombre base y vista previa. Fuente: elaboración propia.

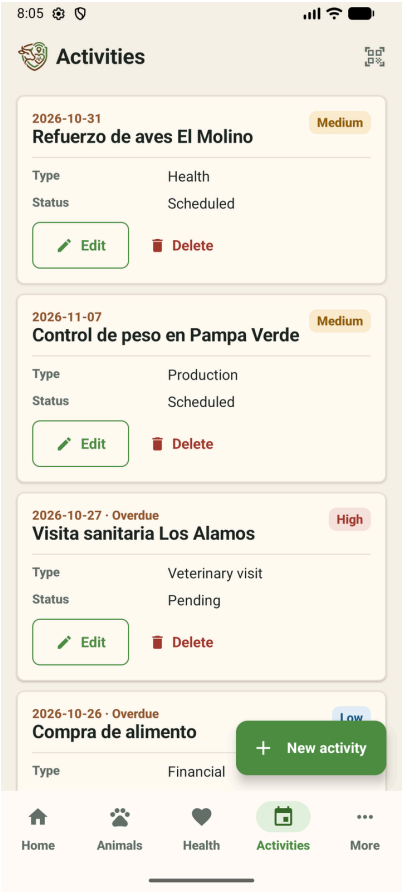


Figura 4.2.2.6.4. Lista de actividades. Fuente: elaboración propia.

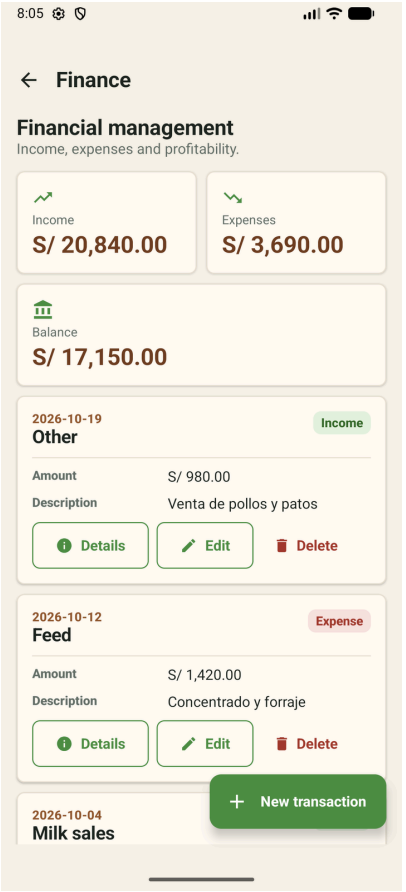


Figura 4.2.2.6.5. Pantalla de finanzas. Fuente: elaboración propia.

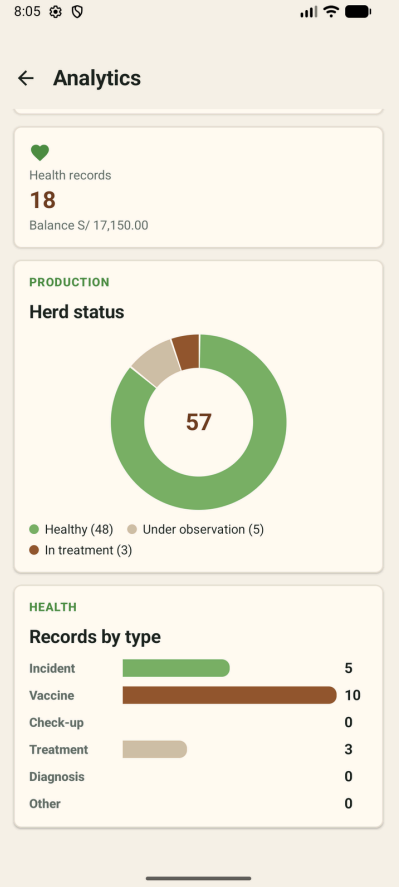


Figura 4.2.2.6.6. Pantalla de analítica del ganadero. Fuente: elaboración propia.

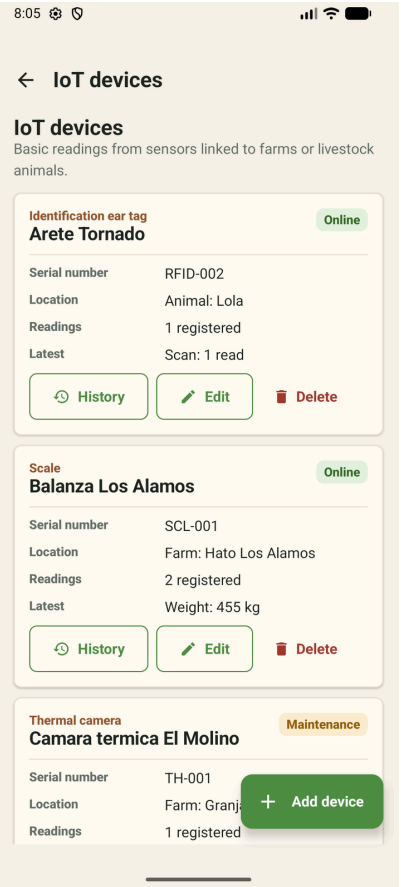


Figura 4.2.2.6.7. Dispositivos IoT con su última lectura. Fuente: elaboración propia.

Aplicación Android: suscripciones. El pago se hace en el modo de prueba de Stripe, con la tarjeta de prueba 4242 4242 4242 4242.

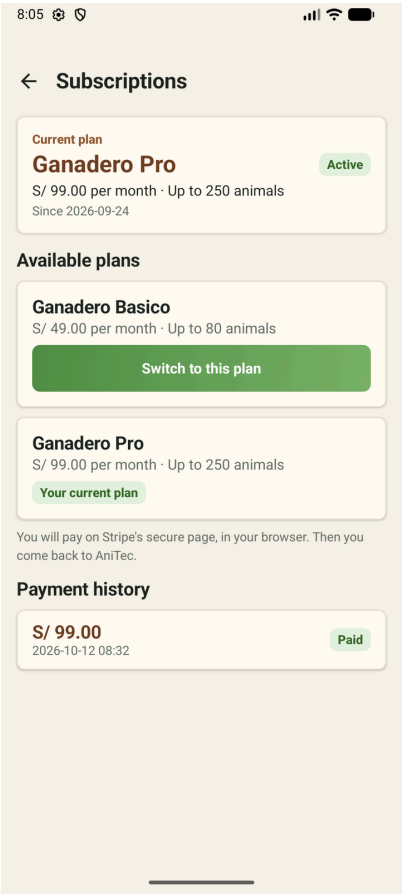


Figura 4.2.2.6.8. Planes de suscripción. Fuente: elaboración propia.

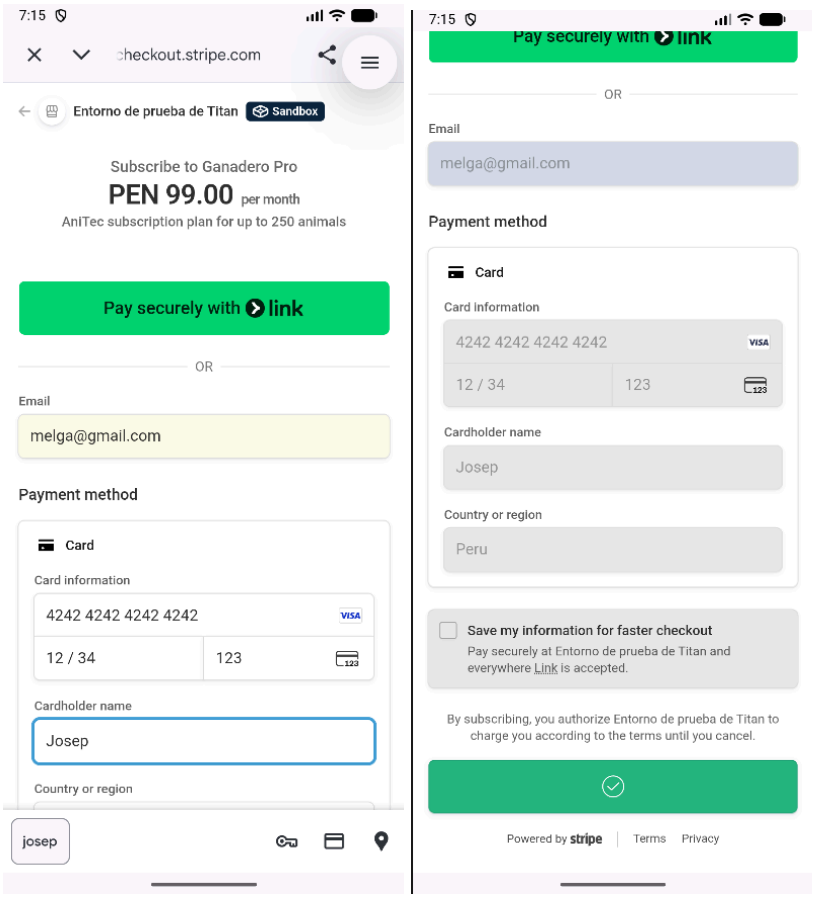


Figura 4.2.2.6.9. Checkout de Stripe en modo de prueba abierto desde la aplicación, con la tarjeta de prueba y el formulario enviado. Fuente: elaboración propia.

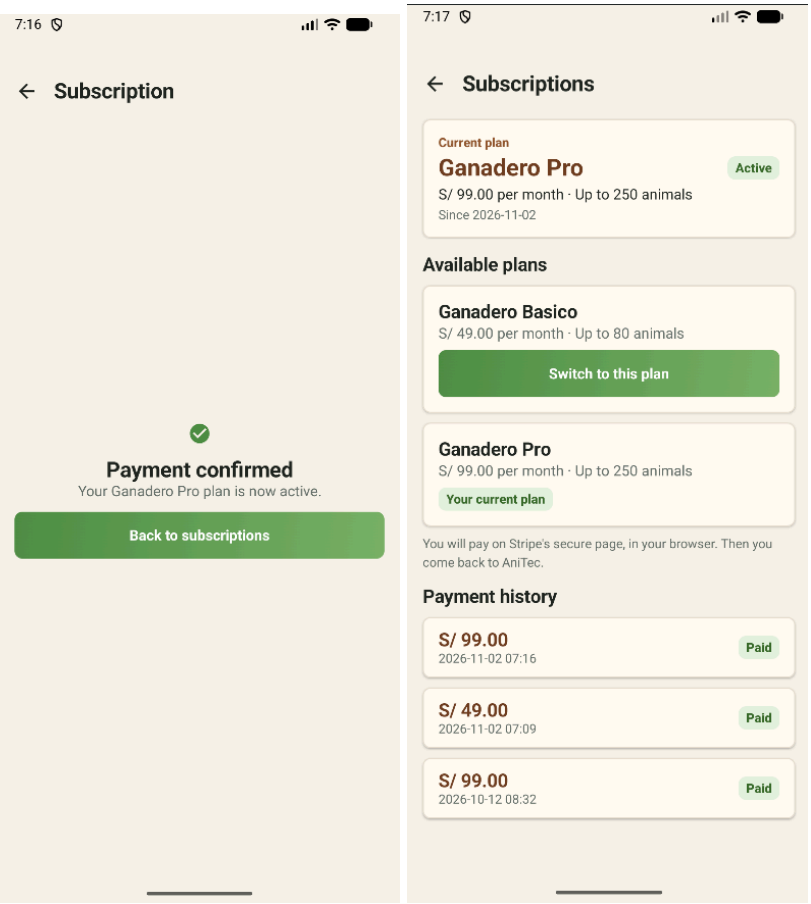


Figura 4.2.2.6.10. Resultado del pago, plan activo e historial de pagos. Fuente: elaboración propia.

Aplicación Android: escáner de códigos QR. Así se ve la pantalla del escáner en el emulador, cuya cámara es virtual.

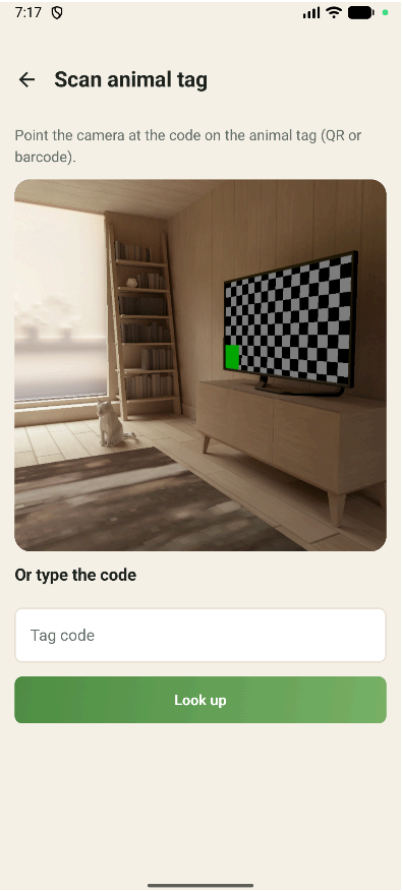


Figura 4.2.2.6.11. Pantalla del escáner de códigos en el emulador. Fuente: elaboración propia.

Figura 4.2.2.6.12. Campo de búsqueda manual del código del animal en el emulador. Fuente: elaboración propia.

Aplicación Android: trabajo sin conexión.

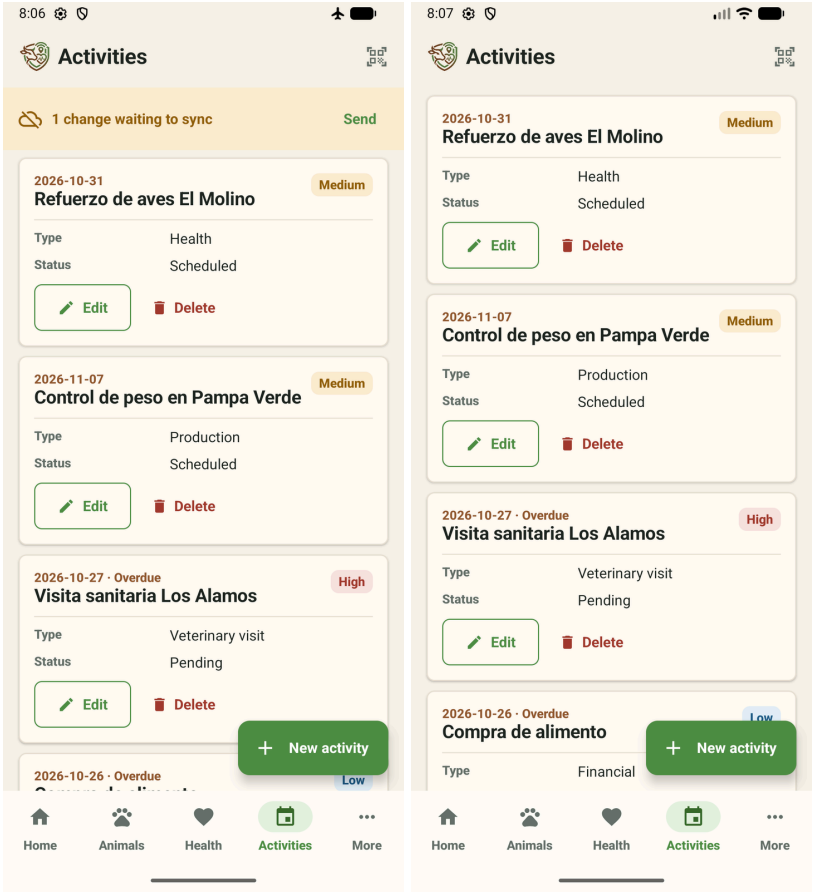


Figura 4.2.2.6.13. Registro sin conexión y aviso de sincronización pendiente. Fuente: elaboración propia.

Aplicación Android: veterinario. Se inicia sesión con el usuario veterinaria.

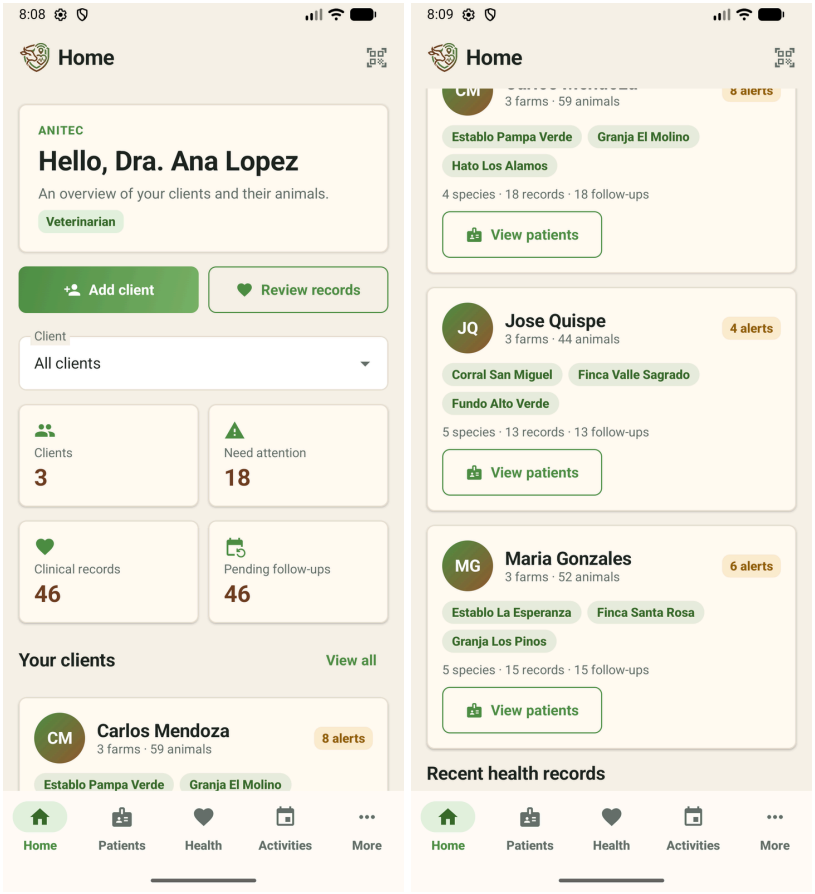


Figura 4.2.2.6.14. Inicio del veterinario. Fuente: elaboración propia.

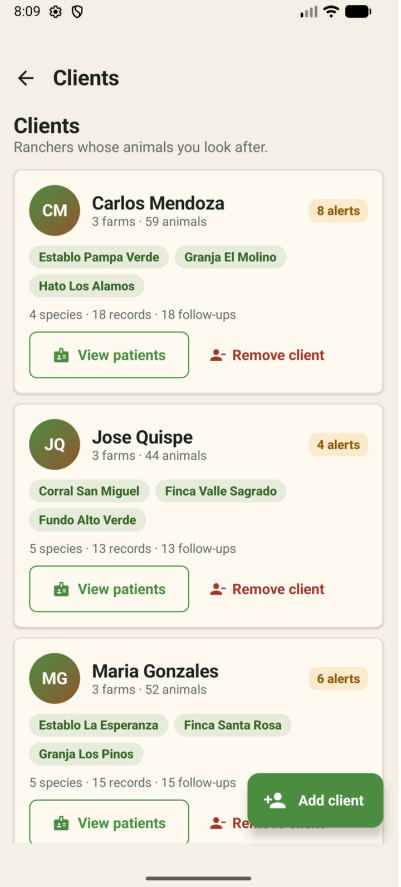


Figura 4.2.2.6.15. Clientes del veterinario. Fuente: elaboración propia.

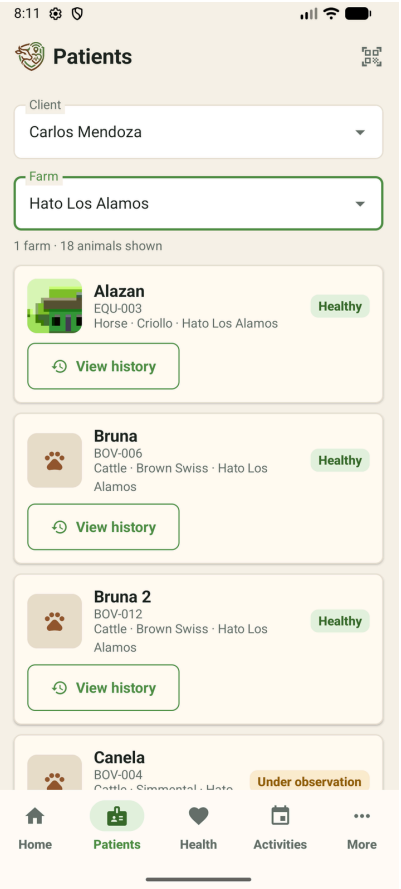


Figura 4.2.2.6.16. Pacientes de un cliente por finca. Fuente: elaboración propia.

Figura 4.2.2.6.17. Historial clínico de un paciente. Fuente: elaboración propia.

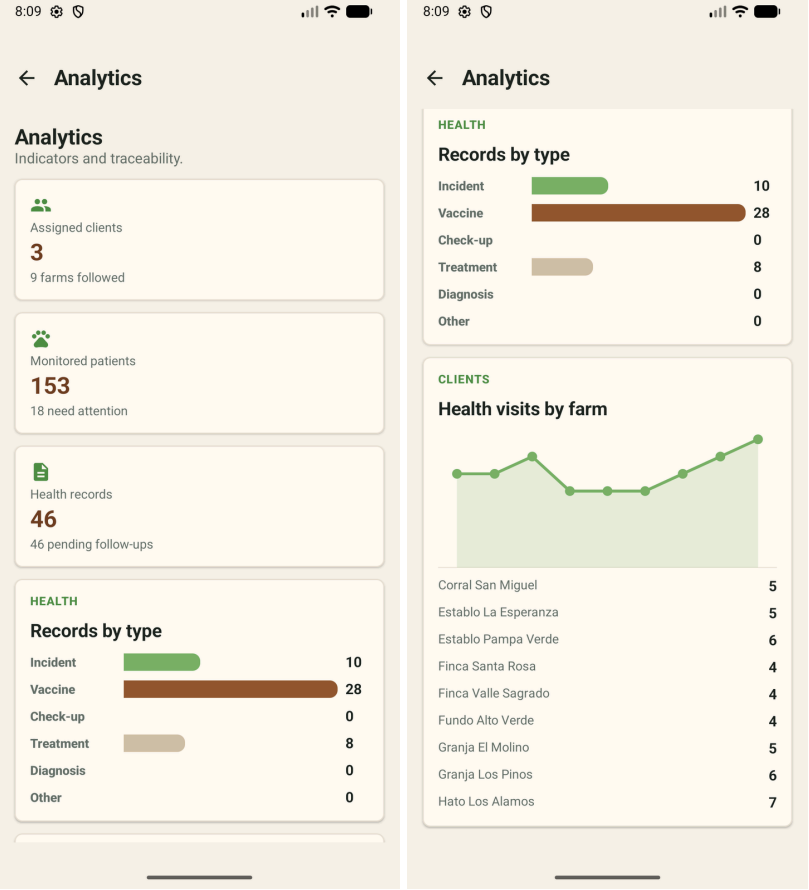


Figura 4.2.2.6.18. Analítica del veterinario. Fuente: elaboración propia.

Aplicación Flutter. Se captura en un dispositivo o emulador, con el usuario ganadero.

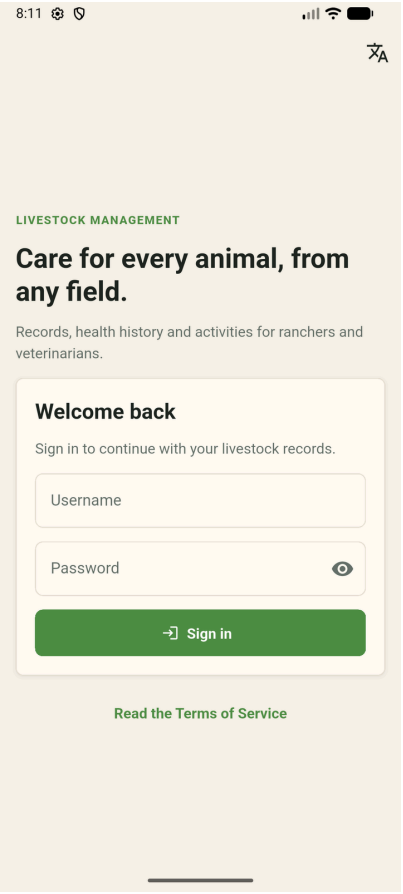


Figura 4.2.2.6.19. Inicio de sesión de la aplicación Flutter. Fuente: elaboración propia.

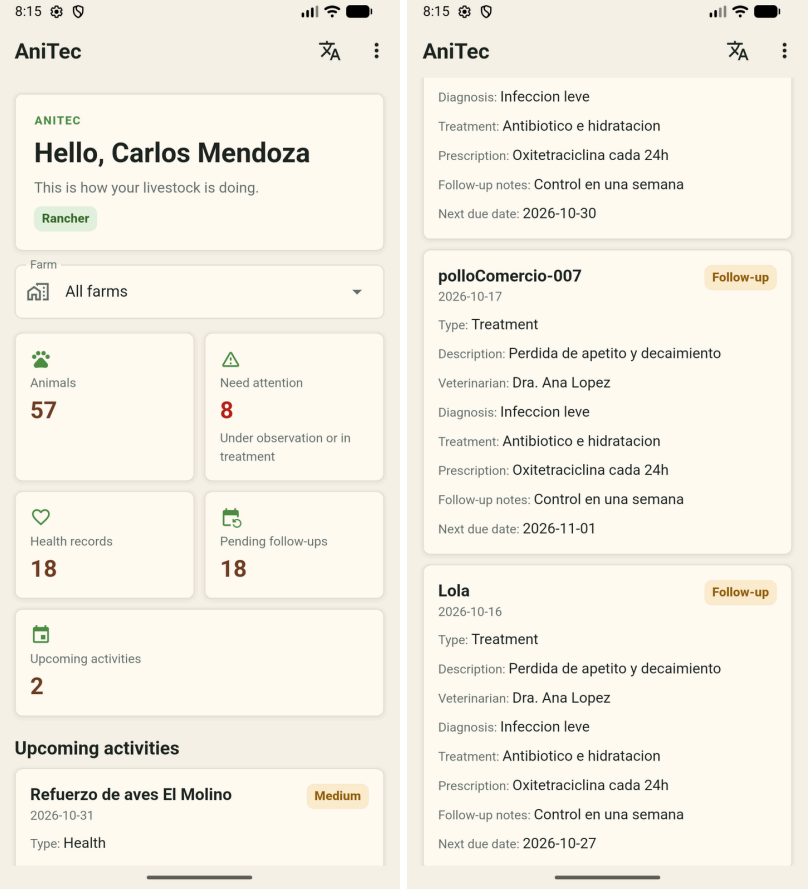


Figura 4.2.2.6.20. Inicio del ganadero en la aplicación Flutter. Fuente: elaboración propia.

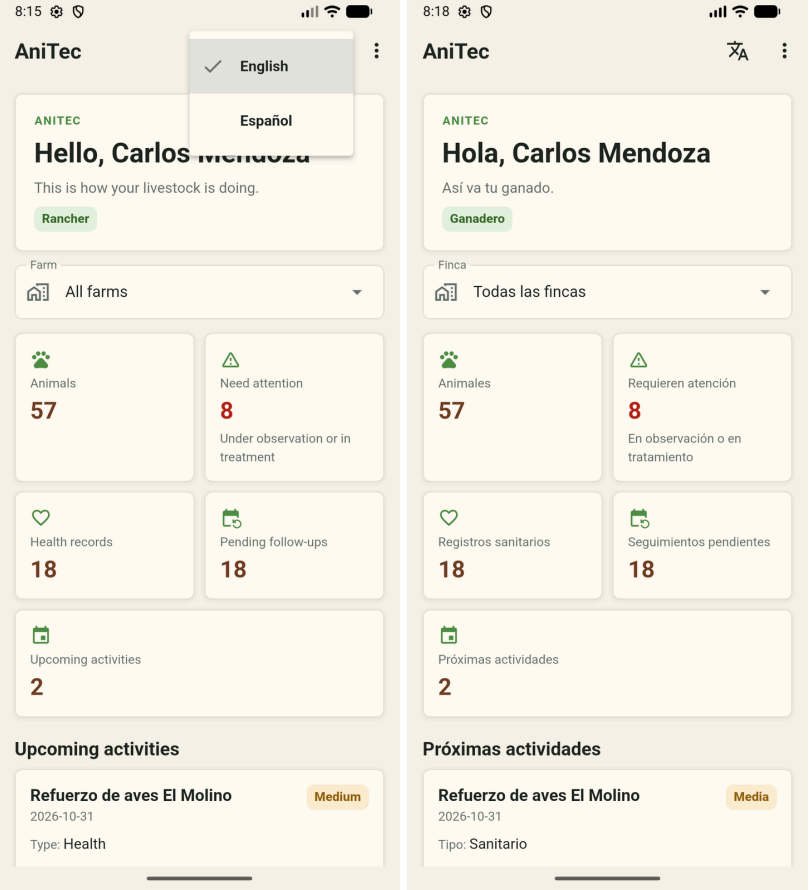


Figura 4.2.2.6.21. Cambio de idioma en la aplicación Flutter. Fuente: elaboración propia.

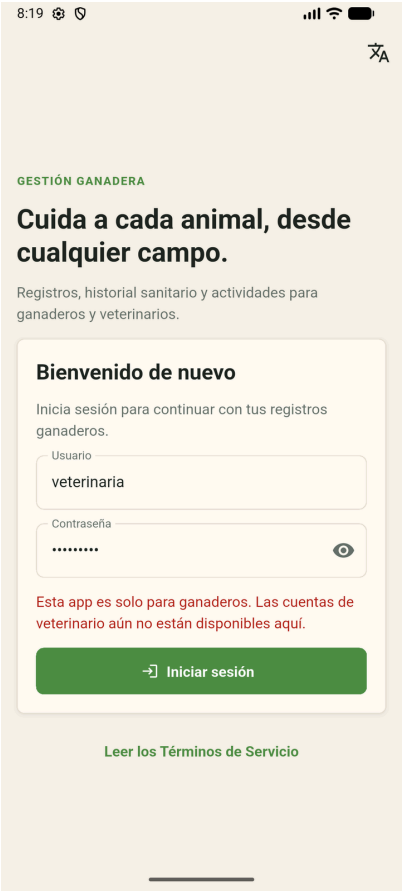


Figura 4.2.2.6.22. Mensaje al iniciar sesión con el rol de veterinario en Flutter. Fuente: elaboración propia.

Distribución.

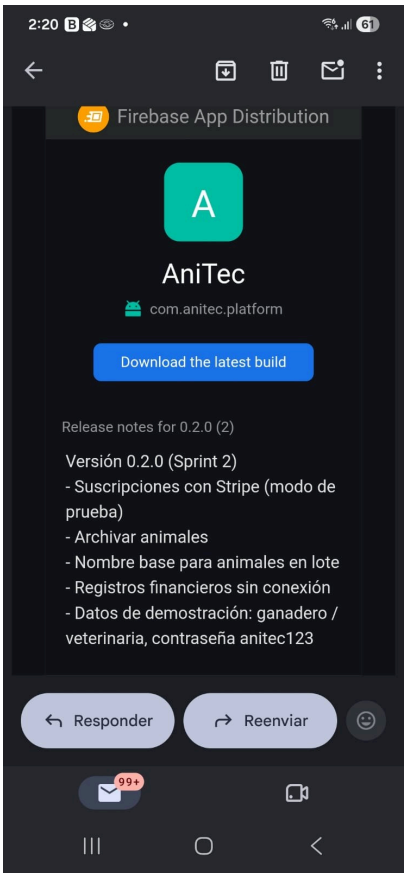


Figura 4.2.2.6.23. Instalación de la aplicación desde Firebase App Distribution. Fuente: elaboración propia.

Videos de ejecución del Sprint 2. Pendiente de completar: los enlaces se agregarán cuando estén grabados.

- **Backend:** pendiente de completar (enlace al video).
- **App Android:** pendiente de completar (enlace al video).
- **App Flutter:** pendiente de completar (enlace al video).

4.2.2.7. Services Documentation Evidence for Sprint Review

Con el Sprint 2 el backend queda completo y publicado, con su documentación OpenAPI en Swagger. Esta sección documenta los endpoints que se añaden o cambian en el sprint: el archivado de animales, el nombre base del alta masiva, las suscripciones y sus planes, las actividades, los movimientos financieros, los dispositivos y sus lecturas, los clientes del veterinario y los paneles analíticos. Los endpoints de autenticación, fincas, corrales, animales y registros sanitarios del Sprint 1 se describen en la sección 4.2.1.7.

Alcance por usuario. El cambio transversal del sprint es que el servidor devuelve y modifica solo lo que corresponde al usuario autenticado: un ganadero ve sus fincas, corrales, animales, registros, actividades, movimientos, dispositivos y suscripciones; un veterinario ve lo de sus clientes. Una lista solo contiene los recursos del usuario, y consultar, crear, modificar o eliminar un recurso ajeno responde 403 Forbidden. Las llamadas con un identificador que no existe siguen respondiendo 404.

Convención de llamada. Es la de la sección 4.2.1.7: las rutas parten de {BASE_URL}/api/v1, las solicitudes con cuerpo usan Content-Type: application/json, cada llamada envía Authorization: Bearer y las fechas usan yyyy-MM-dd. En los ejemplos, representa el token real.

Animales: cambios del sprint (/animals; escritura solo para Rancher)

Related story	HTTP	Call syntax
US-012	PATCH	curl -X PATCH "{BASE_URL}/api/v1/animals/1/status" -H "Authorization: Bearer " -H "Content-Type: a
US-010, US-012	GET	curl "{BASE_URL}/api/v1/animals?archived=false" -H "Authorization: Bearer "
US-011	POST	curl -X POST "{BASE_URL}/api/v1/animals/bulk" -H "Authorization: Bearer " -H "Content-Type: applic '{"name":"polloComercio","species":"Ave","breed":"Cobb","gender":"Mixto","weight":1.8,"status":"Salu

Planes de suscripción (/subscription-plans)

Ejemplo de SubscriptionPlanResource: {"id":1,"name":"Ganadero Basico","price":49.00,"stripePriceId":"mock_price_rancher_basic","maxAnimals":80,"isActive":true}. Los planes se cargan al iniciar la API en cualquier entorno, solo si la tabla está vacía.

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
US-038	GET	curl " {BASE_URL}/api/v1/subscription-plans" -H "Authorization: Bearer "	Ninguno.	200 con la lista de planes: Ganadero Básico (S/ 49, hasta 80 animales), Ganadero Pro (S/ 99, hasta 250) y Veterinario Profesional (S/ 129, hasta 1000).	401; 403.
US-038	GET	curl " {BASE_URL}/api/v1/subscription-plans/1" -H "Authorization: Bearer "	Ruta: id.	200 con el SubscriptionPlanResource.	404 si no existe; 401.

Suscripciones (/subscriptions; Rancher y Veterinarian)

Ejemplo de SubscriptionResource: {"id":1,"userId":1,"planId":2,"stripeCustomerId":"cus_...","stripeSubscriptionId":"sub_...","status":"Active","startedAt":"2026-10-25","endsAt":null}. Ejemplo de PaymentResource: {"id":1,"userId":1,"subscriptionId":1,"amount":99.00,"currency":"PEN","provider":"StripeTest","providerPaymentId":"cs_test_...","status":"Paid","paidAt":"2026-10-25T10:15:00"}.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-039	POST	curl -X POST " {BASE_URL}/api/v1/subscriptions/stripe-checkout" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"planId":2,"platform":"android"}'	Body StripeCheckoutResource: planId y platform. Con android, Stripe vuelve a una página HTTPS de la API que abre la aplicación. El cobro es mensual, en soles y en modo de prueba.	200 con {"sessionId":"cs_test la aplicación abre c
US-040	GET	curl "{BASE_URL}/api/v1/subscriptions/stripe-checkout/cs_test_.../confirm" -H "Authorization: Bearer "	Ruta: sessionId.	200 con {"subscript suscripciones activa Repetir la confirma

Related story	HTTP	Call syntax	Parameters / body	Success response
US-040	GET	curl " {BASE_URL}/api/v1/subscriptions/users/1/active" -H "Authorization: Bearer "	Ruta: userId, que debe ser el del usuario autenticado.	200 con la suscripci
US-040	GET	curl " {BASE_URL}/api/v1/subscriptions/users/1/payments" -H "Authorization: Bearer "	Ruta: userId, el del usuario autenticado.	200 con la lista de p
TS-011	GET	curl "{BASE_URL}/api/v1/subscriptions/mobile-return/success?session_id=cs_test_..."	Query: session_id. Es una página pública a la que Stripe devuelve al usuario; no figura en Swagger.	200 con una página session_id=.... El ide
TS-011	GET	curl "{BASE_URL}/api/v1/subscriptions/mobile-return/cancel"	Ninguno. Página pública.	200 con una página

Los endpoints POST, PUT y DELETE de /subscriptions y de /subscription-plans y el pago simulado POST /subscriptions/mock-checkout siguen disponibles para el desarrollo, pero la aplicación no los usa y el pago simulado responde 404 en producción.

Actividades (/farm-events; Rancher y Veterinarian)

Ejemplo de FarmActivityResource: {"id":1,"ownerId":1,"veterinarianId":2,"title":"Visita sanitaria Los Alamos","type":"Visita veterinaria","date":"2026-10-28","priority":"Alta","status":"Pendiente"}. Un ganadero crea actividades propias; un veterinario las programa para un cliente suyo.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-025	GET	curl "{BASE_URL}/api/v1/farm-events" -H "Authorization: Bearer "	Ninguno.	200 con las actividades del usuario (las suyas y, para el veterinario, las de sus clientes).
US-025	GET	curl "{BASE_URL}/api/v1/farm-events/1" -H "Authorization: Bearer "	Ruta: id.	200 con el FarmActivityResource.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-026	POST	curl -X POST "{BASE_URL}/api/v1/farm-events" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"ownerId":1,"veterinarianId":2,"title":"Visita sanitaria Los Alamos","type":"Visita veterinaria","date":"2026-10-28","priority":"Alta","status":"Pendiente"}'	Body CreateFarmActivityResource: ownerId, veterinarianId (opcional), title, type, date, priority y status.	201 con la actividad creada.
US-026	PUT	curl -X PUT "{BASE_URL}/api/v1/farm-events/1" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{...}'	Ruta: id. Body CreateFarmActivityResource.	200 con la actividad actualizada.
US-026	DELETE	curl -X DELETE "{BASE_URL}/api/v1/farm-events/1" -H "Authorization: Bearer "	Ruta: id.	204 sin contenido.

Movimientos financieros (/financial-records; solo Rancher)

Ejemplo de FinancialRecordResource: {"id":1,"ownerId":1,"type":"Ingreso","category":"Venta de leche","amount":4200.00,"date":"2026-08-06","description":"Venta semanal de leche"}.

Related story	HTTP	Call syntax	Parameters / b
US-037	GET	curl "{BASE_URL}/api/v1/financial-records" -H "Authorization: Bearer "	Ninguno.
US-037	POST	curl -X POST "{BASE_URL}/api/v1/financial-records" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"ownerId":1,"type":"Gasto","category":"Alimento","amount":850.50,"date":"2026-10-20","description":"Compra de alimento"}'	Body CreateFinancialR ownerId, type (I Gasto), category y description.

Related story	HTTP	Call syntax	Parameters / b
US-037	PUT	curl -X PUT "{BASE_URL}/api/v1/financial-records/1" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{...}'	Ruta: id. Body CreateFinancialR
US-037	DELETE	curl -X DELETE "{BASE_URL}/api/v1/financial-records/1" -H "Authorization: Bearer "	Ruta: id.

Dispositivos IoT y lecturas (/devices y /device-metrics; escritura de dispositivos solo para Rancher)

Ejemplo de DeviceResource: {"id":1,"name":"Balanza Los Alamos","type":"Scale","serialNumber":"SCL-001","status":"Online","herdId":1,"animalId":null}. Ejemplo de DeviceMetricResource: {"id":1,"deviceId":1,"type":"Weight","value":455.00,"unit":"kg","recordedAt":"2026-10-24T12:42:40"}. Un dispositivo debe asignarse a una finca o a un animal que el usuario pueda ver.

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
Sin historia	GET	curl "{BASE_URL}/api/v1/devices" -H "Authorization: Bearer "	Ninguno.	200 con los dispositivos visibles para el usuario.	401; 403.
Sin historia	POST	curl -X POST "{BASE_URL}/api/v1/devices" -H "Authorization: Bearer " -H "Content-Type: application/json" -d '{"name":"Balanza Los Alamos","type":"Scale","serialNumber":"SCL-001","status":"Online","herdId":1,"animalId":null}'	Body CreateDeviceResource: name, type, serialNumber, status, herdId y animalId (uno de los dos).	201 con el dispositivo creado.	400 si no tiene una finca ni un animal asignados 403 si la finca o el animal no son visibles para el usuario; 401.
Sin historia	PUT / DELETE	curl -X PUT "{BASE_URL}/api/v1/devices/1" ... y curl -X DELETE "{BASE_URL}/api/v1/devices/1" ...	Ruta: id.	200 con el dispositivo actualizado o 204.	404; 400; 401; 403.
Sin historia	GET	curl "{BASE_URL}/api/v1/devices/1/metrics" -H "Authorization: Bearer "	Ruta: id.	200 con las lecturas del	404 si el dispositivo

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
				dispositivo, la más reciente primero.	no existe; 401; 403.
Sin historia	GET	curl "{BASE_URL}/api/v1/devices/1/latest-metric" -H "Authorization: Bearer "	Ruta: id.	200 con la última lectura.	404 si no existe el dispositivo o no tiene lecturas; 401; 403.
Sin historia	GET	curl "{BASE_URL}/api/v1/device-metrics" -H "Authorization: Bearer "	Ninguno.	200 con las lecturas de los dispositivos visibles para el usuario.	401; 403.

Clientes del veterinario (/veterinarian; solo Veterinarian, y solo sobre su propio identificador)

Ejemplo de cliente: {"id":1,"veterinarianId":2,"rancherId":1,"rancherName":"Carlos Mendoza","status":"Active","herds":3,"animals":59}.

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
US-023	GET	curl "{BASE_URL}/api/v1/veterinarian/2/clients" -H "Authorization: Bearer "	Ruta: veterinarianId, el del usuario.	200 con los clientes del veterinario, con su número de fincas y animales.	403 si el identificador es ajeno; 401.
US-023	GET	curl "{BASE_URL}/api/v1/veterinarian/2/available-ranchers" -H "Authorization: Bearer "	Ruta: veterinarianId.	200 con los ganaderos que aún no son sus clientes (id, username, fullName,	403; 401.

Related story	HTTP	Call syntax	Parameters / body	Success response	Error responses
				herds y animals).	
US-023	POST	curl -X POST " {BASE_URL}/api/v1/veterinarian/2/clients/1" -H "Authorization: Bearer "	Ruta: veterinarianId y rancherId.	201 con el vínculo creado.	409 si ya es su cliente; 403; 401.
US-023	DELETE	curl -X DELETE " {BASE_URL}/api/v1/veterinarian/2/clients/1" -H "Authorization: Bearer "	Ruta: veterinarianId y rancherId.	204 sin contenido; el veterinario deja de ver los datos de ese ganadero.	404 si no es su cliente; 403; 401.

Paneles analíticos (/analytics; Rancher y Veterinarian)

Los endpoints entregan indicadores calculados en el servidor. La aplicación Android calcula sus indicadores con los datos de su caché, por lo que estos endpoints quedan documentados para otros clientes.

Related story	HTTP	Call syntax	Parameters / body	Success response
US-035	GET	curl " {BASE_URL}/api/v1/analytics/ranchers/1/dashboard" -H "Authorization: Bearer "	Ruta: rancherId, el del usuario.	200 con los totales del ga {"herds":...,"animals":...,"he
US-036	GET	curl " {BASE_URL}/api/v1/analytics/veterinarians/2/dashboard" -H "Authorization: Bearer "	Ruta: veterinarianId, el del usuario.	200 con los totales del vet
US-035	GET	curl "{BASE_URL}/api/v1/analytics/ranchers/1/health-summary" -H "Authorization: Bearer "	Ruta: rancherId. Lo consulta el ganadero o su veterinario.	200 con el total de registr
US-037	GET	curl "{BASE_URL}/api/v1/analytics/ranchers/1/financial-summary" -H "Authorization: Bearer "	Ruta: rancherId, el del ganadero.	200 con {"income":...,"expe

- **Web Services repository:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend/tree/main
- **Swagger:** https://anitec-backend-android.onrender.com/swagger/index.html
- **API publicada:** https://anitec-backend-android.onrender.com
- **Documentation commits:** 642e129 (endpoint de archivado, páginas de retorno de Stripe y alcance por usuario) y 0c4f7cc (nombre base del alta masiva).

- **Interaction screenshots:** las capturas siguientes muestran llamadas con datos de muestra desde Swagger.

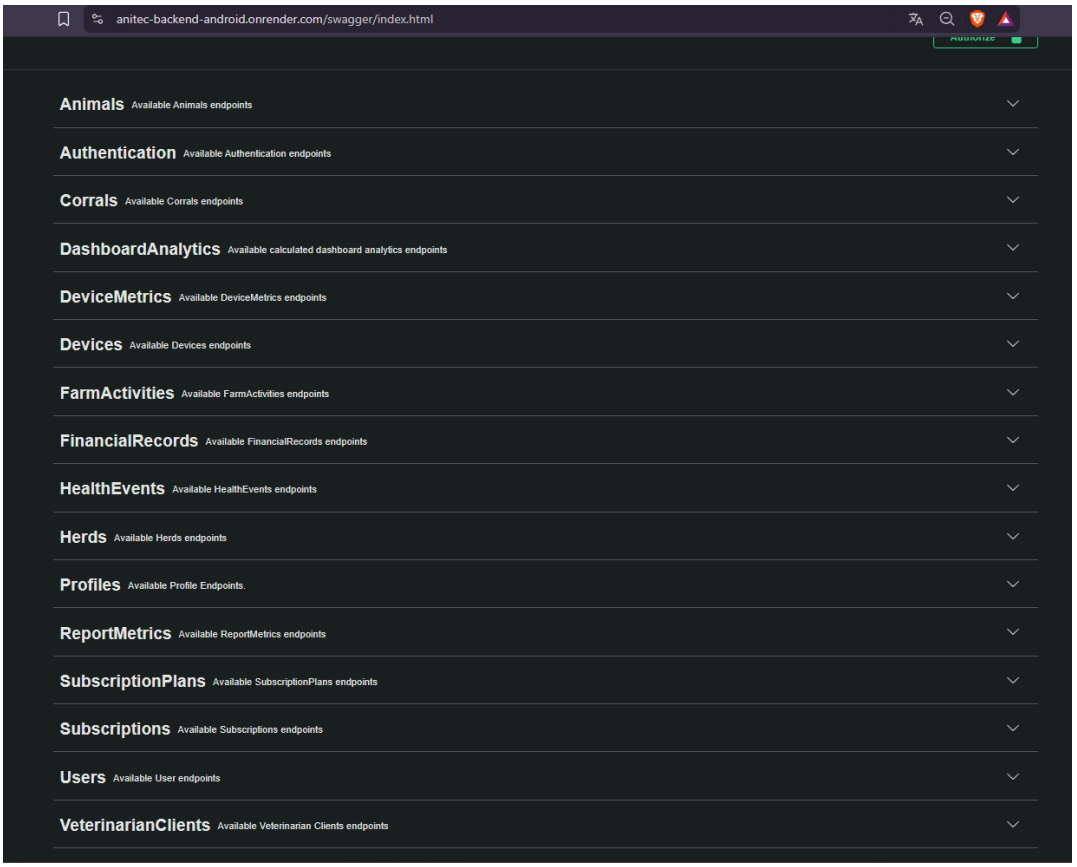


Figura 4.2.2.7.1. Swagger UI con los endpoints del Sprint 2. Fuente: elaboración propia.

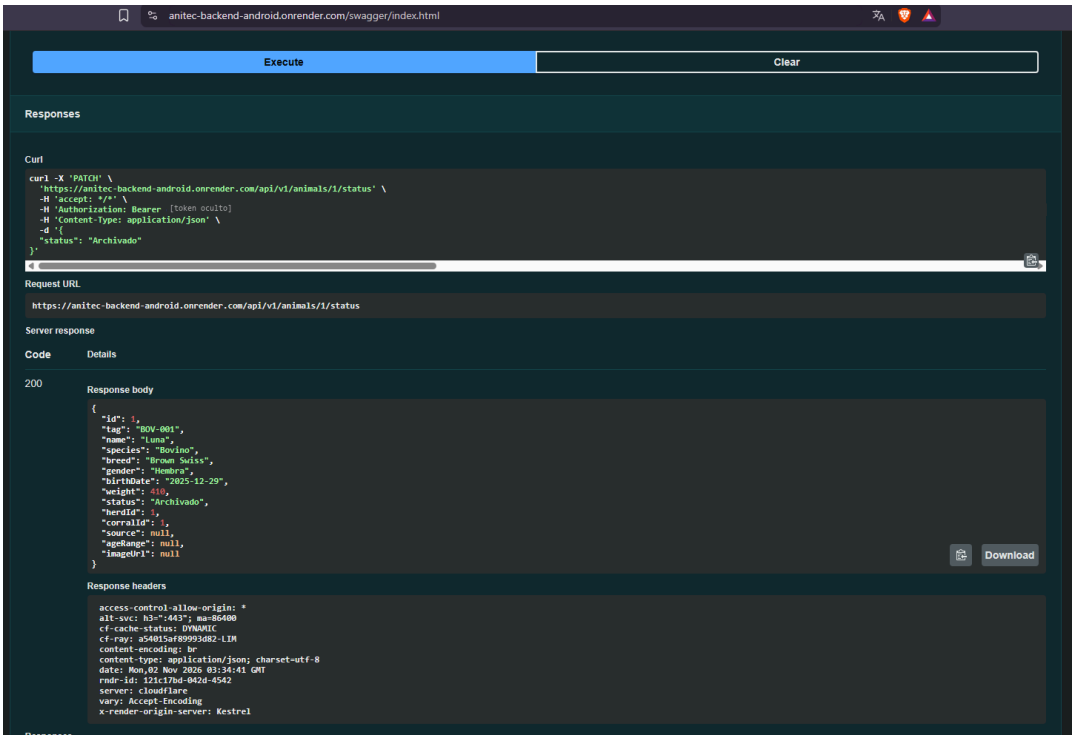


Figura 4.2.2.7.2. PATCH /animals/{id}/status para archivar un animal (con el usuario ganadero; el token de la cabecera está oculto). Fuente: elaboración propia.

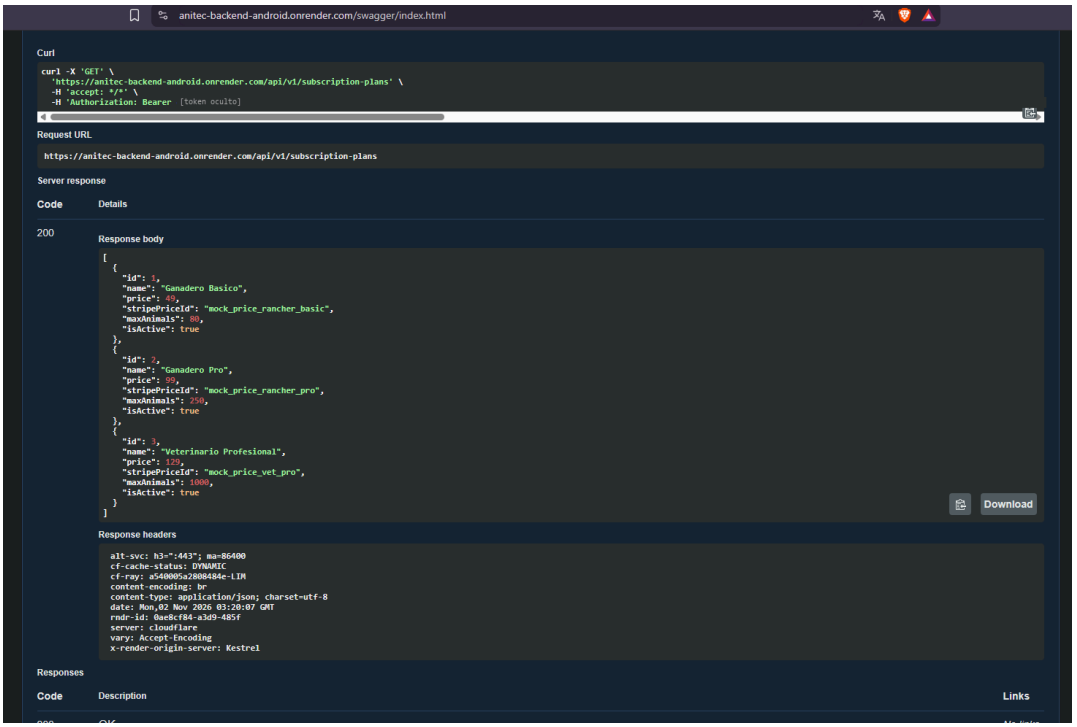


Figura 4.2.2.7.3. GET /subscription-plans con los planes de suscripción. Fuente: elaboración propia.

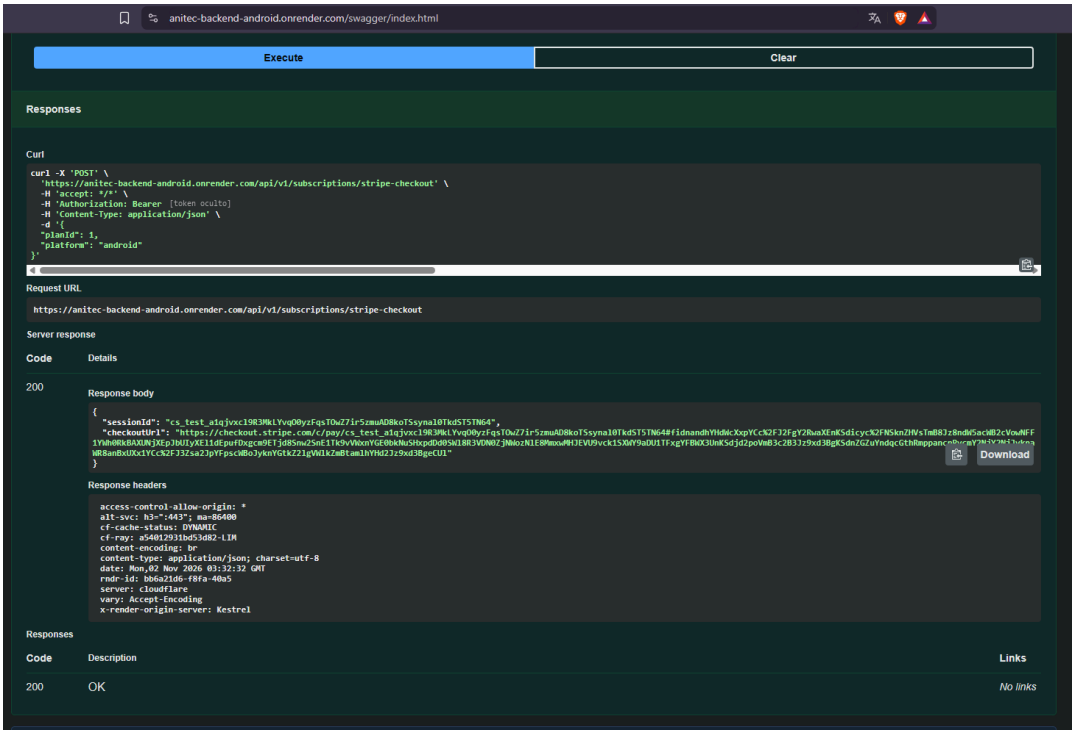


Figura 4.2.2.7.4. POST /subscriptions/stripe-checkout con la sesión de pago creada. Fuente: elaboración propia.

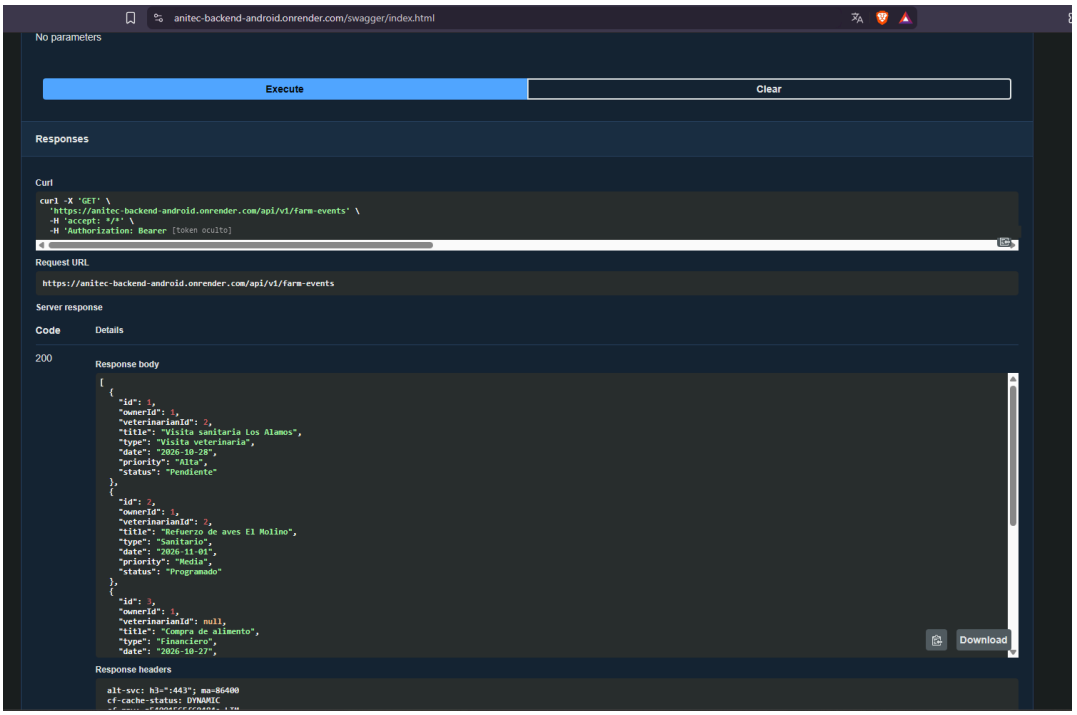


Figura 4.2.2.7.5. Consulta de actividades o movimientos financieros con datos de muestra. Fuente: elaboración propia.

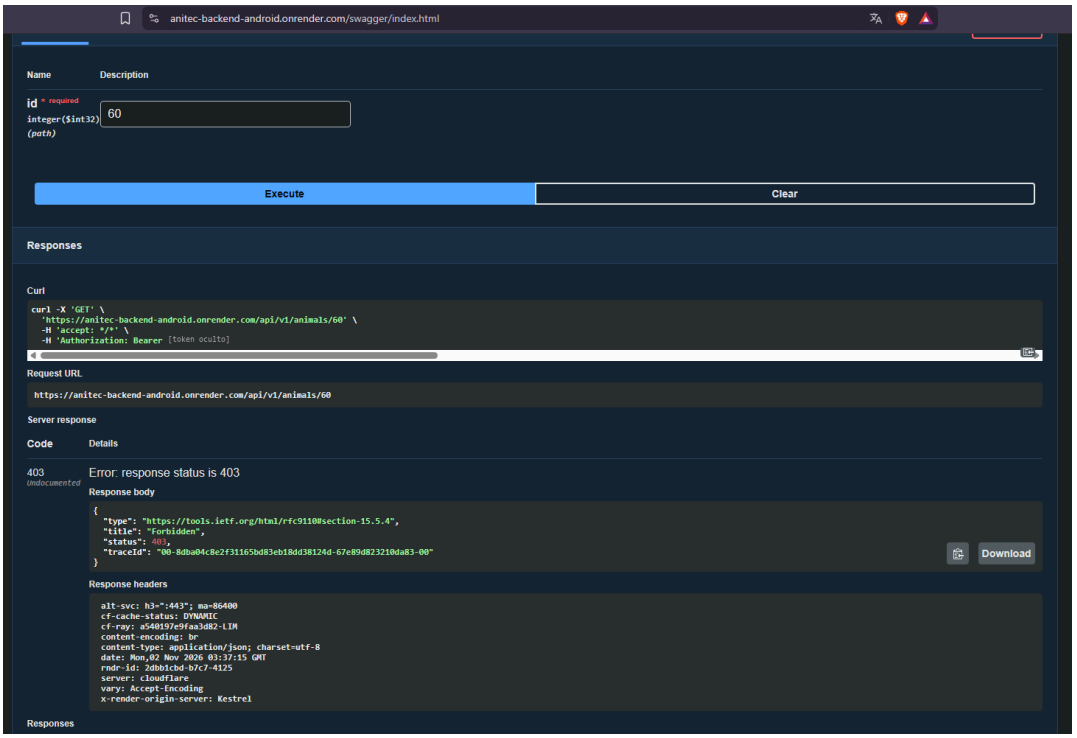


Figura 4.2.2.7.6. Respuesta 403 al consultar un animal de otro ganadero. Fuente: elaboración propia.

4.2.2.8. Software Deployment Evidence for Sprint Review

Durante el Sprint 2 se desplegó la versión completa del backend, se cargaron en el servicio publicado los datos de demostración y se distribuyeron las dos aplicaciones móviles. Para la distribución se creó el proyecto de Firebase «AniTec», se registraron las aplicaciones Android y Flutter, se generaron los APK firmados y se publicaron en App Distribution con un grupo de evaluadores con vínculos de invitación. Los pasos detallados están en la sección 4.1.4.

Product	Platform	Configuration performed	Version / Commit	Public URL or Release
Landing Page	GitHub Pages	El botón de descarga de Android reemplaza su marcador por el vínculo de invitación de Firebase (tarea T-061).	Pendiente	https://adm-1acc0238-2620-13975-grupo01.github.io/anitec-landing-page/
Web Services	Render	Redespliegue del servicio con el backend completo, variable SeedDemoData para cargar los datos de demostración y límite de conexiones en la cadena de conexión.	Commit 309c3b4 (el que Render muestra como desplegado, estado Live)	API: https://anitec-backend-android.onrender.com Swagger: https://anitec-backend-android.onrender.com/swagger/index.html
Android	Firebase App Distribution	Proyecto «AniTec» (anitec-17af9), aplicación com.anitec.platform, APK release firmado con R8 (cerca de 25 MB), grupo «Equipo AniTec» y vínculos de invitación del grupo.	versionName 0.2.0 (versionCode 2); versión definida en el commit 5553a38	Versión 0.2.0 (2) del 2026-10-25. Vínculos del grup https://appdistribution.firebase.dev/i/417dc4e6d8a y https://appdistribution.firebase.dev/i/862507cca1d
Flutter	Firebase App Distribution	Aplicación com.anitec.platform.flutter en el mismo proyecto, APK release firmado con el mismo keystore (52 MB) y el mismo grupo, con los mismos vínculos de invitación.	Versión 1.0.0+1; versión definida en el commit bc56f7e	Versión 1.0.0 (1) del 2026-10-25, en el mismo grup

Pasos ejecutados (Web Services):

1. Integrar los cambios del sprint en la rama main de anitec-backend; el push inicia el despliegue automático en Render.
2. Cargar los datos de demostración en la base publicada, que ya contenía datos de una carga anterior: vaciar sus tablas desde el cliente web del proveedor con DROP TABLE (sin eliminar la base, para conservar sus credenciales), definir SeedDemoData=true en el panel **Environment** de Render y esperar el redespliegue, durante el cual la API crea las tablas con las migraciones y carga los datos. La base quedó con 7 usuarios, 11 fincas, 34 corrales y 182 animales, además de 55 registros sanitarios, 15 actividades, 28 movimientos financieros, 12 dispositivos con 17 lecturas, 3 planes, 4 suscripciones, 4 pagos y 5 vínculos entre veterinarios y clientes (Figura 4.2.2.8.2).

3. Verificar el servicio publicado: iniciar sesión con el usuario ganadero devuelve 200 y sus listas de fincas, corrales y animales contienen sus datos (3 fincas, 10 corrales y 59 animales).
4. Corregir un fallo de carga: al abrir la aplicación en el teléfono aparecía «El servidor tuvo un problema». Las pruebas contra el servicio mostraron que cada petición respondía 200 por separado, pero que al pedir a la vez las ocho listas del inicio la mayoría respondía 500, porque el plan gratuito de la base de datos limita las conexiones simultáneas. Se añadió ;Maximum Pool Size=2 al final de ConnectionStrings__DefaultConnection, con lo que las peticiones esperan turno, y la aplicación cargó sus datos correctamente.
5. Desactivar SeedDemoData (se retira de las variables o se pone en false) para que el servicio no intente cargar datos en los siguientes arranques. En el servicio publicado quedó en false y ASPNETCORE_ENVIRONMENT en Production (Figura 4.2.2.8.1).

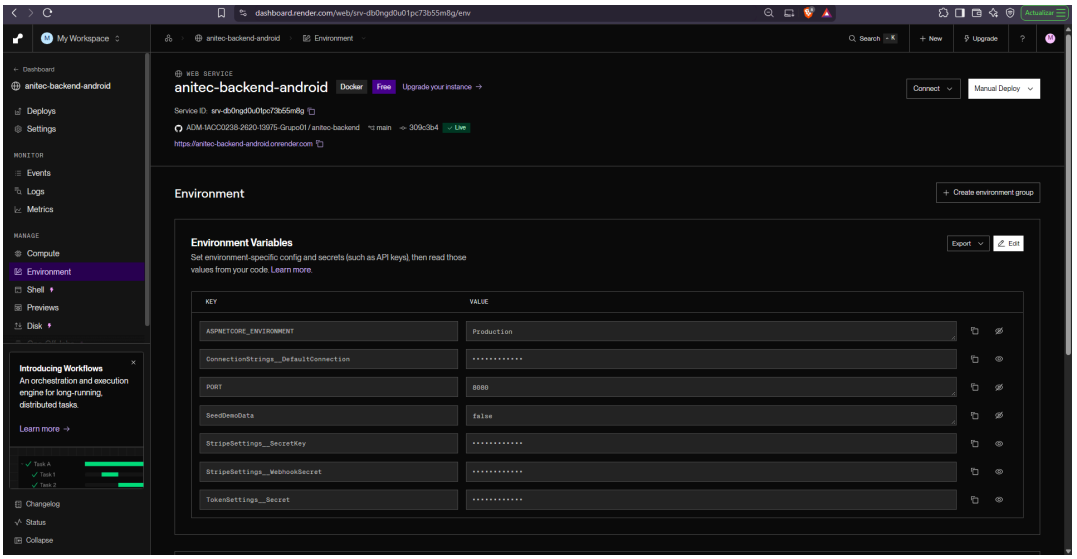


Figura 4.2.2.8.1. Variables de entorno del backend en Render. Los secretos (cadena de conexión, claves de Stripe y secreto del token) tienen sus valores ocultos; ASPNETCORE_ENVIRONMENT está en Production y SeedDemoData en false. Fuente: elaboración propia (captura de Render).

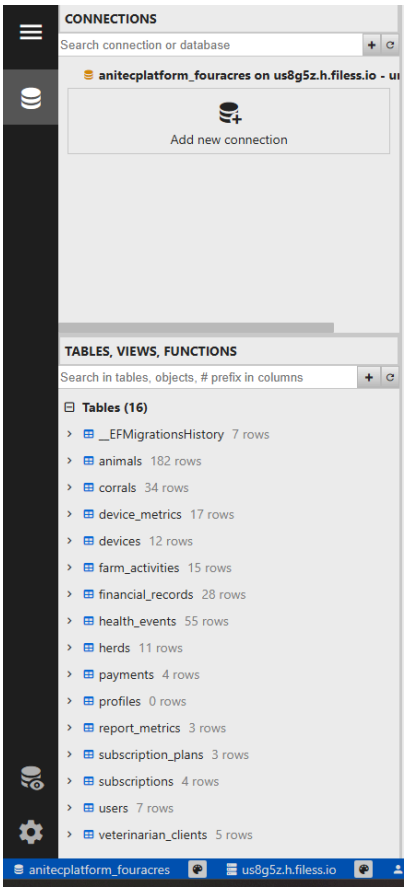


Figura 4.2.2.8.2. Base de datos publicada con los datos de demostración cargados. Fuente: elaboración propia (captura del proveedor).

Pasos ejecutados (aplicaciones móviles):

1. Crear el keystore con keytool y guardarlo fuera del repositorio, con una copia de respaldo; configurar keystore.properties en Android y android/key.properties en Flutter (ambos ignorados por git).
2. Generar el APK de Android con ./gradlew assembleRelease (tras ./gradlew --stop, para cerrar daemons iniciados con otro JDK) y el de Flutter con flutter build apk --release. La firma del APK de Flutter se comprobó con apksigner verify --print-certs.
3. Instalar el APK de Android en un dispositivo físico y comprobar que inicia sesión y carga los datos del backend publicado.
4. Crear el proyecto «AniTec» en Firebase Console, sin Gemini ni Google Analytics, y registrar las dos aplicaciones Android (Figura 4.1.4.4).
5. Subir los APK en App Distribution (Figuras 4.1.4.5 y 4.1.4.6), crear el grupo «Equipo AniTec» y distribuir cada versión.
6. Crear vínculos de invitación para el grupo «Equipo AniTec» (Figura 4.1.4.7), que la consola muestra desde las dos aplicaciones, y comprobar la instalación.

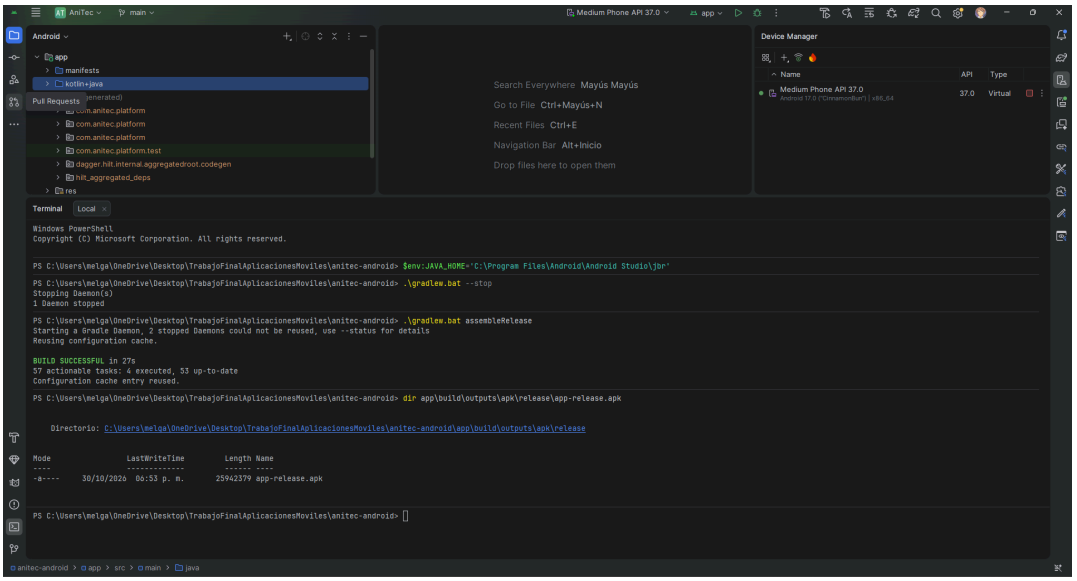


Figura 4.2.2.8.3. Compilación release firmada de la aplicación Android y APK generado. Fuente: elaboración propia.

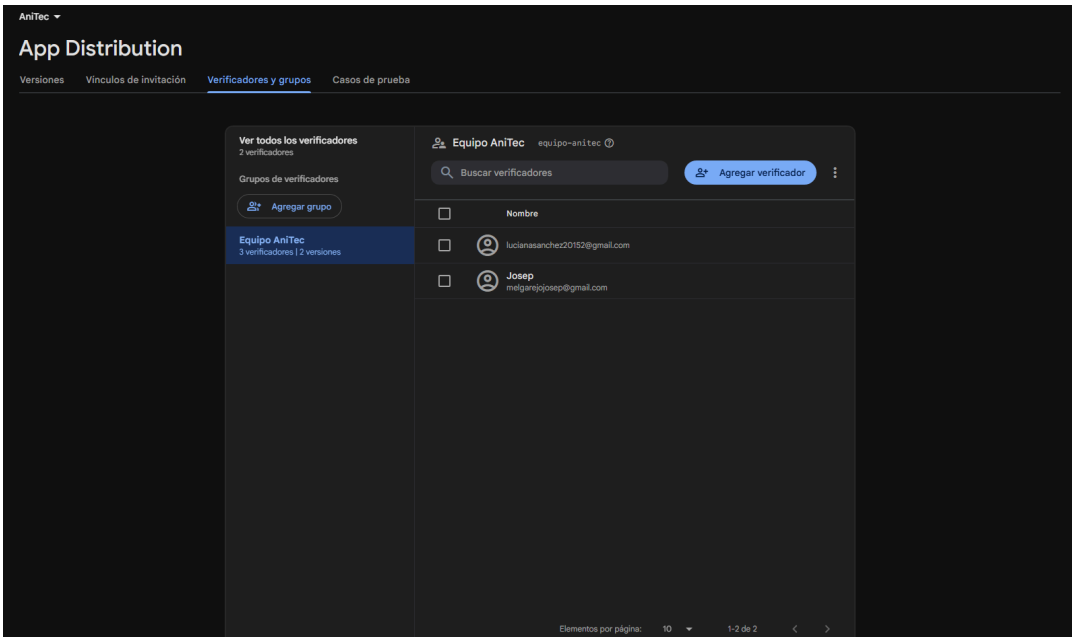


Figura 4.2.2.8.4. Grupo de evaluadores «Equipo AniTec» en App Distribution. Fuente: elaboración propia (captura de Firebase).

4.2.2.9. Team Collaboration Insights during Sprint

Esta sección interpreta la participación del equipo a partir de las tareas del Sprint Backlog (sección 4.2.2.3), de los commits de los repositorios y de la colaboración por producto. Las métricas se analizan en contexto y no se usan de forma aislada para medir aporte. El Sprint 2 repartió las tareas en partes iguales: cada integrante tuvo 40 de las 200 horas estimadas.

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
Sebastian Martin	Backend, aplicación Flutter	Implementó el filtrado de los datos por usuario en el servidor, el retorno móvil de	Tareas T-031 a T-037. Commits:	Hacer que el servidor decida qué ve cada usuario me mostró que la seguridad no puede depender de la

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
Beingolea Montalvo	(pruebas) y despliegue	Stripe y la confirmación del pago, la carga de los planes de suscripción, el archivado de animales y el nombre base del alta masiva con los datos de demostración. Configuró el servicio publicado y escribió las pruebas de la aplicación Flutter.	pendiente de completar.	aplicación, y que cada regla debe tener su prueba de integración, incluida la de quien intenta ver lo ajeno. Con Stripe aprendí que el pago ocurre fuera de nuestro sistema y que lo importante es confirmarlo en el servidor. Al desplegar descubrí que un recurso gratuito, como el límite de conexiones de la base, puede romper lo que funciona en local, y que conviene probar con las mismas condiciones de uso real.
Josep Eliu Melgarejo Quiroz	Aplicación Android e informe	Implementó la vista del veterinario (clientes, pacientes e historial), el trabajo sin conexión con la cola de operaciones, las suscripciones con Stripe en Android, la corrección de la navegación inferior con su prueba y el permiso de red local para depurar. Mantiene la estructura del informe, ajustó el Product Backlog y documentó el sprint.	Tareas T-038 a T-043. Commits: pendiente de completar.	La corrección de la navegación inferior me enseñó a reproducir un error con una prueba antes de arreglarlo: así supe que la causa era la pila guardada y no otra, y quedó cubierta. Implementar la cola sin conexión me mostró que reintentar sin claves de idempotencia puede duplicar registros, algo que queda para el Sprint 3. Documentar el sprint contrastando cada historia con el código me obligó a reconocer cuáles quedaron parciales.
Saul Ortega Muñoz	Aplicación Android (escáner y dispositivos), Flutter, pruebas del backend y Landing Page	Implementó el escáner de códigos QR con ML Kit y la búsqueda manual, los dispositivos IoT, la arquitectura y el diseño del proyecto Flutter, las pruebas de integración y BDD del backend, y los mensajes de error y el idioma. Actualizó la Landing Page con el vínculo de descarga.	Tareas T-056 a T-061. Commits: pendiente de completar.	Integrar ML Kit con CameraX me enseñó a trabajar con un recurso del dispositivo que el emulador no puede probar: el permiso denegado, el código ilegible y la búsqueda manual deben resolverse sin la cámara, y la lectura real se comprueba en un teléfono. Crear el proyecto Flutter con la misma estructura por contextos que Android me mostró que una

Integrante	Productos / aspectos	Contribución en el sprint	Evidencia	Reflexión
				arquitectura clara se traslada entre tecnologías.
Luciana Celeste Sanchez Silva	Diseño UX/UI, aplicación Android (actividades y finanzas) y Flutter	Construyó las pantallas de actividades y de finanzas, el campo de nombre base del formulario de lotes, y en Flutter el inicio de sesión y el inicio del ganadero con su selector de finca, sus contadores y el cambio de idioma. Elaboró los videos del AV2.	Tareas T-044 a T-049. Commits: pendiente de completar.	Llevar el mismo diseño del inicio a Flutter me mostró que los tokens de color y los estados de carga, vacío y error son lo que hace reconocible a una aplicación en cualquier tecnología. En finanzas aprendí que los montos exigen cuidar los decimales y el formato, y que un formulario solo debe mostrar errores después de un intento de guardado.
Giuseppe Villanueva Rodriguez	Aplicación Android (analítica y pruebas), compilación y distribución	Implementó la analítica del ganadero y del veterinario con gráficos propios, el archivado de animales en Android, la compilación firmada con R8, la distribución de las dos aplicaciones en Firebase y las pruebas unitarias e instrumentadas de Android. Verificó el escáner QR y el pago de prueba en un dispositivo físico.	Tareas T-050 a T-055. Commits: pendiente de completar.	Firmar y distribuir las aplicaciones me enseñó pasos nuevos, como el keystore que no puede perderse, la firma que debe distinguir una versión de depuración de una de distribución y los vínculos de invitación. Escribir pruebas de los casos de uso de los contextos nuevos me confirmó que probar a la par de implementar evita errores cuando se añaden pantallas.

Los repositorios del proyecto son:

- Informe: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/Informe
- Landing Page: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-landing-page
- Web Services: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend
- Android: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android
- Flutter: https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-flutter

Las capturas siguientes muestran los analíticos de colaboración y de commits de GitHub (**Insights** → **Contributors** y **Insights** → **Commits**) de cada repositorio.

Figura 4.2.2.9.1. Analíticos de colaboración del repositorio Informe. Fuente: elaboración propia.

Figura 4.2.2.9.2. Analíticos de colaboración del repositorio anitec-android. Fuente: elaboración propia.

Figura 4.2.2.9.3. Analíticos de colaboración del repositorio anitec-backend. Fuente: elaboración propia.

Figura 4.2.2.9.4. Analíticos de colaboración del repositorio anitec-flutter. Fuente: elaboración propia.

Figura 4.2.2.9.5. Analíticos de colaboración del repositorio anitec-landing-page. Fuente: elaboración propia.

Interpretación de los analíticos. Los analíticos de GitHub se leen junto con las tareas del Sprint Backlog, porque el número de commits por autor no mide por sí solo el aporte de cada integrante. El trabajo se concentró en la aplicación Android, que reúne 106 de las 200 horas estimadas (53 %), seguida por el backend, con 40 horas (20 %), la aplicación Flutter, con 34 horas (17 %), y la distribución y la documentación, con 20 horas (10 %). Los cinco integrantes participaron en el desarrollo del backend o de las aplicaciones y cada uno lideró al menos un aspecto: Sebastian Beingolea lideró el backend y colaboró en las pruebas de Flutter, Josep Melgarejo lideró Android, Luciana Sanchez lideró el diseño y Flutter, Giuseppe Villanueva lideró las pruebas y la distribución, y Saul Ortega lideró la Landing Page y colaboró en Android, Flutter y el backend, como indica la matriz de la sección 4.2.2.2. El reparto por persona fue de 40 horas para cada integrante.

Conclusión del trabajo en equipo. Al cierre del Sprint 2 el equipo entregó el backend completo con 150 pruebas automatizadas, una aplicación Android con las funciones del ganadero y del veterinario, una primera versión de la

aplicación Flutter y la distribución de las dos aplicaciones en Firebase App Distribution. Las 31 tareas del sprint quedaron en Done. De las 24 historias comprometidas, 20 se completaron y 4 quedaron parciales, y las historias del consentimiento veterinario, las notificaciones, la persistencia local en Flutter y la sincronización idempotente pasaron al Sprint 3. Como aprendizajes, el equipo destaca cerrar las historias parciales del sprint anterior, probar en las condiciones reales de uso, como las conexiones simultáneas de la base de datos publicada, y verificar en un dispositivo físico lo que el emulador no permite. Para el Sprint 3, el foco será el consentimiento del ganadero sobre el acceso veterinario, las notificaciones, el trabajo sin conexión con sincronización idempotente, la persistencia local en Flutter, la accesibilidad y la automatización de las pruebas de los flujos críticos.

4.3. Validation Interviews

4.3.1. Diseño de Entrevistas

Las entrevistas de validación evalúan la percepción de los usuarios después de interactuar con la Landing Page y la aplicación web funcional de AniTec. Las sesiones se enfocan en la claridad de la propuesta de valor, la facilidad de navegación, la comprensión de las funcionalidades principales y la utilidad percibida por los segmentos objetivo. Sus hallazgos proporcionan una línea base para las decisiones de navegación, lenguaje, retroalimentación y accesibilidad de las aplicaciones móviles. Estas evidencias corresponden a la experiencia efectivamente mostrada durante las grabaciones y no se presentan como pruebas de ejecución de Android o Flutter.

Para esta validación se consideran los dos segmentos principales definidos para AniTec:

- Ganaderos.
- Veterinarios.

La dinámica de la entrevista consiste en presentar primero la landing page de AniTec para observar si el usuario comprende el problema que busca resolver la solución, la confianza que transmite y la claridad de sus secciones. Posteriormente, se muestra la aplicación web iniciando sesión según el segmento correspondiente, con el fin de revisar el dashboard, el menú de navegación y los módulos principales asociados al perfil del usuario.

En el caso del segmento ganadero, la evaluación se orienta a la gestión de hatos, animales, eventos sanitarios, actividades, finanzas, dispositivos IoT y planes. En el caso del segmento veterinario, se evalúa la gestión de clientes asignados, pacientes por ganadero, información clínica, eventos sanitarios, actividades, analíticas e IoT. Al finalizar la demostración, el entrevistado responde un conjunto de preguntas diseñadas para recoger sus opiniones, dificultades y recomendaciones de mejora.

Guía de preguntas para el segmento Ganadero

1. ¿A qué se dedica actualmente dentro de la actividad ganadera y qué tipo de animales maneja?
2. ¿Cómo registra hoy la información de sus animales, fincas, actividades o gastos?
3. Al ver el landing page de AniTec, ¿entiende rápidamente qué problema busca resolver la aplicación?
4. ¿La información del landing page le genera confianza para probar la aplicación? ¿Por qué?
5. ¿Qué sección del landing page le pareció más útil o clara?
6. ¿Hubo alguna parte del landing page que le pareció confusa, innecesaria o poco creíble?
7. Al iniciar sesión como ganadero, ¿le resultó claro hacia dónde debía ir primero?
8. ¿El dashboard ganadero le muestra información útil para tomar decisiones rápidas?
9. ¿Los nombres de las opciones del menú, como hatos, animales, sanidad, finanzas, actividades, IoT y planes, le resultan comprensibles?
10. ¿Le resultó fácil encontrar la lista de animales registrados?
11. ¿El formulario para agregar o editar un animal le parece claro y completo?
12. ¿Qué dato importante sobre un animal cree que falta registrar?
13. ¿Le resulta útil registrar eventos sanitarios como vacunas, tratamientos o incidencias?

14. ¿La sección de actividades le ayudaría a organizar tareas de la finca? ¿Qué tareas agregaría?
15. ¿La sección financiera le parece útil para controlar ingresos y gastos ganaderos?
16. ¿La sección de dispositivos IoT le parece comprensible para asociar dispositivos a animales o fincas?
17. ¿Qué tan fácil le parece interpretar el estado y las métricas de los dispositivos IoT?
18. ¿El lenguaje usado en la aplicación le parece cercano y fácil de entender?
19. ¿Qué parte de la aplicación le resultó más difícil de usar o encontrar?
20. Después de probar AniTec, ¿la usaría en su trabajo diario? ¿Qué tendría que mejorar para que sí la use?

Guía de preguntas para el segmento Veterinario

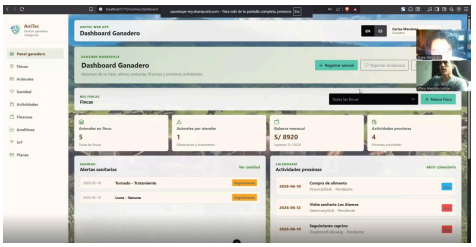
1. ¿Cuál es su experiencia trabajando con ganaderos o productores pecuarios?
2. ¿Cómo organiza actualmente la información de sus clientes, pacientes y visitas?
3. Al ver el landing page de AniTec, ¿queda claro que también está pensada para veterinarios?
4. ¿Qué información del landing page le ayudó más a entender el valor de la aplicación?
5. ¿Qué información agregaría al landing page para que un veterinario confíe más en AniTec?
6. Al iniciar sesión como veterinario, ¿el dashboard le permite entender rápidamente su carga de trabajo?
7. ¿La opción de clientes asignados le parece clara y útil?
8. ¿Le resultó fácil agregar o revisar clientes ganaderos?
9. ¿La vista de pacientes por ganadero le ayuda a encontrar animales bajo seguimiento?
10. ¿Le parece adecuado que primero se seleccione un cliente para revisar sus fincas y animales?
11. ¿La información clínica de cada animal es suficiente para una revisión veterinaria básica?
12. ¿El formulario de eventos sanitarios permite registrar bien vacunas, diagnósticos, tratamientos o seguimientos?
13. ¿Qué campos médicos considera que faltan en el registro sanitario?
14. ¿La sección de actividades le serviría para programar visitas, controles o seguimientos?
15. ¿Las alertas o estados sanitarios son fáciles de identificar dentro del dashboard?
16. ¿La sección de analíticas le da información útil sobre clientes, animales o eventos sanitarios?
17. ¿La sección IoT le parece útil para monitorear animales o fincas de sus clientes?
18. ¿El menú y la organización de la aplicación coinciden con la forma en que usted trabaja?
19. ¿Hubo alguna pantalla, botón o texto que no entendió durante la prueba?
20. Después de probar AniTec, ¿la recomendaría como herramienta de apoyo veterinario? ¿Qué cambios serían prioritarios?

4.3.2. Registro de Entrevistas

A continuación se presentan los registros, evidencias y resultados obtenidos en las sesiones de validación con representantes de los segmentos Ganadero y Veterinario.

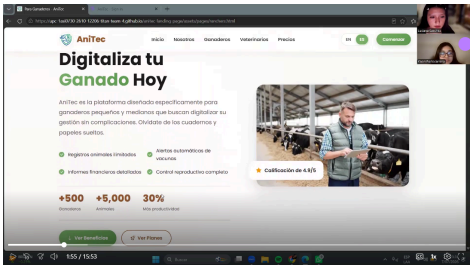
Segmento Objetivo 1: Ganaderos

ENTREVISTA N°1	
Nombre del entrevistado	Rodrigo Alfaro
Edad	25

Profesión	Ganadero
Departamento	Lima
Inicio del video	00:00:13
Fin del video	00:09:36
Link del video	https://tinyurl.com/ValidacionRodrigoAlfaro
Foto entrevista	
Objetivo	Validar si la landing page y la aplicación web AniTec responden a las necesidades del User Persona ganadero, evaluando la organización de información, la utilidad del dashboard y la percepción de valor de la plataforma.
Aspecto del User Persona	**Resultado obtenido / ¿Se valida?**
Frustración: pierde registros	Confirma uso de cuadernos y hojas sueltas; esto causa pérdida o desorden de información. / Sí
Frustración: recordar vacunas	Necesita revisar cuadernos; la aplicación sería útil para no olvidar vacunas y tratamientos. / Sí
Meta: controlar ganado	El dashboard facilita la visualización del estado de sus animales. / Sí
Meta: reducir pérdidas	El registro de ingresos y egresos ayuda a tomar mejores decisiones. / Sí
Motivación: negocio organizado	La plataforma centraliza la información y profesionaliza la gestión. / Sí
Aspectos positivos	Navegación intuitiva, propósito claro y secciones con nombres entendibles.
Dificultades	La sección IoT puede ser difícil de comprender y encontrar; requiere mayor apoyo visual.
Recomendaciones	Agregar alertas automáticas para actividades, fotos de animales y una versión móvil optimizada.
Conclusión	AniTec resuelve problemas reales relacionados con el registro manual. Se requiere simplificar la sección IoT y mejorar la usabilidad móvil.

ENTREVISTA N°2

Nombre del entrevistado	Kiara Gallardo
Edad	22
Profesión	Ganadero

Departamento	Lima
Inicio del video	00:06:30
Fin del video	00:17:25
Link del video	https://tinyurl.com/EntrevistaValidacion2
Foto entrevista	
Objetivo	Validar si la landing page y la aplicación web AniTec responden a las necesidades del User Persona, enfocándose en la gestión, organización y el valor de la herramienta.
Aspecto del User Persona	**Resultado obtenido / ¿Se valida?**
Frustración: desorden manual	Registra información en papeles y libros; le resulta difícil de ordenar. / Sí
Frustración: control de fechas	El control de fechas es un desafío constante; la aplicación ayudaría a no olvidarlas. / Sí
Meta: información organizada	Valora la centralización de información y el filtrado de animales. / Sí
Meta: decisiones basadas en datos	El dashboard y la sección financiera ayudan a visualizar ganancias, pérdidas y salud. / Sí
Aspectos positivos	Interfaz clara e intuitiva, dashboard muy útil y eliminación del uso de papel.
Dificultades	Desconocimiento del concepto y utilidad de IoT, además de familiarización inicial con el dashboard.
Recomendaciones	Incluir una explicación didáctica sobre IoT, versión móvil optimizada y mejores recordatorios automáticos.
Conclusión	AniTec satisface las necesidades de organización y control, validando su valor, aunque requiere mejorar la comunicación sobre tecnologías avanzadas como IoT.

ENTREVISTA N°3

Nombre del entrevistado	Vicente
Edad	61
Profesión	Ganadero
Departamento	Lima

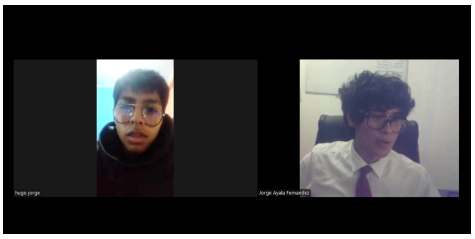
Inicio del video	00:00:00
Fin del video	00:07:30
Link del video	https://tinyurl.com/ValidacionAppWebVicente
Foto entrevista	
Objetivo	Validar si la landing page y la aplicación web AniTec responden a las necesidades del User Persona para la gestión de ganado, con enfoque en la organización sanitaria y administrativa.
Aspecto del User Persona	**Resultado obtenido / ¿Se valida?**
Frustración: registros manuales	Utiliza Excel y anotaciones manuales; el proceso es ineficiente y propenso a errores. / Sí
Meta: control preciso	Valora la centralización de datos estadísticos para mayor control y precisión. / Sí
Meta: decisiones basadas en datos	El dashboard facilita la toma de decisiones rápidas y precisas. / Sí
Motivación: organización sanitaria	Reconoce la importancia del módulo sanitario para gestionar el control de enfermedades. / Sí
Aspectos positivos	Herramienta fácil y efectiva; una versión móvil simplificaría las tareas diarias.
Dificultades	Requiere familiarización con el dashboard y puede existir confusión inicial en registros sanitarios.
Recomendaciones	Ampliar con funciones agrícolas, como suelo y cultivos, e implementar alertas automáticas.
Conclusión	AniTec centraliza la gestión y mejora la eficiencia. La integración de funciones agrícolas consolidaría su valor como solución integral.

Segmento Objetivo 2: Veterinarios

ENTREVISTA N°4	
Nombre del entrevistado	Ariana Fernandez
Edad	21
Profesión	Veterinario

Departamento	Lima
Inicio del video	00:00:00
Fin del video	00:15:45
Link del video	https://tinyurl.com/muxwt3cd
Foto entrevista	
Objetivo	Validar si la aplicación AniTec responde a las necesidades del User Persona del segmento veterinario, evaluando funcionalidades clave como el historial clínico y la utilidad del dashboard.
Aspecto del User Persona	**Resultado obtenido / ¿Se valida?**
Frustración: falta de historial clínico organizado	Destaca la importancia y utilidad del historial médico de los animales como herramienta de trabajo. / Sí
Meta: acceder a información para tratamientos	Considera que disponer del historial médico permite un mejor tratamiento y seguimiento clínico. / Sí
Motivación: mejorar la eficiencia	Valora la centralización de la información y la posibilidad de ver problemas pasados de cada animal de forma organizada. / Sí
Meta: reducir carga administrativa	Considera que el sistema agiliza el trabajo diario en comparación con métodos tradicionales como Excel. / Sí
Aspectos positivos	El historial clínico se percibe como la funcionalidad más valiosa para el trabajo clínico diario. El dashboard permite obtener una visión general rápida del estado de salud de los animales.
Dificultades	Desconocimiento inicial sobre las funciones de los dispositivos inteligentes IoT, por lo que la terminología podría ser un reto.
Recomendaciones	Incluir una explicación didáctica sobre el significado y las ventajas de los dispositivos IoT. Integrar funcionalidades para registrar datos técnicos y clínicos más detallados.
Conclusión	La entrevista confirma que AniTec satisface necesidades clave del veterinario, especialmente la organización y acceso al historial médico. La plataforma optimiza el tiempo y mejora la calidad de la atención veterinaria, aunque requiere mejorar la comunicación sobre IoT.

ENTREVISTA N°5

Nombre del entrevistado	Hugo Jorge
Edad	25
Profesión	Veterinario
Departamento	Lima
Inicio del video	00:00:00
Fin del video	00:04:30
Link del video	https://tinyurl.com/EntrevistaValidacionHugo
Foto entrevista	
Objetivo	Validar si la aplicación AniTec responde a las necesidades del User Persona del segmento veterinario, evaluando funcionalidades clave como el historial clínico y la utilidad del dashboard.
Aspecto del User Persona	**Resultado obtenido / ¿Se valida?**
Frustración: falta de historial organizado	Destaca la importancia y utilidad del historial médico como herramienta de trabajo. / Sí
Meta: acceder a información de tratamiento	Considera que el historial médico permite un mejor tratamiento y seguimiento clínico. / Sí
Motivación: mejorar eficiencia	Valora la centralización de información y la visualización organizada de problemas pasados. / Sí
Meta: reducir carga administrativa	Considera que el sistema agiliza el trabajo diario frente a métodos tradicionales como Excel. / Sí
Aspectos positivos	El historial clínico es la funcionalidad más valiosa; el dashboard resulta útil para una visión general rápida.
Dificultades	Desconocimiento sobre funciones de dispositivos inteligentes IoT y terminología técnica compleja.
Recomendaciones	Agregar explicación didáctica sobre ventajas de IoT e integrar registros clínicos más detallados.
Conclusión	AniTec satisface necesidades de organización y acceso al historial médico, optimizando el tiempo y la atención veterinaria, aunque requiere mejor comunicación sobre el

alcance de IoT.

4.3.3. Evaluaciones según heurísticas

La evaluación heurística de AniTec se realizó a partir de las sesiones de validación con usuarios de los segmentos ganadero y veterinario. Durante estas sesiones, los entrevistados interactuaron con la landing page y con la aplicación web, revisando los flujos principales según su rol. La evaluación considera criterios de usabilidad, arquitectura de información e inclusive design, con el objetivo de identificar fortalezas, problemas de experiencia de usuario y oportunidades de mejora.

Para registrar los hallazgos se utilizó una escala de severidad simple:

Severidad	Descripción
0	No representa un problema de experiencia de usuario.
1	Problema menor que puede corregirse sin afectar el flujo principal.
2	Problema moderado que puede generar dudas o fricción en algunos usuarios.
3	Problema importante que puede dificultar el cumplimiento de una tarea.
4	Problema crítico que impide completar una tarea principal.

Evaluación heurística - Segmento Ganadero:

Criterio evaluado	Evidencia observada	Severidad	Recomendación
Visibilidad del estado del sistema	El dashboard ganadero permite visualizar animales, hatos, eventos sanitarios y actividades.	1	Mantener estados visibles de carga, éxito y error en formularios y listados.
Correspondencia entre el sistema y el mundo real	Los términos animales, hatos, sanidad, actividades y finanzas fueron comprensibles.	0	Mantener vocabulario cercano al contexto ganadero.
Control y libertad del usuario	Los usuarios navegan por módulos, pero faltan acciones de cancelación más visibles.	1	Hacer más claros los botones para cancelar o volver sin guardar.
Consistencia y estándares	La aplicación mantiene navegación lateral, formularios y tarjetas con estructura similar.	0	Mantener la consistencia visual y de interacción.
Prevención de errores	Los formularios tienen campos definidos, pero faltan validaciones adicionales.	2	Añadir mensajes claros para campos obligatorios y formatos esperados.
Reconocimiento antes que memoria	El menú lateral permite reconocer las secciones disponibles sin memorizar rutas.	0	Mantener iconos y nombres de menú visibles.

Criterio evaluado	Evidencia observada	Severidad	Recomendación
Flexibilidad y eficiencia de uso	El dashboard facilita tareas, pero faltan accesos rápidos a registros usados con frecuencia.	1	Añadir accesos directos a registrar animal, evento sanitario o actividad.
Diseño estético y minimalista	Interfaz organizada, aunque algunos dashboards requieren mejor jerarquía visual.	1	Jerarquizar indicadores críticos y reducir contenido secundario.
Ayuda al usuario a reconocer y recuperarse de errores	Mensajes de error insuficientes ante fallas de backend o conexión.	2	Mostrar mensajes de error específicos cuando falle una operación.
Inclusive design y accesibilidad	Se debe seguir cuidando el contraste y legibilidad en dispositivos móviles.	1	Validar contrastes, tamaños de texto y uso en pantallas pequeñas.

Evaluación heurística - Segmento Veterinario:

Criterio evaluado	Evidencia observada	Severidad	Recomendación
Visibilidad del estado del sistema	El dashboard veterinario muestra clientes, pacientes y alertas, lo que ayuda a entender la carga de trabajo inicial.	0	Mantener indicadores visibles de clientes y pacientes asignados.
Correspondencia entre el sistema y el mundo real	Los conceptos clientes, pacientes, historial clínico, eventos sanitarios y actividades coinciden con tareas veterinarias básicas.	0	Reforzar textos específicos para veterinarios en landing page y aplicación.
Control y libertad del usuario	El veterinario puede revisar clientes y pacientes, pero el flujo depende de seleccionar un cliente para ver información asociada.	1	Explicar mejor el paso de selección de cliente antes de revisar animales o historial.
Consistencia y estándares	Las vistas mantienen estructura similar al resto de la aplicación, lo que facilita aprendizaje.	0	Mantener componentes y patrones ya definidos.
Prevención de errores	El registro sanitario funciona para casos básicos, pero los entrevistados sugirieron campos clínicos adicionales.	2	Añadir campos como dosis, vía de administración, signos vitales u observaciones cuando el alcance lo permita.
Reconocimiento antes que memoria	La navegación por menú facilita encontrar clientes, pacientes,	0	Mantener agrupación de módulos por rol.

Criterio evaluado	Evidencia observada	Severidad	Recomendación
	sanidad, actividades e IoT.		
Flexibilidad y eficiencia de uso	El dashboard centraliza información útil, pero podrían existir accesos rápidos para registrar visita o evento sanitario.	1	Agregar accesos directos a tareas veterinarias frecuentes.
Diseño estético y minimalista	La información se presenta de forma ordenada, aunque las vistas clínicas podrían beneficiarse de una jerarquía más clara.	1	Diferenciar visualmente datos clínicos, historial y acciones principales.
Ayuda al usuario a reconocer y recuperarse de errores	Los mensajes de error pueden mejorar cuando no se cargan clientes, pacientes o eventos sanitarios.	2	Mostrar mensajes específicos para errores de conexión o ausencia de datos asignados.
Inclusive design y accesibilidad	La experiencia por rol ayuda a reducir información innecesaria, pero se debe seguir validando legibilidad en dispositivos pequeños.	1	Revisar responsive, contraste y claridad de textos médicos.

Resumen de hallazgos heurísticos:

La evaluación muestra que **AniTec** cumple adecuadamente con criterios de correspondencia con el mundo real, consistencia, reconocimiento y organización general de la información. Los usuarios comprenden la división por roles y reconocen la utilidad de centralizar animales, sanidad, actividades, finanzas, clientes, pacientes, IoT y suscripciones en una sola plataforma.

Los principales puntos de mejora se concentran en **prevención de errores**, **mensajes de retroalimentación** y **detalle de formularios**. En el segmento ganadero, se recomienda reforzar validaciones y accesos rápidos a tareas frecuentes. En el segmento veterinario, se recomienda ampliar progresivamente campos clínicos y explicar con mayor claridad el flujo de selección de cliente antes de revisar pacientes o historial sanitario.

Conclusiones

Conclusiones y recomendaciones

Conclusiones

El proyecto AniTec permitió validar la necesidad de una solución digital orientada a ganaderos y veterinarios, debido a que ambos segmentos requieren mayor orden, trazabilidad y disponibilidad de información sobre animales, fincas, actividades, sanidad y gastos. Las entrevistas y validaciones realizadas confirmaron que el problema identificado es real y que una plataforma web puede aportar valor si mantiene una experiencia simple, clara y cercana al contexto ganadero.

El diseño del sistema basado en Domain-Driven Design permitió organizar AniTec mediante bounded contexts como IAM, Profiles, Livestock, Sanitary, Financial, Activities, Analytics, Clients, Devices, Metrics y Subscriptions. Esta separación favorece la mantenibilidad del sistema, facilita la asignación de responsabilidades dentro del equipo y permite que cada módulo evolucione sin afectar innecesariamente al resto de la aplicación.

Durante el Sprint 1 se logró establecer la presencia digital del producto mediante una landing page funcional desplegada en GitHub Pages. Este avance permitió comunicar la propuesta de valor de AniTec, presentar los beneficios principales para ganaderos y veterinarios, y generar una primera base de validación frente a usuarios potenciales.

Durante el Sprint 2 se implementó la aplicación web frontend con Vue, organizada por bounded contexts y conectada inicialmente a datos de prueba mediante my-json-server. Este sprint permitió construir las principales vistas funcionales de la plataforma, validar flujos de navegación y preparar la estructura necesaria para reemplazar progresivamente los datos mock por servicios reales.

Durante el Sprint 3 se desarrolló el backend real de AniTec con ASP.NET Core, Entity Framework Core y MySQL. La API implementa autenticación con JWT, gestión de usuarios, perfiles, hatos, animales, eventos sanitarios, actividades, registros financieros, clientes veterinarios, analíticas, planes, suscripciones y pagos mock. Con ello, el proyecto dejó de depender solamente de datos simulados y pasó a contar con una base de servicios persistentes.

La documentación de servicios mediante Swagger/OpenAPI permitió verificar los endpoints implementados y facilitar las pruebas manuales del backend. Asimismo, las migraciones de Entity Framework Core permitieron mantener alineada la estructura de la base de datos con las entidades del dominio, reduciendo errores en la configuración y mejorando la trazabilidad técnica del sistema.

El despliegue del backend en Render representa un avance importante para el proyecto, ya que la API queda disponible desde internet y puede ser consumida por el frontend desplegado. Esto permite validar AniTec en un entorno más cercano a producción, comprobar la disponibilidad pública del servicio y preparar futuras pruebas de integración entre frontend, backend y base de datos.

En relación con el Student Outcome, el equipo evidenció trabajo colaborativo y liderazgo conjunto durante los tres avances. Cada integrante asumió responsabilidades claras por módulo o artefacto, participó en la planificación de

tareas, colaboró en la integración del sistema y aportó a la documentación del producto. Esto permitió cumplir los objetivos de cada sprint de forma progresiva y mantener coherencia entre la propuesta, el diseño, el frontend, el backend y el despliegue.

Contraste con Lean UX y validaciones:

El Problem Statement se mantiene vigente, ya que las validaciones confirmaron que ganaderos y veterinarios siguen enfrentando problemas de desorden, registros manuales y falta de trazabilidad. AniTec responde a una necesidad real al centralizar información sanitaria, productiva, económica y operativa en una sola plataforma.

Las Business Assumptions fueron validadas de forma general, debido a que los usuarios reconocieron valor en contar con una solución web para registrar animales, eventos sanitarios, actividades, finanzas y clientes veterinarios. Sin embargo, todavía se requiere reforzar reportes, alertas y campos específicos para aumentar la utilidad percibida.

Las User Assumptions se confirmaron parcialmente, porque los entrevistados comprendieron la navegación por roles y consideraron útiles los dashboards y módulos principales. Aun así, la experiencia debe seguir simplificándose, especialmente en formularios y mensajes de error.

Las Feature Assumptions resultaron pertinentes para una primera versión del producto, ya que los usuarios identificaron valor en el registro de animales, historial sanitario, actividades, finanzas y seguimiento veterinario. Como mejora futura, se deben ampliar los campos clínicos, datos del animal y métricas disponibles para apoyar mejores decisiones.

Las Hypothesis Statements fueron parcialmente validadas. Existe interés inicial y una percepción positiva sobre la utilidad de AniTec, especialmente por la centralización de información; no obstante, la adopción dependerá de fortalecer la confianza, mejorar recordatorios, optimizar reportes y facilitar el uso continuo de la plataforma.

Los criterios de éxito quedan planteados como metas para una siguiente etapa de validación con uso real de la aplicación desplegada. Las entrevistas permiten confirmar intención de uso y valor percibido, pero aún falta medir reducción de errores, mejora en precisión de registros y apoyo efectivo a decisiones basadas en datos.

Recomendaciones

Corto plazo: Se recomienda continuar con la integración completa entre el frontend desplegado en GitHub Pages y el backend desplegado en Render, asegurando que las variables de entorno del frontend apunten a la API real y que los flujos principales funcionen sin depender de my-json-server.

Corto plazo: Se recomienda fortalecer la autenticación y autorización del sistema, definiendo permisos por rol para ganaderos, veterinarios y administradores. Esto permitiría proteger los endpoints sensibles y asegurar que cada usuario solo acceda a la información correspondiente a su perfil.

Mediano plazo: Se recomienda continuar la validación con ganaderos y veterinarios usando la aplicación desplegada, no solo prototipos. Esto permitirá identificar problemas reales de uso, comprensión del lenguaje, dificultad en formularios, utilidad de dashboards y necesidades adicionales en los módulos de sanidad, actividades y finanzas.

Mediano plazo: Se recomienda mejorar la experiencia de usuario del frontend en una siguiente iteración, priorizando claridad en dashboards, simplificación de formularios y mejor organización del flujo veterinario por clientes

asignados.

Mediano plazo: Se recomienda completar y fortalecer las pruebas del backend, incluyendo pruebas de endpoints, validación de reglas de negocio, manejo de errores y pruebas de integración con la base de datos. Esto ayudará a asegurar mayor estabilidad antes de seguir ampliando funcionalidades.

Futuro roadmap: Se recomienda mantener actualizada la documentación técnica, incluyendo endpoints, migraciones, evidencias de despliegue, diagramas C4 y decisiones de arquitectura. A medida que AniTec crezca, esta documentación será clave para sostener la colaboración del equipo y facilitar futuras mejoras del producto.

Video About The Team

El video About The Team presenta la participación de los integrantes del equipo Titan Team 4 durante el desarrollo de AniTec, destacando las actividades realizadas, los logros alcanzados en el curso y el desarrollo de competencias asociadas al trabajo colaborativo, liderazgo conjunto, planificación de tareas y cumplimiento de objetivos.

Datos del video:

Elemento	Información
Título	Video About The Team - AniTec
Duración	
Público objetivo	Docente del curso, visitantes del landing page e interesados en conocer el proceso de trabajo del equipo
URL publicado en Microsoft Stream	
URL publicado en YouTube	
Uso en landing page	El video de YouTube se utiliza como evidencia pública del proceso de trabajo del equipo.

URL del video publicado en Microsoft Stream:

URL del video publicado en YouTube:

Bibliografía

AgriWebb. (s. f.-a). *AgriWebb mobile app*. Recuperado el 13 de septiembre de 2026, de https://www.agriwebb.com/mobile-app/

AgriWebb. (s. f.-b). *Pricing*. Recuperado el 13 de septiembre de 2026, de https://www.agriwebb.com/us/pricing/

Android Developers. (s. f.-a). *Jetpack Compose*. Recuperado el 13 de septiembre de 2026, de https://developer.android.com/develop/ui/compose

Android Developers. (s. f.-b). *Save data in a local database using Room*. Recuperado el 13 de septiembre de 2026, de https://developer.android.com/training/data-storage/room

Bourgau, P. (2022, 29 de marzo). *Step-by-step guide to run your big picture event storming*. Philippe Bourgau's Blog. https://bit.ly/bpes-guide

Chandrasekaran, P. (2023, 20 de marzo). *Remote EventStorming in practice*. Domain-Driven Design: A Practitioner's Guide. https://ddd-practitioners.com/2023/03/20/remote-eventstorming-workshop/

Cohn, M. (2006). *User stories applied: For agile software development*. Addison-Wesley Professional.

DDD by Examples. (s. f.-a). *Big picture EventStorming*. GitHub. Recuperado el 13 de septiembre de 2026, de https://github.com/ddd-by-examples/library/blob/master/docs/big-picture.md

DDD by Examples. (s. f.-b). *Design-level EventStorming*. GitHub. Recuperado el 13 de septiembre de 2026, de https://github.com/ddd-by-examples/library/blob/master/docs/design-level.md

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Firebase. (s. f.). *Firebase App Distribution*. Recuperado el 13 de septiembre de 2026, de https://firebase.google.com/docs/app-distribution

Flutter. (s. f.). *Flutter documentation*. Recuperado el 13 de septiembre de 2026, de https://docs.flutter.dev/

Food and Agriculture Organization of the United Nations. (2022). *Digital agriculture: Technologies transforming agriculture and food systems*. https://www.fao.org/digital-agriculture/en/

Food and Agriculture Organization of the United Nations. (2023). *Livestock and the environment*. https://www.fao.org/livestock-environment/en/

Google for Developers. (s. f.). *Scan barcodes with ML Kit on Android*. Recuperado el 13 de septiembre de 2026, de https://developers.google.com/ml-kit/vision/barcode-scanning/android

Gothelf, J., & Seiden, J. (2021). *Lean UX: Designing great products with agile teams* (3.ª ed.). O'Reilly Media.

iLivestock. (s. f.-a). *iLivestock Argentina: Innovación y tecnología agrícola*. Recuperado el 13 de septiembre de 2026, de https://www.ilivestock.com/es-ar/ar

iLivestock. (s. f.-b). *Livestock management software for sheep and cattle farmers*. Recuperado el 13 de septiembre de 2026, de https://www.ilivestock.com/software

Instituto Nacional de Estadística e Informática. (2014). *Características socioeconómicas del productor agropecuario en el Perú: IV Censo Nacional Agropecuario 2012*. https://www.inei.gob.pe/media/MenuRecursivo/publicaciones_digitales/Est/Lib1177/libro.pdf

Instituto Nacional de Estadística e Informática. (2025). *Estadísticas de las tecnologías de información y comunicación en los hogares: Enero-febrero-marzo 2025*. https://www.inei.gob.pe/media/MenuRecursivo/boletines/informe-tecnico_tecnologiasdelainformacion_ene_feb_mar2025.pdf

Instituto Nacional de Estadística e Informática. (2026). *Estadísticas de las tecnologías de información y comunicación en los hogares: Octubre-noviembre-diciembre 2025*. https://www.inei.gob.pe/media/MenuRecursivo/boletines/boletin-tic-oct_dic2025.pdf

Jošt, G., & Taneski, V. (2025). State-of-the-art cross-platform mobile application development frameworks: A comparative study of market and developer trends. *Informatics, 12*(2), 45. https://doi.org/10.3390/informatics12020045

Livestock Manager. (s. f.). *Livestock Manager: Made by Pila*. Recuperado el 13 de septiembre de 2026, de https://app.livestockfarm.co/

Mahmud, T., Che, M., Ngu, A., & Yang, G. (2025). Why Android app testing falls short: Empirical insights from open-source projects and a practitioner survey. *Empirical Software Engineering, 30*, artículo 163. https://doi.org/10.1007/s10664-025-10726-x

Microsoft. (2026a). *ASP.NET Core documentation*. https://learn.microsoft.com/aspnet/core/

Microsoft. (2026b). *Entity Framework Core documentation*. https://learn.microsoft.com/ef/core/

MySQL. (2026). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

Nsabiyeze, A., Zhang, M., Li, J., Zhao, Q., & Zhang, X. (2025). Precision livestock farming for climate-resilient livestock management: A review of real-time monitoring and decision support systems. *Journal of Cleaner Production, 524*, 146454. https://doi.org/10.1016/j.jclepro.2025.146454

OpenAPI Initiative. (2026). *OpenAPI specification*. https://spec.openapis.org/oas/latest.html

Papadopoulos, G., Papantonatou, M.-Z., Uyar, H., Kriezi, O., Mavrommatis, A., Psiroukis, V., Kasimati, A., Tsiplakou, E., & Fountas, S. (2025). Economic and environmental benefits of digital agricultural technological solutions in livestock farming: A review. *Smart Agricultural Technology, 10*, 100783. https://doi.org/10.1016/j.atech.2025.100783

Pressman, R. S. (2010). *Ingeniería de software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum Guides. https://scrumguides.org/scrum-guide.html

Stripe. (s. f.). *Use a prebuilt Stripe-hosted payment page*. Recuperado el 13 de septiembre de 2026, de https://docs.stripe.com/payments/checkout

Structurizr. (2026). *Structurizr documentation*. https://docs.structurizr.com/

Vue.js. (2026). *Vue.js documentation*. https://vuejs.org/guide/

Weerakoon, S. (2020, 1 de noviembre). *How to write a user story for an API product*. Medium. https://sameera17w.medium.com/how-to-write-a-user-story-for-an-api-product-7af6abd4ad2e

Anexos

Los siguientes anexos reúnen los enlaces disponibles hasta el TB1. Se incluyen las evidencias utilizadas en los capítulos I y II, los repositorios vigentes del curso, los despliegues del producto web tomado como base, el video de exposición del TB1 y los videos de ejecución del Sprint 1. Los recursos correspondientes al AV2 (videos y vínculos de distribución de las aplicaciones) se agregan al anexo F, y los de validación de la aplicación móvil y de los siguientes sprints se incorporarán cuando sean elaborados.

Anexo A. Videos de entrevistas

Segmento de ganaderos

- **Entrevista 1 - Caterina Rodriguez:** https://tinyurl.com/caterinarodri
- **Entrevista 2 - Rebeca Noemi Quiroz Roldan:** https://tinyurl.com/entrevista2RebecaQuiroz
- **Entrevista 3 - Ursula Trigozo Salcedo:** https://upcedupe-my.sharepoint.com/:v:/g/personal/u20231c019_upc_edu_pe/IQDw1Fks32sxQroa9TMJpBhPAffdaO--WIZy3fKexREgNhU?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJTdHJlYW1XZWJBcHAiLCJyZWZlcnJhbFZpZXciOiJTaGFyZURpYWxvZy1MaW5rIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXcifX0%3D&e=H1KzAW

Segmento de veterinarios

- **Entrevista 4 - Angela Mendoza:** https://tinyurl.com/entrevista4AngelaMendoza
- **Entrevista 5 - Jeferson Castro Pariona:** https://tinyurl.com/b3txtyh3

Anexo B. Materiales de investigación y diseño

- **Lean UX Canvas de AniTec:** https://tinyurl.com/LeanUxCanvasMural
- **Big Picture EventStorming y artefactos de Strategic-Level DDD:** https://tinyurl.com/EventSorming

Los enlaces públicos del Impact Mapping y del Product Backlog todavía no han sido incorporados al informe. Se añadirán cuando los tableros utilizados por el equipo estén disponibles públicamente.

Anexo C. Repositorios del curso

- **Repositorio del informe:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/Informe
- **Repositorio de la landing page:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-landing-page
- **Repositorio del frontend web:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-frontend
- **Repositorio del backend:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-backend
- **Repositorio de la aplicación Android:** https://github.com/ADM-1ACC0238-2620-13975-Grupo01/anitec-android

El repositorio de la aplicación Flutter se incorporará cuando se cree el proyecto correspondiente.

Anexo D. Despliegues del producto web tomado como base

Estos enlaces corresponden al trabajo anterior que sirve como punto de partida. No constituyen todavía el despliegue de las aplicaciones móviles del curso actual.

- **Landing page de AniTec:** https://upc-1asi0730-2610-12206-titan-team-4.github.io/anitec-landing-page/
- **Aplicación web de AniTec:** https://anitec-frontend.onrender.com/
- **Documentación Swagger del backend:** https://anitec-backend.onrender.com/swagger/index.html

Anexo E. Herramientas y documentación utilizadas hasta el AV1

Investigación, requisitos y modelado

- **Miro:** https://miro.com/. Se utiliza para Lean UX y EventStorming.
- **UXPressia:** https://uxpressia.com/. Se utiliza para User Personas, Empathy Maps y User Journey Maps.
- **Gherkin:** https://cucumber.io/docs/gherkin/. Se utiliza para expresar criterios de aceptación mediante Given, When y Then.
- **GitHub:** https://github.com/. Se utiliza para control de versiones y colaboración.

Arquitectura y diagramas

- **Structurizr:** https://structurizr.com/. Se utiliza para los diagramas C4.
- **PlantUML:** https://plantuml.com/. Se utiliza para los diagramas de clases del dominio.
- **Mermaid:** https://mermaid.js.org/. Se utiliza como referencia para diagramas y modelos visuales.

Tecnologías consideradas para la solución móvil

- **Jetpack Compose:** https://developer.android.com/develop/ui/compose. Se considera para la interfaz de la aplicación Android nativa.
- **Room:** https://developer.android.com/training/data-storage/room. Se considera para la persistencia local en Android.
- **Flutter:** https://docs.flutter.dev/. Se considera para la aplicación multiplataforma.
- **Google ML Kit Barcode Scanning:** https://developers.google.com/ml-kit/vision/barcode-scanning/android. Se considera para la investigación de identificación mediante códigos QR.
- **Firebase App Distribution:** https://firebase.google.com/docs/app-distribution. Se considera para la distribución de versiones móviles durante la validación.
- **Stripe Checkout:** https://docs.stripe.com/payments/checkout. Se considera para el flujo externo de pagos y suscripciones.

Anexo F. Videos de exposición y validación

Hasta el AV1 no se habían incorporado enlaces públicos de videos. Con el TB1 se incorporan el video de exposición y los videos de ejecución del Sprint 1; las entrevistas de validación de la aplicación móvil, el video About the Product y el video About the Team se añadirán en los hitos que correspondan.

Video de exposición del TB1

- **Exposición TB1:** Ver video de exposición

Videos de ejecución del Sprint 1 (TB1)

- **Landing Page:** Ver video de ejecución (0:00 - 2:04)
- **Backend:** Ver video de ejecución (0:00 - 4:24)
- **Aplicación Android:** Ver video de ejecución (0:00 - 2:59)

Videos del AV2 (Sprint 2)

Pendiente de completar: los enlaces se agregarán cuando los videos estén grabados.

- **Validación de la aplicación (primera versión):** pendiente de completar (enlace al video).
- **About the Product (primera versión):** pendiente de completar (enlace al video).
- **About the Team (primera versión):** pendiente de completar (enlace al video).
- **Ejecución del Sprint 2: Backend, aplicación Android y aplicación Flutter:** pendiente de completar (enlaces a los videos).

Distribución de las aplicaciones (Firebase App Distribution)

Vínculos de invitación del grupo «Equipo AniTec», que tiene acceso a la versión 0.2.0 de la aplicación Android y a la versión 1.0.0 de la aplicación Flutter:

- https://appdistribution.firebase.dev/i/417dc4e6d8ae3e32
- https://appdistribution.firebase.dev/i/862507cca1d4abc6